ffffOn apr 20,2009 saved on 220 on apr20.

Transfer in Linux

To be typed

1. Pointer based sequence to be typed.
2. Evaluation tree to be typed
3. Use of functions

Printout for

4. Binary Relation
5.
6.

Pending assignment

7.

Use of function

```
#include<stdio.h>
int max(int a,int b)
{   if (a>b) return a; else return b; }
main()
{   int p,q,r,s,t;
    printf("Give three numbers");
    scanf("%d %d %d",&p,&q,&r);
    s=max(p,q);
    t=max(s,r);
    printf("%d",t);
}
// read three numbers and find their maximum
#include<stdio.h>
int exch(int x)
{   int y,z,w,p;
    z=x/100;y=x/10%10;w=x%10;
    p=z*100+10*w+y;
    return p;
}
main()
{   int a,b,c,d;
    printf("Give a number");scanf("%d",&a);
    b=exch(a);c=b+a;d=exch(c);
    printf("%d",d);
}
// Read a number. exchange its last two digits. Add it to the number.
again exchange last two digits. Input 2375 output 4723 because
2375+2357=4732
#include<stdio.h>
int exch(int x)
{   int y,z,w,p;
    z=x/100;y=x/10%10;w=x%10;
    p=z*100+10*w+y;
    return p;
}
int pwr(int x,int y)
{   int i,p=1;;
    for (i=1;i<=y;i++)
       p=p*x;
    return p;
}
main()
{   int a,b,c,d;
    printf("Give two numbers");scanf("%d %d",&a,&b);
    c=pwr(a,b);d=exch(c);
    printf("%d",d);
}
Read two numbers and and b. find apowb. Exchange their last two digits.
input 7 3 output 334 because 7pwr3=343
main()
{  float a,b;int p,q;
   printf("Give a number");
   scanf("%f",&a);
   p=a;
```

```
   b=a-p;
   q=10*b;
   printf("%d",q);
}
//read a number and print first digit of fractional part
#include<stdio.h>
main()
{  float a,b,c;int p;
   printf("Give a number");
   scanf("%f",&a);
   p=a;
   b=a-p;
   c=p+0.1+b/10;
   printf("%f",c);
}
//read a number and insert 1 as first digit of fraction part input 12.34
output 12.134
Read a number. exchange it second last and third last digits. [Use
function to exchange last two digits. Delete last digit. Find last
digit. Join a digit at the last]
```

Computer Programming

Simple programs
Clean Programming
Boolean connectives
Switch and condition
Coordinate Geometry
Series
Picture printing
Digits
Factors
Input- Output Format
Reading in a loop
Maximization
History
Rivision
Serial loop and Function
Nested loop and Function
Simulation
Array
Array using function
Array using pointer
Shifting
Matrix using 1D array
Matrix using 2D array
Function
Recursion
String and list
Structure
File
If Less programs
Program Generation
Miscellaneous
Overview
Test Papers

Data Structure and Algorithm

Assignments

Stack & queue
Linked List
Tree (binary and General)
Sorting
Polynomial
Dynamic Programming

Notes

Stack, Queue, Priority queue, Two stack implementation of queue
Pointer, O($\sqrt{n}$) linked list
Height balanced tree, Equivalent classes, Graph, Tree traversal, Huffman Code
Sorting: Quick sort, Merge sort, Bubble sort, Insertion sort, Radix Sort
Dynamic programming: Matrix chain multiplication
Divide and conquer: Polynomial multiplication, n^{th} element in: an array, two sorted arrays
Backtracking: Sum of squares, n queen problem, generation of n digit numbers
Branch and bound: 0-1 Knapsack problem, Greedy Algorithm
Lexical Analyzer,Pattern matching, Automata, Common Substring
Geometric Algorithms: Convex hull, Polygon triangulation

Simple Programs

mkdir hari cd hari	To make directory To change directory
vi t.c i Esc dd :w :q	To enter into editor To enter into insert mode To leave insert mode To delete a line To save To quit from editor
cc t.c ./a.out	To compile file t.c To execute

```
#include<stdio.h>
main( )
{  printf("Rajiv \n Kumar \nGupta");
   printf("Ram prakash");
}

#include<stdio.h>
main( )
{  int a,b;
   printf("Give side of square");
   scanf("%d",&a);
   b=a*a;
   printf("The area is %d",b);
}
```

1. Write program, which reads as input sides of a rectangle and prints its area.
2. Write program, which reads 2 numbers and prints the sum of square of the first and cube of the second. e.g. input 5 3 output 5^2+3^3 =52.
3. Write program, which reads 4 numbers a, b, c and p. Let $f(x)=ax^2+bx+c$ be a function. The program outputs the value of f(p). e.g. input 4 3 –1 2 output $4(2)^2+3(2)-1= 21$.

#include<stdio.h>
main( )
{ int a,b,c,d;
printf(“give two numbers”);
scanf("%d %d",&a,&b);
c=a/b;
d=a%b;
printf(“%d %d”,c,d);
}
Here / is quotient and % is remainder. 87/7 is 12. 87%7 is 3.
	#include<stdio.h>
	main( )
	{ int a,b,c;
	printf(“give a number”);
	scanf("%d",&a);
	b=a%10;
	c=a/10;
	printf(“%d %d”,b,c);
	}
	a%10 finds last digit of a number. a/10 deletes the last digit.
		#include<stdio.h>
		main( )
		{ int a,b,c;
		a=42315;
		b=9;
		c=a*10+b;
		printf(“%d”,c);
		}

4. Write program to delete the last digit. input 13613 output 1361. input 324 output 32.
5. Write program to delete last two digits. input 13613 output 136. input 324 output 3.
6. Write program to print the second last digit. Input 13613 output 1. Input 427 output 2.
7. Program to find the sum of last two digits. For above input output 1+3=4 and 2+7=9
8. Write program to double the last digit. e.g. input 23613 output 23616. input 324 output 328. (assume that last digit is less than 5)
9. Write program to delete the second last digit. e.g. input 23617 output 2367. Input 2365 output 235.
10. Write program to exchange last two digits. e.g. input 23617 output 23671. input 27845345 output 27845354.
11. Write program to exchange the last and the third last digit. e.g. input 23617 output 23716. input 27845345 output 27845543.
12. Write program to double the second last digit. e.g. input 23613 output 23623. input 324 output 344. (assume that second last digit is less than 5)
13. Read a number. Find product after deleting last and second last digit. Input 4358 output 190530 (438*435).
14. Read two numbers. Find their product after exchanging last digits. Input 4270 and 153 output 640950 (4273x150). Input 348 and 31 output 12958 (341*38).

Float Operations

```
#include<stdio.h>
#include<math.h>
main( )
{  float x,y;
   printf("\nGive a number");
   scanf("%f",&x);
   y=cos(x); printf("%f",y);
}
```

The above program finds the cosine of an angle. However the angle must be specified in radians. If input is 1.05 then output is 0.5. It is because 1.05^c is approximately 60^0. To compile the program use cc t.c -lm

```
#include<stdio.h>
#include<math.h>
main( )
{  float x,y,z;
   printf("\nGive a number");
   printf(" between –1 and 1");
   scanf("%f",&x);
   y=acos(x);
   z=y*180/3.14;
   printf("cos inverse in radians %f",y);
   printf(" and in degrees %f",z);
}
```

Here acos(x) means $\cos^{-1}(x)$

Input 0.5 output 60^0. Input .87 output 30^0

1. Write program, which reads a, b and c as sides of a triangle and prints area. Hint: area $= \sqrt{s(s-a)(s-b)(s-c)}$. [Hint: s is (a+b+c)/2] [sqrt(x) will find square root]. Input 5 7 10 output 16.24.
2. Write program, which reads a, b, c and d and finds distance between points (a,b) and (c,d). input 3, 7, 11, 13 output 10.
3. Write program, which reads 6 numbers a, b, c, d, e and f. The program outputs the area of the triangle whose end points are (a,b), (c,d) and (e,f). [Hint: use above two questions]. Input 7 3 11 3 7 6 output 6.
4. Write program, which reads a, b, and c. Let ax + by + c = 0 be equation of a line. The program outputs the slope. Input 3 5 8 output –0.6.
5. Write program, which reads a, b, c, d and e and prints the distance between point (a,b) and line cx+dy+e=0. [Hint: $(ac+bd+e)/(c^2+d^2)^{1/2}$.] input 6 7 3 4 2 output 9.6.
6. Write program, which reads a, b, and c. Let $x^2 + y^2 + ax + by + c = 0$ be equation of a circle. Print its center and radius. Input 10, –6 and –2 then output is center (–5,3) and radius 6. Here circle is $x^2+y^2+10x-6y-2=0$.
7. Write program, which reads a, b, c, p, q and r. Let ax + by + c = 0 and px + qy + r = 0 be equations of lines. Print their point of intersection. Input 4 8 12 2 7 3 output (–5,1).
8. Write program, which reads a, b and c as sides of a triangle and prints the angle A in degree and radian. Hint: $a^2 = b^2 + c^2 - 2bc\cos(A)$. [Hint: use acos. Example: input 13, 12, 5 output 90. input 10, 20, 17.32 output 30. input 7, 7, 7 output 60]
9. Write program, which reads h, k, r, and s. Let a circle has center (h,k) and radius r. Let line x=s intersects the circle. The program, the chord length. Input 2 4 13 7 output 24. [Hint: find distance of the line from the center].
10. Modify above program to find the area of the triangle formed by the points of intersection and the center of the circle. Input 2 4 13 7 output 12x13=60.
11. Modify above program to output the points of intersection of the circle and the line. Input 2 4 13 7 output (7,16) and (7,-8).
12. Read a, b, c, d, e, f, g, and h. Let ax+by+cz+d=0 be a plane and $x^2+y^2+z^2+ex+fy+gz+h=0$ be a sphere. Find the area of circle of intersection of the

plane and the sphere. e.g. input 3 4 12 17 –4 –2 –2 –19 output 50.24. Here the center of the sphere is (2,1,1). Its distance from the plane is 3. The radius of the sphere is 5. Hence the radius of the circle of intersection is 4.

If else

```
#include<stdio.h>
main( )
{  float a,b,m;
   printf("\nGive two numbers");
   scanf("%f %f",&a,&b);
   if (a>b) m=a; else m=b;
   printf("%f is bigger",m);
}
```

```
#include<stdio.h>
#include<math.h>
main( )
{  float a,b,c,det,r1,r2;
   printf("\nGive a b c");
   scanf("%f %f %f",&a,&b,&c);
   det=b*b–4*a*c;
   if (det<0)  printf("Imaginary roots");
   else {
     r1=(–b+sqrt(det))/(2*a);
     r2=(–b–sqrt(det))/(2*a);
     printf("roots %f %f",r1,r2);
   }
}
```

1. Write program, which reads three numbers and prints the biggest.
2. Write program, which reads four numbers and prints the biggest.
3. Write program, which reads 2 numbers and prints both of them. The bigger number is printed first.
4. Write program, which reads a, b and c as sides of a triangle and prints whether angle A is 90^0 or not. [Hint: if (a^2 = = b^2+c^2)] [Do not use $\cos^{-1}$ etc]
5. Write program, which reads a, b, and c. Let ax + by + c = 0 be equation of a line. Print its slope. The program also prints whether the line is vertical or not.
6. Write program, which reads a, b and c. Let ax^2 + bx + c = 0 be a quadratic equation. If roots are real and distinct then both roots are printed. If roots are equal then only one root is printed. If roots are imaginary then real part and complex parts of both roots are printed. e.g. if input is a=1 b=8 c=25 then output is –4, 3 and –3. if input a=2 b=8 c=8 output 2. Input 2 10 12 output –3 and –2.
7. Modify above program to print roots. e.g. for above input output –4+3i and –4–3i.
8. Write program, which reads three numbers. Two of these are same and one of them is different. The program outputs the different number. e.g. input 5 5 2 output 2. Input 4 3 4 output 3. Input 5 2 2 output 5.
9. Write program, which reads 5 numbers a, b, c, d, and x. Here a, b, c, and d are distinct and x is equal to exactly one of a or b or c or d. The program output which is equal to x. e.g. input 5 7 9 6 7 output x is equal to b. input 8 7 1 3 1 output x is equal to c.
10. Write program, which reads 5 numbers a, b, c, d, and x. The program output how many among a, b, c and d are equal to x. e.g. input 5 7 9 7 7 output 2. input 5 3 8 7 2 output 0. input 5 2 2 2 5 output 1.
11. Write program, which reads three numbers. The program outputs the middle of these. e.g. input 5 2 4 output 4. Input 5 6 2 output 5.
12. Read p, q, r, a, b, c. Let ax+by+c=0 be a line. Let (p,q) be the center of a circle and r be its radius. The program finds whether the circle and the line intersect or not. If they intersect let A and B be the points of intersection of the circle and the line. Find the area of the triangle formed by A, B and the center of the circle. [Hint: Find the distance of the line from the center. If it is more than the radius then circle and the

line do not intersect. Otherwise find find the chord length AB]. Input 5 4 10 1 1 20 output “no intersection”. Input 7 4 13 3 4 23 output 60.

13.

14.

19. Write program, which reads 4 numbers a, b, c and d. The program outputs the value of the expression (a+ab+abc+abcd). [Try to do this problem by using only 3 multiplications] Input 2, 1, 3, 4.5 output 37.

1. Write program, which reads x1, y1, x2, y2, and d. Let P and Q be points whose coordinates are (x1,y1) and (x2,y2) respectively. Let R be a point on the line PQ, whose distance from P is d. Assume that the length of PQ is more than d. The program outputs the coordinates of R. e.g. if input 0 0 30 40 30 output (18,24).

2. Write program, which reads h, k, r, and s. Let a circle has center (h,k) and radius r. The program, outputs the points of intersection of the circle and the line x=s. Input 2 4 13 7 output (7,16) and (7,-8).

3. Read p, q, r, a, b, c. Let ax+by+c=0 be a line. Let (p,q) be the center of a circle and r be its radius. Let A and B be the points of intersection of the circle and the line. Find the area of the triangle formed by A, B and the center of the circle. [Hint: Find the distance of the line from the center. Using it find the chord length AB].

4. Read a, b, c, d, e, f, g, and h. Let ax+by+cz+d=0 be a plane and $x^2+y^2+z^2+ex+fy+gz+h=0$ be a sphere. Find the area of circle of intersection of the plane and the sphere. e.g. input 3 4 12 17 –4 –2 –2 -19 output 50.24. Here the center of the sphere is (2,1,1). Its distance from the plane is 3. The radius of the sphere is 5. Hence the radius of the circle of intersection is 4.

5. Write program, which reads coordinates of three points (x1,y1), (x2,y2), and (x3,y3). The program outputs the center and radius the circle which passes through these points.

6. Write program, which reads coordinates of three points (x1,y1), (x2,y2), and (x3,y3). The program, outputs the area of the triangle by following method. Let these points be A, B, and C. Let l be the length of AB. Let h be the length of perpendicular from C to AB. Area=h*l/2. Print the values of h and l also.

Clean programming

Ordinary Program

```
main( )
{int x, a=5;
printf("Give x");
scanf("%d",&x);
if (x>10)
a=a+2;
else
{a=a+10;
a=a+x;}
printf("%d",a);
}
```

Clean program

```
main( )
{  int x, a=5;
   printf("Give x");
   scanf("%d",&x);
   if (x>10)
       a=a+2;
   else
   {   a=a+10;
       a=a+x;
   }
   printf("%d",a);
}
```

program when brackets of else are removed

```
main( )
{  int x,a=5;
   printf("Give x");
   scanf("%d",&x);
   if (x>10)
       a=a+2;
   else
       a=a+10;
   a=a+x;
   printf("%d",a);
}
```

For each of above case observe the output when input is x=1 and x=20.

Write clean programs for each of the following. Also observe the output on the given inputs.

1.
```
scanf("%d",&x);
if (x>10)
a=30;
else if (x>4)
a=40;
else a=50;
printf("%d",a);
```
Input x=3,6,20

2.
```
scanf("%d",&x);
if (x>10)
if (x>20)
a=30;
else a=40;
else a=50;
printf("%d",a);
```
Input x=3,16,25

3.
```
scanf("%d",&x);
if (x>10)
if (x>20)
a=30;
else if (x>15)
a=40;
else a=50;
else a=60;
printf("%d",a);
```
x=3,14,16,25

4.
```
scanf("%d",&x)
a=0;
if (x>10)
a=a+2;
if (x%2==1)
{a=a+x;
a=a+10;}
a=a+100;
printf("%d",a);
```
x=3,8,13,14

5.
```
scanf("%d",&x)
a=0;
if (x>10)
{a=a+2;
if (x%2==1)
{a=a+x;
a=a+10;}
a=a+100;}
printf("%d",a);
```
x=3,8,13,14

6.
```
scanf("%d",&x);
if (x>10)
{a=a+2;
if (x%2==1)
{a=a+x;
a=a+10;}}
a=a+100;
printf("%d",a);
```
x=3,8,13,14

7.
```
scanf("%d",&x);
a=6;
if (x>10)
if (x>20)
a=30;
else a=40;
printf("%d",a);
```
Input x=3,15,25

8.
```
scanf("%d",&x);
a=6;
if (x>10)
{if (x>20)
a=30;}
else a=40;
printf("%d",a);
```
Input x=3,15,25

9.
```
x=5;
while (x<80)
x=x+9;
x=x+10;
printf("%d",x);
```

10.
```
x=5;
while (x<80)
{x=x+9;
x=x+10;}
printf("%d",x);
```

11.
```
x=5;y=5;
while (x<80)
x=x+9;
if (x<60)
y=y+2;
printf("%d,%d",x,y);
```

12.
```
x=5;y=5;
while (x<80)
{x=x+9;
if (x<60)
y=y+2;}
printf("%d,%d",x,y);
```

Boolean connectives

```
main( )
{  int x;
   for (x=0;x<100;x++)
      if (((x>50) && (x<70)) || (x%2==0))
         printf("%d,",x);
}
```

The given program prints numbers between 0 and 100, which are either even or between 50 and 70.

```
main( )
{  int x;
  for (x=0;x<100;x++)
     if (((x>20)&&(x<70))&& !((x>40)&&(x<60)))
         printf("%d,",x);
}
```

The given program prints numbers which are between 20 and 70 but not between 40 and 60.

Write following programs by modifying the line containing *if* only.

1. Write a program, which prints all even numbers between 20 and 70.
2. Write program, which prints all numbers between 20 and 40, and all even numbers between 50 and 80.
3. Write program, which will print all even numbers less then 50 and all odd numbers more than 50.
4. Write program, which prints all even numbers between 20 and 40, and all odd numbers between 50 and 80.
5. Write program, which will print all numbers which are multiple of either 3 or 7.
6. Write program, which will print all numbers which are either between 50 and 70, or less than 20, or more than 90.
7. Write program, which will print all numbers, which are even but not a multiple of either 3 or 5. e.g. 2 4 8 14 16 22 ….
8. Write program, which will print all numbers, which are either a multiple of 3 or 5 but not both.
9. Write program, which will print those numbers whose last digit is multiple of 3. e.g. 0, 3, 6, 9, 10, 13, 16, 19, 20, 23, …..
10. Write program, which will print those numbers whose last digit is between 5 and 8. e.g. 5, 6, 7, 8, 15, 16, 17, 18, 25, 26, ….
11. Write program, which will print those numbers whose sum of both digits is multiple of 7. e.g. 0,7,16, 25, 34, 43, 52, 59, 61, …..
12. Write program, which will print all numbers between 10 and 19, 30 and 39, 50 and 59, … , 90 and 99. [Hint: check condition ((x/10)%2) == 1].
13. Write program, which will print those numbers whose first digit leaves remainder 1 when divided by 3. e.g. 10, 11, ..., 19,40, 41, …, 49, 70, 71, ..., 79.
14. Write program, which will print all odd numbers between 0 and 9, 20 and 29, 40 and 49, … , 80 and 89 and all even numbers between 10 and 19, 30 and 39, … , 90 and 99.

15. Write program, which will print all odd numbers between 0 and 9, 20 and 29, 40 and 49, … , 80 and 89 and all even numbers between 10 and 19, 30 and 39, … , 90 and 99. [Hint: check condition ((x/10)%2) != (x%2)].
16. Write program, which will print all even numbers between 10 and 29, 40 and 59, 70 and 89.
17. Write program, which will print all odd numbers between 0 and 9, 30 and 39, 60 and 69, 90 and 99 and all even numbers between 10 and 29, 40 and 59, 70 and 89.
18. Write program, which will print those numbers between 10 and 99, which after exchange of digits become divisible by 7. e.g. 12, 19, 24, 36, 41, 48, 53, 65, 70, 77, 82, 89, 94.

```
#include<stdio.h>
main()
{  int x,y;
   for (y=0;y<10;y++)
  {   for (x=0;x<20;x++)
        if ((x+y<6) || ((x>10) && (y<8)))
           printf("*");
        else printf("O");
      printf("\n");
  }
}
```

Write programs for each of followings by modifying the line containing ***for*** and ***if*** only.

```
            Picture 1
********************
*O*O*O*O*O*O*O*O*O*O
********************
*O*O*O*O*O*O*O*O*O*O
********************
            Picture 2
OOOOOOOO************
OOOOOOOO************
OOOOOOOO************
********************
********************
********************
********************
***************OOOOO
***************OOOOO
***************OOOOO
            Picture 3
O*O*O*O*O*
*O*O*O*O*O
O*O*O*O*O*
*O*O*O*O*O
             Picture 4
OOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOO
*OO*O**OO**O*OO*OO*O**OO**O*OO
OOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOO
*OO*O**OO**O*OO*OO*O**OO**O*OO
OOOOOOOOOOOOOOOOOOOOOOOOOOOOOOOO
*OO*O**OO**O*OO*OO*O**OO**O*OO
            Picture 5
********************
*OOOO*OOOO*OOOO*OOOO
********************
*OOOO*OOOO*OOOO*OOOO
********************
```

```
The program outputs
******OOOOO*********
*****OOOOOO*********
****OOOOOOO********
***OOOOOOOO********
**OOOOOOOOO********
*OOOOOOOOOO********
OOOOOOOOOOO********
OOOOOOOOOOO********
OOOOOOOOOOOOOOOOOOOO
OOOOOOOOOOOOOOOOOOOO
```

```
          Picture 6
O****O****O****O****
*OOOO*OOOO*OOOO*OOOO
O****O****O****O****
*OOOO*OOOO*OOOO*OOOO
          Picture 7
OOOOOOOOOOOOOOOOOO
OOOOOO*****OOOOOO
OOOO*********OOOO
OOO***********OOO
OO*************OO
OO*************OO
OO*************OO
OOO***********OOO
OOOO*********OOOO
OOOOOO*****OOOOOO
OOOOOOOOOOOOOOOOOO
           Picture 8
OOOOOO*****OOOOOOOOO
OOOOO******OOOOOOOOO
OOOO*******OOOOOOOOO
OOO********OOOOOOOOO
OO*********OOOOOOOOO
O**********OOOOOOOOO
***********OOOOOOOOO
***********OOOOOOOOO
********************
********************
           Picture 9
********************
********************
********************
*****OO*************
****OOOO************
***OOOOOO***********
**OOOOOOOO**********
*OOOOOOOOOO*********
OOOOOOOOOOOO********
```

Picture 6

```
OOOOOOOOOOOOOOO
OO*O*O*O*O*O*O*
O*OOOOOOOOOOOOO
OOOO*O*O*O*O*O*
O*O*OOOOOOOOOOO
OOOOOO*O*O*O*O*
O*O*O*OOOOOOOOO
OOOOOOOO*O*O*O*
O*O*O*O*OOOOOOO
OOOOOOOOOO*O*O*
```

Picture 7

```
********************
OOOOOOOOOOOOOOOOOOOO
********************
OOOOOOOOOOOOOOOOOOOO
********************
OOOO**********OOOOOO
********************
OOOO**********OOOOOO
********************
OOOO**********OOOOOO
```

Picture 10

```
********************
********************
******OO************
*****OOOOO**********
****OOOOOOOO********
***OOOOOOOOOOOO******
**OOOOOOOOOOOOOO****
*OOOOOOOOOOOOOOOOOO**
OOOOOOOOOOOOOOOOOOOO
OOOOOOOOOOOOOOOOOOOO
```

Picture 13

```
OO*O*O*O*O*O*O*
OO*O*O*O*O*O*O*
**OO*O*O*O*O*O*
OOOO*O*O*O*O*O*
****OO*O*O*O*O*
OOOOOO*O*O*O*O*
******OO*O*O*O*
OOOOOOOO*O*O*O*
********OO*O*O*
OOOOOOOOOO*O*O*
```

```
#include<stdio.h>
main()
{   int a,b;float c,d;
    c=5/2;
    d=5.0/2;
    a=5.0/2;
    printf("%f %f %d",c,d,a);
}
```

```
#include<stdio.h>
main()
{   int a,b;float c;
    float p,s;int q,r;
    a=5;b=2;
    p=5;q=2;
    c=a/b;
    r=p/q;
    s=p/q;
    printf("%f %d %f",c,r,s);
}
```

```
#include<stdio.h>
main()
{   int a,b,c;
    float p,q,r;
    a=7;b=2;p=2;
    q=a/b/p;
    r=a/p/b;
    printf("%f %f",q,r);
}
```

```
#include<stdio.h>
main()
{   int a=17,b=3,p,q,r;
    p=a/b + a/b;
    q=(2*a)/b;
    r=2*(a/b);
    printf("%d %d %d",p,q,r);
}
```

```
#include<stdio.h>
main()
{   float a,b,c,d,e;
    scanf("%f%f%f",&a,&b,&c);
    d=a-b-c;
    e=a/b/c;
    printf("%f %f\n",d,e);
}
```

```
#include<stdio.h>
main()
{   int a,x;
    a=0;
    scanf("%d",&x);
    if (x<10){
       a=a+2;
       a=a+4;}
    printf("  %d   \n",a);
}
```

Observe output on inputs x=20 and 3. Also observe the output when brackets in if are removed. You are advised to shift (right) the child statements.

```
#include<stdio.h>
main()
{   int x=70;
    while (x>30){
       x=x-9;
    printf("  %d\n",x);}
}
```

1.Put print before x=x-9.
2.Observe the output when brackets are removed.

```
#include<stdio.h>
main()
{   int x=100,i=0;
    while (x>8)
    {   x=x/2;i=i+1;  }
    printf("%d %d\n",x,i);
}
```

Switch and Condition

```
switch(expression) {

   case constant-expression  :
      statement(s);
      break; /* optional */

   case constant-expression  :
      statement(s);
      break; /* optional */

   /* you can have any number of case statements */
   default : /* Optional */
   statement(s);
}
```

```
#include <stdio.h>

int main () {

   /* local variable definition */
   char grade = 'B';

   switch(grade) {
      case 'A' :
         printf("Excellent!\n" );
         break;
      case 'B' :
      case 'C' :
         printf("Well done\n" );
         break;
      case 'D' :
         printf("You passed\n" );
         break;
      case 'F' :
         printf("Better try again\n" );
         break;
      default :
         printf("Invalid grade\n" );
   }

   printf("Your grade is  %c\n", grade );

   return 0;
```

1. Write program, which reads an integer X and prints an integer Y. Y is X+10 if X is between 10 and 30. Y is 3*X if X is between 50 and 70. Otherwise Y is X-2.

2. A student is awarded Ex grade if he gets more than 90 marks. He is awarded A grade if marks are between 80 and 89. Similarly range for B, C, D and P are 70-79, 60-69, 50-59, and 35-49 respectively. The student is awarded F grade if he gets less then 35 marks. Write a program, which reads marks of a student and prints his grade.

3. No income tax is to be paid if income is less than 5000. If income is between 5000 and 6000 then tax is 10% of the amount by which the income exceeds 5000. If income is between 6000 and 15000 then the tax is 100 + 20% of the amount by which the income exceeds 6000. If income is more than 15000 then the tax is 1900 + 30% of the amount by which the income exceeds 15000. e.g. if income is 10000 then the tax will be 100 + (10000-6000)*20/100 = 900. Write a program, which reads income and calculates the income tax.

4. Write a program, which reads a number X and prints a number Y. Y=X+10 if X is 6. Y is X*X if X is 7. Y is 2*X+4 if X is 12. Otherwise Y is X*6-1.

5. Write a program, which reads three integers X, Y and Z and prints Y+Z if X is 0. If X is 1 then Y-Z is printed. If X is 2 then Y*Z is printed. If X is 3 then Y/Z is printed.

Coordinate Geometry

1. Write program which reads x_1, y_1, r_1, x_2, y_2, r_2. (x_1,y_1) is center of a circle and r_1 is its radius. Similarly (x_2,y_2) and r_2 are center and radius of second sphere. Write program, which outputs their points of intersection. If these circles do not intersect then the output is NO POINT OF INTERSECTION.

2. Write program to output points of intersection of three spheres.

3. Write program which reads x_1, y_1, x_2, y_2, x_3, y_3. (x_1,y_1), (x_2,y_2) and (x_3,y_3) are coordinates of three points on circumference of a circle. The program outputs the center and radius of the circle.

4. Write program, which reads coordinates of four points on the surface of sphere. The program outputs the center and radius of the sphere.

5. Write program which reads coordinates of two points (x_1,y_1) and (x_2,y_2) and outputs three numbers a,b, and c where $ax + by + c = 0$ denotes a line passing through points (x_1,y_1) and (x_2,y_2).

6. Write program which reads coordinates of two points (x_1,y_1,z_1) and (x_2,y_2,z_2) and outputs six numbers h, k, l, p, q, and r. It denotes the following line $\frac{x-h}{p} = \frac{y-k}{q} = \frac{z-l}{r}$.

7. Write program which reads h, k, l, p, q, r, x_1, y_1, and z_1. Here h, k, l , p, q, r denotes a line as described above. (x_1,y_1,z_1) is a point. The program outputs the distance of the point from the line.

8. Write program, which takes above input and outputs four numbers a, b, c, and d. The equation of the plane passing through the given line and the plane is $ax + by + cz + d = 0$.

9. Write program, which reads a, b, c, d, x_1, y_1, and z_1. Here $ax + by + cz + d = 0$ denotes a plane and (x_1,y_1,z_1) denotes a point. The program outputs the distance of the point from the plane.

10. Write program which reads a_1, b_1, c_1, d_1, a_2, b_2, c_2 and d_2. Here $a_1x + b_1y + c_1z + d_1 = 0$ and $a_2x + b_2y + c_2z + d_2 = 0$ are two planes. The program outputs six numbers h, k, l, p, q, and r. These denote a line as described in question no 6.

11. Write program which reads h, k, l, p, q, r, a, b, c, and d. These represent a line and a plane respectively. The program outputs their point of intersection.

12. Write program, which reads two lines and outputs the distance between them.

13. Write program, which reads h, k, l, p, q, r, x_1, y_1, z_1, and r. Here h, k, l, p, q, and r represents a line. (x_1,y_1,z_1) is center of a sphere and r is its radius. The program outputs their points of intersection. If the line does not intersect with the sphere the program outputs NO INTERSECTION.

14. Write program, which reads a, b, c, d, x_1, y_1, z_1, and r. Here a, b, c, and d denote a plane. x_1, y_1, z_1, and r denote a sphere. The program outputs the coordinates of the center and radius of the circle of intersection. If the plane does not intersect with the sphere the program outputs NO INTERSECTION.

Series

```
double x,sum,term,fact; int i,n;
printf("Give x and n");
scanf("%lf  %d",&x,&n);
term=1; sum=1; fact=1;
for (i=1;i<=n;i++)
{   term=term*x;
    fact=fact*i;
    sum=sum+term/fact;
}
printf("The answer is %lf",sum);
printf("\nThe exp(x) is %lf",exp(x));
```

The given program finds the sum of $1+x+\frac{x^2}{2!}+\frac{x^3}{3!}+\ \ldots\ldots+\frac{x^n}{n!}$

When n is large the above become e^x. input x=2 output e^2=7.39

Write program to find sum of following series. Do not use more than one loop. Function 'pow' should not be used. To verify the correctness of the result print the value of the function.

1. $1+2x+3x^2+4x^3+\ldots+(n+1)x^n$ It is $\frac{1}{(1-x)^2}$ for large n and |x|<1 input 0.3 output 2.04
2. $1-x+x^2-x^3+\ldots+(-1)^n x^n$ It is $\frac{1}{(1+x)}$ for |x|<1 input x= 0.3 n=100 output 0.769
3. $1-x-x^2+x^3+x^4-x^5-x^6+x^7+\ \ldots.$ n terms input x= 0.3 n=100 output 0.642
4. $x+\frac{x^2}{2}+\frac{x^3}{3}+\ldots+\frac{x^n}{n}$ It is –log(1– x) for |x|<1 input x= 0.3 n=100 output 0.356
5. $1-\frac{x^2}{2!}+\frac{x^4}{4!}-\frac{x^6}{6!}+\ldots$ It is cos(x) input x=0.3 n=10 output 0.95 x=1.05 output 0.5
6. $x-\frac{x^3}{3!}+\frac{x^5}{5!}-\frac{x^7}{7!}+\ldots$ It is sin(x) input x=0.3 n=10 output 0.29 x=1.05 output 0.866
7. $x+3x^2+6x^3+10x^4+15x^5\ldots$ It is $\frac{x}{(1-x)^3}$ for |x|<1 input x= 0.3 n=100 output 0.874
8. $x+x^2+x^4+x^5+x^7+x^8+x^{10}+x^{11}+x^{13}+\ \ldots\ldots$ For x=2 and n=5 the answer is 182 or 54. For x=1 and n=10 the answer is 10 or 7. Do it for both answers.

For followings, write program which reads x and n and finds f(n).

9. f(0)=x f(1)=x+1 f(2)=2(x+1)+1 f(3)=3(2(x+1)+1)+1 … for x=1 f(5)=326
10. f(0)=x f(1)=x+1 f(2)=2(x+1)+1 f(3)=4(2(x+1)+1)+1 f(4)=8(4(2(x+1)+1)+1)+1 f(5)=16(8(4(2(x+1)+1)+1)+1)+1 f(6)=32(16(8(4(2(x+1)+1)+1)+1)+1)+1 . . . . . for x=1 f(5)=2705, f(7)=5539905
11. f(0)=x f(1)=x^2 f(2)=x^3+x f(3)=x(x + x^3)+x^2 f(i)=xf(i–1)+f(i–2) for x=2 f(5)=140
12. $f(1)=1+\frac{1}{1}$ $f(2)=1+\cfrac{1}{1+\frac{1}{2}}$ $f(3)=1+\cfrac{1}{1+\cfrac{1}{1+\frac{1}{3}}}$ $f(4)=1+\cfrac{1}{1+\cfrac{1}{1+\cfrac{1}{1+\frac{1}{4}}}}$ f(7)=1.616

13. $f(1)=1+\frac{1}{1}$ $\quad f(2)=1+\cfrac{2}{1+\cfrac{1}{2}}$ $\quad f(3)=1+\cfrac{3}{1+\cfrac{2}{1+\cfrac{1}{3}}}$ $\quad f(4)=1+\cfrac{4}{1+\cfrac{3}{1+\cfrac{2}{1+\cfrac{1}{4}}}}$ $\quad f(7)=3.18$

Picture Printing

```
#include<stdio.h>
main()
{  int x;
   printf("Give number");
   scanf("%d",&x);
   printf("ram%com\n",x);
}
```

It outputs the character, whose ASCII code is given.

```
#include<stdio.h>
main()
{  char c;
   c=getchar();
   printf("%d",c);
}
```

It outputs ASCII code of a character.

```
#include<stdio.h>
main()
{  int x;
   printf("Give a number");
   scanf("%d",&x);
   printf("gopal");
   fflush(stdout);sleep(2);
   printf("%c",x);
   fflush(stdout);sleep(2);
   printf("hari\n");
} Give input 8 and 13
```

```
main()
{  int i;
   for (i=1;i<=5;i=i+1)
      printf("%c",65+i);
}
```

The give program outputs BCDEF.
Modify "65+i" to print followings:

1. ABCDE CEGIK FEDCB ABBCC

```
#include<stdio.h>
main()
{  int i;
   for (i=65;i<=70;i=i+1)
      printf("%c",i);
}
```

The give program outputs ABCDEF.
Modify "`for (i=65;i<=70;i++)`" to print followings:

2. BCDEFGHIJ ACEGI FEDCBA

The given program prints following picture.

ABCDE
BCDE
CDE
DE
E

```
for (line=1;line<=5;line++)
{  for(i=1;i<=6-line;i++)
      printf("%c",63+line+i);
   printf("\n");
}
```

In the give program replace “63+line+i” by suitable expression to print following pictures.

3.
ABCDE
ABCD
ABC
AB
A

5.
AAAAA
BBBB
CCC
DD
E

7.
EFGHI
DEFG
CDE
BC
A

9.
ABCDE
CDEF
EFG
GH
I

4.
EDCBA
DCBA
CBA
BA
A

6.
EEEEE
DDDD
CCC
BB
A

8.
EDCBA
FEDC
GFE
HG
I

10.
ABCDE
ACEG
ADG
AE
A

The given program prints following picture.

AAAAA
AAAA
AAA
AA
A

```
for (line=1;line<=5;line++)
{  for(i=1;i<=6-line;i++)
      printf("A");
   printf("\n");
}
```

In the give program replace `6-line` by suitable expression so that following pictures are printed.

11.

```
A
AA
AAA
AAAA
AAAAA
```

12.

```
AA
AAA
AAAA
AAAAA
AAAAAA
```

13.

```
A
AAA
AAAAAA
AAAAAAAAAAA
AAAAAAAAAAAAAAAA
```

14.

```
AAAAAAAAAAA
AAAAAAAA
AAAAAA
AAAA
AA
```

ABCDE
BDFH
CFI
DH
E

```
#include<stdio.h>
main()
{   int i;
    for (i=65;i<=90;i++)
        printf("%c",i);
}
output  ABCD .... Z

int i,gr;
for(gr=1;gr<=5;gr++)
   for (i=1;i<=6-gr;i++)
     printf("%c",64+gr);

output AAAAABBBBCCCDDE
```

What output will be produced if 64+gr is replaced by (A)64+i (B)70–gr (C)70–i (D) 63+i+gr

```
for (gr=1;gr<=5;gr++)
{  for (i=1;i<=gr;i++)
      printf("A");
   for (i=5-gr;i>=1;i--)
      printf("X");
}
AXXXXAAXXXAAAXXAAAAXAAAAA

int i,gr;
for (gr=1;gr<=4;gr++)
{  for (i=65;i<=69;i++)
       printf("%c",i);
   printf("X");
}
ABCDEXABCDEXABCDEXABCDEX

int i,line;
for (line=1;line<=6;line++)
{  for(i=64+line;i<=70;i++)
       printf("%c",i);
   printf("\n");
}
ABCDEF
BCDEF
CDEF
DEF
EF
F
```

In the previous problem the inner loop can be replaced as follows

```
for (i=1;i<=7-line;i++)
    printf("%c",63+i+line);

for (line=1;line<=6;line++)
{ for (i=1;i<=6-line;i++)
     printf(" ");
  for (i=65;i<=64+line;i++)
     printf("%c",i);
  printf("\n");
}    A
    AB
   ABC
  ABCD
 ABCDE
ABCDEF

for (line=1;line<=5;line++)
{ for (i=1;i<=line;i++)
    printf("%c",96+i);
  for (i=70-line;i>=65;i--)
    printf("%c",i);
  printf("\n");
}
aEDCBA
abDCBA
abcCBA
abcdBA
abcdeA

int i,line;char p;
for (line=1;line<=3;line++)
{ for (i=97;i<=100;i++)
  {   p=(char)i;
      printf("%c",p);
  }
  for (i=65;i<=71+line;i++)
      printf("%c",(char)i);
  for (i=line-1;i<=4;i++)
      printf("%d",i);
  printf("\n");
}
abcdABCDEFGH01234
abcdABCDEFGHI1234
abcdABCDEFGHIJ234
```

Write program to print following pictures

1. AAAAAA
2. AAABBBCCCDDD
3. ABCABCABCABC
4. ABBCCCDDDD
5. AAAABBBCCD
6. AAAXAAAXAAAX
7. AaAAaAAAaAAAAa
8. ACEGIKM

9. ABCDEF
   ABCDE
   ABCD
   ABC
   AB
   A

10. A
    AB
    ABC
    ABCD
    ABCDE
    ABCDEF

11.
```
ABCDEF
 BCDEF
  CDEF
   DEF
    EF
     F
```

12.
```
Aabcde
ABabcd
 ABCabc
 ABCDab
 ABCDEa
 ABCDEF
```

13.
```
aBCDEF
abCDEF
abcDEF
abcdEF
abcdeF
```

14. A
    AA
    AAA
    AAAA
    AAAAA
    AAAAAA

[Solve following two problems by replacing 65 by … in above]

15. A
    BC
    CDE
    DEFG
    EFGHI
    FGHIJK

16. ABCDEF
    ACEGI
    ADGJ
    AEI
    AF
    A

17. ABCDEF
    BBCCDDEEFF
    CCCDDDEEEFFF
    DDDDEEEEFFFF
    EEEEEFFFFF
    FFFFFF

18.
```
     A
    ABA
   ABCBA
  ABCDCBA
ABCDEDCBA
```

19.
```
     F
    EF
   DEF
  CDEF
 BCDEF
ABCDEF
```

20.
```
     K
    JI
   IHG
  HGFE
 GFEDC
FEDCBA
```

21. A
    BC
    DEF
    GHIJ
    KLMNO
    PQRSTU

22. A
    AA
    AAAA
    AAAAAAA
    AAAAAAAAAAA

23. A
    BC
    CDEF
    DEFGHIJ
    EFGHIJKLMNO

24.
```
U
ST
PQR
LMNO
GHIJK
ABCDEF
```

25. ABDGKPV
26. 1,2,3,4,5,6,

27. 0,4,8,12,16,20,24,28,
    0,5,10,15,20,25,30,
    0,6,12,18,24,30,
    0,7,14,21,28,
    0,8,16,24,
    0,9,18,27,
    0,10,20,30,

28. 0,4,8,12,16,20,24,28,
    0,5,10,15,20,25,30,

0,6,12,18,24,30,
0,7,14,21,28,
0,8,16,24,
0,9,18,
0,10,

29. 1
1 3
1 3 6
1 3 6 10
1 3 6 10 15
1 3 6 10 15 21

30. A
BC
CDEF
DEFGHIJK
EFGHIJKLMNOPQRST

31. ABCDEBCDECDEDEE
ABCDEBCDECDEDE
ABCDEBCDECDE
ABCDEBCDE
ABCDE

32. ABCDEBCDECDEDEE
ABCDBCDCDD
ABCBCC
ABB
A

33. ABCDEABCDABCABA
BCDEBCDBCB
CDECDC
DED
E

34. ABCDEFGH
ABCDEFGABCDEF
ABCDEFABCDEABCD
ABCDEABCDABCAB

35. ABCDEBCDECDEDEE
BCDECDEDEE
CDEDEE
DEE
E

36. ABCDEABCDABCABA
ABCDABCABA
ABCABA
ABA

37. AABABCABCDABCDE
BBBBCCBBCCDDBBCCDDEE
CCCCCCDDDCCCDDDEEE
DDDDDDDDEEEE
EEEEE

38. ABBCCCDDDDEEEEE
AABBBCCCCDDDDD
AAABBBBCCCCC
AAAABBBBB
AAAAA

39. ABCDEABCDABCABA
BBCCDDEEBBCCDDBBCCBB
CCCDDDEEECCCDDDCCC
DDDDEEEEDDDD
EEEEE

40. EFGHIEFGHEFGEFE
DEFGDEFDED
CDECDC
BCB
A

41. ABCDEFGHIJKLMNOPQRST
ACEGIKMOQS
ADGJMPS
AEIMQ

42. Read two integers a and b and print following picture

(a)(b)
(a)(b)(a+b)
(a)(b)(a+b)(a+2b)
(a)(b)(a+b)(a+2b)(2a+3b)
(a)(b)(a+b)(a+2b)(2a+3b)(3a+5b)
(a)(b)(a+b)(a+2b)(2a+3b)(3a+5b)(5a+8b)

If a=5 and b=3 then picture is

(5)(3)
(5)(3)(8)
(5)(3)(8)(11)
(5)(3)(8)(11)(19)
(5)(3)(8)(11)(19)(30)
(5)(3)(8)(11)(19)(30)(49)

43. 101 102 106 124 220 820
102 106 124 220 820
106 124 220 820
124 220 820
220 820
820

Digits

```
int x,k,i,sum=0;
scanf("%d%d",&x,&k);
for (i=1;i<=k;i++)
{  sum=sum+(x%10);
   x=x/10;
}
printf("%d ",sum);
```

The program to find the sum of last k digits of a number.

```
int x,d;
printf("Give number:");
scanf("%d",&x);
do
{  d=x%10;
   printf("%d,",d);
   x=x/10;
} while (x>0);
```

The program to print all digits of a number

```
int x,d,y=0,t=1;
scanf("%d",&x);
while (x>0)
{  d=x%10;
   if (d%2==0)
   {  y=y+t*d;
      t=t*10;  }
   x=x/10;   }
printf("%d",y);
```

program deletes odd digits.

In following programs only one loop should be used unless essential

1. Write program to delete the last k digits. e.g. input 23617 and k=3 output 23.
2. Write program to print the kth digit from last. e.g. input 23617 and k=4 output 3.
3. Write a program to print first digit. e.g. input 23516 output 2.
4. Write program to print the second digit. e.g. input 23516 the output is 3.
5. Write program to find sum of all digits. Input 23617 output 2+3+6+1+7=19.
6. Write program, which will find sum of product to consecutive digits. e.g. when the input is 23145 the output is 2x3 + 3x1 + 1x4 + 4x5 = 33.
7. Write program, which reads two number (assume that both have same number of digits). The program outputs the sum of product of corresponding digits. Input 327 and 539 output 3x5+2x3+7x9=84.
8. Write program to print positional values of digits. Input 21463 output 3, 60, 400, 1000 and 20000.
9. Write program to find sum of even digits. Input 23617 output 2+6=8.
10. Write program to find number of digits. Input 423 output 3. Input 21151 output 5.
11. Write program to find number of even digits. In above case 2 and 1 respectively.
12. Write program to print the last even digit. e.g. input 23613 output 6. [Hint: while ($x\%2 \neq 0$) x=x/10]
13. Program to print the digit immediately before the last even digit. In above case 3.
14. Write program to print the digit immediately after the last even digit. In above case 1.
15. Write program to print the last digit, which is multiple of 3. e.g. input 23617 output 6.
16. Write program to print the second last even digit. e.g. input 23863 output 8 (do not use ‘if’). Input 325145761 output 4. [Hint: use two loops]
17. Read a number. Do half of number after last odd digit. Input 3 times. Input 61389426 output 184167639 (61389213*3). Input 87 output 261. Input 78 output 222 (74*3).
18. Write program, which finds the sum of numbers formed by consecutive digits. Input 2415 output 24+41+15=80.
19. Find sum of numbers formed by exchanging consecutive digits. In above 42+14+51=107.
20. Write program to print the first even digit. e.g. input 71236 output 2.
21. Write program to print digits, which are multiple of 3. e.g. input 23617 output 6,3.
22. Write program, which outputs the sum of digits at even location. Input 2314 output 3+4=7. Input 72314 output 2+1=3. Input 572314 output 7+3+4=14.

23. Write a program, which reads an integer and prints another integer. In the output all even digits are retained and odd digits are decrement. e.g. input 23145 output 22044.
24. Write program to prints the biggest digit and its location (from last). e.g. in 23176 biggest digit is 7 and its location is 2nd from last.
25. Write program to prints the biggest digit and its location (from beginning). e.g. in 23176 biggest digit is 7 and its location is 4th from beginning.
26. Write a program, which reads an integer and prints another integer. The output is reverse of the input. e.g. for input 27921 the output is 12972. Also print the double of the answer (i.e. 25944).
27. Write program to print the digit immediately before the first even digit. e.g. input 71236 output 1.
28. Write program to print the digit immediately after the first even digit. In above case 3.
29. Write program to print the kth even digit from last. e.g. input 23687 and k=2 output 6.
30. Write program, which deletes all digits except last k digits. e.g. input 23617 and k=3 output 617.
31. Write program to delete the kth digit from last. e.g. input 23617 and k=4 output 2617.
32. Write program to print the kth digit e.g. input 23617 and k=4 output 1.
33. Write program to delete first k^{th} digits e.g. input 23617 and k=2 output 617.
34. Write program, which reads a number the finds special sum. If number is 2314 then output is 4 + 14 + 314 + 2314 = 2646. If input number is 223 then output is 3 + 23 + 223 = 249. If input is 12564 then output is 4 + 64 + 564 + 2564 + 12564 = 15760.
35. Write program, which reads a number the finds special sum in the following way. If number is 2314 then output is 2314 + 231 + 23 + 2 = 2570. If input number is 223 then output is 223 + 22 + 2 = 247.
36. Write program, which reads a number the finds special product. If number is 2314 then output is 4 + 1x4 + 3x1x4 + 2x3x1x4 = 44. If input number is 223 then output is 3 + 2x3 + 2x2x3 = 21.
37. Write program, which reads a number and finds the location of last even digit. If the number is 18263 then the output is 4 since the last even digit 6 occurs at 4th place.
38. Write program, which reads a number and finds the location of first even digit. If the number is 18263 then the output is 2 since the first even digit occurs at 2nd place.
39. Write program, which reads a number and prints a number after deleting the first digit. If input is 2314 then output is 314. If input is 56 then output is 6.
40. Write program, which reads a number x and a digit (integer) y and puts y as first digit in x. If x=2531 and y=8 then output is 82531.
41. Write program, which reads a number and adds 1 to every digit. If input is 2314 then output is 3425. Assume that no digit is 9.
42. Write program, which reads a float number and an integer k and prints number by adding 1 to the first k digits. If input is 0.241573 and k is 4 then output is 0.352673. Assume that given number is between 0 and 1.
43. Write a program, which reads an integer and prints another integer. In the output all even digits are retained and odd digits are removed. e.g. input 23145 output 24.
44. Write a program, which reads an integer (n) and n digits of a number (integer) in reverse order. The program outputs the number. e.g. n=5 and digits are 3 2 5 1 2 the output is 21523.

45. Write a program, which reads two integers a and b and outputs a/b correct to 5 places of decimal. The program should not use any float variable or “%f”. (assume a<b)
46. Write a program, which reads a float number (between 0 and 1) and prints the sum of its first five digits. e.g. input 0.231562 output 2+3+1+5+6 = 17.
47. Write a program, which reads an integer (n) and n digits of a float number (between 0 and 1) in correct order. The program outputs the number. e.g. n=5 and digits are 3 2 5 1 2 the output is 0.32512.
48. Write a program which reads a float x (between 0 and 1) and outputs a float y. Every digit of y is one more than the corresponding digit in x. e.g. input 0.23156 output 0.34267. We assume that all digits of x are less then 9.

Factors

```
int i,x;
printf("Give number");
scanf("%d",&x);
for (i=2;i<=x;i++)
  if (x%i==0)
    printf("%d\n",i);
```

prints all factor

```
int i,x;
scanf("%d",&x);
i=2;
while (x%i!=0)
  i=i+1;
printf("%d",i);
```

Finds smallest factor

```
int i,x;
printf("Give number");
scanf("%d",&x);
for (i=2;x%i!=0;i++);
printf("%d",i);
```

Alternative method to find the smallest factor

The number 385 has factors 5, 7, 11, 35, 55, 77, 385. The smallest factor is 5.

1. Write program, which will print last digits of all factors. Input 385 output 5 7 1 5 5 7 5.
2. Write program, which will output all factors by exchanging last two digits. Input 385 output 50 70 11 53 55 77 358.
3. Write program, which prints sum of all factors (input 24 output 2 + 3 + 4 + 6 + 8 + 12 + 24 = 59).
4. Write program, to find the number of factors. Input 24 output 7. Input 35 outout 3.
5. Write program, which reads a number and prints “yes” if the given number is prime. If the given number is not a prime then “no” is printed. (Hint: Smallest factor of prime number is equal to itself). Input 24 output “no”. Input 13 output “yes”.
6. Write program, which outputs all factors and its count. When number is 24 (1,2) (2,3) (3,4) (4,6) (5,8) (6,12) (7,24) will be printed.
7. Write program, which reads another integer k and prints the kth factor. If given number is 24 and k is 5 then output is 8.
8. Write program, which reads a number and prints only those factors, which leave remainder 2 when divided by 5. e.g. if given number is 24 then its factors are 2, 3, 4, 6, 8, 12, and 24. However, only 2 and 12 will be printed.
9. Write program, which prints biggest factor other than itself.
10. Write program, which prints the smallest and second smallest factors. If number is 24 then outputs are 2 and 3. If number is 65 then outputs are 5 and 13.
11. Write program, which reads two numbers and finds the sum of smallest factors. e.g. input number 35 and 21 output 5+3=8.
12. Write program, which reads a number. Let ‘t’ be its smallest factor. The program finds the smallest factor of t+2. e.g. input 77 output 3 (since smallest factor of 77 is 7 and smallest factor of 7+2 is 3). Input 55 output 7.
13. Write program, which reads two numbers and prints all common factors. If given numbers are 24 and 54 then outputs are 2, 3 and 6.
14. Modify the above program to print the greatest common factor. Input 24 35 output 1.
15. Modify the above program to print the smallest common factor. Input 24, 15 output 3.
16. Write program, which reads two numbers and finds the least common multiplier. e.g. input 24 and 54 then output is 216. [Do not use the method of (a*b/gcd(a,b)).] [Hint: modify above program by replacing x%i by i%x]
17. Write program, which reads a number and prints all factors using ‘while’ (in place of ‘for’).
18. Write program, which finds the sum of product of consecutive factors. e.g. if given number is 12 then answer is 114. Since 2.3+3.4+4.6+6.12=114.

Revision

```
scanf("%d",&x);
do
{  d=x%10;
   x=x/10;
}while (d%2 ! =0);
printf("%d",d);
```

The given program reads a number and finds the last even digit. To find first digit while (d%2 ! =0) is replaced by while (x>0).

19. Modify above program to find the second digit, sum of digits, maximum digit, first even digit.
20. Write program to find the number of even digits, sum of even digits.
21. Write program to print all even digits.
22. Modify above program to find the kth last even digit. If x=214358632 and k=3 then output is 8.

```
scanf("%d",&x);
i=1;
do{
    i=i+1;
} while (x%i ! =0);
printf("%d",i);
```

```
scanf("%d",&x);
i=1; s=0;
do{
   i=i+1;
   if (x%i==0)
      s=s+i;
} while (i<=x);
printf("%d",s);
```

The first program finds the smallest factor. The second program finds the sum of factors.

23. Modify these programs to find number of factors.
24. Modify above program to find k^{th} factor. If x=130 and k=4 then output is 13.
25. Write program, which reads two numbers and finds the smallest common factor of them. If input is 24 and 21 then output is 3.
26. Modify above program to find biggest common factor. Input 42 and 54 output 6.
27. Modify the program of smallest common factor, so that smallest common multiplier is found. Input 24 and 21 output 168.

```
scanf("%d%d",&x,&y);
p=1;
for (i=1;i<=y;i++)
  p=p*x;
printf("%d",p);
```

The given program reads two numbers x and y and finds x^y. Write following programs by modifying it.

28. Write program which reads a number x and finds x!. Input x=5 output 1*2*3*4*5=120.
29. Write program which reads two numbers x and y. The program finds the sum of numbers between. If x=5 and y=8 then output is 5+6+7+8=26. Assume x is less than y.

```
scanf("%d",&x);
i=1;
while(i*i<=x)
  i=i+1;
printf("  %d",i-1);
```

The given program reads a number and finds integer square root. Input 12 output 3. Input 25 output 5. Write following programs by modifying it.

30. Write program which reads two numbers x and z. The program finds smallest y such that sum of numbers x and y exceeds z. Input x=5 z=30 output y=9 because 5+6+7+8+9>30 and 5+6+7+8 ≤ 30. Input x=10 z=33 output y=13 because 10+11+12+13>33 and 10+11+12 ≤ 33.
31. Write program, which reads 2 numbers x and z. The program outputs smallest y such that z< x^y. Input x=3 z=72 output 4. Input x=2 z=64 output 7.
32. Write program, which reads z. The program outputs smallest x such that x!>z. Input 100 output 5. input 22 output 4.

Function

```
#include<stdio.h>
int sum(int x)
{  int s=0,d;
   while (x>0)
   {  d=x%10;
      s=s+d;
      x=x/10;
   }
   return s;
}
void main()
{ int a,b;
  printf("Give number");
  scanf("%d",&a);
  b=sum(a);
  printf("%d",b);
}
```

The above program finds sum of digits of a number. Input 23145 output 15.

Similarly write functions for following.

1. First digit fd(23145)=2 fd(314)=3
2. Second digit sd(23145)=3 sd(314)=1
3. Sum of first two digits sft(27145)=2+7=9
4. Last even digit led(23145)=4 led(683)=8
5. Sum of even digits sed(23145)=2+4=6
6. Last digit multiple of 3 ld(619127)=9
7. Biggest digit bg(247156)=7 bg(253)=5
8. Number of digits nd(247159)=6 nd(251)=3
9. Location of biggest digit loc(247156)=4 loc(215)=1
10. Exchange last two digits ex(21715)=21751
11. Increment even digits inc(23768)=33779
12. Smallest factor sf(175)=5 sf(24)=2
13. Sum of factors sof(35)=5+7+35=47
14. Second smallest factor ssf(130)=5 ssf(24)=3
15. Factorial fact(5)=120 fact(7)=5040
16. Integer square root isr(27)=5 isr(49)=7

```
#include<stdio.h>
int pwr(int x,int n)
{  int p=1,i;
   for (i=1;i<=n;i++)
      p=p*x;
   return p;
}
void main()
{ int a,b,c;
  printf("Give 2 numbers");
  scanf("%d %d",&a,&b);
  c=pwr(a,b);
  printf("  %d",c);
}
```

The above program two numbers. It finds power. Input 3 5 output 3^5=243. Input 7 3 output 7^3=343.

Similarly write functions for following.

17. Sum of smallest factors ssf(24,35)=2+5=7
18. k^{th} last digit kld(2314978,3)=9
19. Sum of numbers sm(5,9)=5+6+7+8+9=35
20. Smallest common factor scf(84,105)=7
21. Smallest common multiplier lcm(24,36)=72
22. Sum of common factors smcf(84,105)=3+7+21=31
23. Greatest common factor gcd(280,350)=70
24. k^{th} smallest factor ksf(24,5)=8 ksf(35,2)=7
25. Sum of digits of a number (x) which are more than y mr(273563,4)=6+5+7=18 mr(15732,3)=7+5
26. Sum of first k factors sm(24,5)=2+3+4+6+8=23
27. Delete k^{th} last digit dl(4231576,3)=423176
28. Replace k^{th} last digit by a given digit rep(231576,3,8)=231876 rep(375468,1,9)=375469
29. Sum of digits between given range sm(23158,3,9)= 8+5+3=16. sm(516984,5,8)=8+6+5=19

Input - Output Format

```
int a=23796;
printf("a=%8d",a);
```

%8d	a= 23796
%9d	a= 23796
%5d	a=23796
%3d	a=23796
%d	a=23796

```
float a=23.796;
printf("a=%9.2f",a);
```

%9.2f	a= 23.80
%7.3f	a= 23.796
%.2f	a=23.80
%f	a=23.796000
%11f	a= 23.796000
%4.3f	a=23.796

```
int a=10,b=20;
printf("%d %d\n",a);
printf("%d\n",a,b);
```

The first line outputs one number (the second number is zero or random). The second line outputs only one number ‘a’.

```
int a,b;
scanf("%3d%2d",&a,&b);
printf("a=%d b=%d",a,b);
```

When input is 427943 then output is a=427 and b=94.

```
float a,b,c,d;
scanf("%f%fi",&a,&b);
c=a*2;d=b*2;
if (d>0)        printf("%f+%fi",c,d);
else if (d<0)   printf("%f%fi",c,d);
else            printf("%f",c);
```

The above program reads a complex number and outputs its double.
Input 2+3i output 4+6i

```
float a,b,c,d;
scanf("%fi%fj%fk",&a,&b,&c);
d=sqrt(a*a+b*b+c*c);
printf("%f",d);
```

The above program reads a vector and prints its magnitude.
Input 3i+4j–12k output 13

```
int i,x,n,s=0;
scanf("%d",&n);
printf("Give numbers separated by ,");
for (i=1;i<=n-1;i++)
{  scanf("%d,",&x);
   s=s+x;
}
scanf("%d",&x);
s=s+x;
printf("%d",s);
```

The above program reads n and n numbers (which are separated by comma). The program output their sum. [Question: why last number is read outside the loop]

1. Write program, which reads 4 (integer) numbers a, b, c and d. The program outputs a and b in first line and c and d in second line. c is outputted below a and d is outputted below b. Assume that any of these numbers may have at most 10 digits. [Caution: cases (A) a=b=7 c=d=2134523 (B) a=b=12653376 c=d=1 and (C) a=d=7 and b=c=5612345 should be handled by same program]
2. Write program, which reads 6 (integer) numbers a, b, c, d, e and f. The program outputs a and b in first line and c, d, e and f in second line. c and d are outputted below a and e and f are outputted below b. [Caution: cases (A) a=b=7 c=d=e=f=2134523 and (B) a=b=12653376 c=d=e=f=1 (C) a=e=f=7 and b=c=d=5612345]
3. Write program, which reads 4 (float) numbers a, b, c and d. The program outputs a and b in first line and c and d in second line. c is outputted below a and d is outputted below b. a and b have 3 decimal places. c has 5 decimal places. d has 1 decimal place.

4. Write program, which reads coordinates of two points and outputs their mid point. If input is (7,2) and (–3,1) then output is (2,1.5). [Hint: scanf("(%f,%f)",&a,&b)]
5. Write program, which reads two vectors and outputs their dot product. If input is 2i+3j–7k and 1i–2j–3k then output is 17.
6. Write program, which reads a vector and outputs a unit vector in same direction.
7. Write program to output the cross product of vectors.
8. Write program, which reads two complex numbers and prints their sum.
9. Write program to print multiplication and division of complex numbers.
10. Write program, which reads equation of a line and outputs its points of intersection with x-axis. If input is 2x+3y–7=0 then output is (3.5,0).
11. Write program, which reads quadratic equation and print its roots. e.g. if input is 5x^2+6x+5=0 then output is –0.6+0.8i and –0.6–0.8i
12. Write program, which reads a complex number and prints it in nice looking form. If input is 2+7i then output is 2+7i. If input is 2–7i then output 2–7i. If input is 0+7i then output is 7i. If input is 2+0i then output is 2. If input is 5+1i then output is 5+i. If input is 0+0i then output is 0.
13. Write program, which reads a vector and prints it in nice looking form. If input is 5i+0j–3k then output is 5i–3k (not as 5i+–3k). If input is 5i+1j–3k then output is 5i+j–3k.
14. Write program, which reads n and n numbers. The program finds their sum. There is comma(,) after n and between all numbers. Input 5,2,3,1,7,9, output 2+3+1+7+9=22.
15. Modify above program so that last (,) is not needed.
16. Modify above program when (;) is used after n. Input 5;2,3,1,7,9.
17. Modify above program when n (;) are used after n. Input 5;;;;2,3,1,7,9.
18. Write program, which reads n and a vector in n dimensional space. The program outputs the magnitude. If n=5 and vector is (3,1,8,5,1) then output is $\sqrt{3^2+1^2+8^2+5^2+1^2}=10$
19. Write program, which reads n and n pair of numbers. The program finds product of every pair. Then their sum is calculated. If n=4 and pairs are (3,2) (6,3) (2,6) (4,3) then output is 3x2+6x3+2x6+4x3=48.
20. Write program, which reads n and m. The program reads n group of numbers. Every group has m numbers. The program finds the sum of all numbers. If n=4 and m=3 and groups are (3,2,1) (6,3,2) (2,6,5) (4,3,4) then output is 3+2+1+6+3+2+2+6+5 +4+3+4=41.
21. Modify above program so that product of every group is calculated. Then their sum is done. For above input the output is 3x2x1+6x3x2+2x6x5+4x3x4=150.
22. Write program, which reads a number and finds its first digit. Do not use any loop. [Hint: scanf("%1d")]
23. Write program, which reads a number and finds its second digit. (Do not use a%10)
24. Write program, which reads a number and finds its 4th digit. Input 152743 output 7.
25. Write program, which reads two numbers. The program finds the product of their first digits.
26. Write program, which reads two numbers (k and x). The program finds the kth digit of x. [Use only one loop]
27. Write program, which reads a number and finds its first even digit. [one loop]
28. Write program, which reads n and n complex numbers and outputs their sum.

29. Write program, which reads n and 2n numbers. If n=4 and numbers are 3,2,6,3,2,6,4,3 then output is $3^2 + 6^3 + 2^6 + 4^3$ =9+216+64+64=353.
30. Read a float number and print it by adding 17. Do not use %f. e.g. input 4788.32 output 4805.32].

File

```
#include<stdio.h>
main( )
{  int i; FILE *fp;
   fp=fopen("abc","w");
   for (i=65;i<=90;i++)
      fprintf(fp,"%c",i);
}
```

The above program prints ABCDEF..Z in a file whose name is “abc”.

```
#include<stdio.h>
main( )
{  int i,s=0,x;FILE *fp;
    fp=fopen("pqr","r");
    for (i=1;i<=10;i++)
   {  fscanf(fp,"%d",&x);
      s=s+x;
   }
   printf("%d",s);
}
```

Make a file whose name is “pqr”. In that file type at least 10 numbers. When above program is executed then sum of first 10 numbers is outputted.

```
#include<stdio.h>
#include<string.h>
      main( )
      {  int x;FILE *fp1,*fp2;char t[10];
         printf("Give filename"); gets(t);
         fp1=fopen("ttt","r");
         fp2=fopen(t,"w");
         fscanf(fp1,"%d",&x);
         fprintf(fp2,"%d",2*x);
      }
```

Make a file whose name is “ttt”. Type few numbers in that file. During the execution of above program a file name is asked. The double of first number of file “ttt” is written in that file.

```
      main( )
      {  FILE *fp;char t[20];
         fp=fopen("abc","r");
         fgets(t,4,fp); puts(t);
         fgets(t,10,fp); puts(t);
         fgets(t,9,fp); puts(t);
      }
```

In a file named “abc” type “1gh45tklc” in the first line and “xyzw” in the second line. Observe the output of the program.

1. Write a program, which prints 1 2 3 4 .. 100 in a file named “ttt”.
2. Write a program, which prints ABCDEABCDABCABA in a file named “pqr”
3. Write program, which prints “ABC..Z” in file “ttt” and “123..9” in file “abc”.
4. Write a program, which reads a number from the terminal and prints its double in file “t”.
5. Write a program, which reads 10 numbers from terminal and prints their sum in file “t” and product in file “q”.
6. Write a program, which reads 10 numbers from terminal and print all numbers in file “t”.
7. Write a program to print complex numbers 2+3i, 3+4i, … 10+11i in a file named “t”.
8. Write program, which reads 10 numbers from the terminal. The program prints the even numbers in file “t” and odd numbers in “q”.

In a file named “pqr” type few numbers.

9. Write program, which outputs the sum of first two numbers (of file “pqr”).
10. Write program, which prints first 10 number of the file “pqr” on terminal.
11. Write program, which prints (in file “abc”) double of the first number (of file “pqr”).
12. Write program, which prints (in file “abc”) double of 10 numbers (of file “pqr”).
13. Write program, which reads a file name. The program writes in that file “gopal”.
14. Write program, which reads a file name and in that file writes “ABC…Z”.
15. Write program, which reads a file name. The program prints the sum of 10 numbers of file “pqr” in that file.

16. Write program, which reads two file names (from the terminal). The program also reads 10 numbers from the terminal. The program prints the even numbers in first file and odd numbers in second file.
17. Read a file name. From that file read 10 numbers and print their sum on terminal. [Before execution of this program, write 10 numbers in that file].
18. Read n from file “abc”. Read n numbers from file “pqr”. Print their sum in file “ttt”.
19. Make two files “abc” and “pqr”. Type 10 numbers in both of them. Write program, which finds the sum of product of corresponding numbers.
20. Make two files “abc” and “pqr”. Type 10 numbers in both of them in ascending order. Write program, which prints all 20 numbers in ascending order.
21. Write program, which prints (on the terminal) all 20 numbers of the files after shuffling. [1st number of abc, 1st number of pqr, 2nd number of abc, 2nd number of pqr, ….]
22. Make 9 files. Their names are f1, f2, …, f9. In file “f1” print “1 2 3 .. 10”. In file “f2” print “2 4 6 … 20”. In file “f9” print “9 18 27 .. 90”.
23. Write program, which reads a number n (assume n is less 10). The program creates n files, whose names are a1, a2, … , an. In the file a1 1,2,3,…,10 is written. In the file a2 2,4,6,..,20 is written. ……. In the file an n, 2n, 3n, ….. , 10n is written.
24. Make 5 files, whose names are a1, a2, …, a5. In every file put few numbers. Write program, which reads n and r. The program finds sum of product of n corresponding numbers of files a1, a2, …, ar. Assume $r \leq 5$ and every file has at least n numbers. Let contents of a1 be 5,2,7 and a2 be 2,4,6 and a3 be 1,2,4 and a4 be 6,2,3 and a5 be 3,1,2. Let n=3 and r=5 then output is 5*2*1*6*3+2*4*2*2*1+7*6*4*3*2=180+32+1008= 1220. When n=2 and r=5 then output is 180+32=212. When n=3 and r=2 then output is 5*2+2*4+7*6=10+8+42=60. Do not use any array variable except FILE *p[6] and char a[3].
25. Write a program, which reads two vectors (from terminal) and prints their sum in file “t” and their dot product in file “q”

Reading in a loop

```
int i,n,x,s=0;
printf("Give n");scanf("%d",&n);
printf("Give %d numbers",n);
for (i=1;i<=n;i++)
{  scanf("%d",&x);  s=s+x;  }
printf("%d",s);
```

This program reads n and n numbers and finds their sum e.g if n is 5 and given numbers are 33, 100, 77, 42 and 12 then output is 33+100+77+42+12=264.

1. Write program, which finds the sum of last digits (in above case 3+0+7+2+2=14).
2. Write program, which finds the sum of second last digits (3+0+7+4+1=15).
3. Write program to find the sum of numbers after deleting last digit. (3+10+7+4+1=25)
4. Write program to find the sum of numbers after deleting second last digit. (3+10+7+2+2=24)
5. Write program, which finds the sum of the product of last two digits (in above case 3x3 + 0x0 + 7x7 + 4x2 + 1x2 = 68). If n=3 and numbers are 137, 41593 and 89 then output is (3x7+9x3+8x9=120).
6. Write program, which finds the sum of numbers after exchanging last two digits. (in above case 33 + 100 + 77 + 24 + 21 = 255).
7. Write program, which finds the last even number (in above case 12). If n=6 and numbers are 14, 17, 16, 19, 21 and 33 then output is 16.
8. Write program, which finds the weighted sum of these numbers. The weight of ith number is i (in above case 1x33 + 2x100 + 3x77 + 4x42 + 5x12 = 692).
9. Write program, which finds the sum of odd numbers (in above case 33+77=110).
10. Write program, which finds how many of these are odd (in above case 2).
11. Write program, which finds sum of all those numbers, whose last digit is multiple of three (in above case 33 + 100 =133).
12. Write program, which finds sum of those numbers, which are between 30 and 90 (in above case 33 + 77 + 42 = 152).
13. Write program, which finds average of odd numbers. (in above case 110/2=55).
14. Write program, to find sum of numbers after putting last two digits in sorted order (smaller digit first). Input n=4 and numbers are 1271, 3189, 42, 317 output 1217+3189+24+317=4747.
15. Write program, which finds the average weighted sum of even numbers. The weight of ith number is i (in above case (2x100 + 4x42 + 5x12)/(2+4+5) = (428/11) = 38.91).
16. Write program, which finds the weighted sum of even numbers. The weight of ith even number is i (in above case 1x100 + 2x42 + 3x12 = 220).
17. Write program, which finds sum of the product of two consecutive numbers (in above case 33x100 + 100x77 + 77x42 + 42x12=14738).
18. Write program, which finds the sum of all numbers from first even number onwards. If n=14 and given numbers are 3, 7, 4, 3, 2, 12, 11, 9, 1, 6, 8, 7, 10 and 17 then the answer is 4+3+2+12+11+9+1+6+8+7+10+17 = 90).
19. Write program, which finds the sum of all numbers after first even number (in above case 3+2+12+11+9+1+6+8+7+10+17 = 86).
20. Write program, which reads an integer k also and finds sum of all numbers after k even numbers. In above case if k=4 then the answer is 8+7+10+17 =42).

21. Write program, which finds sum of all numbers after two consecutive even numbers (in above case 11+9+1+6+8+7+10+17 =69).
22. Write program, which finds the sum of 1st, 2nd, 4th, 7th, 11th, … numbers (in above case 3+7+3+11+8=32).

```
int x,s=0;
printf("Give numbers");
do{
  scanf("%d",&x);
  s=s+x;
}while (x!= –1);
printf("%d",s);
```

This program reads reads numbers till –1 is given. The program finds the sum of the give numbers. If given numbers are 33, 100, 77, 42, 12, then output is 33+100+77+42+12+(–1)=263. In second program –1 is not added.

```
int x,s=0;
scanf("%d",&x);
while (x!= –1)
{  s=s+x;
   scanf("%d",&x);
}
printf("%d",s);
```

23. Write program, which reads numbers till –1 is given. The program finds how many of the given numbers are more than 30. In above case 4.
24. Write program, which reads numbers till –1 is given. The program finds the weighted sum of these numbers. The weight of ith number is 'i'. For above input 1x33 + 2x100 + 3x77 + 4x42 + 5x12 + 6(–1) = 686.
25. Modify above program, when weight of ith last number is taken as 'i'. For above input 6x33 + 5x100 + 4x77 + 3x42 + 2x12 + 1x(–1) = 1155.
26. Write program, which finds the average weighted sum of even numbers. The weight of ith number is i (in above case (2x100 + 4x42 + 5x12)/(2+4+5) = (428/11) = 38.91).
27. Write program, which finds the weighted sum of even numbers. The weight of ith even number is i (in above case 1x100 + 2x42 + 3x12 = 220).
28. Write program, which reads numbers (every number is single digit) till –1 is given. Assume that those digits are digits of a number (given in opposite order). The program outputs the number and its double. e.g. if input is 4, 2, 3, 6, 1, 3, –1 then number is 316324 and its double is 632648.
29. Do above problem, when digits are given in correct order. For above input the output is 423613 and 847226.
30. Write program, which reads 'k' and numbers till 'k' –1's are given. The program finds the sum of those numbers. e.g. if k=3 and numbers are 6, 4, 5, 2, –1, 3, –1, 6, 3, –1 then output is 26.
31. Write program, which reads numbers till two consecutive –1's are given. The program finds the sum of those numbers. e.g. if numbers are 5, 2, –1, 3, –1, 6, 5, 5, 3, –1, –1 then output is 25.
32.

Maximization

```
int i,n,x,max;
printf("Give n");
scanf("%d",&n);
max=0;
for (i=1;i<=n;i++)
{  scanf("%d",&x);
   if (x>max)
      max=x;
}
printf("%d",max);
```

```
int i,n,x,max;
printf("Give n");
scanf("%d",&n);
scanf("%d",&max);
for (i=2;i<=n;i++)
{  scanf("%d",&x);
   if (x>max)
      max=x;
}
printf("%d",max);
```

These programs read n and n numbers and find their maximum e.g if n=5 and given numbers are 32, 100, 77, 83 and 12 then output is 100

1. Write program, which finds the maximum odd number (in above case 83).
2. Write program, which finds the maximum number less then 60 (in above case 32).
3. Write program, which finds location of maximum number (in above case 2, since 2nd number is maximum.)
4. Write program, which finds the number with maximum last digit (in above case 77).
5. Write program, which finds the number immediately after the maximum number (in above case it is 77. Assume that the last number is not the maximum).
6. Write program, which finds the maximum and the second maximum (in above case 100 and 83 respectively).
7. Write program, which finds the minimum number. In above case 12.
8. Write program, which finds the maximum and minimum numbers both.
9. Write program, which finds how many numbers are maximum. e.g. if n=11 and given numbers are 7, 6, 3, 7, 7, 1, 8, 7, 8, 1, and 4 then answer is 2 because the maximum number 8 occurs 2 times.
10. Write program, which finds location of maximum number. If maximum element occurs more then once then the last location is outputted. In above case output is 9 (since the maximum element 8 occurs at locations 7th and 9th, but only the last location is outputted).
11. Modify above program to output the first and the last locations of the maximum number.
12. Write program to find the sum of all numbers except the maximum number. Assume that the maximum number is present only once. Do not use subtraction.
13. Do above problem, when the maximum number is present more than one.
14. Write program, which reads n, k and n numbers. The program finds k^{th} number after maximum number. If n=12 k=3 and numbers are 2, 5, 1, 12, 6, 2, 16, 8, 10, 13, 6 and 10 then answer is 13.
15. Write program, which finds the sum of k numbers after the maximum numbers. For above case 8+10+13=31.
16. Write program, which finds the number with maximum last digit. However when two number with same last digit come then the smaller number is taken. If n=6 and numbers are 32, 100, 77, 74, 67 and 34 then output is 67.
17. Write program, which finds the number with maximum sum of digits. e.g. if numbers are 325, 199, 800 then answer is 199.

18. Write program, which reads a number and deletes the maximum digit. Assume that all digits in the number are distinct. e.g. input 237436 output 23436.
19. Do above problem when the maximum digit can be present many times. e.g. input 137828381 output 137231.

History

```
int i,n,h,x=0,s=0;
printf("Give n");
scanf("%d",&n);
for (i=1;i<=n;i++)
{  h=x;
   scanf("%d",&x);
   s=s+x*h;
}
printf("Sum=%d",s);
```

Program to find sum of product of consecutive numbers.

```
int i,n,h,x=0,ans;
printf("Give n");
scanf("%d",&n);
for (i=1;i<=n;i++)
{  h=x;
   scanf("%d",&x);
   if (x%2==0) ans=h;
}
printf("%d",ans);
```

Program to find the number immediately before the last even number

1. Write program, which reads n and n numbers and finds the sum of all numbers before a even number e.g if n=7 and given numbers are 17, 11, 24, 13, 7, 8, 14 then answer is 26 (since 11+7+8=26). Assume that the first number is not even.
2. Write program, which finds the sum of product of three consecutive numbers e.g. if numbers are 6, 2, 4, 3, 8, 2 then answer is 216 (since 6.2.4+2.4.3+4.3.8+3.8.2=216).
3. Write program, which finds the number immediately after the last even number. e.g. if n=15 and numbers are 11, 14, 31, 27, 21, 26, 47, 18, 32, 37, 41, 38, 17, 45, 15 then output is 17. [Hint: If history is even then record the number]
4. Write program, which finds the number, which is two numbers before the last even number. In above case 37. [Hint: Record two histories]
5. Modify above program to find two numbers after the last even number. In above case 45.
6. Write program, which finds the second last even number. In above case 32. [Hint: history of even numbers]
7. Write program, which finds the number immediately before the second last even number. In above case 18. [Hint: history of numbers before a even number and all numbers]
8. Write program, which finds the number immediately after the second last even number. In above case 37.
9. Write program, which finds the number which is multiple of 3 and is immediately before the last even number. In above case 18.
10. Write program, which finds the number which is multiple of 3 and is immediately before the second last even number. In above case 18.
11. Write program, which finds the number which is multiple of 3 and is before the last even number and there is one more number (which is multiple of 3 between them). In above case 21.
12. Write program, which finds the number which is multiple of 3 and is immediately after the last even number. In above case 45.
13. Write program, which finds the number, which is immediately before the number, which is multiple of 3 and is immediately before the last even number. In above case 47.
14. Write program, which finds the number immediately before the maximum number. For above case 26.

15. Write program, which finds the odd number before the maximum number. For above case 21.
16. Write program, which finds the maximum number before the maximum number. For above case 31.
17. Write program, which finds sum of all numbers before the last even number. e.g. if numbers are 3,4,5,7,8,2,4,9,6,8,7,1 then output 3+4+5+7+8+2+4+9+6=48.
18. Write program, which finds sum of all numbers before the second last even number. In above case 3+4+5+7+8+2+4+9=42.
19. Write program, which finds sum of all numbers after the last even number.
20. Write program, which finds sum of all numbers after the second last even number.
21. Write program, which finds sum of product of consecutive even numbers. In above case 4x8+8x2+2x4+4x6+6x8=128.
22. Write program, which finds sum of product of consecutive even numbers if they occur consecutively. In above case 8x2+2x4+6x8=72.
23. Write program, which reads a number and finds the second even digit. e.g. input 134928651 output 2.
24. Write program, which reads a number and finds the sum of product of consecutive even digits. e.g. input 134928651 output 6x8+8x2+2x4=72.
25. Write program, which reads numbers till a negative number is given. The program finds the sum of all positive numbers.
26. Write program, which reads numbers till two consecutive negative numbers are given. The program finds the sum of all numbers. e.g. input 7,2,4,-2,6,-3,4,5,-6,-7 output 10.
27. Write program, which finds the nth term of following series: 1,1,2,3,5,8,13,21,34,…. . e.g. if n=8 then answer is 21. (assume n>2)
28. Write program, which finds the nth term of following series: 1,1,1,3,5,9,17,31,57,…. (assume n>3).

Serial Loop

1. Write program, which reads n(integer) and n numbers and finds the sum of first n/2 numbers and sum of doubles of remaining n/2 numbers. If n=8 and numbers are 3, 4, 1, 9, 6, 2, 8, and 5 then the output is 3+4+1+9+ 2x(6+2+8+5)=17 + 2x21 = 59. If n=13 then sum of first 6 numbers and sum of doubles of remaining 7 numbers will be calculated.

2. Write program, which reads n and n number and finds the smallest digit of biggest number. If n=5 and given numbers are 33, 427, 36, 2134 and 56 then output is 1. Since the biggest number is 2134 and the smallest digit is 1.

3. Write program, which finds the sum of digits in the sum of given numbers (in above case the sum is 33+427+36+2134+56=2686 hence output is 2+6+8+6=22).

4. Write program, which reads a number and finds the sum of digits in its factorial. If given number is 6 then the answer is 9 because 6!=720.

5. Write program, which reads a number and finds the special sum of digits. Here first digit is multiplied by 1, second digit by 2, third digit by 3 and so on. Then their sum is calculated. If number is 2686 then output is 1x2+2x6+3x8+4x6=62.

6. Write program, which reads a number and finds how many times the first digit occurs. If the number is 34533253 then the answer is 4 since the first digit (3) occurs 4 times. [Hint: Declare long int x; and Read number using scanf("%ld",&x);]

7. Write program, which reads a number and finds the sum of those digits of the number, which are bigger than the first digit. If the number is 3453215 then the output is 4+5+5=14.

8. Write program, which reads a number and finds the last odd digit before last even digit. If number is 341042793 then answer is 1 since last even digit is 2.

9. Write program, which reads two number x and k. The program finds the kth digit of the number x. If x=2341562 and k=3 then output is 4.

10. Write program, which reads two number x and k. and finds the kth digit before last even digit. If x=234156273 and k=3 then output is 1. Here last even digit is 2.

11. Write program, which reads two number x and k. and finds the kth digit after last even digit. If x=23627393 and k=3 then output is 9.

12. Write program, which finds the number of digits before the first even digit. If number is 1372465 then answer is 3 since the first even digit is 2.

Serial Loop and Function

```
#include<stdio.h>
main()
{ int a,b,p,i;
  printf("Give number");
  scanf("%d",&a);
  b=a;
  while (a>9) a=a/10;
  p=1;
  for (i=1;i<=b%10;i++)
     p=p*a;
  printf("%d",p);
}
```

This program reads a number. It finds (first digit)$^{(\text{last digit})}$. Input 4213 output 4^3=64. [Here b stores the copy of given number a. It is done because the given number is destroyed in the computation of first digit.]

```
#include<stdio.h>
int pwr(int x,int n)
{  int p=1,i;
   for (i=1;i<=n;i++)
      p=p*x;
   return p;
}
int fd(int x)
{  while (x>9)
     x=x/10;
   return x;
}
main()
{ int a,p,q;
  printf("Give number");
  scanf("%d",&a);
  p=fd(a);
  q=pwr(p,a%10);
  printf("%d",q);
}
```

```
#include<stdio.h>
void main()
{ int a,b,p,q,i,j;
  printf("Give two number");
  scanf("%d %d",&a,&b);
  i=2;while (a%i!=0) i++;
  j=2;while (b%j!=0) j++;
  p=i+j;
  q=2;while (p%q!=0) q++;
  printf("ans  %d\n",q);
}
```

This program reads two numbers and finds smallest factor of sum of their smallest factors. Input 24 and 121. The sum of smallest factors is 2+11=13.

```
#include<stdio.h>
int sf(int x)
{  int i=2;
   while (x%i!=0)
      i++;
   return i;
}
void main()
{ int a,b,p,q,i,j;
  printf("Give two number");
  scanf("%d %d",&a,&b);
  i=sf(a);j=sf(b);
  p=i+j;
  q=sf(p);
  printf("ans %d\n",q);
}
```

Solve following problems by both method. In the bracket the name of the function is written. In a function only one loop should be

1. Write program, which reads a numbers. The program outputs the first digit of the number obtained by multiplying the number by its first digit. Input 3714 output 1. (because 3*3714=11142). [First digit]
2. Write program, which reads a number. The program finds the sum of numbers between first and last digit. Assume first digit<last digit. Input 27135 output 14. (because 2+3+4+5=14). Input 492376 output 4+5+6=15. [first digit, sum] (sum(3,7)=25)
3. Write program, which reads a number. The program outputs sum of maximum digits in the given number and in its double. input 51716 output 7+4=11. (because double is 103432). Input 136 output 6+7=13 (because 272). [Maximum digit]

4. Write program, which reads a number. The program finds the sum of digits in sum of digits. Input 27956 output 11 (because sum of digits is 2+7+9+5+6=29) [sum of digits]
5. Write program, which reads a number. The program, finds sum of those digits of the number, which are less than first digit] input 42381247 output 2+1+3+2=8. [first digit, sum(x,y) it finds sum of those digits of number x which are less than y sum(23764,6)=9]
6. Write program, which reads a number. The program finds factorial of first digit. Input 571287 output 5!=120. [factorial, first digit]
7. Read a number, if it is prime then output its first digit. Otherwise output second digit. Input 23 output 2. Input 2415 output 4. [smallest factor, first digit, second digit]
8. Write program, which reads a number. The program finds factorial of first digit in factorial of the number. Input 6 output 5040 (because 6!=720 and 7!=5040) [first digit, factorial]
9. Read a number. Multiply it by last digit. Multiply that by second last digit. Multiply that by third last digit. It is done as far as possible. [k^{th} last digit, number of digits]
   Input 23 output 1656 (because 23*3=69, 69*6=414, 414*4=1656, 1656*1=1656)
   Input 342 output 43776 (because 342*2=684, 684*8=5472, 5472*4=21888,
   21888*1=21888, 21888*2=43776)
10. Write program, which reads a number. The program finds sum of prime digits of the number. [sf][Hint: if(sf(d)==d) s=s+d] Assume that the number does not has any digit, which is 0 or 1. Input 27645 output 5+7+2=14] To check whether given number is prime or not, its smallest factor is calculated. If it is equal to the given number then it is concluded that given number is prime.

Nested loop and Function

```
#include<stdio.h>
main( )
{ int x,k,s=0,i,j,t;
  scanf("%d %d",&x,&k);
  for (i=2;i<=x;i++)
    if (x%i==0)
    {     t=1;
          for(j=1;j<=k;j++)
            t=t*i;
          s=s+t;
    }
    printf("%d",s);
}
```

These programs reads two numbers x and k. The program finds the sum of kth power of every factor.

```
#include<stdio.h>
int power(int x,int p)
{  int t=1,i;
   for (i=1;i<=p;i++)
      t=t*x;
   return(t);
}
main( )
{
  int x,k,s=0,i;
  scanf("%d %d",&x,&k);
  for (i=2;i<=x;i++)
    if (x%i==0)
          s=s+power(i,k);
  printf("%d",s);
}
```

```
#include<stdio.h>
main( )
{ int n,i,prod,sum,digit,x;
  scanf("%d",&n);
  sum=0;
  for (i=1;i<=n;i++)
  {  scanf("%d",&x);
     prod=1;
     while (x>0)
     {  digit=x%10;
        x=x/10;
        prod=prod*digit;
     }
     sum=sum+prod;
  }
  printf("Answer=%d",sum);
}
```

The above program reads n and n numbers and finds their sum of product of digits. e.g. if n=5 and given numbers are 33, 100, 77, 42 and 12 then output is 3x3 + 1x0x0 + 7x7 + 4x2 + 1x2 = 68. In this program nested loop is used.

```
#include<stdio.h>
int pduct(int x)
{  int prod,digit;
   prod=1;
   while (x>0)
   {  digit=x%10;
      x=x/10;
      prod=prod*digit;
   }
   return(prod);
}
main( )
{ int n,i,sum,x;
  scanf("%d",&n);
  sum=0;
  for (i=1;i<=n;i++)
  {  scanf("%d",&x);
     sum=sum+pduct(x);
  }
  printf("Answer=%d",sum);
}
```

The above program performs same using function.

Write program for followings by both methods (nested loop and function). In bracket the name of the function is written.

1. Write program, which find the sum of first digits e.g. if n=5 and given numbers are 33, 100, 77, 42 and 12 then output is 3+1+7+4+1=16. [First digit]
2. Write program, which finds sum of those numbers whose first digit is 1. (in above case 100+12=112) [First digit]
3. Write program, which finds the sum of smallest factors (in above case 3+2+7+2+2=16). [Smallest factor]
4. Write program, which finds sum of those numbers whose smallest factor is 2. (in above case 100+42+12=154) [Smallest factor]
5. Write program, which finds the sum of the biggest digit (in above case 3+1+7+4+2=17). [Maximum digit]
6. Write program, which finds special sum of these numbers. It is sum of numbers after multiplying each number by the number of digits (in above case it is 2x33 + 3x100 + 2x77 + 2x42 + 2x12 = 628). [Number of digits]
7. Write program, which finds sum of numbers after deleting first digit. (in above case 3+00+7+2+2=14). [delete first digit]
8. Write program, which finds the sum of numbers after reversing. (in above case 33+001+77+24+21=156). [reverse]
9. Write program, which finds sum of numbers after incrementing every even digit. (in above case 33+111+77+53+13=287). [increment even digit]
10. Write program, which find the first digit of product of first digits. (in above case 8, because 3x1x7x4x1=84 its first digit is 8). [First digit]
11. Write program, which finds the sum of product of smallest factor and first digit. In above case 3x3+2x1+7x7+2x4+2x1=70. [Smallest factor and first digit].
12. Write program, which reads an integer k also and finds sum of kth factor of every number. e.g. if k=3 then answer is 33 + 5 + 77 + 6 + 4 = 125. [k^{th} factor]
13. Write program, which finds the biggest smallest factor. [Hint: Find smallest factor of each number. Find biggest of these]. The program also prints the number, which has that (in above case smallest factors are 3, 2, 7, 2 and 2 respectively. Biggest smallest factor is 7, the number 77 has it). [Smallest factor]
14. Write program, which finds the sum of integer square roots (in above case 5 + 10 + 8 + 6 + 3 = 32). [Integer square root (use while loop till i*i<x)]
15. Write program, which reads n and n numbers and finds how many of them are prime. e.g. if n=6 and given numbers are 7, 13, 16, 27, 11 and 9 then answer is 3 since the prime numbers are 7, 13 and 11. (Hint: Smallest factor of a prime number is equal to itself). [Smallest factor]
16. Write program, which finds the sum of prime numbers. (In above case 7+13+11 = 31).
17. Write program, which finds the biggest prime (in above case 13).
18. Write program, which finds the last prime (in above case 11).
19. Write program, which reads a number and finds $(\text{sum of digits})^{\text{smallest factor}}$. If number is 455 then output is 14^5. [Sum of digits, smallest factor, power]
20. Write program, which reads a number and finds first digit of its factorial. Input 6 output 7 (since 6!=720). [First digit and factorial]

21. Write a program, which reads a number and finds all of its prime factors. [Hint: Find all factors, however print only those which are prime]. If given number is 220, then the output is 2, 5 and 11. [smallest factor]
22. Modify above program so that if a prime factor is present more than once then it is printed multiple times. Input 220 output 2, 2, 5, 11. Input 162 output 2, 3, 3, 3, 3.
23. Read n and 2n numbers. Find sum of greatest common divisors of pair of numbers. If n=4 and numbers are 24, 54, 11, 16, 33, 39, 77, 99 then output is gcd(24,54) + gcd(11,16) + gcd(33,39) + gcd(77,99)=6+1+3+11=21. [gcd]
24. Write program, which reads a number and finds the sum of factorials of the digits of the number. If the number is 2524 then output is 2! + 5! +2! +4! = 2+120+2+24=148. [Factorial]
25. Write program, which reads a number. The program finds (smallest factor)$^{\text{first digit}}$. e.g. input 217 output 7^2=49. [Smallest factor, first digit, power]
26. Write program, which reads a number. The program finds the answer in the following manner. Input 21735 output $2^1 + 1^7 + 7^3 + 3^5$ = 2+1+343+243 = 589. [power]
27. Write program, which reads a number the finds special product. If number is 2314 then output is 2 + 2x3 + 2x3x1 + 2x3x1x4 = 38. If input number is 223 then output is 2 + 2x2 + 2x2x3 = 18. [Product of digits]
28. Write program, which reads a number and finds the sum of the product of digits of every factor. e.g. if given number is 686 then its factors are 2, 7, 14, 49, 98, 343, and 686. Hence the answer is 2 + 7 + 1x4 + 4x9 + 9x8 + 3x4x3 + 6x8x6 = 2 + 7 + 4 + 36 + 72 + 36 + 288 = 445. [product of digits]
29. Write program, which reads three integers a, b and c and finds $\sum_{i=a}^{b}\prod_{j=i}^{i+c-1} j$. If a=2 b=5 and c=3 then answer is 2x3x4 + 3x4x5 + 4x5x6 + 5x6x7 = 24+60+120+210 = 414. [Define function product(a,b): product of numbers between p and q. e.g. product(5,8)=5x6x7x8=1680. Write main program to find the sum of numbers between a and b]
30. Write program, which reads integer k and n and n numbers and finds sum of kth power of every number. If n=5 and k=4 and numbers are 6, 2, 10, 1 and 5 then answer is $6^4 + 2^4 + 10^4 + 1^4 + 5^4$ = 1296 + 16 + 10000 + 1+ 625 = 11938. [power]
31. Write program, which reads numbers x and k and finds k^{th} (integer) root of x. If x=100 and k=4 then answer is 3 because $3^4 \le 100 < 4^4$. If x=219 and k=3 then answer is 6 because $6^3 \le 219 < 7^3$. [power]
32. Write program, which reads two numbers and finds the sum of those digits of the first number, which are also present in the second number. If given numbers are 22676 and 1772 respectively then the output is 7+2+2=11. [Whether a digit is present in a number]
33. Write program, which reads two numbers and finds the sum of product of digits of the number. If numbers are 224 and 37 then answer is 2x3+2x7+2x3+2x7+4x3+4x7=80.
34. Write program, which reads two numbers and computes output in the following manner. If numbers are 273 and 3212 then output is $2^3+2^2+2^1+2^2+7^3+7^2+7^1+7^2+3^3+3^2+3^1+3^2$=514. [power, nested loop can not be removed].

35. Write program, which reads n and n numbers. The program finds the product of sum of factorials of first digit. If n=3 and given numbers are 231, 41, and 322 then answer is 2! + 4! + 3!=2+24+6=32. [factorial, first digit]
36. Write program, which reads n and n numbers. The program finds product of sum of factorials of digits of every numbers. If n=3 and given numbers are 231, 41, and 322 then answer is (2+6+1)*(24+1)*(6+2+2)=2250. [factorial, sum of factorials of digit]
37. Write program, which reads n and n numbers. The program finds the product of sum of prime factors. If n=3 and given numbers are 324, 189 and 35 then output is (2+3)x(3+7)x(5+7)=600. [smallest factor, product of prime factors]
38. Write program, which reads n and n numbers. The program finds sum of product of pair wise smallest factors. If n=3 and given numbers are 325, 5134 and 12 then output is (2x5)+(3x13x2)+(2)=90. [Hint: product of pair wise smallest factors of 35172 is 5x3x17x2, because the smallest factors of 35, 51, 17 and 72 are 5, 3, 17 and 2 respectively.
39. Write program, which reads n, k and n numbers. The program finds the sum of kth (integer) root of every number. If n=3 and k=4 and given numbers are 100, 700, 18 then output is 3+5+2=10.

Simulation

```
#include<stdio.h>
#include<stdlib.h>
main( )
{  int i,t;
   for (i=1;i<=100;i++)
   {  t=rand( );t=t%50;
      printf("%d\n",t);
   }
}
```

The above program generates 100 random numbers. Every number is between 0 and 49. The rand( ) function genetates a random number. This function is defined in the file named "stdlib.h"

```
int die( )
{  return(rand( )%6+1); }
main( )
{  int i,s,t,n,success=0;float prob;
   printf("no of experiments:");
   scanf("%d",&n);
   for (i=1;i<=n;i++)
   {  s=die( ); t=die( );
      if (s+t==10) success++;
   }
   prob=(float)success/n;
   printf("%f",prob);
}
```

A die is thrown two times. What is the probability that sum of outcomes is 10?

```
for (i=1;i<=n;i++)
{  sum=0;count=0;
   do{ sum=sum+die( );count++;
    } while (sum<=20);
    total=total+count;
    if (count<8) success++;
}
p=(float)success/n;average=(float)total/n;
```

A die is thrown, till the sum of outcomes exceeds 20. We want to know the probability the we shall get success in less than 8 throws. We also want to know the average number of throws.

```
int coin( )
{  int t=rand( )%17;
   if (t<7) return 0; else return 1;
}
main( )
{  int i,s,t,n,success=0;float prob;
   printf("Give n");  scanf("%d",&n);
   for (i=1;i<=n;i++)
   {  s=coin( ); t=coin( );
      if ((s==0)&&(t==0)) success++;
   }
   printf("%f",(float)success/n);
}
```

A biased coin (probability of head 7/17) is tossed two times. What is the probability that both times it is head?

```
main( )
{  int i,n,a,b,c,d; float p,total=0;
   printf("Give n"); scanf("%d",&n);
   for (i=1;i<=n;i++)
   {  a=rand( )%101;b=rand( )%101;
      c=rand( )%101;d=rand( )%101;
      p=sqrt((a-c)*(a-c)+(b-d)*(b-d));
      total=total+p;
   }
   printf("%f",total/n);
}
```

Two points are randomly taken inside a square of side 100. What is the expected distance between them?

```
int i,n,a,b,success=0; float prob;
printf(Give n"); scanf("%d",&n);
for (i=1;i<=n;i++)
{  do{
      a=rand( )%201-100;
      b=rand( )%201-100;
   }while(a*a+b*b>100*100);
   if (b>50) success++;
}
printf("%f",(float)success/n);
```

A random point is taken inside the circle $x^2 + y^2 = 100^2$. What is the probability that the point is above the line y=50?

1. Two dies are rolled. What is the probability that difference is 2? [Answer: 0.22]
2. Two dies are rolled. What is expected value of product? [Answer: 12.353]
3. Five coins are tossed. What is the probability that at least 4 are head? [0.1898]
4. Ten coins are tossed. What is the probability that at least 4 are head? [0.8279]
5. A coin is tossed five times. What is the probability two consecutive heads occur? [0.6]
6. A coin is tossed ten times. What is the probability two consecutive heads occur? [0.86]
7. A biased coin (probability of head=2/5) is tossed two times. What is probability that both are head? [Answer: 0.16]
8. A point is randomly generated inside a square of size 100. What is expected distance of the point from the corner? [Answer: 76.72]
9. Two points are randomly generated inside a square. What is the probability the distance between them is more than the side of the square? [Hint: Assume side of the square is 100. Generate 4 random numbers (a,b,c,d) between 0 and 100. Now coordinates of the points are taken as (a,b) and (c,d)] [Answer: 0.024]
10. Two points are randomly taken on the circumference of a circle. What is the probability that the length of chord joining them is more than the radius? [Hint: Assume the radius of the circle is 100. Generate a random number (t) between 0 and 359. Now coordinate of point on the boundary of the circle are (r*cos(t),r*sin(t)). Similarly generate another point.] [Answer: 0.67]
11. What is the expected chord length is above case? [Answer: 126.24]
12. Two points are randomly generated inside a circle of radius 100. What is the expected length of line joining them? [Answer: 90]
13. A random point is generated inside a rectangle whose end points are (0,0) (0,300) (500,0) and (500,300). It is found that the point is inside a triangle, whose end points are (0,0) (0,200) and (200,0). We want to find probability the it is inside circle, whose radius is 200 and center is (200,200). Write program to find that. [Answer: 0.57]
14. Three points are generated inside a square of side 100. What is expected area of triangle formed?
15. A coin is tossed till 3 heads occur. What is the probability that at least 4 tails occur before that?
16. Two dies are rolled. The sum is found to be more than 8. What is the probability that sum is 10?
17. Two dies are rolled. The sum is found to be more than 8. What is the probability that the product is more than 20?
18. Three coins are tossed. At least 2 are found to be head. What is the probability that the last coin is tail?
19. Two biased coins (probability of head as 2/5 and 3/11 respectively) are tossed. The outcome of both is found to be different. What is the probability that the first coin shows head?
20. A coin is tossed till we get 2 heads. What is the probability that we shall get success in less then to equal to 4 tosses. [Answer: 0.6875]
21. A coin is tossed till we get two consecutive heads. What is expected number of tosses? Write program to find that. [Answer 6.0]
22. Write program, which reads an integer k also and finds the number of tosses required in getting k consecutive heads. [Answer: 30 for k=4, 62 for k=5]
23. A coin is tossed till we get 2 consecutive heads. What is the probability that we shall get success in less than or equal to 5 tosses? [Answer: 19/32]
24. A coin is tossed 100 times. What is expected longest sequence of consecutive heads? [Ans: ≈ 6]
25. A coin is tossed till we get HTHHT. What are expected number of tosses? [Ans 36]

26. A float random number (x) is generated between 0 and 1. What is the probability that $x^2 - 1.2x + 0.27$ is more that 0? (Ans 0.4).
    [Hint: float rd( ) { int x;float y; x=rand( )%1000; y=x/1000.0; return(y); }]
27. Two float numbers are generated between 0 and 2. what is the probability that their sum is less than 1? (Ans 1/8).

    Let P(k) denotes the probability to get two consecutive heads in less than or equal to k tosses.
    P(k) = 0 if k<2
    = 0.25 + 0.5P(k-1) + 0.25P(k-2) otherwise

28. Write program, which reads integers k and p also. A coin is tossed till we get (no of heads – no of tails = k). The program finds the probability that number of tosses are less than or equal to p. (Answer= 0.29 for k=3 and p=7).

    P(k,p) = 0 if k>p
    = 1 if k=0
    = 0.5P(k-1,p-1) + 0.5P(k+1,p-1) otherwise

29. A coin is tossed till a head appears after odd number of tails. What are expected number of tosses. (e.g. TTHHTTTTHTH stop). (Ans:6 when p(H)=0.5) (in general expected number of tosses $\frac{2-p(H)}{[1-p(H)]p(H)}$)
30. A biased coin (Probability of head 2/5) is tossed till two consecutive heads or three consecutive tails come. If two consecutive heads come then Ram wins otherwise Mohan. What is the probability that Ram will win? (Ans 28/55).
    [Hint: float rd( ) { int x,y; x=rand( ); y=x%5; if (y<2) return (0) else return(1); }]

31. A die (possible outcomes 1 or 2 or .. or 6) is thrown till sum is multiple of 4 (4 or 8 or 12 or ..). What are expected number of throws? (Ans).

32. A die is thrown till two consecutive numbers (integers 0..4) ‘a’ and ‘b’ appear such that a+b=4. Find the expected number of throws. (Ans 14).

33. Write program, which reads an integer k also. A coin is tossed till the difference in the number of heads and tails becomes k. The program finds the expected number of tosses (Answer = 3.97 for k=2 8.84 for k=3 17.14 for k=4). Let E(x,y) be the expected number of tosses to get the (no of heads – no of tails =x) or the (no of tails – no of heads =y).
    E(x,y) = 0 if x=0 or y=0
    = 0.5E(x-1,y+1) + 0.5E(x+1,y-1) otherwise

Array

```
main()
{  int x[20],sum,i,n;
   printf("How many numbers");
   scanf("%d",&n);
   for (i=1;i<=n;i++)
   {  printf("Give %dth number",i);
      scanf("%d",&x[i]);
   }
   sum=0;
   for (i=1;i<=n-1;i++)
     sum=sum+x[i]*x[i+1];
   printf("%d",sum);
}
```

The given program reads n and finds the sum of product of consecutive numbers. If n=7 and numbers are 4,5,2,5,6,4,7 then output is 4*5+5*2+2*5+5*6+6*4+4*7=122.

Modify the above program to print the value of following expressions.

1. $(x_1 + x_2)*x_3 + (x_2 + x_3)*x_4 + (x_3 + x_4)*x_5 + \ldots..$ For above input answer is answer is 209.
2. $(x_1 + x_3)*(x_2 + x_4)*(x_3 + x_5)* \ldots..$ answer 56160.
3. $(x_1 - x_2)*(x_2 + x_3) + (x_2 - x_3)*(x_3 + x_4) + \ldots.. + (x_{n-2} - x_{n-1}) *(x_{n-1} + x_n)$ answer –7.
4. $(x_1 * x_n) + (x_2 * x_{n-1}) +(x_3 * x_{n-2}) + \ldots..+(x_n * x_1)$ answer 145.
5. $1*(x_1 + x_2) + 2*(x_2 + x_3) + 3*(x_3 + x_4) + \ldots.. + (n-1)*(x_{n-1} + x_n)$ answer 204.
6. $(x_1 + x_2)^1 + (x_2 + x_3)^2 + (x_3 + x_4)^3 + \ldots + (x_{n-1} + x_n)^{n-1}$ answer 1886603.
7. Do above question without using power function.
8. $(((...((x_1 - x_2) - x_3) ..... - x_{n-2}) - x_{n-1}) - x_n)$ answer –25.
9. $(x_1 - (x_2 - (x_3 - ..... (x_{n-2} - (x_{n-1} - x_n)) ...)))$ answer 5.
10. $(...(((((x_1 - x_2) + x_3) - x_4) + x_5) - x_6) ..... \pm x_n)$ answer is 5.
11. $(x_1 * x_n) + (x_3 * x_{n-1}) +(x_5 * x_{n-2}) + \ldots..+(x_n * x_{n/2+1})$ assume n is odd answer 107.
12. $x_1 + x_1*x_2 + x_1*x_2*x_3 + \ldots.. + x_1*x_2*x_3* \ldots *x_{n-1}*x_n$ answer 39864.

Read 'k' also and find followings

13. $x_1*k + x_2*(k+1) + x_3*(k+2) + \ldots.. + x_n*(n+k-1)$ for above input and k=5 answer is 275.
14. $(x_1 * x_k) + (x_2 * x_{k+1}) + (x_3 * x_{k+2}) + \ldots.. + (x_{n-k+1} * x_n)$ for above input and k=4 answer is 93.
15. $x_1 + x_2 + x_3 + \ldots. + x_{k-1} + x_{k+1} + \ldots. + x_n$ Donot use minus(–). When k=4 answer is 28.
16. $(x_1)^k + (x_1+x_2)^k + (x_1+x_2+x_3)^k + \ldots. + (x_1+x_2+x_3 \ldots. x_n)^k$ When k=5 answer is 57440101.
17. $(x_1*x_2*x_3*...*x_k) + (x_2*x_3*x_4*...*x_{k+1}) + (x_3*x_4*x_5*...*x_{k+2}) +\ldots..+ (x_{n-k+1}*x_{n-k+2}*x_{n-k+3}*...*x_n)$ For above input and k=4 answer is 1580.

18. Write program, which outputs local maxims. A number x_i is local maximum if it is more than both x_{i-1} and x_{i+1}. If array elements are 25, 19, 22, 23, 21, 12, 10, 17, 11, 13, 10 then 23, 17 and 13 are local maxims.
19. Write program that outputs smallest 'i' such that x_i is even. In above case 3. (Because first even number is 22).
20. Write program that outputs smallest 'i' such that x_i and x_{i+1} are both even. In above case 6. (Because 12 and 10 are even).
21. Write program to find weighted sum. The weight of an element is w, if it is more than w elements after it. 25.10+19.6+22.7+23.7+21.6+12.3+10.0+17.3+11.1+13.1+10.0=916.
22. Do above problem when weight of an element is w, if it is more than w elements immediately after it. In above case 25.10+19.0+22.0+23.7+21.6+12.1+10.0+17.3+11.0+13.1+10.0=613.

23. Write program, which reads ‘k’ also. The program outputs a x_i’s if it is more than next k numbers (i.e. x_{i+1}, x_{i+2}, …. , x_{i+k}). If k=3 then for above input the output is 25, 23, 21, 17.

Write program, which reads two arrays $x_1 \ldots x_n$ and $y_1 \ldots y_n$. The program outputs the value of

24. $(x_1 * y_n) + (x_2 * y_{n-1}) + (x_3 * y_{n-2}) + \ldots + (x_n * y_1)$
25. $x_1*(y_1+y_2+y_3+ \ldots+y_n) + x_2*(y_2+y_3+y_4+ \ldots+y_n) + x_3*(y_3+y_4+\ldots+y_n) +\ldots.+ x_{n-1}*(y_{n-1}+y_n) + x_n*y_n$
26. $x_1*(y_1+y_2+\ldots+y_n) + (x_1+x_2)*(y_2+y_3+\ldots+y_n) + (x_1+x_2+x_3)*(y_3+y_4+\ldots+y_n) +\ldots+ (x_1+x_2+\ldots+x_n)*y_n$

Let n=4 and array x is 5,7,2,4 and array y is 8,2,5,3 then outputs are 86, 188 and 376 respectively.

Array using function

```
#include<stdio.h>
void rd(float a[],int n)
{  int i;

   for (i=1;i<=n;i++) scanf("%f",&a[i]);
}
float sum(float a[],int n)
{   int i;float s=0;
    for (i=1;i<=n;i++) s=s+a[i];
    return s;
}
main()
{   int size;float x[20],g;
    printf("Give size");scanf("%d",&size);
    rd(x,size); g=sum(x,size); printf("%f",g);
}
```

The above program reads n numbers, stores them in array and finds sum.

1. Define function float ab(float x[],int n). It returns $x_1*x_2 + x_2*x_3 + x_3*x_4+ \ldots\ldots x_{n-1}*x_n$ If n=7 and $x_1, x_2, \ldots, x_n$ are 4,5,2,5,6,4,7 then output is 4*5+5*2+2*5+5*6+6*4+4*7=122.
2. Define function float ac(float x[], int n, int k). It returns $x_1 k + x_2(k+1) + x_3(k+2) +\ldots+ x_n(n+k-1)$ for above input and k=5 answer is 275.
3. Define function float ad(float x[], int n, int k). It returns $x_1+x_2k+x_3k^2+\ldots+ x_nk^{n-1}$. for above input and k=10 answer is 7465254. For k=3 the answer is 6733.
4. Define function float ae(float x[], int n, int k). It returns $x_n+x_{n-1}k+x_{n-2}k^2+\ldots+ x_1k^{n-1}$. for above input and k=10 answer is 4525647. For k=3 the answer is 4501.

```
#include<stdio.h>
void rd(float a[],int n)
{  int i;
   for (i=1;i<=n;i++) scanf("%f",&a[i]);
}
void wr(float a[],int n)
{  int i;
   for (i=1;i<=n;i++) printf("%5.1f",a[i]);
   printf("\n");
}
void add(float a[],float b[],int n,float c[])
{  int i;
   for (i=1;i<=n;i++) c[i]=a[i]+b[i];
}
main()
{   int size;float x[20],y[20],z[20],g;
    printf("Give size");scanf("%d",&size);
    rd(x,size); rd(y,size);
    add(x,y,size,z); wr(z,size);
}
```

The above program reads two arrays and finds sum of corresponding elements.

5. Define function void increment(float x[], float y[], int n). Here array x and n are input. Output is array y. Here every element is 1 more than that of array x. For above input array y will become 5,6,3,6,7,5,8.
6. Define function void shl(float x[], float y[], int n). It performs shift left. For above input array y will become 5,2,5,6,4,7,0.
7. Write similar function for shift right (shr). For above input array y is 0,4,5,2,5,6,4.

8. Write similar function for rotate left (rol). For above input array y is 5,2,5,6,4,7,4.
9. Define function float rcl(float x[], float y[], int n, int cy). It performs rotate left with carry (cy). For above input and cy=3 the array y is 5,2,5,6,4,7,3. The value returned is 4.
10. Define similar functions for ror, rcr.
11. Define function void da(int y, int n, float x[]). y and n are input. It decomposes given number into digits. If y=42456 and n=7 then array x will become 6,5,4,2,4,0,0.
12. Define db which is similar to above. However array x becomes 0,0,4,2,4,5,6.
13. Define function int dc(int y, float x[]). Here y is input. It decomposes given number into digits and returns the number of digits. If y=42416 then array x will become 6,1,4,2,4 and 5 will be returned. If y=279 then array x will become 9,7,2 and 3 will be returned.
14. Define function float ba(float x[], float y[], int n). It returns $x_1*y_1+x_2*y_2+ \ldots + x_n*y_n$. Let n=4 and array x is 5,7,2,4 and array y is 8,2,5,3 then output is 5*8+7*2+2*5+4*3=76.
15. Define followings: [Do not use nested loop. In (ii) and (iii) use sum function]
    i. $(x_1 * y_n) + (x_2 * y_{n-1}) + (x_3 * y_{n-2}) + \ldots + (x_n * y_1)$
    ii. $x_1*(y_1+y_2+y_3+\ldots+y_n) + x_2*(y_1+y_2+ \ldots+y_{n-1}) + x_3*(y_1+y_2+\ldots+y_{n-2}) +\ldots.+ x_{n-1}*(y_1+y_2) + x_n*y_1$
    iii. $x_1*(y_1+y_2+\ldots+y_n) + (x_1+x_2)*(y_1+\ldots+y_{n-1}) + (x_1+x_2+x_3)*(y_1+\ldots+y_{n-2}) +...+ (x_1+x_2+\ldots+x_n)*y_1$

    Let n=4 and array x is 5,7,2,4 and array y is 8,2,5,3 then outputs are 86, 247 and 554 respectively.

16. Define function f(float x[],int n,int t,int p). It finds product of $(p-x_i)$ for i=1..n $i \neq t$.
17. Define function g(float x[],int n,int t). It finds product (x_t-x_i) for i=1..n $i \neq t$.
18. Write program to compute the value of summation $y_j*f(x,n,j,p)/g(x,n,j)$ for j=1..n. For n=5 and p=3 and x_i's 0,1,2,-1,10 and y_i's 0 11 78 9 30710 output is 309. A curve y=y(x) is fitted using Langrange interpolation passing (0,0) (1,11) (2,78) (-1,9) (10,30710). We have calculated the value of y(3). The equation of the curve is $3x^4+7x^2+x$.

Array (using pointers)

```
#include<stdio.h>
void rd(float a[],int k)
{  int i;
   for (i=0;i<=k-1;i++)
      scanf("%f",&a[i]);
}
float sum(float a[],int k)
{  int i;float s=0;
   for (i=0;i<=k-1;i++) s=s+a[i];
   return s;
}
main()
{  int n;float x[20],g;
   printf("Give n");scanf("%d",&n);
   rd(x,n);
   g=sum(&x[n-5],5);
   printf("  %f",g);
}
```

The given program reads n and n numbers. It finds the sum of last n numbers. A function is defined to find of all elements of an array. If n=7 and numbers are 4,5,2,5,6,4,7 then output is 2+5+6+4+7=24.

Modify the above program (do not modify the sum function) to print the value of following

27. $x_0 * (x_0+x_1) * (x_0+x_1+x_2) * \ldots * (x_0+x_1+\ldots+x_{n-1})$. For above input the answer is 119598336
28. $x_{n-1} * (x_{n-2}+x_{n-1}) * (x_{n-3}+x_{n-2}+x_{n-1}) * \ldots * (x_0+x_1+\ldots+x_{n-1})$. answer 661432464 (48)
29. Read p also and find the value of following. For above input and p=4 answer is 107712
    $(x_0+x_1+x_2+\ldots+x_{p-1})*(x_1+x_2+x_3+\ldots+x_p)*(x_2+x_3+x_4+\ldots+x_{p+1})*\ldots*(x_{n-p}+x_{n-p+1}+x_{n-p+2}+\ldots+x_{n-1})$

Write program, which reads two arrays $x_0 \ldots x_{n-1}$ and $y_0 \ldots y_{n-1}$. The program outputs the value of

30. $x_0*(y_0+y_1+y_2+\ldots+y_{n-1}) + x_1*(y_1+y_2+y_3+\ldots+y_{n-1}) + x_2*(y_2+y_3+y_4+\ldots+y_{n-1}) + \ldots + x_{n-1}*y_{n-1}$
31. $x_0*(y_0+y_1+\ldots+y_{n-1}) + (x_0+x_1)*(y_1+\ldots+y_{n-1}) + (x_0+x_1+x_2)*(y_2+\ldots+y_{n-1}) + \ldots + (x_0+x_1+\ldots+x_{n-1})*y_{n-1}$
32. $x_0*(y_1+y_2+\ldots+y_{n-1}) + (x_0+x_1)*(y_2+\ldots+y_{n-1}) + (x_0+x_1+x_2)*(y_3+\ldots+y_{n-1}) + \ldots + (x_0+x_1+\ldots+x_{n-2})*y_{n-1}$

Let n=4 and array x is 5,7,2,4 and array y is 8,2,5,3 then outputs are 188, 376 and 188 respectively.

33. Define function to find the value of $x_0+2x_1+3x_2+\ldots+kx_{k-1}$. Using this write program to find the value of $x_3+ 2x_4+ 3x_5+\ldots+(n-3)x_{n-1}$. If n=7 and numbers are 4,5,2,5,6,4,7 then output is 57.
34. Define a function to perform shift left ($x_i = x_{i+1}$). Using it define a function void del(float a[],int p,int n). It deletes p^{th} element. Let n=5 and initial array is 3,6,2,8,3 then call of del(x,2,5) will make the array as 3,6,8,3. [Hint: print array by wr(x,n-1)]. In the definition of function del no loop should be used.

35. Define a function to perform shift right. Using it define a function void insert(float a[],int p,int n,int b). It inserts p^{th} element as b. Let n=5 and initial array is 3,6,2,8,3 then call of insert(x,2,5,11) will make the array as 3,6,11,2,8,3.
36. A function is defined as follows. Using it write program to find $x_0+x_1+\ldots+x_{n-1}$. O/p 33
37. Read p also and find the value of find $x_p+x_{p+1}+\ldots+x_{n-1}$. above input and p=3 o/p 22

```
float sum(float a[],int k)
{   int i;float s=0;
    for (i=0;i<=k-3;i++) s=s+a[i-5];
    return s;
}
```

---

38. A function is defined as follows. Using it write program to add i to x_i. If n=5 and numbers are 3,6,2,8,3 then after the execution of the program they will be 3,7,4,11,7.
39. Write program so that they become 7,9,4,9,3

```
void f(float a[],int k)
{   int i;float s=0;
    for (i=0;i<=k-1;i++)
        a[i]=a[i]+1;
}
```

---

40. A function is defined as follows. Using it write program so that $x_i= x_i+x_{i+1}+x_{i+2}+\ldots+x_{n-1}$ is performed. If n=5 and numbers are 3,6,2,8,3 then after the execution of the program they will be 22,19,13,11,3.
41. Write program so that they become 3,9,11,19,22

```
void g(float a[],int k)
{   int i;float s=0;
    for (i=0;i<=k-2;i++)
        a[i]=a[i]+a[i+1];
}
```

Read n and n numbers (int) and store them in an array (x). Let n=20

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
3	7	2	8	12	16	31	20	16	19	21	23	24	11	16	19	22	26	29	31

Here {2,8,12,16} {20,16} {24} {16} {22,26} are group of even numbers.
{3,7} {31} {19,21,23} {11} {19} {29,31} are group of odd numbers.

1. Write program to print first group of even numbers. For above output 2 8 12 16.

2. Write program to print the second group of even numbers. For above output 20 16

3. Write program which reads int k also. The program prints the kth group of even numbers. If k=5 then output 22 26.

4. Write program, which finds the number of even groups. For above 5.

5. Write program, which finds the number of all groups. For above 5+6=11. [Caution: We may begin with even number. Similarly we may end by odd number]

6. Write program, which finds the kth group. For k=5 output 19 21 23. For k=6 output 24.

7. Write program, which prints the first group of even numbers in reverse order. For above output 16 12 8 2.

8. Write program, which prints the last group of even numbers in reverse order. For above output 26 22.

9. Write program, which prints the last group of even numbers. For above output 22 26.

10. Write program, which finds the minimum number of every even group. For above output 2 16 24 16 22.

11. Write program, which finds k^{th} number in every even group. If some group does not has k elements then –1 is printed. if k=2 then output 8 16 –1 –1 26.

12. Write program, which prints all numbers. The numbers in even group are reversed. For above output 3 7 16 12 8 2 31 16 20 19 21 23 24 11 16 19 26 22 29 31

13. Print all numbers in group wise reverse order. For above output
    29 31 22 26 19 16 11 24 19 21 23 20 16 31 2 8 12 16 3 7

14. Reverse the position of even groups.
    For above output 3 7 22 26 31 16 19 21 23 24 11 20 16 19 2 8 12 16 29 31

15. Declare a two dimensional array z by int z[20][50]. Store every group of even numbers as follows z[0][0]=2 z[0][1]=8 z[0][2]=12 z[0][3]=16 z[0][4]=-1 z[1][0]=20 z[1][1]=16 z[1][2]=-1 and so on. Now read an integer k. Print k^{th} group of even numbers by for(i=0; z[k][i]!=-1; i++) printf("%d ",z[k][i]);

Shifting

```
int n,i,x,a[20];
printf("Give n");
scanf("%d",&n);
for (i=0;i<n;i++)
   scanf("%d",&a[i]);
```

```
printf("Give
x");□scanf("%d",&x)
;□for(i=x;i<n-
1;i++)□
a[i]=a[i+1];□n=n-1;
```

```
for (i=0;i<n;i++)
   printf("%5d",a[i]);
```

Let initially n=8 and array is as follows.

a_0	a_1	a_2	a_3	a_4	a_5	a_6	a_7
12	17	24	16	13	19	29	16

The above program deletes x^{th} element. If x=3 then finally n=7 and the array will be

a_0	a_1	a_2	a_3	a_4	a_5	a_6
12	17	24	13	19	29	16

Write each of the following programs by modifying the contents of the box only.

1. Given x and y, insert a_x=y. If x=5 and y=34 then finally n=9 and array will be as follows.

a_0	a_1	a_2	a_3	a_4	a_5	a_6	a_7	a_8
12	17	24	16	13	34	19	29	16

2. Write program, which will exchange consecutive elements. Assume that 'n' is even.

For given input the final array will be.

a_0	a_1	a_2	a_3	a_4	a_5	a_6	a_7
17	12	16	24	19	13	16	29

3. Write program, which will delete elements at even position (elements a_0, a_2, a_4 , …. will be deleted). For given input finally n=4 and array will look as

a_0	a_1	a_2	a_3
17	16	19	16

4. Write program, which will delete all even elements. For given input finally n=4 and array will look as

a_0	a_1	a_2	a_3
17	13	19	29

5. Write program, such that between every pair of consecutive elements an element equal to their sum is inserted. For given input finally n=15 and array will look as

a_0	a_1	a_2	a_3	a_4	a_5	a_6	a_7	a_8	a_9	a_{10}	a_{11}	a_{12}	a_{13}	a_{14}
12	29	17	41	24	40	16	29	13	32	19	48	29	45	16

6. Write program, which will insert an element after every even element. The inserted element will be double of the even element (before it). For given input finally n=12 and array will look as

a_0	a_1	a_2	a_3	a_4	a_5	a_6	a_7	a_8	a_9	a_{10}	a_{11}
12	24	17	24	48	16	32	13	19	29	16	32

Matrix(using 1D array)

Any two dimensional matrix can be stored in one dimensional array. e.g. the following matrix

2	2	9	6
1	3	2	4
6	1	2	3

can be stroed as

a_0	a_1	a_2	a_3	a_4	a_5	a_6	a_7	a_8	a_9	a_{10}	a_{11}
2	2	9	6	1	3	2	4	6	1	2	3

Two more variables nr and nc store the number of rows and columns. Here they are 3 and 4 respectively.

The following program reads the elements of a matrix and strores in an array. The program finds [x,y]th element. e.g. [2,3]th element is 2. [3,1]th element is 6. The program also finds at what locations a given element is present. e.g. the element 3 is present at locations [2,2] and [3,4].

```
main( )
{  int nr,nc,i,x,y;
   float a[100],t,element;
   printf("Give size");
   scanf("%d %d",&nr,&nc);
   printf("Give elements");
   for (i=0;i<nr*nc;i++)
      scanf("%f",&a[i]);
   printf("Give coordinates");
   scanf("%d %d",&x,&y);
   t=a[nc*(x-1)+y-1];
   printf("[%d,%d]=%f",x,y,t);
   printf("\nGive element");
   scanf("%f",&element);
   printf("Locations ");
   for (i=0;i<nr*nc;i++)
      if (element = = a[i])
      {  x=i/nc+1;y=i%nc+1;
            printf("[%d,%d]",x,y);
      }
}
```

1. Write program to print xth row
   Input 2 Output 1 3 2 4

2. Write program to print xth column
   Input 2 Output 2 3 1

3. Write program to print matrix.

4. Write program to print transpose.

5. Write program to print maximum element of xth row Input 2 Output 4

6. Write program to print maximum element of xth column.

7. Assume that the given matrix is a square matrix. Write program to print the diagonal elements. e.g.
   Matrix 5 7 9 4
   2 4 7 3
   1 5 9 2
   3 7 4 8
   Output 5 4 9 8

8. Write program to print the element of another diagonal. For above matrix the output will be 4 7 5 3

9. Write program to find MIN-MAX_ROW. Find maximum element of every row and find minimum of those. For the matrix given in the example the output is 4. Here maximum elements of rows are 9, 4 and 6 respectively.

10. Write program to find MIN-MAX_COLUMN. Find maximum element of every column and find minimum of those. For given matrix output 3 (since col max are 6,3,9,6)

The following 3 dimensional matrix

Z=1	Z=2	Z=3	Z=4
2 3 4	4 5 2	2 4 7	2 7 6
5 7 3	3 5 2	3 9 4	2 7 3

will be stored as follows.

a_0	a_1	a_2	a_3	a_4	a_5	a_6	a_7	a_8	a_9	a_{10}	a_{11}	a_{12}	a_{13}	a_{14}	a_{15}	a_{16}	a_{17}	a_{18}	a_{19}	a_{20}	a_{21}	a_{22}	a_{23}
2	3	4	5	7	3	4	5	2	3	5	2	2	4	7	3	9	4	2	7	6	2	7	3

The valuses of nx, ny and nz are 3 2 and 4 respectively.

1. Write program to find (x,y,z) element. If input is x=3 y=1 z=4 then output is 6.

2. Write program, which will print location (xyz) of all those elemetns which are 7. For above case it will be (221), (313), (214), and (224).

3. Write program, which will print the location of the maximum element. In above case the maximum element is 9. Its location is (223). If maximum element is present more than once than any one of its locations can be printed.

4. Write program, whose inputs are y and z and output is all elements with same y and z values. Input y=2 z=3 output 3 9 4.

5. Write program, whose inputs are x and z and output is all elements with same x and z values. Input x=2 z=3 output 4 9.

6. Write program, whose inputs are x and y and output is all elements with same x and y values. Input x=2 y=1 output 3 5 4 7.

7. Write program, whose input is z and output is all elements in same xy plane. For z=2 the output will be
   4 5 2
   3 5 2

8. Write program, whose input is y and output is all elements in same xz plane. For y=2 the output will be
   5 7 3
   3 5 2
   3 9 4
   2 7 3

9. Write program, whose input is x and output is all elements in same yz plane. For x=2 the output will be
   3 7
   5 5
   4 9
   7 7

10. Write program to find MAX-MIN-MAX-ZYX. Find Min-Max for each XY plane (for z=1,2,3, and 4). Now find their maximum. For above example the output is 7. Here Min-Max for every matrix are 4, 5, 7 and 7 respectively.

Matrix(using 2D array)

```
#include<stdio.h>
main( )
{  float a[10][20],p;int i,j,nr,nc,x;
   printf("Give size of matrix ");
   scanf("%d %d",&nr,&nc);
   printf("Give elements of matrix\n");
   for (i=1; i<=nr; i++)
       for (j=1; j<=nc; j++)
           scanf("%f",&a[i][j]);

   printf("Give row no ");
   scanf("%d",&x);
   printf("Give multiplier");
   scanf("%f",&p);
   for (j=1;j<=nc; j++)
       a[x][j]=a[x][j]*p;

   for (i=1; i<=nr; i++)
   {   for (j=1; j<=nc; j++)
           printf("%7.3f ",a[i][j]);
       printf("\n");
   }
}
```

The above program does $R_x = p*R_x$

1. Write program, which prints the diagonal elements of the matrix.

2. Write program, which reads two matrixes and finds their sum.

3. Write program, which reads two matrixes and finds their product.

4. Write program, which finds the minimum element of every row.

5. Write program, which finds the maximum of all elements computed in above question.

6. Write program, which divides every row by corresponding diagonal element.

Input				Output			
3	6	0	9	1	2	0	3
4	2	1	6	2	1	0.5	3
6	6	3	12	2	2	1	4

7. Write program, which reads integers x and y and float p and performs
   $R_x = R_x - p*R_y$

8. Write program, which reads integers x and y and performs the above row operation. Take p such that a_{xy} becomes 0.

9. Write program, which reads integer y and performs the above operation for every x ($x=1..n_r$ and $x \neq y$).

10. Write program, which performs above for all y ($y=1..n_r$).

11. Write program find the solution of simultaneous linear equations. Input is a matrix where $n_c = n_r + 1$.

12. Write program to find the inverse of a matrix. [Hint: form a matrix A:I. Convert it in the form I:B. B is the inverse.

Recursion

```
int sum(int x)
{  if (x<10) return x;
   else return x%10+sum(x/10);
}
main( )
{  int p;
   scanf("%d",&p);
   printf("%d\n",sum(p));
}
```

The above is a recursive function to find sum of digits. Similarly recursive function to delete k^{th} digit from last and maximum digit are defined.

```
int del(int x, int k)
{  if (k==1) return x/10;
   else return del(x/10,k−1)*10+x%10;
}

int max(int x)
{  int t;
   if (x<9) return x;
   else { t=max(x/10);
          if (t > x%10) return t;
          else return x%10;
        }
}
```

1. Define function to find first digit of a number.
2. Define function to find second digit of a number.
3. Define function to find the number of digits.
4. Define function to find sum of digits except second digit. sm(231456)=2+1+4+5+6=18.
5. Define function to find last even digit. f(354683257) returns 2.
6. Define function to find digit before last even digit. f(354683257) returns 3.
7. Define function to find digit after last even digit. f(354683257) returns 5.
8. Define function to find the location of the last even digit from last. e.g. in 354683257 the last even digit is 2 and its location from last is 3.
9. Define function int g(int x, int k). It finds k^{th} digit (from last) of number x. g(2536487,3) returns 4.
10. Define function to find sum of even digits.
11. Define function int h(int x, int k). It finds k^{th} evrn digit (from last) of number x. g(2561487,3) returns 6.
12. Define function int f(int x). It returns (3rd digit)(1st digit)*2. f(21719) will return 144 (72*2).
13. Define function to find factorial of a number.
14. Define function float power(float x, int n). It finds x^n.
15. Define function int sum(int a, int b). It find the value of a+(a+1)+(a+2)+ … + b. Assume a<b. e.g. sum(5,9) will return 5+6+7+8+9=35.
16. Define function int gcd(int a, int b). It finds greatest common divisor of a and b.
17. Define function to delete first digit. del(31452) returns 1452.
18. Define function to delete second digit.
19. Define function to delete all even digits.
20. Define function to increment first digit. Assume that it is not 9.
21. Define function int inc(int x, int k). It increments k^{th} digit (from last) of number x. inc(2536987,4) returns 2537987.
22. Define function int join(int x, int y). It joins numbers x and y. join(273,4167) returns 2734167.

23. Define function int replace(int x, int y). The first digit of the number is replaced by y. replace(27349,5) returns 57349.
24. Define function to increment all even digits.
25. Define function to increment last even digit.
26. Define function to exchange first and second digit.
27. Define function to find difference between first and last digit. diff(231456)=2−6=−4.
28. Define function to exchange first and last digit.
29. Define function float pwr(complex x). It returns $x.r^{x.i}$. pwr(3+4i) will return 81.
30. Define

Write recursive functions for following. No other function should be defined. No loop while, for etc. should be used.

31. Delete first digit. Input 4523 output 523

32. Increment first digit. Input 4523 output 5523. Assume first digit is not 9.

33. Increment second digit. Input 4523 output 4623.

34. Increment all digits. Input 4523 output 5634.

35. Find last even digit. Input 3856641 output 4.

36. Multiply all digits less than 5 by 2. Input 32571 output 64572.

37. Find maximum digit by using following logic. Let the input number x can be written in the form Pqr. Here P is some number and q and r are digits.
    maxdigit(Pqr) = if (q>r) maxdigit(Pq) else maxdigit(Pr)
    e.g. maxdigit(5724)=maxdigit(574)=maxdigit(57).

38. Find first even digit. Input 3856641 output 4.

39. Replace all occurrences of 5 by 6. Input 452554 output 462664.

40. Replace last occurrence of 5 by 6. Input 452554 output 452564. Assume digit 5 is present at least once.

41. Replace first occurrence of 5 by 6.

42. Delete last even digit. Input 452673 output 45273.

43. Delete first even digit. Input 152678 output 15678.

44. Sum of product of consecutive digits. Input 5274 output 5x2+2x7+7x4=52.

45. Sum of product of consecutive even digits. Input 5274618 output 2x4+4x6+6x8=80.

46. Location of last even digit from last. input 56836411 output 3. Assume that the number has at least one even digit.

47. Location of last even digit from beginning. For above input output is 6.

48. Location of first even digit from last. For above input output is 7.

49. Location of first even digit from beginning. For above input the output is 2.

50. Define int location (int x, int k). It returns the kth digit (from end) of x. Assume that x has at least k digits. e.g. location (327965,2) will return 6.

51. Do above problem to return the kth digit from beginning.

52. sum_factors(int x, int y, int z). It returns the sum of all factors of x between y and z. e.g. sum_fators(36,4,13)=31 (because 4+6+9+12).

53. int nCr(int n, int r). It returns the value of ${}^{n}C_{r}$ Use following logic:

$$
\begin{aligned}
{}^{n}C_{r} &= 1 && \text{if } n=r \text{ or } r=0 \\
&= {}^{n-1}C_{r-1} + {}^{n-1}C_{r} && \text{otherwise}
\end{aligned}
$$

54. int location(int x, int y). It returns the location (from last) of digit y in number x. In case more than one occurrences of y in x are present then the location of last y is returned. e.g. location(5273476,7)=2. It is assumed that y is present in x at least once.

55. Do above question when the location of first y is returned. e.g. location(5273476,7)=5. [Hint: use following]

loc(x,y) =-1	if (x<10) and ($x \neq y$)
= 1	if (x<10) and (x=y)
=loc(x/10,y)+1	if ($x \geq 10$) and loc(x/10,y)>0
=-1	if ($x \geq 10$) and loc(x/10,y)<0 and ($x\%10 \neq y$)
= 1	if ($x \geq 10$) and loc(x/10,y)<0 and (x%10=y)

Here also we assume that y is present in x al least once. However, we have over defined the function for the case when it is not present.

56. int location(int x, int y). It returns the location (from beginning) of digit y in number x. In case more than one occurrences of y in x are present then the location of last y is returned. e.g. location(5273476,7)=6. Logic:

loc(x,y) = 1	if (x<10)
=loc(x/10,y)	if ($x\%10 \neq y$)
=1+loc(10*(x/100)+y,y)	otherwise

57. Do above by over defining.

58. Do above question when the location of first y is returned. e.g. location(5273476,7)=3. Logic:

loc(x,y) =-1	if (x<10) and ($x \neq y$)
= 1	if (x<10) and (x=y)
=loc(x/10,y)	if ($x \geq 10$) and loc(x/10,y)>0
=loc(x/10,y)-1	if ($x \geq 10$) and loc(x/10,y)<0 and ($x\%10 \neq y$)
=-loc(x/10,y)+1	if ($x \geq 10$) and loc(x/10,y)<0 and (x%10=y)

Here function is over defined to return (negation of) length of x when y is not present.

59. Replace first occurrence of 5 by 6. Input 258553 output 268553. (Use over definition)

```
int f(int x, int *pr, int *f)
{  int s=0,p=1,t;
   while (x>0){
     t=x%10;
     s=s+t;
     p=p*t;
     x=x/10;
   }
   *pr=p;*f=t;
   return(s);
}
```

```
main( )
{  int a,b,c,d;
   printf("Give number");
   scanf("%d",&a);
   b=f(a,&c,&d);
   printf("sum=%d ",b);
   printf("product=%d ",c);
   printf("first digit=%d ",d);
}
```

Recursive function

```
int g(int x, int *pr, int *f)
{  int s,p;
   if (x<10)
   {  *pr=x;*f=x;return x;}
   else
   {  s=g(x/10,&p,f);
      *pr=p*(x%10);
      return(s+x%10);
   }
}
```

Write recursive and non recursive programs for followings.

1. Define function void f(int x,int *y,int *z). f(12,&a,&b) makes a=24 b=26. Double and triple.
2. Define function void dbl(int *x). dbl(&x) will double the value of x.
3. Define function, which returns first and last digit. e.g. input 32579875 output 3 and 5. Do this problem by following methods:
   (A) When first digit is returned and reference of last digit is passed.
   (B) When last digit is returned and reference of first digit is passed.
4. Define function, which will return two numbers. In one of them first digit is deleted and in the second last digit is deleted. For input 32579875 output will be3257987 and 2579875.
5. Define function, which will return numbers after deleting even and odd digits. input 325768175 output 357175 and 268.
6. Define function, which finds how many digits are less than 5 and how many are more than 7. Input 2356781 output 3 and 1.
7. Define function, which finds how many digits are more than 5 and how many are less than 7. Input 2356781 output 3 and 5.
8. Define function to return maximum digit and how many times, it occurs. input 28688387 output 8 and 4.
9. Define function, which will return maximum and second maximum digit.
10. Define function to return first and last even digit. input 32456875 output 2 and 8.
11. Define function to return location of last even digit from beginning and from last. input 32798175 output 5 and 4.
12. Define function to return location of first even digit from beginning and from last. input 32798175 output 2 and 7.
13. Define function, which will return numbers after deleting first and last even digit. input 325768175 output 35768175 and 32576175.
14. Define function, whose inputs are two numbers. The function returns two numbers. The first number is the number of digits of first number present in the second number. The second number is the number of digits of second number present in first number. input 2322677 and 13237 output 6 and 4. It is because digits 2, 3, 2, 2, 7 and 7 of first number are present in the second number.
15. Define function float power(float x, int n, float *k). It finds x^n and x^{-n}.

16. Define function int f(float a[],int size,int*). It finds location of maximum and second maximum element in array $a_1, a_2, \ldots a_{n-1}$.
17. Define function float f(float x[],int size,float*). It finds maximum and minimum element.

```
float max (float x[], int size)
{  float t;
   if (size==1) return(x[0]);
   else{
        t=max(x,size-1);
        if (t>x[size-1]) return(t);
        else return(x[size-1]);
   }
}
void insert (float x[], int size, float y)
/* insert a given element y
    as x0 in the array */
{  if (size==0) x[0]=y;
   else{
        x[size]=x[size-1];
        insert(x,size-1,y);
   }
}
```

```
main()
{  int i,n; float a[20],ans,p;
   printf("How many elements ");
   scanf("%d",&n);
   for (i=0;i<n;i++)
       scanf("%f",&a[i]);

   ans=max(a,n);
   printf("Maximum  %7.4f\n",ans);

   printf("Give element to be inserted  ");
   scanf("%f",&p);
   insert(a,n,p);

   for (i=0;i<n+1;i++)
       printf("%7.3f ",a[i]);
}
```

1. Write program to find the sum of all elements of the array.
2. Find $((\ldots (((x_0 - x_1) - x_2) - x_3) - \ldots\ldots\ldots - x_{n-2}) - x_{n-1})$
3. Find $x_0*x_1 + x_1*x_2 + x_2*x_3 + \ldots\ldots\ldots\ldots + x_{n-2}*x_{n-1}$
4. Find location of maximum element. i.e. i such that x_i is maximum.
5. Write program to find the biggest i such that x_i is negative. Assume that the array has at least negative element.
6. Write expression when followings are called?

(A) f(x,7) (B) f(x,8)
(C) f(x,9) (D) f(x,10)

```
float f(float x[],int size)
{  float t;
   if(size<=2)return(x[0]);
   if (size%2==0) return(f(x,size-3)+x[size-2]);
   else return(f(x,size-1)-x[size-1]); }
```

7. Write program to delete x_0 .
8. Write program, which reads an integer k and deletes x_k .
9. Write program, which inserts an element as x_k .
10. Assume that the array is sorted. Insert element at correct place.
11. Assume that the size of the array is even. Exchange x_0 and x_1, x_2 and x_3, x_4 and x_5, ….., x_{n-2} and x_{n-1} .
12. Assume that size of the array is even. Delete $x_0, x_2, x_4, \ldots , x_{n-4}, x_{n-2}$.
13. Make array as $x_0, x_0+ x_1, x_1, x_1+ x_2, x_2, x_2+ x_3, x_3, \ldots , x_{n-2}, x_{n-2}+ x_{n-1}, x_{n-1}$.
14. Take two arrays x and y of same length. Form a new array z as follows: $z_0=x_0, z_1= y_0, z_2=x_1, z_3= y_1, z_4=x_2, z_5= y_2, \ldots , z_{2n-2}=x_{n-1}, z_{2n-1}= y_{n-1}$.
    Function header will look as void form(float x[], float y[], int size, float z[]) .

15. Take two arrays x and y of length m and n. Form a new array z as follows: $z_0=x_0$, $z_1=x_1$, $z_2=x_2$, … , $z_{n-1}= x_{n-1}$, $z_n=y_0$, $z_{n+1}= y_1$, …….. , $z_{m+n-1}=y_{m-1}$.
16. Take two arrays x and y. Put elements of array y in array x at the end.
17. Take two sorted arrays x and y. Form a sorted array z.
18. Find $(x_0 - (x_1 - (x_2 \ldots\ldots\ldots\ldots - (x_{n-4} - (x_{n-3} - (x_{n-2} - x_{n-1}))) \ldots)))$
    [Hint: First try to do this problem by destroying the array. Now make appropriate changes so that old array is restored.]
19. Write program to find the smallest i such that x_i is negative.
20. Define function int del(float x[],int size). All negative numbers in array x are deleted. The size of the final array is returned.
21. Define function int ins(float x[],int size). After every negative number, a number which is its double, is put. e.g. initial array 7,-2,1,-4,-3 final array 7,-2,-4,1,-4,-8,-3,-6

String

```
#include<stdio.h>
main()
{  char t[100];int i;
   printf("Give string");
   scanf("%s",t);
   printf("%d",t[2]);
}
```

Read a string and print ascii code of second letter input kbAp output 65 (not 98). The 0th letter is k. Input rm4t output 52 (48+4).

```
#include<stdio.h>
main()
{  char t[100];int p,q;
   printf("Give string ");
   scanf("%s",t);
   t[0]=t[0]+1;
   puts(t);
}
```

Read a string and increment first letter. Input ram output sam.

```
#include<stdio.h>
main()
{  char t[100];
   printf("Give string");
   scanf("%s",t);
   t[3]=0;
   puts(t);
}
```

Read a integer (in string form) and delete all letters except 0th 1st and 2nd input kapil output kap.

1. Write program to replace the 3rd letter by g. Input ramesh output ramgsh.
1. Write program to change the 0th letter from capital to small. Input RaM output raM.
2. Write program to replace 1st letter by 2nd letter. Input hari output hrri.
3. Write program to exchange 1st letter and 2nd letter. Input hari output hrai.
4. Write program to change the case of 0th letter. Input harIF output HarIF. Input YauOh output yauOh.

```
main()
{  char t[100];int p,q;
   printf("Give string");
   scanf("%s",t);
   p=t[0]-48;
   q=t[1]-48;
   r=p+q;
   printf("%d",r);
}
```

Read a integer (in string form) and find sum of first two digits (0th and 1st). Input 2314 output 2+3=5.

```
main()
{  char t[100];int p,q;
   printf("Give string");
   scanf("%s",t);
   p=t[0]-48;
   q=p*2;
   t[0]=q+48;
   puts(t);
}
```

Read a integer (in string form) and doubles of its 0th digit.

5. Write program to find the double of the number formed by first two digits. Input 27395 output 54.
6. Write program to make the first digit half. Input 239 output 139. Input 75 output 35.
7. Write program, which reads a string of 3 letters. 0th and 2nd letters are digits. The 1st digit is +,-,* or /. Input 5+3 output 8. Input 7*4 output 28. Input 8/3 output 2.33.

```
main()
{  char t[100],p;
   printf("Give string");
   scanf("%s",t);
   i=0;
   while (t[i]!=0) i++;
   p=t[i-1];
   printf("%c",p);
}
```

Read a string and find last letter.

Definition: The string is ended by a letter whose ASCII code is 0.

```
main()
{  char t[100];int i,count=0;
   printf("Give string");
   scanf("%s",t);
   i=0;
   while (t[i]!=0)
   {  if ((t[i]>=48)&&(t[i]<=57))
         count++;
      i++;
   }
   printf("%d",count);
}
```

Read a string and find number of digits input w3er47ty output 3

8. Write program to find the length of the string. Input hari output 4.

9. Write program to find the maximum letter. Input ramesh output s.
10. Write program replace all capital letters by small letters. Input RamIs output ramis.
11. Write program to delete the last letter. Input hariprasad output hariprasa.
12. Write program to exchange first (0th) and last letter. Input hari output iarh.
13. Write program to delete the 2nd letter. Input haripal output haipal. [Hint: use shift]
14. Write program, which reads an integer (in string form) and finds sum of digits. Input 2317 output 2+3+1+7=13.
15. Write program to find first even digit. Input 572984379 output 2.
16. Write program to print last even digit. For above input output is 4.
17. Write program to exchange first and last even digits. Input 1325789 output 1385729.
18. Write program to print the double of the number. Input 7354 output 14708.
19. Write program to print the double of the reverse of the number. Input 7354 output 9074 (4537*2). Input 721345 output 1086254. (543127*2)
20. Do both of the above problems using one loop only.

For following problems the string should be read using gets(t) in place of scanf("%s",t);

21. Write program to find first word. Input ram prasad kumar dey. Output ram. [Hint: replace first blank (ascii 32) by 0]
22. Write program to delete the last word. In above case ram prasad kumar.
23. Write program to print short form. For above input output rpkd. [letter after blank]
24. Write program to print the length of second word. In above case 3.
25. Write program to print the last word in reverse order. In above case yed.
26. Write program to print the first word in reverse order. Input mar.
27. Do above 2 problems without using printf("%c",t[i]) [use puts(g)]
28. Write program to print the short form. In above case rpkdey.
29. Write program to print the short form as r.p.k.dey.
30. Write program to find sum of numbers in a string. Input 32 4235 78 11 output 4364.

List

A list is a string, which is enclosed in round parathenses.
e.g. (((5 (7 24)) ram) 7 (2 mohan) ( ) (hari ram) (12 (34)))

1. Write a program, which reads a list (using gets) and prints its first element. (in above case it is ((5 (7 24)) ram).

2. Write program to print the second element (in above case it is 7).

3. Write program, which reads k(integer) also and prints k^{th} element. (in above case if k is 5 then output is (hari ram).

4. Write program which prints the number of elements in the list (in above case 6).

5. Write program, which will print all words. (in above case 5, 7, 24, ram, 7, 2, mohan, hari, ram, 12, and 34.

6. Write program, which will print all words and their depth. (in above case [5 3], [7 4], [24 4], [ram 2], [7 1], [2 2], [mohan 2], [hari 2], [ram 2], [12 2], and [34 3].

7. Write program, which prints the list after deleting the first element. (in above case the output will be (7 (2 mohan) ( ) (hari ram) (12 (34))).

8. Write program, which prints the reverse of the list.
   in above case it is . ((12 (34)) (hari ram) ( ) (2 mohan) 7 ((5 (7 24)) ram))

9. Write program, which reads another string (using gets) puts it as first element of the list. If second string is (12 56) then the output is ((12 56)((5 (7 24)) ram) 7 (2 mohan) ( ) (hari ram) (12 (34))).

10. Write program to print all numbers of the list. (in above case 5, 7 24, 7, 2, 12, 34).

11. Write program, which reads a list. There are only three elements in the list. The first element is ram, hari, bimu or sani. The second and third elements are numbers. If first element is ram then sum of these numbers is printed. If the first element is hari then difference is printed. Similarly in case of bimu and sani their product and division are respectively printed. If input is (ram 12 34) then output is 46. If input is (hari 120 34) then output is 86.

12. Do above problem when the list may have more then two numbers also in case of ram and bimu. If input is (bimu 12 6 5 2) then output is 720.

13. Write program, which reads a list and prints its value. If input is (ram (bimu 12 (hari 10 4)) (sani 130 5)) then output is 98. (since12*(10-4)+(130/5) is 96).

Structure

```
#include<stdio.h>
#include<math.h>
typedef struct {float x,y,z;} point;
float distance (point a, point b)
{  float c,t1,t2,t3;
   t1=a.x – b.x;
   t2=a.y – b.y;
   t3=a.z – b.z;
   c=sqrt(t1*t1+t2*t2+t3*t3);
   return c;
}
main( )
{  float ans; point p1,p2;
   printf("Give first point");
   scanf("%f %f %f",&p1.x,&p1.y,&p1.z);
   printf("Give second point");
   scanf("%f %f %f",&p2.x,&p2.y,&p2.z);
   ans=distance(p1,p2);
   printf("The distance is %f",ans);
}
```

```
#include<stdio.h>
typedef struct {float r,i;} complex;
complex add (complex a, complex b)
{  complex c;
   c.r = a.r + b.r;
   c.i = a.i + b.i;
  return c;
}
main( )
{  int n,i; complex x,sum;
   printf("Give n");scanf("%d",&n);
   printf("Give n complex numbers");
   sum.r =0;  sum.i =0;
   for (i=1;i<=n;i++)
   {   scanf("%f%f",&x.r,&x.i);
       sum=add(sum,x);
   }
   printf("Sum=%f+%fi",sum.r,sum.i);
}
```

1. Define point mid (point a, point b). It finds mid point of A and B.
2. Define vector as follows: typedef struct {float i,j,k;}vector; Using it define following functions float dotproduct (vector a, vector b) and vector crossproduct(vector a , vector b).
   Dotproduct of 2**i**+3**j**+1**k** and 5**i**+2**j**+3**k** is 19 and their cross product is 7**i**–**j**–11**k**.
3. Define float magnitude(vector v) e.g. magnitude 4**i**+3**j**+12**k** is 13. [Call function dot product].
4. Define vector mul(vector v, float t). It returns a vector by multiplying it by t. If a=4**i**+3**j**+12**k** then mul(a,0.5) will return 2**i**+1.5**j**+6**k**.
5. Define vector unitvector(vector v). It returns unit vector in the direction of **V**. [Call magnitude and mul functions]. Unit vector for 4**i**+3**j**+12**k** is 0.31**i**+0.23**j**+0.92**k**.
6. Using the structures of vector and point define vector direction(point a, point b). It returns vector corresponding to the line segment AB. If points are (2,3,1) and (6,2,8) then direction is 4**i**-**j**+7**k**.
7. Define float area(point a, point b, point c). It finds area of triangle ABC. [Find direction corresponding to AB and AC. Now call cross product and magnitude]. Area of the triangle with end points (3,4,5), (7,3,6) and (6,2,3) is 6.36.

8. Define struct {float x,y} point; and struct {float a,b,c;}line;
   Define function line equation(point p, point q) Using it write program which reads two points and finds the equation of line joining them. Let P=(2,3) and Q=(4,7) then line is 2x-y-1=0. Hence a=2 b=–1 and c=–1.
9. Write program, which reads two lines and finds the point of intersection. To accomplish it define a function point intersection(line g, line h). e.g. Input 2x+3y-26=0 and 4x+7y-54=0 output (10,2).

10. Write program, which reads four points and finds the point of intersection of the line joining first two points and the last two points. Let input points be (4,6), (-2,10), (10,2) and (3,6) then the point of intersection is (10,2). [Hint: use above functions]
11. Define function float distance(point p, line l). Using it write program, which reads a point and a line. The program, finds the distance of the point 'p' from the line 'l'. e.g. input point (2,3) and line 8x+6y-12=0 distance = $(8*2+6*3-12)/(8^2+6^2)^{1/2}$ = 2.2.
12. Write program, which reads three points and finds the distance of first point from the line joining other two points. [Hint: use functions to find equation and distance]

13. Define struct {point c;float r;} circle; Now define following functions
14. void print(circle x). It prints the equation of the circle. Let x be a circle whose centre is (2,3) and radius is 7. Call to a function circle(x) will output $x^2+y^2-4x-6y-36=0$. It is because $(x-2)^2+(y-3)^2=7^2$ is the equation of the circle. [It may be printed as $x^2+y^2+-4x+-6y+-36=0$]
15. circle form(float g,float h,float k). x=form(-4,-6,-36) will make a circle whose centre is (2,3) and radius is 7.
16. float chordlength(circle a,line b). It returns the length of the chord corresponding to circle 'a' and line 'b'. If a circle has centre (1,2) and radius 5 and the line is 5x+12y+10z=0 then chord length is 8. [Method: find distance (d) between centre and line by using appropriate function call now use $(r^2-d^2)^{1/2}$.] Assume that the line intersects with the circle.
17. float intersection(circle a, float b). It returns the y-coordinate of point of intersection of circle (s) and line (x=b). Let a circle (s) has centre (7,2) and radius 5. y=intersection(s,10) will make y as 6 (or -2). It is because circle $(x-7)^2+(y-2)^2=5^2$ intersects with the line x=10 at points (10,6) and (10,-2).

18. Define Complex mul(complex a, complex b) for multiplication of two complex numbers. Similarly define function for addition of complex numbers.
19. Read complex number x and integer n. Find x^n. [Call multiplication function]
20. Read a number n and n complex numbers. Output that complex number, which has maximum magnitude. [Hint: Define function for magnitude of complex number]
21. Write program to find $1 + x + x^2 + x^3 + \ldots + x^n$. Here inputs are x(complex) and n(int). Use functions for multiplication and addition of complex number. [Input 0.75–0.433i output 1–1.732i, when large n is taken]
22. Define Complex mult(complex a, float b) for multiplication of a complex number with a float. Using it write program to find $1 + 2x + 3x^2 + 4x^3 + \ldots + nx^{n-1}$.
23. Define complex expo(complex x,int n). It returns $1+x+\frac{x^2}{2!}+\frac{x^3}{3!}+\ldots+\frac{x^n}{n!}$ When n is large the above become e^x. Let x=0+1.05i then call of expo(x,10) returns 0.50+0.87i.

Do following problems by Defining struct {float x,y,z} point; and struct {float i,j,k;}vector; Also define suitable functions.

24. Define float projection (point a, point b, vector v). It returns length of projection of line segment AB in the direction of vector **V**.
25. Define float dist(point a, point b, point c). It finds distance of point A from the line joining points B and C.
26. Define point ppp(point a, point b, float d). It returns a point C on the line joining A and B. Length(AC)=d. Length(BC)<Length(AB).

27. Define qqq. It is similar to ppp but Length(BC)>Length(AB).
28. Define float sd(point a, point b, point c, points d). It returns shortest distance between lines joining A and B with C and D.

If less programs

```
main()
{   int i,n,x,c=0;
    printf("Give n");
    scanf("%d",&n);
    for (i=1;i<=n;i++)
    {   scanf("%d",&x);
        c=c+(x%2);
    }
    printf("%d\n",c);
}
```

This program reads n and n numbers and finds how many of these are odd. Similarly write programs for followings. Do not use if, ?, switch, while , another for.

1. How many of these are even?
2. Find sum of all odd numbers.
3. Find sum of all even numbers.
4. How many of these numbers leave remainder 2 when divided by 3?
5. How many of these are multiple of 3?
6. How many of these are multiple of 7?
7. How many of these numbers leave remainder 2 when divided by 7?
8. How many of these are multiple of 7 but not of 3?
9. Find first even number. Assume that there is at least one even element.
10. Find the last even number.
11. Find second even number. Assume that at least two numbers are even.
12. Find second last even number.
13. Find number immediately before first even number. Assume that first number is not even.
14. Find number immediately after first even number. Assume that last number is not the only even number.
15. Find number immediately before last even number. Assume that first number is not the only even number
16. Find number immediately after last even number. Assume that last number is not even.
17. Sum of numbers immediately before even numbers.
18. Sum of all numbers after first even number.
19. Sum of all numbers after last even numbers.
20. Sum of all numbers before first even number.
21. Sum of all numbers before last even number.
22. Sum of product of consecutive odd numbers.
23. Read a number (x). Assume that input number (x) is 0 or 1. The output is 17 if input is 0. The output is 22 if input is 1.
24. Read a number (x). Assume that input number (x) is 0, 1 or 2. The output is 17 if input is 0, 22 if input is 1, 30 if input is 2.
25. Read a number. Output 1 if the given number is more than 100. The output is 0 otherwise.

26. Read two numbers. Find their maximum. Assume that both numbers are positive integers.
27. Do above problems when both numbers are integers bigger than −100.

Ifless

```
#include<stdio.h>
main()
{
  int i,n,x,s=0;
  printf("Give n");
  scanf("%d",&n);
  for (i=1;i<=n;i++)
  {  scanf("%d",&x);
     s=s+x*((x+2)%3/2);
  }
  printf("%d",s);
}
```

This program reads n numbers and finds sum of those numbers, which are multiple of 3. Let n=6 and numbers are 10, 15, 22, 11, 27, 19 then output is 15+27=42.

```
s=s+2*x/(x+30);
```

How many of them are more than or equal to 30. Assume that all numbers are positive.

```
s=s+(1-(x%2))*(2*x/(x+30));
```

How many of them are even and more than or equal to 30.

```
s=s+1-(x%2)*(2*x/(x+30));
```

How many of them are even or less than 30.

```
p=11;     p=some odd number
for (i=1;i<=n;i++)
{  scanf("%d",&x);
   p=x*(p%2)+(1-p%2)*p;
}
printf("%d",p);
}   It finds first even number.
```

```
p=p*(x%2)+(1-x%2)*x;
```

It finds last even number.

```
s=0;
for (i=1;i<=n;i++)
{  scanf("%d",&x);
   ans=ans*(x%2)+s*(1-x%2);
   s=s+x;
}
printf("%d",ans);
```

It find the sum before last even number. If numbers are 2, 3, 7, 4, 6, 9 then output is 2+3+7+4=16.

```
for (i=1;i<=n;i++)
{  scanf("%d",&x);
   s=s+x;
   s=s*(x%2);
}
```

It find the sum after last even number. If numbers are 2, 3, 7, 4, 6, 9 then output is 6+9=15.

```
p=11;s=0;
for (i=1;i<=n;i++)
{  scanf("%d",&x);
   p=x*(p%2)+(1-p%2)*p;
   s=s+x*(p%2);
}
printf("%d",s);
```

It find the sum before first even number.

```
s=s+x*(1-p%2);
p=x*(p%2)+(1-p%2)*p;
```

It find the sum after first even number.

```
m=1;
for (i=1;i<=n;i++)
{  scanf("%d",&x);
   m=x*((x+x)/(x+m))+m*(1-
(x+x)/(m+x));
}
printf("%d",m);
}
```

It finds the maximum number. Assume all numbers are positive numbers.

```
p=0;s=0;
for (i=1;i<=n;i++)
{  scanf("%d",&x);
   s=s+(x%2)*x*p;
   p=(x%2)*x+(1-x%2)*p;
}
printf("%d",s);
```

sum of product of consecutive odd numbers. Input 2,4,7,9,6,3,8,5 output 7x9+9x3+3x5=105.

Program Generation

The following program reads n and outputs a program. The outputted program reads n numbers and outputs their sum.

```
#include<stdio.h>
main()
{ int i,n;
  scanf("%d",&n);
  printf("#include<stdio.h>\n");
  printf("main()\n{\nfloat sum=0,x;\n");
  printf("printf(%cGive %d numbers%c);\n",34,n,34);
  for (i=1;i<=n;i++)
  {  printf("scanf(%c%cf%c,&x);",34,37,34);
     printf("sum=sum+x;\n");
  }
  printf("printf(%c%cf%c,sum);\n",34,37,34);
  printf("}\n");
}
```

Here 34 and 37 are ASCII codes of " and % respectively.

When we execute this program by ./a.out>m.c (with input 5) then the contents of the file m.c are as follows. When file m.c is executed then it reads 5 numbers and outputs their sum.

```
#include<stdio.h>
main()
{
float sum=0,x;
printf("Give 5 numbers");
scanf("%f",&x);sum=sum+x;
scanf("%f",&x);sum=sum+x;
scanf("%f",&x);sum=sum+x;
scanf("%f",&x);sum=sum+x;
scanf("%f",&x);sum=sum+x;
printf("%f",sum);
}
```

1. Write program that reads n and outputs a program. The outputted program reads n numbers and prints the maximum of given numbers.
   [Hint: The outputted program will have lines if (max<x) max=x;]
2. Write program that reads n and outputs a program. The outputted program reads n numbers and prints the sum of product of consecutive numbers.
   [Hint: The outputted program will have lines sum=sum+x*y; y=x;]
3. Write program that reads n and outputs a program. The outputted program reads n numbers and prints the value of following expression.
   $(x_1-(x_2-(x_3- \ldots\ldots\ldots \quad (x_{n-1}-x_n) \ldots)))$
   The outputted program for n=5 will look as follows.

```
#include<stdio.h>
main( )
{  float ans,x1,x2,x3,x4,x5;
   scanf("%f %f %f %f %f",&x1,&x2,&x3,&x4,&x5);
   ans=x4–x5;
   ans=x3–ans;
   ans=x2–ans;
   ans=x1–ans;
   printf("%f",ans);
}
```

4. Write a program, which reads n and outputs a program. The outputted program reads n numbers and prints them in reverse order. Array should not be used by any program.
5. Write a program, which reads an integer 'r' and outputs a program. The outputted program reads n and prints all possible ways in which 'n' distinct symbols (1,2, ...,n) can be put at 'r' places. Repetition of symbols is allowed. For n=5 and r=3 the output will be (1,1,1) (1,1,2) (1,1,3) (1,1,4) (1,1,5) (1,2,1) ….There are n^r ways.
6. Write program so that given n and k we can find n (non negative) numbers whose sum of squares is k. Let n=4 and k=113 then output will be (8,6,3,2), (10,2,3,0), …
7. In above problem make arrangement so that combinations are not repeated. i.e. among (8,6,3,2) and (6,8,2,3) only one is printed.

8. Write program so that given n we can find n distinct numbers ($x_1x_2\ldots, x_n$) such that all numbers are between 1 and n and difference between x_i and x_j is not equal to i–j (or j–i). It is solution of n-queen problem.
9. Write a program, which reads a number x and outputs a program. The outputted program reads a number y and outputs another program. This program reads a number z and outputs the value of x+y+z.
10. Write program, which reads n and outputs P_n. Program P_1 outputs toy. Program P_i outputs program P_{i-1}.

The following program outputs following sentences

ram is a "good" boy
The shirt is 100%dirty
a\n is 12

The ASCII codes of “, %,\ are 34, 37 and 92 respectively.

```
main()
{   printf("ram is %cgood%c boy",34,34);
    printf("the shirt is 100%cdirty",37);
    printf("a%cn is 12",92);
}
```

The following program reads ‘r’ and outputs 3 digit numbers in base ‘r’.

```
main()
{   int i,j,k,r;
    scanf("%d",&r);
    for (i=0;i<r;i++)
     for (j=0;j<r;j++)
       for (k=0;k<r;k++)
         printf("%d %d %d\n",i,j,k);
}
```

The following program outputs a program, which reads a number and outputs a square.

```
main()
{
  printf("main()\n");
  printf("{  int x,y;\n");
  printf("   printf(%cGive input%c);\n",34,34);
  printf("   scanf(%c%cd%c,&x);\n",34,37,34);
  printf("   y=x*x;\n");
  printf("   printf(%csquare is %cd%c,y);\n",34,37,34);
  printf("}\n");
}
```

The following is the outputted program

```
main()
{  int x,y;
   printf("Give input");
   scanf("%d",&x);
   y=x*x;
   printf("square is %d",y);
}
```

Following program reads a number t and outputs a program. The outputted program reads a number x and outputs the value of t+x.

```
main()
{  int t;
   scanf("%d",&t);
   printf("main()\n");
   printf("{  int x;\n");
   printf("   scanf(%c%cd%c,&x);\n",34,37,34);
   printf("   printf(%c%cd%c,x+%d);\n",34,37,34,t);
   printf("}\n");
}
```

For input 12 the output is as follows

```
main()
{  int x;
   scanf("%d",&x);
   printf("%d",x+12);
}
```

The following program reads n. It outputs a program. The program reads n numbers x_1, x_2, …, x_n and outputs the value of $x_1*x_n + x_2*x_{n-1} + x_3*x_{n-2} + \ldots + x_n*x_1$.

```
main()
{  int i,n;
   scanf("%d",&n);
   printf("main()\n");
   printf("{  int s");
   for (i=1;i<=n;i++) printf(",a%d",i);
   printf(";\n");
   printf("   scanf(%c",34);
   for (i=1;i<=n;i++) printf("%cd",37,i);
   printf("%c",34);
   for (i=1;i<=n;i++) printf(",&a%d",i);
   printf(");\n");
   printf("   s=a1*a%d",n);
   for (i=2;i<=n;i++) printf("+a%d*a%d",i,n-i+1);
   printf(";\n");
   printf("   printf(%c%cd%c,s);\n",34,37,34);
   printf("}\n");
}
```

For n=5 the output is as follows.

```
main()
{  int s,a1,a2,a3,a4,a5;
   scanf("%d%d%d%d%d",&a1,&a2,&a3,&a4,&a5);
   s=a1*a5+a2*a4+a3*a3+a4*a2+a5*a1;
   printf("%d",s);
}
```

Write a program, which reads a number p and outputs a program. The outputted program reads a number t and outputs another program. This program reads a number x and outputs the value of x+t+p.

```
main()
{  int p;
   printf("main()\n");
   scanf("%d",&p);
   printf("{  int t;\n");
   printf("   scanf(%c%cd%c,&t);",34,37,34);
   printf("   printf(%cmain()%cn%c);\n",34,92,34);
   printf("   printf(%c{  int x;%cn%c);\n",34,92,34);
   printf("   printf(%c   scanf(%cc%ccd%cc,&x);%cn%c,34,37,34);\n",
                                                 34,37,37,37,92,34);
   printf("   printf(%c   printf(%cc%ccd%cc,x+%cd+%d);%cn%c,
                           34,37,34,t);\n",34,37,37,37,37,p,92,34);
   printf("   printf(%c}%cn%c);\n",34,92,34);
   printf("}\n");
}
```

For input 57 the output is as follows

```
main()
{  int t;
   scanf("%d",&t);   printf("main()\n");
   printf("{  int x;\n");
   printf("   scanf(%c%cd%c,&x);\n",34,37,34);
   printf("   printf(%c%cd%c,x+%d+57);\n",34,37,34,t);
   printf("}\n");
}
```

For input 17 the output is as follows

```
main()
{  int x;
   scanf("%d",&x);
   printf("%d",x+17+57);
}
```

Miscellaneous

1. Write program, which reads numbers till five even numbers are given. The program outputs the sum of all numbers given.

2. Write program, which reads numbers in a loop till two consecutive even numbers are given. The program outputs how many numbers were given.

3. Write program, which reads n and n integers. The output of the program is TRUE or FALSE. The output is TRUE if 3 consecutive even numbers are present.

4. Write program, which reads a number and outputs the product of sum of numbers between consecutive digits. Input 3571 output (3+4+5)x(5+6+7)x(1+2+3+4+5+6+7) = 12*18*28 = 6048.

5. Write program, which reads a number and outputs the sum of x^y. Here x and y are consecutive digits. Input 81924 output $8^1 + 1^9 + 9^2 + 2^4$ = 8+1+81+16 = 106.

6. Write program, which reads a number and finds the special sum of digits. Here first digit is multiplied by 1, second digit by 2, third digit by 3 and so on. Then their sum is calculated. If number is 2686 then output is 1x2+2x6+3x8+4x6=62. Use only one loop

7. Write program, which reads n and n numbers. The program finds the sum of 1st, 3rd, 6th, 10th , 15th , 21st, 28th, … numbers. If n=16 and numbers are 2, 5, 1, 12, 6, 7, 16, 8, 10, 13, 3, 5, 17, 6, 4 and 10 then answer is 2+1+7+13+4=27. Use only one loop.

8. Write program, which reads numbers x and k and finds the special sum of product of digits. Here first digit is multiplied by k^{th} digit, second digit is multiplied by $(k+1)^{th}$ digit, 3rd digit by $(k+2)^{th}$ and so on. If x= 241567839 and k=4 then answer is 162 (since 2x5 + 4x6 + 1x7 + 5x8 + 6x3 + 7x9 = 162). Do not use array.

9. Write program, which reads a string of words. The program outputs the length of longest word. Input Ram Chandra Singh Dey output 7.

10. Write program, which outputs the longest word. For above case Chandra. Assume that there is only one word of longest length.

11. Write program, which reads a list and prints the word at maximum depth. Assume that there is only one word at maximum depth. Input ((Ram (Om) Kapil) Hari (Dipu (Hari))) output Kapil.

12. Write program, which reads coordinates of three points. The program finds the area of triangle using vector method. Find vectors corresponding to each edge. Then find vector triple product. Define functions for dot product and cross product of vectors.

13. Write program, which reads two sorted arrays and outputs all elements in sorted form.

14. Write program, which reads an array x (i.e. x_0 , x_1 , x_2 , ….. , x_{n-1}). The program reads an integer k and deletes x_k. [Hint: Shift left x_{k+1} , x_{k+2} , ….. , x_{n-1}].

15. Write program, which reads an integer k and an element y. The program inserts y as x_k. [Hint: Shift right x_k , x_{k+1} , ….. , $x_{n-1}$and in the kth place put y].

16. Write program, which reads n and n numbers and finds sum of all numbers after last even number. If n=10 and numbers are 17, 19, 18, 17, 21, 19, 17, 16, 19 and 18 then output is 19+18=37.

17. Write program, which reads n and n numbers and finds how many times the maximum and second maximum number takes place. For above input the answer is 1 and 3 because maximum number is 21, which occurs once and the second maximum number is 19, which occurs 3 times.

18. Write program, which reads x and k. The program finds sum of product of k consecutive digits in x. If x=231789 and k=3 then output is 7*8*9+1*7*8+3*1*7+2*3*1=504+56+21+6=587.

19. Write program, which reads a number and finds the digit after second even digit. e.g. if number is 341567248132 then output is 7, because second even digit is 6. Do not use more than one loop.

Overview Programs must be efficient

1. Write program, which reads a number. Program exchanges the digits in the following manner. Input 27139 output 27391. Input 482 output 824. Input 3875649 output 3875496.
2. Write program, which reads two numbers. They are area and length of a rectangle. The program output the breadth. Input 24 5 output 4.8.
3. Write program, which reads three numbers a, b and c. The program outputs the distance of line ax+by+c=0 from origin. [Hint: it is c/sqrt(a^2+b^2)]. Input 3 4 7 output 1.4.

4. Write program, which reads a number. If it is even then output its square. If it is odd then output its cube. Input 4 output 16. Input 5 output 125.
5. Write program, which reads two numbers. The program outputs the number, which has smaller last digit. Input 314 and 76852 output 76852. Because their last digits are 4 and 2 respectively. Input 37256 and 4562 output 4562. Assume that their last digits are not same.

6. Write program, which reads a number and finds its last even digit. Input 324679 output 6.
7. Write program, which reads a number and finds its maximum digit. Input 372346 output 7.
8. Write program to find minimum digit. In above case 2.

9. Write program, which outputs those numbers between 0 and 100 which are even but not a multiple of 7. e.g. 2 4 6 8 10 12 16 18 …..
10. Write program to print picture ABCDEABCDABCABA.

11. Write program, which reads n and n numbers. The program finds, the sum of even and odd numbers separately. Let n=7 and numbers are 4, 8, 3, 2, 5, 4, 6 then output 24 and 8.
12. Write program, which finds the sum of all numbers and multiplies it by the last number. In above case output is 192 (because sum of all numbers is 32 and 32*6=192).
13. Modify above program to multiply sum by first number. In above case 128 (because 32*4=128)

14. Define function to find product of first and last digit of a number. p(2413) output 6.
15. Define function to find product of first and second digit of a number. q(2413) output 8.
16. Using above function write program which reads two numbers. The program finds product of first and second digits for both numbers independently. The program outputs their sum. Input 65234 and 824 output 46 (because 6*5=30 and 8*2=16).
17. Define above functions. Using them write program which reads two numbers. The program finds product of first and last digit of first number. The program finds the product of first and second digit of second number. The program outputs their sum. Input 65234 and 824 output 40 (because 6*4=24 and 8*2=16).

18. Write program to output the value of: $x_1*x_2*x_3 + x_2*x_3*x_4 + x_3*x_4*x_5 + ...$
    If n=7 and numbers are 4,5,2,5,6,4,7 then output is 438.
    because 4*5*2 + 5*2*5 + 2*5*6 + 5*6*4+6*4*7 = 438
19. Write program to output the value of $x_1*x_k + x_2*x_{k-1} + x_3*x_{k-2} + ...$
    If n=7 and k=4 and numbers are 4,5,2,5,6,4,7 then output is 60 because 4*5 + 5*2 + 2*5 + 5*4
    Here if k=5 then output is 102 because 4*6+5*5+2*2+5*5+6*4
20. Write program, which reads two numbers x and y. The program outputs smallest multiple of x which is greater than y. Input 13 and 57 output 65. Do not use *. [Hint: use %]

21. Write program, which reads two numbers x and y. The program outputs biggest multiple of x which is smaller than y. Input 13 and 57 output 52. Do not use *. [Hint: use %]
22. Read a number (x) and find x!.

TEST

GG and EX

1. Write program, which reads n and n numbers. The program finds the weighted sum of these numbers. The weight of the number is the previous even number. Do not use array. Let n=10 and numbers are 3, 7, 2, 5, 4, 6, 1, 5, 3, 8 then output is 144 because 3x0+7x0+2x0+5x2+4x2+6x4+1x6+5x6+3x6+8x6=144. Since there is no even number before 1st, 2nd and 3rd number hence it is taken as 0.

2. Write program, which reads n numbers and store them in an array a_1, a_2, a_3, a_4, …, a_n. The program finds the number of even elements in the array. Let it is 'p'. Find the value of $(a_1+p)*(a_2+p)*(a_3+p)*\ldots*(a_n+p)$. Let n=7 and numbers are 4,5,2,7,8,3,1 then output is 739200 because (4+3)*(5+3)*(2+3)*(7+3)*(8+3)*(3+3)*(1+3)=739200. Here 3 is added because there are 3 even numbers 4, 2 and 8.

3. Write program, which reads a number. The program outputs sum of product of digits, which are at a distance of 2. Input 26179 output 53 because 1x9+6x7+2x1=53. Input 527683 output 121 because 6x3+7x8+2x6+5x7=121.

4. Write program, which reads a number. Let p be its first digit. The program finds p^{th} digit from last. Input 31673892 output 8. It is because the first digit is 3 and 3rd digit from last is 8.

MA and SI

1. Read a number. Find smallest factor of FLS. 'F' is first digit. 'L' is last digit and 'S' is second digit. Input 15390 output 3. Because FLS is 105. It smallest factor is 3.

2. Write program, which reads n and n numbers. The program finds the weighted sum of these numbers. The weight of the number is the previous even number. Do not use array. Let n=10 and numbers are 3, 7, 2, 5, 4, 6, 1, 5, 3, 8 then output is 144 because 3x0+7x0+2x0+5x2+4x2+6x4+1x6+5x6+3x6+8x6=144. Since there is no number before 1st, 2nd and 3rd number hence it is taken as 0.

3. Write program, which reads n numbers and store them in an array a_1, a_2, a_3, a_4, …, a_n. The program outputs all numbers after last even number. Assume that there is at least one even number. Let n=7 and numbers are 4,6,2,8,3,1,5 then output is 3,1,5.

4. Write program, which reads a number (x). Let n be its second last digit. The program finds its first digit. x is multiplied by that. The program again finds its first digit and multiplies it by that. This process is repeated n times. The final number is outputted. Let input x is 2738 then output is 54760. Because n=3. 2738*2=5476. 5476*5=27380. 27380*2=54760.

5. Write program, which reads n numbers and store them in an array a_1, a_2, a_3, a_4, …, a_n. Let $p=a_1$ $q=a_p$ $r=a_q$ $s=a_r$ and so on. This process is repeated till 0 is obtained. The program finds the

value of p+q+r+s+… Let n=9 and numbers are 5,8,0,9,2,3,2,6,3 then output is 27 because 5+2+8+6+3+0=27.

Here $a_1=5$ $a_2=8$ $a_3=0$ $a_4=9$ $a_5=2$ $a_6=3$ $a_7=2$ $a_8=6$ $a_9=3$.

$a_1=5$ $a_5=2$ $a_2=8$ $a_8=6$ $a_6=3$ $a_3=0$

Rough

What are outputs of following program?

```
scanf("%d",&x);
while(x<80)
  x=x+4;
printf("%d",x);

input x=75,90
```

```
scanf("%d",&x);
do{
  x=x+4;
}while(x<80);
printf("%d",x);
input x=75,90
```

```
x=21;
while (x%7!=0)
  x=x+4;
printf("%d",x);
```

```
x=21;
do{
  x=x+4;
}while(x%7!=0);
printf("%d",x);
```

What are outputs of following program? Write clean programs also.

```
b=5;c=6;
scanf("%d",&a);
if (a>50)
b=8;
c=9;
printf("%d %d",b,c);
input a=40,60
```

```
b=5;c=6;
scanf("%d",&a);
if (a>50)
{b=8;
c=9;}
printf("%d %d",b,c);
input a=40,60
```

```
b=5;c=6;
scanf("%d",&a);
if (a>50)
b=8;
else
c=9;
printf("%d %d",b,c);
input a=40,60
```

```
b=5;c=6;
scanf("%d",&a);
if (a>50)
{b=8;
else
c=9;}
printf("%d %d",b,c);
input a=40,60
```

Write clean program for followings. Also state the output.

```
if (x<22)
while(x<80)
if (x%2==1)
x=x+5;
else
x=x+15;
x=x+100;
printf("%d",x);

Input x=16,17,25
```

```
if (x%2==0)
if (x<10)
a=30;
else
a=40;
else
if (x%3==0)
a=50;
else
a=60;
printf("%d",a);

Input x=5,6,15,16
```

```
while(x<108)
{if(x>30)
x=x+11;
else
if(x<70){
x=x+2;
x=x+10;
}
printf("%d",x);
}
```

Let input x=5
do above problem after
removing else
removing {}

Initial value problem using single step methods

```
#include<stdio.h>
#include<math.h>
double f(double x,double y) { return x*x+y*y; }
double yp(double x,double y){ return f(x,y); }
double ypp(double x,double y){ return 2*x+2*y*yp(x,y); }
double y3p(double x,double y)
{   return 2+2*y*ypp(x,y)+2*yp(x,y)*yp(x,y); }
double euler(double a,double b,double h){ return b+f(a,b)*h; }
double eupc(double a,double b,double h)
{   double y;
    y=euler(a,b,h);
    return b+(f(a,b)+f(a+h,y))*h/2;
}
double mid(double a,double b,double h)
{   double y;
    y=euler(a,b,h/2);
    return b+f(a+h/2,y)*h;
}
double rk(double a,double b,double h)
{   double k1,k2,k3,k4,k;
    k1=h*f(a,b);
    k2=h*f(a+h/2,b+k1/2);
    k3=h*f(a+h/2,b+k2/2);
    k4=h*f(a+h,b+k3);
    k=(k1+2*k2+2*k3+k4)/6;
    return b+k;
}
double taylor(double a,double b,double h)
{   return b+yp(a,b)*h+ypp(a,b)*h*h/2+y3p(a,b)*h*h*h/6; }
main()
{   double x0,y0,x1,y1;
    printf("give initial values");scanf("%lf %lf",&x0,&y0);
    printf("give final x");scanf("%lf",&x1);
    y1=euler(x0,y0,x1-x0);printf("%lf\n",y1);
    y1=taylor(x0,y0,x1-x0);printf("%lf\n",y1);
}
```

Convergence

```
#include<stdio.h>
#include<math.h>
double f(double x,double y){ return x/y; }
double exact(double x,double x0,double y0)
{   return sqrt(x*x+y0*y0-x0*x0); }
double euler(double a,double b,double h){ return b+f(a,b)*h; }
main()
{   double x0,y0,h1,h2,e1,e2,convergence;
    printf("give initial values");scanf("%lf %lf",&x0,&y0);
    printf("give widths");scanf("%lf %lf",&h1,&h2);
    e1=euler(x0,y0,h1)-exact(x0+h1,x0,y0);if (e1<0) e1=-e1;
    e2=euler(x0,y0,h2)-exact(x0+h2,x0,y0);if (e2<0) e2=-e2;
    printf("%.15lf %.15lf\n",euler(x0,y0,h1),e1);
    printf("%.15lf %.15lf\n",euler(x0,y0,h2),e2);
    convergence=log(e1/e2)/log(h1/h2);
    printf("%lf\n",convergence);
}
```

Initial value problem using multi step methods

```
double f(double x,double y){ return x/y; }
double rk(double a,double b,double h)
{   double k1,k2,k3,k4,k;
    k1=h*f(a,b);
    k2=h*f(a+h/2,b+k1/2);
    k3=h*f(a+h/2,b+k2/2);
    k4=h*f(a+h,b+k3);
    k=(k1+2*k2+2*k3+k4)/6;
    return b+k;
}
double global(double x0,double y0,double xn,int n)
{   double h,x[100],y[100];int i;
    h=(xn-x0)/n;x[0]=x0;y[0]=y0;
    for (i=1;i<=n;i++)
    {   y[i]=rk(x[i-1],y[i-1],h);
        x[i]=x[i-1]+h;
    }
    return y[n];
}
double milne(double x0,double y0,double xn,int n)
{   double h,x[100],y[100];int i;
    h=(xn-x0)/n;x[0]=x0;y[0]=y0;
    for (i=1;i<=n;i++) x[i]=x[i-1]+h;
    y[1]=rk(x0,y0,h);y[2]=rk(x[1],y[1],h);y[3]=rk(x[2],y[2],h);
    printf("%.10lf %.10lf\n",x[1],y[1]);
    printf("%.10lf %.10lf\n",x[2],y[2]);
    printf("%.10lf %.10lf\n",x[3],y[3]);
    for (i=4;i<=n;i++)
    {   y[i]=y[i-4]+4*h/3*(2*f(x[i-3],y[i-3])-f(x[i-2],y[i-2])+
             2*f(x[i-1],y[i-1]));
        printf("%.10lf %.10lf\n",x[i],y[i]);
```

Correction

```
        y[i]=y[i-2]+h/3*(f(x[i-2],y[i-2])+4*f(x[i-1],y[i-1])+f(x[i],y[i]));
        printf("%.10lf %.10lf\n",x[i],y[i]);
        y[i]=y[i-2]+h/3*(f(x[i-2],y[i-2])+4*f(x[i-1],y[i-1])+f(x[i],y[i]));
        printf("%.10lf %.10lf\n",x[i],y[i]);
    }
    return y[n];
}
main()
{   double x0,y0,xn,yn;int n;
    printf("give initial values");scanf("%lf %lf",&x0,&y0);
    printf("Give final x ");scanf("%lf",&xn);
    printf("give no of parts");scanf("%d",&n);
    yn=milne(x0,y0,xn,n);
    printf("%lf\n",yn);
}
```

Convergence

```
main()
{   double x0,y0,xn,h1,h2,e1,e2,convergence;int n1,n2;
    printf("give initial values");scanf("%lf %lf",&x0,&y0);
    printf("Give final x ");scanf("%lf",&xn);
    printf("give no of parts");scanf("%d %d",&n1,&n2);
    e1=global(x0,y0,xn,n1)-exact(xn,x0,y0);if (e1<0) e1=-e1;
```

```
    e2=global(x0,y0,xn,n2)-exact(xn,x0,y0);if (e2<0) e2=-e2;
    printf("%.15lf %.15lf\n",global(x0,y0,xn,n1),e1);
    printf("%.15lf %.15lf\n",global(x0,y0,xn,n2),e2);
    h1=(xn-x0)/n1;h2=(xn-x0)/n2;
    convergence=log(e1/e2)/log(h1/h2);
    printf("The convergence = %lf\n",convergence);
}
```

Initial value problem using muti step methods (without array)

```
#include<stdio.h>
double f(double x,double y) { return x-y*y; }
double eu(double x0,double y0,double h)
{   return y0+f(x0,y0)*h;  }
double eupc(double x0,double y0,double h)
{   double y;int i,n=4;
    y=eu(x0,y0,h);
    for (i=1;i<=n;i++)
    {
      printf("%.10lf\n",y);
      y=y0+(f(x0,y0)+f(x0+h,y))*h/2;
    }
    return y;
}
double rk(double x0,double y0,double h)
{   double k1,k2,k3,k4,k;
    k1=h*f(x0,y0);
    k2=h*f(x0+h/2,y0+k1/2);
    k3=h*f(x0+h/2,y0+k2/2);
    k4=h*f(x0+h,y0+k3);
    k=(k1+2*k2+2*k3+k4)/6;
    return y0+k;
}
double global(double x0,double y0,double xn,int n)
{   double h,y1;int i;
    h=(xn-x0)/n;
    for(i=1;i<=n;i++)
    {  y0=eupc(x0,y0,h);
       x0=x0+h;
       printf("%.10lf %.10lf\n",x0,y0);
    }
    return y0;
}
double milne(double x0,double y0,double xn,int n)
{   double h,y1,y2,y3,y4,y,x1,x2,x3,x4;int i;
    h=(xn-x0)/n;
    y1=rk(x0,y0,h);x1=x0+h;
    y2=rk(x1,y1,h);x2=x1+h;
    y3=rk(x2,y2,h);x3=x2+h;
    printf("%.10lf %.10lf\n",x1,y1);
    printf("%.10lf %.10lf\n",x2,y2);
    printf("%.10lf %.10lf\n",x3,y3);
    for (i=4;i<=n;i++)
    {   y=y0+4*h/3*(2*f(x1,y1)-f(x2,y2)+2*f(x3,y3));x4=x3+h;
        printf("%.10lf %.10lf\n",x4,y);
```

Correction

```
        y=y2+h/3*(f(x2,y2)+4*f(x3,y3)+f(x4,y));
        printf("%.10lf %.10lf\n",x4,y);
        y=y2+h/3*(f(x2,y2)+4*f(x3,y3)+f(x4,y));
        printf("%.10lf %.10lf\n",x4,y);
        y0=y1;x0=x1;y1=y2;x1=x2;y2=y3;x2=x3;y3=y;x3=x3+h;
    }
    return y;
}
```

```
main()
{
    double x0,y0,xn; int n;
    printf("Give initial values "); scanf("%lf %lf",&x0,&y0);
    printf("Give final value "); scanf("%lf",&xn);
    printf("Give number of intervals "); scanf("%d",&n);
    printf("%lf",global(x0,y0,xn,n));
    printf("%lf",milne(x0,y0,xn,n));
}
```

Backward Euler

```
#include<stdio.h>
double f(double x,double y)
{   return -2*x*y*y; }
double g(double x0,double y0,double h,double y)
{   return y+2*h*(x0+h)*y*y-y0;  }
double gp(double x0,double y0,double h,double y)
{   return 1+2*h*(x0+h)*2*y;  }
double beu(double x0,double y0,double h)
{   int i;double y;y=y0;
    for (i=1;i<=5;i++)
    {    y=y0;
         printf("%.10lf\n",y);
         y=y-g(x0,y0,h,y)/gp(x0,y0,h,y);
    }
    return y;
}
double global(double x0,double y0,double xn,int n)
{
    double h,y1;
    int i;
    h=(xn-x0)/n;
    for(i=1;i<=n;i++)
    {
       y0=beu(x0,y0,h);
       x0=x0+h;
       printf("%.10lf %.10lf\n",x0,y0);
    }
    return y0;
}
main()
{
    double x0,y0,xn;
    int n;
    printf("Give initial values ");
    scanf("%lf %lf",&x0,&y0);
    printf("Give final value ");
    scanf("%lf",&xn);
    printf("Give number of intervals ");
    scanf("%d",&n);
    printf("%lf",global(x0,y0,xn-x0,n));
}
```

Second order equation using Taylor series method

```
#include<stdio.h>
#include<math.h>
double ypp(double x,double y,double yp){ return 2*y*yp; }
double y3p(double x,double y,double yp,double ypp)
{   return 2*y*ypp+2*yp*yp; }
double y4p(double x,double y,double yp,double ypp,double y3p)
{   return 2*y*y3p+6*yp*ypp; }
double taylor(double x0,double y0,double yp,double h)
{   double s,t;
    s=ypp(x0,y0,yp);t=y3p(x0,y0,yp,s);
    return y0+h*yp+h*h/2*s+h*h*h/6*t;
}
double global(double x0,double y0,double xn,double yp,int n)
{   double h,x[100],y[100];int i;
    h=(xn-x0)/n;x[0]=x0;y[0]=y0;
    for (i=1;i<=n;i++)
    {   y[i]=taylor(x[i-1],y[i-1],yp,h);
        yp=yp+h*ypp(x[i-1],y[i-1],yp)+
               h*h/2*y3p(x[i-1],y[i-1],yp,ypp(x[i-1],y[i-1],yp));
        x[i]=x[i-1]+h;
        printf("%lf %lf %lf\n",x[i],y[i],yp);
    }
    return y[n];
}
main()
{   double x0,y0,xn,yn,yp;int n;
    printf("give initial values");
    scanf("%lf %lf %lf",&x0,&y0,&yp);
    printf("give final x");scanf("%lf",&xn);
    printf("give number of parts");scanf("%d",&n);
    yn=global(x0,y0,xn,yp,n);
    printf("%lf\n",yn);
}
```

Second order equation using vector method

```
#include<stdio.h>
typedef struct{double i,j;}vector;
vector mul(double t,vector v)
{   vector w;
    w.i=t*v.i;
    w.j=t*v.j;
    return w;
}
vector add(vector v,vector w)
{   vector m;
    m.i=v.i+w.i;
    m.j=v.j+w.j;
    return m;
}
void pt(vector v)
{   printf("%.8lfi+%.8lfj\n",v.i,v.j);  }
vector f(double x,vector z)
{   vector w;
    w.i=-3*z.i+2*z.j;
    w.j=3*z.i-4*z.j;
    return w;
}

/*vector f(double x,vector z)
{   vector w;
    w.i=z.j;
    w.j=x*(z.j)*(z.j)-(z.i)*(z.i);
    return w;
}*/

vector euler(double a,vector b,double h)
{   return add(b,mul(h,f(a,b)));  }
vector rk(double a,vector b,double h)
{   vector k1,k2,k3,k4,k;
    k1=mul(h,f(a,b));
    k2=mul(h,f(a+h/2,add(b,mul(1.0/2,k1))));
    k3=mul(h,f(a+h/2,add(b,mul(1.0/2,k2))));
    k4=mul(h,f(a+h,add(b,k3)));
    k=mul(1.0/6,add(k1,add(mul(2,k2),add(mul(2,k3),k4))));
    pt(k1);pt(k2);pt(k3);pt(k4);
    return add(b,k);
}
main()
{   double x0,x1;vector y0,y1;
    printf("give initial values");
    scanf("%lf %lf %lf",&x0,&y0.i,&y0.j);
    printf("Give final x");scanf("%lf",&x1);
    y1=rk(x0,y0,x1-x0);
    pt(y1);
}
```

Given

$u' = -3u+2v$

$v' = 3u-4v$

$u(0)=0 \quad v(0)=0.5$

find $u(0.2)$

Given

$y'' = xy'^2 - y^2$

$y(a)=b \quad y'(a)=b$

find $y(a+h) \quad y'(a+h)$

Boundary value problem using secent method

```
#include<stdio.h>
#include<math.h>
double ypp(double x,double y,double yp){ return 2*y*yp; }
double y3p(double x,double y,double yp,double ypp)
{   return 2*y*ypp+2*yp*yp; }
double y4p(double x,double y,double yp,double ypp,double y3p)
{   return 2*y*y3p+6*yp*ypp; }
double taylor(double x0,double y0,double yp,double h)
{   double s,t;
    s=ypp(x0,y0,yp);t=y3p(x0,y0,yp,s);
    return y0+h*yp+h*h/2*s+h*h*h/6*t;
}
double global(double x0,double y0,double xn,double yp,int n)
{   double h,x[100],y[100];int i;
    h=(xn-x0)/n;x[0]=x0;y[0]=y0;
    for (i=1;i<=n;i++)
    {   y[i]=taylor(x[i-1],y[i-1],yp,h);
        yp=yp+h*ypp(x[i-1],y[i-1],yp)+
               h*h/2*y3p(x[i-1],y[i-1],yp,ypp(x[i-1],y[i-1],yp));
        x[i]=x[i-1]+h;
        printf("%lf %lf %lf\n",x[i],y[i],yp);
    }
    return y[n];
}
void secent(double x0,double y0,double xn,double yn,int n,
                                    double yp1,double yp2, int m)
{   double yn1,yn2,e1,e2,t;int i;
    for (i=1;i<=m;i++)
    {   printf("[%lf %lf %lf %lf]\n",e1,e2,yp1,yp2);
        yn2=global(x0,y0,xn,yp2,n);e2=yn-yn2;
        yn1=global(x0,y0,xn,yp1,n);e1=yn-yn1;
        t=yp1-e1*(yp2-yp1)/(e2-e1);
        yp1=yp2;yp2=t;
    }
}
main()
{   double x0,y0,xn,yn,yp1,yp2;int n,m;
    printf("give initial values");scanf("%lf %lf",&x0,&y0);
    printf("give final values");scanf("%lf %lf",&xn,&yn);
    printf("give two guesses for derivative");
    scanf("%lf %lf",&yp1,&yp2);
    printf("give number of parts");scanf("%d",&n);
    printf("give number of iterations");scanf("%d",&m);
    secent(x0,y0,xn,yn,n,yp1,yp2,m);
}
```

Linked List (singly: for parallel processing)

```
main()
{   int data[20],next[20],t[20],i,j,size,head,x;
    printf("How many nodes");scanf("%d",&size);
    printf("Give location of head node");scanf("%d",&head);
    printf("Give location array");
    for (i=1;i<=size;i++)scanf("%d",&j);
    printf("Give data array");
    for (i=1;i<=size;i++)scanf("%d",&data[i]);
    printf("Give next array");
    for (i=1;i<=size;i++)scanf("%d",&next[i]);

    printf("\nThe given list is :");
    i=head; while (i!=-1){ printf("%d,",data[i]);i=next[i];}
    printf("\n");

        for (i=1;i<=size;i++) t[i]=data[i];
        for (i=1;i<=size;i++)
              if ((next[i]!=-1)&&(t[next[i]]%2==0)) data[i]++;

    printf("\nThe final list is :");
    i=head; while (i!=-1){ printf("%d,",data[i]);i=next[i];}
}
```

The above program increments elements before all even elements. Following data structure defines sequence **69 49 44 46 45 67 29 28 39**. After the execution of the above program it will become 69 50 45 46 45 67 30 28 39

```
Size=16    Head=11
Location  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16
Data     46 44 45 39 28 21 67 49 29 41 69 49 39 29 28 46
Next      3  1  7 -1  4  4  9  2  5 12  8 -1  2  1  3 11
```

Location	11	8	2	1	3	7	9	5	4
Sequence	69	49	44	46	45	67	29	28	39
Previous	46(16)		39(13)	29(14)	28(15)				21(6)

The following program segment deletes the next element of a given element (X). For input X=29 the link list will become 69 49 44 46 45 67 29 39.

```
    printf("\nGive the element whose next element is to be deleted");
    scanf("%d",&x);
    for (i=0;i<=size;i++) if (data[i]==x) next[i]=next[next[i]];
```

For the doubly link list the data structure becomes

```
Size=16    Head=11
Location  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16
Data     46 44 45 39 28 21 67 49 29 41 69 49 39 29 28 46
Next      3  1  7 -1  4  4  9  2  5 12  8 -1  2  1  3 11
Prev      2  8  1  5  9  8  3 11  7  3 -1  2  -1 3 16  9
```

The link list should be printed as

```
    printf("\nThe Forward list is :");
    i=head; while (next[i]!=-1){ printf("%d,",data[i]);i=next[i];}
    printf("%d \nThe backward list is :",data[i]);
    while (i!=-1){ printf("%d,",data[i]);i=prev[i];}
```

```
printf("\n");
```

Write programs for followings by modifying the contents of the box only.

(A) In these lines do not use while loop unless permitted.
(B) Do not make copy of data or next array into ‘t’ unless permitted.
(C) Few of the following problems can not be done. Identify those problems. Observe the output when they are done in normal manner.

The link list for following problems is 69 49 44 46 45 67 29 28 39

1. Output last element. Output 39 *!
2. Double the last element. 69 49 44 46 45 67 29 28 78
3. Program, which adds 1 (increments) to all even element. 69 49 45 47 45 67 29 29 39
4. Increment all elements after even elements. 69 49 44 47 46 67 29 28 40 *
5. Add 2 to all numbers before even number 69 51 46 46 45 67 31 28 39 (copy not needed). However copy is needed when we add 1 to numbers before even numbers.
6. Delete last element. (copy permitted) 69 49 44 46 45 67 29 28
7. Delete second last element (copy permitted) 69 49 44 46 45 67 29 39
8. Add second last element to the last element. 69 49 44 46 45 67 29 28 67 *
9. Add last element to the second last element. 69 49 44 46 45 67 29 67 39
10. Delete elements at even position (copy permitted) 69 44 45 29 39
11. Delete elements at odd position (copy permitted) 49 46 67 28
12. Insert a given element (X) at last. Input x=57 the list 69 49 44 46 45 67 29 28 39 57! [Hint: to insert an element in the beginning data[size]=x next[size]=head head=size]
13. Insert an element before last element. Input x=57 list 69 49 44 46 45 67 29 28 57 39 *!
14. Delete all elements on or after first even element. 69 49
15. Delete all elements after first even element. 69 49 44
16. Delete all elements after next element after first even element. 69 49 44 46 *
17. Read a number (X) and delete its next element. x=46 list 69 49 44 46 67 29 28 39
18. Read a number (X) and delete one element after next element. For x=67 there is no problem. The list is 69 49 44 46 45 67 29 39. The element 28 is deleted. However for x=46 output is not 69 49 44 46 45 29 28 39. One more element (49) is deleted. *
19. Delete all elements which are multiple of 3. Assume that they are not consecutive. 49 44 46 67 29 28

For following problems while loop of size log(n) is permitted. Copy is also permitted.

20. Find sum of all elements 416 (copy not needed)
21. Find last element. 39 (copy not needed)
22. Find second last element. 28 (copy not needed)
23. Find k^{th} element. For k=7 output 29.
24. Add second last element to the last element. 69 49 44 46 45 67 29 28 67
25. Print all elements of the link list in any order.
26. Given an element (X) find its location. For X=44 output is 3.
27. Find sum of first k elements. For k=5 output 320.

Linked List

(singly: array based)

```
#include<stdio.h>


main()
{  int data[20],next[20],head=-1,size=0,i,j,choice,loc,y;
   do
   {
      printf("\nGive choice:  -1:find third element -2:put at last");
      printf(" -3:put at begining -4:delete -5:print -6:exit    ");
      scanf("%d",&choice);
      if(choice==-1) printf("%d",data[next[next[head]]]);
      if(choice==-2)
      {  printf("Give the element to be put at last "); scanf("%d",&y);
         size++;data[size]=y;next[size]=-1;
         if (head==-1) head=size;
         else {i=head;while (next[i]!=-1) i=next[i]; next[i]=size; }
      }
      if(choice==-3)
      {  printf("Give the element to be put at the begining ");
         scanf("%d",&y);
         size++;data[size]=y;next[size]=head;head=size;}
      if(choice==-4)
      {  printf("Give location of the element to be deleted");
         scanf("%d",&loc);
         if(loc==1) head=next[head];
         else
         {  i=head;for(j=2;j<loc;j++) i=next[i];
            next[i]=next[next[i]];
         }
      }
      if(choice==-5)
      {  printf("The sequence is: ");
         i=head;while(i!=-1) {printf("%4d ",data[i]);i=next[i];}
         printf("\nhead=%d",head);
         printf("\nIndex");for (i=1;i<=size;i++) printf("%3d",i);
         printf("\nData ");for (i=1;i<=size;i++) printf("%3d",data[i]);
         printf("\nNext ");for (i=1;i<=size;i++) printf("%3d",next[i]);
      }
   }while(choice!=-6);
}
```

Give following input

-2 34 -2 56 -3 97 -3 27 -2 64 -2 87 -3 45 -2 34 -3 67 -2 88 -3 39 -4 1 -5 -6

The output The sequence is 67 45 27 97 34 56 64 87 34 88

```
head=9
Index  1  2  3  4  5  6  7  8  9 10 11
Data  34 56 97 27 64 87 45 34 67 88 39
Next   2  5  1  3  6  8  4 10  7 -1  9
```

Let sequence be 67 45 27 97 34 56 64 87 34 88

28. Write program to print the second element. In above case 45.
29. Write program to find the last element of the linked list. Answer 88.
30. Write program to find the second last element of the linked list. Answer 34.

31. Write program, which reads k also and finds k^{th} element of the linked list. For k=3, 27
32. Write program to find sum of all elements. Answer 599
33. Write program to find the maximum element. Answer 97
34. Write program to print all even elements. Answer 34 56 64 34
35. Write program to find the first even element. Answer 34
36. Write program to print the maximum even element. Answer 64
37. Read an element and find its location in linked list. Input 56 output 6.
38. Write program to find local maximum elements. [Definition: An element is called local maximum if it is bigger than both of its neighbors.] In above example 97 and 87.

<u>No printing should be done in followings. Correctness should be checked by (-5).</u>

39. Write a program to delete the first element. 45 27 97 34 56 64 87 34 88
40. Write program to delete the second element. 67 27 97 34 56 64 87 34 88
41. Write program to delete the last element. 67 45 27 97 34 56 64 87 34
42. Write program to delete the second last element. 67 45 27 97 34 56 64 87 88
43. Write program which will delete first k elements. For k=3 The link list is 97 34 56 64 87 34 88 Use only one loop. For k=7 the link list is 87 34 88
44. Write program to delete all elements after k^{th} element. For k=3 the list is 67 45 27.
45. Write program which will delete the last k elements. For k=3 the list is 67 45 27 97 34 56 64. Two loops may be used but they should not be nested.
46. Write program which reads an element and puts it at second place in the link list. If new element is 29 then the link list after insertion is 67 29 45 27 . . . . .
47. Write program to put the element at second last position. 34 29 88
48. Read two elements x and y. y is already in the link list. Put x before y. Input 39 56 print outputs 67 45 27 97 34 39 56 64 87 34 88.

<u>In following programs, exchange of data field should not be done.</u>

49. Write program to exchange the second and third element. 67 27 45 97 34 . . . .
50. Program to exchange second and second last element. 67 34 27 97 34 56 64 87 45 88
51. Program to exchange first and second element. 45 67 27 97 34 56 64 87 34 88
52. Write program, which reads two numbers p and q and exchanges p^{th} and q^{th} element. Let p=5 and q=8 then sequence is 67 45 27 97 87 56 64 34 34 88

53. Insert an element before the first even element. It is double of that. 67 45 27 97 68 34 … (one loop)
54. Read two locations. Write program to delete all elements between them. If locations are 2 and 5 then final link list is 67 56 64 87 34 88
55. Write program to find the element before maximum element. Answer 27. (one loop)
56. Write program, which inserts an element between every pair of consecutive elements. The new element is sum of neighbor. 67 112 45 72 27 …….. 87 121 34 122 88
57. Delete k^{th} element from last. Let k=3 then 67 45 27 97 34 56 64 34 88
58. Write program which will delete 1st, 3rd, 5th, … elements. 45 97 56 87 88
59. Write program which will delete 2st, 4th, 6th, … elements. 67 27 34 64 34
60. Write program to exchange consecutive elements. 45 67 97 27 56 34 87 64 88 34
61. Write program to delete all even elements. 67 45 27 97 87 88

62. Write program to reverse the link list.

Linked List (multi linked list : array based)

```
main()
{  int data[20],next[20],head1=-1,head2=-1,size=0,i,j,choice,y;
   do
   {  printf("\nGive choice:  -1:put at the begining of first list ");
      printf(" -2:put at the begining of second list -3:print -4:exit");
      scanf("%d",&choice);
      if(choice==-1)
      {  printf("Give the element to be put at the begining ");
         scanf("%d",&y);
         size++;data[size]=y;next[size]=head1;head1=size;}
      if(choice==-2)
      {  printf("Give the element to be put at the begining ");
         scanf("%d",&y);
         size++;data[size]=y;next[size]=head2;head2=size;}
      if(choice==-3)
      {  printf("\nThe first sequence is: ");
         i=head1;while(i!=-1) {printf("%4d ",data[i]);i=next[i];}
         printf("\nThe second sequence is: ");
         i=head2;while(i!=-1) {printf("%4d ",data[i]);i=next[i];}
         printf("\nhead1=%d  head2=%d",head1,head2);
         printf("\nIndex");for (i=1;i<=size;i++) printf("%3d",i);
         printf("\nData ");for (i=1;i<=size;i++) printf("%3d",data[i]);
         printf("\nNext ");for (i=1;i<=size;i++) printf("%3d",next[i]);}
   }while(choice!=-4);
}
```

Give input `-2 34 -2 56 -1 55 -2 67 -2 88 -1 33 -1 64 -1 83 -2 85 -3 -4`

The output: The first list is: 83 64 33 55 The second list is: 85 88 67 56 34

```
head1=8  head2=9
Index  1  2  3  4  5  6  7  8  9
Data  34 56 55 67 88 33 64 83 85
Next  -1  1 -1  2  4  3  6  7  5
```

<u>No printing should be done in followings. Correctness should be checked by (-3).</u>

1. Write program to exchange first elements of the lists. The lists will become 85 64 33 55 and 83 88 67 56 34 respectively.
2. Write program to exchange second elements of the lists. The lists will become 83 88 33 55 and 85 64 67 56 34 respectively.
3. Write program to exchange their last elements.
4. Put second list after first. The first list will become 83 64 33 55 85 88 67 56 34.
5. Put second list after first element of the first list. The first list will become 83 85 88 67 56 34 64 33 55.
6. Delete last element of the first list and put it in the beginning of the second list. The lists will become 83 64 33 and 55 85 88 67 56 34 respectively.
7. Delete second last element of the first list and put it in the beginning of the second list.
8. Delete first element of the first list and put it in the beginning of the second list.
9. Delete first element of the first list and put it after first element in the second list.
10. Delete last element of the first list and put it at the end of the second list.
11. Put last element of the first list at the end of the second list. The first list remains unchanged. The lists shall be 83 64 33 55 and 85 88 67 56 34 55 respectively.

12. Output elements of both lists alternatively. Assume that second list has more elements. Excess elements of the second list are ignored. In above example output is 83 85 64 88 33 67 55 56. (printing permitted)
13. In above question excess elements of the second list are outputted at the end.
14. Shuffle lists. The first list will become 83 85 64 88 33 67 55 56. The access elements are ignored.
15. The first list contains elements at even position. The second list contains the elements at odd position. The lists will become 64 55 88 56 and 83 33 85 67 34 respectively.
16. The first list contains even element and the second list contains odd elements. The lists will become 64 88 56 34 and 83 33 55 85 67 respectively.
17. Assume they are sorted lists. Print sorted system. Let lists are 12 34 56 78 and 39 42 57 58 62 then output is 12 34 39 42 56 57 58 62.
18. In above case form a sorted list. The first list will become 12 34 39 42 56 57 58 62.

Linked List (singly function & pointer based)

In a file named “sf” type the following program.

```
typedef struct  node { int data; struct node *next; } node;
node* construct()
{   node *i,*j,*head,*t; int b,g,n;

    printf("Howmany elements the link list has "); scanf("%d",&n);
    head=(node *)malloc(sizeof(node));
    i=head;
    for (g=1;g<n;g++)
    {   printf("Give  %dth element ",g);
        scanf("%d",&(i->data));
        i->next=(node *)malloc(sizeof(node));
        i=i->next;
    }
    printf("Give the last element ");
    scanf("%d",&(i->data)); i->next=NULL;
    return head;
}
void print(node* head)
{   node *i;
    printf("\nThe linked list is ");
    i=head;
    while (i!=NULL)
    {   printf("%d, ",i->data);
        i=i->next;
    }
}
int second(node* head) { return head->next->data; }
node* third(node* head) { return head->next->next; }
node* delete_first(node* head)
{   node *i,*j;
    i=head->next;return i;
}
node* delete_second(node* head)
{   node *i,*j,*k;
    i=head;j=i->next;k=j->next;
    i->next=k;
    return head;
}
node* insert_second(node* head, int g)
{   node *i,*j,*k,*t;
    i=head;j=i->next;
    t=(node *)malloc(sizeof(node));
    t->data=g;i->next=t;t->next=j;
    return head;
}
```

In a file “t.c” type the following program. Compile “t.c” and execute.

```
#include<stdio.h>
#include "sf"
main()
{  node *head,*i;int t;
   head=construct();
   print(head);
   i=third(head);printf("\nThe third element is %d",i->data);
   t=second(head);printf("\nThe second element is %d",t);
```

```
   head=delete_first(head);print(head);
   head=delete_second(head);print(head);
   printf("Give an element to insert");scanf("%d",&t);
   head=insert_second(head,t);print(head);
}
```

Let a link list has 8 elements and the list is 17, 15, 42, 56, 78, 89, 23, 67.

1. Define function int first(node* head) to return the first element. For above list the output will be 17.
2. int product(node* head) to return the product of first two elements. Output 255.
3. Define function node* delete_third(node* head) to delete the third element. The list becomes 17, 15, 56, 78, 89, 23, 67.
4. node* del_three(node* head) to delete first three elements. 56, 78, 89, 23, 67.
5. node* del_after(node* head) to delete all elements after third element. 17, 15, 42.
6. node* exchange12(node* head) to exchange the first two elements. The list becomes 15, 17, 42, 56, 78, 89, 23, 67.
7. node* exchange23(node* head) The list becomes 17, 42, 15, 56, 78, 89, 23, 67.
8. node* exchange13(node* head) The list becomes 42, 15, 17, 56, 78, 89, 23, 67.
9. node* exchange24(node* head) The list becomes 17, 56, 42, 15, 78, 89, 23, 67.
10. node* insert_first(node* head, int g) to insert g as the first element. Let input g is 32 then list becomes 32,17, 15, 42, 56, 78, 89, 23, 67.
11. node* insert_third(node* head, int g) to insert g as the third element. 17, 15, 32, 42, 56, 78, 89, 23, 67.

12. int kth(node* head, int k) to return the kth element. For k=4 the output is 56.
13. node* del_after_k(node* head, int k) to delete all elements after k^{th} element. For k=4 the list becomes 17, 15, 42, 56.
14. node* del_k(node* head, int k) to delete first k elements. For k=4 list is 78, 89, 23, 67.
15. void printk(node* head, int k) to print first k elements. For k=3 the output 17, 15, 42.
16. int sum_k(node* head, int k) to return the sum of first k elements. For k=4 sum 130.
17. node* delete_kth(node* head, int k) to delete the kth element. For k=4 the list will become 17, 15, 42, 78, 89, 23, 67.
18. node* insert_kth(node* head, int g, int k) to insert g as the kth element. For g=32 and k=4 the list will become 17, 15, 42, 32, 56, 78, 89, 23, 67

19. int last(node* head) to return the last element. For above list 67.
20. node* laast(node* head) to return the pointer of the last element.
21. node* second_last(node* head) to return the pointer of the second last element.
22. int first_even(node* head) to return the first even element. For above list 42.
23. node* delete_last(node* head) to delete the last element. The list will become 17, 15, 42, 56, 78, 89, 23.
24. node* insert_last(node* head, int g) to insert g as the last element. For g=32 the list will become 17, 15, 42, 56, 78, 89, 23, 67, 32.
25. node* insert_second_last(node* head, int g) to insert g as the second last element. For g=32 the list will become 17, 15, 42, 56, 78, 89, 23, 32, 67.

26. Define function node* join(node *head1, node* head2) to join the two link lists. Let link lists be 12, 34, 56, 98, 23 and 45, 67, 89 then after the execution of following program a list 12, 34, 56, 98, 23, 45, 67, 89 will be formed.

```
main()
{  node *head1, *head2, *head;
   head1=construct(); head2=construct();
```

```
        head=join(head1,head2);print(head);
    }
```

27. node* joinn(node *head1, node* head2). Here second list is put after first element in the first list. The list 12, 45, 67, 89, 34, 56, 98, 23 will be formed.
28. node* shuffle(node *head1, node* head2). Here alternate elements both lists are joined. Assume that both lists have same size. If lists are 12, 34, 56 and 18, 13, 29 then a list 12, 18, 34, 13, 56, 29 is formed.

Linked List (doubly function & pointer based)

```
typedef struct  node { int data; struct node *next,*prev; } node;
node* construct()
{   node *i,*j,*head,*t; int b,g,n;

    printf("Howmany elements the link list has "); scanf("%d",&n);
    head=(node *)malloc(sizeof(node));
    i=head;(*head).prev=NULL;
    for (g=1;g<n;g++)
    {   printf("Give  %dth element ",g);
        scanf("%d",&(i->data));
        i->next=(node *)malloc(sizeof(node));
        i->next->prev=i;i=i->next;
    }
    printf("Give the last element ");
    scanf("%d",&(i->data));
    i->next=NULL;
    return head;
}
void print(node* head)
{   node *i,*j;
    printf("\nThe linked list is ");
    i=head;
    while (i!=NULL)
    {   printf("%d, ",i->data);
        j=i;i=i->next;
    }
    printf("\nThe reverse linked list is ");
    while (j!=NULL)
    {   printf("%d, ",j->data);
        j=j->prev;
    }
}
node* third(node* head) { return head->next->next; }
int first_even(node* head)
{   node *i=head;
    while (i->data%2!=0) i=i->next;
    return (i->data);
}
node* delete_first(node* head)
{   node *i,*j;
    i=head->next;i->prev=NULL; return i;
}
node* delete_second(node* head)
{   node *i,*j,*k;
    i=head;j=i->next;k=j->next;
    i->next=k;k->prev=i;
    return head;
}
node* insert_second(node* head, int g)
{   node *i,*j,*k,*t;
    i=head;j=i->next;
    t=(node *)malloc(sizeof(node));
    t->data=g;
    i->next=t;t->prev=i;
    t->next=j;j->prev=t;
```

```
    return head;
}
```

Linked List (singly pointer based)

```
typedef struct node {float data;struct node *next; }node;
main()
{  node *i,*t,*head; int j,choice,loc;float y;
   head=NULL;
   do
   {  printf("Give choice:  -1:find third element -2:put at last”);
      printf(“ -3:delete -4:print -5:exit    ");scanf("%d",&choice);
      if(choice==-1) printf("%f",head->next->next->data);
      if(choice==-2)
      {  printf("Give the element to be put at last "); scanf("%f",&y);
         t=(node *)malloc(sizeof(node));
         t->data=y;t->next=NULL;
         if (head==NULL) head=t;
         else {i=head;while (i->next!=NULL) i=i->next; i->next=t; }
      }
      if(choice==-3)
      {  printf("Give location of the element to be deleted");
         scanf("%d",&loc);
         if(loc==1) head=head->next;
         else
         {  i=head;for(j=2;j<loc;j++) i=i->next;
            i->next=i->next->next;
         }
      }
      if(choice==-4)
      {  i=head;
         while(i!=NULL) {printf("%4.1f ",i->data);i=i->next;}
         printf("\n");
      }
   }while(choice!=-5);
}
```

Let a linked list has following elements 17 12 34 26 78 23 75.

63. Write a program to print the second element. In above case 12.
64. Write a program to find last element of the linked list. Answer 75.
65. Write a program, which reads k also and finds k^{th} element of the linked list. For k=3, 34
66. Write a program to find sum of all elements.
67. Write a program to find the maximum element. Answer 78
68. Write a program to find the first even element. Answer 12
69. Read an element and find its location in linked list. Input 23 output 6.
70. Write program to output the number of elements in the linklist. In above case 6.

<u>No printing should be done in followings. Correctness should be checked by (-4).</u>

71. Write a program to delete the first element. 12 34 26 78 23 75.
72. Write program to delete the second element. 17 34 26 78 23 75.
73. Write program to delete the last element. 17 12 34 26 78 23.
74. Write program to delete the second last element. 17 12 34 26 78 75.
75. Write program which will delete first k elements. For k=3 The link list is 26 78 23 75. Use only one loop.
76. Write program which will delete the last k elements. For k=3 the list is 17 12 34 26. Two loops may be used but they should not be nested.

77. Read two elements x and y. y is already in the link list. Put x before y. Input 26 88 print outputs 17 12 34 88 26 78 23 75.
78. Write program which reads an element and puts it at second place in the link list. If new element is 29 then the link list after insertion is 17 29 12 34 26 78 23 75.
79. To put the element in the beginning. In above case 29 17 12 34 26 78 23 75.
80. Write program to put the element at last position. 17 12 34 26 78 23 75 29.

<u>In following programs, exchange of data field should not be done.</u>

81. Write program to exchange the second and third element. 12 17 34 26 78 23 75.
82. Write program to exchange second and second last element. 17 23 34 26 78 12 75.
83. Write program to exchange first and second element. 12 17 34 26 78 23 75.

Linked List (doubly pointer based)

```
#include<stdio.h>
typedef struct  node
              { float data;
                struct node *next,*prev;
              } node;
main()
{   node *i,*j,*head,*t,*hd;
    int g,n,k; float b;

    printf("Howmany elements the link list has ");
    scanf("%d",&n);
    head=(node *)malloc(sizeof(node));
    i=head;(*head).prev=NULL;
    for (g=1;g<n;g++)
    {   printf("Give  %dth element ",g);
        scanf("%f",&(*i).data);
        (*i).next=(node *)malloc(sizeof(node));
        (*(*i).next).prev=i;i=(*i).next;
    }
    printf("Give last element ");
    scanf("%f",&(*i).data);
    (*i).next=NULL;

      // Delete second element
      i=head->next;j=i->next;
      head->next=j;j->prev=head;

    printf("\nThe linked list is ");
    i=head;
    while (i!=NULL)
    {   printf("%f ",(*i).data);
        j=i;i=(*i).next;
    }
    printf("\nThe reverse linked list is ");
    while (j!=NULL)
    {   printf("%f ",(*j).data);
        j=(*j).prev;
    }
}
```

The above program deletes the 2nd element. Let a link list has following elements 17 12 34 26 78 23 75.After deleting 2nd element it will become 17 34 26 78 23 75. The reverse printing will output 75 23 78 26 34 17

To get the third last element, the contents of the box should be replaced by

```
    i=head;
    while (i->next!=NULL) i=i->next;
    printf("The third last element %f",i->prev->prev->data);
```

Let a link list has following elements 17 12 34 26 78 23 75.

<u>No printing should be done in followings. Carefully check the reverse printing also.</u>

1. Program to delete the first element. print outputs 12 34 26 78 23 75, 75 23 78 26 34 12
2. Exchange first two elements. print outputs 12 17 34 26 78 23 75, 75 23 78 26 34 17 12
3. Exchange 2nd and 3rd element. print 17 34 12 26 78 23 75, 75 23 78 26 12 34 17
4. Delete last element. 17 12 34 26 78 23 , 23 78 26 34 12 17
5. Delete first even element.17 34 26 78 23 75, 75 23 78 26 34 17
6. Exchange 1st and last elements. print 75 12 34 26 78 23 17, 17 23 78 26 34 12 75
7. Perform rotate left operation. Print 12 34 26 78 23 75 17, 17 75 23 78 26 34 12
8. Perform rotate right operation. Print 75 17 12 34 26 78 23, 23 78 26 34 12 17 75
9. Delete a given element. Input 34 output 17 12 26 78 23 75, 75 23 78 26 12 17
10. Delete kth element from last. k=6 output 17 34 26 78 23 75, 75 23 78 26 34 17
11. Write program which will delete 2nd, 4th, 6th, … elements. 17 34 78 75 , 75 78 34 17
12. Write program which will delete 1st, 3rd, 5th, … elements. 12 26 23 , 23 26 12
13. Read two locations. Write program to delete all elements between them. If locations are 2 and 5 then final link list is 17 12 78 23 75. The reverse is 75 23 78 12 17.

Read one more element and put it after first element. The contents of the box should be replaced by. Suppose the new element is 33 then the list will become17 33 12 34 26 78 23 75 The reverse printing will be 75 23 78 26 34 12 33 17

```
printf("Give an element to put");scanf("%f",&b);
i=head->next;
t=(node *)malloc(sizeof(node));
t->next=i;t->prev=head;t->data=b;
head->next=t;i->prev=t;
```

14. Read one element and put it in the begining. Input 33, output 33 17 12 34 26 78 23 75, 75 23 78 26 34 12 17 33
15. Put it at last. 17 12 34 26 78 23 75 33, 33 75 23 78 26 34 12 17
16. Put it at second last place. 17 12 34 26 78 23 33 75, 75 33 23 78 26 34 12 17
17. Put it after first even element. 17 12 33 34 26 78 23 75, 75 23 78 26 34 33 12 17
18. Put copy of last element in the beginning. 75 17 12 34 26 78 23 75, 75 23 78 26 34 12 17 75
19. Put k copies of the last element in the beginning. Let k=3 output 75 75 75 17 12 34 26 78 23 75, 75 23 78 26 34 12 17 75 75 75
20. Write program, which inserts an element between every pair of consecutive elements. The new element is sum of neighbor. In above case 17 29 12 46 34 60 26 104 78 101 23 98 75. The reverse is 75 98 23 101 78 104 26 60 34 46 12 29 17.

Printing is permitted in following programs.

21. Read k and print first k elements in reverse order. For k=3 output is 34 12 17.
22. Write program to output the maximum element. Output 78.
23. Find the element before the maximum element. In above list 26.
24. Write program to output all even numbers in reverse order. Output 78 26 34 12.
25. Write program to find local maximum elements. An element is local maximum, if it greater then both its neighbors. In above list 34 and 78 are local maximum.
26. Output all elements after kth element in reverse order. For k=2 output 75 23 78 26 34.
27. Print last k elements in reverse order. For k=2 the output is 75 23. For k=5 the output is 75 23 78 26 34

28. Read two link lists. Put the second list after the first.
29. Read two link lists of same size. Perform product of corresponding elements. Then add them. Let link lists are 12, 44, 56, 78 and 22, 45, 13, 17 output is 12*22+44*45+56*13+78*17 = .
30. Read two link lists of same size. Perform element by element addition. Let link lists are 12, 44, 56, 78 and 22, 45, 13, 17 then first list is modified as 34, 89, 69, 95.

```
typedef struct node {float data;struct node *next,*prev; }node;
main()
{
   node *i,*t,*head; int j,choice,loc;float y;
   head=NULL;
   do
   {
      printf("choice: -1:put at begining -2:delete -3:print -4:exit ");
      scanf("%d",&choice);
      if(choice==-1)
      {  printf("Give the element to be put"); scanf("%f",&y);
         t=(node *)malloc(sizeof(node));
         if (head!=NULL) head->prev=t;
         t->data=y;t->next=head;t->prev=NULL;head=t;
      }
      if(choice==-2)
      {  printf("Give location of element to be deleted");
         scanf("%d",&loc);
         if(loc==1) { head=head->next;if (head!=NULL) head->prev=NULL;}
         else
         {  i=head;
            for(j=1;j<=loc-1;j++) i=i->next;
            i=i->prev;
            i->next=i->next->next;
            if (i->next!=NULL) i->next->prev=i;
         }
      }
      if(choice==-3)
      {  i=head;
         printf("Forward ");
         while(i->next!=NULL) {printf("%4.1f ",i->data);i=i->next;}
         printf("%4.1f    Backward ",i->data);
         while(i!=NULL) {printf("%4.1f ",i->data);i=i->prev;}
         printf("\n");
      }
   }while(choice!=-4);
}
```

Linked List (singly circular)

```
#include<stdio.h>
typedef struct node {float data;struct node *next; }node;
main()
{
   node *i,*t,*head; int j,choice,loc;float y;
   head=NULL;
   do
   {
      printf("choice: -1:put at begining -2:delete -3:print -4:exit");
      scanf("%d",&choice);
      if(choice==-1)
      {  printf("Give the element to be put at begining ");
         scanf("%f",&y);
         t=(node *)malloc(sizeof(node));
         t->data=y;
         if(head==NULL){head=t;head->next=head;}
         else
         {  t->next=head;
            i=head;
            do{i=i->next;}while(i->next!=head);
            i->next=t;
            head=t;
         }
      }
      if(choice==-2)
      {  printf("Give location of element to be deleted  ");
         scanf("%d",&loc);
         i=head;
         if(loc==1)
         {  if(head->next==head)head=NULL;
            else
            {
            do{i=i->next;}while(i->next!=head);i->next=head->next;
            head=head->next;
            }
         }
         else
         {  i=head;for(j=2;j<loc;j++) i=i->next;
            i->next=i->next->next;
         }
      }
      if(choice==-3)
      {  i=head;
         if(head==NULL)printf("List is empty\n");
         else
         {   do{printf("%4.1f ",i->data);i=i->next;}while(i!=head);  }
      }
   }while(choice!=-4);
}
```

Singly Linked List

```
#include<stdio.h>

typedef struct  node
              { float data;
                struct node *next;
              } node;

main()
{   node *i,*j,*k,*head,*t,*hd;
    int g,n; float b;

    printf("Howmany elements the link list has ");
    scanf("%d",&n);
    head=(node *)malloc(sizeof(node));
    i=head;
    for (g=1;g<n;g++)
    {   printf("Give  %dth element ",g);
        scanf("%f",&(*i).data);
        (*i).next=(node *)malloc(sizeof(node));
        i=(*i).next;
    }
    printf("Give last element ");
    scanf("%f",&(*i).data);
    (*i).next=NULL;

    printf("Give the element to be inserted ");
    t=(node *)malloc(sizeof(node));
    scanf("%f",&(*t).data);
    printf("Before which element it is to be inserted ");
    scanf("%f",&b);
    if ((*head).data==b)
    {   (*t).next=head;
        head=t;
    }
    else
    {   i=head;
        while ((*((*i).next)).data!=b) i=(*i).next;
        (*t).next=(*i).next;
        (*i).next=t;
    }

    i=head;
    while (i!=NULL)
    {   printf("%f ",(*i).data);
        i=(*i).next;
    }
}
```

The above program inserts an element in linked list. Notation: in place of (*p).q one can use p->q. Following is program segment to get duplicate copy of linked list.

Doubly Linked List

```
#include<stdio.h>
typedef struct  node
              { float data;
                struct node *next,*prev;
              } node;
main()
{   node *i,*j,*k,*head,*t,*hd;
    int g,n; float b;

    printf("Howmany elements the link list has ");
    scanf("%d",&n);
    head=(node *)malloc(sizeof(node));
    i=head;(*head).prev=NULL;
    for (g=1;g<n;g++)
    {   printf("Give  %dth element ",g);
        scanf("%f",&(*i).data);
        (*i).next=(node *)malloc(sizeof(node));
        (*(*i).next).prev=i;i=(*i).next;
    }
    printf("Give last element ");
    scanf("%f",&(*i).data);
    (*i).next=NULL;

    printf("\nThe linked list is ");
    i=head;
    while (i!=NULL)
    {   printf("%f ",(*i).data);
        j=i;i=(*i).next;
    }
    printf("\nThe reverse linked list is ");
    while (j!=NULL)
    {   printf("%f ",(*j).data);
        j=(*j).prev;
    }
}
```

Link list

84. Write program to find last element of the linked list.
85. Write program, which reads k also and finds k^{th} element of the linked list.
86. Write program to find sum of all elements.
87. Write program to find the maximum element.
88. Write program to find the first even element. In 3 7 6 5 4 8 the first even element is 6.
89. Read an element and find its location in linked list. e.g. if linked list is 17 12 34 26 78 23 95 and given element is 78 then its location is 5.
90. Write program to find local maximum elements. An element is local maximum, if it greater then both its neighbors. In above list 34 and 78 are local maximum.

In following programs the correctness should be checked by printing the entire linked list.

91. Write a program to delete the first element.
92. Write program to delete the second element.
93. Write program to delete the last element.
94. Write program to delete the second last element.
95. Write program to delete the kth element.
96. Write program which will delete first k elements.
97. Write program which will delete the last k elements.
98. Write program, which reads one more element and puts it in the beginning of the linked list. If the given link list is 12 34 56 78 15 65 72 and the new element is 63 then after the operation the linked list will be 63 12 34 …
99. Write program to put the element at second position. In above case 12 63 34 …
100. Write program to put the element at last position.
101. Write program, which inserts an element between every pair of consecutive elements. The new element is sum of neighbor. In above case 12 46 34 90 56 134 78 93 15 80 65 137 72.
102. Write program to exchange the second and third element. If given list is 12 34 7 56 78 then after the operation it will be 12 7 34 56 78.
103. Write program to exchange last and second last elements. In above case 12 34 7 78 56.
104. Write program to exchange the first two elements. In above case 34 12 7 56 78.
105. Write program to exchange first and last elements. In above case 78 34 7 56 12.
106. Write program which will delete 1st, 3rd, 5th, … elements.
107. Write program to delete the first even number.
108. Write program to delete the maximum number. (Assume it is unique)
109. Read two locations. Write program to delete all elements between them. e.g. if linked list is 12 34 56 78 15 65 72 and locations are 2 and 5 then final linked list is 12 65 72.
110. Write program, which will read two linked lists and joins them. If given linked lists are 12 34 13 and 56 78 then final linked list will be 12 34 13 56 78.

To check correctness of following programs print the linked list in reverse order also.

111. Write program, which converts a singly linked list into doubly linked list. [Hint: use one more field ‘prev’ in the node].
112. Write program to delete the first element.
113. Write program to delete the second element.

114. Write program to delete the last element.
115. Write program to delete the second last element.
116. Write program to delete the kth element.
117. Write program, which will read one more element and put it at beginning.
118. Write program, which will put it at last place.

Deletion in Linked List

```
#include<stdio.h>
main()
{   int data[20],next[20],i,j,size,head,x;
    printf("How many nodes");scanf("%d",&size);
    printf("Give data array");
    for (i=1;i<=size;i++)scanf("%d",&data[i]);
    printf("Give next array");
    for (i=1;i<=size;i++)scanf("%d",&next[i]);
    printf("Give location of head node");scanf("%d",&head);

    printf("The given list is :");
    i=head; while (i!=-1){ printf("%d ",data[i]);i=next[i];}

    printf("\nGive the node to be deleted");scanf("%d",&x);
    if (data[head]==x) {head=next[head];}
    else{
        i=head;while(data[next[i]]!=x) i=next[i];
        next[i]=next[next[i]];}

    printf("\nThe final list is :");
    i=head; while (i!=-1){ printf("%d ",data[i]);i=next[i];}
}
```

Insertion in Linked List

```
#include<stdio.h>
main()
{   int data[20],next[20],i,j,size,head,x,y;
    printf("How many nodes");scanf("%d",&size);
    printf("Give data array");
    for (i=1;i<=size;i++)scanf("%d",&data[i]);
    printf("Give next array");
    for (i=1;i<=size;i++)scanf("%d",&next[i]);
    printf("Give location of head node");scanf("%d",&head);

    printf("The given list is :");
    i=head; while (i!=-1){ printf("%d ",data[i]);i=next[i];}

    printf("\nGive the node to be inserted");scanf("%d",&x);
    printf("Before which node it is to be inserted");
    scanf("%d",&y);
    j=size+1;size=size+1;
    data[j]=x;
    if (data[head]==y) {next[j]=head;head=j;}
    else{
        i=head;while(data[next[i]]!=y) i=next[i];
        next[j]=next[i];next[i]=j;}

    printf("\nThe final list is :");
    i=head; while (i!=-1){ printf("%d ",data[i]);i=next[i];}
}
```

Linked list

Let the sequence be
(17,42,26,19,18,28,27,33).
It may be stored as
head 3

Index	1	2	3	4	5	6	7	8
Data	18	19	17	27	28	42	33	26
Next	5	1	6	7	4	8	-1	2

1. Write program to find the number of even elements in a sequence. For above sequence it will be 4.

2. Write program which will find the last element of the sequence. In above case 33.

3. Write program which will find the second last element of the sequence. In above case 27.

4. Write program which will delete the last element of the sequence. The above sequence after deleting the last element will be
   (17,42,26,19,18,28,27).
   It will be stored as

head 3

Index	1	2	3	4	5	6	7	8
Data	18	19	17	27	28	42	33	26
Next	5	1	6	-1	4	8	-1	2

5. Write program which will delete the last two elements of the sequence. The above sequence after deleting will be (17,42,26,19,18,28).
   It will be stored as

head 3

Index	1	2	3	4	5	6	7	8
Data	18	19	17	27	28	42	33	26
Next	5	1	6	7	-1	8	-1	2

6. Write program to delete the second last element.

7. Write program, which will delete first half of the elements. Assume that the sequence has even number of elements. Since in the above example there are 8 elements hence first four elements will be deleted. The final sequence will be (18,28,27,33).

The storage will look like
head 1

Index	1	2	3	4	5	6	7	8
Data	18	19	17	27	28	42	33	26
Next	5	1	6	7	4	8	-1	2

8. Write program, which will delete second half of the elements. The final sequence will be (17,42,26,19).

The storage will look like
head 3

Index	1	2	3	4	5	6	7	8
Data	18	19	17	27	28	42	33	26
Next	5	-1	6	7	4	8	-1	2

9. Write program, which will decompose the sequence in two parts. The above sequence will be broken into (17,42,26,19) and (18,28,27,33).

The storage will look like
$head_1$ 3
$head_2$ 1

Index	1	2	3	4	5	6	7	8
Data	18	19	17	27	28	42	33	26
Next	5	-1	6	7	4	8	-1	2

10. Write program which will delete all elements after the first even element. For given example the sequence will be (17,42). It will be stored as

head 3

Index	1	2	3	4	5	6	7	8
Data	18	19	17	27	28	42	33	26
Next	5	1	6	7	4	-1	-1	2

11. Write program which will delete all elements after the last even element. For given example the sequence will be (17,42,26,19,18,28).

It may be stored as

head 3

Index	1	2	3	4	5	6	7	8
Data	18	19	17	27	28	42	33	26
Next	5	1	6	7	-1	8	-1	2

12. Write program which will delete the first even element. For given exapmle the sequence will be (17,26,19,18,28,27,33).

It may be stored as

head 3

Index	1	2	3	4	5	6	7	8
Data	18	19	17	27	28	42	33	26
Next	5	1	8	7	4	8	-1	2

Care should be taken if the first element is even.

13. Write program which will delete the smallest element. For given example the sequence will be (42,26,19,18,28,27,33).

It may be stored as

head 6

Index	1	2	3	4	5	6	7	8
Data	18	19	17	27	28	42	33	26
Next	5	1	6	7	4	8	-1	2

14. Write program, which will decompose the sequence into two parts. The first sequence will contain elements at odd position and the second sequence will contain elements at even position. For above case the sequences will be (17,26,18,27) and (42,19,28,33) respectively. The storage will look as

headodd 3

headeven 6

Index	1	2	3	4	5	6	7	8
Data	18	19	17	27	28	42	33	26
Next	4	5	8	-1	7	2	-1	1

15. Write program, which will delete all even numbers from a sequence. For given example after deleting even elements the sequence will be (17,19,27,33).

The storage will look as

head 3

Index	1	2	3	4	5	6	7	8
Data	18	19	17	27	28	42	33	26
Next	5	4	2	7	4	8	-1	2

16. Write program, which will decompose the sequence into two parts. The first part will contain the even elements and the other will contain odd elements. For above case the sequences will be (42,26,18,28) and (17,19,27,33) respectively.

The storage will look as

headeven 6

headodd 3

Index	1	2	3	4	5	6	7	8
Data	18	19	17	27	28	42	33	26
Next	5	4	2	7	-1	8	-1	1

17. Write program, which exchanges consecutive elements of a linked list. Assume that the list contains even number of elements. For above case the sequence will be (42,17,19,26,28,18,33,27)

The storage will look as

head 6

Index	1	2	3	4	5	6	7	8
Data	18	19	17	27	28	42	33	26
Next	7	8	2	-1	1	3	4	5

Care should be taken so that the storage is as above only. No change in the data array should be made.

18. Write program, to reverse the linked list. For above example the sequence will be (33,27,28,18,19,26,42,17)
    The storage will look as

head 7

Index	1	2	3	4	5	6	7	8
Data	18	19	17	27	28	42	33	26
Next	2	8	-1	5	1	3	4	6

19. Write program, which will join two sequences by putting the second sequence after the first. Assume that both sequences have equal number of elements. e.g. let the input sequences be (17,26,18,27) and (42,19,28,33) then output sequence will be (17,26,18,27,42,19,28,33).

Let input storage is

$head_1$ 3

$head_2$ 6

Index	1	2	3	4	5	6	7	8
Data	18	19	17	27	28	42	33	26
Next	4	5	8	-1	7	2	-1	1

The final storage will look as

head 3

Index	1	2	3	4	5	6	7	8
Data	18	19	17	27	28	42	33	26
Next	4	5	8	6	7	2	-1	1

20. Write program, which will merge two sequences by taking one element from each alternatively. Assume that both sequences have equal number of elements. For above input lists the final list will be (17,42,26,19,18,28,27,33).

The final storage will look as

head 3

Index	1	2	3	4	5	6	7	8
Data	18	19	17	27	28	42	33	26
Next	5	1	6	7	4	8	-1	2

21. Write program, which will read a number (x) also. It is an element of the first sequence. The second sequence will be put after x in the first sequence. If x= 26 then the final sequence will be (17,26,42,19,28,33,18,27)
    The final storage will look as

head 3

Index	1	2	3	4	5	6	7	8
Data	18	19	17	27	28	42	33	26
Next	4	5	8	-1	7	2	1	6

22. Write program, which inserts an element between every pair of elements. The inserted element is the sum of adjacent elements. If the input list is (28,16,13) then the final list will be (28,44,16,29,13).
    Suppose the input storage is

head 2 size(k)=5

Index	1	2	3	4	5	6	7	8
Data	13	28			16			
Next	-1	5			1			

The final storage will look as

head 2 size(k)=7

Index	1	2	3	4	5	6	7	8
Data	13	28			16	44	29	
Next	-1	6			7	5	1	

23. Write recursive program to reverse a linked list. [Caution: No change in the data array should be made.]

Dynamic Programming

Write program for following using recursive programming and dynamic programming

1. f(1)=1 f(n)=f(n-1)+f(n/2) if n>1 [Answer: f(10)=30 f(40)=2062]
2. f(1)=2 f(2)=7 f(n)= $\sum_{i=1}^{i=n-1}$ (f(i)*f(n-i))%100 if n>2 [f(10)=322 f(15)=624 f(20)=934]
3. f(1)=2 f(2)=7 f(n)= $\min_{i=1}^{i=n-1}$ (f(i)*f(n-i))%100 if n>2 [f(10)=36 f(15)=16 f(20)=12]

<u>Recursive program: Top Down approach</u>

```
int f(int n,int a[])
{  int t;
   if (n==1) return a[1];
   if (n==2) return a[2];
   t=(f(n-1,a)+f(n-2,a))*a[n];
   return t;
}
main()
{  int i,n,a[20],p;
   scanf("%d",&n);
   for (i=1;i<=n;i++)
      scanf("%d",&a[i]);
   p=f(n,a);
   printf("%d",p);
}
```

<u>Dynamic program: Bottom up approach</u>

```
main()
{  int i,n,a[20],p,f[20];
   scanf("%d",&n);
   for (i=1;i<=n;i++)
      scanf("%d",&a[i]);
   f[1]=a[1];f[2]=a[2];
   for (i=3;i<=n;i++)
   {  f[i]=(f[i-1]+f[i-2])*a[i];
      printf("%d %d\n",i,f[i]);
   }
}
```

A function f(n,a) is defined as follows (n is a positive integer):

f(n,a)=a[i] if n<3

f(n)=(f(n-1,a)+f(n-2,a))*a_n otherwise

[Let n=6 and $a_1,\ldots,a_6$ are 3,2,4,1,5,6 then f(3,a)=20 f(4,a)=22 f(5,a)=210 f(6,a)=1392]

4. f(1,a)= a_1 f(2,a)= a_2 f(n,a)= $\sum_{i=1}^{i=n-1}$ f(i,a)*f(n–i,a) if n>2 [f(6,a)=3472 f(5,a)=504]
5. f(1,a)=a_1 f(2,a)=a_2 f(n,a)= $\min_{i=1}^{i=n-1}$ f(i,a)*f(n–i,a) if n>2 [f(6,a)=8 f(5,a)=12]
6. Write a program, which reads two numbers ‘x’ any ‘y’ Let x=3 y=7 then output is (3,3)(4,4)(5,5)(6,6)(7,7)(3,4)(4,5)(5,6)(6,7)(3,5)(4,6)(5,7)(3,6)(4,7)(3,7)
   When x=4 y=6 output (4,4)(5,5)(6,6)(4,5)(5,6)(4,6)

The program for following function is given f(5,9)=841 f(2,7)=761
f(m,n) = m if m=n
= f(m,n-1) + f(m+1,n) + m*n if n>m
	`for (i=m;i<=n;i++) f[i][i]=i;`
	`for (d=1;d<=(n-m);d++)`
	`  for (j=m;j<=(n-d);j++){`
	`  x=j;y=j+d;`
	`  f[x][y]=f[x][y-1]+f[x+1][y]+x*y;}`

7. f(m,n)=0 if m=n-1 f(m,n)= $\sum_{i=m+1}^{i=n-1}$ [f(m,i)+f(i,n)+m*n]if m<n [f(3,7)=303f(5,12)=25965]
8. f(m,n)=0 if m=n-1 f(m,n)= $\min_{i=m+1}^{i=n-1}$ [f(m,i)+f(i,n)+((m*n)%20)] if m<n [f(3,7)=13]
9. f(m,n,a) = 0 if m=n-1 f(m,n,a)= $\min_{i=m+1}^{i=n-1}$ (f(m,i,a)+f(i,n,a)+a_m*a_i* a_n) if(m<n)

The above is solution of matrix chain multiplication. f(1,6,a) = 62.

```
#include<stdio.h>
main()
{  int i,j,p,q,a,b;
   printf("Give p and q");
   scanf("%d %d",&p,&q);
   for (i=1;i<=(q-p+1);i++)
   {  for (j=p;j<=q-i+1;j++)
         printf("%d,",j);
      printf("\n");
   }
}
```

The program takes 2 inputs p and q and produces a picture. Pictures for various values of p and q are shown.

p=5 q=9	p=2 q=7
5,6,7,8,9, 5,6,7,8, 5,6,7, 5,6, 5	2,3,4,5,6,7 2,3,4,5,6, 2,3,4,5, 2,3,4, 2,3, 2,

1. Modify the above program so that pictures become as shown. It can be done by replacing print as: printf("(%d,%d)",j,j+i-1);	(5,5)(6,6)(7,7)(8,8)(9,9) (5,6)(6,7)(7,8)(8,9) (5,7)(6,8)(7,9) (5,8)(6,9) (5,9)	(2,2)(3,3)(4,4)(5,5)(6,6)(7,7) (2,3)(3,4)(4,5)(5,6)(6,7) (2,4)(3,5)(4,6)(5,7) (2,5)(3,6)(4,7) (2,6)(3,7) (2,7)

2. Using above write program to evaluate 'g' g(5,9)=2239 g(20,27)=606802	g(m,n) = m if m=n = 2g(m,n-1) + g(m+1,n) + m*n if n>m

It can be done by replacing
`printf("(%d,%d)",j,j+i-1);`
By given program segment.
	`{ m=j; n=j+i-1;`
	`if (m==n) g[m][n]=m;`
	`else g[m][n]= 2*g[m][n-1] + g[m+1][n] + m*n;`
	`}`
	`printf("g(%d,%d)=%d",p,q,g[p][q]);`

3. Write program to print following pictures.		4. Modify this program so that pictures become as following. Make modifications in the print statement only.	
p=5 q=9	p=2 q=7	p=5 q=9	p=2 q=7
5,6,7,8,9, 6,7,8,9, 7,8,9, 8,9, 9,	2,3,4,5,6,7, 3,4,5,6,7 4,5,6,7, 5,6,7 6,7, 7,	(5,5)(6,6)(7,7)(8,8)(9,9) (5,6)(6,7)(7,8)(8,9) (5,7)(6,8)(7,9) (5,8)(6,9) (5,9)	(2,2)(3,3)(4,4)(5,5)(6,6)(7,7) (2,3)(3,4)(4,5)(5,6)(6,7) (2,4)(3,5)(4,6)(5,7) (2,5)(3,6)(4,7) (2,6)(3,7) (2,7)
5. Using this find the value of above function. g(5,9)=2239 g(20,27)=606802			

6. Write program to print following pictures.		7. Modify this program so that pictures become as following. Make modifications in the print statement only.	
p=5 q=9	p=2 q=7	p=5 q=9	p=2 q=7
5, 6,5, 7,6,5, 8,7,6,5, 9,8,7,6,5,	2, 3,2 4,3,2, 5,4,3,2, 6,5,4,3,2, 7,6,5,4,3,2,	(5,5) (6,6)(5,6) (7,7)(6,7)(5,7) (8,8)(7,8)(6,8)(5,8) (9,9)(8,9)(7,9)(6,9)(5,9)	(2,2) (3,3)(2,3) (4,4)(3,4)(2,4) (5,5)(4,5)(3,5)(2,5) (6,6)(5,6)(4,6)(3,6)(2,6) (7,7)(6,7)(5,7)(4,7)(3,7)(2,7)
8. Using this find the value of above function. g(5,9)=2239 g(20,27)=606802			

<table>
<tr><td colspan="2">9. Write program to print following pictures.</td><td colspan="2">10. Modify this program so that pictures become as following. Make modifications in the print statement only.</td></tr>
<tr><td>p=5 q=9
5,
6,6
7,7,7,
8,8,8,8,
9,9,9,9,9,</td><td>p=2 q=7
2,
3,3,
4,4,4,
5,5,5,5,
6,6,6,6,6,
7,7,7,7,7,7,</td><td>p=5 q=9
(5,5)
(6,6)(5,6)
(7,7)(6,7)(5,7)
(8,8)(7,8)(6,8)(5,8)
(9,9)(8,9)(7,9)(6,9)(5,9)</td><td>p=2 q=7
(2,2)
(3,3)(2,3)
(4,4)(3,4)(2,4)
(5,5)(4,5)(3,5)(2,5)
(6,6)(5,6)(4,6)(3,6)(2,6)
(7,7)(6,7)(5,7)(4,7)(3,7)(2,7)</td></tr>
<tr><td colspan="4">11. Using this find the value of above function. g(5,9)=2239 g(20,27)=606802</td></tr>
</table>

Following program takes 2 inputs p and q and produces a picture.

```
for (i=1;i<=q+1;i++)
{   a=(p-q+i-1);
    b=(p+q-i+1);
    for (j=a;j<=b;j++)
         printf("%d,",j);
    printf("\n");
}
```

p=9 q=4
5,6,7,8,9,10,11,12,13,
6,7,8,9,10,11,12,
7,8,9,10,11,
8,9,10,
9,

p=15 q=3
12,13,14,15,16,17,18,
13,14,15,16,17,
14,15,16,
15

12. Modify the above program so that pictures become as following. Modify only print.

(5,0)(6,0)(7,0)(8,0)(9,0)(10,0)(11,0)(12,0)(13,0)
(6,1)(7,1)(8,1)(9,1)(10,1)(11,1)(12,1)
(7,2)(8,2)(9,2)(10,2)(11,2)
(8,3)(9,3)(10,3)
(9,4)

(12,0)(13,0)(14,0)(15,0)(16,0)(17,0)(18,0)
(13,1)(14,1)(15,1)(16,1)(17,1)
(14,2)(15,2)(16,2)
(15,3)

13. Using above write program to evaluate 'h'
h(9,4)=2304 h(20,12)=335544320

h(m,n) = m if n=0
= h(m+1,n-1) + h(m-1,n-1) + 2h(m,n-1) if n>0

14. Write program to print following pictures.

p=9 q=4
5,7,9,11,13
6,8,10,12
7,9,11
8,10
9,

p=15 q=3
12,14,16,18
13,15,17,
14,16,
15,

15. Modify this program so that pictures become as following. Make modifications in the print statement only.

p=9 q=4
(5,0)(7,0)(9,0)(11,0)(13,0)
(6,1)(8,1)(10,1)(12,1)
(7,2)(9,2)(11,2)
(8,3)(10,3)
(9,4)

p=15 q=3
(12,0)(14,0)(16,0)(18,0)
(13,1)(15,1)(17,1)
(14,2)(16,2)
(15,3)

16. Using above write program to evaluate 'k'
k(17,5)=7763 k(20,12)=22054670

k(m,n) = m if n=0
= 2k(m+1,n-1) + k(m-1,n-1) + m*n if n>0

17. Write program to print following pictures.

p=5 q=9
5,6,…., 13,
5,6,…., 11,
5,6,7,8,9,
5,6,7,
5,

p=2 q=5
2,3,4,5,6,7,8,
2,3,4,5,6,
2,3,4,
2,

18. Modify this program so that pictures become as following. Make modifications in the print statement only.

p=5 q=9
(5,5)(6,6) ……. (13,13)
(5,6)(6,7) ……. (11,12)
(5,7)(6,8)(7,9)(8,10)(9,11)
(5,8)(6,9)(7,10)
(5,9)

p=2 q=5
(2,2)(3,3)(4,4)(5,5)(6,6)(7,7)(8,8)
(2,3)(3,4)(4,5)(5,6)(6,7)
(2,4)(3,5)(4,6)
(2,5)

19. Using above write program to evaluate 'z'
z(5,9)=729 z(20,27)=59049

z(m,n) = m if m=n
= z(m,n-1) + z(m+1,n) + z(m+2,n+1) if n>m

20. Write program to print following pictures.

p=5 q=9
5,6,…., 13,
6,7,…., 12,
7,8,9,10,11,

p=2 q=5
2,3,4,5,6,7,8,
3,4,5,6,7,
4,5,6,

21. Modify this program so that pictures become as following. Make modifications in the print statement only.

p=5 q=9
(5,5)(6,6) ……. (13,13)
(5,6)(6,7) ……. (11,12)
(5,7)(6,8)(7,9)(8,10)(9,11)

p=2 q=5
(2,2)(3,3)(4,4)(5,5)(6,6)(7,7)(8,8)
(2,3)(3,4)(4,5)(5,6)(6,7)
(2,4)(3,5)(4,6)

8,9,10, 9,	5,	(5,8)(6,9)(7,10) (5,9)	(2,5)
22. Using this find the value of above function. z(5,9)=729 z(20,27)=59049			

23. Write program, whose input is a number (k). For k=6 and k=4 given pictures are printed.	(1,1) (1,2)(2,1) (1,3)(2,2)(3,1) (1,4)(2,3)(3,2)(4,1) (1,5)(2,4)(3,3)(4,2)(5,1)	(1,1) (1,2)(2,1) (1,3)(2,2)(3,1)
24. Using above write program to evaluate 'd' d(17,5)=11481975	d(m,n) = m+n if n=0 or m=0 = 2d(m,n-1) + d(m-1,n) + d(m-1,n-1) if n>0 & m>0	
Hint: Picture printing for k=17+5		

25. Write program, whose input are two numbers (x and y). For (x,y)=(9,4) and (15,3) given pictures are printed.	(6,1)(7,1)(8,1)(9,1)(10,1)(11,1)(12,1) (7,2)(8,2)(9,2)(10,2)(11,2) (8,3)(9,3)(10,3) (9,4)	(13,1)(14,1)(15,1)(16,1)(17,1) (14,2)(15,2)(16,2) (15,3)
Using above write program to evaluate 'h' h(9,4)=2304 h(20,12)=335544320	h(m,n) = m if n=0 = h(m+1,n-1) + h(m-1,n-1) + 2h(m,n-1) if n>0	

26. Write program, whose input are two numbers (x and y). For (x,y)=(9,4) and (15,3) given pictures are printed.	(6,1)(8,1)(10,1)(12,1) (7,2)(9,2)(11,2) (8,3)(10,3) (9,4)	(13,1)(15,1)(17,1) (14,2)(16,2) (15,3)
Using above write program to evaluate 'k' k(17,5)=7763 k(20,12)=22054670	k(m,n) = m if n=0 = 2k(m+1,n-1) + k(m-1,n-1) + m*n if n>0	

27. Pictures for (x,y)=(5,9) and (2,8)	(5,6) (6,7)(5,7) (7,8)(6,8)(5,8) (8,9)(7,9)(6,9)(5,9)	(2,3) (3,4)(2,4) (4,5)(3,5)(2,5) (5,6)(4,6)(3,6)(2,6) (6,7)(5,7)(4,7)(3,7)(2,7) (7,8)(6,8)(5,8)(4,8)(3,8)(2,8)
Using above write program to evaluate 'f' f(5,9)=2239 f(20,27)=606802	f(m,n) = m if m=n = 2f(m,n-1) + f(m+1,n) + m*n if n>m	

28. Pictures for (x,y)=(5,9) and (2,5)	(5,6)(6,7)(7,8)(8,9)(9,10)(10,11)(11,12) (5,7)(6,8)(7,9)(8,10)(9,11) (5,8)(6,9)(7,10) (5,9)	(2,3)(3,4)(4,5)(5,6)(6,7) (2,4)(3,5)(4,6) (2,5)
Using above write program to evaluate 'z' z(5,9)=729 z(20,27)=59049	z(m,n) = m if m=n = z(m,n-1) + z(m+1,n) + z(m+2,n+1) if n>m	

Functions

```
#include<stdio.h>
int f(int x)
{  if (x==2) return 5;
   else return f(x-1)+x;
}
main()
{ int p,q;
  printf("Give a number");
  scanf("%d",&p);
  q=f(p);
  printf("f(%d)=%d\n",p,q);
}
```

```
main()
{ int p,x,f[100];
  printf("Give a number");
  scanf("%d",&p);
  f[2]=5;
  for (x=3;x<=p;x++)
     f[x]=f[x-1]+x;
  printf("f(%d)=%d\n",p,f[p]);
}
```

Both programs compute function

f(x)=5 if x=2

f(x)=f(x-1)+x if x>2

```
main()
{  int p,s,i;
   printf("Give a number");
   scanf("%d",&p);
   s=0;
   for (i=1;i<=p;i++)
     s=s+i;
   printf("%d\n",s);
}
```

Program for $\sum_{i=1}^{i=n} i$

```
main()
{  int p,m,i;
   printf("Give a number");
   scanf("%d",&p);
   m=10000;
   for (i=3;i<=p;i++)
     if (m>i*i%17) m=i*i%17;
   printf("%d\n",m);
}
```

Program for $\min_{i=3}^{i=n}$ i*i%17

```
int f(int x)
{  int i,s;
   if (x==2) return 5;
   else
   {  s=0;
      for (i=2;i<=x-1;i++)
         s=s+f(i)*i;
      return s;
   }
}
main()
{ int p,q;
  printf("Give a number");
  scanf("%d",&p);
  q=f(p);
  printf("f(%d)=%d\n",p,q);
}
```

```
main()
{ int p,x,f[100],s,i;
  printf("Give a number");
  scanf("%d",&p);
  f[2]=5;
  for (x=3;x<=p;x++)
  {  s=0;
     for (i=2;i<=x-1;i++)
        s=s+f[i]*i;
     f[x]=s;
  }
  printf("f(%d)=%d\n",p,f[p]);
}
```

f(x)=5 if x=2

f(x)= $\min_{i=2}^{i=x-1}$ f(i)*i if x>2

```
int f(int x)
{  int i,m;
   if (x==2) return 5;
   else
   {  m=1000;
      for (i=2;i<=x-1;i++)
         if (m>f(i)*i%87)
              m=f(i)*i%87;
      return m;
   }
}
main()
{ int p,q;
  printf("Give a number");
  scanf("%d",&p);
  q=f(p);
  printf("f(%d)=%d\n",p,q);
}
```

```
main()
{ int p,x,f[100],m,i;
  printf("Give a number");
  scanf("%d",&p);
  f[2]=5;
  for (x=3;x<=p;x++)
  {  m=1000;
     for (i=2;i<=x-1;i++)
        if (m>f[i]*i%87)
        m=f[i]*i%87;
     f[x]=m;
  }
  printf("f(%d)=%d\n",p,f[p]);
}
```

$$f(x)=5 \qquad \text{if } x=2$$

$$f(x)=\sum_{i=2}^{i=x-1} f(i)*i\%87 \qquad \text{if } x>2$$

Partition

```
main()
{  int i,j,m,n,a[20],x,y,p;
   printf("Give set size ");scanf("%d",&n);
   printf("Give number of pairs ");scanf("%d",&m);
   for (i=1;i<=n;i++) a[i]=i;
   printf("Give %d pairs ",m);
   for (i=1;i<=m;i++)
   {   scanf("%d %d",&x,&y);
       if (a[x]!=a[y]) printf("(%d,%d)",x,y);
       p=a[x];
       for (j=1;j<=n;j++) if (a[j]==p) a[j]=a[y];
   }
   printf("\nPartitions are ");
   for (i=1;i<=n;i++)
   {  printf("(");
      for (j=1;j<=n;j++) if (a[j]==i) printf("%d,",j);
      printf(")");
   }
}
```

1. Define functions void rd(int x[],int size) and void wr(int a[],int size). They perform reading and writing elements.
2. Define function wreven(int x[],int size). It outputs only even numbers. In following output is 12 32 56 32 26

X_1	X_2	X_3	X_4	X_5	X_6	X_7	X_8	X_9	X_{10}	X_{11}	X_{12}	X_{13}	X_{14}	X_{15}	X_{16}
12	35	17	35	21	19	32	35	56	17	32	49	19	17	26	35

3. Modify above program to output locations of even numbers. In above case 1 7 9 11 15
4. Define function wrlocation(int x[],int size,int y). It outputs locations of all those elements in array which are equal to y. In above case call of wrlocation(x,16,35) will output 2 4 8 16
5. Define function void replace(int x[],int size,int y,int z). Every element, which is y will be replaced by z. In above case call of replace(x,16,35,17) will make the array as following:

X_1	X_2	X_3	X_4	X_5	X_6	X_7	X_8	X_9	X_{10}	X_{11}	X_{12}	X_{13}	X_{14}	X_{15}	X_{16}
12	17	17	17	21	19	32	17	56	17	32	49	19	17	26	17

6. Use above functions to output equivalent partitions.
   (A) Read the value of size (n) and initialize the array as x[i]=i
   (B) Read the value of number of pairs (m).
   (C) Read m pairs. After reading a pair (p,q) call replace(x,n.,x[p],x[q]).
   (D) Output partitions by calling wrlocation(x,size,i) for i=1..size.
   (E) Make arrangement so that comma(,) is put between partitions.
7. Define function to output even numbers in list form. In above case (12,32,56,32,26,).
8. Modify above program to produce output as (12,32,56,32,26).
9. Modify above program. It output “none” if there is no even number. Otherwise all even numbers are outputted.
10. Modify above program. It outputs “ttt” if there is exactly one even number and “none” if there is no even number. Otherwise all even numbers are outputted.

11. Use above programs to output partitions in good form. e.g. input n=14 m=10 and pairs are 1 10, 2 3, 7 11, 4 9, 10 12, 2 8, 9 7, 4 11, 3 8, 1 6. Output (1,6,10,12) (5) (2,3,8) (4,9,7,11) (13) (14).
12. Modify function so that partition of size 1 are not outputted. In above case output (1,6,10,12) (2,3,8) (4,9,7,11).
13. Write program, which will output those pairs which are not needed. (cyclic edge). In above case (4,11)(3,8).
    When pairs are 1 2, 3 4, 5 6, 7 8, 1 4, 2 3, 5 9, 9 10, 6 10, 11 12, 1 12, 4 11 then output is (2,3)(6,10) (4,11).

Sequence (Shifting based)

```
main()
{  int x[100],i,j,k,element,choice;x[1]=0;
   do{printf("\nGive choice: -1:put at last  -2:put at begining  ");
      printf(" -3:delete kth element -4:print  -5:exit  ");
      scanf("%d",&choice);
      if (choice==-1){
         printf("Give element to be put at last ");scanf("%d",&element);
         i=1;while(x[i]!=0) i++;
         x[i]=element; x[i+1]=0;}
      if (choice==-2){
         printf("Give element to be put at begining ");
         scanf("%d",&element);
         i=1;while(x[i]!=0) i++;
         for (j=i;j>=1;j--) x[j+1]=x[j];
         x[1]=element; }
      if (choice==-3){
         printf("Give k");scanf("%d",&k);
         i=k;while(x[i]!=0) {x[i]=x[i+1]; i++;}}
      if (choice==-4){
         i=1;while(x[i]!=0) {printf("%d ",x[i]);i++;}}
      }while(choice!=-5);
}
```

Let a sequence has numbers 33, 45, 12, 27, 13, 89, 24, 88, 34, 78, 85, 16. After deleting 5th element the sequence will appear as 33, 45, 12, 27, 89, 24, 88, 34, 78, 85, 16.
In following programs no printing should be done. To check the correctness use (-4).
Put this sequence in a file by typing -1 33 -1 45 -1 12 …… -1 85 -1 16 -4 -5.

Do not use nested loop (one loop inside another)

1. Write portion of the program to delete all elements after k^{th} element. Function should not use any loop. If k=7 then sequence will be 33, 45, 12, 27, 13, 89, 24.
2. Delete all elements after first even number. Assume at least one even number. The new sequence will be 33, 45, 12. The program should be efficient.
3. Increment all even numbers. 33, 45, 13, 27, 13, 89, 25, 89, 35, 79, 85, 17.
4. Find k^{th} last number. If k=7 then output is 89. [printing permitted]
5. Find k^{th} number after first even number. If k=7 then output 78. [printing permitted]
6. Find first even number after k^{th} number. If k=7 then output 88. [printing permitted]
7. Delete first k numbers. For k=7 the sequence is 88, 34, 78, 85, 16.
8. Increment all elements between two given locations. If given locations are 5 and 9 then sequence will be 33, 45, 12, 27, 14, 90, 25, 89, 35, 78, 85, 16.
9. Reverse all elements between two given locations. If given locations are 5 and 9 then sequence will be 33, 45, 12, 27, 34, 88, 24, 89, 13 , 78, 85, 16.
10. Reverse the sequence. 16, 85, 78, 34, 88, 24, 89, 13, 27, 12, 45, 33.
11. Exchange consecutive elements. Assume there is even number of elements. Output is 45, 33, 27, 12, 89, 13, 88, 24, 78, 34, 16, 85. Do this problem when odd number of elements is also permitted.
12. Add location of an even number to that. 33, 45, 15, 27, 13, 89, 31, 96, 43, 88, 85, 28.
13. Add even location of an even number to that. 33, 45, 13, 27, 13, 89, 26, 91, 38, 83, 85, 22. [Hint: 12 is 1st even number, 24 is 2nd even number. Hence 12+1 and 24+2]
14. Delete all elements before first even number. 12, 27, 13, 89, 24, 88, 34, 78, 85, 16.

15. Insert a number at k^{th} place. If k=7 and element is 49 then new sequence is 33, 45, 12, 27, 13, 89, 49, 24, 88, 34, 78, 85, 16.
16. Delete elements x_1, x_3, x_5, …. 45, 27, 89, 88, 78, 16. [Use only one loop]
17. Insert a copy of all even elements after last element. 33, 45, 12, 27, 13, 89, 24, 88, 34, 78, 85, 16, 12, 24, 88, 34, 78, 16.
18. Delete all even elements. 33, 45, 27, 13, 89, 85. [Try by using one loop only]
19. Exchange consecutive even numbers. Assume there is even number of even numbers. 33, 45, 24, 27, 13, 89, 12, 34, 88, 16, 85, 78.
20. Insert an element before first bigger. If new element is biggest then it is put at the last place. In the given sequence 40 will be inserted before 45 and 97 will be inserted at the end.
21. Between every pair of elements insert a new element. It is the sum of the neighboring elements. 33, 78, 45, 57, 12, ……., 78, 163, 85, 101, 16. [Try by using only one loop]
22. Insert an element after every even element. It is half of that element. 33, 45, 12, 6, 27, 13, 89, 24, 12, 88, 44, 34, 17, 78, 39, 85, 16, 8.

Sequence (Permutation based)

```
#include<stdio.h>
main()
{  int data[20],location[20],i,j,choice,loc,y,z;
   for (i=1;i<20;i++){data[i]=0;location[i]=0;}
   do
   {
      printf("\nGive choice:  -1:insert -2:delete -3:print");
      printf(" -4:print elements -5:exit    ");
      scanf("%d",&choice);
      if(choice==-1)
      {  printf("Give element (to be inserted) and location");
         scanf("%d %d",&y,&z);
         for(i=1;i<20;i++) if(location[i]>=z)location[i]++;
         i=1;while(location[i]!=0) i++;
         data[i]=y;location[i]=z;
      }
      if(choice==-2)
      {  printf("Give location to perform deletion");scanf("%d",&z);
         for(i=1;i<20;i++) {
             if (location[i]==z) location[i]=0;
             if (location[i]>z)  location[i]--;
         }
      }
      if(choice==-3)
      {  printf("\nThe sequence is: ");
         for(i=1;i<20;i++)
         {  j=1;while((j<20)&&(location[j]!=i))j++;
            if (j==20) break;printf("%4d",data[j]); }
         printf("\nIndex");for (i=1;i<20;i++) printf("%3d",i);
         printf("\nData ");for (i=1;i<20;i++) printf("%3d",data[i]);
         printf("\nLoc  ");for (i=1;i<20;i++) printf("%3d",location[i]);
      }
      if(choice==-4)
      {  printf("\nThe elements of the sequence are: ");
         for(i=1;i<20;i++) if(location[i]!=0) printf("%4d",data[i]);}
   }while(choice!=-5);
}
```

Give following input

```
-1 34 1 -1 33 1 -1 86 3 -1 99 2 -1 67 4 -1 85 3 -1 69 7 -1 37 2 -1 57 1
-1 44 3 -1 78 3 -1 82 4 -1 97 6 -2 4 -2 7 -2 3 -3 -4 -5
```

The output is

The sequence is: 57 33 44 97 37 85 34 67 86 69

Index	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
Data	34	33	86	99	67	85	69	37	57	44	78	82	97	0	0	0	0	0	0
Loc	7	2	9	0	8	6	10	5	1	3	0	0	4	0	0	0	0	0	0

In following programs use only one loop.

1. Read one more element. Find its location. Input 67 output 8.
2. Write program to print elements at odd locations. The order of elements is unimportant. Output 34 86 37 57 44.
3. Write program to print elements at even locations. Output 33 67 85 69 97
4. Write program to output the sum of elements. output 609(not 868)
5. Write program to print all odd elements. Output 33 67 85 69 37 57 97
6. Write program to output the first element. Output 57
7. Read one more number k and output the kth element. For k=3 output is 44.
8. Write program to output the number of elements. Output 10. [max location]
9. Write program to output the last element. Output 69 [data at index of max location]
10. Write program to print the maximum element of the sequence. Output 97 (not 99)
11. Print the location of maximum element of the sequence. Output 4.
12. Print the location of last even element. output 9 [max location with even data]
13. Print the last even element. output 86 [data at index of max location with even data]
14. The location of first even element. output 3 [min (non zero) location with even data]
15. First even element. output 44 [data at index of min (non zero) location with even data]

In following programs no printing should be done. Correctness should be checked by (-4). No change in the data array should be done unless permitted.

16. Write program to increment all elements in most efficient manner. [Data array can be modified] 58 34 45 98 38 86 35 68 87 70
17. Write program to increment all elements at odd locations in most efficient manner. [Data array can be modified] 58 33 45 97 38 85 35 67 87 69
18. Write program to increment all elements at even locations. [Data array can be modified] 57 34 44 98 37 86 34 68 86 70
19. Program to exchange first and second element. 33 57 44 97 37 85 34 67 86 69
20. Exchange consecutive elements. 33 57 97 44 85 37 67 34 69 86
21. Exchange the last element and the maximum element. 57 33 44 69 37 85 34 67 86 97
22. Delete elements at odd location. 33 97 85 67 69
23. Delete elements at even location. 57 44 37 34 86
24. Write program to delete the first element. 33 44 97 37 85 34 67 86 69
25. Delete all elements beyond k^{th} element. For k=7 57 33 44 97 37 85 34
26. Delete k^{th} element. For k=7 57 33 44 97 37 85 67 86 69
27. Delete first k elements. For k=7 67 86 69
28. Write program to delete the last element. [Find index of max location] 57 33 44 97 37 85 34 67 86
29. Write program to exchange first and last element. 69 33 44 97 37 85 34 67 86 57
30. Insert first element of the sequence as 86. [More than one loop permitted but try it by using one loop] [Data array can be modified] 86 57 33 44 97 37 85 34 67 86 69 [Hint: Find index (p) such that location[p]=0 and increment the location array]
31. Insert the last element of the sequence as 86. [More than one loop permitted but try it by using one loop] [Data array can be modified] 57 33 44 97 37 85 34 67 86 69 86

Binary Search Tree (array based)

```
#include<stdio.h>
void put(int x[],int y,int z){ x[y]=z; }
int take(int x[],int y){ return(x[y-1]); }
void inorder(int h,int x[],int y[],int z[])
{    if (h!=-1)
     {  inorder(y[h],x,y,z);
        printf("%d ",x[h]);
        inorder(z[h],x,y,z);
     }
}
void print(int x[],int y[], int z[],int n)
{    int i;
     printf("\nindex"); for (i=1;i<=n;i++) printf("%3d ",i);
     printf("\ndata "); for (i=1;i<=n;i++) printf("%3d ",x[i]);
     printf("\nleft "); for (i=1;i<=n;i++) printf("%3d ",y[i]);
     printf("\nright"); for (i=1;i<=n;i++) printf("%3d ",z[i]);
}
main()
{  int data[50],left[50],right[50],sys[50];
   Int element,size=0,j,choice,i,root=-1,free=0;
   do{printf("\nGive choice: -1:put  -2:search  -3:DFT   -4:inorder");
      printf("  -5:print  -6:exit "); scanf("%d",&choice);
      if (choice==-1){
         printf("Give element to put ");scanf("%d",&element);
         size++;data[size]=element;left[size]=-1;right[size]=-1;
         if (root==-1) root=size;
         else{ i=root;
               while (i!=-1)
               {    j=i;
                    if (data[i]>element) i=left[i]; else i=right[i];
               }
               if (data[j]>element) left[j]=size; else right[j]=size;
             }}
      if (choice==-2){
         printf("Give element to search ");scanf("%d",&element);
         i=root;printf("Elements observed ");
         while ((i!=-1)&&(data[i]!=element))
         {   printf("%d ",data[i]);
             if (data[i]>element) i=left[i]; else i=right[i];
         }
         if (i!=-1) printf(" present"); else printf(" absent");}
      if (choice==-3){
         put(sys,free,root);free++;
         while (free!=0)
         {  i=take(sys,free);free--;printf("%d ",data[i]);
            if (left[i]!=-1) {put(sys,free,left[i]);free++;}
            if (right[i]!=-1){put(sys,free,right[i]);free++;}
         }
      }
      if (choice==-4) inorder(root,data,left,right);
      if (choice==-5) print(data,left,right,size);
      }while(choice!=-6);
}
```

After inserting 50, 20, 67, 89, 45, 56 and 58 in the given order the tree (data structure) looks as (root is 1)

```
index  1   2   3   4   5   6   7
data  50  20  67  89  45  56  58
left   2  -1   6  -1  -1  -1  -1
right  3   5   4  -1  -1   7  -1

              50

20                          67

      45              56              89

                              58
```

Output on search of various numbers are as following:

```
element to search 56    Elements observed 50 67  present
element to search 57    Elements observed 50 67 56 58  absent
element to search 18    Elements observed 50 20  absent
element to search 21    Elements observed 50 20 45  absent
```

1. In order traversal of the above tree is 20 45 50 56 58 67 89. Write program for pre order traversal. The pre order traversal of the tree is 50 20 45 67 56 58 89.
2. Write program for post order traversal. For above tree it is 45 20 58 56 89 67 50
3. Program for depth first traversal of the tree is given. The DFT is 50 67 89 56 58 20 45. Modify the program so that it becomes 50 20 45 67 56 58 89.
4. Modify functions put and take in the context of queue. Now breadth first traversal is found in place of depth first traversal. BFT is 50 20 67 45 56 89 58.
5. Another BFT is 50 67 20 89 56 45 58. Write program for both cases.
6. Write program to find depth of an element. Input 67 output 1. Input 58 output 3.
7. Write program to find father of a node. Input 67 output 50. Input 58 output 56.
8. Write program to find sum of all elements. Here it is 385.
9. Write program to output all elements below a given numbers. Input 67 output 56 58 89. Input 20 output 45. Input 50 output 20 45 67 56 58 89.
10. Do the program of pre order traversal using stack (do not use recursion).[Hint: pre order is special case of DFT, when right child is first put in the stack]
11. Do the program of post order traversal using stack (do not use recursion). [Hint: when left child is first put in the stack then DFT produces post order traversal in reverse].
12. Do the programs of in order traversal using stack (do not use recursion).
13. Modify above program, so that post order is generated. [Here only one stack is used]
14. Write program to find height of the tree. [Hint: use recursion]
15. Write program to find how many elements are even. [Hint: use recursion]
16. Write program to output all nodes at a given depth. Input 1 output 20 67. Input 3 output 58. input 2 output 45 56 89. [Hint: use recursion]
17. Whether there is any element, whose last digit is k. For input k=2 answer is no. For input 6 output is yes.

18. Write program, which outputs a number whose last digit is k. Assume that there is at least one such number. If there is more than one such number then any number can be outputted. Input k=6 output 56. Input k=0 output 50 (or 20).
19. Modify above program so that complete path from that number to the root is outputted. Input k=6 output 56 67 50. Input k=5 output 45 20 50.
20. Write program so that complete path from root to that number is outputted. Input k=6 output output 50 67 56. Input k=5 output 45 20 50.
21. Implement sequence tree. Add one more function delete.

Binary Search Tree (Pointer based)

```
#include <stdio.h>
typedef struct node{ float data; struct node *left,*right;}node;
node *root;
void put(float e)
{  node *t,*i,*j;
   t=(node *)malloc(sizeof(node));
   t->data=e;t->left=NULL;t->right=NULL;
   if (root==NULL) root=t;
   else
   {   i=root;
       while(i!=NULL)
       {  j=i;
          if(i->data>e) i=i->left; else i=i->right;
       }
       if (j->data>e) j->left=t; else j->right=t;
   }
}
int height(node* t)
{   int p,q;
    if (t==NULL) return -1;
    else
    {  p=height(t->left);
       q=height(t->right);
       if (p>q) return p+1;
       else return q+1;
    }
}
int count(node* head,int t)
{   int p,q,r;
    if (head==NULL) return 0;
    else
    {  p=count(head->left,t);
       q=count(head->right,t);
       if ((int)head->data%10==t) r=1;
       else r=0;
       return p+q+r;
    }
}
void preorder(node *head)
{   if (head!=NULL)
    {   printf("%5.1f ",head->data);
        preorder(head->left);
        preorder(head->right);}
}
main()
{   int t,i,n;float x;
    printf("Give n");scanf("%d",&n);
    for (i=1;i<=n;i++) {  scanf("%f",&x);put(x); }
    preorder(root);
    t=height(root);
    printf("height=%d\n",t);
    printf("Give last digit");scanf("%d",&t);
    i=count(root,t);
    printf("%d elements have last digit %d\n",i,t);
}
```

After inserting 49, 67, 53, 27, 19, 45, 37, 47, 86, 58 and 66 in an empty tree in the given order the tree looks as

```
                    49

        27                                  67

19          45                      53              86

          37      47                      58

                                              66
```

The function height returns the height of the tree. (4)
Count returns the number of elements whose last digit is ‘t’. Input 7 output 4.
Preorder outputs the preorder traversal of the tree. 49 27 19 45 37 47 67 53 58 66 86.

In the following problems recursion should not be used.

1. Define function void search(float e). It finds, whether an element ‘e’ is present in the tree or not. Path from root to the place of search is also printed.
   Input 58 output 49,67,53,58 present Input 22 output 49 27 19 absent
2. float father(float e). It returns the father of element e. Assume ‘e’ is present and is not at root. Input 58 output 53. Input 47 output 45.
3. void smallest( ). It outputs the smallest element of the tree. Output 19
4. void just_bigger(float e). It outputs the smallest number bigger than e. Assume that ‘e’ is present in the tree.[Caution: check for both cases (A) when e has right child (B) when e has no right child] Input 49 output 53. Input 27 output 37. Input 47 output 49.
5. void LL( ). It applies LL rotation at root. To check the correctness output the preorder traversal. Preorder is 27 19 49 45 37 47 67 53 58 66 86.
6. void LR( ). It applies LR rotation at root. Preorder 45 27 19 37 49 47 67 53 58 66 86.
7. void LL(float e). It applies LL rotation at element e. Assume ‘e’ has left child. LL rotation at 67 will make the tree as 49 27 19 45 37 47 53 67 58 66 86

In the following problems recursion should be used.

8. float sum(node *t). It returns the sum of all elements. (554)
9. float sld(node *t). It returns the number with smallest last digit. (53)
10. float just_bigger(node *t, float e). It returns the smallest number bigger than e. If all elements are smaller than e then -1 is returned. Input 50 output 53. Input 28 output 37.
11. Modify above so that complete path from root to that node is printed. Input 28 output 49 27 45 37. The path may be printed in reverse.
12. float bigget_even(node *t) to return biggest even number. If there is no even number then -1 is returned. (86)
13. float smallest_even(node *t) to return smallest even number. If there is no even number then -1 is returned. (58)
14. float first_e(node *t, int e) to return first number whose last digit is e. If there is no even number then -1 is returned. Input 5 output 45. Input 8 output 58.
15. node* ptrsld(node *t). It returns the pointer of the number with smallest last digit.
16. void leaf(node *t). It outputs all leafs of the tree. 19 37 47 66 86

17. void depth( ). It output depth of all nodes. [Hint: Define one more function void dpth(node *t, int p). Call dpth(root,0).] (49 0)(27 1) (19 2) (45 2)(37 3)(47 3)(67 1)(53 2)(58 3)(66 4)(86 2)
18. Define suitable function to return the element at maximum depth. In case of equal depth any element may be returned. More than one function may be needed. (66)
19. ads
20. adg
21. adg
22. adg
23. adg

Binary Tree (Pointer based)

```
#include <stdio.h>
typedef struct node{ float data; struct node *left,*right;}node;
node* search(node* root,float e)
{  node *a,*b;
   if (root->data==e) return root;
   else
   {  if ((root->left!=NULL) && (root->right!=NULL))
      {  a=search(root->left,e); b=search(root->right,e);
         if (a!=NULL) return a; else return b;}
      if ((root->left!=NULL) && (root->right==NULL))
      {  a=search(root->left,e); return a;  }
      if ((root->left==NULL) && (root->right!=NULL))
      {  b=search(root->right,e); return b;  }
      if ((root->left==NULL) && (root->right==NULL)) return NULL;
   }
}
```

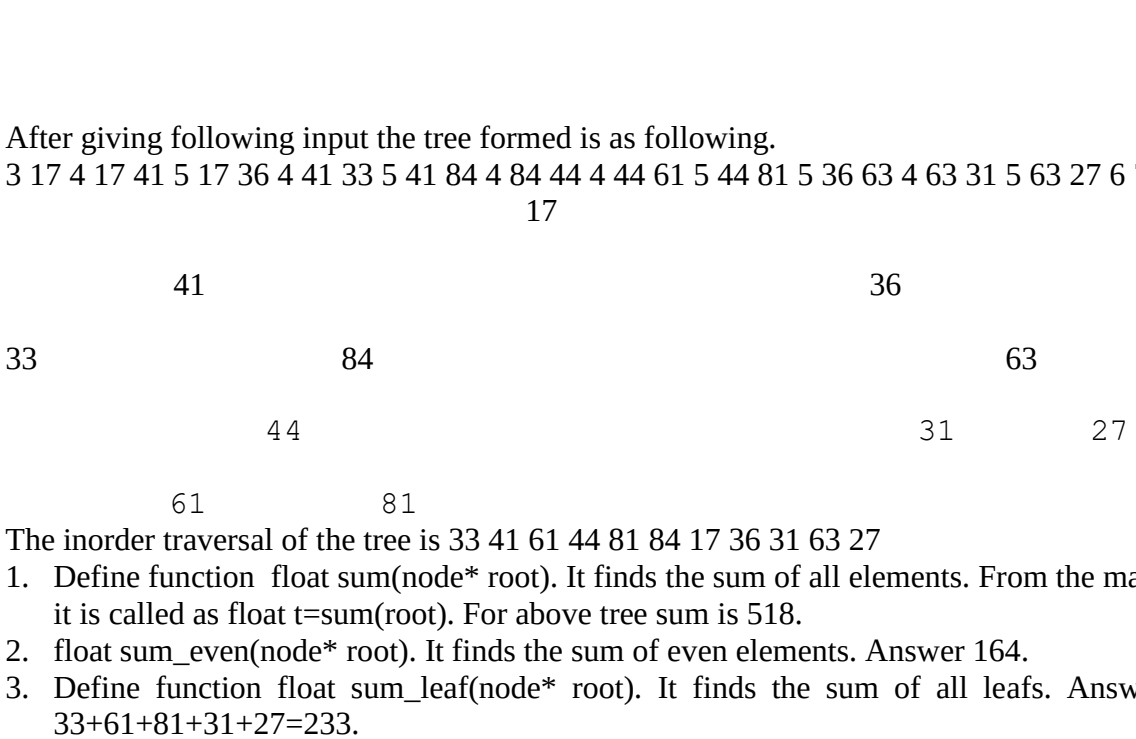
Search may be written as following also.

```
node* search(node* root,float e)
{  node *a,*b;
   if (root==NULL) return root;
   if (root->data==e) return root;
   a=search(root->left,e);
   b=search(root->right,e);
   if (a!=NULL) return a;
   return b;
}
```

```
void inorder(node *head)
{   if (head!=NULL)
    {   inorder(head->left);
        printf("%5.1f ",head->data);
        inorder(head->right);}
}
void bft(node *root)
{   node * a[100],*t;int i,j;
    a[0]=root;i=0;j=1;
    while (i<j)
    {  t=a[i];printf("%5.1f",t->data);i++;
       if (t->left!=NULL) {a[j]=t->left;j++;}
       if (t->right!=NULL) {a[j]=t->right;j++;} }
}
main()
{   int i,choice;float y,z;node *root,*t,*s;
    do
    {   printf("Give choice 1:inorder  2:search  3:insert at root
                4:insert at left  5:insert at right  6:BFT  7:exit");
        scanf("%d",&choice);
        if (choice==1) inorder(root);
        if (choice==2) {printf("Give element to search");scanf("%f",&y);
            t=search(root,y);
            if (t==NULL) printf("absent\n");else printf("present\n"); }
        if (choice==3) {printf("Give element to put at root");
            scanf("%f",&y);t=(node *)malloc(sizeof(node));
            t->data=y;t->left=NULL;t->right=NULL;root=t;}
        if (choice==4) {printf("Where and what to put?");
            scanf("%f %f",&y,&z);t=(node *)malloc(sizeof(node));
```

```
            s=search(root,y);if (s!=NULL)s->left=t;
            t->data=z;t->left=NULL;t->right=NULL;}
        if (choice==5) {printf("Where and what to put?");
            scanf("%f %f",&y,&z);t=(node *)malloc(sizeof(node));
            s=search(root,y);if (s!=NULL)s->right=t;
            t->data=z;t->left=NULL;t->right=NULL;}
        if (choice==6) bft(root);
    }while(choice!=7);
}
```

After giving following input the tree formed is as following.
3 17 4 17 41 5 17 36 4 41 33 5 41 84 4 84 44 4 44 61 5 44 81 5 36 63 4 63 31 5 63 27 6 7



The inorder traversal of the tree is 33 41 61 44 81 84 17 36 31 63 27

1. Define function float sum(node* root). It finds the sum of all elements. From the main it is called as float t=sum(root). For above tree sum is 518.
2. float sum_even(node* root). It finds the sum of even elements. Answer 164.
3. Define function float sum_leaf(node* root). It finds the sum of all leafs. Answer 33+61+81+31+27=233.
4. float sum_non_leaf(node* root). It finds the sum of all non leafs. Answer 285. Do not use subtraction.
5. float sum_special(node* root). It finds the sum of those elements which have two children. Answer 17+41+44+63=165.
6. float sum_specal(node* root). It finds the sum of elements at height 1. ans 44+63=107
7. int count(node* root). It finds the number of elements. Answer 11.
8. int count_even(node* root). It finds the number of even elements. Answer 3.
9. float sum_depth(node* root, int k). It finds the sum of those elements, which are at depth k. For k=3 answer 44+31+27=102.
10. float max(node* root). It finds the maximum element. Answer 84.
11. node* max_ptr(node* root). It finds the pointer to the maximum element. To check the result { node* t; t=max_ptr(root); printf("%f",t_data);} Answer 84.
12. float max_odd(node* root). It finds the maximum odd element. Answer 81.
13. int height(node* root). It finds the height of the tree. Answer 4.
14. float father(node* root, float e). It finds the father of element e. For e=44 answer is 84. If father is absent then -1 is returned.
15. float sum_sons(node* root, float e). It finds the sum of children of 'e'. For e=44 answer is 61+81=142. For e=84 it is 44. For e=41 it is 33+84=117.
16. float smallest_even(node* root). It finds the smallest even element. Answer 36.
17. float first_even_inorder(node* root). It finds the first even element in inorder traversal. ans 44.
18. Define function to find first even element in preorder traversal. Answer 84.
19. Define function of output all leafs. Answer 33 61 81 31 27.
20. Define function to output pre order traversal. 17 41 33 84 44 61 81 36 63 31 27.
21. node* LL(node* root) to apply LL rotation at root. Pre order traversal after LL rotation at root will be
22. LR rotation at root. Pre order will be
23. node* LL(node* root, float e). LL rotation at element e.
    If e is 41 then preorder will be
    For e= 84 it will be

24. Similarly do for LR raotaion. For e=41 the preorder will be

Do not use recursion in following programs.

25. Program to output depth first traversal. 17 36 63 27 31 41 84 44 81 61 33 (reverse of postorder)
26. Modify above program so that depth first is 17 41 33 84 44 61 81 36 63 31 27 (preorder)
27. Write program for inorder traversal. 33 41 61 44 81 84 17 36 31 63 27

Multi child Tree (Generalized link list)

```
#include <stdio.h>
typedef struct node{ int data,type; struct node *first,*next;}node;
void remove_blank(char a[])
{  int i=0,j=0;
   while (a[i]!='\0')
   {  if (a[i]!=' ') {a[j]=a[i];j++;}
      i++;}
   a[j]='\0';
}
int decompose(char a[], char b[], char c[])
{  int i,j,k,count;
   if (a[0]!='(') return 0;
   else if (a[1]==')') return 1;
   else if (a[1]!='(')
   {  b[0]=a[1];b[1]=a[2];b[2]='\0';
      i=3;c[0]='(';
      while (a[i]!='\0') {c[i-2]=a[i];i++;}
      c[i-2]='\0'; return 2; }
   else
   {   i=1;count=0;
       do{  if(a[i]=='(') count++;if(a[i]==')') count--;
            b[i-1]=a[i];i++;
       } while (count!=0);
       b[i-1]='\0';c[0]='(';j=1;
       while (a[i]!='\0') {c[j]=a[i];i++;j++;}
       c[j]='\0'; return 2;
   }
}
node* construct(char a[])
{   int p;node *x,*y,*t;char b[200],c[200];
    p=decompose(a,b,c);
    if (p==1) return NULL;
    t=(node *)malloc(sizeof(node));
    if (p==0) {t->type=0;t->data=(a[0]-48)*10+a[1]-48; return t;}
    x=construct(b);y=construct(c);
    t->type=1;t->first=x;t->next=y; return t;
}
void print(node *root)
{   if (root==NULL) printf("()");
    else if (root->type==0) printf("%d ",root->data);
    else{ printf("(");
          while (root!=NULL)
          {   print(root->first);
              root=root->next; }
          printf(")");}
}
int sum(node *root)
{   int s=0,t;
    if (root==NULL) return 0;
    else if (root->type==0) return root->data;
    else{ while (root!=NULL)
          {   t=sum(root->first);s=s+t;
              root=root->next; }
           return s; }
}
```

```
main()
{   char a[200],b[200],c[200];int p;node *root,*t;
    printf("Give a tree");gets(a);remove_blank(a);
    root=construct(a);
    print(root);printf("\nThe sum=%d\n",sum(root));
    node *s;int sum=0;t=root;
    while(t!=NULL)
    {  s=t->first;
       if (s->type==0) sum=sum+s->data;
       t=t->next;
    }
    printf("The sum of base lavel elements %d",sum);
}
```

1. Given a list of two pairs. Find product of numbers of every pair. Find its sum. Input((12 16)(34 10)) output 12*16+34*10=532. No loop should be used.
2. Given a list of two pairs. Form a average pair. For above input (23 13). (No loop)
3. Given a list of many pairs. Find product of numbers of each pair. Find their sum. Input((12 16)(34 10)(15 20)(22 11)) output 1074. (loop needed)
4. Form an average pair (20 14)

The above program takes input a general list. For the following input the sum is 362.
((77 24 27)(11 (19 23) 18) 16 (44 (28 (27) 29)) 19) All numbers must be in two digits.
The sum of all elements at base level is 16+19=35.

5. Program to exchange first and second element. No printing should be done. Print list by calling print(root). ((11 (19 23) 18) (77 24 27) 16 (44 (28 (27) 29)) 19)
6. Exchange first two elements of first element. ((24 77 27)(11 (19 23) 18) …)
7. Exchange first elements of first two elements. ((11 24 27)(77 (19 23) 18) …)
8. Program to output first element of first element. Assume that the first element of the given list is a list. Output 77.
9. Delete first element of first element. ((24 27)(11 (19 23) 18) 16 (44 (28 (27) 29)) 19)
10. Read a number and put it at the beginning of the list. Hint: Use malloc twice. Let input is 45 then list is (45(77 24 27)(11 (19 23) 18) 16 (44 (28 (27) 29)) 19). Printing should be done using print(root) only.
11. Put number at second position. ((77 24 27) 45 (11 (19 23) 18) 16 (44 (28 (27) 29)) 19)
12. Put number at first place in first element. ((45 77 24 27)(11 (19 23) 18) 16 (44 (28 (27) 29)) 19)
13. Put number at the end. ((77 24 27)(11 (19 23) 18) 16 (44 (28 (27) 29)) 19 45)
14. Read two numbers and form list. Input 45 67. List (45 67). Print list by calling print root. [Hint: use malloc 4 times]
15. Program to output all elements in separate line. Output (77 24 27), (11(19 23)18), 16, (44 (28 (27) 29)), 19. They are outputted in 5 lines.
16. Program to output the kth element. For k=4 output (44 (28 (27) 29)).
17. Program to output p^{th} element of first element. For p=3 output 27.
18. Program to output p^{th} element of kth element. Assume that k^{th} element is a list. For k=4 and p=2 output (28(27)29). For k=2 and p=3 output 18. k=p=2 output (19 23).
19. Program to output the maximum of all numbers at base level. Output 19.
20. Find number of elements in every element at base level. Find their sum of squares. In above case $3^2+3^2+1^2+2^2+1^2=24$.
21. int max(node* root) to return the maximum of all numbers. output 77.
22. int height(node* root) to return the height of the tree. Output 3. (recursive function)
23. int sm(node* root, int k) to find the sum of the all numbers at depth k. For k=1 the output is 16+19=35. For k=2 it is 77+24+27+11+18+44=201. For k=3 it is 19+23+28+29=99. For k=4 it is 27.
24. Read a list and put it at the beginning. Hint: Call remove_blank and construct. Let input is (12(34 56)) then list is ((12(34 56))(77 24 27)(11 (19 23) 18) 16 (44 (28 (27) 29)) 19).

The father of 29 is the list (28 (27) 29). Following function returns the pointer of father.
The father of 44 is (44 (28 (27) 29)).

The father of 16 is ((77 24 27)(11 (19 23) 18) 16 (44 (28 (27) 29)) 19)

```
node* father(node *root,int a)
{   node *i,*t,*j;j=root;
    while (j!=NULL)
    {   i=j->first;
        if ((i->type==0) && (i->data==a)) return root;
        if (i->type==1){ t=father(i,a); if (t!=NULL) return t; }
        j=j->next;
    }
    return NULL;
}
main()
{   printf("Whose father do you want");scanf("%d",&p);
    t=father(root,p);print(t);
}
```

25. Read a number. Find the sum of all numbers in the list, in which this number is found. For input 29 output is 28+27+29=84.
26. Delete second element of that list. For input 29 the output is ((77 24 27)(11 (19 23) 18) 16 (44 (28 29)) 19). For input 19 the output is ((77 24 27) 16 (44 (28 (27) 29)) 19)
27. Output the last element of that list. Input 29 output 29. Input 77 output 27.
28. Find sum of base level. For input 29 output 28+29=57.

Binary Relation (many one)

```
#include<stdio.h>
main()
{  int m,n,i,j,x,y,next[100];
   printf("Give m and n");scanf("%d%d",&m,&n);
   for (i=1;i<=m+n;i++) scanf("%d",&next[i]);
   printf("Give x to find (?,x) x>m");scanf("%d",&x);
   i=next[x];
   while (i>0) i=next[i];
   printf("%d",-i);
   printf("Give y to find (y,?) y≤m");scanf("%d",&y);
   i=next[y];
   while (i>0)
   {  printf("%d ",i);
      i=next[i];
   }
}
```

Let P={1,2,3,4,5) Q={6,7,8,9,10,11,12,13,14,15,16}. Here m=5 and n=11
There is a many one retation ($R \subseteq PxQ$)
R={(3,13),(2,11),(5,8),(3,6),(1,10)(2,7),(5,12),(3,9),(3,16),(2,15),(3,14)}.

Owner Sets: 1={10} 2={11,7,15} 3={13,6,9,16,14} 4={} 5={8,12}
Representation

Index	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Location	10	11	13	-4	8	9	15	12	16	-1	7	-5	6	-3	-2	14

Here m and n are sizes of P and Q respectively.

1. Write program, which reads m and n and a sequence of n pair of numbers. Let m=5 and n=11 and pairs are (3,14)(2,15)(3,16)(3,9)(5,12)(2,7)(1,10)(3,6)(5,8)(2,11)(3,13) then above structure will be outputted. [To insert a pair (x,y) use next[y]=next[x] next[x]=y] [Initially next[i]=-i for i=1..m]

Binary Relation (many many)

```
#include<stdio.h>
main()
{  int m,n,r,t,i,j,x,y,next[100],next1[100],next2[100];
   printf("Give m and n");scanf("%d%d",&m,&n);
   for (i=1;i<=m+n;i++) next[i]=-i;
   printf("Give number of pairs");scanf("%d",&r);
   for (i=1;i<=r;i++)
   { t=m+n+i;
     scanf("%d %d",&x,&y);
     next1[t]=next[x];
     next2[t]=next[y];
     next[x]=t;next[y]=t;
   }
   printf("\n");for (i=1;i<=m+n;i++) printf("%3d ",i);
   printf("\n");for (i=1;i<=m+n;i++) printf("%3d ",next[i]);
   printf("\n");for (i=m+n+1;i<=m+n+r;i++) printf("%3d ",i);
   printf("\n");for (i=m+n+1;i<=m+n+r;i++) printf("%3d ",next1[i]);
   printf("\n");for (i=m+n+1;i<=m+n+r;i++) printf("%3d ",next2[i]);
   printf("\nGive x to find (x,?)");
   scanf("%d",&x);
   i=next[x];
   while (i>0)
   {  j=next2[i];
      while (j>0) j=next2[j];
      printf("%d ",-j);
      i=next1[i];
   }
}
```

Let P={1,2,3,4) Q={5,6,7,8,9,10}. Here m=4 and n=6
There is a many many retation ($R \subseteq PxQ$)
R={(3,5)(2,10)(1,7)(4,5)(2,6)(1,9)(4,7)(2,7)(2,9)(4,10)(1,6)(4,6)}
11 12 13 14 15 16 17 18 19 20 21 22

Owner sets: 1={21,16,13} 2={19,18,15,12} 3={11} 4={22,20,17,14}
5={14,11} 6={22,21,15} 7={18,17,13} 8={ } 9={19,16} 10={20,12}

Index	1	2	3	4	5	6	7	8	9	10
Next	21	19	11	22	14	22	18	-8	19	20

Index	11	12	13	14	15	16	17	18	19	20	21	22
$Next_1$	-3	-2	-1	-4	12	13	14	15	18	17	16	20
$Next_2$	-5	-10	-7	11	-6	-9	13	17	16	12	15	21

Dynamic Set [Hashing: Chain based]

```
#include<stdio.h>
int hash(int p){ return (p%10+p/10)%5; }
int insert(int x[],int y[],int free,int z){int i=hash(z); missing; return free;}
void delete(int x[],int y[],int z){ int i=hash(z); missing;   }
int search(int x[],int y[],int z){ int t,i=hash(z); missing;  return t;  }
void print(int x[],int y[], int free)
{  int i;
   printf("\nindex");for (i=0;i<free;i++) printf("%3d ",i);
   printf("\ndata ");for (i=0;i<free;i++) printf("%3d ",x[i]);
   printf("\nnext ");for (i=0;i<free;i++) printf("%3d ",y[i]);
}
main()
{  int data[50],next[50],element,free=5,t,choice,i;
   for (i=0;i<free;i++) { data[i]=-1; next[i]=-1; }
   do{printf("\nGive choice: 0:put  1:take  2:search ");
      printf(" 3:printing set  4: exit  ");
      scanf("%d",&choice);
      if (choice==0){
         printf("Give element to put ");scanf("%d",&element);
         free=insert(data,next,free,element);}
      if (choice==1){
         printf("Give element to delete ");scanf("%d",&element);
         delete(data,next,element);}
      if (choice==2){
         printf("Give element to search ");scanf("%d",&element);
         t=search(data,next,element);
         if (t==0) printf("  absent"); else printf("  present");}
      if (choice==3)print(data,next,free);
      }while(choice!=4);
}
```

1. Write missing portion in function Insert so that after inserting 12 41 29 the structure appears as following: (size=5)

Index	0	1	2	3	4	5	6	7	8	9	10	11	12
Data	41	29	-1	12	-1								
Next	-1	-1	-1	-1	-1								

2. After inserting 12 41 80 29 60 44 35 the structure appears as following: (size=9)

Index	0	1	2	3	4	5	6	7	8	9	10	11	12
Data	41	29	-1	12	-1	80	60	44	35				
Next	-1	6	-1	8	-1	-1	-1	5	7				

3. Write missing portion in search so that the successful search is properly carried out.
   The search of 29 outputs “Locations observed 1 present”.
   The search of 44 outputs “Locations Observed 3 8 7 present”.
4. Update search for unsuccessful search also. For 83“Locations observed 1 6 -1 absent”.
5. Write missing portion in function delete so that after deleting 44 and 29 the structure appears as following: (size=9)

Index	0	1	2	3	4	5	6	7	8	9	10	11	12
Data	41	-1	-1	12	-1	80	60	44	35				
Next	-1	6	-1	8	-1	-1	-1	5	5				

The search of 44 outputs “Locations observed 3 8 5 -1 absent”.

Now after inserting 33 and 50 the structure appears as follows: (size=10)

Index	0	1	2	3	4	5	6	7	8	9	10	11	12
Data	41	33	-1	12	-1	80	60	44	35	50			
Next	9	6	-1	8	-1	-1	-1	5	5	-1			

Dynamic Set (using variable size)[Binary Search]

```
#include<stdio.h>
void insert(int x[],int size,int y) {x[size]=y; }
void delete(int x[],int size,int y)
{  int i=0;
   while (x[i]!=y) i++;
   x[i]=x[size-1];
}
int search(int x[],int size,int y)
{  int i=0;
   printf("locations observed:    0");
   while ((i<size)&&(x[i]!=y)) {i++;printf("%3d",i);}
   if (i==size) return 0; else return 1;
}
void print(int x[],int size)
{  int i;
   for (i=0;i<size;i++) printf("%3d ",i);
   printf("\n");
   for (i=0;i<size;i++) printf("%3d ",x[i]);
}
main()
{  int x[50],element,size=0,t,choice;
   do{printf("\nGive choice: 0:put  1:take  2:search ");
      printf(" 3:printing set  4: exit  ");
      scanf("%d",&choice);
      if (choice==0){
         printf("Give element to put ");scanf("%d",&element);
         insert(x,size,element);size++;}
      if (choice==1){
         printf("Give element to delete ");scanf("%d",&element);
         delete(x,size,element);size--;}
      if (choice==2){
         printf("Give element to search ");scanf("%d",&element);
         t=search(x,size,element);
         if (t==0) printf("absent"); else printf("present");}
      if (choice==3)print(x,size);
      }while(choice!=4);
}
```

1. Modify the functions for insert, delete and search to maintain sorted array.
   (A) To insert an element we find first element in array x, which is bigger than y. y is put there. Other elements are shifted. If y is bigger than all elements then it is put at last.
   (B) To delete an element other elements are shifted.
   (C) Binary search is used for searching. Let search of 45 is done on following array (size 14) then output will be "locations observed 6, 2, 4 present"

X_0	X_1	X_2	X_3	X_4	X_5	X_6	X_7	X_8	X_9	X_{10}	X_{11}	X_{12}	X_{13}	X_{14}	X_{15}	X_{16}
12	17	29	32	45	68	71	73	75	77	79	80	82	83			

2. During deletion the element is replaced by corresponding negative element. After deleting 77 and 29 the array will be as following. (size will remain unchanged as 14)

X_0	X_1	X_2	X_3	X_4	X_5	X_6	X_7	X_8	X_9	X_{10}	X_{11}	X_{12}	X_{13}	X_{14}	X_{15}	X_{16}
12	17	-29	32	45	68	71	73	75	-77	79	80	82	83			

(A) In above case modify the search function [Hint: if abs(x[mid])>y) …]

(B) Modify the insert function. When an element is inserted then shifting is done only up to negative element. After inserting 42 and 81 the array will be as following (size will become 15 (not 16)). Since during insert the 'size' may or may not change hence prototype is int insert(int x[],int size,int y). It is called as size=insert(x,size,element).

X_0	X_1	X_2	X_3	X_4	X_5	X_6	X_7	X_8	X_9	X_{10}	X_{11}	X_{12}	X_{13}	X_{14}	X_{15}	X_{16}
12	17	-29	32	42	45	68	71	73	75	79	80	81	82	83		

3. sg
4. sdg
5. dsg
6. ds
7. gd
8. sfg
9. dsf

Dynamic Set (using constant size) [Hashing: open Address]

```
#include<stdio.h>
void insert(int x[],int size,int y)
{  int i=0;
   while (x[i]!=0) i++;
   x[i]=y;
}
void delete(int x[],int size,int y)
{  int i=0;
   while (x[i]!=y) i++;
   x[i]=0;
}
int search(int x[],int size,int y)
{  int i=0;
   printf("locations observed:    0");
   while ((i<size)&&(x[i]!=y)) {i++;printf("%3d",i);}
   if (i==size) return 0; else return 1;
}
void print(int x[],int size)
{  int i;
   for (i=0;i<size;i++) printf("%3d ",i);
   printf("\n");
   for (i=0;i<size;i++) printf("%3d ",x[i]);
}
main()
{  int x[50],element,size=0,t,choice,i;
   printf("Give size ");scanf("%d",&size);
   for (i=0;i<size;i++) x[i]=0;
   do{printf("\nGive choice: 0:put  1:take  2:search ");
      printf(" 3:printing set  4: exit  ");
      scanf("%d",&choice);
      if (choice==0){
         printf("Give element to put ");scanf("%d",&element);
         insert(x,size,element);}
      if (choice==1){
         printf("Give element to delete ");scanf("%d",&element);
         delete(x,size,element);}
      if (choice==2){
         printf("Give element to search ");scanf("%d",&element);
         t=search(x,size,element);
         if (t==0) printf("absent"); else printf("present");}
      if (choice==3)print(x,size);
      }while(choice!=4);
}
```

1. Insert an element at location "home: sum of digits". It is hash function. Let size be 17. Assume all inputs are two digit numbers. After inserting 92 76 73 52 23 69 31 the table will be following:

X_0	X_1	X_2	X_3	X_4	X_5	X_6	X_7	X_8	X_9	X_{10}	X_{11}	X_{12}	X_{13}	X_{14}	X_{15}	X_{16}
0	0	0	0	31	23	0	52	0	0	73	92	0	76	0	69	0

2. When a location is occupied then the element is inserted at next free location. After inserting 55 and 41 the table will be as following:

X_0	X_1	X_2	X_3	X_4	X_5	X_6	X_7	X_8	X_9	X_{10}	X_{11}	X_{12}	X_{13}	X_{14}	X_{15}	X_{16}
0	0	0	0	31	23	41	52	0	0	73	92	55	76	0	69	0

3. During search an element is observed from home till element (or blank location). When 55 is searched then locations observed are 10 11 12. When 32 is searched then locations observed are 5 6 7 8.
4. When an element is deleted then corresponding location is freed by '-1'. After deleting 92, 55, 23 the table will be

X_0	X_1	X_2	X_3	X_4	X_5	X_6	X_7	X_8	X_9	X_{10}	X_{11}	X_{12}	X_{13}	X_{14}	X_{15}	X_{16}
0	0	0	0	31	-1	41	52	0	0	73	-1	-1	76	0	69	0

5. However, if next location is free then 0 can be put. (e.g deletion of 69).
6. When an element is deleted and next location is free(0) and previous locations are '-1' then they can also be made as 0. After deleting 76 the table will be

X_0	X_1	X_2	X_3	X_4	X_5	X_6	X_7	X_8	X_9	X_{10}	X_{11}	X_{12}	X_{13}	X_{14}	X_{15}	X_{16}
0	0	0	0	31	-1	41	52	0	0	73	0	0	0	0	69	0

7. gdsf
8. gd

Expression Evaluation

1. Define a function int f(int a, int b, int c). If c is -91 then value returned is a+b, If it is -92 then a-b, if -93 then a*b, if -94 then a/b.
   main( ){int p,q,r,s;printf("Give 3 numbers");scanf(p,q,r);s=f(p,q,r); print(s);}
   If input numbers are 6,7,-93 then output is 42.
2. Define functions for reading and writing arrays as follows.

```
void rd(int a[],int n)        void wr(int a[],int n)
{  int i;                     {  int i;printf("\narray ");
   for (i=1;i<=n;i++)            for (i=1;i<=n;i++) printf("%d,",a[i]);
      scanf("%d",&a[i]);         printf("\n");
}                             }
```

   Now execute the following main program
   main( ) { int x[20],size; printf("Give size");scanf("%d",&size);rd(x,size);wr(x,size); }
   Let input n=9 and array is 5,70,60,-91,15,8,-93,-92,-91.
   It correspond to the post fix expression 5,70,60,+,15,8,*,-,+
3. Define a function int g(int a[], int k). It returns the value of f(a[k-2],a[k-1],a[k]). For k=7 the output is 15*8=120.
   main( ) { int x[20],n,k,t; printf("Give n,k");scanf(n,k);rd(x,n);t=g(x,k);print(t); }
4. Define void del(int a[], int n, int k). It deletes the kth element. For k=2 the final array will be 5,60,-91,15,8,-93,-92,-91.
   main( ) { int x[20],n,k; printf("Give n,k");scanf(n,k);rd(x,n);del(x,n,k);wr(x,n-1); }
5. void h(int a[], int n, int k). It replaces the kth element by f(a[k-2], a[k-1], a[k])] (i.e. g(a,k)). For k=7 the array will be 5,70,60,-91,15,8,120,-92,-91. [In function 'h' call 'g' (do not call 'f')]
   main( ) { int x[20],n,k; printf("Give n,k");scanf(n,k);rd(x,n);h(x,n,k);wr(x,n); }
6. void hh(int a[], int n, int k). It calls 'h' and deletes $(k-1)^{th}$ and $(k-2)^{th}$ elements. [Hint: call h and delete (care is needed in calling delete)]. For k=7 the final array will be 5,70,60,-91,120,-92,-91.
7. int loc(int a[], int n). It returns the location of first negative element. For above input the location of first negative element is 4.
   main( ) { int x[20],n,t; printf("Give n");scanf(n);rd(x,n);t=loc(x,n);print(t); }
8. void step(int a[], int n) to perform a step in the evaluation of post fix expression. It finds the location (k) of first negative element. After that it calls hh(a,n,k). For above input 5,130,15,8,-93,-92,-91.
   main( ){ int x[20],n; scanf(n);rd(x,n);step(a,n);wr(x,n-2); }
9. int value(int a[], int n) to find the value of a postfix expression. [Hint: In a loop call function step (n/2) times. Print array after every step. Finally it returns a[1]] Output in above case will be 15.
   . main( ){ int x[20],n,t; printf("Give n");scanf(n);rd(x,n);t=value(a,n);print(t); }
   (A) The array after second step 5,130,120,-92,-91.
   (B) The array after third step 5,10,-91.
   (C) The array after fourth step 15.

Modify above functions. Take n=11 and input array -91,-93,4,-92,7,2,-92,-91,2,9,5.
It corresponds to the pre fix expression +,*,4,-,7,2,-,+,2,9,5.

10. The function g returns the value of f(a[k+1],a[k+2],a[k]). For k=8 output is 2+9=11.
11. The function 'h' replaces the kth element by f(a[k+1], a[k+2], a[k]) (i.e. g(a,k)) For k=8 the final array will be -91,-93,4,-92,7,2,-92,11,2,9,5 [No modification in function h is needed if function 'g' is called]
12. The function 'hh' deletes $(k+1)^{th}$ and $(k+2)^{th}$ elements. -91,-93,4,-92,7,2,-92,11,5
13. The function 'loc' returns the smallest 'i' such that a[i+1] and a[i+2] are both positive. Here it is 4 because 5th and 6th elements are 7 and 2 (positive).
14. The function 'step' performs a step in the evaluation of pre fix expression. For above input the final array will be -91,-93,4,5,-92,-91,2,9,5. [No modification needed]
15. The function 'value' finds the value of a pre fix expression. [No modification needed]
    The array after second step -91,20,-92,-91,2,9,5.
    Third: -91,20,-92,11,5. Fourth: -91,20,6. Fifth: 26

Modify above functions. Take n=11 and input array 18,-91,5,-93,14,-92,42,-94,5,-91,16.
It corresponds to infix expression 18,+,5,*,14,-,42,÷,5,+,16.

16. The function g returns the value of f(a[k-1],a[k+1],a[k]). For k=4 output is 5*14=70.
17. The function 'h' replaces the kth element by f(a[k-1], a[k], a[k+1]). For k=4 the final array will be 18,-91,5,70,14,-92,42,-94,5,-91,16
18. Function 'hh' deletes $(k-1)^{th}$ and $(k+1)^{th}$ elements.18,-91,70,-92,42,-94,5,-91,16.
19. The function 'loc' returns the location of smallest element. Here it is 8 because 8th element is smallest (-94). It corresponds to division. Hence division is done first.
20. The function 'step' performs a step in the evaluation of infix expression. [No modification needed] The final array will be 18,-91,5,-93,14,-92,8,-91,16.
21. The function 'value' finds the value of an infix expression. [No modification needed]
    The array after second step 18,-91,70,-92,8,-91,16.
    Third: 18,-91,62,-91,16. Fourth: 80,-91,16 Fifth: 96

Expression Evaluation (link list based)

28. Define a function int f(int a, int b, int c). If c is -91 then value returned is a+b, If it is -92 then a-b, if -93 then a*b, if -94 then a/b. e.g. input numbers 6,7,-93 output is 42.
    main( ){int p,q,r,s;printf("Give 3 numbers");scanf(p,q,r);s=f(p,q,r); print(s);}
29. Write function for constructing and printing link list. Let input n=9 and list is 5,70,60,-91,15,8,-93,-92,-91. (Corresponding post fix expression 5,70,60,+,15,8,*,-,+)
30. Define a function int kth(node *head, int k). It returns the kth element. For k=4 the output is -91. For k=2 output is 70.
31. int g(head *node, int k). It returns the value of f(p,q,r). Here p is $(k-2)^{th}$, q is $(k-1)^{th}$ and r is k^{th} element. For k=7 the output is 15*8=120. The main program is
    node *head; int k,t; printf("Give k");scan(k);head=construct();t=g(head,k);printf(t);
32. node* del(node *head, int k). It deletes the kth element. For k=2 the final list will be 5,60,-91,15,8,-93,-92,-91. The main program is
    int k; printf("Give k");scan(k);head=construct();head=del(head,k);print(head);
33. node* h(node *head, int k). It replaces the kth element by g(head,k). For k=7 the list will be 5,70,60,-91,15,8,120,-92,-91. [In function 'h' call 'g' (do not call 'f')]
    main: scan(k); head=construct( ); head=h(head,k);print(head);
34. node* hh(node *head, int k). It calls h and deletes $(k-1)^{th}$ and $(k-2)^{th}$ elements. (care is needed in calling delete)]. For k=7 the final list will be 5,70,60,-91,120,-92,-91.
    main: head=construct( ); head=hh(head,k);print(head);
35. int loc(node *head). It returns the location of first negative element. For above input the location of first negative element is 4. main: int t; t=loc(head); printf(t);
36. node* step(head *node) to perform a step in the evaluation of post fix expression. It finds the location (k) of first negative element. After that it calls hh(head,k). For above input the final list is 5,130,15,8,-93,-92,-91. main: head=step(head); print(head);
37. int value(head *node) to find the value of a postfix expression. [Hint: In a loop call function step (n/2) times. Print list after every step. Finally it returns the first element.] The list after second step 5,130,120,-92,-91. Third step 5,10,-91. fourth step 15.

Modify above functions. Take n=11 and input list -91,-93,4,-92,7,2,-92,-91,2,9,5.
It corresponds to the pre fix expression +,*,4,-,7,2,-,+,2,9,5.

38. The function g returns the value of f(p,q,r). Here p, q and r are $(k+1)^{th}$, $(k+2)^{th}$ and k^{th} elements respectively. For k=8 output is 2+9=11.
39. The function 'h' replaces the kth element by g(head,k). For k=8 the final list will be -91,-93,4,-92,7,2,-92,11,2,9,5 [No modification in function h is needed]
40. The function 'hh' deletes $(k+1)^{th}$ and $(k+2)^{th}$ elements. -91,-93,4,-92,7,2,-92,11,5
41. The function 'loc' returns the smallest 'i' such that $(i+1)^{th}$ and $(i+2)^{th}$ elements are both positive. Here it is 4 because 5th and 6th elements are 7 and 2 (positive).
42. The function 'step' performs a step in the evaluation of pre fix expression. For above input the final list will be -91,-93,4,5,-92,-91,2,9,5. [No modification needed]
43. The function 'value' finds the value of a pre fix expression. [No modification needed]
    The list after second step -91,20,-92,-91,2,9,5.
    Third: -91,20,-92,11,5. Fourth: -91,20,6. Fifth: 26

Modify above functions. Take n=11 and input list 18,-91,5,-93,14,-92,42,-94,5,-91,16.
It corresponds to infix expression 18,+,5,*,14,-,42,÷,5,+,16.

44. The function 'g' returns the value of f(p,q,r). Here p, q and r are $(k-1)^{th}$, $(k+1)^{th}$ and k^{th} elements respectively. For k=4 output is 5*14=70.
45. The function 'h' replaces the kth element by g(head,k). For k=4 the final list will be 18,-91,5,70,14,-92,42,-94,5,-91,16
46. Function 'hh' deletes $(k-1)^{th}$ and $(k+1)^{th}$ elements.18,-91,70,-92,42,-94,5,-91,16.
47. The function 'loc' returns the location of smallest element. Here it is 8 because 8th element is smallest (-94). It corresponds to division. Hence division is done first.
48. The function 'step' performs a step in the evaluation of an infix expression. [No modification needed] The final list 18,-91,5,-93,14,-92,8,-91,16.
49. The function 'value' finds the value of an infix expression. [No modification needed]
    The list after second step 18,-91,70,-92,8,-91,16.
    Third: 18,-91,62,-91,16. Fourth: 80,-91,16 Fifth: 96

Expression Evaluation (doubly link list)To be deleted

1. Define a function int f(int a, int b, int c). If c is -91 then value returned is a+b, If it is -92 then a-b, if -93 then a*b, if -94 then a/b. f(6,7,-93) will return 42.
2. Write program, which reads 'k' and finds the value of f(p,q,r). Here p q and r are $(k-2)^{th}$, $(k-1)^{th}$ and k^{th} elements respectively. Let n=9 k=7 and input list is 5,70,60,-91,15,8,-93,-92,-91 then output is 15*8=120.
3. Write program, which reads 'k' and deletes the kth element. In above case the final list will be 5,70,60,-91,15,8,-92,-91.
4. Write program, which reads k and replaces the kth element by f(p,q,r). Here p q and r are $(k-2)^{th}$, $(k-1)^{th}$ and k^{th} elements respectively. For above input the final list will be 5,70,60,-91,15,8,120,-92,-91
5. Write program to output the first negative element (less than -90, it is operator). For above input it is -91.
6. Write program to perform a step in the evaluation of post fix expression. It finds the first negative element. Now suitable replacement and deletion is done. For above input the final list will be 5,130,15,8,-93,-92,-91.
7. Write program to find the value of a postfix expression. Output in above case will be 15. [Hint: execute above in a loop till the list has only one element]
   (A) The list after second step 5,130,120,-92,-91.
   (B) The list after third step 5,10,-91.
   (C) The list after fourth step 15.

8. Write program, which reads 'k' and finds the value of f(p,q,r). Here p q and r are $(k-1)^{th}$, k^{th} and $(k-2)^{th}$ elements respectively. Let n=11 k=10 and input list is -91,-93,4,-92,7,2,-92,-91,2,9,5 then output is 2+9=11.
9. Write program, which reads k and replaces the $(k-2)^{th}$ element by of f(p,q,r). Here p q and r are $(k-1)^{th}$, k^{th} and $(k-2)^{th}$ elements respectively. For above input the final list will be -91,-93,4,-92,7,2,-92,11,2,9,5.
10. Write program which finds the first element, which is positive and its next element is also positive. For above input it is 7.
11. Write program to perform a step in the evaluation of pre fix expression. For above input the final list will be -91,-93,4,5,-92,-91,2,9,5.
12. Write program to find the value of a prefix expression. Output in above case will be 26. [Hint: execute above in a loop till the list has only one element]
    (A) The list after second step -91,20,-92,-91,2,9,5.
    (B) Third: -91,20,-92,11,5. Fourth: -91,20,6. Fifth: 26

13. Write program, which reads 'k' and finds the value of f(p,q,r). Here p q and r are $(k-1)^{th}$, $(k+1)^{th}$ and k^{th} elements respectively. Let n=11 k=8 and input list is 7,-91,4,-93,6,-92,3,-93,8,-94,4 then output is 3*8=24.
14. Write program, which reads k and replaces the kth element by of f(p,q,r). Here p q and r are $(k-1)^{th}$, $(k+1)^{th}$ and k^{th} elements respectively. For above input the final list will be 7,-91,4,-93,6,-92,3,24,8,-94,4.
15. Write program which finds the smallest element. In above case -94.
16. Write program to perform a step in the evaluation of infix expression. For above input the final list will be 7,-91,4,-93,6,-92,3,-93,2.

17. Write program to find the value of a infix expression. Output in above case will be 25.
    [Hint: execute above in a loop till the list has only one element]
    Second step 7,-91,24,-92,3,-93,2. third 7,-91,24,-92,6 fourth 7,-91,18 fifth 25.

Stack-queue

```
#include<stdio.h>
void put_last(float a[],int n,float b) {a[n+1]=b;}
void remove_last(float a[],int n){}
float last(float a[],int n){return a[n];}
void print(float a[],int n){int i;for (i=1;i<=n;i++) printf("%.1f,",a[i]);}
main()
{  float x[50],element; int size=0,i,choice;
   do{printf("\nGive choice: 0:put  1:remove  2:top  3:print  4:exit");
      scanf("%d",&choice);
      if (choice==0){
         printf("Give element to be put ");scanf("%f",&element);
         put_last(x,size,element);size++;}
      if (choice==1){remove_last(x,size);size--;}
      if (choice==2){printf("The element is %.1f\n",last(x,size));}
      if (choice==3){print(x,size);}
   }while(choice!=4);
}
```

1. Define following functions. Use them in place of last and remove_last respectively. The queue behavior will be shown.
   (A) `float first(float a[],int n)` to return the first element.
   (B) `void remove_first(float a[],int n)` to remove the first element.
2. Define function `void put_begining(float a[],int n,float b)`. It puts the element 'b' in the beginning of array. Use this function in place of put_last. The queue behavior will be shown. The functions last and remove_last remain unchanged.
3. Define following functions. Use max in place of last and remove_max in place of remove_last respectively. The priority queue behavior will be shown. The function put last remains unchanged.
   (A) `float max(float a[],int n)` to return the max element.
   (B) `int location(float a[],int n,int p)` to find location of an element p. Let n=7 and array is 12,56,78,23,45,55,26 and p=56 then the function will return 2.
   (C) `void remove_kth(float a[],int n,int k)` to remove the kth element. For above input if k=4 then the final array will be 12,56,78,45,55,26.
   (D) `void remove_max(float a[],int n)` to remove the maximum element. For above input the final array will be 12,56,23,45,55,26. In this function no loop should be used. [Method: Find the maximum element (using A), then its location (using B), then remove the element at that location (using C).
   (E) The function remove_kth can also be defined as a[k]=a[n]. Let n=7 k=4 and array is 12,56,78,23,45,55,26 then final array will be 12,56,78,26,45,55. Here no shifting is done.
4. Define following functions. Use put_proper in place of put_last. The priority queue behavior will be shown.
   (A) Define function `int count(float a[],int n,float p)`. It finds the number of elements less then p. Let n=7 and array is 12,56,78,23,45,55,26 and p=52 then the function will return 4. (12, 23, 45, 26 are less)
   (B) Define function `void put_kth(float a[],int n,float b,int k)`. It puts the element 'b' at the kth place. For above array if k=4 and b=62 then the final array will be 12,56,78,62,23,45,55,26.

(C) Define function `void put_proper(float a[],int n,float b)`. It finds the number of elements less than ‘b’. Let these are ‘q’. The element ‘b’ is put at $(q+1)^{th}$ place. Let n=7 and array is 12,56,78,23,45,55,26 and b=32 then the final array will be 12,56,78,32,23,45,55,26. Since 32 is less than 3 elements (12, 23, and 26). Hence it will be put at 4th place.

5. Implement queue using following method: Use two variables: size and count. They store the number of elements put and removed respectively. An element is put at the location x[size+1]. An element is removed from the position x[count+1].

6. Implement priority queue by using heap. An element is inserted at x[size] and taken from x[1]. However after every insert and delete operation adjustment is done so that heap property is satisfied. [Heap property: $x_i > x_{2i}$ and $x_i > x_{2i+1}$ i.e. $x_i < x_{i/2}$.]

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Insert 70 60 40 ...	70	60	40	18	12	25	30	2	4	6	8	20		
Insert 35	70	60	40	18	12	35	30	2	4	6	8	20	25	
Insert 65	70	60	65	18	12	35	40	2	4	6	8	20	25	30
Remove	65	60	40	18	12	35	30	2	4	6	8	20	25	

(A) After inserting 70,60,40,18,12,25,30,2,4,6,8,20 the array will look the same.
(B) Now after inserting 35 the array will look as 70,60,40,18,12,35,30,2,4,6,8,20,25
(C) Now after inserting 65 the array will look as 70,60,65,18,12,35,40,2,4,6,8,20,25,30
(D) Here the top element is 70.
(E) After removing an element the array looks as 65,60,40,18,12,35,30,2,4,6,8,20,25. The intermediate array will be 30,60,65,18,12,35,40,2,4,6,8,20,25

7. Following is implementation of evaluation of postfix expression. Operators +, -, * and / are represented as -91, -92, -93 and -94 respectively. The stack configuration during the evaluation of 12 6 9 13 + 4 - 5 * + / is
(A) 12 6 9 13
(B) 12 6 22 (9+13)
(C) 12 6 22 4
(D) 12 6 18 (22-4)
(E) 12 6 18 5
(F) 12 6 90 (18*5)
(G) 12 96 (6+90)
(H) 0.125 (12/96)

```
float f(float a,float b,float c)
{  if (c==-91) return a+b;
   if (c==-92) return a-b;
   if (c==-93) return a*b;
   if (c==-94) return a/b;
}
void put_last(float a[],int n,float b) {a[n+1]=b;}
void remove_last(float a[],int n){}
float last(float a[],int n){return a[n];}
void print(float a[],int n)
   {int i;for (i=1;i<=n;i++) printf("%.1f,",a[i]);}
main()
{  float x[50],element,t,p,q; int size=0;
   do{printf("\nGive element");scanf("%f",&element);
      if (element>-90){put_last(x,size,element);size++;print(x,size);}
      else {p=last(x,size);size--;q=last(x,size);size--;
            t=f(q,p,element);printf("%f\n",t);
            put_last(x,size,t);size++;print(x,size);}
   }while(1);
}
```

8. Write program for evaluation of prefix expression. The stack configuration during the evaluation of + * 4 - 7 2 - + 2 9 5 is
(A) + * 4 - 7 2

(B) + * 4 5 (7-2)
(C) + 20 (4*5)
(D) + 20 - + 2 9
(E) + 20 - 11 (2+9)
(F) + 20 - 11 5
(G) + 20 6 (11-5)
(H) 26 (20+6)

Method: Every thing is pushed on the stack. If top two elements of the stack are operands then 3 elements are removed. Let these are p, q and r. Now operation qrp is done and the result is pushed on the stack. [Hint: define second last]

```
void put_last(float a[],int n,float b) {a[n+1]=b;}
void remove_last(float a[],int n){}
float last(float a[],int n){return a[n];}
float second_last(float a[],int n){return a[n-1];}
void print(float a[],int n){int i;for (i=1;i<=n;i++)
printf("%.1f,",a[i]);}
main()
{  float x[50],element,t,p,q,r; int size=0;
   do{if (size>2 && last(x,size)>-90 && second_last(x,size)>-90)
      {  p=last(x,size);size--;q=last(x,size);size--;
         r=last(x,size);size--;
         t=f(q,p,r);printf("result %f\n",t);
         put_last(x,size,t);size++;print(x,size);}
      else
      {  printf("\nGive element");scanf("%f",&element);
         put_last(x,size,element);size++;print(x,size);}
   }while(1);
}
```

Rearrange

```
#include <stdio.h>
void rd(int a[],int n)
{  int i;
   for (i=1;i<=n;i++)
      scanf("%d",&a[i]);
}
void wr(int a[],int n)
{  int i;
   printf("\narray ");
   for (i=1;i<=n;i++)
      printf("%d,",a[i]);
   printf("\n");
}
```

```
void decompose(int a[],int n,int b[],int c[])
{   int i;
    for (i=1;i<=n/2;i++) b[i]=a[i];
    for (i=1;i<=n/2;i++) c[i]=a[i+n/2];
}
main()
{ int x[20],size,y[20],z[20];
  printf("Give size");scanf("%d",&size);
  rd(x,size);
  decompose(x,size,y,z);
  wr(y,size/2);wr(z,size/2);
}
```

Let n=8 and input array is 12, 34, 56, 78, 98, 37, 59, 23 then it is decomposed into two arrays 12, 34, 56, 78 and 98, 37, 59, 23.

1. Modify function, so that arrays as decomposed as 12, 56, 98, 59 and 34, 78, 37, 23. The first array has elements at even position. The second array has at odd position.

```
void join(int a[],int b[],int n,int c[])
{  int i;
   for (i=1;i<=n;i++) c[2*i-1]=a[i];
   for (i=1;i<=n;i++) c[2*i]=b[i];
}
```

This function joins two arrays. If n=4 and input arrays are 12, 56, 98, 59 and 34, 78, 37, 23 then final array is 12, 34, 56, 78, 98, 37, 59, 23

```
main()
{ int x[20],y[20],z[20],size;
  printf("Give size");
  scanf("%d",&size);
  rd(x,size);rd(y,size);
  join(x,y,size,z);
  wr(z,2*size);
}
```

2. Modify function so that arrays are joined as 12, 56, 98, 59, 34, 78, 37, 23.
3. Define function `void join(int a[],int b[],int n,int m,int c[])` to join arrays of different sizes. If m=3 and n=5 and arrays are 12, 34, 56 and 13, 67, 23, 14, 21 then final array is 12, 34, 56, 13, 67, 23, 14, 21. The main program looks as follows: scanf(p,q); rd(x,p); rd(y,q); join(x,y,p,q,z); wr(z,p+q)

```
int counteven(int a[],int n)
{  int i,s=0;
   for (i=1;i<=n;i++)
      if (a[i]%2==0) s=s+1;
   return s;
}
```

It counts the number of even elements. Let n=5 and array is 12, 55, 33, 22, 17 then output is 2. (12 and 22 are even)

```
main()
{ int x[20],g,size;
  printf("Give size");
  scanf("%d",&size);
  rd(x,size);
  g=counteven(x,size);
  printf("%d",g);
```

4. Define function `int countbigger(int a[],int n,int k)`. It finds how many elements in the array are bigger than ‘k’. If n=5 and k=30 and array is 12, 45, 56, 23, 52 then output is 3 (because 45, 56 and 52 are bigger).

```
void evens(int a[],int n,int b[])
{  int i,j=1;
   for (i=1;i<=n;i++)
      if (a[i]%2==0)
      {   b[j]=a[i];
          j++;
      }
}
```

```
main()
{ int g,x[20],size,y[20];
  printf("size");scanf("%d",&size);
  rd(x,size);
  evens(x,size,y);
  g=counteven(x,size);
  wr(y,g);
}
```

The above function forms an array of even numbers. If n=6 and array is 12, 45, 23, 34, 46, 77 then final array is 12, 34, 46.

5. Define function `void biggers(int a[],int n,int k,int b[])`. It forms array of those elements, which are bigger than 'k'. Let n=5 and k=30 and array is 12, 45, 56, 23, 52 then final array is 45, 56, 52.
6. Define function to form array of odd numbers.
7. Write program, which forms arrays y and z of even and odd numbers. Now these arrays are joined to form a single array. In the final array even numbers are in the beginning and odd at end. [Use functions evens, odds, counteven, join]
8. Define function `void rearrange(int a[],int n,int b[])`. In array 'b' even elements are put in the beginning and odd at end. [Algorithm: Find the number of even elements using counteven. Let it be 'k'. Put odd elements from location 'k' onwards]. Let n=8 and array is 12, 57, 87, 34, 28, 44, 79, 42 then final array is 12, 34, 28, 44, 42, 57, 87, 79.
9. Define function `void rearrange(int a[],int n,int b[])` using following algorithm: Take two pointers p and q. Initially p=1 and q=n. Put even element at 'p' and odd element at 'q'. Here odd elements will appear in reverse order. For the array of previous example the final array will be 12, 34, 28, 44, 42, 79, 87, 57.
10. Define function `void rearrange(int a[],int n)`. Here additional array is not used. The same array is modified. Use following algorithm: Take two pointers p and q. Initially p=1 and q=n. If p has even element or q has odd element then these points are incremented (or decremented). Otherwise p^{th} and q^{th} elements are exchanged.

Sorting(Selection, Bubble, Location)

```
#include <stdio.h>
void rd(int a[],int n)
{  int i;
   for (i=1;i<=n;i++)
      scanf("%d",&a[i]);
}
void wr(int a[],int n)
{  int i;
   for (i=1;i<=n;i++)
      printf("%d,",a[i]);
   printf("\n");
}
void mul(int a[],int n,int k)
{  int i;
   for (i=1;i<=n;i++)
      a[i]=a[i]*k;
}
```

```
main()
{ int g,x[20],size;
  printf("Give size");scanf("%d",&size);
  rd(x,size);
  printf("Give a number to multiply");
  scanf("%d",&g);
  mul(x,size,g);
  wr(x,size);
}
```

This program reads n numbers, stores them in an array. The program reads one more number ‘g’. The program multiplies all elements of the array by ‘g’. Let n=8 and elements are 23, 47, 28, 97, 21, 34, 44, 22. Let g=5 then the array will change to 115, 235, 140, 485, 105, 170, 220, 110.

Following function finds the sum of even elements. For above input 28+34+44+22=128.

```
int sumeven(int a[],int n)
{  int i,s=0;
   for (i=1;i<=n;i++)
      if (a[i]%2==0)
         s=s+a[i];
   return s;
}
```

```
main()
{ int g,x[20],size;
  printf("Give size");scanf("%d",&size);
  rd(x,size);
  g=sumeven(x,size);
  printf("sum of even numbers=%d",g);
}
```

The following function finds the location of first even element. For above input output 3.

```
int locfirsteven(int a[],int n)
{  int i=1;
   while (a[i]%2!=0) i++;
   return i;
}
```

```
main()
{ int x[20],g,size;
  printf("Give size");scanf("%d",&size);
  rd(x,size);
  g=locfirsteven(x,size);printf("%d",g);
}
```

The following function outputs first ‘g’ even numbers. For g=3 the output is 28, 34, 44.
It uses following algorithm:

(A) Find location of first even element. Let it be z.
(B) Output a[z].
(C) Replace z^{th} element by some odd number.
(D) Repeat above steps ‘g’ times.

```
void feweven(int a[],int n,int k)
{  int i,j;
   for (i=1;i<=k;i++)
   {  j=locfirsteven(a,n);
      printf("%d,",a[j]);
      a[j]=11;
   }
}
```

```
main()
{ int x[20],g,size;
  printf("Give size");
  scanf("%d",&size);
  rd(x,size);
  printf("howmany even");
  scanf("%d",&g);
  feweven(x,size,g);
}
```

The following function adds two arrays of same size.

```
void add(int a[],int b[],int n,int c[])
{  int i;
   for (i=1;i<=n;i++) c[i]=a[i]+b[i];
}
```

```
main()
{ int x[20],y[20],z[20],size;
  printf("Give size");
  scanf("%d",&size);
  rd(x,size);rd(y,size);
  add(x,y,size,z);
  wr(z,size);
}
```

Let n=4 and first array is 12, 34, 23, 45 and the second array is 16, 44, 11, 39 then result is 12+16, 34+44, 23+11, 45+39 i.e. 28, 78, 34, 84.

Sorting(Selection, Bubble, Location)

Let n=8 and array is 12, 34, 17, 29, 11, 37, 28, 35

1. Define function int minimum(int a[], int n). It returns the the minimum element. For above input output is 11.
2. Define function int location(int a[], int n, int k). It returns the location of the element 'k'. For k=34 the output is 2.
3. Define function int minlocation(int a[], int n). It returns the location of the minimum element. For above input output is 5. (since minimum element 11 is at location 5). In the definition of this function do not use loop. Use above functions.
4. Define function void replace(int a[], int n, int k, int t). It replaces the tth element of the array by k. Let k=33 and t=6 then array will be 12, 34, 17, 29, 11, 33, 28, 35.
5. Define function void selectionsort(int a[], int n). It outputs the elements in the sorted order. It uses following algorithm:
   (B) Find the location of the minimum element. Let it be z.
   (C) Output a[z].
   (D) Replace z^{th} element by 100. Assume all elements of the array are less than 100.
   (E) Repeat above steps 'n' times.

6. Define function void scan(int a[], int n). It scans the array. Whenever it finds $a_i > a_{i+1}$ it exchanges them. For above input the final array is 12, 17, 29, 11, 34, 28, 35, 37.
7. Define function void bubblesort(int a[], int n). It calls the scan function 'n-1' times.

8. Define function void increment(int a[], int n). It adds 1 to all elements of the array. For above input the final array will be 13, 35, 18, 30, 12, 38, 29, 36.
9. Define function int even(int a[], int n). It returns the number of even elements in the array. For above input output is 3 (since 12, 34 and 28 are even).
10. Define function int smaller(int a[], int n, int k). It returns how many numbers are less than 'k'. Let k=30 then output is 5 (since 12, 17, 29, 11 and 28 smaller then 30)
11. Define function void order(int a[], int b[], int n). It outputs the a[b[i]]. Let n=4 and array 'a' is 12, 34, 15, 23 and array 'b' is 2, 4, 2, 1 then output is 34, 23, 34, 12.
12. Define function void rearrange(int a[], int b[], int n, int c[]). It forms c[b[i]]=a[i]. Let n=4 and array 'a' is 12, 34, 15, 23 and array 'b' is 2, 4, 3, 1 then array 'c' will become 23, 12, 15, 34.
13. Define function void locationsort(int a[], int n, int c[]). Use following algorithm:
    (B) Declare an array b in side the function (int b[100])
    (C) Compute b[i]=number of elements less than a[i]. For above input array 'b' will be 1, 5, 2, 4, 0, 7, 3, 6.
    (D) Increment array 'b'. It will become 2, 6, 3, 5, 1, 8, 4, 7.
    (E) Call rearrange. The array c will be the sorted array.

Following is efficient implementation of Bubble sort.

14. Define function int scan(int a[], int n). It scans the array. Whenever it finds $a_i > a_{i+1}$ it exchanges them. It returns the location of last exchange. If no exchange is made then 0 is returned. Let n=8 and array is 12, 17, 29, 11, 34, 28, 35, 37 then 5 will be returned and final array will be 12, 17, 11, 29, 28, 34, 35, 37.

15. Define function void bubblesort(int a[], int n). It calls the scan function till it returns 0. If scan function returns z then next time scan(a,z) is called.

Sorting

```
void rd(int a[],int k)
{  int i;
   for (i=1;i<=k;i++)
      scanf("%d",&a[i]);
}
void wr(int a[],int k)
{  int i;
   for (i=1;i<=k;i++)
      printf("%d ",a[i]);
}
int delete(int a[],int k)
{  int i,p;p=a[1];
   for (i=1;i<k;i++) a[i]=a[i+1];
   return p;
}
main()
{  int n;int x[20],g;
   printf("n=");scanf("%d",&n);
   rd(x,n);
   g=delete(x,n);
   wr(x,n-1);
   printf("Deleted=%d",g);
}
```

This program (at left) reads n numbers, stores them in array. The delete function deletes first element. Other elements are shifted. Let n=6 and elements are 23,47,2,974,828,95 then element 23 will be deleted and array will be 47,2,974,828,95.

The following program reads an additional number and puts it al last. If additional element is 126 then the array will be 23,47,2,974,828,95,126.

```
void insert(int a[],int k,int b)
{   a[k+1]=b; }
main()
{  int n;int x[20],g;
   printf("n=");scanf("%d",&n);
   rd(x,n);
   printf("Give element to put");
   scanf("%d",&g);
   insert(x,n,g);
   wr(x,n+1);
}
```

1. Modify the delete function to delete the second last element. For above example the deleted element will be 828 and the array will be 23,47,2,974,95.
2. Delete last element. The deleted element will be 95 and the array will be 23,47,2,974,828.
3. Delete first even element. After deletion other elements are not shifted. The free space is filled by the last element. For above example the deleted element is 2 and the array will be 23,47,95,974,828.
4. Delete biggest element. The deleted element is 974 and array is 23,47,2,828,95.
5. Delete first even element. After deletion elements are shifted. For above example the deleted element is 2 and the array will be 23,47,974,828,95.

6. Modify the insert function to put the new element at second last place. The array will be 23,47,2,974,828,126,95.
7. Put the new element at 1st place. However elements are not shifted. The first element is put at last. The array will be 126,47,2,974,828,95,23.
8. Do above problem by shifting element. The array will be 126,23,47,2,974,828,95.
9. Insert it at the place of the location of first bigger. Do not shift other elements. The corresponding element is put at last. The 23,47,2,126,828,95,974. [Hint: there is a difference between first bigger and biggest. In 2,4,8,3,9,7 the first bigger element of 5 is 8. Here 5 will be inserted as 2,4,5,3,9,7,8.] If new element is biggest then it is put at last.

10. Do above problem by shifting. The array 23,47,2,126,974,828,95 and 2,4,5,8,3,9,7.

```
void exch(int a[],int k)
{  int i=1,p;
   while (a[i]%2!=0) i++;
   p=a[i];a[i]=a[k];a[k]=p;
}
```

```
main()
{
   rd(x,n);exch(x,n);wr(x,n);
}
```

11. The above function exchanges first even number by the last number. If n=8 and elements are 23,45,67,42,31,89,62,17 then after the execution they will be 23,45,67,17,31,89,62,42.
12. Define function which exchanges the maximum element by the last element. In above case 23,45,67,42,31,17,62,89.
13. Using above implement Selection Sort. for (i=n;i>1;i--) exch(x,i);
14. Let ‘t’ be the first element which is greater then the next element. ($t=a_i$ here ‘i’ is smallest number such that such that $a_i>a_{i+1}$.) Exchange ‘t’ with the next element. In above case array will be 23,45,42,67,31,89,62,17.
15. Using above implement bubble sort. x[n+1]=0;while(x[n]$\neq$0) exch(x,n); x[n]=x[n+1];
16. Let ‘p’ be the next element of ‘t’. Let ‘q’ be the first element which is bigger than ‘p’. Exchange p and q. In above case 23,42,67,45,31,89,62,17.
17. Put ‘p’ in place of ‘q’. Perform shifting. In above case 23,42,45,67,31,89,62,17.
18. Using above implement insertion sort. x[n+1]=0;while(x[1]$\neq$0) exch(x,n); shift left
19. Define int count(int a[],int k,int t). It returns the number of elements in array ‘a’, which are less then ‘t’.

    Use above to perform location sort. for(i=1;i$\leq$n,i++) b[1+count(a,n,a[i])]=a[i]

Sorting

32. Define function void rd(int a[],int n) for reading ‘n’ elements of array a_1, a_2, a_3, …., a_n. Also define function void wr(int a[],int n) for printing array. Write following main program to check the result.

    int n;int x[20]; printf("Give n ");scanf("%d",&n); rd(x,n); wr(x,n);

33. Define function void decompose(int a[],int n,int b[],int c[]) for decomposing array into two parts. The array b has first n/2 elements. The array c contains the remaining element. Let n=8 and array elements are 12,37,15,66,89,33,62,19 then following main program will output 89,33,62,19 and 12,37,15,66.

    decompose(x,n,y,z);wr(z,n-n/2);printf(“\n”);wr(y,n/2)

34. Define function int count(int a[],int n) for finding the number of even elements. On above input the execution of following main program will output 3.

    p=count(x,n);printf(“%d”,p);

35. Define function void break(int a[],int n,int b[],int c[]) for decomposing array into two parts. The array b has even elements. The array c has odd elements. On above input the execution of following main program will output 12,66,62 and 37,15,89,33,19.

    k=count(x,n);break(x,n,y,z);wr(y,k);printf(“\n”);wr(z,n-k)

36. Define function void partition(int a[],int n,int b[]) for partitioning array into two parts. The array b has even elements in the beginning and odd elements at end. On above input the main program partition(x,n,y);wr(y,n); will output 12,66,62,37,15,89,33,19.
37. Define function void bs(int a[],int n) for sorting array by bubble sort technique. On above input the main program bs(x,n);wr(x,n); outputs 12,15,19,33,37,62,66,89.
38. Define function int loc(int a[],int n) to find location of maximum element. The main program p=loc(x,n);print(p) outputs 5.
39. Define function void ss(int a[],int n) for sorting array by selection sort technique. The function looks as follows: p=loc(x,n);x[p]↔x[n];bs(x,n-1);
40. Define function void insert(int a[],int n,int b). Here input array ‘a’ is sorted. The element b is inserted in that at proper place. Let n=5 and array is 12,27,34,56,89 and b=43 then main program insert(x,n,b);wr(x,n+1) will output 12,27,34,43,56,89. If n=4 and array is 16,35,39,56 and b=67 then output is 16,35,39,56,67.
41. Define function void is(int a[],int n) for sorting array by insert sort technique. The function looks as follows: bs(x,n-1);insert(x,n-1,x[n]);
42. Define function void permute(int a[], int b[], int n,int c[]) The size of array b is also n. The array ‘c’ contains the element of array ‘a’. Their locations are according to ‘b’. Let n=5 and array ‘x’ is 12,34,15,29,42 and array ‘y’ is 4,3,1,5,2 then execution of permute(x,y,n,z);wr(z,n) will output 15,42,34,12,29.
43. Define permute in such a manner so that output in above case is 29,15,12,42,34.
44. Define function int less(int a[],int n,int t) for finding the number elements of array ‘a’ which are less than ‘t’. Let n=6 and array is 12,34,16,78,25,33 then execution of p=less(x,n,18);printf(“%d”,p); will output 2.
45. Define function void lesscount(int a[],int n,int b[]). The b[i] has the number of elements of array ‘a’ less than a[i]. Let n=6 and array ‘a’ 42,34,16,78,25,33 then execution of lesscount(a,n,b);wr(b,n); will output 4,3,0,5,1,2.
46. Define function void increment(int a[],int n) to add 1 to all elements of array.
47. Define function void ls(int a[],int n) for sorting array by location sort technique. Take the help of functions lesscount, increment and permute.

Matrix(using function)

```
void readmatrix(int a[][10],int n)
{   int i,j;
    for (i=1;i<=n;i++)
        for (j=1;j<=n;j++)
            scanf("%d",&a[i][j]);
}
void mulrow(int a[][10],int p,int q,int n)
{   int i;
    for (i=1;i<=n;i++) a[p][i]=a[p][i]*q;
}
void printmat(int a[][10], int n)
{   int i,j;
    printf("\n");
    for (i=1;i<=n;i++)
    {   for (j=1;j<=n;j++)
            printf("%4d",a[i][j]);
        printf("\n");
    }
    printf("\n");
}
main()
{   int x[10][10],size,k,t;
    printf("Give size");
    scanf("%d",&size);
    readmatrix(x,size);
    printf("Give k and t");
    scanf("%d %d",&k,&t);
    mulrow(x,k,t,size);
    printmat(x,size);
}
```

The given program reads a matrix and multiplies k^{th} row by t. Let size of the matrix is 6 and the matrix is

```
 2  3  1  4  5  6
 6  3  4  5  6  8
 2  6  3  4  5  6
 7  5  8  9  4  3
 8  3  4  5  2  6
 4  2  5  6  2  4
```

Let k=4 and t=2 output matrix is

```
 2  3  1  4  5  6
 6  3  4  5  6  8
 2  6  3  4  5  6
14 10 16 18  8  6
 8  3  4  5  2  6
 4  2  5  6  2  4
```

In a file named "gg" give following input and execute by command ./a.out<gg

```
6
2 3 1 4 5 6
6 3 4 5 6 8
2 6 3 4 5 6
7 5 8 9 4 3
8 3 4 5 2 6
4 2 5 6 2 4
4 2
```

1. Define function to add t to k^{th} column. Let k=4 and t=2 then output matrix is as follows: [2 3 1 6 5 6][6 3 4 7 6 8][2 6 3 6 5 6][7 5 8 11 4 3][8 3 4 7 2 6][4 2 5 8 2 4]
2. Write program to multiply the first row by 1, second row by 2, and so on. For above input the output matrix will be
   [2 3 1 4 5 6][12 6 8 10 12 16][6 18 9 12 15 18] [28 20 32 36 16 12][40 15 20 25 10 30][24 12 30 36 12 24]
3. Program to multiply each row by the last element of that row. In above case first row will be multiplied by 6, second by 8, third by 6 and so on. Output
   [12 18 6 24 30 36][48 24 32 40 48 64][12 36 18 24 30 36][21 15 24 27 12 9][48 18 24 30 12 36][16 8 20 24 8 16]

```
int minrow(int a[][10],int k,int n)
{   int i,min=a[k][1];
    for (i=2;i<=n;i++)
        if (a[k][i]<min) min=a[k][i];
    return min;
}
main()
{   int x[10][10],size,k,t;
    printf("Give size");scanf("%d",&size);
    readmatrix(x,size);
    printf("Give k");scanf("%d",&k);
    t=minrow(x,k,size);
    printf("The minimum of %dth row is %d\n",k,t);
}
```

This program finds the minimum element of kth row. When input k=4 then output is 3. For k=5 the output is 2.

4. Define function to return sum of all elements of kth row. For k=4 the output is 36.
5. Define function to return maximum element of kth column. For k=4 the output is 9.
6. Function to return maximum even number of kth row. For k=4 the output is 8.
7. Define function to return the location of minimum element. For k=4 it is 6 because the minimum element 3 is at 6th column. For k=5 it is 5.
8. Write program to find sum of minimum elements of all rows. Ans:1+3+2+3+2+2=13.
9. Write program to replace first element of every row by sum of all elements of that row. Output [**21** 3 1 4 5 6] [**32** 3 4 5 6 8] [**26** 6 3 4 5 6] [**36** 5 8 9 4 3] [**28** 3 4 5 2 6] [**23** 2 5 6 2 4]
10. Program to multiply k^{th} row by minimum element of that row. For k=4 the output is [2 3 1 4 5 6][6 3 4 5 6 8][2 6 3 4 5 6][**21 15 24 27 12 9**][8 3 4 5 2 6][4 2 5 6 2 4]
11. Program that finds the sum of minimum elements of kth and t^{th} rows. For k=4 and t=2 the minimum elements are 3 and 3 respectively. Hence output is 6.
12. Program finds the location of minimum element of kth row. Let it be 'L' The program multiplies the Lth column by 2. [Define suitable function to multiply k^{th} column by 2]
    For k=4 [2 3 1 4 5 **12**][6 3 4 5 6 **16**][2 6 3 4 5 **12**][7 5 8 9 4 **6**][8 3 4 5 2 **12**][4 2 5 6 2 **8**]
    For k=2 [2 **6** 1 4 5 6][6 **6** 4 5 6 8][2 **12** 3 4 5 6][7 **10** 8 9 4 3][8 **6** 4 5 2 6][4 **4** 5 6 2 4]
13. Write program to multiply every row by minimum element of that row. Output
    [2 3 1 4 5 6] [18 9 12 15 18 24] [4 12 6 8 10 12] [21 15 24 27 12 9] [16 6 8 10 4 12] [8 4 10 12 4 8]

```
void addrow(int a[][10],int p,int q,int n)
{   int i;
    for (i=1;i<=n;i++)
        a[p][i]=a[p][i]+a[q][i];
}
main()
{   int x[10][10],size,k,t;
    printf("Give size");
    scanf("%d",&size);
    readmatrix(x,size);
    printf("Give k and t");
    scanf("%d %d",&k,&t);
    addrow(x,k,t,size);
    printmat(x,size);
}
```

This function performs operation
Row(p) = Row(p) + Row(q)

If k=4 and t=2 then final matrix is

Input	Output
2 3 1 4 5 6	2 3 1 4 5 6
6 3 4 5 6 8	6 3 4 5 6 8
2 6 3 4 5 6	2 6 3 4 5 6
7 5 8 9 4 3	13 8 12 14 10 11
8 3 4 5 2 6	8 3 4 5 2 6
4 2 5 6 2 4	4 2 5 6 2 4

14. Define function to perform Row(p) = max(Row(p) , Row(q))For input 4 and 2 output
    [2 3 1 4 5 6] [6 3 4 5 6 8] [2 6 3 4 5 6] [**7 5 8 9 6 8**] [8 3 4 5 2 6] [4 2 5 6 2 4]
15. Define function to exchange p^{th} and q^{th} rows. For input 4 and 2 the output is
    [2 3 1 4 5 6] [**7 5 8 9 4 3**] [2 6 3 4 5 6] [**6 3 4 5 6 8**] [8 3 4 5 2 6] [4 2 5 6 2 4]
16. Write program, which will add two rows. It will then output the minimum of that row. For input 4 and 2 output is 8. [Do not use any loop in it]

```
void addmatrix(int a[][10],int b[][10],int c[][10],int n)
{   int i,j;
    for (i=1;i<=n;i++)
        for (j=1;j<=n;j++)
            c[i][j]=a[i][j]+b[i][j];
}
main()
{   int x[10][10],y[10][10],z[10][10];
    int size;
    printf("Give size");
    scanf("%d",&size);
    readmatrix(x,size);
    readmatrix(y,size);
    addmatrix(x,y,z,size);
    printmat(z,size);
}
```

Sum of given matrixes

2 3 1 4 5 6	2 3 1 4 5 6
6 3 4 5 6 8	6 2 3 4 5 8
2 6 3 4 5 6	8 3 4 5 6 7
7 5 8 9 4 3	9 2 3 4 5 6
8 3 4 5 2 6	9 5 6 7 8 3
4 2 5 6 2 4	6 4 5 6 8 7

Input matrixes (⇑) Output (⇓)

4	6	2	8	10	12
12	5	7	9	11	16
16	9	7	9	11	13
16	7	11	13	9	9
17	8	10	12	10	9
10	6	10	12	10	11

17. Define function, which will perform $c_{i,j}=\max(a_{i,j},b_{i,j})$ Output for above input

[2 3 1 4 5 6] [6 3 4 5 6 8] [8 6 4 5 6 7] [9 5 8 9 5 6] [9 5 6 7 8 6] [6 4 5 6 8 7]

18. Define function which will perform $c_{i,j}=a_{i,k}+b_{t,j}$. For k=4 and t=2 the matrix will be

[10 6 7 8 9 12][11 7 8 9 10 13][10 6 7 8 9 12][15 11 12 13 14 17][11 7 8 9 10 13] [12 8 9 10 11 14]

Graph (Shortest Path)

```
int dotproduct(int a[][10],int b[][10],int n,int p,int q)
 {  int i,sum=0;
    for (i=1;i<=n;i++)
        sum=sum+a[p][i]*b[i][q];
    return sum;
 }
```

This finds dotproduct of pth row of matrix 'a' and q^{th} column of matrix 'b'. For above matrixes and k=4 t=2 the output is 105. Because dot product of (7 5 8 9 4 3) and (3 2 3 2 5 4) is 7.3+5.2+8.3+9.2+4.5+3.4=105.

```
 main()
 {  int x[10][10],y[10][10],size,k,t,g;
    printf("Give size");scanf("%d",&size);
    readmatrix(x,size);readmatrix(y,size);
    printf("Give k and t");scanf("%d %d",&k,&t);
    g=dotproduct(x,y,size,k,t);
    printf("The dot product of %dth row and %dth column is %d\n",k,t,g);
 }
```

19. In the function for addition of matrixes if c[i][j]=dotproduct(a,b,n,i,j) is used then matrix multiplication is done. The output for above matrixes is

Input Matrixes		Output Matrix
2 3 1 4 5 6	2 3 1 4 5 6	147 72 87 112 139 124
6 3 4 5 6 8	6 2 3 4 5 8	209 108 122 166 206 192
2 6 3 4 5 6	8 3 4 5 6 7	181 84 104 134 166 162
7 5 8 9 4 3	9 2 3 4 5 6	243 105 120 170 209 225
8 3 4 5 2 6	9 5 6 7 8 3	165 86 90 134 168 178
4 2 5 6 2 4	6 4 5 6 8 7	156 69 80 111 138 145

20. Define function, which will perform $b_{ij}=a_{ij.}$ It makes copy of the matrix. Check correctness by main( ){ readmatrix(x,size);copy(x,y,size); printmat(x,size); }
21. Modify the definition of dot product. Here $\min(a_i+b_i)$ is used in place of $\text{sum}(a_i*b_i)$. Output for k=4 t=2 is 7. Because dot product of (7 5 8 9 4 3) and (3 2 3 2 5 4) is min(7+3,5+2,8+3,9+2,4+5,3+4)=7.
22. Under above definition of dotproduct for matrix multiplication. The multiplication of given matrixes is [4 4 3 6 7 8][8 5 6 7 8 9][4 5 3 6 7 8][9 7 8 9 10 7][9 5 6 7 8 5][6 4 5 6 7 5]

    Let matrix 'a' be [0 1 4 6 3][6 0 7 2 6][8 4 0 9 1][3 6 1 0 4][1 3 4 8 0]
23. Write a program, which reads a matrix 'a'. It performs its dotproduct with itself. It is called a^2. Output matrix a^2 [0 1 4 3 3][5 0 3 2 6][2 4 0 6 1][3 4 1 0 2][1 2 4 5 0]
24. Define function void power(int a[][10],int n,int k,int b[][10]). It finds $b=a^k$. For above matrix a^{10} is [0 1 4 3 3][5 0 3 2 4][2 3 0 5 1][3 4 1 0 2][1 2 4 4 0]
    a^3 is [0 1 4 3 3][5 0 3 2 4][2 3 0 6 1][3 4 1 0 2][1 2 4 4 0]

    If $a_{i,j}$ denotes the distance from i^{th} to j^{th} node and $b=a^{n-1}$ then $b_{i,j}$ is the length of shortest path from i^{th} to j^{th} node. Here it is obtained using the method of matrix multiplication.

25. Define function void mul(int a[][10],int n,int k,int b[][10]). It performs $b_{ij}=a_{ik}+a_{kj}$. It is represented as $a_k a$. For above matrix 'a' $a_2 a$ is
    [7 1 8 3 7][6 0 7 2 6][10 4 11 6 10][12 6 13 8 12][9 3 10 5 9].
26. Function void min(int a[][10],int b[][10],int n,int c[][10]) to perform $c_{ij}=\min(a_{ij},b_{ij})$. min of [2 3 8][5 7 2][4 6 8] and [6 1 5][3 4 2][9 6 2] is [2 1 5][3 4 2][4 6 2].

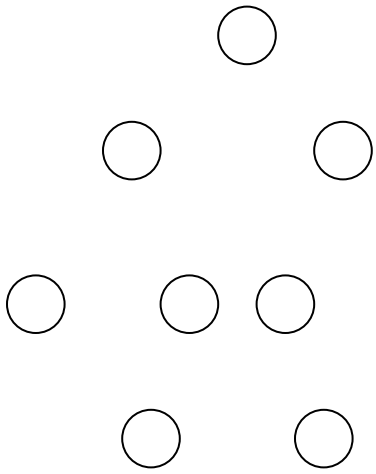
27. Function void FWs(int a[][10],int n,int k,int b[][10]). It performs $b_{ij}=\min(a_{ij}, a_{ik}+a_{kj})$. It is represented as $a^k a$. It is a single step in Floyd Warshall algorithm. Do not use loop. User above functions.
    For above matrix 'a' $a^2 a$ is [0 1 4 3 3][6 0 7 2 6][8 4 0 6 1][3 6 1 0 4][1 3 4 5 0].
28. Write program, which reads k and t and performs $p=a^k a$ $q=p^t p$. For above 'a' and k=2 t=3 the matrix is [0 1 4 3 3][6 0 7 2 6][8 4 0 6 1][3 5 1 0 2][1 3 4 5 0]
29. Write program to compute $b=a^1 a$ $c=b^2 b$ …. $q=p^{n-1} p$ $r=q^n q$. The final matrix is all pair shortest path. [0 1 4 3 3][5 0 3 2 4][2 3 0 5 1][3 4 1 0 2][1 2 4 4 0]

    Let 'a'=[0 1 0 4 6 3][6 0 7 4 2 6][8 0 4 0 9 0][3 6 7 8 0 4][1 1 3 4 8 0][2 1 1 1 1 1]
30. Define functions for reading and writing matrix.
31. Define function printrow(int a[][10],int n,int k). It prints the kth row of the matrix.
32. Define function int min(int a[][10],int n,int k). It finds the minimum of kth row except 0. For k=2 the minimum is 2. For k=5 minimum is 1.
33. Define function to find the location of the minimum element. For k=2 it is 5. For k=5 it is 1 or 2. [In case of equal elements any location may be taken]
34. Define function void add(int a[][10],int n,int k,int t). It adds t to the kth row. Let k=2 t=5 [0 1 0 4 6 3][**11 5 12 9 7 11**][8 0 4 0 9 0][3 6 7 8 0 4][1 1 3 4 8 0][2 1 1 1 1 1]
35. Define function void minrow(int a[][10],int n,int p,int q) to perform $a_{pi}=\min(a_{pi},a_{qi})$. It does Row(p) = min(Row(p) , Row(q)) For p=4 q=2 output
    [0 1 0 4 6 3][6 0 7 4 2 6][8 0 4 0 9 0][**3 0 7 4 0 4**][1 1 3 4 8 0][2 1 1 1 1 1]
36. Implement Dijkstra's Algorithm to find single source shortest path. It finds the shortest path from k^{th} node to all other nodes.
    (A) Print k^{th} row.
    (B) Find location of minimum element of kth row. Let it be 'z'.
    (C) Output z and $a_{k,z}$.
    (D) Add $a_{k,z}$ to all elements of zth row.
    (E) Print z^{th} row also.
    (F) Perform Row(k) = min(Row(k) , Row(z))
    (G) $a_{k,z}=0$
    (H) Repeat above steps (n-1) times.

Matrix	k^{th} row k=1	z	$a_{k,z}$	z^{th} after adding $a_{k,z}$
Row 1: 0 9 7 3 17 19	0 9 7 3 17 19	4	3	8 6 10 3 12 16
Row 2: 5 0 3 5 13 0	0 6 7 0 12 16	2	6	11 6 9 11 19 6
Row 3: 5 5 0 5 1 5	0 0 7 0 12 6	6	6	13 13 13 13 11 6
Row 4: 5 3 7 0 9 13	0 0 7 0 11 0	3	7	12 12 7 12 8 12
Row 5: 3 3 3 3 0 3	0 0 0 0 8 0	5	8	11 11 11 11 8 11
Row 6: 7 7 7 7 5 0				

20.

Tree



The above tree will be stored as
Root 2

Index	1	2	3	4	5	6	7	8
Data	18	19	17	27	28	42	33	26
Left	-1	5	1	-1	8	4	-1	-1
Right	-1	6	-1	7	3	-1	-1	-1

1. Write program to find the maximum element in the tree. In given example 42.

2. Write program, to find the height of a tree. In above case 3. [Hint: the height is the length of a longest path from root to leaf.]

3. Write program to find index of a node. e.g. input 17 output 3.

4. Write program to print inorder traversal. For given tree it is
   26,28,18,17,19,27,33,42,

5. Write program, to find children of a node. e.g. children of 28 are 26 and 17. Child of 17 is 18.

6. Write program to find father of a node. e.g. father of 17 is 28.

7. Write program (using recursive function), which will print the tree as follows:
   <<<26>28<<18>17>>19<<27<33>>42>>

8. Read an element x and search it. If x=18 then answer is 17 28 19. Define a function int search (int root, int x). It will return 1 if x is present in the tree and –1 if it is absent.

9. Write program, which will print the breadth first traversal of tree as follows: 19 28 42 26 17 27 18 33. Do not use any function.

10. Assume that the tree is binary search tree. Read x and search it. Find whether x is present or not. If it is present find the path from root to it.

11. Read x and insert it in binary search tree.

Try to do following problems, without using function [Hint: Use logic of breath first traversal]
(1) Maximum element (2) Height of tree (3) All elements at height h
(4) Path of a given node from x (it is not binary search tree).
(5) Write program for LL and LR rotation

Polynomial

```
void copy(float a[],int n, float b[])
{   int mid;
    if (n==0) b[0]=a[0];
    else
    {   mid=n/2;
        copy(a,mid,b);
        copy(&a[mid+1],n-mid-1,
                        &b[mid+1]);
    }
}
```

```
void makezero (float a[],int n)
{   if (n>=0)
    {   a[0]=0;
        makezero(&a[1],n-1);
    }
}
```

```
int add (float a[],int n,float b[],int m, float c[])
{   int t;
    if (m<0) {copy(a,n,c); return n;}
    if (n<0) {copy(b,m,c); return m;}
    c[0]=a[0]+b[0];
    t=add(&a[1],n-1,&b[1],m-1,&c[1]);
    return t+1;
}
main()
{   float x[20],y[20],z[20];int i,j,d1,d2,d3;
    printf("Give degree of polynomials");scanf("%d %d",&d1,&d2);
    printf("Give elements of first polynomial ");
    for (i=0;i<=d1;i++) scanf("%f",&x[i]);
    printf("Give elements of second polynomial");
    for (i=0;i<=d2;i++) scanf("%f",&y[i]);
    d3=add(x,d1,y,d2,z);
    printf("The addition is:");
    for (i=0;i<=d3;i++) printf("%.1fx^%d+",z[i],i);
}
```

A polynomial x of degree ‘n’ has ‘n+1’ terms. These are x_0, x_1,... x_n. The function int add(a,n,b,m,c) does addition of two polynomials ‘a’ (degree n) and ‘b’ (degree m). The result is stored in polynomial ‘c’. The function returns the degree of resulting polynomial.

1. Define a function g(a,n,t,b) to perform $b=x^t a$. Use logic mult(a,n,t-1,c)
2. Do above problem without using recursion. Use mkaezero and copy.
3. Define function int addmul(a,n,b,m,t,c). Polynomial ‘b’ is added to ‘a’ after multiplying by x^t. i.e. $c=a+x^t b$. Use following logic:
   If ($n \geq 0$) and (t>0) {addmul(&a[1],n-1,b,m,t-1,&c[1]) …} else if (t=0) {…} else if (n<0) {…}
4. Do above problem by using following logic: copy(a,t-1,c); if (t>n) {makezero(…)…} else {…}
5. Define function add (assume m=n)by using following logic:
   t=max(m,n);
   if (t=0) {c[0]=a[0]+b[0]; return 0;} else {mid=t/2; ….}
6. (A) Do above problem when $n \geq m$. (B) Do above problem when n and m can be anything.
7. Define function mult(a,n,b,c). The polynomial ‘a’ is multiplied by number ‘b’.
8. Define function int mul(a,n,b,m,c) for multiplication of polynomial. Use following logic: Let a=A+xB. Here A is a number and B is polynomial (degree n-1). c=Ab+xbB.
9. Define polynomial multiplication by using following logic: $a=A+Bx^t$ and $b=C+Dx^t$. $c=AC+(AD+BC)x^t+BDx^{2t}$. Take t=n/2. Assume m=n. Here A, B, C and D are polynomials.
10. In above program calculate AD+BC as [(A+B)*(C+D)]–AC–BD.

11. Define function void div(a,n,b,m,c,d). c and d are polynomials such that bc+d=a. Degree of them is n–m and m–1 respectively.

Dynamic Programming

Write program to find fib(n) using recursive function and otherwise. Try to find fib(50) using recursive program. It is difficult because same function is recomputed.
Hint: fib(1)=fib(2)=1. fib(n)=fib(n-1)+fib(n-2) when n>2

f(m,n) = m if m=n
= f(m,n-1) + f(m+1,n) + m*n+m if n>m

Program for calculation of above recursive function by dynamic programming is as follows.
To find f(5,9) we find f(5,5) f(6,6) f(7,7) f(8,8) f(9,9), f(5,6) f(6,7) f(7,8) f(8,9), f(5,7) f(6,8) f(7,9), f(5,8) f(6,9) then f(5,9)

```
#include<stdio.h>
main()
{    int d,x,y,m,n,i,j,f[100][100];
     printf("Give m and n. m is less than n");
     scanf("%d %d",&m,&n);
     for (i=m;i<=n;i++) f[i][i]=i;
     for (d=1;d<=(n-m);d++)
         for (j=m;j<=(n-d);j++){
             x=j;y=j+d;
             f[x][y]=f[x][y-1]+f[x+1][y]+x*y+x;
             printf("f(%d,%d)=%d\n",x,y,f[x][y]);
          }
}
```

Write program for following recursive functions using dynamic programming.

1. For following program, assume n is positive
   f(n)=1 if n<3
   f(n)=f(n-1)+f(n/2) otherwise

2. For following assume m<n.
   f(m,n) = 0 if m=n-1
   $= \sum_{i=m+1}^{i=n-1} [f(m,i)+f(i,n)+m*n]$ otherwise

3. For following assume m<n and both positive.
   f(m,n) = 0 if m=n-1
   $= \min_{i=m+1}^{i=n-1} [f(m,i)+f(i,n)+((m*n)\%20)]$ otherwise

4. For following assume m<n and positive. Write program (using dynamic programming, without using function) to implement function, whose prototype is int f(int m, int n, int a[])

$$f(m,n,a) = 0 \qquad \text{if } m=n-1$$
$$= \min_{i=m+1}^{i=n-1} \left(f(m,i,a)+f(i,n,a)+a[m]*a[i]*a[n] \right) \quad \text{otherwise}$$

The above is solution of matrix chain multiplication. If $a_0, a_1,\ldots,a_5$ are 3,2,4,1,5,6 then f(0,5,a) is answer is 62.

5. For following program assume n is positive.

$$f(m,n) = m \qquad \text{if } n=0$$
$$= f(m-2,n-1) + 2*f(m-1,n-1) + m*n \qquad \text{otherwise}$$

6. For following program assume $m \le n$.

$$f(m,n) = m^2 \qquad \text{if } n=m$$
$$= f(m+1,n) + 2*f(m,n-1) + f(m-1,n-2) \qquad \text{if } m<n$$

7. For following program assume m and n are positive.

$$f(m,n) = m \qquad \text{if } n=0$$
$$= n \qquad \text{if } m=0$$
$$= f(m-1,n-1) + 2*f(m-1,n) + 3*f(m,n-1) + m*n \quad \text{otherwise}$$

Hint: find in the order f(1,1) f(2,1) f(1,2) f(3,1) f(2,2) f(1,3).
In inner loop find f(i,j) where i+j=constant.
Do above problem by adding f(m-21,n+1) and f(m+1,n-2).

8. For following program, assume m and n are positive.

$$f(m,n) = m \qquad \text{if } n=0$$
$$= n \qquad \text{if } m=0$$
$$= f(m-1,n+1) + 2*f(m-1,n) + 3*f(m,n-1) + m*n \quad \text{otherwise}$$

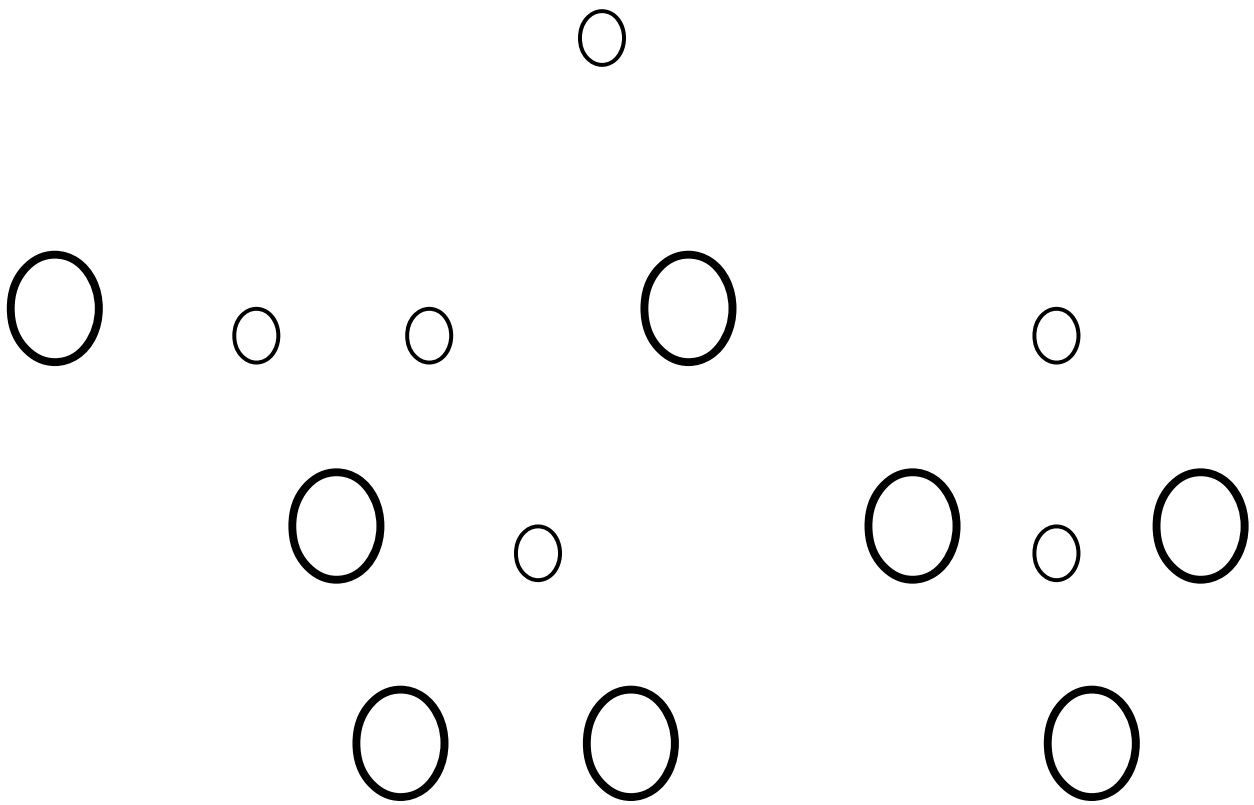
Do above problem when f(m+1,n-1) is used in place of f(m-1,n+1).
Do above problem by adding f(m-2,n+1) and f(m+1,n-2).

9. Write function given in the example so that to find f(5,9) the order of evaluation is f(5,5), f(6,6) f(5,6), f(7,7) f(6,7) f(5,7), f(8,8) f(7,8) f(6,8) f(5,8), f(9,9) f(8,9) f(7,9) f(6,9) f(5,9).

10. For following write program. Assume $n \ge m \ge 0$.

$$f(m,n) = n \qquad \text{if } m=0$$
$$= n^2 \qquad \text{if } m=1$$
$$= m \qquad \text{if } m=n$$
$$= f(m-2,n-1) + f(m,n-1) + f(m+1,n) + m*n+m \qquad \text{if } n>m$$

Tree (general)



The above tree is represented using following structure. Tag stores whether a node is data (1) or tree (0). Br stores pointer to next younger brother. DC stores data element if tag is 1. It stores pointer to first child if it is tree.

Index	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Tag	0	1	1	1	0	0	1	1	1	1	0	0	0	1	
DC	-1	23	44	81	9	3	71	91	77	41	14	10	2	42	
Br	12	8	11	6	-1	-1	-1	-1	1	13	7	4	-1	-1	

1. Write program to find index of children from index of node. e.g. child index of 12 are 10 and 13.
2. Write program to find index of grandchildren from index of node. e.g. input 12 output 2 and 8.
3. Write program to print breadth first traversal (BFT). For given tree it is 77 81 41 44 71 23 91 42.
4. Define function int Max (int root). Max(6) will output 71. Max(5) will output 91. Use both methods: BFT and recursion.
5. Define suitable function to find height of tree. For above case it is 3.
6. Define function to find index of father. e.g. index of father of 41 is 12.
7. Define function to search an element (X). If X is present then path (index on path) from root should be printed. e.g. input 91 output 5, 12, 13, 8.

8. Write program to print the tree. Define a function void Pt(int root). Pt(4) will print 81. Pt(11) will print (42). Pt(5) will print (77 ( ) (41 (23 91)) 81 (44 (42) 71)).
9. Write structure for following trees. Observe the output of above Pt function.
   [A] (17 29 (34 56 67) ((16) 24) 19) [B] (26 ((17 34 78)) 19 (( )) 90) [C] (24 (( ) ( )) 45)
10. Delete first child of a node, whose index is given. e.g. input 12 output (77 ( ) ((23 91)) 81 (44 (42) 71)).
11. Write program, which reads an integer X and deletes that from the tree. After deleting 23 the above tree will look as (77 ( ) (41 (91)) 81 (44 (42) 71)).
12. Delete all even elements. Given tree would look as (77 ( ) (41 (23 91)) 81 (( ) 71)).
13. Write program, which reads X and Y. The program inserts X as younger brother of Y. If X=98 and Y=44 then tree will look as (77 ( ) (41 (23 91)) 81 (44 98 (42) 71)).
14. Write program to insert as elder brother.
15. Write program, which will delete a node, whose index is given. If that node is tree then its successors are also deleted. After deleting 12 the above tree will look as (77 ( ) 81 (44 (42) 71)).
16. Modify above delete, so that when node is tree then its children are made sons of their grandfather. After deleting 12 the tree will look as (77 ( ) 41 (23 91) 81 (44 (42) 71)).
17. Delete x^{th} child of every node (which has atlest x children). If x=2 then above tree would look as (77 (41) 81 (44 71)). If x=3 then given tree will look as (77 ( ) 81 (44 (42))).
18. Given input in the form of binary tree (data, left, right are given), write program to convert it in above structure. Observe output using Pt function.

Sorting

1. Read n and n numbers and store them in array x_0, x_1 …. x_{n-1}. Assume array x is sorted. Now read another element y. Put the element at proper place in array x.

2. Using above concept define a recursive function insertsort(float a[], int n). It sorts 'n' elements of array 'a'. [Hint: Sort 'n-1' elements and then put n^{th} element at proper place]

3. Read n and n numbers and store them in array x_0, x_1 …. x_{n-1}. Similarly read m and m numbers and store them in array y_0, y_1…. y_{m-1}. Assume that both x and y is sorted arrays. Form another sorted array z (size m+n) from the elements of both given arrays. Use one for loop each for reading array x, reading y and outputting z. Use only while loop for forming z.

4. Using above concept define a recursive function mergesort(a,low,up,b). Sort elements of array 'a' between index low, low+1, …, up. The sorted array is stored in array 'b'. [Hint: Define mid=(low+up)/2. Sort the array elements in the range low…mid and mid+1…up. Now merge both parts, using the concept of above question.]

5. Rearrange the elements of an array, so that even elements are in the beginning and odd elements at end. Use only one loop.

6. Do above problem without using any other array.

7. Using above concept define a recursive function quicksort(a,low,up). Rearrange elements of array 'a' between limits low+1 and up in such a manner so that elements less than a_{low} are in the beginning and bigger at the end. Let 'k' be the index of boundary. Now exchange element a_{low} and a_k. Sort the array elements in the range low…k-1 and k+1…up.

8. Read n and n numbers x_1, x_2 …. x_n. Let these numbers satisfy property $x_i>x_{2i}$ and $x_i>x_{2i+1}$. Read another element and put it in such a manner so that given property is satisfied.
9. In above case delete x_1 and change location of elements so that above property is satisfied.
10. Use above concept to sort a array (heap sort)

11. Read n and n numbers x_1, x_2 …. x_n. Assume that given numbers are sorted. Now read another element and find its location in the array in O(log i) time. Here 'i' location. It is output size complexity.

12. Read n, k and n numbers a_0, a_1, … , a_{n-1}. Output numbers 0..n-1. 'i' is outputted before 'j' if $a_i<a_j$ or (($a_i=a_j$)and(i<j)). For the following input output is 3 0 2 4 6 1 5. Assume each a_i is between 1 and k.

a_0	a_1	a_2	a_3	a_4	a_5	a_6
2	3	2	1	2	3	2

13. Read m, n and n pair of numbers. Let 1..m be numbers. Every pair shows equivalent relation between them. Output equivalent classes. e.g. input m=14 n=10 and pairs are 1 10, 2 3, 7 11, 4 9, 10 12, 2 8, 9 7, 4 11, 3 8, 1 6. Output 1 6 10 12, 5, 2 3 8, 4 9 7 11, 13, 14.

Rearrangement of numbers such that even numbers are at the beginning and odd at end. Relative order of numbers within even and odd should not change.

```
#include<stdio.h>
main()
{  int i,j,k,t,n,a[100],b[100];
   printf("How many numbers ");scanf("%d",&n);
   for (i=1;i<=n;i++) scanf("%d",&a[i]);

   i=1;j=n;
   for (k=1;k<=n;k++)
      if (a[k]%2==0) { b[i]=a[k]; i++;}
      else           { b[j]=a[k]; j--;}
   for (j=j+1,k=n;j<k;j++,k--) {t=b[j];b[j]=b[k];b[k]=t;}

   for (i=1;i<=n;i++) printf("%4d",b[i]);
}
```

Radix sort using rearrangement

```
#include<stdio.h>
main()
{  FILE *fp; int t,r,i,j,k,n,p[100],q[100],a[100][10];
   fp=fopen("data","r");
   printf("How many numbers ");scanf("%d",&n);
   printf("How many digits  ");scanf("%d",&r);
   for (i=1;i<=n;i++) for (j=1;j<=r;j++)
       fscanf(fp,"%d",&a[i][j]);

   for (i=1;i<=n;i++) p[i]=i;
   for (t=r;t>=1;t--)
   {   i=1;j=n;
       for (k=1;k<=n;k++)
           if (a[p[k]][t]==0) { q[i]=p[k]; i++;}
           else               { q[j]=p[k]; j--;}
       for (i=1;i<=j;i++)          p[i]=q[i];
       for (j=j+1,k=n;j<=n;j++,k--) p[j]=q[k];
       for (i=1;i<=n;i++) printf("%3d",p[i]);
       printf("\n");
   }

   for (i=1;i<=n;i++)
   {   for (j=1;j<=r;j++) printf("%1d",a[p[i]][j]); printf("\n"); }
}
```

	a	p	q	p	q	p	q	p	q	p
1	1001	1	2	2	3	3	6	6	7	7
2	0110	2	3	3	6	6	1	1	5	5
3	1100	3	4	4	1	1	7	7	2	2
4	0110	4	6	6	5	5	4	3	4	4
5	0101	5	7	1	7	7	2	5	3	6
6	1000	6	5	5	4	2	5	2	1	1
7	0001	7	1	7	2	4	3	4	6	3

Pattern matching using automata

```
#include<stdio.h>
main()
{   int n,m=0,i,j,t,a[20],state[20][10];
    printf("\nGive length of pattern");scanf("%d",&n);
    printf("Give pattern");
    for (i=1;i<=n;i++) {scanf("%d",&a[i]); if (m<a[i]) m=a[i];} m++;

    for (i=1;a[i]==a[1];i++)
    {    for (j=0;j<m;j++) state[i-1][j]=0; state[i-1][a[i]]=i; }
    for (j=0;j<m;j++) state[i-1][j]=0;state[i-1][a[1]]=i-1;

    for (;i<=n;i++)
    {   t=a[i];
      for (j=0;j<m;j++) state[i][j]=state[state[i-1][t]][j];
      state[i-1][t]=i;
    }
    printf("     ");for(i=1;i<=n;i++) printf("%d ",a[i]);
    printf("\n   ");for(i=0;i<=n;i++) printf("%d ",i);
    for (j=0;j<m;j++)
    { printf("\n%d  ",j);for (i=0;i<=n;i++) printf("%d",state[i][j]); }
}
```

Searching

We are given two sorted arrays. We want to find n^{th} element in combined array in logn time. Assume that it is present in the first array.

```
#include<stdio.h>
main()
{   float a[50],b[50];
    int i,low,up,n,s1,s2;
    printf("Give sizes of sorted arrays ");
    scanf("%d %d",&s1,&s2);
    printf("Give first sorted array ");
    for (i=1;i<=s1;i++) scanf("%f",&a[i]);
    printf("Give second sorted array ");
    for (i=1;i<=s2;i++) scanf("%f",&b[i]);
    printf("Give the index of element to be searched");
    scanf("%d",&n);

    if (n-s2>1) low=n-s2; else low=1;
    if (s1<n) up=s1; else up=n;
    //    b[0]=a[0]-1;b[s2+1]=a[s1]+1;

    do{ i=(low+up)/2;
        printf("%d %d %d %d\n",low,up,i,n-i);
        if (((n==i)||(a[i]>b[n-i]))&&((n-i==s2)||(a[i]<b[n-i+1]))) break;
        if (a[i]<b[n-i])  low=i+1;
        if (a[i]>b[n-i+1]) up=i-1;
    } while (low<=up);
    if (low>up) printf("Element absent\n");
    else printf("%d %f\n",i,a[i]);
}
```

Equivalent Class

```
void replace(int a[],int n,int x,int y)
{ int i,t=a[x];
  for(i=1;i<=n;i++) if(a[i]==t) a[i]=a[y];
}
void initialize(int a[],int n)
{  int i;  for(i=1;i<=n;i++) a[i]=i;    }
void print(int a[20],int n,int  r)
{  int i,t=r;
   for(i=1;i<=n;i++) if(a[i]==t) printf("%d,",i);
   printf("\n");
}
main()
{   int i,n,m,p,q,w;int a[20];
    printf("give the no.of vertices");scanf("%d",&n);
    printf("give the no.of edges");scanf("%d",&m);
    initialize(a,n);
    for(i=1;i<=m;i++)
    { scanf("%d %d",&p,&q);
      if (a[p]!=a[q]) replace(a,n,p,q);
    }
    for(i=1;i<=n;i++) print(a,n,i);
}
```

The above program reads n and m and m pair of numbers. Let 1..n be numbers. Every pair shows equivalent relation between them. The program outputs equivalent classes.

Input m=14 n=10 and pairs are 1 10, 2 3, 7 11, 4 9, 10 12, 2 8, 9 7, 4 11, 3 8, 1 6.

Output 1 6 10 12, 5, 2 3 8, 4 9 7 11, 13, 14.

The following program achieves it in more efficient manner.

```
int next[100],size[100];
int root (int x)
{   int y=x;
    while (next[y]!=y) y=next[y];
    return y;
}
void unin (int x,int y)
{   int p,q;
    p=root(x);q=root(y);
    if (p!=q)
       if (size[p]>size[q]) { next[q]=p;size[q]+=size[p];}
       else                 { next[p]=q;size[p]+=size[p];}
}
main()
{   int gr[100][100],i,j,n,x,y;
    printf("How many elements");scanf("%d",&n);
    for (i=0;i<n;i++) {size[i]=1;next[i]=i;gr[i][0]=0;}
    do{printf("Give tuple");scanf("%d %d",&x,&y);
       if ((x==-1)||(y==-1)) break;
       unin(x,y);
       for (i=0;i<n;i++) printf("%3d",i);      printf("\n");
       for (i=0;i<n;i++) printf("%3d",next[i]);printf("\n");
       for (i=0;i<n;i++) printf("%3d",size[i]);printf("\n");
    }while (2==2);
    for (i=0;i<n;i++)
    {  j=root(i); gr[j][0]++; gr[j][gr[j][0]]=i; }
    for (i=0;i<n;i++){
```

```
        for (j=1;j<=gr[i][0];j++) printf("%d ",gr[i][j]);
        if (gr[i][0]!=0) printf("\n");
    }
}
```

Graph

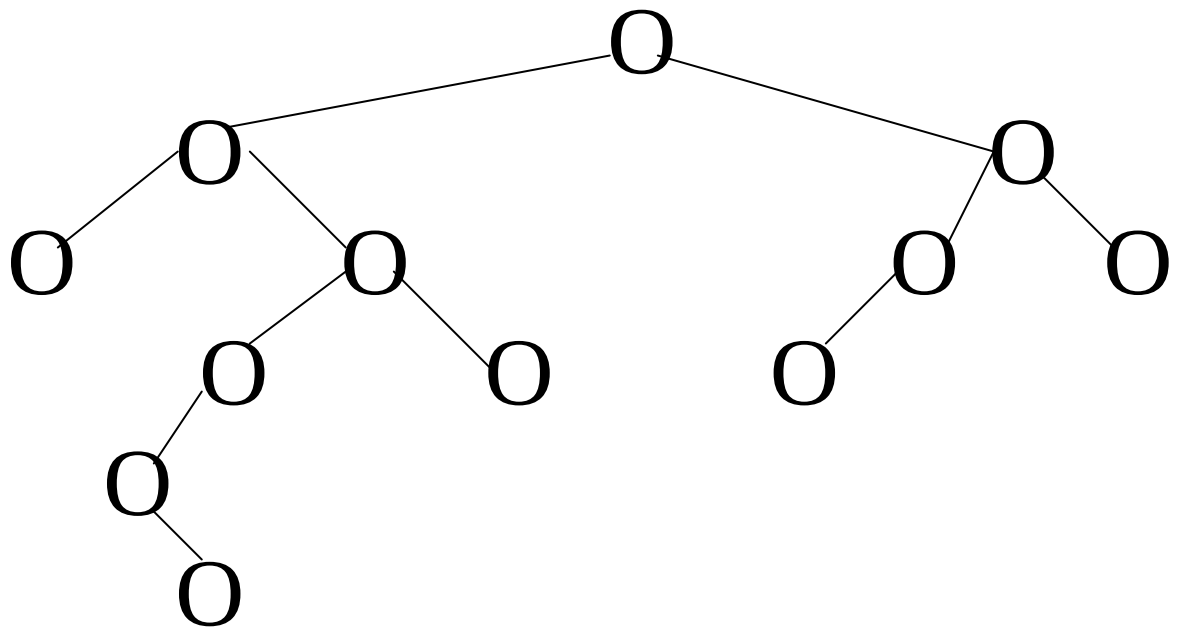
```
/* Find single source shortest paths using Dijkstra's Algorithm
   It finds shortest paths all vertex from vertex 1. */

#include<stdio.h>
main()
{   int i,j,n,c,d[10],e[10][10],t[10],min;
    printf("howmany nodes");scanf("%d",&n);
    printf("Give edge matrix for nodes 1..%d",n);
    for (i=1;i<=n;i++)
        for (j=1;j<=n;j++)
            scanf("%d",&e[i][j]);
    printf("\n");
    for (i=2;i<=n;i++) printf(" %2d ",i); printf("\n");
    for (i=2;i<=n;i++) t[i]=0;
    for (i=2;i<=n;i++) d[i]=e[1][i];
    for (c=1;c<n;c++)
    {   for (i=2;i<=n;i++) printf("%3d ",d[i]);
        min=100000;
        for (i=2;i<=n;i++)
            if ((d[i]<min)&&(t[i]==0)) { j=i; min=d[i]; }
        t[j]=1;printf ("  taken %d\n",j);
        for (i=2;i<=n;i++)
            if (d[i]>d[j]+e[j][i])
                d[i]=d[j]+e[j][i];
    }
}
6
13 09 07 03 17 19
05 05 03 05 13 00
05 05 05 05 01 05
05 03 07 09 09 13
03 03 03 03 03 03
07 07 07 07 05 07

howmany nodes 6
Give edge matrix for nodes 1..6
  2   3   4   5   6
  9   7   3  17  19   taken 4
  6   7   3  12  16   taken 2
  6   7   3  12   6   taken 6
  6   7   3  11   6   taken 3
  6   7   3   8   6   taken 5

When 01 is replaced by -6 then this algorithm fails.
```

Tree Traversal



The representation of above binary tree is as

Size=12 Root=9

Index	1	2	3	4	5	6	7	8	9	10	11	12
Left	11	10	12	-1	2	-1	-1	-1	3	-1	8	-1
Right	7	-1	1	-1	4	-1	-1	6	5	-1	-1	-1
Father	3	5	9	5	9	8	1	11	-1	2	1	3

The various traversals are as follows

Preorder: 9, 3, 12, 1, 11, 8, 6, 7, 5, 2, 10, 4

Inorder: 12, 3, 8, 6, 11, 1, 7, 9, 10, 2, 5, 4

Postorder:12, 6, 8, 11, 7, 1, 3, 10, 2, 4, 5, 9

Leaforder:12,6,7,10,4

Preorder Traversal

```
int left[20],right[20],father[20],i,j,size,root;
printf("How many nodes");scanf("%d",&size);
printf("Give root");scanf("%d",&root);
printf("Give index");for (i=1;i<=size;i++)scanf("%d",&j);
printf("Give left array");for (i=1;i<=size;i++)scanf("%d",&left[i]);
printf("Give right array");for (i=1;i<=size;i++)scanf("%d",&right[i]);
printf("Give father array");for (i=1;i<=size;i++)scanf("%d",&father[i]);
i=root;
while (i>0)
{  printf("%d \n",i);
   if (left[i]>0) i=left[i];
   else if (right[i]>0) i=right[i];
   else
   {   while ((father[i]>0)&&((right[father[i]]<0)||(right[father[i]]==i)))
           i=father[i];
       if (father[i]<0) break; else i=right[father[i]];
   }
}
```

Inorder Traversal

```
i=root;while (left[i]>0) i=left[i];
while (i>0)
{  printf("%d \n",i);sleep(1);
   if (right[i]>0){
        i=right[i];
        while (left[i]>0) i=left[i]; }
   else {
        while ((father[i]>0) && (right[father[i]]==i))
             i=father[i];
        i=father[i]; }
}
```

Postorder Traversal

```
i=root;
while ((left[i]>0)||(right[i]>0))
     if (left[i]>0) i=left[i]; else i=right[i];
while (i>0)
{  printf("%d \n",i);sleep(1);
   j=father[i]; if (j<=0) break;
   if ((right[j]<=0)||(right[j]==i)) i=j;
   else{
        i=right[j];
        while ((left[i]>0)||(right[i]>0))
             if (left[i]>0) i=left[i]; else i=right[i];
        }
}
```

Leaforder Traversal

```
i=root;
while ((left[i]>0)||(right[i]>0))
     if (left[i]>0) i=left[i]; else i=right[i];
while (i>0)
{  printf("%d \n",i);sleep(1);
   while ((father[i]>0)&&((right[father[i]]<0)||(right[father[i]]==i)))
        i=father[i];
   if (father[i]<0) break;
   i=right[father[i]];
   while ((left[i]>0)||(right[i]>0))
        if (left[i]>0) i=left[i]; else i=right[i];
}
```

Location of leaf

```
int cnt(int x,int left[], int right[])
{   if ((left[x]<=0) && (right[x]<=0)) return 1;
    if ((left[x]>0)  && (right[x]<=0)) return cnt(left[x],left,right);
    if ((left[x]<=0) && (right[x]>0))  return cnt(right[x],left,right);
    if ((left[x]>0)  && (right[x]>0))
              return cnt(left[x],left,right)+cnt(right[x],left,right);
}    This function finds how many leafs are reachable from x
main()
{                                  ...........
                                   ...........
    for (i=1;i<=size;i++)count[i]=cnt(i,left,right);
    for (i=1;i<=size;i++)
    {   if ((left[i]<=0)&&(right[i]<=0)) c[i]=1;
        else if (left[i]>0) c[i]=count[left[i]]; else c[i]=0;
    }
    for (j=1;j<=size;j++)
    {   if ((left[j]<=0)&&(right[j]<=0))
        {   s=1;i=j;
            while (father[i]>0)
            { if (right[father[i]]==i) s=s+c[father[i]];
              i=father[i];
            }
            printf("Location[%d]=%d\n",j,s);
        }
    }
}
```

Location of a node in inorder traversal

```
int cnt(int x,int left[], int right[])
{   if (x<=0) return 0;
    else return cnt(left[x],left,right)+cnt(right[x],left,right)+1;
}      This function finds how many nodes are reachable from x
main()
{                                   ...........
    for (i=1;i<=size;i++)count[i]=cnt(i,left,right);
    for (i=1;i<=size;i++)
         if (left[i]>0) c[i]=count[left[i]]; else c[i]=0;
    for (j=1;j<=size;j++)
    {   s=c[j]+1;i=j;
        while (father[i]>0)
        {   if (right[father[i]]==i)
               s=s+c[father[i]]+1;
            i=father[i];
        }
        printf("Location[%d]=%d\n",j,s);
    }
}
```

Location in postorder traversal

```
   s=count[i];i=j;
   while (father[i]>0)
   {   if (right[father[i]]==i)
           s=s+c[father[i]];
       i=father[i];
   }
   printf("Location[%d]=%d\n",j,s);
```

Location in preorder traversal

```
   s=1;i=j;
   while (father[i]>0)
   {   if (right[father[i]]==i)
          s=s+c[father[i]];
       i=father[i]; s=s+1;
   }
   printf("Location[%d]=%d\n",j,s);
```

Given location find corresponding leaf

```
int cnt(int x,int left[], int right[])
{   if ((left[x]<=0) && (right[x]<=0)) return 1;
    if ((left[x]>0)  && (right[x]<=0)) return cnt(left[x],left,right);
    if ((left[x]<=0) && (right[x]>0))  return cnt(right[x],left,right);
    if ((left[x]>0)  && (right[x]>0))
          return cnt(left[x],left,right)+cnt(right[x],left,right);
}           This function finds how many leafs are reachable from x
main()
{                                   ...........
                                    ...........
    for (i=1;i<=size;i++)count[i]=cnt(i,left,right);
    for (i=1;i<=size;i++)
    {   if ((left[i]<=0)&&(right[i]<=0)) c[i]=1;
        else if (left[i]>0) c[i]=count[left[i]]; else c[i]=0;
    }
    for (j=1;j<=count[root];j++)
    {   i=root;L=j;
        while ((left[i]>0)||(right[i]>0))
        {   if (L<=c[i]) i=left[i];
            else {L=L-c[i];i=right[i];}
        }
        printf("Location[%d]=%d\n",j,i);
    }
}
```

Given location find corresponding node in inorder traversal

```
int cnt(int x,int left[], int right[])
{   if (x<=0) return 0;
    else return cnt(left[x],left,right)+cnt(right[x],left,right)+1;
}      This function finds how many nodes are reachable from x
main()
{                                   ...........
    for (i=1;i<=size;i++)count[i]=cnt(i,left,right);
    for (i=1;i<=size;i++)
         if (left[i]>0) c[i]=count[left[i]]; else c[i]=0;
    for (j=1;j<=count[root];j++)
    {   i=root;L=j;
        while (L!=c[i]+1)
        {   if (L<=c[i]) i=left[i];
            else {L=L-c[i]-1;i=right[i];}
        }
        printf("Location[%d]=%d\n",j,i);
    }
}
```

Node in preorder traversal

```
i=root;L=j;
while (L!=1)
{   if (L<=c[i]+1) i=left[i];
    else {L=L-c[i];i=right[i];}
    L=L-1;
}
printf("Location[%d]=%d\n",j,i);
```

Node in postorder traversal

```
i=root;L=j;
while (L!=count[i])
{   if (L<=c[i]) i=left[i];
    else {L=L-c[i];i=right[i];}
}

printf("Location[%d]=%d\n",j,i);
```

Huffman Code

Given two sorted arrays, output elements as follows: First pair of two smallest elements, then pair of next two smallest elements, then pair of next two smallest elements and so on. Assume that total number of elements are even and within a pair the elements can be outputted in any order e.g. input arrays (6,9,14) and (2,4,5,7,8) output (2,4)(6,5)(7,8)(9,14)

```
main()
{   int a[20],b[20],i,j,m,n;
    printf("Give sizes of arrays");scanf("%d %d",&m,&n);
    for (i=1;i<=m;i++) scanf("%d",&a[i]);
    for (i=1;i<=n;i++) scanf("%d",&b[i]);
    a[m+1]=1000;b[n+1]=1000;i=1;j=1;
    while ((i<=m)||(j<=n))
        if (a[i+1]<b[j]) {printf("(%d %d)",a[i],a[i+1]);i=i+2;}
        else if (b[j+1]<a[i]) {printf("(%d %d)",b[j],b[j+1]);j=j+2;}
        else {printf("(%d %d)",a[i],b[j]);i++;j++;}
}
```

Following is program for Huffman code. Given few letters and their frequencies, the letters are put on leafs of binary tree such that weighted sum of frequencies is minimum. The weight of the frequency is taken as the depth of corresponding letter.

```
#include<stdio.h>
main()
{   int frequency[20],father[20],i,j,k,n;
    printf("How many letters");scanf("%d",&n);
    printf("Give frequency in nondecreasing order");
    for (i=1;i<=n;i++)scanf("%d",&frequency[i]);
    for (i=n+1;i<=2*n+1;i++) frequency[i]=10000;
    i=1;j=n+2;k=n+2;father[2*n]=-1;
    while (k<=2*n)
    {   if (frequency[i+1]<frequency[j])
        {   frequency[k]=frequency[i]+frequency[i+1];
            father[i]=k;father[i+1]=k;i=i+2; }
        else if (frequency[j+1]<frequency[i])
        {   frequency[k]=frequency[j]+frequency[j+1];
            father[j]=k;father[j+1]=k;j=j+2; }
        else
        {   frequency[k]=frequency[i]+frequency[j];
            father[i]=k;father[j]=k;j=j+1;i=i+1; }
        k=k+1;
    }
    for (i=1;i<=n;i++)
    {   printf("\n");j=i;
        while(j>0){printf("%d ",frequency[j]);j=father[j];}
    }
}
```

If input is 6 elements, 6, 9, 12, 13, 16 and 44 then output is

```
 6 15 31 56  100
 9 15 31 56  100
12 25 56 100
13 25 56 100
16 31 56 100
44 100
```

Common Substring

```
#include<stdio.h>
main()
{  int i,j,k,m,n,max=0,p,q;
   char s1[20],s2[20];
   printf("Give both strings\n");gets(s1);gets(s2);
   n=strlen(s1);m=strlen(s2);
   for (j=0;j<m;j++)
      for (i=0;i<n;i++){
      for (p=i,q=j;s1[p]==s2[q] && p<n && q<m;p++,q++);
      if (p-i>max) {max=p-i;k=i;}
      }
   printf("\nLongest lenth substring is between ");
   printf("%d and %d in first string",k,k+max-1);
}
```

```
#include<stdio.h>
main()
{  int i,j,max=0,p,q,m,n,t[20];
   char s1[20],s2[20];FILE *fp;
   fp=fopen("output","w");
   printf("Give both strings\n");gets(s1);gets(s2);
   n=strlen(s1);m=strlen(s2);
   fprintf(fp,"\n %s",s1);
   for (i=0;i<n;i++) t[i]=0;
   for (j=0;j<m;j++){
      for (i=n-1;i>=0;i--){
      if (s2[j]==s1[i])
          if (i>0) t[i]=t[i-1]+1; else t[i]=1;
      else t[i]=0;
      if (t[i]>max) {max=t[i];p=i;q=j;}
      }
      fprintf(fp,"\n%c",s2[j]);
      for (i=0;i<n;i++) fprintf(fp,"%d",t[i]);
   }
   printf("\nLongest lenth substring is between ");
   printf("%d and %d in first string",p-max+1,p);
}
```

```
 babaacabdcc
a01011010000
a01012010000
b10200002000
a02031010000
c00000200011
a01011030000
b10200004000
d00000000500
c00000100061
b10100001000
a02021010000
c00000200011
```

```
 ababca
a101001
a101001
b020200
c000030
b010100
```

Greedy Algorithm: Task Scheduling Problem

Deadlines and utility of jobs are given. Every job requires unit time. Which jobs should be done and during which time slot, so that total utility is maximum.

Job	A	B	C	D	E	F	G	H	I
Deadline	1	2	4	3	2	1	4	3	3
Utility	75	20	70	30	30	80	20	40	60

Nearest deadline first: Arrange jobs in non decreasing order of deadline. In case of equal deadline in the non increasing order of utility. 1-F 2-E 3-I 4-C

Job	F	A	E	B	I	H	D	C	G
Deadline	1	1	2	2	3	3	3	4	4
Utility	80	75	30	20	60	40	30	70	20

Maximum profit first: Arrange jobs in non increasing order of utility. 1-F 2-C 3-I 4-G

Job	F	A	C	I	H	D	E	B	G
Deadline	1	1	4	3	3	3	2	2	4
Utility	80	75	70	60	40	30	30	20	20

Maximum profit must: Do the job which gives maximum profit as late as possible
1-F 4-C 3-I 2-H

```
#include<stdio.h>
main()
{  int i,j,m,n,deadline[100],utility[100],slot[100],full[100];
   printf("Give number of jobs and slots"); scanf("%d %d",&n,&m);
   printf("Give utility (in non increasing order) and deadline of jobs ");
   for (i=0;i<n;i++) scanf("%d %d",&utility[i],&deadline[i]);
   full[0]=2; for (i=1;i<=m;i++) full[i]=0;

   for (i=0;i<n;i++)
      if (full[deadline[i]]!=2)
      {   for (j=deadline[i];full[j]==1;j--);
          slot[j]=i; full[j]=1;
          while ((full[j-1]==2) && (full[j]==1))
          {   full[j]=2; j++; }
      }

   for (i=1;i<=m;i++) printf("%dth slot allocated to %d\n",i,slot[i]);
}
```

full[i]=0 means that ith slot is free. full[i]=1 means that it is allocate. full[i]=2 means that it is allocated and earlier slots are also allocated. The complexity of above algorithm is $O(n+m^2)$. When the for loop is replaced by following the complexity becomes O(mn). When jobs are not given in sorted order then complexity becomes $O(n\log_2 n+n+m^2)$ and $O(n\log_2 n+mn)$ respectively.

```
for (i=0;i<n;i++)
   {   for (j=deadline[i];(full[j]==1 && j>0);j--);
       if (j!=0) {   slot[j]=i; full[j]=1;   }
   }
```

rr_1

5	**7**	**9**	**3**	**4**
7	9	11	5	6
6	8	10	4	5
2	4	6	0	1

```
pt(int x) {printf("%3d",x);}
cm()      {printf(",");}
nl()      {printf("\n");}

pl (int a)
{    pt(a); pt(a+2); pt(a+4); pt(a-2); pt(a-1); nl(); }

main()
{    pl(5);
     pl(7);
     pl(6);
     pl(2);
}
```

rrr_1

5	**7**	**9**	**3**	**4,**	**2**	**3**	**4**	**1**	**5,**	**4**	**6**	**3**	**8**	**2**
7	9	11	5	6,	**3**	4	5	2	6,	**5**	7	4	9	3
6	8	10	4	5,	**2**	3	4	1	5,	**5**	7	4	9	3
2	4	6	0	1,	**1**	2	3	0	4,	**8**	10	7	12	6

```
pg1 (int a)
{    pt(a); pt(a+2); pt(a+4); pt(a-2); pt(a-1); cm(); }
pg2 (int a)
{    pt(a); pt(a+1); pt(a+2); pt(a-1); pt(a+3); cm(); }
pg3 (int a)
{    pt(a); pt(a+2); pt(a-1); pt(a+4); pt(a-2); cm(); }

pl (int x, int y, int z)
{    pg1(x); pg2(y); pg3(z); nl();}

main()
{    pl(5,2,4);
     pl(7,3,5);
     pl(6,2,5);
     pl(2,1,8);
}
```

rrr_2

```
5  7  3  1,  2  4  6  2,  8  5  3  1
4  2  6  1,  1 -1  9  2,  7  0  6  1
6  3  1  2,  3  0  4  3,  9  1  1  2
1  1  6  9, -2 -2  9 10,  4 -1  6  9
```

```
pl (int a, int b, int c, int d)
{    pt(a);   pt(b);   pt(c);   pt(d);   cm();
     pt(a-3); pt(b-3); pt(c+3); pt(d+1); cm();
     pt(a+3); pt(b-2); pt(c);   pt(d);   cm(); nl();
}
main()
{    pl(5,7,3,1);
     pl(4,2,6,1);
     pl(6,3,1,2);
     pl(1,1,6,9);
}
```

rrr_{12}

```
5  7  3  1,  2  4  6  2,  8  5  3  1
4  2  6  1,  6  4 14  7,  8  1  7  2
6  3  1  2,  4  1  5  4,  1 -7 -7 -6
1  1  6  9,  2  2 13 14,  3 -2  5  8
```

```
pl (int a, int b, int c, int d,int k, int l)
{    pt(a); pt(b);       pt(c);       pt(d);       cm();
     pt(k); pt(k-a+b)  ; pt(k-a+c+6); pt(k-a+d+4); cm();
     pt(l); pt(l-a+b-5); pt(l-a+c-3); pt(l-a+d-3); cm(); nl();
}
main()
{    pl(5,7,3,1,2,8);
     pl(4,2,6,1,6,8);
     pl(6,3,1,2,4,1);
     pl(1,1,6,9,2,3);
}
```

rRr

3	**7**	**6,**	4	8	7,	5	9	8
4	**8**	**9,**	5	9	10,	6	10	11
2	**3**	**1,**	3	4	2,	4	5	3
3	**2**	**6,**	4	3	7,	5	4	8

$rrr_{1,2}$

5	**7**	**2,**	**6**	**9**	**1,**	**4**	**2**	**3**
8	10	5,	9	12	4,	7	5	6
2	4	-1,	3	6	-2,	1	-1	0

$rr_1r_{1,2}$

5	**7**	**2,**	**6**	8	3,	**4**	6	1
8	10	5,	9	11	6,	7	9	4
2	4	-1,	3	5	0,	1	3	-2

```
  5  7  9  3,  7  9 11  5,  8 10 12  6,  4  6  8  2,  9 11 13  7,  6  8 10  4,
  2  4  6  2,  4  6  8  4,  5  7  9  5,  1  3  5  1,  6  8 10  6,  3  5  7  3,
  1  3  2  1,  3  5  4  3,  4  6  5  4,  0  2  1  0,  5  7  6  5,  2  4  3  2,
  4  6 10  1,  6  8 12  3,  7  9 13  4,  3  5  9  0,  8 10 14  5,  5  7 11  2,
  7  6 12  5,  9  8 14  7, 10  9 15  8,  6  5 11  4, 11 10 16  9,  8  7 13  6,
XXXXX
  2  4  1  6,  4  6  3  8,  5  7  4  9,  1  3  0  5,  6  8  5 10,  3  5  2  7,
 -1  1 -2  5,  1  3  0  7,  2  4  1  8, -2  0 -3  4,  3  5  2  9,  0  2 -1  6,
 -2  0 -6  4,  0  2 -4  6,  1  3 -3  7, -3 -1 -7  3,  2  4 -2  8, -1  1 -5  5,
  1  3  2  4,  3  5  4  6,  4  6  5  7,  0  2  1  3,  5  7  6  8,  2  4  3  5,
  4  3  4  8,  6  5  6 10,  7  6  7 11,  3  2  3  7,  8  7  8 12,  5  4  5  9,
XXXXX
  1  6  2  3,  3  8  4  5,  4  9  5  6,  0  5  1  2,  5 10  6  7,  2  7  3  4,
 -2  3 -1  2,  0  5  1  4,  1  6  2  5, -3  2 -2  1,  2  7  3  6, -1  4  0  3,
 -3  2 -5  1, -1  4 -3  3,  0  5 -2  4, -4  1 -6  0,  1  6 -1  5, -2  3 -4  2,
  0  5  3  1,  2  7  5  3,  3  8  6  4, -1  4  2  0,  4  9  7  5,  1  6  4  2,
  3  5  5  5,  5  7  7  7,  6  8  8  8,  2  4  4  4,  7  9  9  9,  4  6  6  6,
XXXXX
  4  1  5  8,  6  3  7 10,  7  4  8 11,  3  0  4  7,  8  5  9 12,  5  2  6  9,
  1 -2  2  7,  3  0  4  9,  4  1  5 10,  0 -3  1  6,  5  2  6 11,  2 -1  3  8,
  0 -3 -2  6,  2 -1  0  8,  3  0  1  9, -1 -4 -3  5,  4  1  2 10,  1 -2 -1  7,
  3  0  6  6,  5  2  8  8,  6  3  9  9,  2 -1  5  5,  7  4 10 10,  4  1  7  7,
  6  0  8 10,  8  2 10 12,  9  3 11 13,  5 -1  7  9, 10  4 12 14,  7  1  9 11,
XXXXX
  1  1  2  4,  3  3  4  6,  4  4  5  7,  0  0  1  3,  5  5  6  8,  2  2  3  5,
 -2 -2 -1  3,  0  0  1  5,  1  1  2  6, -3 -3 -2  2,  2  2  3  7, -1 -1  0  4,
 -3 -3 -5  2, -1 -1 -3  4,  0  0 -2  5, -4 -4 -6  1,  1  1 -1  6, -2 -2 -4  3,
  0  0  3  2,  2  2  5  4,  3  3  6  5, -1 -1  2  1,  4  4  7  6,  1  1  4  3,
  3  0  5  6,  5  2  7  8,  6  3  8  9,  2 -1  4  5,  7  4  9 10,  4  1  6  7,
XXXXX
  2  3  1  5,  4  5  3  7,  5  6  4  8,  1  2  0  4,  6  7  5  9,  3  4  2  6,
 -1  0 -2  4,  1  2  0  6,  2  3  1  7, -2 -1 -3  3,  3  4  2  8,  0  1 -1  5,
 -2 -1 -6  3,  0  1 -4  5,  1  2 -3  6, -3 -2 -7  2,  2  3 -2  7, -1  0 -5  4,
  1  2  2  3,  3  4  4  5,  4  5  5  6,  0  1  1  2,  5  6  6  7,  2  3  3  4,
  4  2  4  7,  6  4  6  9,  7  5  7 10,  3  1  3  6,  8  6  8 11,  5  3  5  8,
XXXXX
```

The type of above picture is

$$\mathbf{rr_1r_2r_{2,3}}$$

No printf statement should be used. Except in the definition of following procedures.

```
pt (int x){printf("%3d",x);}
cm(){printf(",");}
nl(){printf("\n");}
px(){printf("XXXXX\n");}
```

The number of (;) and no of (,) should be least. The program given at right hand side can be used to count the number of (,)+(;).

```
#include<stdio.h>
#include<string.h>
main()
{   FILE *fp;int count=0;
    char s[40],c;
    printf("Give filename");
    gets(s);
    fp=fopen(s,"r");
    do{
       fscanf(fp,"%c",&c);
       if ((c==',')||(c==';'))
           count++;
    }while (!(feof(fp)));
    printf(" %d",count);
}
```

Operating Systems Lab

Multi valued Functions

Parallel Processing

Common Variables

Signal

File Low Level System Calls

Lock

Pipe

Buffer

File High level System Calls

Buffer manager

Shell Programming (making commands)

Test Papers

Fork

```
printf("X\n");
fork( );
printf("Y\n");
```

The above program prints one X and two Y's. When fork system call is executed, a duplicated copy of the program is made. Both copies of the program are executed independently. Each copy is called a process.

1. Without typing the following program on the computer, state the number of X, Y and Z will be printed.

```
fork( );fork( );
printf("X\n");
fork( );
printf("Y\n");
fork( );fork( );fork( );
printf("Z\n");
```

Process Identifier (pid)

When we execute a program, a process is created. Every process has got an identifier (henceforth id). It can be obtained by calling getpid( ). Different execution of the same program create different processes, hence their identifiers are different. Every process has got a father. It is the process, which created it. When we give a command to execute a program, the father is the login process. If we execute the following one line program, two numbers will be outputted.

```
printf("%d %d\n",getpid( ),getppid( ));
```

On different executions of the above program, the first number will be different, but the second number will be same. It is because we are in same login process. However, if we logout and again login, then the second number will also be different.

```
printf("%d  %d\n",getpid( ),getppid( ));
fork( );
printf("%d  %d\n",getpid( ),getppid( ));
sleep(1);
```

The above program outputs

A L
A L
B A

Here L is the id of the login process. A is the id of the parent process and B is the id of the process created by fork( ). The father of the process B is A. The second and the third lines of the above may be exchanged.

If we remove sleep(1) then following two outputs are possible

A L		A L
A L	or	B A
B 1		A L

It is because, if after the fork the parent executes first, then it will print its id, its parent (login) id, and will terminate. Now child will be adopted by a process, whose id is 1 (It may be different on some system). When parent of a process dies then it is adopted by process of id 1.

```
printf("%d\n",getpid( ));
p=getpid( );
fork( );
while (getppid( )==p);
printf("%d\n",getpid( ));
sleep(2);
```

The output of the above program is A A B. A and B are parent and child id's respectively. B is printed 2 seconds after both A's are printed. It is because, the child keeps on waiting, till the parent is over by (while (getppid( )==p);). A child can know, whether its parent is over, by comparing its current parent id by the id of the parent at the time when it was created by fork.

The following program shows that parent and child execute alternatively.

```
p=getpid( ); fork( );
for ( i=1 ; i<1000000 ; i++)
    if (p==getpid( )) printf("X");
    else printf("Y");
```

Here X and Y are mixed.

The following program creates a child. The parent prints X and child prints Y. These may be printed in any order.

```
p=getpid( );
fork( );
if (p==getpid( )) printf("X");
if (p==getppid( )) printf("Y");
sleep(1);
```

If we remove sleep(1) then child may not print any thing. Hence possible outputs are YX or X. The output XY is possible only in case parent prints X and before it halts the child finds its parent's id.

2. How many X and Y are printed by following program? What is output if sleep(1) is removed?

```
fork( );fork( );
p=getpid( );
fork( );fork( );fork( );
if (p==getpid( )) printf("X");
if (p==getppid( )) printf("Y");
sleep(1);
```

3. How many X and Y are printed by following program? What is output if the line containing sleep(2) is removed?

```
fork( );fork( );
p=getpid( );
fork( );fork( );fork( );
if (p==getpid( ))  sleep(4);
fork( );
if (p==getppid( )) sleep(2);
if (p==getppid( )) printf("Y");
sleep(1);
```

Answer: 12 Y's. If we remove the line then 4 more Y's are printed.

4. How many X and Y will be printed?

```
p=getpid( );
for ( i=1 ; i≤10 ; i++) fork( );
if (p==getpid( )) printf("X");
if (p==getppid( )) printf("Y");
sleep(1);
```

5. Create 11 process using 4 fork( )'s. Do not use getppid( ). The number of getpid( )'s used should be minimum. Do not use loop.

6. Suppose there is a function getapid(x). It returns the id of xth ancestor of a process. e.g. getapid(0) is same as getpid( ). getapid(1) is same as getppid( ). getapid(2) returns the id of grandfather. How many A's will be the printed by the following program for
   (A) n=5 and r=2
   (B) n=5 and r=3
   (C) Express the number of A's as a function of n and r.

```
p=getpid( );
for ( i=1 ; i≤n ; i++)  fork( );
if  ( p==getapid(r) ) printf("A");
sleep(1);
```

The return value

The system call fork( ) is a function with side effect. As the side effect, two copies of the running process are made. The function call fork( ) returns 0 in the child process and the child's id in the parent process. The output of the following program will be

```
A L                 A L
A L B        or     B A 0
B 1 0               A L B
```

```
int q;
printf("%d %d\n",getpid( ),getppid( ));
q=fork( );
printf("%d%d%d\n",getpid(),getppid(),q);
```

7. How many X will be printed by following program?

```
p=fork( ); q=fork( );
if (p==0) fork( );
fork( );
printf("X");
```

8. How many X and Y are printed by the following program?

```
p=fork( );fork( );q=fork( );
if (p==q) printf("X");
else printf("Y");
```

The following program creates process B and C. The parent of B is A and the parent of C is B.

```
int q;
q=fork( );
if (q==0) fork( );
printf("%d%d\n",getpid( ),getppid( ));
sleep(1);
```

The output of the above program will look as follows:

```
A L
B A
C B
```

The order of lines may be exchanges.

9. Write a program, which creates 10 additional processes. Every process prints its id. Use only 4 fork's.

10. Write a program, which reads (int n) and creates n additional processes. Use less then $(\log_2 n)+1$ fork's.

11. Write a program, which creates processes B, C, D, and E. The parent of B, C and D is A. The parent of E is B.

12. Read n and create a system of 2^n-1 processes. Every process has exactly two children. If n=3 and following line is written at the end

```
printf("[%d %d]",getpid( ),getppid( ));
sleep(1);
```

then the output will be

[AL],[BA],[CA],[DB],[EB],[FC],[GC]

In following program a function aork( ) is defined. It creates one additional process. In the parent the id of child is returned. In child the id of parent is returned.

```
int aork( )
{   int p;
    p=fork( );
    if (p==0) return(getppid( ));
    else return(p);
}
main( )
{   int q;
    q=aork( );
    printf("%d%d%d\n",getpid( ),
                              getppid( ),q);
    sleep(1);
}
```

The output of above will be

```
A  L  B
B  A  A
```

If we remove sleep(1) then parent of B may be outputted as 1.

13. Define a function bork( ). It creates two additional processes. Let A be the id of parent and B and C are id's of the child. The parent of both B and C is A. The function returns C in parent process, B in C, and A in B. Hence the same main program (when function call q=bork( ) is made) will output (lines may be exchanged)

    A L C
    B A A
    C A B

14. Define a function cork( ). It creates 2 addition processes. Let parent be A, child be B and grandchild be C. A returns id of B. B returns id of C and C returns id of A.

15. Define a function cark( ). It creates 3 addition processes. Let parent be A, child be B, grandchild be C, and great grandchild be D. A returns id of B. B returns id of C, C returns id of D and D returns id of A.

16. Define a function dork(int n). It creates 2^n-1 more processes by using n fork( ). The parent returns 1. Every child (these shall be n children) returns 2. Every grand child returns 3. The process, whose father's father 's father's … (n times) ... father is the original process, returns n.

17. Define a function eork( ). It creates 10 more processes. Let A be the id of main process and B, C, D, … ,K are id's of other processes. A is father of B, B is father of C, C is father of D, …, J is father of K. In C the function call returns A, in D it returns B, … , in K it returns I, and in A and B it returns 0.

Separate Memory Image

The following program illustrates that both processes (created by fork( )) run independently. Both have got different copies of the variables. The variables modified by one process are not visible to the other process.

```
int a=10,b=20,q;
q=fork( );
if (q==0)  a=a+1;
else       b=b+1;
printf("%d %d %d\n",q,a,b);
```

The output is

```
0   11   20
X   10   21
```

X is id of child (some non-zero number).

Here the child process modifies 'a' and the parent process modifies 'b'. The value of 'b' in child process (and 'a' in parent process) remains unchanged.

However, value of a variable can be sent from one process to another (parent child) by a pipe. One of them writes into the pipe and another reads.

```
int a,b,p[2],q,t,sum,prod,ans;
scanf("%d %d",&a,&b);
pipe(p);
q=fork( );
if (q==0)
 { sum=a+b;
   read(p[0],&t,4);
   ans=t/sum;
   printf("%d",ans);
 }
else
 { prod=a*b;
   write(p[1],&prod,4);
 }
```

In above program (a*b)/(a+b) is computed with the help of two processes. The computation of (a+b) is done by the child and that of (a*b) is done by the parent. The parent sends its result to the child. (The parent writes the result in a pipe and the child reads from pipe).

18. Write a program, which computes 1+2+3+ . . . + 100. The sum of first 50 numbers is calculated by child and the sum of the remaining number is calculated by parent. The child sends its result to parent. The parent calculates the final answer.

19. Define a function gork( ). It creates one additional process. In the parent the id of child is returned. In child the id of parent is returned. Do not use getppid( ) and value returned by fork.

20. Define function hork( ). It creates one additional process. In parent the id of child is returned. In child 0 is returned. In the definition do not use the value returned by fork ( ). [i.e. do not use p=fork( ) (i.e. return value of fork). You may use fork( )]

21. Define a function iork( ). It creates 2 addition processes. Let parent be A, child be B and grandchild be C. A returns id of C. B returns id of A and C returns id of B. [Hint: make a pipe between A-B. B will communicate its child's id to A using pipe.]

22. Define a function jork( ). It creates 3 addition processes. Let parent be A, child be B, grandchild be C, and great grandchild be D. A returns id of D. B returns id of C, C returns id of D and D returns id of A. [Hint: make two pipes one between A-B and another between B-C. C will send its child's id to B and B will send it (whatever C has send) to A]

23. Define a function kork( ). It creates 2 addition processes. Let parent be A and B and C be children of A. A returns 0. B returns id of C and C returns id of B.

24. Define a function lork( ). It creates 10 more processes. Let A be the id of main process and B, C, D, … ,K are id's of other processes. A is father of B, B is father of C, C is father of D, …, J is father of K. In A the function call returns C, in B it returns D,…, in I it returns K, in J and K it returns 0.

```
int a,b,p[2],q,r,t,sum,i;
scanf("%d %d",&a,&b);
pipe(p);  q=fork( ); r=fork( );
if ((q==0)&&(r==0))
 { t=a-b;
   write(p[1],&t,4);
 }
if ((q==0)&&(r!=0))
 { t=a*b;
   write(p[1],&t,4);
 }
if ((q!=0)&&(r==0))
 { t=a/b;
   write(p[1],&t,4);
 }
if ((q!=0)&&(r!=0))
 { sum=0;
   for (i=1;i<=3;i++)
    { read(p[0],&t,4);
      sum=sum+t; }
   printf("%d",sum);
 }
```

In above program, the value of (a-b)+(a*b)+(a/b) is calculated by using 4 processes. One process each finds (a-b) (a*b) and (a/b). The final addition is done by another process.

25. Write program, which reads n and r and finds the value of 1+2+3+…+ n by following method: The program creates 2^r process (by using fork r times). The entire range (1..n) is divided into 2^r (approximately) equal parts. All processes (except one) write their result in a pipe. One process adds all results.

Multi valued Functions

```
int f(int x)
{   int t;
    if (x<10)  return x*x;
    if (x<=15) exit(0);
    if (x<=20) return x+5;
    if (x>=25) return 2*x;
    t=fork();
    if (t==0) return x+5;
    else return 2*x;
}
main()
{   int x,y;
    printf("Give number");
    scanf("%d",&x);
    printf("Solutions are: \n");
    y=f(x);
    printf("%d ",y);
}
```

The above program implements following function

f(x)=x*x if x<10
  =2*x if x>20
  =x+5 if (x<25) and (x>15)

Observe the output for x=7, 12, 17, 22, 27

Observe the difference when printf("Solutions are: "); is written after y=f(x)

Observe the output for
y=f(x)+3, y=f(x)+f(x) and y=2*f(x)

```
int insert (int x,int y)
{  int p;
   p=fork();
   if (p!=0)
     if ((x%2==0)&&(x>0))
         return(10*x+y);
     else exit(0);
   if (p==0)
     if (x>0)
         return

(insert(x/10,y)*10+(x%10));
     else exit(0);
}
main()
{  int x,y;
   printf("Give numbers x and y");
   scanf("%d %d",&x,&y);
   printf("%d ",insert(x,y));
}
```

The above program inserts a digit(Y) after some every even digit in number (X)

Logic is:

insert(X,Y)=10X+Y if (X>0) and (X%10=0)
  =insert(X/10,Y)*10+(X%10) if (X>0)

insert(1283654,7) will return 4 solutions. These are 12783654, 12873654, 12836754, and 12836547.

Write programs for following problems by defining one function only.

1. Define function g(x). g(x)=x+3 if (x<10) g(x)=2*x if x>3 g(x)=x^2 if x>6 and x<20
2. Define mult(x,y,z). All numbers between x and z are multiplied by y. mult(3,2,7) will return 5 solutions. These are 6, 8, 10, 12 and 14.
3. Define a function factor(x,y). It returns all factors of number x, which are bigger than y.
4. Define a function f to find product of neighboring digits. f(2365) will return 6, 18 and 30.
5. Define a function g to find product of neighboring even digits. g(23659438) will return 12, 24, 32.
6. Define a function even(x). It returns all even digits of a number. even(1432526) will return 4, 2, 2, 6.
7. Define a function location(x). It returns locations of all even digits. location(1432526) will return 1, 2, 4, and 6. Here location is taken from last.
8. Define a function h(x). It returns two digit numbers. The second digit of the answer is some even digit of x and first digit it its location. h(81432526) will return 88, 64, 42, 22, 16.

9. Define a function m(x). It returns two digit numbers. The second digit of the answer is maximum digit of x and first digit it its location. h(38842838) will return 78, 68, 38, 18. [It is a hard problem, when done using only one function. Even if same solution is outputted multiple times, it is acceptable].
10. Define replace(X,Y,Z). In the number X, any one Y will be replaced by X. replace (21343763,3,8) will return 21843763, 21348763, 21343768.

In following program assume that all digits of number are distinct.

11. Define a function c(x,y) to find all numbers formed from combinations of digit of a number (x) taken y at a time. c(2536,2) will return 6 numbers. These are 25, 23, 26, 53, 56 and 36. c(1428563,3) will return ${}^{7}C_{3}$ numbers. .
12. Define a function p(x) to find all numbers formed by permuting the digits of number x. p(2764) will return 24 numbers. Few solutions are 7264, 4627, 4762, … You may use more than one function.
13. Define a function subset(x). It will return all numbers formed from the digits of x (all digits different). subset(2764) will give 16 solutions. Few solutions are 764, 27, 4, 0, 64, …. Here in place of 764, 674 can also be returned, because both mean {7,6,4}. 0 is returned for empty set.

```
void f(char s[],int n)
{   int p;
    if (n<0) exit(0);
    p=fork();
    if (p==0)
       if (s[n-1]=='t')
return;
       else exit(0);
    else
    {  f(s,n-1);
       s[n-2]=s[n-1];
       return;
    }
}
```

```
main()
{   char s[40];int n;
    gets(s);
    n=strlen(s);
    f(s,n);
    s[n-1]='\0';
    puts(s);
}
```

The above program reads a string and deletes some ‘t’ from it. In input is ktretuyttg then outputs are kretuyttg, ktreuyttg, ktretuytg are ktretuytg.

14. Define function replace(char s[],int n). Some ‘t’ is replaced by ‘g’. For input ktretuyttg outputs are kgretuyttg, ktreguyttg, ktretuygtg, and ktretuytgg.
15. Define function insert(char s[],int n). ‘g’ is inserted after some ‘t’. For input ktretuyttg outputs are ktgretuyttg, ktretguyttg, ktretuytgtg, and ktretuyttgg.
16. Define function int del(char s[],int n). Few t’s are deleted. The size of final string is returned. For input ktretuy outputs are kretuy, ktreuy, kreuy, and ktretuy. If given string has ‘n’ t’s then 2^n solutions are returned. The function returns the size of final string. Hence in main program, we print it as s[n]=’\0’;put(s);.
17. Define function int locations(char s[],int n). All locations of ‘t’ are returned. For input ktretuyttg then outputs are 1, 4, 7 and 8.
18. Define a function int subset(char x[], int n, char y[]). String y includes few letters of x. The function returns the size of x. If x is krc there are 8 possible y’s. Assume all letters of string x are distinct.
19. Define a function void permutation(char x[], char y[]). String y is obtained by permuting the letters of string x. There are n! solutions. You may define an additional function. Assume all letters of string x are distinct.

Parallel Processing

Output some prime number

```
#include<stdio.h>
#include<fcntl.h>
int create(int n)
{  int p;
   if (n==1) return 0;
   else
   {   p=fork();
       if (p==0) return create(n/2);
       else return n/2+create(n-n/2);
   }
}
main()
{  int i,id,n,p[2],a[50],c;
   printf("Give n");scanf("%d",&n);
   printf("Give n numbers");
   for (i=0;i<n;i++) scanf("%d",&a[i]);
   pipe(p);
   id=create(n+1);
   if (id!=n)
   {   i=2;
       while (a[id]%i!=0) i++;
       if (a[id]==i)
         write(p[1],&i,4);
   }
   else
   {   close(p[1]);
       c=8;
       read(p[0],&c,4);
       if (c==8) printf("No prime number");
       else printf("%d",c);
   }

}
```

Read n numbers and find sum using m processors

```
main()
{  int i,id,n,m,p[2];float a[50],sum,c;
   printf("Give n and m");scanf("%d %d",&n,&m);
   printf("Give n numbers");
   for (i=0;i<n;i++) scanf("%f",&a[i]);
   pipe(p);
   id=create(m);
   sum=0;
   for (i=id*n/m;i<(id+1)*n/m;i++) sum=sum+a[i];
   printf("%f ",sum);
   if (id!=0) write(p[1],&sum,4);
   else
   {   for (i=0;i<m-1; i++)
       {   read(p[0],&c,4);
           sum=sum+c;
       }
       printf("\n%f",sum);
   }
}
```

Common Variables

Common variables are those variables, which can be accessed by more than one process. Following is the method of **semaphore**.

```
#include<stdio.h>
#include<sys/sem.h>
main()
{   int a,b,c,d,t,s,k,q,ans;
    int p;p=semget(0,1,IPC_CREAT|0666); ≡create semaphore p
    printf("Give a, b c, and d");
    scanf("%d %d %d %d",&a,&b,&c,&d);
    q=fork();
    if (q==0)
    {  t=a*b;
       semctl(p,0,SETVAL,t);            ≡ p=t
    }
    else
    {  s=c/d;
       sleep(1);
       k=semctl(p,0,GETVAL,0);          ≡ k=p
       ans=k+s;
       printf("%d",ans);
    }
}
```

In the above program, the child calculates the value of (a*b) and the parent of (c/d). Finally the parent performs the sum.

If we have already created a large number of semaphores, then additional semaphores are not created properly. To check, whether a semaphore p is properly created or not use printf("%d\n",p); If output is –1 then it is not properly created. We can delete semaphores by executing the following program.

```
#include<stdio.h>
#include<sys/sem.h>
main()
{   int i;
    for (i=0;i<=50000000;i++)
       semctl(i,0,IPC_RMID,0);
}
```

The following is the method of **shared memory**. Here the parent calculates the value of (a+b), (a-b) and (a/b) and the child of (c+d), (c-d) and (c/d). Finally the parent finds (a+b)*(c+d)+(a-b)*(c-d)+(a/b)*(c/d).

```
#include<stdio.h>
#include<sys/types.h>
#include<sys/ipc.h>
#include<sys/shm.h>
main()
{   int a,b,c,d,s,t,k,f,ans;
    int *p,*q,*r,*ptr,shmid;
    shmid=shmget((key_t)1,20,IPC_CREAT|0666);
    ptr=shmat(shmid,0,0);
                The above two lines create the shared memory segment of size 20.
                Every integer variable takes 4 bytes. Hence it can store 5 integers.
    p=ptr;q=ptr+1;r=ptr+2;
                p, q, and r are pointers in shared memory region.
    scanf("%d %d %d %d",&a,&b,&c,&d);
    f=fork();
    if (f==0)
    {  *p=a+b; *q=a-b; *r=a/b; }
    else
    {  s=c+d; t=c-d; k=c/d;
       sleep(1);
       ans=s*(*p)+t*(*q)+k*(*r);
       printf("%d",ans);
    }
}
```

1. Do above problems without using sleep(1). Here synchronization is achieved by using a common variable ‘flag’. In the beginning the value of flag is zero. When the child completes its work, it makes its value 1. The parent keeps on checking the value of flag by executing the loop while (flag==0);

2. Write a program, which creates two processes. First parent prints PPP, then child prints CCC, then parent prints QQQ and at last child prints DDD. Achieve synchronization by using common variables. Do not use signal.

3. Write a program, which creates two processes. The child prints the id of father. Do not use getppid( ).

4. What should be the missing lines in the following program, such that every process prints its id, its father’s id and its grandfather’s id.
   fork( );fork( );fork( ); sleep(1);<missing>;sleep(1);
   printf(“%d%d%d\n”,getpid( ),getppid( ),x);

5. Write program, which reads n number and finds the sum of these numbers by following manner. Two processes are created. Both processes keep on adding numbers, till the work is complete. The process which adds the last number prints the answer.

6. Write program using one semaphore (or 4 bytes of shared memory) only to compute the value of (((a*b)+c)/d)+e. The parent process can only add and child can only multiply and divide. Assume a, b, c, d and e are integer numbers (all are more than 2).

7. Write program in which parent reads a sequence of numbers. The child computes product of consecutive numbers and outputs it. e.g. input 2 5 7 3 6 …… output 10 35 21 18 …….

The semaphores created by a process can be used by another process.

```
main()
{   int p,q;
    p=semget(0,1,IPC_CREAT|0666);
    printf("Sem no%d\n",p);
    sleep(30);
    q=semctl(p,0,GETVAL,0);
    printf("%d",q+17);
}
```

Put the following and above programs in two different windows and different directory. Run both program. Input to the following program is the semaphore number created by the previous program.

```
#include<stdio.h>
#include<sys/sem.h>
main()
{   int t;
    printf("Give semiphore no");
    scanf("%d",&t);
    semctl(t,0,SETVAL,411);
}
```

After 30 seconds the first program produces output 411+17=428.

Signal

Signal	When it is send
SIGINT	When Control C (CTRL C i.e. ^C) is pressed.
SIGQUIT	When Control slash (CTRL \ i.e. ^\) is pressed.
SIGHUP	When CTRL D is pressed.
SIGFPE	When illegal mathematical operation such as division by zero is done.
SIGPIPE	When a process tries to write in a pipe with all read ends closed
SIGCLD	When a child process terminates it send to its parent
SIGALRM	When a process calls alarm(t) then it is send to the process after time 't'
SIGKILL	When command kill <process id> is given
SIGUSR1 SIGUSR2	These are open signals. Any process can send to any other process

Signal	What it does
	All signals except SIGCLD, SIGSTOP and SIGCONT terminate a process.
SIGSTOP	Stops a process execution.
SIGCONT	Resumes execution of a stopped process.
SIGKILL	Certain termination of a process.

System Call	Operation
Signal(signal name,SIG_IGN)	The corresponding signal is ignored. This does not work for SIGKILL.
Signal(signal name,funtion name)	On the arrival of the signal the corresponding function is called. It does not work for SIGKILL
Signal(signal name,SIG_DFL)	On the arrival of the signal the default action is taken.
Kill(process id, signal name)	Corresponding signal is send to the process whose identifier is given. Logically it is equivalent to Send(process id, signal name)

```
#include<stdio.h>
#include<signal.h>
void abc(){printf("AA\n");}
main()
{   signal(SIGINT,abc);
    for(;;);
}
```

The above program outputs AA whenever ^C is pressed. To terminate the above program use ^\ (CRTL \).

```
void abc(){printf("AA\n");}
void pqr(){printf("PP\n");}
main()
{long int n=10000,i;
 signal(SIGINT,abc);
 for (i=1;i<=100000*n;i++);
 printf(" XXX \n");
 signal(SIGINT,pqr);
 for (i=1;i<=100000*n;i++);
}
```

The program prints AA if any ^C is pressed before XXX is outputted. The program prints PP if any ^C pressed after that.

```
int count=0;
void abc()
{  count++;
   printf("%dth\n",count);
}
main()
{  signal(SIGINT,abc);
   for(;;);
}
```

The above program prints how many ^C's we have pressed.

```
void abc()
{   printf("AA\n");
    signal(SIGINT,SIG_DFL);
}
main()
{   signal(SIGINT,abc);
    for(;;);
}
```

On first ^C the above program outputs AA. On second ^C the program terminates.

1. Write a program, which ignores 5 ^C's. On 6th ^C the program terminates.

```
void abc(){printf("AA\n");}
void pqr()
{   printf("PP\n");
    signal(SIGINT,abc);
}
main()
{   signal(SIGINT,pqr);
    for(;;);
}
```

On first ^C the above program prints PP. On successive ^C's AA is printed.

.

2. Write a program, which first time prints PP, second time AA and the third time terminates.

3. Write a program, which first time prints PP, second time AA, third time PP, fourth time AA, and so on.

4. When ^C is pressed, the program asks a question, whether you want to terminate. If user types 0 the program terminates. If user types 1 then the program does not terminate.

5. Write program, which prints X when ^C is pressed first time. The program prints XX when ^C is pressed second time. On third ^C the program prints XXX and so on.

Time

```
void abc(){printf("AA\n");}
main()
{   alarm(3);
    signal(SIGALRM,abc);
    sleep(20);
}
```

Three seconds after the execution begins the output AA is produced, because a signal SIGALRM is sent to itself.

6. Write program, which prints XY. The Y is printed 3 seconds after X is printed. Do not use sleep(3). You may use sleep(20).

```
void abc(){printf("AA\n");}
main()
{   int n=12;
    alarm(3);
    signal(SIGALRM,abc);
    printf("Give n\n");
    scanf("%d",&n);
    printf("n=%d",n);
}
```

The above program behaves differently on UNIX and LINUX. On UNIX if value of N is not given within 3 seconds the program takes the value of N as 12. On LINUX the program keeps on waiting for the value of N.

7. Write program, which displays the message after every 3 seconds to give the value of N.

8. Write program, which prints AAA…. for 1 second.

9. Write program, which prints 1 2 3 4 5 …. The next number is outputted after 1 sec. Do not use sleep(1).

```
void abc()
{printf("%dX\n",getpid());}
void pqr()
{printf("%dY\n",getpid());}

main()
{  int q;
   printf("%d\n",getpid());
   q=fork();
   if (q==0)
       signal(SIGINT,abc);
   else
       signal(SIGINT,pqr);
   for(;;);
}
```

When ^C is pressed the signal SIGINT goes to both processes. The parent prints Y and the child prints X. However, if we run the program in background then SIGINT does not go to either parent or child.

10. Write program, when we press ^C parent always prints X. Child prints Y and Z alternatively

```
main()
{  int q;
   signal(SIGHUP,SIG_IGN);
   q=fork();
   printf("%d\n",getpid());
   if (q==0)
       sleep(100);
}
```

When we execute the above program, the parent is over immediately. The child process continues running in background. Press ^D (signal SIGHUP is send to child, but it is ignored). Even after logout the child continues running. It can be verified by giving command "ps –e". (This command shows the id's of all running processes). If the child process had not ignored SIGHUP, it would have been killed. The above does not hold on LINUX.

```
int count=0;
void abc() { count=1;}
main()
{int q;
 signal(SIGUSR1,abc);
 q=fork();
 if (q==0)
 { while (count==0);
   printf("%dABC\n",
                 getpid());
   count=0;
   kill(getppid(),SIGUSR1);
   while (count==0);
   printf("%dTTT\n",
                 getpid());
 }
 else
 { printf("%dPQR\n",
                 getpid());
   kill(q,SIGUSR1);
   while (count==0);
   printf("%dLMN\n",
                 getpid());
   kill(q,SIGUSR1);
 }
}
```

In above program the parent prints PQR, then child prints ABC, then parent prints LMN, then child prints TTT. Here synchronization is achieved by sending signals. A process keeps on waiting (busy waiting) by using

```
while (count==0);
```

As soon as signal arrives, the value of count changes, hence wait is no more performed.

```
main()
{   int q;
    q=fork();
    if (q!=0){
        sleep(5);
        kill(q,SIGSTOP);
        sleep(4);
        kill(q,SIGCONT);
        sleep(15);
        kill(q,SIGKILL);
    }
    else
        for (;;)
        {  printf("X\n");
           sleep(1);}
}
```

In above program, the child prints one X in one second. This goes on for 5 seconds. For 4 seconds the child does not print any thing. Again for 15 seconds the child prints X's.

1. Write a program, which creates an infinite loop by for(;;). ^C and ^\ is given to the program. The program terminates when three consecutive ^C's are pressed. If (^\) is pressed in between then the program does not terminates. e.g. ^C\^\^C^\^C^C^C terminates. To terminate this program press ^Z.

2. Do above program, without using exit(0) or kill(gertpid( ), signal)

3. Write a program, which creates an infinite loop by for(;;). A sequence of ^C and ^\ is given. After every ^C (except first two ^C's) a number is outputted. It is the number of ^\'s between last and third last ^C. e.g. input ^\^C^\^\^C^\^C^C^\^\^\^C^\^\^C the output is 3 1 3 5

4. Write a program, which creates an infinite loop by for(;;). The program terminates when ^C^C^\^C^\ is pressed. e.g. ^C^C^\^\^C^C^C^\^C^C^\^C^\ terminates.

5. Write a program, which creates an infinite loop by for(;;). ^C and ^\ is given to the program. ^C means 0 and ^\ means 1. The program executes for 10 seconds. Then outputs the value of number. e.g. input ^C^\^\^\^C^C^\ output is 57 (0111001).

6. Write program, which finds the sum of the following series. (Use double to store answer). As many terms are added as can be added within one second.

$$\frac{1}{10000}+\frac{1}{10001}+\frac{1}{10002}+\frac{1}{10003}+\frac{1}{10004}+\ldots\ldots$$

7. Write program, which reads a float x. Program computes the sum of the above series, till it exceeds x. The program outputs the time taken.

8. Write program, which creates an infinite loop by for(;;). When ^C (except first ^C) is pressed an integer is outputted. It is time lapsed between two consecutive ^C 's. e.g. if ^C are pressed at time t=10, 22, 29,36, 40, 48, 49, 57, 61, 65, 77 then the output will be 12, 7, 7, 4, 8, 1, 8, 4, 4, 12.

9. Write program, which outputs an X for every ^C after 10 second. If ^C's are pressed at time t=4, 17, 29, 36, 48, 52, 53, 68 then X will be outputted at t=14, 27, 39, 46, 58, 62, 63, 78.

10. Write program which terminates when 20 ^C's are pressed. The program outputs the time difference between 4th and 20th ^C.

11. Write program, which creates an infinite loop by for(;;). The program terminates when two ^C's are pressed within 5 seconds. For above input the program will terminate at t=40.

12. Write program, which terminates if three ^C's are pressed within 10 seconds. For above case the program will terminate at t=49.

13. Write program, which terminates if three ^C's are pressed such that difference between consecutive ^C's is less than 5 seconds. For above case termination will be at t=65.

14. Write program, which prints XX as soon as command ./a.out is given. After 20 seconds YY is printed. During these 20 seconds user can give at most 2 ^C's. (If ^\ or more than 2^C's are given then the program terminates immediately). Do not use SIG_IGN.

15. What given program does

```
int i,j=1,k=1,a[80];
void g(){a[k++]=i;}
main()
{  signal(SIGINT,g);
   for (i=1;;i++)
   if (a[j]*2==i)
       printf("%d\n",j++);
}
```

16. Write program which prints 1,2,3,4,5 … Every number is printed after approximately 0.5 second. (If at t=0 1 is printed then at time t=60 (approximately) 121 will be printed.)

17. Write program, which creates two processes parent and child. Both print their id's alternatively. i.e. $n_p,n_c,n_p,n_c,n_p,n_c,n_p,n_c,n_p,n_c$, . . .Numbers should be printed in fast speed (in one seconds at least 10 numbers).

18. Let A, B, and C be distinct numbers. The program outputs ABCABCABC. . . at fast speed. There should be no printf statement [except printf("%d",getpid( ));] in the program.

19. Write program, which terminates when two ^C's are pressed. The program outputs the time difference between them. Do not use alarm.

20. Write a program, whose inputs are two ^C's. The program outputs the time difference between them. [Model answer: 1.2 second] [Hint: Make a while loop. The program enters the loop when first ^C is pressed and exits the loop when second ^C is pressed. Find how many times the loop is executed.]

21. Write program, which terminates if 10 ^C's are given. Let time difference between consecutive ^C's be t_1, t_2, t_3, … t_9. The program outputs minimum of t_i's. [Model answer: 7.3 seconds].

22. Write program, which finds the value of $t_1+ t_2+ t_3+ t_4+ t_5$ for above case.

File Low level System Calls

```
#include<stdio.h>
#include<fcntl.h>
main()
{ int fd,i;char c;
  fd=open("xyz",O_RDONLY);
  i=lseek(fd,6,SEEK_SET);
  read(fd,&c,1);
  printf("%d%c",i,c);
}
```

The above program opens a file named xyz and prints its sixth character. Let the contents of file xyz be abcdefghi then output is 6g. The lseek system call carries the file pointer at sixth location.

0	1	2	3	4	5	6	7	8	9		
a	b	c	d	e	f	g	h	i	\n		

Few editors (e.g. emacs) do not put '\n' at the end.

The file pointer can be moved by using lseek system call. SEEK_SET means relative to the beginning to the file. SEEK_CUR means relative to the current position and SEEK_END means from end of the file. In place of SEEK_SET, SEEK_CUR and SEEK_END 0,1 and 2 respectively can also be used.

```
int fd,i,j,k;char c;
fd=open("xyz",O_RDONLY);
i=lseek(fd,6,SEEK_SET);
j=lseek(fd,-2,SEEK_CUR);
read(fd,&c,1);
printf("%d%c\n",j,c);
k=lseek(fd,-2,SEEK_END);
read(fd,&c,1);
printf("%d%c\n",k,c);
```

In the above program the first lseek carries the file pointer to 6th location. The second lseek carries to $(6-2)=4^{th}$ location. Hence the output is 4e. There are 10 (9 if file is made using emacs) characters in the file hence the last lseek will carry the file pointer to $(10-2)=8^{th}$ position. Hence the output will be 8i (7h in case of emacs).

```
int fd,i;char fn[20];
printf("Give filename");
gets(fn);
fd=open(fn,O_WRONLY);
write(fd,"BCD",3);
lseek(fd,2,SEEK_CUR);
write(fd,"KLC",2);
```

The above program writes into a file. If a file is given whose contents are abcdefghi then after the execution of the above program the contents will be BCDdeKLhi

The third argument in write system call says how many characters should be written. In write(fd,"KLC",2) though three characters are given, yet only first two will be written. write(fd,a,4) will write a[0], a[1], a[2], and a[3].

```
fd=open("exp",O_CREAT|O_WRONLY);
write(fd,"BC",2);
```

The above program will create a file (if it does not exist) named "exp" and writes BC in it.

```
int fd,i;char a[10],fn[20];
fd=open("xyz",O_RDWR);
lseek(fd,-4,SEEK_END);
read(fd,a,3);
lseek(fd,0,SEEK_SET);
write(fd,a,3);
```

The above program copies last three characters of file "xyz" in the beginning. If initial contents of file "xyz" are abcdefgh then after execution of the above program the contents will be fghdefgh.

```
int fd1,fd2;char c;
fd1=open("xyz",O_RDONLY);
fd2=open("pqr",O_WRONLY);
lseek(fd1,6,SEEK_SET);
read(fd1,&c,1);
write(fd2,&c,1);
```

The above program reads 6th character from file xyz and puts it in the beginning of file pqr. If in above we replace file name pqr by xyz then in the same file the above change will be made. Now there are two open instances of same file xyz.

```
int fd,i;char a[10];
a[2]=78;
fd=open("exp",O_RDONLY);
i=read(fd,a,5);
```

printf("%d %d %d %d",i,a[0],a[1],a[2]);

The read system call returns the number of the characters actually read. In a file named "exp" if we put three characters abc, the output will be 4 97 98 99. If the file is made in emacs then the output will be 3 97 98 99.

If in file exp we write only two characters BC the output will be 3 66 67 10. When we make a file (in vi editor) an extra character, whose ASCII code is 10, is added. If the file is made in emacs then the output will be 2 66 67 78. (The value of a[2] remains unchanged)

The read system call returns –1 if read can not be performed. For example in the following we have opened a file for writing and we are trying to read.

```
fd=open("exp",O_WRONLY);
i=read(fd,a,5);
```

If the file is opened for reading and we perform read, but there are no characters then 0 is returned.

s=read(fd,a,100000) can be used to find size of a file (if the file has less than 100000 characters).

Representation of Integer/Float

```
int fd,i;float j;
fd=open("pqr",O_RDONLY);
read(fd,&i,4);
read(fd,&j,4);
printf("%d %f",i,j);
printf("%d",256*256*256*65+
   256*256*70+256*102+102);
```

Let contents of file pqr be ffFAffFA.

From above example the representation (in UNIX) of an integer and float variable at low level can be understood.

An integer (or float) is stored in four bytes. Let the contents of these bytes be A, B, C and D (here D is most significant byte). Then value of the integer is $D256^3+C256^2+B256+A$. It can be verified by observing that the first and the last outputs are same.

The representation of float as follows.

D	sg	e	e	e	e	e	e	e
C	e	m	m	m	m	m	m	m
B	m	m	m	m	m	m	m	m
A	m	m	m	m	m	m	m	m

sg is sign bit. It is 0 if the number is a positive number and 1 if the number is a negative number.

There are 8 bits for storing exponent (in access 128 representation) and 23 bits for storing mantissa. It is assumed that point is put after two bits and most significant bit (MSB) is 1. Since MSB is 1, hence it is removed from representation. The representation of 12.4 is as follows.

12.4=1100.011001100110…..

=1<u>1.00011001100110</u>….x 2^2.

Hence the mantissa is 100011001100…..

Sign bit is 0 since the number is positive.

Exponent (in access 128) is 2+128=130

130=10000010

	0	**1**	**0**	**0**	**0**	**0**	**0**	**1**	65A
	0	1	0	0	0	1	1	0	70 F
	0	1	1	0	0	1	1	0	102f
	0	1	1	0	0	1	1	0	102f

1. Write program, which finds the location of first a. Let a file has erapouseraty then output is 3.
2. Write program to find 4th character after first ‘a’. In above case ‘s’.
3. Write program to find character previous to first ‘a’. In above case ‘r’. Assume that first character is not ‘a’.
4. Write a program to find the size of a file. Do not use SEEK_END. [Hint: Read every character of file till the system call i=read(fd,&c,1) returns 0.]
5. Write program to find product of difference of characters around ‘z’. Let file has pczatxzwazdzc. The output will be (c-a)*(x-w)*(a-d)*(d-c)=2*1*(-3)*1=-6. Assume that first and last characters of the file are not z and two consecutive z’s do not occur.
6. Write program to print the entire file. It is like cat command.
7. Write program to replace 4th character of a file by ‘A’. Hence the above file after execution of this program will look as eraAouseraty
8. Write program to find the location of last ‘a’. In above case 10.
9. Write program to replace the first ‘a’ by ‘A’. Hence above file will look as erApouseraty
10. Write program to replace all a’s by A’s.
11. Write program, which reads two file names. In first file the contents of the second file are copied. It is like (cp) copy command.
12. Write program, which reads two file names. In first file the contents of the second file are put at the end. Here we are joining both files.
13. Write program to print a file in reverse order (Do not use array).
14. Write program, which reads two file names. In first file the contents of the second file are put in reverse order (Do not use array).
15. Write program to reverse a file (Do not use array). Hence after the execution of this program the above file look as ytaresuopare.
16. Write program, which reads a file name and a string. The string is put at the end of the file.
17. Write program, which puts a character ‘x’ in the beginning of a file.
18. Write program, which puts a ‘x’ after every ‘a’. After the execution of this program the above file will look as eraxpouseraxty.
19. Write program, which deletes the first character of a file.
20. Write program, which exchanges adjacent characters. After the execution of this program the above file will look as repauoesaryt. Assume that file has even number of characters.
21. Write program, which inserts a character ‘x’ in between adjacent characters. After the execution of this program the above file will look as exrxaxpxoxuxsxexrxaxtxy.
22. Write program, which reads two file names. Assume that characters in those files are in increasing order of ASCII code. The program prints all characters in sorted order. Let the contents of files are acfxyz and bdeghjkl then output is abcdefghjklxyz.
23. Assume that file has numbers. Print the last number.
24. Assume that the file has words. Write program to find the kth word. Let the file has ram shyam hari gopal and k is 3 then output is hari..
25. Write program to replace every word by its reverse. After the execution of this program the above file will look as mar mayhs irah lapog.

File Pointer

It is the position, at which character will be accessed. File pointers is not associated with a file but with an instance of opening file. A file may have more than one open instances hence different file pointers.

```
int fd1,fd2; char c,d;
fd1=open("exp",O_RDONLY);
fd2=open("exp",O_RDONLY);
lseek(fd1,2,SEEK_CUR);
lseek(fd2,5,SEEK_CUR);
read(fd1,&c,1);
read(fd2,&d,1);
printf("%c%c\n",c,d);
```

Let file "exp" contains abcdefghijkl. The output of above program is *cf*. If file pointers were associated with file then the output had been *hi*. If we replace fd2 by fd1 in lseek and read then we get this output.

File tables (FDT and SFT)

Information (file pointer etc) about open instance of a file is not stored in the process (which opened it). It is stored in System File Table (SFT). It is unique in the system. Every process has its file description table (FDT), which keeps a pointer to corresponding open instance of the file in (SFT). If a process has opened a file and the process forks then the child process gets a copy of FDT. Hence it can access the same open instance of file.

```
int fd,p; char c;
fd=open("exp",O_RDONLY);
p=fork();
if (p==0)
    lseek(fd,4,SEEK_SET);
else{ sleep(2);
      read(fd,&c,1);
      printf("%c\n",c);
    }
```

In above program the parent opens a file and child positions the file pointer. Now parent uses the same file pointer.

Let the contents of file exp are abcdefgh. The output is *e*. If file pointer were stored in FDT then output had been *a*. This output is produced if fork and open are exchanged. It is because there are two open instances of same file

File descriptor (FD)

When a process opens a file a descriptor is associated with the open instance. It is an entry in FDT. It stores a pointed to an entry (corresponding to open instance of file) in SFT. When an open instance of file is closed, the corresponding file descriptor is freed. If in future another file opening is made then the same file descriptor can hold pointer corresponding to that opening.

```
int fd1,fd2,fd3,fd4,fd5;
char c,d;
fd1=open("abc",O_RDONLY);
fd2=open("pqr",O_RDONLY);
fd3=open("xyz",O_RDONLY);
printf("%d %d %d\n",
              fd1,fd2,fd3);
read(fd1+2,&c,1);
read(4,&d,1);
printf("%c %c\n",c,d);
close(fd2);
fd4=open("lmn",O_RDONLY);
fd5=open("stu",O_RDONLY);
printf("%d %d\n",fd4,fd5);
```

Let the file abc contains abc, file pqr contains pqr, xyz has xyz, lmn has lmn, and stu has stu.

The output of above program is 3 4 5 x p 4 6. It is because file descriptors are numbers starting from 3. fd1+2 is (3+2=5), which corresponds to the xyz. Similarly 4 corresponds to pqr.

Since file descriptor 4 becomes free after closing hence it can store FD corresponding to lmn

Closing of file

When a process closes a file the corresponding instance of file-open may still remain live. This happens if it is referenced by some other process. Only from that process the value of file descriptor is removed (hence that process looses its right to access that file)

```
int fd,p;char c='u';
fd=open("abc",O_RDONLY);
p=fork();
if (p==0)
{  close(fd);
   read(fd,&c,1);
   printf("child=%c  ",c);
}
else
{  sleep(1);
   read(fd,&c,1);
   printf("Parent=%c\n",c);
}
```

Let the contents of file abc be abc. The output of above program is Child=u Parnet=a. It is because the child can not access the file (hence the value of c is unchanged), but parent can access the file. If we remove close the output is Child=a Parent=b.

Count of open instances

Every entry in system file table keeps the count of the number of processes from where that file-open instance is referenced. When a process closes a file that count is decremented. When a process forks the count is incremented. Only when the count becomes zero the entry from system file table is deleted.

```
int fd1,fd2,fd3,p;char c;
fd1=open("abc",O_RDONLY);
fd2=open("pqr",O_WRONLY);
p=fork();
if (p==0)  close(fd1);
fd3=open("xyz",O_RDWR);
read(3,&c,1);
printf("%d %d %c \n",
                  p,fd3,c);
```

Let the file abc contains abc, file pqr contains pqr, xyz has xyz. The output of above program is A 5 a and 0 3 x, where A is child id. The child closes abc hence FD=3 is free for him. Hence child opens file xyz in FD=3 and parent in FD=5. For parent FD=3 means abc and for child it means xyz. Various tables looks as follows :

System File Table

Sl. No	File Name	Access Right	Count	File Pointer
. . .	. . .	. . .	. . .	. . .
42	abc	RD	1	1
43	pqr	WR	2	0
44	xyz	RW	1	0
45	xyz	RW	1	1

File Description Table for Parent

Serial no	Pointer
3	42
4	43
5	44

File Description Table for Child

Serial no	Pointer
3	45
4	43

Lock

```
#include<stdio.h>
#include<fcntl.h>
main()
{  int p,fd,i,j;
   fd=open("xyz",O_WRONLY|
                   O_CREAT);
   p=fork();
   if (p==0)
   { sleep(2);
     printf("PPP\n");
     lockf(fd,F_LOCK,0);
     printf("QQQ\n");
     for (i=1;i<=5;i++)
     {  write(fd,"B",1);
        sleep(1);
     }
   }
   else
   { lockf(fd,F_LOCK,0);
     for (i=1;i<=5;i++)
     {  write(fd,"A",1);
        sleep(1);
     }
  }
}
```

The output of above program is
(wait 2) PPP (wait 3) QQQ
In the file XYZ 5A's and 5B's are printed as AAAAABBBBB. It is because when the child tries to lock, it finds that the file is already locked. Hence it waits. When the parent process becomes over, it removes its own lock. Now the child process locks and resumes execution.
When we remove the lock in the parent process the output is
(wait 2) PPPQQQ
In the file XYZ 5A's and 5B's are printed in mixed manner.
We get the same effect when we remove the lock from child. Here a process is not stopped from writing into a file, which is locked by another process. It is because it is an advisory lock. The lock does not stop any one from reading or writing. The lock only stops a process (which is trying to lock a file) if someone has already locked it. A process, which is not executing the lock system call can never be stopped.

```
int p,fd,i,j;char c;
p=fork();
if (p==0)
{ sleep(2);
  fd=open("xyz",O_RDWR);
  lseek(fd,10,0);
  lockf(fd,F_LOCK,3);
  lseek(fd,7,0);
  for (i=1;i<=5;i++)
  {   read(fd,&c,1);
      printf("%c\n",c);
      sleep(1);
  }
}
else
{ fd=open("xyz",O_RDWR);
  lockf(fd,F_LOCK,10);
  for (i=1;i<=5;i++)
  {   read(fd,&c,1);
      printf("%c\n",c);
      sleep(1);
  }
}
```

Assume the file XYZ has
0123456789AB
The output of the above program is 01234 and 789AB. The characters of both sequences are mixed. The parent has locked character number 0-9. The child tries to lock characters 10-12. There is no conflict.
If 10 is replaced by 11 in lockf(fd,F_LOCK,10) the output is 01234789AB. The child stops because it is trying to lock characters 10-12, and the 10th character is already locked. When the parent is over this lock is removed.
It should be observed that lock applies to a file (not to an open instance).
It should also be observed that lock is not applicable if a process has opened file in Read-only mode.

Pipe

```
#include<stdio.h>
#include<fcntl.h>
main()
{
  int p[2],i;char c;
  p[0]=open("exp",O_RDONLY);
  p[1]=open("exp",O_WRONLY);
  printf("%d%d",p[0],p[1]);
  write(p[1],"PQR",3);
  close(p[1]);
  for (i=1;i<=4;i++)
  {  c='x';
     read(p[0],&c,1);
     printf("%c",c);
     fflush(stdout);
  }
}
```

When we make an empty file "exp" and execute this program, the file descriptors are outputted as 3 and 4 and characters are outputted as PQRx. When attempt is made to read the 4th character, the read system call fails because there are only 3 characters in the file.

When we replace

```
p[0]=open("exp",O_RDONLY);
p[1]=open("exp",O_WRONLY);
```

by `pipe(p)`

we get the same output. However the difference can be understood by following.

When we remove close(p[1]); the program does not halt after printing PQR. Since write end of the pipe is still open hence it is expected that some characters can be put in the pipe. Hence indefinite wait takes place.

If we put following system call after pipe(p)then wait is removed. It is because a flag (F_SETFL) is set to no delay (O_NDELAY).

```
fcntl(p[0],F_SETFL,O_NDELAY);
```

Another difference between pipe and file is that lseek does not work in pipe.

```
pipe(p);
write(p[1],"ABC",3);
lseek(p[0],2,0);
read(p[0],&c,1);
printf("%c",&c);
```

When we try to write in a pipe after closing (all) read end(s) the program terminates immediately and a message "broken pipe" is displayed and abc is not printed.

```
pipe(p);
close(p[0]);
write(p[1],"KLT",3);
printf("abc\n");
```

In this case a signal SIGPIPE is send. In the following program we have caught the signal. Hence TTabc is printed and the program does not terminate with message "broken pipe".

```
void pqr(){printf("TT");}
main()
{ int p[2];
  signal(SIGPIPE,pqr);
  pipe(p);
  close(p[0]);
  write(p[1],"KLT",3);
  printf("abc\n");
}
```

```
#include<stdio.h>
#include<fcntl.h>
main()
{
  int p[2],i,q;char c;
  pipe(p);
  q=fork();
  if (q==0){
    sleep(4);
    write(p[1],"ABC",3);
    sleep(1);
    close(p[1]);
    sleep(20);
    printf("child over ");
  }
  else{
    sleep(1);
    fcntl(p[0],F_SETFL,
                O_NDELAY);
    write(p[1],"PQR",3);
    close(p[1]);
    for (i=1;i<=7;i++){
      c='x';
      read(p[0],&c,1);
      printf("%c",c);
      fflush(stdout);
    }
  }
}
```

The output of the above program is PQRxxxx. When no delay is removed output is PQRABCx. Here ABC is displayed after 3 seconds and x is displayed 1 second after that. The parent does not wait to read the 7th character, when all write ends are closed.

When we remove close(p[1]) from the parent, the parent process waits indefinitely, because there is a hope that something can be put in the pipe.

When we remove close(p[1]) from the child x is printed after 20 seconds. It is because when a process (in this case child) terminates close(p[1]) is executed even when it is not explicitly written.

```
#include<stdio.h>
main()
{  int p[2],i,n,q;
   pipe(p);
   q=fork();
   if (q==0){
    close(p[0]);
    sleep(2);
    write(p[1],"AB",2);
    printf("X");
   }
   else {
     sleep(1);
     close(p[0]);
     printf("Y");
     sleep(3);
   }
}
```

In above program the write system call fails. It is because when the child tries to write in the pipe it finds that no read end is open hence it terminates and X is not printed (only Y is printed).

When we remove any one of the close(p[0]) the child finds that some read end is open hence write system call works. Hence YX is printed.

If do not remove any of the close(p[0]), but remove sleep(2) we find that write is successful. It is because at the time of execution of write system call some read end was open.

If we remove close(p[0]) from the parent the write does not fails. However, when we remove sleep(3) also, write fails (hence only Y is printed). It is because when a process halts it closes its read and write ends.

Buffer

When a printf statement is executed, it is not necessary that output be immediately shown on the monitor. The output is put into a buffer. The contents of the buffer are displayed later. It may give rise to following anomalies:

```
main( )
{    printf("AAA");
     sleep(1);
     printf("BBB");
}
```

It is expected that AAA will be printed immediately after execute command is given. BBB will be printed 1 second after AAA is printed. However we observe that AAA is displayed 1 second after the command to execute is given. BBB is displayed immediately after (with no time delay) AAA is displayed. It is because **printf("AAA")** instruction puts AAA in the buffer. Buffer is normally displayed when

1. Buffer is full (Normal size 1024 characters).
2. New line character is printed.
3. By system call fflush(stdout);
4. By instruction Scanf.
5. End of the execution.

Hence if we put printf("\n"); or fflush(stdout); or scanf("%d",&i); immediately after printf("AAA"); we shall get the expected output.

```
main( )
{    int i;
     printf("XXX");
     for (i=1;i<2;);
}
```

In this program output XXX is expected. However nothing is printed. It is because the program falls into infinite loop and XXX is never displayed (it remains in the buffer).

```
main( )
{    int i;
     for (i=1;i<=348;i++)
     printf("AAA");
     sleep(3);
     printf("BBB"):
}
```

In this program 1024 A's are immediately printed (because buffer is full). Remaining 20 A's are printed after 3 seconds along with BBB.

```
main( )
{    printf("A");
     fork( );
     printf("B");
}
```

In this program expected output is ABB. However ABAB is printed. It is because buffer is also copied when duplicated copy of the process is made by fork() system call.

File High level System Calls

```
FILE *fs; int i=36426;
fs=fopen("exp","w");
fprintf(fs,"ppp%dkkk",i);
fseek(fs,-1,SEEK_END);
fprintf(fs,"ttt");
fseek(fs,-2,SEEK_CUR);
write(3,"ABC",3);
```

In high level file opening, an stream (in above case fs) is also (another is file descriptor) associated corresponding to every open instance. It enables us to print integers, floats etc. The file pointer can be moved using fseek. Since it is the first file open hence file descriptor will be 3. Hence the file can also be modified using low level system calls like write.

```
fd=open("exp",O_WRONLY);
fs=fdopen(fd,"w");
write(fd,"AAAA",4);
fprintf(fs,"ppp");
```

Even if a file is originally opened using low level system call, a stream can be obtained. Using it high level file manipulation system calls can be used.

```
pipe(p);
fs=fdopen(p[1],"w");
fprintf(fs,"p"); fflush(fs);
write(p[1],"T",1);
read(p[0],&c,1); printf("%c\n",c);
read(p[0],&c,1); printf("%c\n",c);
```

An stream can be associated with pipe also

```
fs1=fdopen(3,"w");
fs2=fdopen(3,"w");
fprintf(fs1,"X");
fprintf(fs2,"Y");
fclose(fs2);
write(3,"T",1);
```

When more than one stream is associated with same file descriptor (FD) and one of the streams is closed then the other is also closed. The above program writes only "Y" in the file. This shows that fclose removes file descriptor also.

```
fs1=fopen("exp","w");
fprintf(fs1,"A");
fs2=freopen("stu","w",fs1);
fprintf(fs1,"B");
fprintf(fs2,"C");
```

The freopen system call opens the file in the file descriptor corresponding to fs1. Now there are two streams (fs1 and fs2). Hence in file exp A is written and in the file stu BC is written.

```
FILE *fs1,*fs2,*fs3;
fs1=fopen("exp","w");
fs2=fopen("pqr","w");
fclose(fs1);
fs3=freopen("abc","w",fs2);
write(4,"KKK",3);
```

File descriptors (FD) associated with files exp and pqr are 3 and 4 respectively. Since file exp is closed hence FD=3 becomes free. However freopen("abc","w",fs2) opens file abc in the same FD, which was associated with fs2(pqr). Hence FD=4 is associated with abc. However if we replace freopen by fopen then FD=3 will be associated with abc. It can be verified by looking contents of various files.

```
fs=fopen("abc","w");
fs=fopen("pqr","w");
write(3,"TT",2);
```

If another file is opened in same stream then corresponding file descriptor is not closed. The above program writes TT in file “abc”. However if second fopen is replaced by freopen("pqr","w",fs) then TT is written in pqr.

Standard Input and output

```
write(1,"PQR",3);
fprintf(stdout,"TTT");
```

File descriptor=1 and stream stdout are associated with standard output. Hence statements printf("x"), write(1,"x",1) and fprintf(stdout,"x") are equivalent.

```
close(0);
fd=open("exp",O_RDONLY);
scanf("%c",&c);
printf("%c",c);
```

Let the contents of file exp be pqrst. File descriptor=0 and stream stdin are associated with standard input. close(0) frees FD=0 hence new file is opened in it. Since scanf("%c",&c) is translated (via fscanf, see next example) as read(0,&c,1). Hence the first character of file exp is read. Hence c='p'.

```
stdout=fopen("abc","w");
printf("RTY");
fprintf(stdout,"KK");
write(3,"SS",2);
write(1,"LP",2);
```

Since printf("RTY"); is translated as fprintf(stdout,"RTY") which is translated as write(3,"RTY",3) hence in the file abc RTY is written (KK and SS are also written). LP is outputted on monitor. If we replace fopen("abc","w") by freopen("abc",w,stdout) then LP also goes to file "abc" (SS goes no where).

In the following example stream fs and stdout are both associated with FD=1.

```
fs=freopen("abc","w",stdout);
printf("TTT");
```

Hence TTT is written in file "abc". Observe the difference when freopen is replaced by fopen.

Buffer

```
fs=fopen("exp","w");
fprintf(fs,"pqr");
fflush(fs);
write(3,"Y",1);
```

A file exp is opened. The file stream is fs and corresponding file descriptor is 3. Hence in the file pqrY is written. However if we remove fflush then pqr is written in buffer. Y is immediately written. Buffer is written when program is over. Hence the file looks Ypqr. Here buffer size is 4096. However if buffer size were 2 then file will look as pqYr.

```
for (i=1;i<=4100;i++)  fprintf(fs,"A");
write(3,"B",1);
```

Last few characters of the file are .....ABAAAA. It is because the buffer size is 4096. On some systems the buffer size is 1024.

```
fs=fopen("exp","w");
write(3,"PQRSTUVW",8);
fprintf(fs,"ABC");
lseek(3,2,SEEK_SET);
fflush(fs);
write(3,"Z",1);
```

Since lseek makes file pointer (FP) as 2, hence fflush writes the buffer at that point. Hence file looks as PQABCZVW.

```
FILE *fs1,*fs2; int fd;
fd=open("exp",O_WRONLY);
fs1=fdopen(fd,"w");
fs2=fdopen(fd,"w");
fprintf(fs1,"P");
fprintf(fs2,"Q");
fprintf(fs1,"R");
```

More than one stream can be associate with same open instance of a file. Both have different buffer. The above program will write PRQ or QPR in the file (depending upon which buffer is written first).

```
fs1=fopen("pqr","w");
fs2=freopen("abc","w",fs1);
fprintf(fs1,"X");
fprintf(fs2,"Y");
write(3,"Z",1);
fflush(fs2);
```

The output of the above program is ZXY. This shows that both stream fs1 and fs2 have same buffer.

```
write(3,"AAA",3);
fprintf(fs,"ppp");
lseek(3,-2,SEEK_CUR);
write(3,"Y",1);
```

Observe the result by replacing lseek by fseek(fs,-2,SEEK_CUR); It forces the buffer to be written in file. Because positioning of a file pointer can not be done in buffer.

```
fprintf(fs,"ppp");
write(3,"YYYY",4);
j=lseek(3,0,SEEK_CUR);
i=ftell(fs);
k=lseek(3,0,SEEK_CUR);
printf("%d %d %d\n",i,j,k);
```

ftell finds the current position of file pointer. Difference in values of j and k shows that buffer is flushed because of ftell. If we remove it then j and k are same (4).

```
FILE *fs;char c,d;
fs=fopen("abc","r+");
fprintf(fs,"HJ");
fscanf(fs,"%c",&c);
fprintf(fs,"PQ");
fscanf(fs,"%c",&d);
printf("%c%c\n",c,d);
lseek(3,6,0);
```

Let the initial contents of the file be 012345678. The final contents of the file are HJ2PQ5678 and output is 25. It is because read and write buffer are flushed when fscanf is performed.

```
fs=fopen("pqr","r+");
fscanf(fs,"%c%c",&c,&d);
lseek(3,-1,1);
fprintf(fs,"TTTT");
fscanf(fs,"%c",&e);
printf("%c%c%c\n",c,d,e);
```

Let in the file 0123456 is put. The output of the above program is 015 and the file contents are 0TTTT56. However when fprintf is removed then the output is 012. It shows that both buffers (read and write) are flushed when fscanf is done and write buffer is not empty. When write buffer is empty then fscanf does not flushes buffers.

```
FILE *fs; char c,d,e;
fs=fopen("exp","r");
fscanf(fs,"%c",&c);fflush(fs);
d='R'; read(3,&d,1);
fscanf(fs,"%c",&e);
printf("%c%c%c\n",c,d,e);
```

Let the contents of the file are xyzwst. The output of the above program is xyz. When we remove fflush then the output is xRy. It is because the entire file is put in buffer and read(3,&d,1) fails. If the size of buffer were 4 then the output had been xsy. It is because xyzw is put in buffer hence file pointer is at location 4.

```
fs=fopen("exp","r+");
fscanf(fs,"%c",&c);
fflush(fs);
write(3,"AA",2);
```

After reading 0th character the file pointer is 1. Hence AA is written after that. However when fflush is removed, characters 0 to 4095 are put in buffer. Hence file pointer is at 4096th character. Hence AA is written there. This can be verified by making a file in whose size is more than 4100 characters.

```
fs=fopen("exp","r+");
fscanf(fs,"%c",&c);
printf("%c",c);
write(3,"KKKKKK",6);
lseek(3,-2,SEEK_CUR);
fscanf(fs,"%c",&c);
printf("%c",c);
fflush(fs);
fscanf(fs,"%c",&c);
printf("%c",c);
```

Let the file has abcdefgh. After first fscanf the entire file is put into buffer and file pointer (FP) is 8 and the buffer has 7 letters (bcdefgh). After the execution of write and lseek the file pointer becomes 12. After the execution of second fscanf the buffer has 6 letters. When read buffer is flushed, the file pointer decrements by the size of buffer. Hence it becomes 6. Hence g is read in the last fscanf.

```
fs=fopen("exp","r+");
fscanf(fs,"%c%c%c%c",&c,&c,&c,&c);
fprintf(fs,"T");
lseek(3,-2,SEEK_CUR);
fflush(fs);
```

Fflush first flushes read buffer. Let initial file contents be abcdefgh. After the execution of fscanf the buffer has 4 letters efgh and FP=8. The lseek makes FP=6. When the read buffer is flushed FP=2. The contents of the file are abTdefgh. If the write buffer were flushed first then the file had been abcdefTh.

```
fs1=fopen("exp","w");
fs2=fopen("exp","w");
fprintf(fs1,"ABC");fflush(fs1);
fprintf(fs2,"PQ");fflush(fs2);
```

The contents of file are PQC. It is because first ABC is written then PQ. If we remove fflush(fs1) then file contents are ABC because PQ is immediately written. ABC is written when program is over (hence buffer is written). If we remove both fflushs then contents of the file may be any of these depending upon which buffer is written first.

```
fs=fopen("exp","w");
p=fork( );
if (p!=0)
{ fprintf(fs,"AA");
  sleep(2);
  fprintf(fs,"BB");
  sleep(3);
}
else
{ sleep(1);
  fprintf(fs,"xx");
  sleep(2);
  fprintf(fs,"yy");
}
```

It is expected that in the file AAxxBByy is written. However xxyyAABB is written. It is because both processes write in buffer. Only when a process is over (or buffer is full) the buffer is outputted in the file. If buffer size were 3 then in the file AABxxyyB would have been written. If we write fflush(fs) after every fprintf then we get the expected result.

Buffer management

```
#include<stdio.h>
#include<fcntl.h>
typedef  struct
        {int fd,rp,wp,rb,wb;
         char r[100],w[100];} FLE;
FLE* gopen(char* fname, int rb, int wb)
{  FLE *t;
   t=(FLE *)malloc(sizeof(FLE));
   t->fd=open(fname,O_RDWR);
   t->rp=rb; t->wp=0; t->rb=rb; t->wb=wb;
   return(t);
}
FLE* gdopen(int fd, int rb, int wb)
{  FLE *t;
   t=(FLE *)malloc(sizeof(FLE));
   t->fd=fd;
   t->rp=rb; t->wp=0; t->rb=rb; t->wb=wb;
   return(t);
}
void gprintf(FLE *fs,char *a)
{  int i=0;
   while ((a[i]!=(char)0))
   {  if (fs->wp==fs->wb)
      {  fs->wp=0;
         write(fs->fd,fs->w,fs->wb);
      }
      fs->w[fs->wp]=a[i];
      (fs->wp)++; i++;
   }
}
void gscanf(FLE *fs, char *a, int n)
{  int i;
   for (i=0;i<n;i++)
   {  if ((fs->rp)==(fs->rb))
      {   read(fs->fd,fs->r,fs->rb);
          fs->rp=0;
      }
      a[i]=fs->r[fs->rp];
      (fs->rp)++;
   }
}
void gflush(FLE *fs)
{ int temp;
   temp= (fs->rp)-(fs->rb);
   lseek(fs->fd,temp,SEEK_CUR);
   fs->rp=fs->rb;
   write(fs->fd,fs->w,fs->wp);
   fs->wp=0;
}
```

In a file named (say buffer) type the given program.

A file is opened and buffers for read and write are associated. Their sizes are rb and wb respectively.

A file, which was originally opened using low level system calls, a stream is associated.

When a write is performed in file then it is put in buffer. Only when buffer is full the contents are actually written.

When read from a file is performed then it is first checked whether buffer is empty. If it is so then number of characters (equal to the size of buffer) are read.

Read buffer is flushed by carrying the file pointer back (equal to the number of characters in the buffer). The write buffer is flushed by writing the contents of buffer in the file. First read and then write buffer is flushed.

```
#include<stdio.h>
#include<fcntl.h>
#include "buffer"
main( )
{  FLE *fs;
   fs=gopen("xyz",3,4);
   gprintf(fs,"ABCDEF");
   write(3,"PQ",2);
   gflush(fs);
}
```

The above program writes ABCDPQEF in the file. It is because the size of write buffer is 4

```
main( )
{   FLE *fs;char a[10],c;
    fs=gopen("xyz",3,4);
    gscanf(fs,a,2);
    printf("%c%c\n",a[0],a[1]);
    read(3,a,2);
    printf("%c%c\n",a[0],a[1]);
    gscanf(fs,a,2);
    printf("%c%c\n",a[0],a[1]);
}
```

When contents of file are abcdefgh then output of above program is abdecf.

```
main( )
{   FLE *fs;char a[10];
    fs=gopen("xyz",5,4);
    gscanf(fs,a,2);
    lseek(3,8,SEEK_SET);
    gflush(fs);
    gscanf(fs,a,2);
    printf("%c",a[0]);
}
```

When contents of file are 0123456789 then output is 5. After first gscsnf 234 is in read buffer. lseek carries the file pointer (FP) at 8th position. When buffer is flushed then FP goes 3 positions back (since the buffer has 3 characters).

```
main( )
{   FLE *fs1,*fs2;int fd;
    fd=open("xyz",O_WRONLY);
    fs1=gdopen(fd,6,6);
    fs2=gdopen(fd,6,6);
    gprintf(fs1,"A");
    gprintf(fs2,"B");
    gprintf(fs1,"C");
    gflush(fs1);
    gflush(fs2);
}
```

The function gdopen associates a stream with open instance of a file. More than one stream can be associated with same open instance of file a file. Both have different buffer.

1. Modify gscanf. If write buffer is not empty then read and write buffers are flushed.

2. Define gseek, gtell, greopen.

Shell Programming (Making Commands)

date outputs Fri Aug 1 12:01:55 GMT 1997

cal 5 1992 outputs the calendar of May 1992.

cal 2001 outputs the calendar of 2001**. cal 5** outputs the calendar of year 5.

cal 5 > tt1 outputs the calendar of year 5 in file tt1. It can be seen by **cat tt1** command.

ram=23 sets the value of variable ram as 23. It can be used in following commands till new setting is done**. x=hari** will set the value of variable x as hari.

echo Hari is friend of $ram kapil is ram outputs Hari is friend of 23 kapil is ram. $x means put the value of variable x.

echo ram is `date` mohan outputs ram is Fri Aug 1 12:01:55 GMT 1997 mohan. `x` means the output of command x. ` (back quote) is north west key of the key board.

cal 12 1997; sleep 5; cal 1 1999 outputs the calendars of Jan 1999 after 5 seconds.

set ram 12 om kapil sets the value of variable 1 as ram, 2 as 12, 3 as om, 4 as kapil, # as 4, and * as ram 12 om kapil. Here * is set as entire string. # is set as number of words in the string.

echo 3 ram $2 $4 ttt $# $* $1 outputs 3 ram 12 kapil ttt 4 ram 12 om kapil ram.

set `date`; echo $3-$2-$6 outputs 1-Aug-1997. `date` is replaced by output of command date. Value of variable 1 is set as Fri, 2 as Aug, …, 6 as 1997, # as 6, * as Fri Aug 1 12:01:55 GMT 1997.

ls outputs the name of all files in current directory.

date | cut –c2-6 outputs ri Au. The output of command date is cut between second and sixth characters. Similarily **echo ram kapil kapil | cut -c3-12** outputs m kapil ka.

```
for i in Ram Shyam Mohan
do
    echo hello $i  good morning
done
```

This command outputs
hello Ram good morning
hello Shyam good morning
hello Mohan good morning

Here value of i is taken as Ram, Shyam and Mohan respectively at different times in the loop.

y=date; z=te; w=a; p=3; q=2; k3=ram; pq=hari will set the values of various variables. Output of each of the following commands will be same as that of date. (type in separate lines) da`echo te` $y da$z d`echo $w`$z.

echo $pq outputs hari. To get the output 3q give **echo "$p"q** .
cal `echo $p` 2002 outputs calendar of March 2002. Same holds for **cal $p 2002** .
cal 1$q 2002 outputs the calendar of December 2002 (No blank between 1 and $).
cal 4 200`echo $p` outputs the calendar of April 2003.
p=23`echo $z`$w sets the value of p as 23tea. Hence **echo $p** outputs 23tea.

However followings do not work
echo $k`echo $p` does not output value of variable k3 (ram).
p`echo $p`=kapil does not set the value of p3 as kapil.
`echo $z`=32 does not set the value of variable te as 32

Making new commands

Sequence of commands are put in a file (say file name is t1). The file is made executable by command **chmod +x t1** (or by chmod 777 t1). Now a command **./t1** is ready.

In a file named t2 write echo $2 $# $* $3.
Make the file executable by command **chmod +x t2** and give command **./t2 ram om go** the output is om 3 ram om go go. It is because value of variable 1 is set as ram, variable 2 is set as om, variable 3 is set as go, variable # is set as 3, variable * is set as ram om go.

In a file named ttt write cal $1 1995; cal $2 1994 and give command **./ttt 3 5** the calendars of March 1995 and May 1994 are outputted.

Create a file named ttt1 as following.

```
for i in $*
do
   cal $i 1995
done
```

The command **./ttt1 3 5 12 8** will output the calendars of March, May, December, and August 1995.

Create a file named ttt2 as following.

```
m=$1
set `date`
cal $m $6 .
```

The command **./ttt2 7** will print the calendar of July current year. Here the value of variable 1 is saved in variable m because it is updated by set `date` (which is used to find current year).

```
case $1 in
Jan) cal 1 1997;;
Feb) cal 2 1997;;
   ….
Dec) cal 12 1997;;
esac
```

./ttt3 Mar will print the calendar of March 1997. Here month name (not number) is given.

```
set `ls`
echo I have $# files
```

It finds number of files.

```
set `ls`
for i in $*
do
    echo $i is my file
done
```

The above finds all files.

In a file named ttt5 write following.

```
case  $#  in
1)  m=$1;
    set `date`;
    cal  $m  $6;;
2)  cal  $1  $2;;
esac
```

./ttt5 7 1995 will print calendar of July 1995. **./ttt5 7** will print the calendar of July current year. The following will also do the same.

```
if  [   "$#"   =   2  ]
then
   cal  $1  $2
else
   month=$1
   set `date`
   cal  $month  $6
fi

x=`date | cut –c12-13`
y=`date | cut –c15-16`
echo it is $x hours $y minutes
```

```
set `cat temp`
for i  in  $*
do
    echo hello $i
done
```

Hello is done to all those persons, whose names appear in file temp.

```
echo Give month
read x
echo Give year
read y
cal  $x  $y

for  i  in  $*
do
    y=$z
    z=$i
done
echo last $z second last $y
```

./ttt8 ran om 12 6 hari
outputs last hari second last 6

Suppose today is Tuesday.
In a file named Tue type following.

```
for  i  in  $*
do
  echo  $i  good morning
done
```

Make the file executable by **chmod +x Tue** and observe the output of command **./`date`**

Fundamentals

Suppose in a file named x we write echo t, in a file named t we write echo wert, in a file named tw we write echo you, in a file named q we write echo y, and we make these files executable. Now command ./x outputs t, command ./t outputs wert, command ./tw outputs you and command ./q outputs y.

./`./x` outputs wert (output of command ./x is t and that of ./t is wert)
./`./x`w outputs you (output of command ./x is t and that of ./tw is you)
y=x; t=ram; date=123 does appropriate setting.
./$y outputs t (value of variable y is x and output of command ./x is t).
echo t $t `./t` outputs t ram wert.
echo date ram $date hari `date` outputs date ram 123 hari Fri Aug 1 12:01:55 GMT 1997. Here variable date and command date are different.
./$`./q` does not output t.

1. Define command cala. **cala 1995 7** will print the calendar of July 1995.

2. Define command calb. **calb Mar 1995** will print calendar of March 1995. Here month name and year are given.

3. Define calc. **calc 3 8 5** will print calendars of March, August, and May 1995. Any number of month numbers can be given.

4. Define cald. It is same as calc except year is current year.

5. Define cale. **cale Mar 1999** will print calendar of March 1999**. cale Mar** will print that of March current year. If year is not given, it is taken as current year.

6. Define calf. **calf 2 1999** will print calendar of March 1999. Calendar of next month is printed. (except **calf 12 1995** will print the calendar of January 1995).

7. Define calg. **calg Mar Aug May** will print calendars of March, August, and May 1995. Any number of month names can be given.

8. Define dayname. **dayname 7** will print Sunday. **dayname 5** will print Friday.

9. Define daynumber. **daynumber Sunday** will print 7**. dayname Friday** will print 5.

10. Define today. It will print weekday of today (Hint: variable 1 in command date).

11. Define data1. Current date will be printed in the format 1-8-1997.

12. Define data2. Current date will be printed in the format 1-8-97.

13. Define inshort. **inshort rajiv 3** will output raj. **inshort kapilkumar 7** will output kapilku.

14. Define latetime. **latetime 5** will not print anything for 5 seconds. After that it will print the time which was at the time when this command was given (Hint: use sleep).

15. Define putcal. **putcal 10 1994 t1** will put the calendar of October 1994 in file t1.

16. Define currentcalender. It makes the calendar of current month and year.

17. Define filecontents. It displays the contents of all files in current directory.

18. Define calall. **calall 1994 t1 t2** will make calendars of all months listed in file t1. The output will be put in file t2. Assume file t1 has only numbers between 1 and 12.

19. Define calall1. It is like above except the output file name may or may not be given. If no output file name is given then the output will come on the monitor.

20. Define wel. **wel Ram Hari Gopal** will say hello for Ram. For Hari and Gopal it will say good morning. To the first person hello is said. To the remaining persons good morning is said. There may be any number (more than one) of persons and all persons have different names.

21. Define reverse. **reverse hari ram 12 16** will output 16 12 ram hari. **reverse 12 3 4** outputs 4 3 12.

22. Define short. **short Ram Chandra Gupta** will output R C Gupta. **short 12 34 123 45** will output 1 3 1 45.

23. Define short1. It works as above except output is R.C.Gupta and 1.3.1.45 respectively.

24. Define calw. **calw 3 8 5 1995** will print calendar of March, August, and May 1995. Any number of month numbers can be given. The last number is year.

25. Define calw. **Calwp 1995 3 8 5** will print calendar of March, August, and May 1995. Any number of month numbers can be given. The first number is year.

26. Define calx. **calx 6 12** will print calendar of June 1712. Only last two digits of year in 18th century are specified. **calx 4 46** will print calendar of April 1746.

27. Define caly. It is like above except first two digits of year are taken as those of current century. Now **caly 4 46** will print calendar of April 2046. Few years back **caly 4 46** would have printed the calendar of April 1946.

28. Define warmwelcome. **warmwelcome 20** will print hello 20 times.

29. Define specialwelcome. **specialwelcome 20 3** will print hello 20 times. The time difference between two hellows will be 3 seconds.

30. Define cfiles. It will display the number of C files in the current directory.

31. Define calz. The command will ask for month name. If user says current then calendar of current month and current year is printed. If user gives some month name (say Mar) then the year is asked. If user says current, then the calendar of March current year is printed. If user says 2050, calendar of March 2050 is printed.

32. Define picture1. **Picture1 a b ram hari** will print a a b b ram ram hari hari

33. Define picture2. **Picture2 a b ram hari** will print a b ram hari a b ram a b a

34. Define duplicate. It will print those words, which occur more than once.

duplicate a b ram a a hari hari will output a hari

35. Define command find. **find 3 ram shyam mohan kapil ravi om** will outputs mohan. The program finds the nth word. The value of n is given as first argument of command.

36. Do above problem, when we want n^{th} word from last. The output in above case will be kapil.

37. Define command break. It breaks given string into words. X is treated as separator. **Break ramXomXkapilXravi** will output ram om kapil ravi. The words may be outputted in separate line.

38. Define replace. Every occurrence of 'a' is replaced by 'b'. **replace ram** will output rbm. **replace waerasa** will output wberbsb.

39. Define length. It finds the length of a word. **length ram** will output 3.

40. Define command reverse. **reverse ram** will output mar.

41. Define command gm. It says good morning to every alternate person. **gm ram om shyam kapil ravi hari** will say good morning to ram shyam and ravi.

42. Define command, which reads ten words and prints them in reverse order. In input words are in separate lines. In output words are in same line.

43. Define remove. The first word is removed and others are printed in same line. **remove ram shyam hari om gopal** will output shyam hari om gopal.

44. Define command cmda. **cmda 27152** will output the calender of July 2152. **cmda 68731** will output the calender of August 6731. The input is a 5 digit number. 2nd digit is month number. Other digits are year.

45. Define command, which outputs good morning 100 times in separate lines. In the definition of this command there should be less than 10 lines. Every line should have less than 25 characters.

46. Define command count. **count hari ram kapil tom ram gopal** outputs 2. **count ram tom ram ram ram kapil ram** outputs 5. The command outputs the number of ram.

47. Define command xyz. It takes few words and a number (n) and outputs those words n times. **xyz ram om hari 3** will output ram om hari ram om hari ram om hari.

48. Define command lmn, which takes two number (the first is smaller) and prints all numbers between them. **lmn 3 7** will output 3 4 5 6 7.

File Compression

We want to code a string of 0's and 1's. There are a large number of 0's. A string is decomposed in blocks. If all elements of block are 0 then block is coded as 0. Otherwise the block is coded as 1x. Here x are contents of block. Following program finds the compression.

```
#include<stdio.h>
#include<stdlib.h>
#include<math.h>
int rd(int r) { int j,k; j=rand()%(r+1); if (j==r) k=1; else
k=0;return(k);}
main()
{    int i,j,k,n,r,h,b,count=0,total=0;FILE *fp;float p,t,ans;
     fp=fopen("data","w");
     printf("Give string length:");scanf("%d",&n);
     printf("Give ratio of probabilities of getting 0 and 1:");
     scanf("%d",&r);
     printf("Give block size");scanf("%d",&b);
     for (i=1;i<=n;i++){
          k=rd(r);fprintf(fp,"%d",k);
          if (k==1) count++;
          if (i%b==0)
          {   fprintf(fp,"\n");
              if (count==0) total=total+1;
              if (count>=1)  total=total+1+b;
              printf("%d %d\n",count,total);count=0;
          }
     }
     printf("Compression ration by simulation  %8.6f\n",(float)total/n);
     p=1.0/(1+r);t=pow((1-p),b);ans=t*1+(1-t)*(1+b);
     printf("Compression ration through calculation %8.6f ",ans/b);
}
```

The above scheme is used because a block is likely to have all elements as 0. However if all elements are not zero then most likely there shall be only one 1. A block of all zero's is coded as 0. A block, which has exactly one 1, is coded as 10y. Here y is the location of 1 in the block. A block, which has two or more 1's, is coded as 11x. Where y are block contents.

```
     if (count==0) total=total+1;
     if (count==1) total=total+ulg(b)+2;
     if (count>1)  total=total+2+b;
```

Compression ration through calculation is ans/b. Where
p=1.0/(1+r);t=pow((1-p),b);s=b*p*pow((1-p),(b-1));ans=t*1+s*(ulg(b)+2)+(1-t-s)*(2+b);

The function ulg (upper $\log_2$ is defined as follows)

```
int ulg(int a) { int i=1,c=0; while (i<a) {i=i*2;c++;} return(c);}
```

The following table shows comparison of compression ratio's in above two schemes. P(0)=0.99 P(1)=0.01

Block size	First scheme	Second Scheme
20	0.23	0.11
10	0.19	0.15
5	0.24	0.23

The string is coded as sequence of numbers. The numbers at even position signify continuous number of zeros. The numbers at odd position signify continuous number of 1's. Every number of the sequence is coded in b_i bits. However if a number (x) is more than or equal to 2^{bi} then it is coded as (b_i 0's)(code of x-2^{bi} -1). If b_i=3 then 22 will coded as 000000000001.

```
int rd(int r){ int j,k; j=rand()%(r+1); if (j==r) k=1; else
k=0;return(k);}
int power(int x){ int i,y=1; for (i=1;i<=x;i++) y=y*2; return(y); }
main()
{    int i,j,k,n,r,h,b,count=1,total=0,bi;FILE *fp;
     fp=fopen("data","w");
     printf("Give string length:");scanf("%d",&n);
     printf("Give ratio of probabilities of getting 0 and 1:");
     scanf("%d",&r);
     printf("Give no of bits in representation:");
     scanf("%d",&bi);b=power(bi);
     k=rd(r);fprintf(fp,"%d",k);
     for (i=1;i<=n-1;i++)
     {    h=k;
          k=rd(r);fprintf(fp,"%d",k);
          if (h==k) count++;
          else
          {  total=total+((count-1)/(b-1)+1)*bi;
             printf("%d %d\n",count,total);count=1;
             if (k==1) fprintf(fp,"\n");
          }
     }
     printf("Compression is %8.6f\n",(float)total/(n-count));
}
```

The continuous number of 0's is coded in b_i bits. If they are more than or equal to 2^{bi} –1 then they are coded as (b_i 0's)(code of x – 2^{bi} –2). However to code the number of 1's the following scheme is used. If there is exactly one 1 then it is not at all coded. If there are x (more than 1) 1's then it is coded as (b_i 1's)(x-2 1's) 0. This scheme will be useful when P(0)>>P(1).

```
          if (h==k) count++;
          else
          {  if (k==1) total=total+((count-1)/(b-2)+1)*bi;
             if ((k==0)&&(count>1)) total=total+bi+(count-2)+1;
             printf("%d %d\n",count,total);count=1;
             if (k==1) fprintf(fp,"\n");
          }
```

The following table shows comparison of compression ratio's in above two schemes. P(0)=0.99 1

Bits used in representation	First scheme	Second Scheme
6	0.18	0.12
7	0.16	0.09
8	0.16	0.08
9	0.18	0.09

A number (x) is coded as follows: x is written as 2^t+y where $y<2^t$.
code(x) = (t 1's)0(binary representation of y in t bits).

```
int rd(int r){ int j,k; j=rand()%(r+1); if (j==r) k=1; else
k=0;return(k);}
int llg(int a){ int i=2,c=0; while (i<=a) {i=i*2;c++;} return(c);}
main()
{    int i,j,k,n,r,h,b,count=1,total=0;FILE *fp;
     fp=fopen("data","w");
     printf("Give string length:");scanf("%d",&n);
     printf("Give ratio of probabilities of getting 0 and 1:");
     scanf("%d",&r);
     k=rd(r);fprintf(fp,"%d",k);
     for (i=1;i<=n-1;i++)
     {    h=k;k=rd(r);fprintf(fp,"%d",k);
          if (h==k) count++;
          else
          {  total=total+2*llg(count)+1;
             printf("%d %d\n",count,total);count=1;
             if (k==1) fprintf(fp,"\n");
          }
     }
     printf("compression ratio %8.6f\n",(float)total/n);
}
```

Code for 1: If there are x continuous 1's then they are coded as follows: x=1 is not coded at all. Otherwise code is 0(x-2 1's)0.
Code for 0: Since in the previous scheme of coding x (x=1) continuous 0's are coded as 0. To avoid conflict code of (x+1) is written.

```
int rd(int r){ int j,k; j=rand()%(r+1);
                        if (j==r) k=1; else k=0;return(k);}
int llg(int a){ int i=2,c=0; while (i<=a) {i=i*2;c++;} return(c);}
main()
{    int i,j,k,n,r,h,b,count=1,total=0;FILE *fp;
     fp=fopen("data","w");
     printf("Give string length:");scanf("%d",&n);
     printf("Give ratio of probabilities of getting 0 and 1:");
     scanf("%d",&r);
     k=rd(r);fprintf(fp,"%d",k);
     for (i=1;i<=n-1;i++)
     {    h=k;k=rd(r);fprintf(fp,"%d",k);
          if (h==k) count++;
          else
          {  if (k==1) total=total+2*llg(count+1)+1;
             if ((k==0)&&(count>1)) total=total+(count-2)+2;
             printf("%d %d\n",count,total);count=1;
             if (k==1) fprintf(fp,"\n");
          }
     }
     printf("compression ratio %8.6f\n",(float)total/n);
}
```

The following table shows comparison of compression ratio's in above two schemes.

P(0)	First scheme	Second Scheme
0.99	0.12	0.11
0.999	0.019	0.018

```
fs=fopen("exp","r+");
fprintf(fs,"TT");
lseek(3,2,SEEK_SET);
fscanf(fs,"%c",&c);
printf("%c\n",c);
lseek(3,4,SEEK_SET);
```

Using r+ the file is opened for the purpose for reading and writing. The above example shows that write buffer is flushed when fscanf is performed. Let the initial contents of the file be abcdef. The final contents are abTTef. If we remove fscanf then the contents of the file are abcdTT. Obsever the output by replacing fscanf by read.

Pipe should not be used in first four problems.

1. Read n and create a system of 2^n-1 processes. Every process has exactly two children. If n=3 and following line is written at the end

   ```
   printf("[%d %d]",getpid( ),getppid( ));
   sleep(1);
   ```

   then the output will be

   [AL],[BA],[CA],[DB],[EB],[FC],[GC]

2. Define a function cork( ). It creates 2 addition processes. Let parent be A, child be B and grandchild be C. A returns id of B. B returns id of C and C returns id of A.

3. Define a function eork( ). It creates 10 more processes. Let A be the id of main process and B, C, D, … ,K are id's of other processes. A is father of B, B is father of C, C is father of D, …, J is father of K. In C the function call returns A, in D it returns B, . . , in K it returns I, in A and B it returns 0.

4. Define a function gork( ). It creates 2 addition processes. Let parent be A, child be B and grandchild be C. A returns id of C. B returns id of A and C returns id of B.

5. Define a function jork( ). It creates 3 addition processes. Let parent be A, child be B, grandchild be C, and great grandchild be D. A returns id of B. B returns id of D, C returns id of A and D returns id of C. [Hint: make two pipes one between B and C another between C and D.]

6. Define a function lork( ). It creates 10 more processes. Let A be the id of main process and B, C, D, … ,K are id's of other processes. A is father of B, B is father of C, C is father of D, …, J is father of K. In A the function call returns C, in B it returns D, . . , in I it returns K, in J and K it returns 0.

Assembly Language Programming

Picture printing

Simple Loops

Conditional

Input

Big Numbers

Macro

Function

Shift and Rotate

Float representation

Word size register

Mul and Div

Array

Matrix

String

Linked List

Picture Printing

```
.model small
.code
   Mov AH,2
   Mov BH,1
L1:Mov DL,65
   Add DL,BH
   Int 33
   Add BH,1
   Cmp BH,5
   JLE L1
   Mov AH,76
   Int 33
   End
```

The above program outputs BCDEF

Modify the contents of the box to print followings. You may use more than one instruction

1. ABCDE
2. CEGIK
3. FEDCB
4. AAAAA

```
   Mov AH,2
   Mov CH,1
L2:Mov BH,1
L1:Mov DL,70
   Sub DL,BH
   Add DL,CH
   Add DL,CH
   Int 33
   Add BH,1
   Cmp BH,4
   JLE L1
   Mov DL,','
   INT 33
   Add CH,1
   Cmp CH,4
   JLE L2
   Mov AH,76
   Int 33
   End
```

Output GFED,IHGF,KJIH,MLKJ,

Modify the contents of the box so that followings are printed.

5. GHIJ,GHIJ,GHIJ,GHIJ,
6. GGGG,HHHH,IIII,JJJJ,
7. AAAA,AAAA,AAAA,AAAA,
8. ABCD,ABCD,ABCD,ABCD,
9. DEFG,FGHI,HIJK,JKLM,
10. BCDE,BCDE,BCDE,BCDE,
11. FGHI,EFGH,DEFG,CDEF,

```
.model small
.code
   Mov AH,2
   Mov CH,1
L2:Mov BH,1
L1:Mov DL,65
   Int 33
   Add BH,1
   Mov BL,6
   Sub BL,CH
   Cmp BH,BL
   JLE L1
   Mov DL,','
   INT 33
   Add CH,1
   Cmp CH,4
   JLE L2
   Mov AH,76
   Int 33
   End
```

Output AAAAA,AAAA,AAA,AA

Modify the contents of the box so that followings are printed.

12. A,AA,AAA,AAAA,
13. AAA,AAAA,AAAAA,AAAAAA,

```
.model small
.code
Mov AH,1
Int 33
Mov DL,AL
Int 33
Sub DL,AL
Add DL,48
Mov AH,2
Int 33
Mov AH,76
Int 33
end
```

The above program reads two digits and outputs their difference. It is assumed that first digit is bigger than second.

14. Modify above program to output sum of two digits. Assume sum is less than 10.
15. Write program to compute sum, when sum may be more than 10 also. Input 57 output 12. Input 24 output 6.
16. Write program to compute difference. Any digit may be bigger. Input 57 output 2. Input 83 output 5.

17. Write program to subtract the second digit from the first digit. Input 45 output -1. Input 93 output 6.
18. Write program, which reads a digit and a letter. If letter is ‘A’ then double of the digit is outputted. If letter is ‘B’ then digit is incremented. Assume that digit is less than 5. Input 3A output 6. Input 4B output 6.

The end

```
.model small
.code
   Mov AH,1
   Int 33
   Cmp AL,97
   JGE L1
   Mov DL,225
   Jmp L2
L1:Mov DL,224
L2:Mov AH,2
   Int 33
   Mov AH,76
   Int 33
end
```

The program reads a letter. It outputs if ASCII code of the letter is more than or equal to 97. Otherwise it outputs β. Input Y output β. Input c output α.

1. Write program, which reads a letter. It outputs α if ASCII code of the letter is between 65 and 90. Otherwise it outputs β. Input 5 output β. Input C output α. Input c output β.
2. Write program, which reads a letter. It outputs α if ASCII code of the letter is more than or equal to 97. It outputs β if ASCII code is between 65 and 96. It outputs γ if ASCII code is less then 65. Input F output β. Input c output α. Input 5 output γ.
3. Write program, which reads a letter. It outputs α if ASCII code of the letter is even. Otherwise it outputs β. Input 5 output β. Input D output α.
4. Write program, which reads two letters. It outputs α if ASCII code of both letters is even. It outputs β if ASCII code of any of the letters is odd. Input 5B output β. Input Bd output α. Input 6C output β. Input 5E output β.
5. Write program, which reads a letter. It outputs α if ASCII code of the letter is between 65 and 90 or between 97 and 122. Otherwise it outputs β. Input 5, [or { output β. Input c or C output α. [ASCII codes of [and { are 91 and 112 respectively].

Machine Codes

Instruction	Machine code
Mov reg, number	1011wreg number
Add reg, number	1000000w 11000reg number
Sub reg, number	1000000w 11101reg number
Cmp reg, number	1000000w 11111reg number
Add AL, number or Add AX,number	0000010w number
Sub AL, number or Sub AX,number	0010110w number
Cmp AL, number or Cmp AX,number	0011110w number
Mov reg_1,reg_2	100010dw 11reg_1,reg_2
Add reg_1,reg_2	000000dw 11reg_1,reg_2
Sub reg_1,reg_2	001010dw 11reg_1,reg_2
Cmp reg_1,reg_2	001110dw 11reg_1,reg_2
Mul reg	1111011w 11100reg
Div reg	1111011w 11110reg
Inc reg	1111111w 11000reg
Dec reg	1111111w 11001reg
Inc reg (word size)	01000reg
Dec reg (word size)	01001reg

d:1 for correct w:0 for byte

000	001	010	011	100	101	110	111
AL AX	CL CX	DL DX	BL BX	AH SP	CH BP	DH SI	BH DI

Int number	11001101 number
Jmp	11101011 disp

Jmp conditional: 0111 (in case of negation 1 is added)

JO: 0000	JB: 0010	JE: 0100	JBE: 0110	JS: 1000	JP: 1010	JL: 1100	JLE: 1110

Shift/Rotate 110100cw 11xyzreg

c=0 for count=1 x=0 for rotate z=0 for left y=0 for logical shift or rotate without carry

```
.model  small
.code
  db 10110100b
  db 2d
  db 10110010b
  db 70
  db 11001101b
  db 33d
  db 10110100b
  db 76d
  db 11001101b
  db 33d
end
```

```
.model small
.code
 Mov AH,2

 Mov DL,70

 Int 33

 Mov AH,76

 Int 33

end
```

A program is written in machine code. It prints ‘F’.

6. Write program in machine code, which reads a letter and outputs next letter. Input K output L.
7. Write program, which reads two letters and ouputs bigger. Input tk output t.
8. Write program, which reads two digits and outputs their sum. Input 59 output 14,

9. Write program, which outoputs their difference. Input 59 output -4.
10. Write program in machine code to print ABCD..Z.
11.
12. sdag

Picture printing

```
.model small
.code
        Mov     AH,2
        Mov     DL,82
        Int     33
        Mov     DL,97
        Int     33
        Mov     DL,'m'
        Int     33
        Mov     AH,76
        Int     33
        END
```

The above program prints Ram
Make a file under the name t.asm
Give following commands
masm t;
link t;
t;

Mov a,b means a=b
Add a,b means a=a+b

Execution of int 33

a) if AH is 2 : prints the character corresponding to the contents of DL.
b) if AH is 76 : Stops the execution of program

```
.model small
.code
        Mov     AH,2
        Mov     DL,65
L1:     Int     33
        Add     DL,1
        Cmp     DL,90
        JLE     L1
        Mov     AH,76
        Int     33
        END
```

The above program prints
ABCD . . . Z

JLE is conditional jump. It means jump if less than or equal to
JG means jump if greater than
JNE means jump if not equal to
JGE JGL JE JNG JNGE
are other conditional jumps.
JMP is unconditional jump.

```
.model small
.code
        Mov     AH,2
        Mov     CH,1
L2:     Mov     DL,64
        Add     DL,CH
L1:     Int     33
        Add     DL,1
        Cmp     DL,70
        JLE     L1
        Mov     DL,10
        Int     33
        Mov     DL,13
        Int     33
        Add     CH,1
        Cmp     CH,6
        JLE     L2
        Mov     AH,76
        Int     33
        END
```

The above program prints
ABCDEF
BCDEF
CDEF
DEF
EF
F

Any of the following registers can be used as variables:
AH AL BH BL CH CL DH DL
Caution: int 33 destroys the contents of AL
For going to new line characters whose ASCII codes are 10 and 13 are printed

sadfsdf

Write program to print following pictures

44. AAAAAA

45. AAABBBCCCDDD

46. ABBCCCDDDD

47. AAAABBBCCD

48. AAAXAAAXAAAX

49. AaAAaAAAaAAAAa
,

50. ACEGIKM

51. ABCDEF
ABCDE
ABCD
ABC
AB
A

52. A
AB
ABC
ABCD
ABCDE
ABCDEF

53. A
BC
CDE
DEFG
EFGHI
FGHIJK

54. A
BC
DEF
GHIJ
KLMNO
PQRSTU

55. ABCDEF
ACEGI
ADGJ
AEI
AF
A

```
56.  ABCDEF
      BCDEF
       CDEF
        DEF
         EF
          F
```

```
57.  Aabcde
     ABabcd
      ABCabc
      ABCDab
      ABCDEa
      ABCDEF
```

```
58.  aBCDEF
     abCDEF
     abcDEF
     abcdEF
     abcdeF
```

```
59.        F
          EF
         DEF
        CDEF
       BCDEF
      ABCDEF
```

```
60.        K
          JI
         IHG
        HGFE
       GFEDC
      FEDCBA
```

```
61.       A
         ABA
        ABCBA
       ABCDCBA
      ABCDEDCBA
```

62. A
BC
CDEF
DEFGHIJ
EFGHIJKLMNO

```
63.  U
     ST
     PQR
     LMNO
     GHIJK
     ABCDEF
```

64. ABCDEF
BBCCDDEEFF
CCCDDDEEEFFF
DDDDEEEEFFFF
EEEEEFFFFF
FFFFFF
.

65. ABDGKPV

66. 1,2,3,4,5,6,

67. 0,4,8,12,16,20,24,28,
0,5,10,15,20,25,30,
0,6,12,18,24,30,
0,7,14,21,28,
0,8,16,24,
0,9,18,27,
0,10,20,30,

68. 0,4,8,12,16,20,24,28,
0,5,10,15,20,25,30,
0,6,12,18,24,30,
0,7,14,21,28,
0,8,16,24,
0,9,18,
0,10,

69. 1
1 3
1 3 6
1 3 6 10
1 3 6 10 15
1 3 6 10 15 21

70. A
BC
CDEF
DEFGHIJK
EFGHIJKLMNOPQRST

71. ABCDEBCDECDEDEE
ABCDEBCDECDEDE
ABCDEBCDECDE
ABCDEBCDE
ABCDE

72. ABCDEBCDECDEDEE
ABCDBCDCDD
ABCBCC
ABB
A

73. ABCDEABCDABCABA
BCDEBCDBCB
CDECDC
DED
E

74. ABCDEFGH
ABCDEFGABCDEF
ABCDEFABCDEABCD
ABCDEABCDABCAB

75. ABCDEBCDECDEDEE
BCDECDEDEE
CDEDEE
DEE
E

76. ABCDEABCDABCABA
ABCDABCABA
ABCABA
ABA

77. AABABCABCDABCDE
BBBBCCBBCCDDBBCCDDEE
CCCCCCDDDCCCDDDEEE
DDDDDDDDEEEE
EEEEE

78. ABBCCCDDDDEEEEE
AABBBCCCCDDDDD
AAABBBBCCCCC
AAAABBBBB
AAAAA

79. ABCDEABCDABCABA
BBCCDDEEBBCCDDBBCCBB
CCCDDDEEECCCDDDCCC
DDDDEEEEDDDD
EEEEE

80. EFGHIEFGHEFGEFE
DEFGDEFDED
CDECDC
BCB
A

81. ABCDEFGHIJKLMNOPQRST
ACEGIKMOQS
ADGJMPS
AEIMQ

82. Read two integers a and b and print following picture

(a)(b)
(a)(b)(a+b)
(a)(b)(a+b)(a+2b)
(a)(b)(a+b)(a+2b)(2a+3b)
(a)(b)(a+b)(a+2b)(2a+3b)(3a+5b)
(a)(b)(a+b)(a+2b)(2a+3b)(3a+5b)(5a+8b)

If a=5 and b=3 then picture is

(5)(3)
(5)(3)(8)
(5)(3)(8)(11)
(5)(3)(8)(11)(19)
(5)(3)(8)(11)(19)(30)
(5)(3)(8)(11)(19)(30)(49)

83. 101 102 106 124 220 820
102 106 124 220 820
106 124 220 820
124 220 820
220 820
820

Simple loops

The following program finds $\sum_{i=x}^{i=y} i$

Corresponding C program is as

```
AL=x
BL=y
DL=0
do{
      DL=DL+AL
      AL=AL+1
   } while (AL≤BL)
print(DL)
```

```
        Mov     AL,x
        Mov     BL,y
        Mov     DL,0
L1:     Add     DL,AL
        Add     AL,1
        Cmp     AL,BL
        JLE     L1
        Mov     AH,2
        Int     33
        Mov     AH,76
        Int     33
```

To check the answer we print the character whose ASCII code is the result. For x=5 and y=12 the result is 68. Hence D is printed.

```
e
do{
    p
    q
  } while (s<t)
f
```

is translated as

```
      e
L1:   p
      q
      Cmp  s,t
      JL   L1
      f
```

Print a two digit number x

```
CL=x
CH=0
while (CL≥10)
{     CH=CH+1
      CL=CL-10
}
print(CH,CL)
```

```
        Mov     CL,x
        Mov     CH,0
L1:     Cmp     CL,10
        JL      L2
        Add     CH,1
        Sub     CL,10
        JMP     L1
L2:     Mov     DL,CH
        Add     DL,48
        Mov     AH,2
        Int     33
        Mov     DL,CL
        Add     DL,48
        Int     33
        STOP
```

Digits are calculated as

CH=x div 10

CL=x mod 10

Each digit is printed by adding 48 because ASCII code of a digit and its value differ by 48

```
e
while (s<t)
{    p
     q
}
f
```

is translated as

```
      e
L1:   Cmp  s,t
      JGE  L2
      p
      q
      JMP  L1
L2:   f
```

1. Write program to find x*y.

2. Write program to find (x div y) and (x mod y).

3. Write program to find $\sum_{i=1}^{i=x} i*(i+1)$.

   For x=5 it is

   1*2+2*3+3*4+4*5+5*6=70.

4. Write program to find gcd of x and y using following logic:

```
while (a≠b){
    if (a>b)  a=a-b;
   else b=b-a; }
print(a);
```

5. Write program to find x^y.

6. Write program to find gcd of x and y using following logic:

```
for (i=x; ((x%i!=0)||(y%i!=0)) ;i--);
print(i);
```

7. Write program, which checks whether x is a perfect square or not. The program outputs T if x is a perfect square and F otherwise. [Hint: Test whether 1^2=x or 2^2=x or 3^2=x or …. i^2=x till i^2>x].

8. Write program to find biggest perfect square less than x. If x=56 then answer is 49.

9. Write program to find smallest factor of a number x. [Hint: Find smallest i such that (x mod i =0)].

10. Write program to test whether x is prime or not. Output will be T of F.

11. Write program to find sum of all factors of a number. If x=24 then answer is 2+3+4+6+8+12+24=59.

12. Write program to print x in base 3. e.g. base 3 representation of 113 is 11012. You may print the reverse of representation e.g. in above case 21011. Use following logic

```
while (x≠0) {
    print(x mod 3);
    x=x div 3;   }.
```

13. Write program to find gcd of x and y using following logic:

```
do {
    r=a mod b; a=b; b=r;
   }while (r≠0); print(a);
```

14. Write program to print ASCII codes of all those characters which are prime and are between x and y. If x=67 and y=85 then output will be CGIOS, because corresponding ASCII codes are 67, 71, 73, 79 and 83 respectively. These are primes between 67 and 85.

15. Write program to find x*y without using Add. You can use Inc. [Hint: Inc AL ≡ Add AL,1].

16. Write program to find x^y. without using Add. You may use Inc.

17. Write program to find biggest power of x which divides y. If x=3 and y=108 then answer is 3 because 3^3 divides 108 but 3^4 does not. If x=2 and y=96 then answer is 5. If x does not divide y then answer is 0. You should use following method:

```
ans=0;
while (y mod x==0) {
    y=y div x;
    ans=ans+1;}
print(ans);
```

Conditional

```
a
if (p>q)
{    r
     s
}
b
```

is translated as

```
     a
     Cmp   p,q
     JNG  L1
     r
     s
L1:  b
```

Program which prints
α β if x>y and β otherwise
(ASCII α =224 β =225 Γ =226)

```
if  (x>y)
    print(α)
print(β)
```

```
        Mov     BL,x
        Mov     BH,y
        Mov     AH,2
        Cmp     BL,BH
        JNG     L1
        Mov     DL,224
        Int     33
L1:     Mov     DL,225
        Int     33
        Mov     AH,76
        Int     33
        END
```

See output on
x=20 y=30
x=30 y=20

```
a
if (p>q)
{    r
     s
}
else
{    t
     u
}
b
```

is translated as

```
     a
     Cmp   p,q
     JNG  L1
     r
     s
     JMP  L2
L1:  t
     u
L2:  b
```

Program which prints
α Γ if x>y and β Γ otherwise

```
if  (x>y)
    print(α)
else
    print(β)
print(Γ)
```

```
        Mov     BL,x
        Mov     BH,y
        Mov     AH,2
        Cmp     BL,BH
        JNG     L1
        Mov     DL,224
        Int     33
        JMP     L2
L1:     Mov     DL,225
        Int     33
L2:     Mov     DL,226
        int     33
        STOP
```

```
a
if (p>q) or (t>u)
{   r
    s
}
b
```

is translated as

```
    a
    Cmp   p,q
    JG    L1
    Cmp  t,u
    JNG L2
L1: r
    s
L2: b
```

Program that prints
α if x>10 or y>20
nothing is printed
otherwise

```
if (x>10) or (y>20)
    print(α)
stop
```

```
        Mov     BL,x
        Mov     BH,y
        Mov     AH,2
        Cmp     BL,10
        JG      L1
        Cmp     BH,20
        JNG     L2
L1:     Mov     DL,224
        Int     33
L2:     Mov     AH,76
        Int     33
        END
```

See output for
x=7 y=17
x=7 y=27
x=17 y=17
x=17 y=27

```
a
if (p>q) and (t>u)
{   r
    s
}
b
```

is translated as

```
    a
    Cmp  p,q
    JNG  L1
    Cmp  t,u
    JNG  L1
    r
    s
L1: b
```

Program that prints
α if x>10 and y>20
nothing is printed
otherwise

```
if (x>10) and (y>20)
    print(α)
stop
```

```
        Mov     BL,x
        Mov     BH,y
        Mov     AH,2
        Cmp     BL,10
        JNG     L1
        Cmp     BH,20
        JNG     L1
        Mov     DL,224
        Int     33
L1:     Mov     AH,76
        Int     33
        END
```

1. Write program, which prints α if x>10 and y>20. β is printed otherwise.

2. Write program, which prints α if x>10 or y>20. β is printed otherwise.

3. Write program, which prints α if x is even. Nothing is printed otherwise. [Hint: calculate (x mod 2)]

4. Write program, which prints α if x is even and x>10. Nothing is printed otherwise. See output for x=12, 13, 7, 8. Only for x=12 the output will be α.

5. Write program, which prints α if x is even and x>10 or x is odd and x<10. Nothing is printed otherwise. See output for x=12, 13, 7, 8. For x=7 and 12 the output will be α. Program segment to find (x mod 2) should be written only once.

6. Write program, which prints α if x is even and x>10 and x<30. Nothing is printed otherwise. See output for x=12, 13, 7, 8, 37, 38. Only for x=12 the output will be α.

7. Write program, which prints α if x is even or x<10 or x>30. Nothing is printed otherwise. For above case only for x=13 no output will be printed.

8. Write program, which prints α if x<10, β if x>30, and Γ otherwise.

9. Write program, which prints α if x>10 and x<20, or x>50 and x<60. β is printed otherwise. See output for x=5, 15, 30, 55, and 65. Only for 15 and 55 α will be printed.

10. Write program, which prints α if x>10 and x<20 and y>50 and y<60. β i s printed otherwise. See output for all combinations of x=5, 15, and 30 and y=45, 55 and 65. Only for x=15 and y=55, α will be printed. For remaining 8 combinations β will be printed.

11. Write program, which prints α if exactly one among x and y is less than 20. β is printed otherwise. See output for (x,y)=(5,5), (5,25), (25,5) and (25,25). For (5,25) and (25,5) α will be printed.

12. Write program, which prints α if at least one among x and y is less than 20. β is printed otherwise. For above case only for x=25 and y=25 β will be printed.

13. Write program, which prints α if bigger of x and y is even. β is printed otherwise. See output for (x,y)=(5,15), (15,5), (20,5), (5,20) and other exhaustive combinations. Program segment to find (mod 2) should be written only once.

14. Write program, which prints α if exactly two among x, y and z are less than 10. β is printed otherwise. x=5 y=15 z=7 will print α. x=5 y=15 z=17 will print β.

15. Write program, which prints α if x is between 0-9, 20-29, 40-49, 60-69, 80-89. β is printed otherwise. e.g For x=8, 27, 28 α will be printed. For x=17, 18, 37 β will be printed.

16. Write program, which prints α if x is an even number between 0-9, 20-29, 40-49, 60-69, 80-89 or an odd number between 10-19, 30-39, 50-59, 70-79, 90-99. β is printed otherwise. e.g For x=8, 17, 28 α will be printed. For x=7, 18, 27 β will be printed.

Input

Execution of int 33
if AH is 1 : Reads a character and puts its ASCII code in register AL.

```
        Mov     AH,1
        Int     33
        Mov     DL,AL
        Mov     AH,2
        Add     DL,1
        Int     33
        Mov     AH,76
        Int     33
```

The above program reads a character and prints the next character.

```
        Mov     AH,1
        Mov     CL,1
        Mov     DL,0
L2:     Int     33
        Cmp     AL,DL
        JLE     L1
        Mov     DL,AL
L1:     Add     CL,1
        Cmp     CL,10
        JLE     L2
        Mov     AH,2
        Int     33
        Mov     AH,76
        Int     33
```

The above program reads 10 characters and outputs the character with maximum ASCII code.

```
        Mov     AH,1
        Int     33
L1:     Mov     DL,AL
        Int     33
        Cmp     AL,97
        JNE     L1
        Mov     AH,2
        Int     33
        Mov     AH,76
        Int     33
```

The above program reads a sequence of characters till ‘a’ is typed. The program outputs the second last character. The first character of the sequence should not be ‘a’. e.g. input wersa output s

```
        Mov     AH,1
        Mov     CL,1
        Mov     DL,0
L1:     Int     33
        Sub     AL,48
        Add     DL,AL
        Cmp     AL,0
        JNE     L1
        Add     CL,1
        Cmp     CL,5
        JLE     L1
        Mov     AH,2
        Int     33
        STOP
```

The above program reads characters (digits) till five 0’s are given. The program outputs the character whose ASCII code is sum of digits. e.g. input 90990990910920 Output B. Since sum of digits is 67.

CAUTION: Let we intend to type characters as bdakfghjly. Suppose by mistake we type ‘t’ after ‘f’ and correct the mistake by pressing backspace key. Hence sequence of characters typed will be b, d, a, k, f, t, <backspace>, g, h, j, l and y. <Backspace> will not erase ‘t’. It will also be taken as a character (ASCII code 8). Hence first 10 characters shall be taken up to ‘j’ only. ‘l’ and ‘y’ will not be read. Maximum of these characters will be outputted as ‘t’. The output will be produced even before we can type ‘ly’.

1. Read a character and print it 10 times.

2. Read a character and print a character whose ASCII code is half of the input. Input d output 2.

3. Read two characters and print a character whose ASCII code is sum of their ASCII codes. e.g. Input A7 output x, because 65+55=120.

4. Read two characters and print second character first and first character later. e.g. input px output xp.

5. Read 4 characters and print them in reverse order.

6. Read two characters and print all intermediate characters. e.g. input px output pqrstuvwx. Assume ASCII code of first character is more.

7. Read two characters and print the character with bigger ASCII code. e.g. input HA output H. Input AH output H.

8. Read 5 characters and print their next characters. e.g. input a7psa output b8qtb. On monitor it will look as a**b**7**8**p**q**s**t**a**b**. Output is shown in **bold** letters.

9. Read three characters and print the character with middle ASCII code. Assume all characters are different. e.g. for each of the following input sequences the output H will be produced. AHK, AKH, KAH, KHA, HKA, and HAK.

10. Read two characters (digits). Print character whose ASCII code is equal to the number formed by these digits. Input 67 output C. [Hint: Subtract 48 from each of the inputs to find values of the respective digits. Then compute 10*First digit + Second digit].

11. Read a character and print its next character. It is done in a loop till a character ‘a’ (ASCII 97) is typed. e.g. input pe5a output qf6. Input and output may be mixed.

12. Read 9 characters. How many of these are equal to ‘a’. e.g. input catbaqada output 4.

13. Read 10 characters. How many of these are equal to the first character. e.g. input pweprtypqw output 3. Since there are 3 p’s.

14. Read 10 characters and output the characters with maximum and second maximum ASCII codes.

15. Read few characters. The first character is a digit (say x). Read x more characters. Print the character with maximum ASCII code. e.g. input 7psatklc output t. Input 3cpa output p.

16. Read 14 characters (digits) and output character whose ASCII code is equal to their sum. e.g. input 49802978431291 output C, because sum is 67.

17. Read sequence of characters till a character ‘a’ (ASCII 97) is typed. Print the character with maximum ASCII code. e.g. input kpcla output p. Input SYCa output Y.

18. Read a sequence of characters, till a character ‘a’ is typed. Print the third last character. e.g. input qwetyqfga output f. Assume first two characters are not ‘a’.

19. Read a sequence of characters till 5 a’s are given. Find maximum character which is followed by ‘a’. e.g. input dtpacamatyakzca output t, because maximum of c, m, t, and k is t. Assume two consecutive a’s are not present.

20. In above example find maximum character before ‘a’. Assume the first character is not ‘a’. For above case the output will be y, because maximum of p, c, m, y and c is y.

21. Read a sequence of character till 8 a’s are given. The first (and the last) characters must be ‘a’. Find maximum number of characters between two consecutive a’s. e.g. input acpakaatpcamakcaba output 3, because number of characters before 1st, 2nd, 3rd, .. 8th ‘a’ are 0, 2, 1, 0, 3, 1, 2, and 1 respectively.

22. For the above problem print before which ‘a’ maximum number of characters are found. For above problem before 5th ‘a’ maximum number of characters are found.

23. Read characters till 2 consecutive a’s are typed. The program outputs the maximum character. e.g. input efsagtasdaa output t.

24. Read characters till 5 consecutive a’s are typed. The program outputs the maximum character. e.g. input kpaaaambaaqcmaaaaa output q.

25. Read 10 characters. Find how many of these are bigger than the previous character. e.g. input tawerydkhh output 4 since w, r, y and k are bigger than a, e, r and d respectively.

26. Read 20 characters. Find biggest sequence of consecutive a’s. e.g. input aaabaaquaaaaartaawer output 5.

27. Read 20 characters. The program outputs the length of longest sequence of characters in ascending order. e.g. input capydacmtzpqdfmkstab output 5, because the sequences in ascending order are c, apy, d, acmtz, pq, dfm, kst, and ab.

28. In above program which sequence is longest. For above case output will be 4, because 4th sequence has maximum length.

29. Read 8 characters. Each character is 0 or 1. Assume that input is a binary number. Print the character whose ASCII code is that. e.g. input 01001011 output K.

30. In above question assume that input is reverse of a binary number. Input 11010010 output K.

31. Read two characters.
    If input is PS output is
    PQRSQRSRSS10
    PPQQRRQQRRRR12
    PPPQQQQQQ09
    PPPP04

    If input is AE output is as follows
    ABCDEBCDECDEDEE15
    AABBCCDDBBCCDDCCDDDD20
    AAABBBCCCBBBCCCCCC18
    AAAABBBBBBBB12
    AAAAA05

32. Read 9 characters and print the length of sequence displayed. Let the characters typed be lpw**B**rtret, the sequence displayed will be lprtret (length 7). Here we are using **B** as a short form of backspace. If input is w**BBB**asdfg then displayed input will be asdfg (length 5).

Big Numbers

A big number is represented as a pair of integers. Each integer is between 0 and 59. (a,b) means 60*a+b. Addition, subtraction and comparison of big numbers are defined as follows:

(a,b) + (c,d) = (a+c,b+d) if (b+d < 60)
= (a+c+1,b+d-60) otherwise

(a,b) - (c,d) = (a-c,b-d) if (b-d $\geq$0)
= (a-c-1,b-d+60) otherwise

(a,b) > (c,d) = TRUE if (a > c)
= TRUE if (a=c) and (b>d)
= FALSE if (a < c)

Following program computes the product of a big number x by a small number y. x is stored as (BL,CL) and y is in AL. The answer is in (DL,DH). e.g. (2,33)*18=(45,54). Following logic is used:

answer=0;
do{ answer = answer + x ;y=y-1;
} while (y>0);

```
        Mov     BL,a
        Mov     CL,b
        Mov     AL,c
        Mov     DL,0
        Mov     DH,0
L2:     Add     DH,CL
        Add     DL,BL
        Cmp     DH,60
        JL      L1
        Sub     DH,60
        Add     DL,1
L1:     Sub     AL,1
        Cmp     AL,0
        JG      L2
        Mov     AH,2
        Int     33
        Mov     DL,DH
        Int     33
```

The following program prints digits of a big number (x) in reverse order. x is stored as (BH,CH). Following logic is used:

do { p=x div 10; x = x mod 10;
print(x); x=p;
} while (x>0);

p is stored as (BL,CL). Translation of
p = x div 10; x = x mod 10;
is done using following logic:
p=0;
while (x $\geq$10)
{ x=x-10; p=p+1; }

```
    Mov  BH,a
    Mov  CH,b    x=input
L5: Mov  BL,0    do{
    Mov  CL,0    p=0
L4: Cmp  BH,0    while x≥10
    JNE  L1       {
    Cmp  CH,10
    JL   L2
L1: Sub  CH,10      x=x-10
    Cmp  CH,0
    JGE  L3
    Add  CH,60
    Sub  BH,1
L3: Add  CL,1       p=p+1
    Cmp  CL,60
    JL   L6
    Sub  CL,60
    Add  BL,1
L6: JMP  L4       }
L2: Mov  AH,2
    Mov  DL,CH   print(x)
    Add  DL,48
    Int  33
    Mov  BH,BL   x=p
    Mov  CH,CL   }while x>0
    Cmp  BH,0
    JNE  L5
    Cmp  CH,0
    JNE  L5
```

1. Write program to find (x mod y). x and y are big numbers. e.g. (30,41) mod (2,4) = (1,45). Since 1841 mod 124 = 105.

2. Write program to find (x div y). x and y are big numbers, but their division is expected to be a small number. e.g. (30,41) div (2,4) = 14.

3. Write program, which reads 20 characters and stores the sum of their ASCII codes in the form of a big number. e.g. input 22222222222222222233 the stored number is (16,42). Since the sum of ASCII codes is 1002.

4. Write program to find the product of a small number x and a big number y, The following logic should be used. [Caution: See the difference with example program.]
   answer=0; do{ answer = answer + x ; y=y-1; } while (y>0);

5. Write program to find the smallest factor of a big number x. Assume that smallest factor is less than 60. Smallest factor of (21,13) is 19. Since (21,13)=1273=19*67. [Hint: Here you have to find (big number mod small number). The small number will take values 2,3,4, …]

6. Write program to find gcd of two big numbers x and y using following logic:
   while (x ≠ y){ if (x>y) x=x-y; else y=y-x; } print(x);

7. A very big number is represented as (a,b,c). It means 60^2a+60b+c. Write program to perform the addition of two very big numbers. e.g. (27,37,24)+(11,46,19)=(39,23,43).

8. Write program to perform the subtraction of two very big numbers. e.g. (27,37,24)-(11,46,19)=(15,51,5).

9. Write program to multiply a big number and a small number. The answer may be a very big number. e.g. (30,41)*52=(26,35,32) since 1841*52=95732.

10. Write program to multiply two big numbers. The answer may be a very big number (but not very very big number) e.g. (3,21)*(3,20)=(11,10,0).

11. Write program to print the digits of a very big number (in reverse order). e.g. (36,35,32) will output 237131. It is because (36,35,32)=131732.

12. Write program, which will read 6 digits and store the number formed by them in the form (a,b,c). e.g. 131732 will be stored as (36,35,32).

13. Write program, which reads three characters and outputs three characters. Let x, y and z be ASCII codes of input and p, q and r that of output characters. Assume an hour has x minutes and a minutes has y seconds. Convert 100z seconds into hours, minutes and seconds. i.e. Find p, q and r such that pxy+qy+r=100z and q<x, r<y. e.g. if x=2 y=76 z=101 then p=66 q=0 r=68.

Macro

```
.model small
.code
   Mov  AH,1
   Mov  DL,0
   Int  33
   Cmp  DL,AL
   JGE  L1
   Mov  DL,AL
 L1:
   Int  33
   Cmp  DL,AL
   JGE  L2
   Mov  DL,AL
 L2:
   Int  33
   Cmp  DL,AL
   JGE  L3
   Mov  DL,AL
 L3:
   Int  33
   Cmp  DL,AL
   JGE  L4
   Mov  DL,AL
 L4:

   Mov  Ah,2
   Int  33
   Mov  Ah,76
   Int  33
   End
```

The above program reads 4 characters and outputs the character with maximum ASCII code. Here following lines are repeated four times (with the difference of label L).

```
   Int  33
   Cmp  DL,AL
   JGE  L
   Mov  DL,AL
 L:
```

To save the typing effort we define them as macro with parameter L. Before the assembly language program is converted into machine code the Macro expansion takes place and the previous program is generated. This can be seen by looking at a.lst file (We assume a.asm is the program file).

```
max  macro  L
   Int  33
   Cmp  DL,AL
   JGE  L
   Mov  DL,AL
 L:
endm

.model small
.code
   Mov  AH,1
   Mov  DL,0
   max  L1
   max  L2
   max  L3
   max  L4
   Mov  Ah,2
   Int  33
   Mov  Ah,76
   Int  33
   End
```

```
newline macro
   Mov  DL,10
   Int  33
   Mov  DL,13
   Int  33
endm

pqr  macro
  Local  L
    Mov  CL,1
  L:Int  33
    Add  DL,1
    ADD  CL,1
    Cmp  CL,5
    JLE  L
endm

lmn  macro  a1,a2,a3
    Mov  DL,a1
    pqr
    Mov  DL,a2
    pqr
    Mov  DL,a3
    Pqr
    newline
 endm

.model small
.code
   Mov  AH,2
   lmn  97,'P',48
   lmn  'l','5','C'
   Mov  AH,76
   Int  33
   End
```

The above program prints
abcdePQRST01234
lmnop56789CDEFG

```
abc  macro n
   if n EQ 0
     Mov  Ah,2
     Mov  DL,65
   else
     abc  n-1
     Int  33
     Add  DL,1
   endif
endm
.model small
.code
   abc 6
   Mov  AH,76
   Int  33
   End
```

The above program prints ABCDEF
The above was recursive macro. GE (≥), LE(≤), GT(>), LT(<) and NE(≠) can also be used.

```
.model small
.code
    Mov  Ah,2
    n=1
    rept 5
       m=1
       rept 7-n
          Mov  DL,64+m
          Int  33
          m=m+1
        endm
        Mov  DL,10
        Int  33
        Mov  DL,13
        Int  33
        n=n+1
    endm
    Mov  ah,76
    Int  33
    End
```

The above program prints
ABCDEF,ABCDE,ABCD,ABC,AB,

```
ttt macro k
   Mov  DL,k
   Int  33
endm
.model small
.code
    Mov  Ah,2
    n=0
    rept 3
        ttt 'a'+n
        ttt 'P'+n
        ttt '0'+n
        n=n+1
    endm
    Mov  ah,76
    Int  33
    end
```

The above program prints
aP0bQ1cR2

```
ppp macro t
   ttt t+65
   ttt t+48
   ttt t+'a'
endm
.model small
.code
    Mov  Ah,2
    ppp 0
    ppp 2
    ppp 1
    ppp 7
    Mov  AH,76
    Int 33
End
```

The above program prints
A0aC2cB1bH7h

```
.model small
.code
    Mov  Ah,2
    n=0
    rept 2
     ppp 0+n
     ppp 2+n
     ppp 1+n
     ppp 7+n
    newline
    n=n+1
    endm
    Mov  AH,76
    Int 33
End
```

The above program prints
A0aC2cB1bH7h
B1bD3dC2cI8i

All of the following problems should be done without using loop (i.e. backward jump).

1. Picture type RR
   ABCDEF
   CDEFGH
   FGHIJK
   JKLMNO

2. Picture type Rr
   ABCDEFGH
   PQRSTUVW
   01234567
   bcdefghi

3. Picture type rR
   AP1sQa
   BQ2tRb
   CR3uSc

4. Picture type rr_1
   ADBEJ
   EHFIN
   CFDGL
   BECFK
   DGEHM

5. Picture type RRR
ABCDE,BCDEF,CDEFG,
CDEFG,DEFGH,EFGHI,
FGHIJ,GHIJK,HIJKL,
JKLMN,KLMNO,LMNOP,

6. Picture type RRr
ABCDEF,BCDEFG,CDEFGH,
IJKLMN,JKLMNO,KLMNOP,
BCDEFG,CDEFGH,DEFGHI,
PQRSTU,QRSTUV,RSTUVW,

7. Picture type RrR
ABCD,PQRS,1234,bcde,
BCDE,QRST,2345,cdef,
CDEF,RSTU,3456,defg,
DEFG,STUV,4567,efgh,

8. Picture type Rrr_2
ABCD,DEFG,BCDE,EFGH,
EFGH,HIJK,FGHI,IJKL,
CDEF,FGHI,DEFG,GHIJ,
BCDE,EFGH,CDEF,FGHI,

9. Picture type Rrr
ABCD,DEFG,BCDE,EFGH,
EFGH,KLMN,IJKL,LMNO,
CDEF,DEFG,FGHI,GHIJ,
BCDE,GHIJ,ABCD,CDEF,

10. Picture type rRR
AP1B,BQ2C,CR3D,DS4E,
CR3D,DS4E,ET5F,FU6G,
FU6G,GV7H,HW8I,IX9J,

11. Picture type rRr
ADBE,BECF,CFDG,DGEH,
EHFI,FIGJ,GJHK,HKIL,
CFDG,DGEH,EHFI,FIGJ,
BECF,CFDG,DGEH,EHFI,
DGEH,EHFI,FIGJ,GJHK ,

12. Picture type rRr_1, rrR, rr_1R and various pictures types in rrr.

13. Read a character and print it 10 times.

14. Read sequence of characters till a character ‘a’ (ASCII 97) is typed. Print the character with maximum ASCII code. e.g. input kclpdla output p. Input SYCa output Y. You may assume that ‘a’ will be typed within first 10 characters. [Hint: you may define a macro of following type and expand it 10 times. Caution: Among L1 and L2 which is local]

```
  Int    33
  Cmp    AL,97
  JE     L1
  Cmp    AL,DL
  JLE    L2
  Mov    DL,AL
L2:
```

15. Read two characters and print all intermediate characters. e.g. input px output pqrstuvwx. Assume ASCII code of first character is more and difference in ASCII codes of given characters is less than 10.

16. Read 9 characters and print the length of sequence displayed. Let the characters typed be lpw**B**rtret, the sequence displayed will be lprtret (length 7). Here we are using **B** as a short form of backspace (ASCII 8). If input is w**BBB**asdfg then displayed input will be asdfg (length 5).

17. Read 12 characters and print a character whose ASCII code is equal to the sum of digits. In calculating the sum of digits, those digits are not added, which are more than their positions e.g. input

513273948999 output 0. Here sum is 1+3+2+3+4+8+9+9+9=48. Here 5, 7 and 9 are not added because their positions are 1, 5 and 7 respectively, which are less than respective digits.

Function

```
.model small
.code
xyz     proc
        Mov     DL,0
   L2:  Cmp     BL,CL
        JG      L1
        Add     DL,BL
        Add     BL,1
        JMP     L2
   L1:  RET
xyz     endp
main    proc
        Mov     BH,d1
        Mov     CH,d2
        Mov     DH,d3
        Mov     BL,BH
        Mov     CL,CH
        CALL    xyz
        Mov     AL,DL
        Mov     BL,CH
        Mov     CL,DH
        CALL    xyz
        Sub     DL,AL
        Mov     AH,2
        Int     33
        Mov     AH,76
        Int     33
main    endp
end     main
```

The above program computes the value of $\sum_{i=d2}^{i=d3} i - \sum_{i=d1}^{i=d2} i$

It has been achieved by defining the function DL= $\sum_{i=BL}^{i=CL} i$

CALL L1 means JMP L1 However the address of the location from where the call is performed is saved. So that it can be used by RET.
RET means jump to the location from where call was made.

1. Define a function CL=BL*BH. Using it write program, which reads 4 digits and prints the character, whose ASCII code is the value of following: $d_1+3d_2+5d_3+4d_4$. e.g. input 8357 output F.

2. Define a function to find CL=AL div AH and CH=AL mod AH. Using it write program, which reads a character and prints its ASCII code. e.g. input Y output 089 input x output 120. Since only 3 characters are outputted, hence do not use any loop (in main program).

3. Give the shape of a function to the above program. Using it write program which reads 5 characters and prints the ASCII codes of biggest, smallest, second biggest and second smallest characters. e.g. input CATPK output 84 65 80 67.

4. Using the function mod (defined in one of the above questions) define a function CL= smallest factor of CH. Using it write program which reads 9 characters and prints how many of these are prime.

5. Define a function xyz (using Int 33 not more than six times) so that following main program outputs the given picture.
   ADBEJ,EHFIN,BECFK,
   Mov CL,'A'
   Call xyz
   Mov CL,'E'
   Call xyz
   Mov CL,'B'
   Call xyz

```
.model small
.code
main    proc
        Mov     AL,x
        CALL    f
        Output  DL
        Stop
main    endp
f       proc
        Cmp     AL,3
        JG      L1
        Mov     DL,1
        RET
   L1:  Push    AX
        Sub     AL,1
        CALL    f
        Pop     AX
        Mov     BL,DL
        Push    AX
        Push    BX
        Sub     AL,2
        Call    f
        Pop     BX
        Pop     AX
        Add     BL,DL
        Sub     AL,3
        Push    BX
        Call    f
        Pop     BX
        Add     DL,BL
        RET
f       endp
end     main
```

The above function finds DL=f(AL). Here f(x) is defined as

f(x) = 1 if $x \leq 3$

f(x) = f(x-1)+f(x-2)+f(x-3) otherwise

PUSH AX saves the values of registers AH and AL on stack.

POP AX restores the values of AH and AL.

Push and Pop should be used in opposite order. e.g. following should not be used.

PUSH AX
PUSH BX
POP AX
POP BX

1. Define a recursive function f(BL,BH). It prints all characters whose ASCII codes are between BL and BH. e.g. f(67,72) will print CDEFGH. Use following logic:
   f(x,y)=print(x);f(x+1,y) if $x \leq y$

2. Define a recursive function CL= AL div AH by using following logic
   CL = 0 if AL < AH
   =[(AL-AH)div AH]+1 otherwise

3. Define a recursive function CL= AL*AH. Use following logic:
   CL = 0 if AH=0
   =[(AL*(AH-1)]+AL otherwise

4. Define a recursive function to find CL=gcd(BL,BH).
   CL=BL if BH=0
   =gcd(BH, BL mod BH) else

5. Define a recursive function to find sum of digits (in base 3) of contents of AL e.g. AL=103 output 5. Since $103=(10211)_3$. To define this function, take the help of recursive function mod.

6. Define ${}^nC_r$ by using following:
   ${}^nC_r=1$ if n=r or r=0
   $={}^{n-1}C_{r-1} + {}^{n-1}C_r$ else

Shift and Rotate

```
        Mov     AH,1
        Int     33
        Mov     BL,AL
        Mov     AH,2
        Mov     BH,0
L3:     SHL     BL,1
        JC      L1
        Mov     DL,48
        JMP     L2
L1:     Mov     DL,49
L2:     Int     33
        Add     BH,1
        Cmp     BH,8
        JL      L3
        STOP
```

The above program reads a number and prints the binary (base 2) representation of ASCII code. Every number is stored in a register in 8 bits, each bit is 0 or 1. e.g. 70 is represented as 01000110.

Let in register BL a number is stored whose binary representation is abcdefgh. Let us see how following shift and rotate operations will modify BL.

Shift left: SHL BL,1 will make BL as bcdefgh0. The carry flag will have 'a'. It means "Shift the contents of register BL by 1 bit in left direction". When we want to shift by more than one then count is stored in CL. e.g. SHL BL,5 does not work on most of the systems. It can be achieved by

Mov CL,5
SHL BL,CL

The above operation will make BL as fgh00000. 'e' will go in carry flag.

Rotate Left: ROL BL,1 will make BL as bcdefgha. The carry flag will have 'a'.

Rotate Left with Carry: RCL BL,1 will make BL as bcdefghx. The carry flag will have 'a'. 'x' was the contents of the carry, which have now moved into the register as the least significant bit (LSB).

Following instructions are used for right shift and rotate: SHR, ROR, RCR

1. Write program to print base 4 representation of a number. Logic: Shift the number twice
   Print3 if both times carry.
   Print0 if both times no carry.
   Print1 if only second times carry.
   Print2 if only first time carry.

2. Write program to print the base 2 representation using following logic: Let the number is in BL. SHL BL,1 will bring most significant bit (MSB) in Carry. The carry can be taken into DL by RCL DL,1(Rotate left with carry). Now contents of DL are added by adding 48.

3. Use above logic to print base 4 representation.

4. Write program to print the base 2 representation using following logic: Obtain k^{th} digit by shifting left k-1 times and then shifting right 7 times.

5. Use above logic to print base 4 representation.

6. Write a program, which reads a character and outputs another character. The binary representation of ASCII code of the output character is reverse of the binary representation of ASCII code of the input character. e.g. input F output b.
   Since F=70=$(01000110)_2$
   Since b=98=$(01100010)_2$

7. Write a program, which reads a character and outputs another character. The base 4 representation of ASCII code of the output character is reverse of the base 4 representation of ASCII code of the input character. e.g. input m output y.
   Since m=109=$(1231)_4$
   Since y =121=$(1321)_4$

8. Write program, which reads 8 digits. Each digit is 0 or 1. The program outputs the character, whose binary representation of ASCII code is as given. Use following logic: Read a digit and put it into carry flag. (The carry flag can be made 1 by STC and 0 be CLC instruction). Now bring the contents of the carry flag into the register by "rotate with carry".

9. Do above question be using following logic: Answer is the sum of positional values of all digits. The positional value of kth digit is 0 if the digit is 0. The positional value is 2^{8-k} if the digit is 1. [Hint: BL=BL*2^i can be done by using SHL BL,i].

10. Do the above problems for base 4 representation using both of the above logics.

11. Write program to find z=x*y. Logic:
```
ans=0;
for (i=1; i≤8; i++)
{     if (x mod 2 ≠ 0)
            ans=ans+y;
       x=x div 2;
       y=y*2;
}
```
    SHL does multiplication by 2.
    SHR does div 2. If mod 2 is 1 then carry is 1.

12. Write program, which reads a character and outputs another character. Following relationship holds between input and output characters. Let abcdefgh be the binary representation of input character then acegbdfh will be the binary representation of output character. Input c output Y.
    Since c=99=$(01100011)_2$
    and Y=89=$(01011001)_2$

13. Do above question when output is efghabcd. Input F output d.
    Since F=70=$(0100\ 0110)_2$
    And d=100=$(0110\ 0100)_2$

14. Do above question when output is badcfehg. Input 6 output 9.
    Since 6=54=$(00\ 11\ 01\ 10)_2$
    And 9=57=$(00\ 11\ 10\ 01)_2$

Float representation

Let a float be represented in 8 bits abcdefgh. Its value is 1d.efgh x 2^{abc}.

11. Write a program, which reads a character (X) and outputs another character (Y). ASCII code of Y is float representation of ASCII code of X. e.g. input K output Ñ (ASCII 165).
    K = 75 = $(01001011)_2$ = 10.0101 x 2^{101} = 101 00101 = 165

12. Write a program, which reads character Y and outputs X. e.g. input j output § (ASCII 21).
    j = 106 = $(011\ 01010)_2$ = 10.1010 x 2^{011} = 10101 = 21

13. Write program, which reads two characters Y and Z. The program outputs a character W. Y and Z are representations of two floats. W is a representation of a float, which is sum of Y and Z. e.g. input h and I output '|'(ASCII 124).
    h= 104 = $(011\ 01000)_2$ = 10.1000 x 2^{011} = 10.1000 x 2^{011}
    I= 73 = $(010\ 01001)_2$ = 10.1001 x 2^{010} = 1.0100 x 2^{011}
    11.1100 x 2^{011} = $(011\ 11100)_2$= 124

    Care is required for inputs h and R. Output Ç (ASCII 128)
    h= 104 = $(011\ 01000)_2$ = 10.1000 x 2^{011} = 10.1000 x 2^{011}
    R= 82 = $(010\ 10010)_2$ = 11.0010 x 2^{010} = 1.1001 x 2^{011}
    100.0001 x 2^{011}
    10.0000 x 2^{100} = $(100\ 00000)_2$= 128

14. Do above problem when W=Y-Z. (Assume Y>Z) e.g. input h and I output 'H'(ASCII 72).
    h= 104 = $(011\ 01000)_2$ = 10.1000 x 2^{011} = 10.1000 x 2^{011}
    I= 73 = $(010\ 01001)_2$ = 10.1001 x 2^{010} = 1.0100 x 2^{011}
    01.0100 x 2^{011}
    10.1000 x 2^{010} = $(010\ 01000)_2$ = 72

15. Do above problem when W=Y*Z. e.g. input h and I output π (ASCII 210).
    h= 104 = $(011\ 01000)_2$ = 10.1000 x 2^{011}
    H= 72 = $(010\ 01000)_2$ = 10.1000 x 2^{010}
    Product = 11.0010 x 2^{110} = $(110\ 10010)_2$ = 210

Let a float be represented in 8 bits abcdefgh. Its value is 1d.efgh x 2^{abc} if a, b and c are not all zero. If all three are zero then value is de.fgh

16. Write program, which reads two characters Y and Z. The program outputs a character W. Y and Z are representations of two floats. W is a representation of a float, which is sum of Y and Z. e.g. input h and a character with ASCII code 25 output ‘n’(ASCII 110).

$h = 104 = (011\ 01000)_2 = 10.1000 \times 2^{011} = 10.1000 \times 2^{011}$

$25 = (000\ 11001)_2 = 11.001 = .0110 \times 2^{011}$

$10.1110 \times 2^{011} = (011\ 01110)_2 = 110$

input z and a character with ASCII code 25 output Ç (ASCII 128).

$z = 122 = (011\ 11010)_2 = 11.1010 \times 2^{011} = 11.1010 \times 2^{011}$

$25 = (000\ 11001)_2 = 11.001 = .0110 \times 2^{011}$

100.0000×2^{011}

$10.0000 \times 2^{100} = (100\ 00000)_2 = 128$

input two characters with ASCII code 25 output 2 (ASCII 50).

$25 = (000\ 11001)_2 = 11.001$

$25 = (000\ 11001)_2 = 11.001$

$110.010 = 11.0010 \times 2^{001} = (001\ 10010)_2 = 50$

input two characters with ASCII code 15 output ▲ (ASCII 30).

$15 = (000\ 01111)_2 = 01.111$

$15 = (000\ 01111)_2 = 01.111$

$11.110 = (000\ 11110)_2 = 30$

Word Size registers

Here we shall consider four word registers. AX, BX, CX, and DX. Each of these can hold 16 bit data (or a number between 0 and 65535 in base 10). Let the contents of AX be abcdefghijklmnop. First 8 bits are AH and last 8 bits are AL. Hence AX can be thought as 256*AH+AL. Alternatively
AH=AX div 256 and AL=AX mod 256

```
        Mov     AH,1
        Int     33
        Mov     AH,0
        Mov     CX,0
        Mov     BL,100
L1:     Add     CX,AX
        Sub     BL,1
        Cmp     BL,0
        JG      11
        Mov     AH,2
        Mov     DL,CH
        Int     33
        Mov     DL,CL
        Int     33
        STOP
```

The above program reads a character and finds 100 times of its ASCII code. The result is in CX. The program prints the characters whose ASCII codes are in CH and CL respectively. Hence the answer is in the form of 2 characters (say) $\alpha\beta$. Where α=Ans div 256 and β=Ans mod 256. Input g output (<. Since g=103 hence answer 10300. Hence contents of CH and CL are 40[(] and 60[<] respectively.

In Mov, Add, Sub, Cmp etc. instructions either both registers should be word size or byte size. e.g. Add AX,BL is not allowed. However it can be done by one of the following ways:

```
      Mov    BH,0
      Add    AX,BX
```

or

```
      Add    AL,BL
      Adc    AH,0
```

If AL+BL $\geq$ 256 then Carry flag is set.

ADC AH, α $\equiv$ AH=AH+ α +1.
If carry flag is set then
Otherwise AH=AH+ α .

```
        Mov     AH,1
        Int     33
        Mov     DX,20000
        Mov     AH,0
L1:     Sub     DX,AX
        Cmp     DX,AX
        JGE     L1
        Mov     AH,2
        Int     33
        STOP
```

The above program reads a character and finds 20000 mod (ASCII code of input). Alternatively the program can be written as following.

```
        Mov     AH,1
        Int     33
        Mov     DX,20000
L1:     Sub     DL,AL
        Sbb     DH,0
        Cmp     DH,0
        JA      L1
        Cmp     DL,AL
        JAE     L1
        Mov     AH,2
        Int     33
        StOP
```

Here JA is used in place if JG. Similarly JB can be used in place of JL. These work for unsigned comparison.

1. Write program, which reads a character and finds (20000 div ASCII code of input). Assume that answer is less then 256.

2. Write program, which reads two characters. Let their ASCII codes be X and Y respectively. The program prints α if 100X>57Y. β is printed otherwise. e.g. inputs AZ output α. Inputs 2d output β.

3. Read 10 characters and print the character whose ASCII code is the average of their ASCII codes. Input dZAdZAFddZ output W (ASCII 87). Here the sum of ASCII codes of input characters is 870.

4. Read two characters and find the product of their ASCII codes. Let the answer is in CX. Print CH and CL. e.g. input kx output 2(. Since k=107 x=120 Result is 12840. It can be written as 50*256+40. ASCII codes of 2 and (are 50 and 40 respectively.

5. Write program, which reads two characters. Let their ASCII codes be X and Y respectively. The program finds $\sum_{i=X}^{i=Y} i$.

6. Write a program, which reads five digits and stores the value of the number in CX. e.g. if input is 12897 and CH and CL are printed then the output will be 2a. Since the contents of CX will be 50*256+97=12897.

7. Read two characters and store them in CH and CL respectively. Print the character whose ASCII code is CX mod 120. If the inputs are 2a then the output is 9. Since 12897 mod 120 is 57.

8. Read two characters and store them in CH and CL respectively. Print the contents of CX (in base 10) in reverse order. For above example the output will be 79821. [Caution: CX div 10 will be more than 256, hence a word size register may be required to store it].

9. In above question find the smallest factor of CX.

Problems on SHIFT and ROTATE

10. Read a character and print binary representation of ASCII code. Use following logic: Let the character is in BL. Perform SHL BX,1. The most significant bit (MSB) of BL will become the least significant bit(LSB) of BH.

11. Read 8 digits. Each digit is 0 or 1. Print the character, whose ASCII code is that.
    Logic: Let the digit is in BL.
    Perform SHL BL,7 and SHL BX,1.

12. Use both of the above logic to print base 4 representation.

13. Print base 16 representation of contents of BX. Logic: Let initially DL=0. Perform SHL BX,1. If there is carry then add 8 to DL (otherwise nothing). Similarly perform SHL BX,1 three more times and add 4 (or nothing), 2(or nothing) and 1 (or nothing) respectively to DL. The above method will give one digit of base 16. To add 8,4,2,1 put 8 in DH. Every time make the value of DH half by SHR instruction.

14. Write program, which reads two octal digits (0..7). The program outputs the character, whose ASCII code is the number formed by those digits. e.g. input 65 output 7 (since $(65)_8$ is 53). Input 53 output + (since $(53)_8$ is 43). Do not use any instruction other than Mov, Int, and Shl.

MUL and DIV

MUL BL ≡ AX =AL*BL
MUL BX ≡ DX:AX=AX*BX
DIV BL ≡ AL=AX div BL
and AH=AX mod BL
DIV BX ≡ AX=DX:AX div BX
and DX=DX:AX mod BX

In place of BL any byte size register and in place of BX any word size register can be used.

Mov BL,100
Mov AL,150
MUL BL
AX=150*100=15000

Mov BX,300
Mov AX,500
MUL BX
DX:AX=300*500=150000
Hence DX=150000 div 65536 = 2
and AX=150000 mod 65536 = 18928

Mov AX,10000
Mov BL,166
DIV BL
AH=10000 mod 166 = 40
AL=10000 div 166 = 60

Mov AX,60000
Mov BL,166
DIV BL
Since 60000 div 166 is more than 256 hence overflow error will result. The equivalent effect can be achieved by following.
Mov DX,0
Mov AX,60000
Mov BX,166
DIV BX
DX=60000 mod 166 = 76
AX=60000 div 166 = 361

Mov DX,2
Mov AX,10000
Mov BX,1000
DIV BX
DX:AX=2*65536+10000=141072
DX=141072 mod 1000 = 72
AX=141072 div 1000 = 141

The following program reads a character and prints the smallest factor of ASCII code.

```
        Mov     AH,1
        Int     33
        Mov     BL,2
        Mov     AH,0
        ;AX=AL
        Mov     CX,AX
        ;retain a copy of input
L1:     DIV     BL
        ;AL=AX  div  BL
        ;AH=AX  mod  BL
        Cmp     AH,0
        JE      L4
        Mov     AX,CX
        Add     BL,1
        JMP     L1
L4:     Mov     DL,BL
        Mov     AH,2
        Int     33
        Stop
```

1. Write program, which reads a character and prints its ASCII code. Input Z output 90.

2. Write program, which reads 5 digits and prints the character whose ASCII code is equal to their product. e.g. input 21534 output x because 2*1*5*3*4=120. Assume that the answer is less than 256.

3. Write program, which reads a character and prints base 3 representation of ASCII code. Use following logic: (Let ASCII code of input be x)
   while ($x \neq 0$) {print(x mod 3); x=x div 3} You may print the reverse of representation
   e.g. input q output 21011 since q=113 and base 3 representation of 113 is 11012.

4. Write program to find $\sum_{i=1}^{i=x} i*(i+1)$. For x=5 it is 1*2+2*3+3*4+4*5+5*6=70. Assume that the answer is less than 256.

5. Write program, which reads 6 digits d_1, d_2, d_3, d_4, d_5, and d_6. The program outputs the character, whose ASCII code is $d_1*d_2+ d_2*d_3+ d_3*d_4+ d_4*d_5+ d_5*d_6$. e.g. input 215348 output B since 2*1+1*5+5*3+3*4+4*8=66. Assume that the answer is less than 256.

6. Write program, which reads two digits x and y and outputs the character whose ASCII code is x^y . e.g. input 35 output $\leq$ (ASCII 243). Assume x^y is less than 256.

7. Write program, which reads two characters. Let their ASCII codes be X and Y. The program outputs the value of approximate value of $\sqrt{X*Y}$. e.g. input Fd output S Since $\sqrt{70*100}$ = 83.

8. Write program, which reads a character and finds the sum of factors of the ASCII code of the input character. Assume that the answer is less than 256.

9. Write program, which reads a character and tests whether its ASCII code is prime or not. Output will be T of F.

10. Write program, which reads a character and finds the sum of prime factors of ASCII code. Assume that the answer is less than 256.e.g. if input is H then sum is 2+2+2+3+3=12. Since $72=2^3.3^2$. Use logic:
    Read(x); S=0; y=2; while (x>1) { if (x mod y = 0) { S=S+y; x=x div y} else y=y+1 }

11. Write program, which reads two characters. Let their ASCII codes be x and y. The program outputs the GCD of x and y. Use following logic:
    do { r=x mod y; x=y; y=r; } while (r!=0); print(x);

12. Write program to find gcd of x and y by using following logic:
    for (i=x; ((x%i!=0)||(y%i!=0)) ;i--); print(i);

13. Write program, which reads two characters and stores them in BH and BL. The program prints the value of BX. Digits may be printed in reverse order. e.g. input dA

BX=100*256+65=25665. Hence output is 56652. [Caution: 25665 div 10 will cause overflow hence store 25665 in DX:AX and 10 in a word size register.]

14. Write program, which reads 5 digits. Let x be the number formed by the digit. The program outputs ASCII of (x mod 88). e.g. input 54361 output A (since 54361 mod 88 is 65).

15. Write program, which reads few digits till a non-digit character is given. Let x be the number formed by the digits (taken in opposite order). The program outputs ASCII of (x mod 88). Assume that there are less than six digits. e.g. input 16345E output A (since 54361 mod 88 is 65). Input 7165E output I (since 5617 mod 88 is 73).

Array

```
.model small
.data
   a     db  100 dup(?)
.code
         Mov      AX,Dgroup
         Mov      DS,AX
         Lea      BX,a
         Mov      SI,0
         Mov      AH,1
L1:      Int      33
         Mov      [BX+SI],al
         Add      SI,1
         Cmp      SI,5
         JL       L1
         Mov      AH,2
         Mov      CL,1
         Lea      SI,a
L2:      Mov      DL,[SI+4]
         Int      33
         Sub      SI,1
         Inc      CL
         Cmp      CL,5
         JLE      L2
         Mov      AH,76
         Int      33
         END
```

The above program reads 5 characters and prints them in reverse order.

In square brackets any of the followings can be used: BX, BX+SI, BX+DI, SI, DI, BX+n, BX+SI+n, BX+DI+n, SI+n, and DI+n. Here 'n' is a positive or negative number.

Let us understand the difference between Mov CX,BX and Mov CX,[BX]. If contents of BX are 1000 then the first instruction will make the contents of CX as 1000. The second instruction will put the contents of memory location 1000 into CX.

Here SI and DI are word size registers. Hence Mov SI,CL is invalid. However it can be done by following sequence of instructions: Mov CH,0 Mov SI,CX

In a single instruction two memory references can not be made. Hence Mov [BX],[SI] is invalid. However it can be done as:
Mov DH,[SI] Mov [BX],DH
Mov CL,[BX] means the contents of the memory location pointed by BX are put into CL. Mov CX,[BX] means that the contents of memory location pointed by BX are put into CL and pointed by BX+1 are put into CH. Hence it is equivalent to
Mov CL,[BX] Mov CH,[BX+1]

Size plays a very important role when immediate data transfer is done into memory. Mov [BX],80 is confusing. It does not state whether a word or a byte is to be transferred into memory. If we want to transfer a word, then we should write Mov [BX], byte ptr 80. If we want to transfer a word, then we should write Mov [BX], word ptr 80. In this case the contents of memory location pointed by BX+1 will become 0. The instruction Mov [BX], word ptr 1000 will make the contents of memory location pointed by BX as 232 and that of pointed by BX+1 as 3. It is because 1000=3*256+232.

It should be noted Mov CL,BX is illegal however Mov CL,[BX] is valid. It is because in first case an attempt is made to transfer the contents of a word size register BX into a byte size register CL. In second case attempt is made to transfer the contents of a memory byte (whose address is in BX) into CL.

1. Write program, which reads 5 characters. Let these be qwert. The program outputs qwertwertertrtt.

2. qwert
   wert
   ert
   rt
   t

3. qwertqwerqweqwq
   wertwerwew
   ertere
   rtr
   t

4. qwertwertertrtt
   qwerwererr
   qwewee
   qww
   q

5. qwertqqwweerrqqqwwweee

6. Write program, which reads 13 characters. Let these be qptanbdkxegsw The program outputs qpta,qtnd,qade,qnxw,

7. qnxw,pnkg,tndx,anbd,

8. Write program, which reads 10 characters and finds how many of these are capital letters. (ASCII code between 65 and 90). e.g. input Dyw-H#qwLM output 4.

9. Write program, which reads 10 characters and outputs, how many of these are bigger than the past character. e.g. input fadkbczywx output 5. It is because d, k, c, z and x are bigger than the past character, which are a, d, b, c and w respectively.

10. Write program, which reads names and marks of 10 students. In input names and marks of all students are given in the same line. There is a blank between name and marks of a student. After the marks of a student comma is put. In the output there are 10 lines. Every line contains the name and mark of a single student.

11. Write program, which reads 10 characters and prints them in increasing order of ASCII code.

12. Write program, which reads 17 characters. The first 16 characters are in increasing order of ASCII code. The program finds, whether the last character is present in the sequence of 16 characters or not. Use binary search. The program should not use more than 4 comparisons.

13. Write program, which inserts the last character in the proper place in the above sequence of 16 characters.

14. Write program, which reads 5 digits of a number. The program outputs the double of the number. e.g. input 61284 output 122568.

15. Write program, which reads 8 characters. Seven of these characters are digits and one of these is '.' (ASCII 46). The program outputs the double of the number. e.g. input 23.65218 output 47.30436

16. Write program, which reads 10 characters and rearranges them in such a manner so that characters with ASCII code less than or equal to 90 are in the beginning and others are at the end. Use following logic:

```
i=1;   j=10;
While (i<j)
{    if (a[i]≤90) i++;
     else (if a[j]>90) j--;
          else   a[i]↔a[j]
}
```

17. Write program, which reads 10 characters and print them in reverse order. There may be backspace between characters. Let the characters typed be lpw**B**rtresab, the output will be basertrpl. Here we are using **B** as a short form of backspace. If input is w**BBB**asd**BB**fgbjhklpt then output will be tplkhjbgfa.

18. Write program, which reads 20 characters. First 10 characters are in ascending order of ASCII codes and last 10 are also. Output all characters in the ascending of ASCII code. Do not use nested loop. Use the logic of merge sort.

19. Read 12 characters from the set {A..Z,-,*}. Write program, which maintains a queue. A..Z means put that element in the queue. '-' means take a element from the queue. '*' means display the queue. e.g. input PQKI--NC-*-* output (P)(Q)(K)(INC)(I)(NC) The output need not come at the end. It may be displayed immediately after '-' or '*'. The above will appear as
PQKI-(P)-(Q)NC-(K)*(INC)-(I)*(NC)

20. Modify the above program to implement stack. For above input the output (I)(K)(C)(PQN)(N)(PQ)

21. Write program, which reads a sequence of character till a character '%' (ASCII 37) is typed. There are few blank between characters. The program outputs the first word. e.g input ram kumar prasad dey% output ram. [Caution: There may be blanks before the first word and after the last word, There may be multiple blanks between consecutive words.]

22. The program outputs the second word.

23. The program outputs the last word.

24. The program outputs the number of words.

25. The program outputs the short form. For above input rkpd.

26. The program outputs the short name. For above input rkpdey

27. The program outputs the short name as r.k.p.dey

28. The program outputs opposite as: dey prasad kumar ram

29. The program outputs word wise opposite: mar ramuk dasarp yed

Matrix

```
.model small
.data
    mat  db  2,2,9,6,1,3,2,4,6,1,2,3
    nr   db  3
    nc   db  4
  ;The above makes matrix
  ;      2  2  9  6
  ;      1  3  2  4
  ;      6  1  2  3
.code
        Mov     AX,Dgroup
        Mov     DS,AX
        Mov     AH,1
        Int     33
        Mov     BL,AL
        Sub     BL,48
        Int     33
        Sub     AL,48
;BL=row number x
;AL=Col number y
        Lea     SI,mat
        Mov     CL,nc
        Mov     CH,0
L2:     Cmp     BL,1
        JE      L1
        Add     SI,CX
        Dec     BL
        JG      L2
;SI has  address of element
in Xth row and 1st column
L1:     Mov     AH,0
        Add     SI,AX
        Dec     SI

        Mov     AH,2
        Mov     DL,[SI]
        Add     DL,48
        Int     33
        Mov     AH,76
        Int     33
        END
```

The above program finds the element in x^{th} row and y^{th} column.

1. Write program to print x^{th} row
   Input 2 Output 1 3 2 4

2. Write program to print x^{th} column
   Input 2 Output 2 3 1

3. Write program to print matrix.

4. Write program to print transpose.

5. Write program to print maximum element of xth row Input 2 Output 4

6. Write program to print maximum element of xth column.

7. Assume that the given matrix is a square matrix. Write program to print the diagonal elements. e.g.

   Matrix 5 7 9 4
           2 4 7 3
           1 5 9 2
           3 7 4 8

   Output 5 4 9 8

8. Write program to print the element of another diagonal. For above matrix the output will be 4 7 5 3

9. Write program to find MIN-MAX_ROW. Find maximum element of every row and find minimum of those. For the matrix given in the example the output is 4. Here maximum elements of rows are 9, 4 and 6 respectively.

10. Write program to find MIN-MAX_COLUMN. Find maximum element of every column and find minimum of those. For given matrix output 3 (since col max are 6,3,9,6)

The following 3 dimensional matrix

Z=1	Z=2	Z=3	Z=4
2 3 4	4 5 2	2 4 7	2 7 6
5 7 3	3 5 2	3 9 4	2 7 3

will be stored as follows.

```
.data
    mat  db   2,3,4,5,7,3,4,5,2,3,5,2,2,4,7,3,9,4,2,7,6,2,7,3
    nx   db   3
    ny   db   2
    nz   db   4
```

17. Write program to find (x,y,z) element. If input is x=3 y=1 z=4 then output is 6.

18. Write program, which will print location (xyz) of all those elemetns which are 7. For above case it will be (221), (313), (214), and (224).

19. Write program, which will print the location of the maximum element. In above case the maximum element is 9. Its location is (223). If maximum element is present more than once than any one of its locations can be printed.

20. Write program, whose inputs are y and z and output is all elements with same y and z values. Input y=2 z=3 output 3 9 4.

21. Write program, whose inputs are x and z and output is all elements with same x and z values. Input x=2 z=3 output 4 9.

22. Write program, whose inputs are x and y and output is all elements with same x and y values. Input x=2 y=1 output 3 5 4 7.

23. Write program, whose input is z and output is all elements in same xy plane. For z=2 the output will be

    4 5 2
    3 5 2

24. Write program, whose input is y and output is all elements in same xz plane. For y=2 the output will be

    5 7 3
    3 5 2
    3 9 4
    2 7 3

25. Write program, whose input is x and output is all elements in same yz plane. For x=2 the output will be

    3 7
    5 5
    4 9
    7 7

26. Write program to find MAX-MIN-MAX-ZYX. Find Min-Max for each XY plane (for z=1,2,3, and 4). Now find their maximum. For above example the output is 7. Here Min-Max for every matrix are 4, 5, 7 and 7 respectively.

String

```
.model small
aaa segment
    a db 'abcdefghijklmn'
aaa ends
bbb segment
    b db 'ABCDEFGHIJKLMN',36
bbb ends
ccc segment
    assume CS:ccc
L1: mov ax,aaa
    mov ds,ax
    mov ax,bbb
    mov es,ax
    mov si,offset a
    mov di,offset b
    add si,4
    add di,6
    mov cx,3
     CLD
     REP MOVSB
    mov ah,9
    mov bx,bbb
    mov ds,bx
    mov dx,offset b
    int 33
    mov ah,76
    int 33
ccc ends
end L1
```

Movsb is equivalent to
ES:[DI]←DS:[SI]; DI++; SI++;
Output of given program is ABCDEFefg...

Replace cld by std output: ABCDcde....
In this case DI--and SI-- are done

Replace movsb by movsw output: ABCDEFefghij...
In this case DI=DI+2 and SI=SI+2 are done

Replace movb by stosb, before that put Mov AL,'x'
output ABCDEFxxx...
Stosb is equivalent to Mov ES:[DI],AL; DI++

Replace it by stosw, before that put Mov AX,25665
output ABCDEFAdAdAd...

Replace it by lodsb, after that print AL. Output: g
It is equivalent to Mov AL,DS:[SI]; SI++;
Replace it by lodsw, after that print AX. Output: ji

Rep T is equivalent to while (CX≠0) {T, CX--}

```
bbb segment
    b db 'abcdeeeeefghijxyzw'
bbb ends
ccc segment
    assume CS:ccc
L1: mov es,bbb
    mov di,offset b
    add di,6
    mov al,'e'
    mov cx,9
     CLD
     REPE SCASB
    mov ah,2
    mov dx,cx
    add dl,48
    int 33
    mov dl,es:[di]
    int 33
    mov ah,76
    int 33
ccc ends
end L1
```

Scasb is equivalent to cmp AL,ES:[DI]
It does flag setting.

REPE T is equivalent to:
if (CX≠0)
do{ T ; CX--} while (CX≠0 and ZF≠0)
On every execution CX is decremented.
output of above program is 5g,
because SCASB is executed 4 times.
if AL is 'h' then output is 8e.

REPNE is similar to REPE, except ZF=0 is used in place of ZF≠0.
In above program if AL is 'h' and REPNE is used then output is 3i, because SCASB is executed 6 times.

If AL is 'w' then output is y0, because loop is executed 9 times

```
mov es,bbb
mov ds,aaa
mov si,offset a
mov di,offset b
add di,3
mov cx,9
 CLD
 REPE CMPSB
mov ah,2
mov dx,cx
add dl,48
int 33
print(es:[di])
```

```
aaa segment
    a db 'eeefgjkxyilw'
aaa ends
bbb segment
    b db 'deeeeefghijxyzw'
bbb ends
```

Cmpsb≡cmp ds:[si],es:[di]
Output of given program is 3i, because Cmpsb is executed 6 times
When Repe is replaced by Repne and Add Si,3 is used then output becomes 2j, because Cmpsb is executed 7 times.

Input to string shall be in following format a db ‘ Ram is a good boy `,36

1. Write program to find length of string. In above case 24. [Hint: Find location of 36 using Scasb, Initialize CX by a big number say 1000].
2. Write program to find number of words in string. In above case 5.
3. Find length of longest word. In above case 4 (good).
4. Find how many letters are ‘o’. In above case 3.
5. Write program to find initial letter of every word. In above case Riagb.
6. Write program to find number of capital letters in string.
7. Make another copy of string. Replace every word of string by string of same number of x’s. e.g Ram is replaced by xxx, good by xxxx.
8. Make another copy of string. In which unnecessary blanks are removed. In above case new string will look as ‘Ram is a good boy',36.
9. Make another copy of string in which words are in reverse order. In above case new string will look as ‘ boy good a is Ram `,36
10. Reverse every word. In above case new string will look as ‘ maR si a doog yob `,36.
11. Find the lexicographical biggest word. (e.g. Ram is bigger than Anil, but smaller than Ramesh).

Let qptan are first five letters of a string. Print following pictures by executing int 33 only twice. Once for stop and once when ah is 9.

12. qqqqqpppppttttttaaaaannnnn
13. qqqqqpppptttaan
14. qptanqptanqptanqptanqptanqptan
15. qptanptantanann
    ptantanann
    tanann
    ann
    n

16. qptanqptaqptqpq
    qptanqptaqptqp
    qptanqptaqpt
    qptanqpta
    qptan

17. qptanqptaqptqpq

18. qptanqptaqptqpq
    ptanptaptp
    tantat
    ana
    n

19. qptan
    ppttaann
    tttaaannn
    aaaannnn
    nnnnn

Linked List

```
.model small
.data
  head db   3
  node db "APBKXCXL      "
  next db 5,7,-1,0,0,1,0,2,"  "
  size db 8
.code
main    proc
        Mov     AX,Dgroup
        Mov     DS,AX
        CALL    printlist
        Mov     AH,1
        Int     33
        Mov     CL,AL
        Int     33
        CALL    insertafter
        CALL    printlist
        Stop
main    endp

printlist       proc
        Mov     AH,2
        Mov     BL,head
        Mov     BH,0
L1:     Cmp     BL,0
        JL      L2
        Mov     DL,node[BX]
        Int     33
        Mov     BL,next[BX]
        JMP     L1
L2:     RET
printlist       endp

insertafter     proc
;Insert AL after CL
        Mov     BL,head
        Mov     BH,0
L4:     Cmp     CL,node[BX]
        JE      L3
        Mov     BL,next[BX]
        JMP     L4
L3:     Mov     DL,size
        Mov     DH,next[BX]
        Mov     next[BX],DL
        Mov     BL,size
        Mov     next[BX],DH
        Mov     node[BX],AL
        ;node[size]=AL
        ;next[CL]=size
        ;next[size]=next[CL]
        Add     size,1
        RET
insertafter     endp
        end     main
```

A sequence can be represented in the form of a linked list. e.g. KACPLB can be represented as follows:

head 3

Index	0	1	2	3	4	5	6	7	8
Node	A	P	B	K		C		L	
Next	5	7	-1	0		1		2	

1. Write program, which reads a digit ‘n’ and prints the nth element. e.g. input 3 output C.

2. Write program, which prints the last element of the sequence.

3. Write program, which finds the biggest element of sequence. e.g. for above case P.

4. Define a procedure, which reads an (element) character and deletes that e.g. for input P the internal representation after delete will be:

head 3

Index	0	1	2	3	4	5	6	7	8
Node	A	P	B	K		C		L	
Next	5	7	-1	0		7		2	

The output of function printlist will be KACLB. [Caution: Care should be taken to delete the first node. Now head will change.]

5. Define a procedure, which reads two characters and inserts the first character, before the second character. e.g. for input ZL the internal representation will look as:

head 3

Index	0	1	2	3	4	5	6	7	8
Node	A	P	B	K		C		L	Z
Next	5	8	-1	0		1		2	7

The output of function printlist will be KACPZLB

Puzzles

```
.model small
.code
   mov ah,2
   mov dl,byte ptr cs:[20]
   int 33
   mov ah,76
   int 33
   db 65
   db 66
   db 87
   db 68
   db 69
   db 70,71,72,73,74,75
   db 90
   end
```

```
.model small
.code
   mov ah,2
   mov cs:[11],byte ptr 68
   mov ah,2
   mov dl,65
   int 33
   mov ah,76
   int 33
   end
```

```
.model small
.code
   mov ah,2
   sub cs:[11],byte ptr 1
   mov dl,65
   add bl,5
   int 33
   mov ah,76
   int 33
   end
```

Observe the difference between the translations of add bl,5 and add dl,5. The machine code of bl is one more than that of dl.

```
.model small
.code
   mov ah,2
   mov cl,10
   mov bl,2
   add cs:[15],byte ptr 2
   mov dl,65
   add dl,cl
   int 33
   mov ah,76
   int 33
   add dl,bl
   end
```

```
.model small
.code
   mov ah,2
   mov bl,40
   mov cs:[12],byte ptr 2
   mov dl,65
   mov dl,bl
   int 33
   mov ah,76
   int 33
   end
```

Observe the difference between the machine codes of mov reg,reg and add reg,reg.

```
.model small
.code
   mov bl,65
   mov cl,70
   sub cs:[10],byte ptr 2
   mov cl,bl
   mov ah,2
   mov dl,cl
   int 33
   mov dl,bl
   int 33
   mov ah,76
   int 33
   end
```

mov reg, reg can be translated in two ways.

```
.model small
.code
        Mov AH,2
        Jmp L2
L1:     Mov DL,65
        Int 33
        Mov AH,76
        Int 33
L2:     Mov cs:[6],byte ptr 66
        Jmp L1
        END
```

```
.model small
.code
        Mov AH,2
        Jmp L2
L1:     Mov DL,65
        Int 33
        Add DL,40
        Int 33
        Mov AH,76
        Int 33
L2:     Mov cs:L1[1],word ptr
45570
        Jmp L1
        END
Since 45570 = 256*178 + 2
Int is replaced by Mov
DL,(number)
```

```
.model small
.code
        Mov AH,2
        Mov DL,70
        Jmp L2
L1:     Mov CL,72
        Int 33
        Mov AH,76
        Int 33
L2:     Mov SI,offset L1
        Add cs:[SI],byte ptr 1
        Jmp L1
        END
It is because the machine code
of DL is 1 more than that of CL.
Replace byte ptr by word ptr
513.
```

```
.model small
.code
        Mov AH,2
        Mov DL,48
        Mov BL,75
        Jmp L2
L1:     Mov BL,DL
        Int 33
        Mov AH,76
        Int 33
L2:     Mov SI,offset L1
        Sub cs:[SI],byte ptr 2
        Jmp L1
        END
Change of direction
```

```
.model small
.code
        Mov AH,2
        Mov BL,5
        Jmp L2
L1:     Mov DL,65
        Mov BL,DL
        Int 33
        Mov AH,76
        Int 33
L2:     Mov SI,offset L1
        Mov cs:[8],byte ptr 67
        Jmp L1
        END
Observe the output be replacing
byte ptr with word ptr
```

Machine codes:

Mov reg,number: (10110reg)(number)

Mov reg_1,reg_2 : (100010d0)(11$reg_1$$reg_2$)

Add reg_1,reg_2 : (000000d0)(11$reg_1$$reg_2$)

Jmp : (11101001)(disp)( )

Int number : (11001101)(number)

AL:000 BL:011 CL:001 DL:010

Macro

```
.model small
.code
        Mov     AH,1
        Int     33
        Mov     AH,2
        Mov     DL,65
        Mov     BL,AL
l2:
     rept    10
        Int     33
        Dec     BL
        Cmp     BL,0
        JE      l1
     endm
        Jmp      l2

L1:     Mov     AH,76
        Int     33
END
```

Read a number and print no of A's equal to the ASCII code of input number

```
abc macro n
        Mov     DL,n
endm
.model small
.code
        Mov     AH,2
        Mov     CL,90
        abc CL  ; n=CL
                ; Mov  DL,n
        Int     33
        Mov     DL,CL
        Mov     DL,90
        Mov     DL,cs:[9]
        Int     33
        Mov     DL,cs:[11]
        Int     33
        Mov     DL,cs:[5]
        Int     33
        Mov     AH,76
        Int     33
End
```

n=cl is stopped. It may be thought that it is the value of CL.

abc CL is translated as Mov DL,CL and not as Mov DL,90

```
abc macro x
   if x eq al
      mov dl,65
   else
      mov dl,66
   endif
endm
.model small
.code
        Mov     AH,2
        abc  al
        int     33
        abc  dl
        int     33
        Mov     AH,76
        int     33
END
```

Register name can be used as parameter

```
aa macro x
   x dl,bl
endm
.model small
.code
        Mov      ah,2
        Mov      bl,30
        aa mov
        aa add
        aa add
        Int     33
        Mov     AH,76
        Int     33
END
```

Instruction name can be used as parameter

```
abc macro x
   Mov dl,65+x
   Int  33
   Int  33
endm
pqr macro x
   Mov dl,66
   Int  33
endm
ttt macro x
   x 5
endm
.model small
.code
        Mov     AH,2
        ttt     abc
        ttt     pqr
        Mov     AH,76
        Int     33
END
```

Macro name can be parameter

```
.model small
.code
        Mov     AH,2
        Mov     DL,0
        n=65
        Jmp     L1
        n=n+1
        Add     DL,5
l1:     Add     DL,n
        Int     33
        Mov     AH,76
        Int     33
End
```

Macro expansion and execution are two different phases

```
.model small
.code
        m=50
        t=65
        if m gt t
            Mov  CL,kk
            t=t+1
        else
            Mov  DL,t
        endif
        Mov     AH,2
        Int     33
        Mov     AH,76
        Int     33
End
```

The above program does not show error because of Mov CL,kk but the following program shows.

```
.model small
.code
        Mov     BL,50
        t=65
        Cmp     BL,t
        JNG     L1
        Mov     CL,kk
        t=t+1
        Jmp     L2
L1:     Mov     DL,t
L2:     Mov     AH,2
        Int     33
        Mov     AH,76
        Int     33
END
```

```
abc macro
local n
   n=70
endm

.model small
.code
        n=65
        abc
        Mov     DL,n
        Mov     AH,2
        Int     33
        Mov     AH,76
        Int     33
END
```

When n is not local

```
ttt macro k
   Mov   DL,k
   Int   33
   Int   33
endm

ppp macro t
   local ttt
   ttt macro k
       Mov  dl,k+10
       Int  33
   endm
   ttt t+12
endm
.model small
.code
    Mov  Ah,2
    ttt 65
    ppp 65
    ttt 65
    Mov  AH,76
    Int 33
end
```

Remove local ttt

Remove first macro defination ttt also

To remove the error remove the first ttt 65 also

```
ttt macro
   Mov  DL,65
   Int  33
endm
ppp macro
   Mov  DL,66
   Int  33
endm
abc macro
  local ttt
  ttt macro
     ppp macro
        Mov  DL,67
        Int  33
     endm
     Mov  DL,97
     Int  33
   endm
   ttt
   Mov DL,90
   Int 33
endm
.model small
.code
        Mov     AH,2
        ttt
        ppp
        abc
        ppp
        ttt
        ppp
        Mov     AH,76
        Int     33
END
```

Remove local ttt

Remove ttt from abc

```
abc macro
   65
endm
.model small
.code
    Mov DL,2
    Mov DL,abc
    a  db  65
    n=20
    cl=20
    cl=cl+1
    a=25
    Mov BL,CL
    Mov BL,a
    rept CL
    Int 33
    endm
    cmp=20
    l1=20
l2: Mov BL,CL
l1: Mov BL,CL
    l2=20
end
```

Which symbols can be used as variable

Packed BCD

```
.model small
.code
        Mov     AH,1
        Int     33
        Mov     BL,AL
        Int     33
        Add     BL,AL
        Int     33
        DAA
        Mov     DL,AL
        Mov     ah,2
        Int     33
        Mov     AH,76
        Int     33
END
.Model small
.Code
        Mov ah,1
        Int 33
        Mov bl,al
        Int 33

        Add al,bl
        daa

        Mov dl,al
        Mov ah,2
        Int 33

        Mov DL,0
        Jnc l1
        Add dl,'1'
L1:     Int 33

        clc
        Mov al,41h
        daa
        Mov dl,al
        Int 33
        Mov ah,76
        Int 33
END

rd_hex_digit macro x
local l1,l2
        Mov     AH,1
        Int     33
        Mov     x,AL
        Sub     x,48
        Cmp     x,32
        Jle     l2
        Sub     x,32
l2:     Cmp     x,9
        Jle     l1
        Sub     x,7
l1:
endm

rd_two_digits macro x
        rd_hex_digit x
        rept 4
          Shl     x,1
        endm
        rd_hex_digit AL
        Add     x,AL
endm

pt_hex_digit  macro x
Local l1
        Cmp     x,10
        Jl      l1
        Add     x,7
l1:     Add     x,48
        Mov     AH,2
        Int     33
endm

pt_two_digits macro x
        Mov     DL,x
        rept    4
          Shr     DL,1
        endm
        pt_hex_digit DL
        Mov     DL,x
        And     DL,0Fh
        pt_hex_digit DL
endm

.model small
.code
        rd_two_digits CH
        rd_two_digits CL
        rd_two_digits BH
        rd_two_digits BL

        Add     CL,BL
        Mov     AL,CL
        DAA
        Mov     CL,AL

        Adc     CH,BH
        Mov     AL,CH
        DAA
        Mov     CH,AL

        pt_two_digits CH
        pt_two_digits CL
        Mov     AH,76
        Int     33
END
```

Word Size Register

```
.model small
.code
     mov dx,25665
     mov ah,2
     int 33
     mov dl,dh
     int 33
     mov ah,76
     int 33
end
```

```
.model small
.code
     mov ah,1
     int 33
     mov ch,al
     int 33
     mov cl,al
     mov dl,'0'
     cmp cx,25665
     jne l1
     mov dl,'1'
l1:  mov ah,2
     int 33
     mov ah,76
     int 33
end
```

```
.model small
.code
     mov cl,255
     mov ch,30
     add cl,1
     mov dl,'n'
     jnc l1
     mov dl,'c'
l1:  mov ah,2
     int 33
     mov ah,76
     int 33
end
```

Replace ADD CL,1 by ADD CX,1

```
.model small
.code
        jmp l1
    a  dw 12850,51
    b  db 48,49,50
    c  db 101,102,103
    d  db 56,57,58
    e  db 90,89,88,10,13,36,70
l1:     Mov     AX,CS
        Mov     DS,AX
        Mov     DX,offset a
        Mov     AH,9
        Int     33
        Mov     a,53
        Int     33
        Mov     b,53
        Int     33
        Mov     BX,offset b
        Mov     [BX],byte ptr 50
        Int     33
        Mov     BX,offset c
        Mov     [BX],word ptr 50
        Int     33
        Mov     CX,25670
        Mov     BX,offset d
        Mov     [BX],CL
        Int     33
        Mov     BX,offset e
        Mov     [BX],CX
        Int     33
        Mov     AH,76
        Int     33
END
```

```
.Model small
.Code
        Mov     AH,1
        Int     33
        Add     AL,AL
        CBW
        Mov     BX,10000
        Add     BX,AX
        Mov     DX,AX
        Mov     AH,2
        Int     33
        Mov     DL,DH
        Sub     DL,100
        Int     33
        Cmp     BX,10000
        Mov     DL,'g'
        Jg      L1
        Mov     DL,'l'
l1:     Int     33
        Mov     DL,155
        Int     33
        Mov     DL,156
        Int     33
        Mov     AH,76
        Int     33
END
```

Instruction Pointer

```
.model small
.code
        mov ah,2
        mov dl,67
        mov cx,17
        push cx
        ret
        int 33
        int 33
        int 33
        int 33
        int 33
        int 33
        int 33
        mov ah,76
        int 33
        end
```

```
.model small
.code
      mov ah,2
      mov dl,65
      call l1
      int 33
      int 33
      int 33
      int 33
      int 33
      mov ah,76
      int 33
l1:   pop cx
      add cx,6
      push cx
      ret
      end
```

```
jump macro l
    Mov CX,offset l
    Push CX
    RET
endm
.model small
.code
        Mov     AH,2
        Mov     DL,65
        Jump    l1
        Int     33
        Inc     DL
        Int     33
l1:     Inc     DL
        Int     33
        Mov     AH,76
        Int     33
end
```

```
cll macro l
 local t
    Mov CX,offset t
    Push CX
    Jmp l
 t:
endm
.model small
.code
pqr  proc
        Inc     DL
        RET
pqr  endp
main proc
        Mov     AH,2
        Mov     DL,65
        cll    pqr
        Int     33
        Mov     AH,76
        Int     33
main endp
end main
```

```
jumc macro l
local l1
    Mov CX,0
    Rcl CX,1
    Mov BX,CX
    Shl CX,1
    Add CX,BX
    Mov BX,offset l
    Add CX,offset l2
    Push CX
    Ret
l2: Mov BX,offset l1
    Push BX
    RET
l1:
endm
.model small
.code
        Mov     AH,1
        Int     33
        ADD     AL,AL
        ADD     AL,AL
        Mov     AH,2
        Mov     DL,'C'
        Jumc      L6
        Mov     DL,'N'
l6:     Int     33
        Mov     AH,76
        Int     33
end
```

Segment

```
.model small
.code
kkk segment
        Assume CS:kkk
L3:     Mov     AH,2
        Mov     DL,65
L2:     Int     33
        Jmp     far ptr L1
kkk ends
ppp segment
        Assume  CS:ppp
       Add     DL,1
       Int     33
L1:    Add     DL,1
       Cmp     DL,90
       Jg      L4
       Jmp     far ptr L2
L4:    Mov     DX,offset L1
       Add     DL,48
       Int     33
       Mov     DL,offset L2
       Add     DL,48
       Int     33
       Mov     AH,76
       Int     33
ppp ends
        end    L3
```

We can write programs in more than one segment

The last two outputs show that offset of two addresses can be same

```
.model small
.code
kkk segment
Assume CS:kkk
L2:     Mov     DL,AL
        Mov     DL,AL
        Jmp short l2
        Jmp near ptr  l2
        Jmp far ptr l2
        Jmp l2
        Jmp l3
        Jmp far ptr l3
        Jmp near ptr l3
        jmp near ptr l31
        jmp near ptr l31
        Jmp short l3
        Jmp far ptr l4
        Jmp far ptr l4
;       Jmp near ptr l4
;       Jmp short l4
        Mov dl,al
l3:
   rept 200
   mov bl,cl
   endm
l31:
kkk ends
ppp segment
Assume CS:ppp
        mov dl,al
L4:
ppp ends
end
```

Observe the LST file to observe difference among JMP short, near and far

```
.model small
.code
kkk segment
   a db 97
   b db 98
kkk ends
ppp segment
   f db 102
   g db 103,104
ppp ends
mmm segment
   assume cs:mmm, ds:kkk
l1:
        Mov     AX,ppp
        Mov     DS,AX
        Mov     DL,a
        Mov     AH,2
        Int     33
        Mov     AH,76
        Int     33
mmm ends
        End l1
```

What happens when we make wrong assumptions

```
.model small
.code
kkk segment
   a db  97
   b db  98
kkk ends
ppp segment
   f db  102
   g db  103,104
ppp ends
mmm segment
   assume cs:mmm
   assume ds:kkk,es:ppp
l1:mov ah,2
   Mov BX,kkk
   Mov DS,BX
   Mov BX,ppp
   Mov ES,BX
   Mov Dl,a
   rept  5
      Mov DL,a
      Int 33
      assume ds:ppp, es:kkk
   endm
   Mov ah,76
   Int 33
mmm ends
   end l1
```

First macro expansion takes place then translation is done

```
.model small
.code
kkk segment
        Mov     AL,DL
        n=5
        if n eq 6
        Mov     DL,CL
        ASSUME  DS:ppp
        endif
        Mov     DL,a
kkk ends
ppp segment
        a db 97,98
ppp ends
end
Put n=6
```

Stack

```
.model small
.code
ppp segment
  Mov AX,aaa
  Mov DS,AX
  Mov SS,AX
  Mov AH,9
  Mov DX,offset p
  Int 33
  Mov SP,4
  Mov BX,3031h
  Push BX
  Mov DX,offset p
  Mov AH,9
  Int 33
  Mov SP,10
  Pop DX
  Mov AH,2
  Int 33
  XCHG DH,DL
  int 33
  Mov AH,76
  Int 33
ppp ends

aaa segment
  p db 65,66,67,68,69,70,
  r db 71,72,73
  q db 97,98,99,100,101,
  t db 102,103,104,10,13,36
aaa ends
end
Replace Mov SP,10 by
Mov SP,9
```

```
.model small
.code
  Mov BX,3031h
  Push BX
  Mov BX,4461h
  Push BX
  Mov SI,SP
  Mov AX,SS
  Mov DS,AX
  Mov DL,[SI+1]
  Mov AH,2
  Int 33
  Mov AH,76
  Int 33
END
```

```
.model small
.data
   a db 'abcdefghijklmnopqrst'
.code
   Mov AX,Dgroup
   Mov DS,AX
   Add AX,1
   Mov ES,AX
   Mov AH,2
   Mov SI,offset a
   Mov DL,[SI+16]
   Int 33
   Mov DL,ES:[SI]
   Int 33
   Mov AH,76
   Int 33
   End
```

Segment Register

```
.model small
.code
ppp segment
  Mov AX,aaa
  Mov DS,AX
  Mov AH,9
  Mov DX,offset p
  Int 33
  Mov DX,offset p
  Mov AX,DS
  Add AX,1
  Mov DS,AX
  Mov AH,9
  Int 33
  Mov AH,76
  Int 33
ppp ends

aaa segment
  p db 65,66,67,68,69,70,
  r db 71,72,73,74,75,76,77
  q db 97,98,99,100,101,
  t db 102,103,104,10,13,36
aaa ends
end
```

```
.model small
.code
   Mov AH,2
   Mov CX,8
   Mov  BX,3040h
L1:
   Push BX
   Add BX,1
   Push BX
   Add BX,1
   Mov DL,CL
   Add DL,48
   Int 33
   Loop L1
   Mov BX,SS
   Add BX,1
   Mov SS,BX
   Pop DX
   Int 33
   Mov AH,76
   Int 33
END
```

Loop L1 is equivalent to Sub CX,1; If CX!=0 goto L1
Put sub CL,1 after Push BX
Put sub CL,2

```
printword macro t,n
   Mov  DX,t
   Add  DX,n
   Mov  AH,2
   Int  33
   Mov  DL,DH
   Int  33
endm
.model small
.code
   Mov  DX,4297h
   Push DX
   Mov  BX,SP
   Call l1
   rept 10
       Int 33
   endm
   Mov  AH,76
   Int 33
l1:Sub  BX,SP
   printword BX,48
   Pop  BX
   printword BX,40
   Pop  CX
   printword CX,0
   Pop  BX
   printword BX,0
   Mov  BX,CS
   printword BX,0
   Mov  AH,76
   Int  33
END
```

Put following lines after

```
printword CX,0
Mov AH,2
Mov DL,65
Push CX
Add BX,6
Push BX
```

```
RET   or   db   11000011b   ;
Machine   code   of   within
segment return
From above remove Push CX
Use call far ptr L1
Put  following  lines  after
printword CX,0
Add CX,1
Push CX
Push BX
Mov  AH,2
Mov  DL,65
db    11001011b   ; Machine
code of Intersegment return

.model small
.code
        Mov     AH,2
        Mov     DL,65
        Mov     BX,CS
        Inc     BX
        db      8Eh, 0CBh
        ;Mov  CS,BX   is  not
permitted
        rept 10
                Int     33
        endm
        Mov     AH,76
        Int     33
        Mov     DS,BX
        Mov     SS,BX
        Mov     ES,BX
End

.model small
.code
        Mov     AH,2
        Mov     DL,65
        Mov     BX,CS
        Inc     BX
;       Push    BX
        Mov      BX,  offset
L1
        Push    BX
;       db      0CBh
;M/c  code  of  intersegment
return
        Ret
L1:
        rept 10
                Int     33
        endm
        Mov     AH,76
        Int     33
End

.model small
.code
kkk segment
Assume CS:kkk
l1: Mov AH,2
    Mov DL,65
    Int 33
    Mov BX,ttt
    Push BX
    Mov BX,2
    Push BX
    db 11001011b
    Add DL,5
    Int 33
    Mov Ah,76
    Int 33
kkk ends
ttt segment
Assume CS:ttt
    Int 33
    Int 33
    Mov AH,76
    Int 33
ttt ends
end l1
;In Mov BX,2 put 0 in place
of 2
;Observe  the  output  when
return far is not used
```

```
.model small
.code

L2:
abc proc far
L3:
  pop bx
  pop cx
  add bx,20
  add cx,1
  push cx
  push bx
  ret
abc endp
  ret

L1:
  mov bx,65
  mov ah,2
  push bx
  mov dh,1
  mov dl,48
  call abc
  rept 20
    int 33
  endm
  pop dx
  int 33
  pop dx
  int 33
  mov ah,76
  int 33
  call abc
  call L2
  call L3
  call far ptr L3
  jmp far ptr abc
  jmp abc
  jmp short abc
end L1
```

Call far ptr L_2 is equivalent to

Push CS
Push offset L_1
Jmp far ptr L_2
L_2:

Return far is equivalent to

Pop IP
Pop CS

Observe the difference in the translations of various returns and calls.
Output of above program is 00At. (t:some random number)
When we remove far from abc proc then output is 0000000000Bt
Same effect can be obtained by removing the first return and making call as call l2 or call l3.
When we remove far from abc proc but make call as far ptr then output produced is 0000000000jA. Here j is character corresponding to ASCII code of CS.
Let the program remain as it is. Call program as call L2. Program hangs. When we put Push CS before call then output is 00At.
abc proc far and abc endp only assist the assembler in translation of RET.

Linking **(common)**

a.asm

```
.model small
.code
  extrn l2:far
aaa segment
  Mov ah,2
  jmp far ptr l2
aaa ends
end
```

Link a+b+c ⇒ bc
Link a+c+b ⇒ Bc

b.asm

```
.model small
.code
  public l1
bbb segment common
   assume cs:bbb
l1:mov dl,65
   int 33
   mov dl,66
   int 33
bbb ends
end
```

c.asm

```
.model small
.code
public l2
bbb segment common
assume cs:bbb
   mov dl,97
   int 33
l2:mov dl,98
   int 33
   mov dl,99
   int 33
   mov ah,76
   int 33
bbb ends
end
```

p.asm

```
.model small
public l1
aaa segment
assume cs:aaa,assume ds:bbb
l2:mov ah,2
   mov dl,'t'
   int 33
l1:mov ax,bbb
   mov ds,ax
   mov ah,2
   mov dl,ds:[3]
   int 33
   mov dl,ds:[1]
   int 33
   mov dx,offset a
   mov ah,9
   int 33
   mov ah,76
   int 33
aaa ends
bbb segment common
   a db 65,66
bbb ends
end l2
```

q.asm

```
.model small
extrn l1:far
bbb segment common
    p db 48,49,50,51,52,36
bbb ends
end l1
```

Link p+q ⇒ t3101234
Link q+p ⇒ 3BAB234

a1.asm

```
aaa segment
assume cs:aaa
l1:mov ds,bbb
   mov dx,0
   mov ah,9
   int 33
   aaa ends
bbb segment common
   db 'abcdefghi'
bbb ends
end l1
```

a2.asm

```
bbb segment common
   a db '012345678901'
bbb ends
end
```

a3.asm

```
bbb segment public word
   a db 65,66,67,68
bbb ends
end
```

a1+a2+a3 ⇒ 012345678901ABCD
Because a1+a2 ⇒ 012345678901

a1+a3+a2 ⇒ 012345678901CD
Since a1+a3 ⇒ abcdefghi ABCD

Let a2 is modified as 0123
a1+a2+a3 ⇒ 0123efghi ABCD
Because a1+a2 gives 0123efghi

Alignment

```
                          .model small
                          .code
 0000                     aaa segment
 0000  0C 11 13                   a db 12,17,19
 0003  00                    even
 0004  22 38                      b db 34,56
 0003                        org 3
 0003  61 62                      c db 97,98
 0005                     aaa ends
 0000                     bbb segment
                             assume cs:bbb
 0000  BA 020C                    mov dx,524
 0003  90                    even
 0004  B2 37                      mov dl,55
 0006  43 44 45                   p db 67,68,69
 0009  90                    align 2
 000A  30 31                      q db 48,49
 000C                     bbb ends
                          end
```

```
.model small
.code
   mov dl,65
   add dl,2
   org 2
   add dl,1
   mov ah,2
   int 33
   int 33
   int 33
   mov ah,76
   int 33
   org 0
   mov dl,97
   org 9
   mov dl,90
   end
Output bZ
```

```
.model small
.code
   mov dx,942
   org 1
   Add BH,CL
   add dl,67
   mov ah,2
   int 33
   mov ah,76
   int 33
end
```

Output E, C is also possible
if Add [BX],CL Output C
If Add BX,CX is used Output F, D is also possible
If Add [BX],CX is used output D
M/C code of add reg,r/m (000000dw)(mdregr/m)

x.asm

```
.model small
.code
public l1
aaa segment
assume cs:aaa
l1:mov ax,bbb
   mov ds,ax
   mov dx,0
   mov ah,9
   int 33
   mov ah,76
   int 33
aaa ends
bbb segment public word
   a db 65,66,67
bbb ends
end l1
```

y.asm

```
.model small
extrn l1:far
bbb segment public para
    p db 48,49,50,51,52
bbb ends
end l1
```

Link x+y ⇒ ABC 01234 .. (13 blanks)
These are not blanks. These are characters whose ACSII code is 0. It can be verified by Mov DL,ds:[5]; Add dl,70 and printing DL
Link y+x ⇒ 01234 ABC.... (1 blank)
When word is replaced by Dword 01234 ABC (3 blanks)
When word is replaced by page 01234 … ABC (251 blanks)

JAVA

String
Loop
String Buffer
Arithmetic
Function
Recursion
Object representation
Copy
Link list
Exception
Abstract Data Type
Inheritance
Interface
Visibility/Error
Thread
File
Graphics
Mouse
Key

C++

Class Overloading Abstract Data Type

String

In a name (say a.java) type following program. The file can be compiled by command **javac a.java**. To run the program give command **java kapil**.

```
import java.io.*;
import java.lang.*;
class kapil
{  public static void main(String args[])
   { String a,b;
     try
     { DataInputStream o=new DataInputStream(System.in);
       a=o.readLine();
       b=a.substring(2,5);
       System.out.println(b);
     }
     catch(IOException e) { System.out.println(e);  }
   }
}
```

The program outputs sub string between positions 2 and 5 (including 2 but excluding 5). The first character is at 0th position. e.g. input qwertyuiuo output ert.

```
b=a.substring(4);System.out.println(b);
```

The program outputs string on and after 4th position. input qwertyuiuo output tyuiuo

```
String a,b,c; a=o.readLine();b=o.readLine();
c=a+b; System.out.println(c);
```

The program takes two strings and joins them.

1. Write program to remove 2nd letter. Let the input string is pwsxtpbcderxrtxgt then output is pwxtpbcderxrtxgt.
2. Write program to add 0th letter in the beginning. In above case ppwsxtpbcderxrtxgt
3. Write program to exchange first two letters. In above case wpsxtpbcderxrtxgt.
4. Write program to exchange 4th and 10th letter. In above case pwsxrpbcdetxrtxgt.
5. Write program to insert ‘t’ between 1st and 2nd letter. In above case pwtsxtpbcderxrtxgt. [Hint: a+”t”+b].

```
a=o.readLine();
i=a.indexOf('x');
System.out.println(i);
```

At what location 'x' is present. If more than one occurrence of 'x' is there then the location of first 'x' is returned. If 'x' is absent then –1 is returned. e.g. input wedxtyhxu output 3

```
int i;String a,b,c,d;i=a.indexOf('x');
b=a.substring(0,i);c=a.substring(i+1);
d=b+c; System.out.println(d);
```

The first 'x' in the given string is deleted.

6. Write a program, which reads a string and finds string after the first x. Let the input string is pwsxtpbcderxrtxgt then output is tpbcderxrtxgt.

7. Write program to replace first x by y. In above case pwsytpbcderxrtxgt.
8. Write program to output the location of second x. In above case 11.
9. Write program to print the string between 1st and 2nd x. In above case tpbcder.
10. Write program to find string before 2nd x. In above case pwsxtpbcder.
11. Write program to delete the string between 1st and 2nd x. In above case pwsxxrtxgt.
12. Program to exchange the string between 1st and 2nd x, with the string before 1st x. In above case tpbcderxpwsxrtxgt
13. Write program to exchange neighbors of first x. In above case pwtxspbcderxrtxgt.

```
char b;a=o.readLine();b=a.charAt(2);
System.out.println(b);
```

Program outputs character at location 2. e.g. input qwertyuiuo output e

14. Write program to find 2nd location of 0th letter. Input pwerpty output 4.
15. Write program, which will delete 1st y immediately after 1st x. If input string is pgyeryuyixaysdyexer then output is pgyeryuyixasdyexer.
16. Write program to exchange neighbors of first occurrence of left neighbors of first 'x'. e.g. input abcdefxgh output abcdxfegh. input abfcdefxgh output acfbdefxgh.
17. Write program to replace first occurrence of right neighbor of 2nd x by left neighbor of 1st x. Input imgpxugxutkl output imgpxpgxutkl. Input bcxdefxgh output bcxdefxch.
18. Write program, which reads a string. Let x and y be respectively left and right neighbors of the second occurrence of the 0th letter. Find the substring between first occurrence of y and (first occurrence of x after first occurrence of y). e.g. input patkgfmpkst output kgfm. Input pastgksfsptse output tgks. Input raklfrgmcfd output gmcf. Input ywetyykjhtl output ywet.

```
a=o.readLine();b=o.readLine();
i=a.compareTo(b);System.out.println(i);
```

Input two strings. Output is 0 if both are same. If second string is (lexicographically) bigger then a negative number is outputted. If first string is bigger then some positive number is outputted.

Use of "if" is permitted in following programs.

19. Read two strings. Print lexicographically bigger string first and smaller later.
20. Read two string. Print 1 if first string is bigger, 2 if second string is bigger, 0 if both are same.
21. Read three strings. Print 1 if first string is biggest, 2 if second string is biggest, 3 if 3rd string is biggest, 0 if all are same, -1 if 1st and 2nd string are biggest, -2 if 2nd and 3rd string are biggest, -3 if 1st and 3rd string are biggest.

```
a=o.readLine();a=a.trim();
i=a.indexOf(' ');b=a.substring(0,i);
System.out.println(b);
```

Print first word. Trim removes blank spaces at the beginning and at the end. If it is not used then the problem will arise if blanks are given at the beginning. The program will not work if string has only one word. If string is ram Prasad dey then output is ram.

22. Print second word. In above case Prasad.
23. Delete second word. In above case ram dey.
24. Exchange first and second word. In above case Prasad ram dey.
25. Exchange first letters of first two words. In above case Pam rrasad dey.

26. Exchange last letters of first two words. In above case rad Prasam dey.
27. Find the location of first ‘a’ in second word. In above case 6.
28. Find location of first letter of first word in second word. In above case 5.

Loop

```
a=o.readLine();
for (i=1;i<=4;i++){
  j=a.indexOf('x');
  b=a.substring(0,j);
  a=a.substring(j+1);
  System.out.println(b);}
```

Prints first 4 substrings before x input qwexrx123xtyxasdfx output qwe, r, 123, ty

```
a=o.readLine();t="";
for (i=1;i<=4;i++){
  j=a.indexOf('x');
  b=a.substring(0,j);t=t+b;
  a=a.substring(j+1);
}
System.out.println(t+a);
```

Delete first 4 x's

1. Write program to print the string after 4th x. Let input string be pwsxtxaxxrxd then output is rxd.
2. Write program, which will find the string letters between 3th and 4th x. In above case output is null string.
3. Write program, which finds the letter after 4th x. In above case r.
4. Write program, which finds the letter before 3rd x. In above case a.

```
a=o.readLine();count=0;
while (a.indexOf('x')!=-1){
  i=a.indexOf('x');
  a=a.substring(i+1);
  count++;
}
```

Find Number of x's

5. Write program, which will find the string after last x. In above case d.
6. Write program, which finds letters after every x. In above case taxrd.
7. Write program, which deletes last x. In above case pwsxtxaxxrd.
8. Write program, which will find location of all x's. Let input string be pwsxtxaxxrxd the output is 3 5 7 8 10.
9. Write program, which will remove all x's. In above case output pwstard.
10. Write program, which will replace every x by y. In above case output pwsytyayyryd.
11. Write program, which will insert y after every x. In above case pwsxytxyaxyxyrxyd
12. Write program, which will find the location of last 'x'. In above case 10.
13. Write program, which will find the length of string. Do not use length function.

```
a=o.readLine();
for (i=1;(a.length()!=0);i++){
  b=a.substring(0,1);
  a=a.substring(1);
  System.out.println(b);  }
```

Print every characters of the string in separate line.

```
    a=o.readLine();loc=0;
    do{
      i=a.indexOf('x');
      loc=loc+(i+1);
      a=a.substring(i+1);
    } while(a.indexOf('y')!=0);
    System.out.println(loc-1);
  }
```

Find location of first xy. Input yerxryyttxysdfxu output 9.

14. Write program, which finds the number of non-consecutive x's. In above case 3.
15. Write program, which will find the location of sub string xy. If more than one xy is present then location of first xy will be printed. If no xy is present then –1 is printed. e.g. input wyqxaexytrxixyty output 6.
16. Write a program, which will remove x's which are at the start of the string. Input string xxxwexrxxsxx output wexrxxsxx.
17. Write program to remove x's which are at the end. In above case output xxxwexrxxs.
18. Write program, which delete 1st y immediately before 1st x. If input string is pgyeryuyixasdyexer then output is pgyeryuixasdyexer.
19. Write program, which will find the location of first x. Do not use *indexOf*.
20. Write program, which will insert x in between consecutive letters. e.g. input srwe output sxrxwxe.
21. Write program, which will reverse a string.
22. Write program, which will reverse sub string between consecutive x's. e.g. input wexcapxxzmtxabx output wexpacxxtmzxbax.
23. Read a string of words. Delete unnecessary blanks. Input I am bad output I am bad.
24. Read a string of words. Output all words at even location. Input ram hari gopal ravi anil ramesh output hari ravi ramesh.
25. Exchange consecutive words of string (assume string has even number of words). For above input output is hari ram ravi gopal ramesh anil.
26. Find lexicographically biggest word. For above case ravi.
27. Output all those words, which are smaller than raja. For above case hari gopal anil.
28. Output words of the string in sorted order [Output: anil gopal hari ram ramesh ravi]
29. Find the last word beginning with 'r'.
30. Reverse every word. For above input mar irah lapog ivar lina hsemar.
31. Find wordwise reverse. For above input ramesh anil ravi gopal hari ram.

    Write a program which will read a string (Let string is qwert) and print following pictures

32. qwert
    wert
    ert
    rt
    t

33. qwert
    qwer
    qwe
    qw
    q

34. qwertwertertrtt

35. qwweeerrrrttttt

36. qwertqwerqweqwq
    qwerqweqwq
    qweqwq
    qwq
    q

37. qwertqwerqweqwq
    wertwerwew
    ertere
    rtr
    t

38. qqqqqwwwweeerrt
    wwwwweeeerrrtt
    eeeeerrrrttt
    rrrrrtttt
    ttttt

39. Write program to find the product of number of y's separated by x's. Input yatyxyyxcyyyxyy output 2x2x3x2=24.

String Buffer

```
import java.io.*;
import java.lang.*;
class kapil
{  public static void main( String args[])
   { StringBuffer a;String b;int i;
     try
     { DataInputStream o=new DataInputStream(System.in);
       a=new StringBuffer("");b=o.readLine();a.append(b);
       System.out.println("The input string is"+a);
       i=a.indexOf("x");System.out.println(i);
       i=a.indexOf("xy");System.out.println(i);
       a.setCharAt(6,'X');System.out.println(a);
       a.insert(4,"tty");System.out.println(a);
       a.append("ghij");System.out.println(a);
       a.reverse();System.out.println(a);
       a.delete(4,7);System.out.println(a);
       a.deleteCharAt(5);System.out.println(a);
       a.replace(3,7,"was");System.out.println(a);
       a.setLength(5);System.out.println(a);
       b=a.toString();System.out.println(b);
     }
     catch(Exception e)
     { System.out.println("Error"+e);  }
   }
}
```

All the programs should be done without using +(plus), –(minus) or append (except during converting input string to string buffer).

1. Read a string and replace first x by y. Input artxuxtxxp output artyuxtxxp.
2. Replace last x by y. (do not use loop)
3. Replace second x by y.
4. Replace all x's by y.
5. Find the number of x's. In above case 4.
6. Whether given string is a Palindrome. A Palindrome is word which remains same when reversed. e.g. malayalam.
7. Find length of the string. (Do not use *length( )*)
8. Suppose string has words. e.g. Ram is a good boy. Find first and second word.
9. Find the first word of 4 letters. In above case *good*. Assume such word present at least once.
10. Remove all blanks. In above case *Ramisagoodboy*.
11. Find last letter of first word.
12. Find first letter of last word.
13. Find first letter of every word.
14. Replace first letter of every word by x. In above case x*am xs x xood xoy*.
15. Word wise reverse of the string. In above case *boy good a is Ram*.
16. Reverse each word. In above case output is *maR si a doog yob*.
17. Interchange first two words. In above case *is Ram a good boy*.

18. Write program, which reads an expression and outputs its value. Assume that parenthesis is not used. Assume all operations are left associative and all numbers are integers. Take only '+' and '*' as operators. e.g. input 12+17*10 output 290.
19. Do above problem when parentheses are used. e.g. input 12+(17*10+5*2) output 362.

Arithmetic

```
int k;String a;
try
{ DataInputStream o=new DataInputStream(System.in);
  a=o.readLine();k=Integer.parseInt(a);
  if (k%2==0) System.out.println(a+" is even");
  else System.out.println(a+" is odd");
}
```

Program reads a number and finds whether it is even or odd. The program does not work if blanks are left in the beginning or at the end. Hence a=a.trim() should be used. It removes blanks at the beginning and at the end.

```
int i;float x,y,k,l;String a,b;
try
{ DataInputStream o=new DataInputStream(System.in);
  a=o.readLine();a=a.trim();
  i=a.indexOf(" ");b=a.substring(0,i);x=Float.parseFloat(b);
  b=a.substring(i+1);y=Float.parseFloat(b.trim());
  k=x+y;l=x*y;System.out.println(k+" "+l);
}
```

Program reads two numbers and outputs their sum and product. Both numbers are given in same line.

1. Read a string of digits. Let n be its first digit. Find the number formed by next n digits. Output double of the number. Input 4773962 output 7739*2=15478.
2. Write program, which reads two numbers. It prints the bigger number first and smaller later.
3. Write program, which reads a complex number and outputs its modulus. Input 4+3i output 5. [Hint: p=(float)Math.sqrt(q) will find square root of q]
4. Write program, which reads a vector and outputs its magnitude. Input 3i+4j+12k output 13.
5. Read two vectors and print their dot product. Input 5i+8j+0k and 6i–3j+2k output 6.
6. Write program, which behaves in the following manner: Input 12+10 output 22. Input 12-10 output 2.
7. Read a string. It contains a word and two numbers. The word is one of the followings: add, sub, mul, and div. The program outputs the sum of given numbers, if the word is add. If word is sub the program outputs difference and so on. e.g. input mul 12 19 output 228.
8. Write program, which reads a string. Its first number is either 1 or 2. If it is 1 then square of second number is returned. If it is 2 then product of 2nd and 3rd number is returned. Input 1 12 output 144. Input 2 5 12 output 60.
9. Read a float number and output its double. Do not use parseFloat. Assume fraction part has exactly two digits. Input 12.34 output 24.68. Input 862.97 output 1725.94.
10. Read a string. It has 2 or 3 numbers. If it has two numbers then output their product. If it has three numbers then output their sum.
11. Write a program, which reads a number *n* and prints a string with *n* x's. There will be only one print statement. Input 6 output xxxxxx.
12. Read a string of numbers and find 5th number. Input 6 2 6 4 17 8 9 18 output 17.
13. Read a string of 5 numbers and find their sum. Input 6 403 2 5 7 output 423.
14. Read a string of numbers and find first even number. Input 61 3 6 43 17 8 9 18 output 6.

15. Read a string of numbers and find their sum. Input 6 3 2 5 7 output 23. Input 2 5 3 output 10. The number of numbers in the string is unknown.
16. Write program, which deletes all even numbers from the string. Input 12 17 19 6 17 output 17 19 17. [Caution: all numbers should come in same line]
17. Read a complex number and print it in simplified form. Input 7i+12i+9+4i-8i+13 output 22+15i.
18. Read a string. The first word is a number (say k). The program finds k^{th} word in remaining string. e.g. input 3 Ram Hari Gopal Om Ravi output Gopal.
19. Write program, which reads an expression and finds its value. Input (((6+3)*8)–(4*3)) output 60.
20. Write a program, which reads number n and n strings. The program outputs the string, which has the longest length.
21. Write program, which reads a complex number and outputs its modulus. Input 4+3i output 5. Input 4 output 4. Input 3i output 3. Input 4-3i output 5.
22. Read two complex numbers and find their sum. Input 5-7i and 3 output 8-7i.
23. Write program, which will delete maximum number from the string. Assume all numbers are distinct. Try to do it using only one for loop.
24. Write program for deleting maximum number, when numbers are not distinct. If maximum number occurs more than once then first occurrence is deleted. Input 12 17 6 13 17 6 17 9 output 12 6 13 17 6 17 9.
25. Write program, which will delete all occurrences of maximum number.
26. The program reads two strings of numbers. In both the numbers are in increasing order. The program outputs all numbers in increasing order. e.g. if input strings are 6 9 14 20 22 25 and 11 12 15 16 21 then output is 6 9 11 12 14 15 16 20 21 22 25.
27. Do above problem to find k^{th} word from last. Use only one for.
22. Read a string of numbers. Let these are $x_0, x_1, x_2, \ldots, x_{n-1}$. Without using array find the value of $(x_0 - (x_1 - (x_2 \ldots\ldots\ldots - (x_{n-4} - (x_{n-3} - (x_{n-2} - x_{n-1}))) \ldots)))$

Function

```
import java.io.*;
import java.lang.*;
class f
{  public static String rdstring()
   {  String a="";
      try
      { DataInputStream o=new DataInputStream(System.in);
        a=o.readLine();
      }
      catch(IOException e) {}
      return a;
   }
   public static int second(String a,char e)
   {  String b;int p,q;
      p=a.indexOf(e);
      b=a.substring(p+1);
      q=b.indexOf(e);
      return(p+q+1);
   }
   public static String del(String a,int k)
   {  String b,c;
      b=a.substring(0,k)+a.substring(k+1);
      return b;
   }
   public static String delsec(String a,char e)
   {  int p;
      p=second(a,e);
      return del(a,p);
   }
}
class kapil
{  public static void main( String args[])
   { String a,k;int g,m;
     a=f.rdstring();m=Integer.parseInt(f.rdstring());
     g=f.second(a,'x');System.out.println(g);
     k=f.del(a,m);System.out.println(k);
     System.out.println(f.delsec(a,'y'));
   }
}
```

The function second returns the location of second occurrence of e in string a.
The function del removes kth letter.The function delsec deletes the second e.
The first input to this program is a string which has at least 2x's and at least 2y's.The second input is a number.

In following functions at most one loop should be used.

1. Define function String delkth(String a, int k). It returns a string after deleting k^{th} letter. [call of delkth("ramharikumar",4) will return "ramhrikumar".
2. Define function String delfirst(String a). It deletes first 'x' in the string. Do not use + in this function. Do not use loop. [Hint: use delkth function.]
3. Define function String delfirsttwo(String a). It deletes first two x's in the string. Do not use indexOf in this function. Do not use loop or +. [Hint: use delfirst function]
4. Define function String delfirstk(String a, int k). It deletes first k x's in the string. Do not use indexOf or + in this function. [Hint: use delfirst function]
5. Define function String delall(String a). It deletes all x's in the string. Do not use + and substring in this function [Hint: use delfirst function]
6. Define function int loc(String a, int k). It finds location of kth x in the string. Do not use loop.
7. Define function String aft(String a). It finds first letter after every 'x'. aft("axwxthxm") will return "wtm".
8. Define function String mr(String a). It returns "yes" if first letter is present more than once. mr("hari") returns "no". mr("abhay") returns "yes". Do not use loop.
9. Define function String more(String a). It returns those words of string 'a' whose first letter is present in the same word more than once. more("gtygwe kiuip eleet klimp tyutty gtgy") will return "gtygwe eleet tyutty gtgy".
10. Define function int count(String a, String b). Its returns the number of letters of string a present in string b. count("acgcklm","rdgclp") will return 4. Here c, g, c and l are present. a, k and m are not present.
11. Define function String subset(String a, String b). It returns "yes" if every letter of first string is present in second string. subset("hari","iphyearus") returns "yes". subset("klcm","kldm") returns "no". Do not use loop. [Hint: use count]
12. Define function, which returns the string of first letter of every word. first("ram hari om kapil ravi") will return "rhokr".
13. Define function String present(String a, String b). It returns string of those words of string 'a' whose every letter is present in 'b'. present("ram is a good boy","tsoaigkdm") will return "isagood". [Hint: use subset]
14. Define function String presentfirst(String a, String b). It returns string of first letter of those words of string 'a' whose every letter is present in 'b'. presentfirst("ram is a good boy","tsoaigkdm") will return "iag".
15. Define function String prfirst(String a, String b). It returns string of first letter of those words of string 'a' whose every letter after first letter is present in 'b'. prfirst("ram is good boy","asodg") will return "ig".
16. Define function String abcfirst(String a). It returns string of those words of string 'a' whose every letter is first letter of some word. abcfirst("yes ret mrts toy srtk om") will return "mrts toy om". Do not use any loop in this function. [Hint: use first and present]
17. Define function, which returns string of those words, whose first letter is capital. Capital("kapil Ram Gopal sani") will return "RamGopal".
18. Define reverse. Reverse("hari") returns "irah".
19. Define rev. It reverses string a given range. rev("hariprasad",4,8) returns harisarpad.
20. Define function to perform word wise reverse. Rv1("ram is bad") returns "bad is ram".
21. Define function to perform within word reverse. Rv2("ram is bad") returns "mar si dab".

22. shuffle(“abc pqr ghk tih jkl”) returns “apgtj bqhik crkhl”. Assume all words have same length.

Recursion

```
   public static int loc(String a,char e,int k)
   {  String b;int i,j;
      if (k==1) return a.indexOf(e);
      else{ i=a.indexOf(e);b=a.substring(i+1);
            j=loc(b,e,k-1);return(j+i+1); }
   }
   public static String del(String a,int k)
   {   String b,c,d;int i,j;
       if (k==0)  return a;
       else{  i=a.indexOf('x');b=a.substring(i+1);c=a.substring(0,i);
              d=del(b,k-1);return(c+d); }
   }
class kapil
{  public static void main( String args[])
   { String a,k;int g;
     a=f.rdstring();g=Integer.parseInt(f.rdstring());
     System.out.println(f.loc(a,'x',g));
     System.out.println(f.del(a,g));
   }
}
```

The function loc finds the location of kth e in string a. The function del deletes first k x's.

Define following functions without using any loop.

1. Define function int count(String a, char e). It returns the number of e's in the string 'a'. count("wrtkcrygm",'r') will return 2.
2. Define a function String delfirst(String a). It deletes first 'x' in string x. Using it define function to delete (A) first k x's. (B) all x's. In them indexOf should not be used.
3. Define function to return short form. short("Ram Prasad Kumar Dey") returns "R.P.K.Dey". short("shyam singh") returns s.singh.
4. Define function, which returns the string of those words, which has at least one 'x'. abc("ram haxi pxxy wertm xrt") will return "haxi pxxy xrt".
5. Define function, which returns the string of second letter of those words whose first letter is 'x'. ghi("xram mhaxi tpxxy xxwertm xgrt") will return "rxg".
6. Modify the function "loc" (given in example). It returns -1 if string does not have k e's.
7. Define function String subset(String a, String b). It returns "yes" if every letter of first string is present in second string. It returns "no" otherwise.
8. Define function String present(String a, String b). It returns string of those words of string 'a' whose every letter is present in 'b'. present("ram is a good boy","tsoaigkdm") will return "isagood". [Hint: use subset].
9. Define function common. It returns "yes" both strings have got some common letter. common("hari","ram") returns "yes". common("hari","som") returns "no".
10. Define function to remove duplicates. rd("hariram") returns "harim" (or "hiram").
11. Define function, which returns the location of last 'x'. If the string does has any 'x' then -1 is returned.
12. Define function to delete the last 'x'. Assume that string has at least one 'x'.
13. Define function to delete second last 'x'. Assume that string has at least two x's.
14. Define function to find the location of kth last x. If the string does not have k x's then -1 is returned. kthlast("ramxttxharixxkapilxravi",4) will return 6.
15. Define function to find the location of kth last x from last. If the string does not have k x's then -1 is returned. kthlast("ramxttxharixxkapilxravi",4) will return 17.

16. Define function to find the number of letters after last ‘x’. Assume that string has at least one ‘x’. [Hint: To find the length of string a.length() may be used]
17. Define function to delete all x’s after kth x. del(“exrtxxuixojkxr”,3) returns exrtxxuiojkr.
18.

Object

```
import java.io.*;
import java.lang.*;
class point
{  private float x,y;
   public point(float a,float b){x=a;y=b;}
   public point(){}
   public void print() { System.out.println("("+x+","+y+")"); }
   public void abc() {x=2*x;y=3*y;}
   public static point pqr(point a)
                              {point t;t=new point(2*a.x,3*a.y);return t;}
   public void ghi(point a) {x=2*a.x;y=3*a.y;}
   public float getx(){return x;}
}
class abc
{  private float a,b,c;
   public abc(float p,float q,float r){a=p;b=q;c=r;}
   public abc(){}
   public void print() { System.out.println("("+a+","+b+","+c+")"); }
}
class hari
{  public static void main(String args[])
   {   point a,b,c;float t;abc k;
       a=new point(2,4);a.print();a.abc();a.print();
       b=point.pqr(a);b.print();
       a=new point(2,4);c=new point();c.ghi(a);c.print();
       t=a.getx();System.out.println(t);
       k=new abc(2,4,9);k.print();
   }
}
```

In above if point p is (x,y) then p.abc( ) will make it (2x,3y).
If point r is (x,y) then q=point.pqr will make q as (2x,2y). r is unmodified.
If point k is (x,y) then r.ghi(k) will make r as (2x,3y)

In the definition of following the use of local variables should be minimum.

1. Define fa. If point p is (x,y) then p.fa() will make it (x+y,2y). (20,4)→(24,8)
2. Define fb. If point p is (x,y) then p.fb() will make it (2x,x+y). (20,4)→(40,24)
3. Define fc. If point p is (x,y) then p.fc() will make it (x+y,x*y). (20,4)→(24,80)
4. Define fd. If point p is (x,y) then q=p.fd() will make q as (x+y,x*y). [local variable]
5. Define ga. If point p is (x,y) then q=p.ga() will make q as (x+y,2y).
6. Define gb. If point p is (x,y) then q=point.gb(p) will make q as (x+y,2y).
7. Define gb using ga. In its definition + and * should not be used.
8. Define gc. If point p is (x,y) then q.gc(p) will make q as (x+y,2y).
9. Define gc using ga. In its definition + and * should not be used.
10. Define gd. If point p is (x,y) then p.gd(k) will make it (x+k,2y).
11. Define ga, gb and gc using gd. In their definition + and * should not be used.

In the definition of followings the use of getx, gety, geta, etc should be minimum. Care should be taken to choose the class in the definition of following functions.

12. Define faa. If point p is (x,y) then q=point.faa(p) will make q as (x+y,x*y,x−y).
13. Define fab. If point p is (x,y) then q=abc.fab(p) will make q as (x+y,x*y,x−y).

14. Define faa using fab. In its definition +, * and – should not be used.
15. Define fab using faa. In its definition +, * and – should not be used.
16. Define fac. If q is (x,y,z) then p=q.fac() will make p as (x+y,y*z).
17. Define fad. If q is (x,y,z) then p.fad(q) will make p as (x+y,y*z).
18. Define fac and fad with the help of each other (without using +, * and –).
19. In class xyz, define functions f, g and h so that following main program outputs (x,y) (x+y,2x) and (10(x+y)+1,30x+1). In function g only * and in h only + should be used. No other arithematic operatror should be used in them.
    main( ){ xyz a,b; a=new xyz(x,y); a.pt();b=xyz.h(a.f().g());a.pt();b.pt();}
20. Do above problem when in g only *5 and in h only +1 is used.
21. Do above problem without using new in function f.
22. Do above problem without using new in function g.

Object representation-using Class

```
import java.io.*;
import java.lang.*;
class complex
{  float real,img;
   public complex(float a,float b){real=a;img=b;}
   public complex(){}
   public void st(float x,float y){ real=x; img=y; }
   public void print1() { System.out.println(real+"+"+img+"i"); }
   public void duble() { real=2*real; img=2*img;}
   public void ktimes(int k) {real=k*real;img=k*img;}
   private float magsq(){ return real*real+img*img;}
   public float magnitude(){float t;t=(float)Math.sqrt(this.magsq());
return(t);}
   public static void print2(complex a)
         { System.out.println(a.real+" +  "+a.img+"i"); }  or a.print1( )
   public void add(complex x){ real=real+x.real;img=img+x.img; }
   public void add(complex x,complex y)
         { real=x.real+y.real;img=x.img+y.img; }  or { real=x.real; img=x.img;
this.add(y);}
   public complex add3(complex x)
         { complex c;float a,b;c=new complex();
           a=real+x.real;b=img+x.img;c.st(a,b);  or c.add(this,x); or
c=complex.add4(this,x);
           return(c); }
   public static complex add4(complex x,complex y)
         { complex c;float a,b;a=x.real+y.real;b=x.img+y.img;
           c=new complex();c.st(a,b);return(c); }
   public float getreal() { return(real);}
   private float getimaginary() { return(img);}
}
class hari
{  public static void main( String args[])
   {   complex t,s,r,m;float j;
       t=new complex();s=new complex();r=new complex();
       t.st(4,2);t.duble();t.print1();
       t.st(4,2);t.ktimes(10);t.print1();
       t.st(4,3);j=t.magnitude();System.out.println(j);
       t.st(4,2);t.print1();s.print2(t);complex.print2(t);
       s.st(2,7);t.st(4,2);s.add(t);s.print1();
       s.st(2,7);t.st(4,3);r.add(s,t);r.print1();
       s.st(2,7);t.st(4,23);r=s.add3(t);r.print1();
       s.st(2,7);t.st(4,2);r=complex.add4(s,t);r.print1();
       System.out.println(s.getreal());
       m=new complex(23,-45);m.print1();
   }
}
```

Since getreal( ) is public s.getreal( ) can be written in class hari. However s.getimaginary( ) can not be written.

s.add(t) means s+=t; r.add(s,t) means r=s+t; Two definitions show function overloading.

r=s.add3(t) means r=s+t; Same meaning is with add4.

In add4 current object (real and img) is not used hence it is declared static. In calling add4 the name of class (not that of object is passed).

To implement a=a*b as a.mul(b) a temporary variable is needed.
a=b*c can be implemented as a=b.mul(c) or a=complex.mul(a,b) without using temporary variable. But when it is implemented as a.mul(b,c) a temporary variable should be used otherwise a=a*c will give wrong result.

Copy

```
import java.io.*;
import java.lang.*;
class complex
{  private float r,i;
   public void pt() { System.out.println(r+"+"+i+"i"); }
   public void st(float x,float y){ r=x; i=y; }
   public complex cpy(){return this;}
   public void cqy(complex k) { r=k.r;i=k.i; }
   public complex cry()
   { complex c;float x,y;c=new complex();
     x=r; y=i;c.st(x,y);                    or  c.cqy(this);
     return(c); }
   public void aaa(complex a, int b, complex c)
   {   a=this;b=12;c.r=r*9; }
}
class hari
{  public static void main( String args[])
   {   complex t,s,r,a,b;int k,m;
       a=new complex();b=new complex();a.st(5,2);b=a;b.st(7,9);a.pt(); b.pt();
```

Let us differentiate between copy and assignment. In assignment only pointer is copied. Hence when b is changed, a is also changed. It does not happen (see below) when copy is done. The copy can be defined by many ways. The following shows difference.

Replace b=a by b=a.cpy( ) etc.

Statement	a	B
b=a	7+9i	7+9i
b=a.cpy( )	7+9i	7+9i
b.cqy(a)	5+2i	7+9i
b=a.cry( )	5+2i	7+9i

```
       r=new complex();t=new complex();
       s=r;r.st(9,3);t.st(5,7);s=t;s.st(1,2);s.pt();t.pt();r.pt();
```

Replace s=t by s=t.cpy() etc.

Statement	s	t	r
s=t	1+2i	1+2i	9+3i
s=t.cpy( )	1+2i	1+2i	9+3i
s.cqy(t)	1+2i	5+7i	1+2i
s=t.cry( )	1+2i	5+7i	9+3i

```
       s.st(2,3);t.st(2,3);if (s==t) m=0; else m=1; System.out.println(m);
```

Even when complex numbers s and t are both 2+3i their comparison shows unequal. It is because s and t are pointing at different locations. Hence the value of m is 1.

```
       s.st(2,3);t.st(4,5);r.st(6,7);k=231;s.aaa(t,k,r);
       s.pt();t.pt();r.pt(); System.out.println(k);
```

Call to function aaa helps to understand that parameter passing is call by value. t and k remain unchanged while r is modified. k remains unchanged because b (int) is in build object. Its value (not pointer) is passed. t remains unchanged because only pointer of ‘a’ is modified. r is changed because contents of object are modified.

```
   }
}
```

1. Define a class vector. Define dotproduct of vectors so that a=v.dot(w) computes the dot product of vectors v and w. Dotproduct of 2**i**+3**j**+1**k** and 5**i**+2**j**+3**k** is 19.
2. Define magnitude such that a=v.mag() computes the magnitude of vector v. To compute the magnitude, use the function of dot product defined above. Magnitude 4**i**+3**j**+12**k** is 13. [Hint: a=(float)Math.sqrt(b) can be used to find square root of b]
3. Define cross product such that v.cross(w,h) performs v=wxh. Also define suitable function for printing a vector. Cross product of 2**i**+3**j**+1**k** and 5**i**+2**j**+3**k** is 7**i**–**j**–11**k**.
4. Define divide such that v.divide(float k) divides vector v by k. Let vector v be 4**i**+3**j**+12**k** then after v.divide(2) it will become 2**i**+1.5**j**+6**k**.
5. Define unit vector such that v.unit() will modify the vector v so that it becomes a vector of unit magnitude in the same direction. Use the functions magnitude and divide defined above. Unit vector for 4**i**+3**j**+12**k** is 0.31**i**+0.23**j**+0.92**k**.
6. Define another class point. In that class define function to compute mid point of two given points. p=point.mid(q,r) should find p as mid point of q and r. Define suitable function to print point.
7. Define function direction such that v=point.dir(p,q) finds vector v in the direction of given points p and q. If points are (2,3,1) and (6,2,8) then direction is 4**i**-**j**+7**k**.
8. Define function direction such that v.direction(p,q) will find vector v. Use above function.
9. Define function to compute area. a=point.area(p,q,r) should find the area of triangle PQR. [Find direction corresponding to PQ and PR. Now call cross product and magnitude]. Area of the triangle with end points (3,4,5), (7,3,6) and (6,2,3) is 6.36.

10. In a separate file define class point (in 2 dimensional space, components x and y). Now define a function to find mid point. p.mid(q) will find p as mid point of p and q. Define suitable function to print point.
11. In the same file define another class line. It has 3 components a, b and c. ax+by+c=0 is equation of a line. Define function to find equation of a line. g.eq(p,q) will find the equation of the line (g) passing through p and q. Define suitable function to print the equation of line. Let P=(2,3) and Q=(4,7) then line is 2x-y-1=0. Hence a=2 b=–1 and c=–1. The line may be printed as 2x+-1y+-1=0.
12. Define intersection. p.intersection(g,h) finds the point of intersection of the line g and h. Input 2x+3y-26=0 and 4x+7y-54=0 output (10,2).
13. Define intersection. p.intersection(a,b,c,d) finds the point of intersection of line joining points a and b with the line joining points c and d. Use above function. Let input points be (4,6), (-2,10), (10,2) and (3,6) then the point of intersection is (10,2).
14. Define distance. k=line.distsance(p,l) finds the distance of the point ‘p’ from the line ‘l’. e.g. input point (2,3) and line 8x+6y-12=0 distance = $(8*2+6*3-12)/(8^2+6^2)^{1/2}$ = 2.2.
15. In the same file define another class circle. It has two components c and r. c(point) is centre of the circle. r(float) is radius of the circle. Define a function to print the equation of the circle. Let there is a circle (x) whose radius is 7 and centre is (2,3) then x.print() will print x^2+y^2-4x-6y-36=0. [It may be printed as x^2+y^2+-4x+-6y+-36=0]
16. Define a function form. a.form(g,h,k) will form a circle whose equation is x^2+y^2+gx+hy+k=0. a.form(-4,-6,-36) will make centre and radius of circle ‘a’ as (2,3) and 7 respectively. Define a function to print the circle. a.pt() will print centre and radius of the circle ‘a’.

17. Define a function chord. a.chord(b) will return the chord length. Here ‘a’ is a circle and ‘b’ is a line. If a circle has centre (1,2) and radius 5 and the line is 5x+12y+10z=0 then chord length is 8. [Method: find d=line.distance(c,b) and use $(r^2-d^2)^{1/2}$.]
18. Define a function intersection. y=s.intersection(b) returns the y-coordinate of point of intersection of circle (s) and line (x=b). Let a circle (s) has centre (7,2) and radius 5. y=intersection(s,10) will make y as 6 (or -2). It is because circle $(x-7)^2+(y-2)^2=5^2$ intersects with the line x=10 at points (10,6) and (10,-2).

Link List

```
import java.io.*;
import java.lang.*;
class node
{   float data;node next;
    public node(){}
    public node(float a, node b){ data=a;next=b;}
    public float second()
    {   return next.data;}
```

Ordinary

```
    public void pt()
    {   node I;
        i=this;
        while(i!=null)
        {  System.out.print(i.data+" ");
           i=i.next;
        }
        System.out.println(" end");
    }
    public void insertafter(int x,int y)
    {   node I,j;
        i=this;
        while(i.data!=x)
            i=i.next;
        j=new node();
        j.data=y;
        j.next=i.next;
        i.next=j;
    }
    public void deleteafter(int x)
    {   node I;
        i=this;
        while(i.data!=x)
            i=i.next;
        i.next=i.next.next;
    }
    public node del(int x)
    {  node I;
       if(data==x)
            return(next);
       else
       {    i=this;
            while(i.next.data!=x)
                i=i.next;
            i.next=i.next.next;
            return(this);
        }
     }
```

Recursion

```
    public void pt()
    {   if(next!=null)
        {   System.out.print(data+" ");
            next.pt();
        }
        else
            System.out.println(data+" end");
    }

    public void insertafter(int x,int y)
    {   if(data==x)
            next=new node(y,next);
        else
            next.insertafter(x,y);
    }

    public void delafter(int x)
    {   if(data==x)
             next=next.next;
         else
             next.delafter(x);
    }

    public node del(int x)
    {    node k;
         if(next.data==x)
         {    next=next.next;
              return(this);
         }
         else if(data==x)
             return(next);
         else
         {    k=next.del(x);
              return(this);
         }
    }
```

```
public void form(int n)
    {   node I;
        String a;
        try
        {   DataInputStream o=new DataInputStream(System.in);
            System.out.print( " Give "+n+" more elements " );
            a=o.readLine();
            data=Integer.parseInt(a);
            if(n==1)
                next=null;
            else
            {   i=new node();
                i.form(n-1);
                next=I;
            }
        }
        catch(IOException e)
        {   System.out.println(e); }
    }
}
class hari
{   public static void main(String args[])
    {   String a;
        node head,head1;
        int n,x,y;
        float t;
        try
        {   DataInputStream o=new DataInputStream(System.in);
            System.out.print(" How many elements the link list has");
            a=o.readLine();n=Integer.parseInt(a);
            head=new node();
            head.form(n);head.pt();
            System.out.println("The second element"+head.second());
            System.out.print(" After which number you want to insert ");
            a=o.readLine();
            x=Integer.parseInt(a);
            System.out.print(" What  number you want to insert ");
            a=o.readLine();
            y=Integer.parseInt(a);
            head.insertafter(x,y);
            head.pt();
            System.out.print(" After which number you want to delete ");
            a=o.readLine();
            x=Integer.parseInt(a);
            head.deleteafter(x);
            head.pt();
            System.out.print(" Which number you want to delete ");
            a=o.readLine();
            x=Integer.parseInt(a);
            head=head.del(x);
            head.pt();
            System.out.print(" Which number you want to delete ");
        }
        catch(IOException e){   System.out.println(e); }
    }
}
```

```
import java.io.*;
import java.lang.*;
class node
{   float data;node next;
    public node(){}
    public node(float a, node b) { data=a;next=b;}
    public void pt()
    {   node i;i=this;
        while(i!=null)
        {   System.out.print(i.data+" ");
            i=i.next;
        }
        System.out.println(" end");
    }
    public void inaf(int x,int y)
    {   node i,j;i=this;
        while(i.data!=x) i=i.next;
        j=new node();j.data=y; j.next=i.next; i.next=j;
    }
    public void delaf(int x)
    {   node i;i=this;
        while(i.data!=x) i=i.next;
        i.next=i.next.next;
    }
    public node del(int x)
    {  node i;
       if(data==x) return(next);
       else
       {    i=this;
            while(i.next.data!=x)i=i.next;
            i.next=i.next.next;
            return(this);
        }
     }
}
class kapil
{  public static void main( String args[])
   { String a,b="";int i;
     node head,head1,m;int n,x,y;float t;
     head=null;
     do{
       try{ DataInputStream o=new DataInputStream(System.in);
            b=o.readLine();
       if (b.compareTo("put")==0)
       {    System.out.print("Give number to be inserted at end");
            a=o.readLine();x=Integer.parseInt(a);
            m=new node();m.data=x;m.next=null;
            if (head==null) head=m;
            else {j=head;while (j.next!=null) j=j.next; j.next=m;}
       }
       if (b.compareTo("put")==0)
       {    System.out.print("Give number to be inserted at end");
            a=o.readLine();x=Integer.parseInt(a);
            m=new node();m.data=x;m.next=null;
            if (head==null) head=m;
            else {j=head;while (j.next!=null) j=j.next; j.next=m;}
       }
```

```
        if (b.compareTo("insertafter")==0)
        {    System.out.print(" After which number you want to insert ");
             a=o.readLine();x=Integer.parseInt(a);
             System.out.print(" What  number you want to insert ");
             a=o.readLine(); y=Integer.parseInt(a);head.inaf(x,y);
        }
        if (b.compareTo("delete")==0)
        {    System.out.print(" Which number you want to delete ");
             a=o.readLine();x=Integer.parseInt(a);head=head.del(x);
        }
        if (b.compareTo("print")==0) head.pt();
        }catch(IOException e) {}
      }while (b.compareTo("exit")!=0);
    }
}
```

Doubly Linked List

```
import java.io.*;
import java.lang.*;
class node
{   float data;node next,prev;
    public void pt()
    {   node i,j;
        System.out.print(data+" ");
        i=next;j=i;
        while(i!=null)
        {   System.out.print(i.data+" ");
            j=i;i=i.next;
        }
        System.out.print(" rev");
        while(j!=null)
        {   System.out.print(j.data+" ");
            j=j.prev;
        }
        System.out.println(" end");
     }
     public void form(int n)
     {  node i;
        try
        {   DataInputStream o=new DataInputStream(System.in);
            System.out.print("Give "+n+" more elements");
            String a=o.readLine();data=Integer.parseInt(a);
            if(n==1) {next=null;prev=null;}
            else
            {   i=new node();
                i.form(n-1);
                next=i;prev=null;i.prev=this;
            }
        }
        catch(IOException e)
        {   System.out.println(e); }
     }
}
class hari
{   public static void main(String args[])
    {   node head;int n;String a;
        try
        {   DataInputStream o=new DataInputStream(System.in);
            System.out.print(" How many elements the link list has");
            a=o.readLine();n=Integer.parseInt(a);
            head=new node();
            head.form(n);
            head.pt();
        }
        catch(IOException e)
        {   System.out.println(e); }
    }
}
```

Exception

```
import java.io.*;
import java.lang.*;
class xyz
{ public static void main (String args[])
  { String a,b;char c; a="abc";
    try{ b=a.substring(2,5); System.out.println(b);
         c=a.charAt(9);System.out.println(c);
       } catch (Exception t){ System.out.println("The exception is"+t); }
    System.out.println("Hari");
  }
}
```

The above program outputs "Hari" irrespective of input string 'a'. When input is abc or abcdef then the exception is java.lang.StringIndexOutOfBounds. In case of "abc" no additional output is produced. In case of "abcdef" output "cde" is produced. When input is abcedfghijkl then there is no exception. When try-catch is removed then program terminates when input is abc or abcedf. In case of "abc" no output is produced. In case of "abcdef" output "cde" is produced. Because of try-catch the impact of exception is felt only within the block. The normal execution is carried out after that.

```
class xyz
{
  public static void main (String args[])
  {   int a,b,c,d,e,f;a=25;b=4;c=0;d=3;
      e=a/b;System.out.println(e);
      e=a/c;System.out.println(e);
      e=a/d;System.out.println(e);
      System.out.println("Hari");
  }
}
```

The above program terminates after outputting 6.

```
Class xyz
{
  public static void main (String args[])
  {   int a,b,c,d,e,f;a=25;b=4;c=0;d=3;
      try{
        e=a/b;System.out.println(e);
        e=a/c;System.out.println(e);
        e=a/d;System.out.println(e);
       }
      catch (ArithmeticException t){System.out.println("mistake");}
      System.out.println("Hari");
  }
}
```

The above program outputs 6 mistake hari

```
Class xyz
{
  public static void main (String args[])
  {   int a,b,c,d,e,f;
      a=25;b=4;c=0;d=3;
      try{e=a/b;System.out.println(e);}
         catch (ArithmeticException t){System.out.println("Error");}
```

```
      try{e=a/c;System.out.println(e);}
         catch (ArithmeticException t){System.out.println("Error");}
      try{e=a/d;System.out.println(e);}
         catch (ArithmeticException t){System.out.println("Error");}
  }
}
```

The above program outputs 6 error and 8.

```
Class xyz
{ private static int read1()
  { int i=2,j;String a;
    try{
        DataInputStream o=new DataInputStream(System.in);
        a=o.readLine();i=Integer.parseInt(a);}
    catch (Exception t){ System.out.println("Not a number"+t); }
    return i;
  }
  public static void main (String args[])
  {   int i,a,f[]={8,12,34,16};
      a=xyz.read1();
      try
      {  System.out.println(f[a]);
         i=25/0;
      }
      catch (Exception t){ System.out.println("stop"+t);}
      /*catch (ArrayIndexOutOfBoundsException t)
          {System.out.println("Illegal Memory reference"+t);}
      catch (ArithmeticException t)
          {System.out.println("Wrong Division"+t);}*/
  }
}
```

input 2 Output 34 stop java.lang.ArithmeticException: / by zero

input 7 Output stop
java.lang.ArrayIndexOutOfBoundsException: 7

input hari Output Not a number java.lang.NumberFormatException
34 stop java.lang.ArithmeticException: / by zero

For this input initialize i as 3 and 5 in function read1.

When catch exception is removed and other two catches are activated then for input 2 the output is "wrong division". For input 7 the output is "Illegal Memory Reference".

When all three catches remain then program refuses to compile. It is because catch exception suppresses other two exceptions. However, when catch exception is written at the last, the program compiles. When catch arithmetic exception and catch exception is written then for input 2 the output is wrong division. For input 7 the output is "Stop".

```
Class xyz
{
  public static void main (String args[])
  {   int a,b,c,d,e,f; a=25;b=4;c=0;d=3;
      try{
        e=a/b;System.out.println(e);
        e=a/c;System.out.println(e);
        e=a/d;System.out.println(e);
     }
     finally{System.out.println("Continue");}
     e=a/d;System.out.println(e);
  }
}
```

The above program outputs 6 Continue.

When c=0 is replaced by c=10 then output is 6 2 8 Continue 8.

When “finally” is replaced by “catch(Exception t)” then output is 6 Continue 8 when c is 0.
When “finally” is replaced by “catch(Exception t)” then output is 6 2 8 8 when c is 10.
“Finally” means in case of exception the program will execute the corresponding statement. The program will terminate after that. In case of “catch exception” the program continues after that. When exception does not occur then the statement in finally is executed. But statement is catch exception is not executed.

```
class kapil
{  public static void main( String args[])
   { int i;float x,y,k,l;String a="",b;
     try
     { DataInputStream o=new DataInputStream(System.in);
       a=o.readLine();a=a.trim();
     }catch(IOException e){ }
     i=a.indexOf(" ");b=a.substring(0,i);
     try{x=Float.parseFloat(b);}catch(Exception e){x=5;}
     b=a.substring(i+1);y=Float.parseFloat(b.trim());
     k=x+y;l=x*y;System.out.println(k+" "+l);
   }
}
```

Program reads two numbers and outputs their sum and product. If first number is wrong then it is taken as 5.

```
k=Integer.parseInt(a);k=k%2;
try{p="ram";p=p.substring(k+3);
    System.out.println(a+" is an even number");
}catch(Exception m){System.out.println(a+" is an odd number");}
```

Program reads a number and finds whether it is even or odd.

1. Read a string. If it is a number then output its double otherwise output 0. Input 12 output 24. Input 3.9 output 7.8. Input "12 14" output 0. Input ram output 0.
2. Read a number. If it a integer then output its double. If it is a float then output its triple. Do not use indexOf or if.
3. Read a string. If it is a number then output "AC". Otherwise output "BC". You should use only 3 print statements. Every print statement should print only one letter.
4. Modify above program: in case of number output ABC. Otherwise output "B". [Hint: use finally]
5. Read a string. It has 1 or 2 numbers. If it has one number the output its square. If it has two numbers then output their sum. Do not use if.
6. Read a string. It has 2 or 3 numbers. If it has two numbers then output their product. If it has three numbers then output their sum. Do not use if.
7. Read a string of two (distinct) integers. Output bigger integer. Do not use "if".
8. Read a string of two (distinct) numbers (floats). Output bigger number. Do not use "if".
9. Read a string of two numbers. Output 1 if first number is bigger. Output 2 if second number is bigger. Output 0 if they are equal. Do not use "if".
10. Read a string. If it is a number then output 0. Otherwise let 'p' be the 0th letter. If p is not an integer then output 1. If p is an integer and size of the string is less than p+1 then output 2. Otherwise if string becomes integer after deleting p^{th} letter then output 3. Otherwise output 4. $157 \rightarrow 0$ $r23er \rightarrow 1$ $5123a \rightarrow 2$ $332a67 \rightarrow 3$ $2r7 \rightarrow 4$ $33abc45 \rightarrow 4$.
11. Read a number and output its double. Do not use parseFloat. Assume fraction part has at most one digits. Input 12.3 output 24.6. Input 218 output 436. Do not use if.
12. Read a string of two numbers (words) and output their sum. If first word is not a number then it is taken as 5. If second word is not a number then it is taken as 7. Input 12 34 output 46. Input ram 30 output 35. However if both are not numbers then output is 100. Do not use if.
13. Write program, to find number of integers in a string. Input rat 12 3.0 34 om output 2.

14. Write program, which reads a string of digits and finds how many digits are more than 5. Input 2145835769 output 3. Do not use if. Use only ArrayIndexOutOfBoundsException.
15. Write program, which reads a string of digits and finds how many digits are equal to 5. Input 2145835769 output 2. Do not use if. Use only ArrayIndexOutOfBoundsException.
16. Write program, which reads a string. It finds how many of them are integers and how many of them are floats. E.g. Input “2 ram 4 10.3 20 4.78 20 3” output 5,7.

Write following program without using length function, “if” and any loop except while(1==1).

17. Write program, which reads a string and find its length.
18. Write program, which finds number of words in a string.
19. Given a string of integers, find program to find how many of them are more than 15.

Abstract Data Type

```
class stack
{  private String x[]=new String[10];int sp;
   public stack(){int sp=0;}
   public void put(String e) {x[sp]=e;sp++;}
   public String take() {sp--; return x[sp];}
   public String top() {return x[sp-1];}
   public String empty()
                  {if (sp==0) return "empty"; else return "not empty";}
   public int size() { return sp;}
   public void pt(){int I;for (i=0;i<sp;i++) System.out.print(x[i]+" ");}
}
class kapil
{  public static void main( String args[])
   { String a="",b;int I;stack x;
     x=new stack();
     do{
       try
       { DataInputStream o=new DataInputStream(System.in);
         a=o.readLine();
       }
       catch(IOException e) {}
       i=a.indexOf(' ');
       if (i!=-1) b=a.substring(0,i);else b=a;
       if (b.compareTo("put")==0)x.put(a.substring(i+1));
       if (b.compareTo("take")==0) System.out.println("Element"+x.take());
       if (b.compareTo("top")==0) System.out.println("First ele"+x.top());
       if (b.compareTo("empty")==0) System.out.println("System is empty");
       if (b.compareTo("size")==0) System.out.println(x.size()+" ele");
       if (b.compareTo("print")==0) {x.pt();System.out.println();}
     }while (b.compareTo("exit")!=0);
   }
}
```

1. Add one more operation "initialize". After that the stack becomes empty.
2. Define Queue using two stacks,
3. Define priority system using two stacks. The element taken is biggest of existing elements.
4. Define queue using front and rear.
5. Define stack using two queues. An element is put in stack by putting in first queue. An element is taken from stack by taking last element of first queue. Another queue acts as auxiliary queue.
6. Define stack using single queue.
7. Using queue define priority system. In that element taken is the biggest of existing elements.
8. Define stack using priority system. Assume all elements are positive. Do not use empty( ).
9. Define non abstract data type "sequence". Its operations are put(e,k), take(k), empty and print. After operations put(12,1) put(37,2) put(41,1) put(67,2) put(95,3) take(4) then sequence will look as 41,67,95,37. The element taken will be 12.
10. Define abstract data type sequence using stack.

Inheritance

```
import java.io.*;
import java.lang.*;

class d2vectors {  float i,j;
    public d2vectors(float i1,float j1){  i=i1; j=j1;    }
    public float slope(){return (j/i); }
    public float magsqr(){return (i*i+j*j);}
    public void dble(){i=2*i;j=2*j;}
    public void pt(){System.out.println(i+"i+"+j+"j");}
    public float geti(){return (i);}
    public float getj(){return (j);}
}

class d3vectors extends d2vectors{float k;
    public d3vectors(float i1, float j1, float k1){super(i1,j1);k=k1; }
    public float magsqr(){return(super.magsqr()+k*k);}
    public void ttt(){super.dble();k=k*3;} //Remove super
    public void dble(){i=2*i;j=2*j;k=2*k;}
    public double direction(){return (k/Math.sqrt(i*i+j*j));}
    // public d2vectors project( ){return super;}
    // super can not be used like keyword "this"
    public void pt(){System.out.println(i+"i+"+j+"j+"+k+"k");}
}

class xyz{
      public static void main(String ar[])
      { d2vectors a,c;double t;d3vectors b,d;
        b=new d3vectors(6,8,7);c=new d2vectors(6,7);
        t=c.magsqr();System.out.println(t);
        t=b.magsqr();System.out.println(t);
        t=b.slope();System.out.println(t);
        t=b.direction();System.out.println(t);
        c.pt();b.pt();b.dble();b.pt();b.ttt();b.pt();
      }
}
```

1. Define class stack { String x[]; int sp; ….put take top print empty size …}
2. Define class queue extends stack{ int front; …. take top print empty size ….} Do not define "put". The program should be efficient.
3. Define class queue extends stack{ …. put….} Do not define other operations. The program may be inefficient because of shifting.
4. Define class priority_system extends stack{ ….take….} Element taken is biggest element. Do not define other operations. Do not do shifting.
5. Define class priority_system extends stack{ ….put….}. Do not define other operations. [Hint: use shifting]
6. Define class queue in a normal manner using front and rear.
7. Define class priority_system extends queue{ ….put….}. Do not define other operations
8. Define class priority_system extends queue{….take….}. Do not define other operations.
9. Define class stack extends queue{ ….put….}. Do not define other operations
10. Define class stack extends queue{ ….take….}. Do not define other operations

Interface

```
import java.io.*;
import java.lang.*;
interface picture { float area (); }
class rectangle implements picture
{  float p,q;
   public rectangle(float a, float b){p=a;q=b;}
   public float area (){ return p*q;}
}
class circle implements picture
{  float r;
   public circle(float a){r=a;}
   public float area () {return (float)3.14*r*r;}
}
class xyz
{  public static void main( String args[])
    {  picture p;float a,b;int i;String s,t,u,v;
       try{
        DataInputStream o=new DataInputStream(System.in);
        s=o.readLine();s=s+" 10 ";
        i=s.indexOf(' ');t=s.substring(0,i);
        v=s.substring(i+1);i=v.indexOf(' ');u=v.substring(0,i);
        s=v.substring(i+1);i=s.indexOf(' ');s=s.substring(0,i);
        a=Float.parseFloat(u.trim()); b=Float.parseFloat(s);
        if (t.compareTo("circle")==0) p=new circle(a);
        else p=new rectangle(a,b);
        a=p.area();
        System.out.println("area="+a);
      }
      catch (IOException e){ System.out.println(e);}
   }
}
```

Give input circle 4 and rectangle 3 7

1. Define class stack { private String x[]; int sp; ….put take top print empty size …}
2. In same file Define class queue { private String x[]; int front, rear; ….put take top print empty size …}
3. Write main program which reads a word. If it is “stack” then remaining operations are according to stack. If it is “queue” then remaining operations are according to queue.

```
import java.io.*;
import java.lang.*;
interface abc {
   public void put(String e);
   public String take();
   public String top() ;
   public String empty() ;
   public int size() ;
   public void pt() ;
}
class stack implements abc
{  private String x[]=new String[10];int sp;
   public stack(){int sp=0;}
   public void put(String e) {x[sp]=e;sp++;}
   public String take() {sp--; return x[sp];}
   public String top() {return x[sp-1];}
   public String empty() {if (sp==0) return "empty"; else return "not";}
   public int size() { return sp;}
   public void pt(){int i;for (i=0;i<sp;i++) System.out.print(x[i]+" ");}
}
class queue implements abc
{  private String x[]=new String[10];int front,rear;
   public queue(){int front=0;rear=0;}
   public void put(String e) {x[rear]=e;rear++;}
   public String take() {front++; return x[front-1];}
   public String top() {return x[front];}
   public String empty() {if (front==rear) return "E"; else return "N";}
   public int size() { return rear-front;}
   public void pt()
        {int i;for (i=front;i<rear;i++) System.out.print(x[i]+" ");}
}
class kapil
{  public static void main( String args[])
   { String a="",b;int i;abc x;x=new stack();
     do{
       try
       { DataInputStream o=new DataInputStream(System.in);
         a=o.readLine();
       }
       catch(IOException e) {}
       i=a.indexOf(' ');
       if (i!=-1) b=a.substring(0,i);else b=a;
       if (b.compareTo("put")==0)x.put(a.substring(i+1));
       if (b.compareTo("stack")==0)x=new stack();
       if (b.compareTo("queue")==0)x=new queue();
       if (b.compareTo("take")==0) System.out.println(x.take()+" taken");
       if (b.compareTo("top")==0) System.out.println("first "+x.top());
       if (b.compareTo("empty")==0) System.out.println(+x.empty());
       if (b.compareTo("size")==0)
            System.out.println("The system has "+x.size()+" elements");
       if (b.compareTo("print")==0) {x.pt();System.out.println();}
     }while (b.compareTo("exit")!=0);
   }
}
```

Visibility

```
import java.io.*;
import java.lang.*;
class abc
{  int a;public int b;private int c;   int d;
   public void set(int p, int q,int r,int s) {a=p;b=q;c=r;d=s;}
   protected void pt(){System.out.println(a+" "+b+" "+c+" "+d);}
}
class pqr
{  public static void main(String args[])
   { abc x;
     x=new abc();
     x.set(2,3,4,5);
     x.pt();
     System.out.println(x.a+" "+x.b+" "+/*x.c+*/" "+x.d);
     //error x.c is not visible
   }
}
class xyz extends abc
{  public static void main(String args[])
   { abc x;
     x=new abc();
     x.set(2,3,4,5);
     x.pt();
     System.out.println(x.a+" "+x.b+" "+/*x.c+*/" "+x.d);
     //error x.c is not visible
   }
}
```

Errors

```
1. Uninitilized variable
import java.io.*;
import java.lang.*;
class xyz
{  public static void main(String args[])
   {  int i,a=4;
      if (a>3) i=5;
      System.out.println(i);
   }
}
```

Replace a>3 by 4>3 and 2>3 seperately.

Thread

```
class xyz implements Runnable
{   public void run()
    {   int i;
        for (i=1;i<5;i++)
        {   System.out.print(i);
            try{Thread.sleep(1000);}catch(Exception e){}
        }
    }
}
class kapil
{
    public static void main(String ar[])
    {   xyz k;Thread a,b;
        k=new xyz();a=new Thread(k);b=new Thread(k);
        a.start();b.start();
        System.out.print("X");
    }
}
```

The above program outputs X1**1**2**2**3**3**4**4** (or 1X**1**2**2**3**3**4**4** or 1**1**X2**2**3**3**4**4)** When a.start( ) is replaced by a.run( ) then the output is 1234X**1234**. When b.start( ) is replaced by b.run( ) then the output is 1**1**2**2**3**3**4**4**X.

```
class xyz implements Runnable
{   int k;
    xyz(int g) {k=g;}
    public void run()
    {   int i;
        for (i=k;i<k*2;i++)
        {   System.out.print(i);
            try{Thread.sleep(1000);}catch(Exception e){}
        }
    }
}
class kapil
{
    public static void main(String ar[])
    {   xyz x,y;Thread a,b;
        x=new xyz(5);a=new Thread(x);
        y=new xyz(3);b=new Thread(y);
        a.start();b.start();
    }
}
```

The above program outputs 5**3**6**4**7**5**89.

```
class xyz implements Runnable
{   public void run()
    {   int i;
        for (i=1;i<6;i++)
        {   System.out.print(i);
            try{Thread.sleep(1000);}catch(Exception e){}
        }
    }
}
class pqr implements Runnable
{   public void run()
    {   xyz t;Thread c;
        System.out.print("X");
        try{Thread.sleep(3000);}catch(Exception e){}
        System.out.print("Y");
        t=new xyz();c=new Thread(t);c.start();
    }
}
class kapil
{
    public static void main(String ar[])
    {   xyz k;pqr g;Thread a,b;
        k=new xyz();a=new Thread(k);
        g=new pqr();b=new Thread(g);
        a.start();b.start();
    }
}
```

The above program outputs 1**X**23**Y**1425**345**

```
class xyz implements Runnable
{   int k,count=0;;
    xyz(int g) {k=g;}
    public void run()
    {   xyz h;Thread b,c;
        try{Thread.sleep(k*1000);}catch(Exception e){};
        if (k>1)
        {   h=new xyz(k-1);
            b=new Thread(h);c=new Thread(h);
            b.start();c.start();
        }
        count++;
        System.out.print("["+k+","+count+"]");
    }
}
class kapil
{
    public static void main(String ar[])
    {   xyz x;Thread a;
        x=new xyz(5);a=new Thread(x);
        a.start();
    }
}
```

```
class f implements Runnable
{  public void run()
   {  int i;
      for (i=1;i<10;i++)
      {   System.out.print(i);
          try {Thread.sleep(2000);}catch(Exception e){}
      }
   }
}
class g implements Runnable
{  public void run()
   {  int i;
      for (i=65;i<=73;i++)
      {   System.out.print((char)i);
          try {Thread.sleep(1000);}catch(Exception e){}
      }
   }
}
class kapil
{  public static void main(String ar[])
   {  f x;g y;Thread a,b;
      x=new f();a=new Thread(x);
      y=new g();b=new Thread(y);
      a.run();b.start();
      try {Thread.sleep(3000);}catch(Exception e){}
      System.out.print("ravi");
   }
}
```

The output is 123456789ABCraviDEFGHI

File

```
import java.io.*;
class hari
{
    public static void main(String arguments[])
    {
        String temp="",fileName="st";
        try{
          FileReader fr= new FileReader(fileName);  open a new file stream
          BufferedReader br= new BufferedReader(fr);   wrap it in a buffer
          while((temp=br.readLine())!=null)
              System.out.println("> "+temp);
          fr.close();
        }
        catch(Exception e)
        { System.out.println("Error is "+e); }
    }
}
```

The above program reads from file whose name is "st"

```
String fileName = "abc";
BufferedWriter out = new BufferedWriter(new FileWriter(fileName),8);
out.write("Gopal is good");
out.close();
```

The above program writes into a file whose name is "abc"

1. Write program, which reads 10 numbers from a file and outputs their sum.
2. Write program, which reads a file name (from key board) and in that file writes ABCD..Z.
3. Write program, which reads a string. The string has a word and a number. Let the string be “abc 7”. In file abc1 1,2,3,..,10 is written. In file abc2 2,4,6,..,20 is written. …… In file abc7 7,14,21,..,70 is written. Assume that the number of the string is less than 7.
4. Define function read(x,y). Here x is a string and y is a number. It returns y^{th} letter (char) of the file whose name is x. Assume that file x has only one string. Let contents of a file “abc” are “hariisgood”. The function call t=read(“abc”,6) will make t=’g’. Call read(“abc”,0) will return h.
5. Define function write(x,y,z). Here x is a string, y is a number and z is a char. It modifies y^{th} letter of the file (name x) as z. Assume that the file x has only one string. In the context of above file the system call write(“abc”,4,’m’) will modify the file contents as “harimsgood”.
6. Define functions move(x), read(), write(y). Here x is a number and y is a letter. All the functions are in the context of a file whose name is “abc”. The function move(x) will move the file pointer at x^{th} location. The read() will return the letter pointed by the file pointer. The function write(y) will write the letter y at position pointed by the file pointer. Let the contents of file “abc” are “hariisgood” then move(5);t=read();move(7);write(t) will make the file contents as “hariisgsood”
7. Define function seek(x,y). Here x and y are numbers. y is either 0, 1 or 2. seek(x,0) is equivalent to move x. seek(x,2) moves file pointer at x^{th} location from last. seek(x,1) advances file pointer by x letters. seek(7,2); seek(3,1); t=read() will make t=s.

8. Using function seek and read write program to print the contents of a file in opposite order.
9. Modify read and write functions. Here file pointer is advanced after reading and writing. Hence first 3 letters of a file can be read as move(0);p=read();q=read();r=read();
10. Using read, write and seek write program to print file contents.
11. Using function seek and read write program to print the contents of a file in opposite order.
12. Using read, write and seek write program to increment every letter of file. Assume that letter z is not present. Let initial file contents are “hariisgood” then final file contents will be “ibsjjthppe”.
13. Define function seek(s,x,y), read(s), write(s,z). s is string, x is number, y is 0, 1 or 2, z is char. ‘s’ is name of file. Corresponding operations are done in file ‘s’

Graphics

```
import java.awt.*;
import java.io.*;
import java.lang.*;
import javax.swing.*;
import java.applet.*;
public class gr extends Applet
{  public void init()
   {  setBackground(Color.white);
      setForeground(Color.red);
   }
   public void paint(Graphics g)
   {  g.drawRect(10,100,50,70);
      g.fillOval(10,100,50,70);
      g.drawString("Kapil",50,7);
      g.drawLine(100,20,400,70);
      g.setColor(Color.blue);
      g.drawOval(100,200,50,10);
   }
}
```

In a file whose name is **gr.java** type this program. Compile it by command **javac gr.java**. In a file **a.html** type following.

```
<html>
<applet code="gr.java" height=300 width=700>
</applet>
</html>
```

Now give command **appletviewer a.html**

g.drawRect(10,100,50,70) draws a rectangle whose north west corner is (10,100) and sides are 50 and 70.

g.drawOval(10,100,50,70) draws a ellipse inside a rectangle whose north west corner is (10,100) and sides are 50 and 70.

g.drawLine(100,20,400,70) draws a line joining (100,20) and (400,70).

In the above program replace the paint function as follows:

```
public void paint(Graphics g)
{  String a="";
   do{try { DataInputStream o=new DataInputStream(System.in);
            a=o.readLine();
      } catch(Exception t){}
      if (a.compareTo("hari")==0)  g.fillOval(300,100,50,70);
      if (a.compareTo("kapil")==0) g.drawRect(30,100,50,70);
      if (a.compareTo("anil")==0)  g.setColor(Color.blue);
      if (a.compareTo("ravi")==0)  break;
   }while (1==1);
   repaint();
}
```

1. When square 12 is typed then a square of side 12 is displayed. When square 80 is typed then a square of side 80 is displayed. The north west corner is 100 100.
2. When square x is typed then a square of side 12 is displayed. The center is 100 100.
3. When ram is typed then a red circle is displayed. When gagan is typed then green circle is displayed. When bimu is typed then blue circle is displayed. The centre is (100,100). The radius is 80.
4. When rectangle a b is typed then a rectangle of length a and breadth b is displayed. The north west corner is (100,100).
5. When circle a b c is typed then a circle with center (a,b) and radius r is displayed.

```
public class gr extends Applet
{  int x=100;
   public void init()
   {  setBackground(Color.white);setForeground(Color.red);}
   public void paint(Graphics g)
   {  g.drawLine(x,20,200,70);
      try { DataInputStream o=new DataInputStream(System.in);
            String a=o.readLine();
      } catch(Exception t){}
      x=x+5; repaint();
   }
```

} It displays a line. After pressing ENTER the line appears moving.

6. Write a program, which initially displays a line (200,100),(200,250). After pressing ENTER the line rotates by 10 degree (anticlockwise) around (200,250). After pressing ENTER 9 times the line will become (50,250),(200,250) [Hint: (int)Math.sqrt(q) and Math.sin(q)]
7. Write program that reads a sequence of pair of numbers and draws (connected) lines joining them.
8. In above, if a pair 0,? (a pair of numbers with first element 0 is given) then window is cleaned.
9. In above window is cleaned except last line survives.
10. In above last two lines survive.
11. In above, if a pair 0,n is given then last n lines survive. [Hint: use array]

Mouse

```
import java.awt.*;
import java.awt.event.*;
import java.applet.*;
public class muse extends Applet implements MouseListener, MouseMotionListener
{   int p,q;
    public void init( ){   addMouseListener(this);addMouseMotionListener(this);    }
    public void mouseClicked(MouseEvent k) { p=k.getX( );q=k.getY( );}
    public void mouseEntered(MouseEvent k) {}
    public void mouseExited(MouseEvent m) { repaint( ); }
    public void mousePressed(MouseEvent n) {}
    public void mouseReleased(MouseEvent k){}
    public void mouseDragged(MouseEvent p){}
    public void mouseMoved(MouseEvent e){}
    public void paint(Graphics f){ f.drawOval(p,q,10,10); }
}
```

When mouse is moved out of applet a circle is displayed. It is at location of last mouse click

```
public class muse extends Applet implements MouseListener, MouseMotionListener
{  int x[]=new int[20],y[]=new int[20],size=0;
    public void mouseClicked(MouseEvent k)
            { x[size]=k.getX( ); y[size]=k.getY( );size++;repaint( );}
    public void paint(Graphics f) {  for (int i=0;i<size;i++) f.drawOval(x[i],y[i],10,10); }
} When mouse is clicked a circle is displayed. Earlier circles are also displayed.

    int p,q,k;
    public void mouseClicked(MouseEvent g)  {  p=g.getX( );q=g.getY( );k=1;repaint( );}
    public void mousePressed(MouseEvent n) {  p=n.getX( );q=n.getY( );k=2;repaint( ); }
    public void paint(Graphics f)
    {   int i; if (k==1) f.drawOval(p,q,10,10); if (k==2) f.drawOval(p,q,100,100);   }
```

When mouse is clicked a small circle is drawn. When it is pressed a big circle is drawn

1. When mouse is clicked a circle (radius 70) is displayed. The center is the place of click.
2. Circle when mouse pressed. Rectangle when mouse released.
3. Circle when mouse released at the place where the mouse was pressed.
4. Line when mouse is released. The end points are places of mouse press and release.
5. Rectangle when mouse is released. The end points are places of mouse press and release. Assume that location of mouse release is east south of the place of mouse press.
6. Do above problem, when no such assumption is made.
7. Circle when mouse is released. Centre is the place of mouse pressed. The radius is the distance between mouse pressed and mouse released.
8. Circle when mouse clicked. (A) The location is the place where the mouse was previously clicked. (B) The location where mouse was clicked two steps back.
9. Line is drawn from the place of mouse pressed to mouse dragged.
10. Line is drawn from the place of mouse pressed to mouse released.
11. All such lines are displayed.

12. Initially a large number of rectangles are displayed. Their north west corners are (5,10i) i=1..30. Their length and breath are 100 and 8 respectively. When mouse is clicked inside a rectangle then it disappears.
13. Initially a large number of circles (radius 20) are displayed. Their centers are (5,40i) i=1..15. When mouse is clicked inside a circle then it disappears.
14. Initially a large number of circles (radius 20) are drawn. Their centers are (0,0),(50,0),(100,0), ... (500,0),(0,70),(50,70),..(500,70),(100,140),..(500,140),..,(500,700). When mouse is clicked then nearest circle is removed.
15. Do above problem when a circle disappears when mouse is clicked inside it.

Key

```
import java.awt.*;
import java.awt.event.*;
import java.applet.*;
public class key extends Applet implements KeyListener
{  String msg= "";
   public void init()
   {  addKeyListener(this);requestFocus();}
   public void keyPressed(KeyEvent ke)
   {  int key=ke.getKeyCode();
      if ((key>30)&&(key<122)) msg=msg+(char)key;
      switch(key)
      {
         case KeyEvent.VK_F1:msg=msg.substring(1);break;
         case KeyEvent.VK_F2:msg=msg.substring(0,msg.length()-2);break;
         case KeyEvent.VK_F3:repaint();break;
         case KeyEvent.VK_PAGE_DOWN:msg +="<pgDn>";break;
         case KeyEvent.VK_PAGE_UP:msg +="<pgUp>";break;
         case KeyEvent.VK_LEFT:msg +="<Left Arrow>";break;
         case KeyEvent.VK_RIGHT:msg +="<right arrow>";break;
         case KeyEvent.VK_ENTER:msg +=',';break;
      }
   }
   public void keyReleased(KeyEvent ke)
   {System.out.println(msg+ke.getKeyCode());}
   public void keyTyped(KeyEvent ke){}
   public void paint(Graphics g)
   { String a,b,c;int I,j;
     g.drawString(msg,50,10); a="ttttt"+msg;i=a.length();
     if (a.charAt(i-2)=='C') g.drawOval(50,60,10,20);
   }
}
```

1. Modify above program so that old string vanishes when ENTER is typed.
2. Modify above program so that output is updated after typing every letter.
3. When circle is typed then a circle is drawn.
4. When "circle 10" is typed then a circle of radius 10 is drawn.
5. When "circle 30 40 10" is typed then a circle of centre (30,40) and radius 10 is drawn.
6. When "circle red" is typed then a circle of red color is drawn.
7. When circle is typed then circle is drawn. When rectangle is typed then rectangle is drawn. And so on. Also specify their parameters.
8. When F_1 is pressed a circle is drawn. When F_2 is pressed a rectangle is displayed.

9. When 5 is typed a circle of 50 is displayed. When 7 is typed a circle of radius 70 is displayed.
10. A cursor is displayed. It can be modified using arrow keys.

```
 Public void init()
{ setBackground(Color.cyan);setForeground(Color.red);a="Ram is good";}
public void paint(Graphics g)
{ g.drawString(a,10,30);g.drawLine(100,20,400,70);
  g.setColor(Color.blue);g.drawString("kapil is good",10,85);
  g.drawRect(90,100,500,300);g.fillRect(700,100,500,300);
  g.drawOval(100,500,500,300);g.fillOval(700,500,500,300);
}
```

Class

```
#include<iostream.h>
#include<math.h>
class point
  { public: float x,y; };
float dist(point a, point
b)
{   float t1,t2,t;
    t1=a.x-b.x;
    t2=a.y-b.y;
    t=t1*t1+t2*t2;
    return sqrt(t);
}
point readpoint()
{   point p;
    cout<<"Give a point";
    cin>>p.x>>p.y;
    return p;
}
main()
{   point p,q;
    p=readpoint();
    q=readpoint();
    cout<<dist(p,q);
}
```

Read two points and find distance between them

```
#include<iostream.h>
#include<math.h>
class point { public: float x,y,z; };
class vector{ public: float i,j,k; };
point readpoint()
{   point p;
    cout<<"Give a point";
    cin>>p.x>>p.y>>p.z;
    return p;
}
vector direction(point a,point b)
{   vector v;
    v.i=b.x-a.x;
    v.j=b.y-a.y;
    v.k=b.z-a.z;
    return v;
}
void writevector(vector v)
{ cout<<v.i<<"i+"<<v.j<<"j+"<<v.k<<"k"; }
main()
{   point p,q;vector a;
    p=readpoint();q=readpoint();
    a=direction(p,q);
    writevector(a);
}
```

read two points and find corresponding vector direction. If points are (2,3,1) and (6,2,8) then direction is 4**i**-**j**+7**k**.

```
class point { public: float x,y; };
class line{ public: float a,b,c; };
point readpoint()
{   point p;
    cout<<"Give a point";
    cin>>p.x>>p.y;
    return p;
}
line equation(point a,point b)
{   line p;
    p.a=b.y-a.y;
    p.b=a.x-b.x;
    p.c=-p.b*a.y-p.a*a.x;
    return p;
}
void writeline(line a)
{   cout<<a.a<<"x+"<<a.b<<"y+"<<a.c<<"=0"; }
main()
{   point p,q;line a;
    p=readpoint();q=readpoint();
    a=equation(p,q);
    writeline(a);
}
```

Reads two points and find the equation of line joining them. Let P=(2,3) and Q=(4,7) then line is 2x–y–1=0. Hence a=2 b=–1 and c=–1.

Point and vectors is three dimensional space

29. Define point mid (point a, point b). It finds mid point of A and B.
30. Define float dotproduct (vector a, vector b) Dotproduct of 2**i**+3**j**+1**k** and 5**i**+2**j**+3**k** is 19.
31. Define vector crossproduct(vector a, vector b) Cross product in above case is 7**i**–**j**–11**k**.
32. Define float magnitude(vector v) e.g. magnitude 4**i**+3**j**+12**k** is 13. [Call dot product].
33. Define vector mul(vector v, float t). It returns a vector by multiplying it by t. If a=4**i**+3**j**+12**k** then mul(a,0.5) will return 2**i**+1.5**j**+6**k**.
34. Define vector unitvector(vector v). It returns unit vector in the direction of **V**. [Call magnitude and mul functions]. Unit vector for 4**i**+3**j**+12**k** is 0.31**i**+0.23**j**+0.92**k**.
35. Define float area(point a, point b, point c). It finds area of triangle ABC. [Find direction corresponding to AB and AC. Now call cross product and magnitude]. Area of the triangle with end points (3,4,5), (7,3,6) and (6,2,3) is 6.36.

Line and point in two dimensional space

36. Write program, which reads two lines and finds the point of intersection. To accomplish it define a function point intersection(line g, line h). e.g. Input 2x+3y-26=0 and 4x+7y-54=0 output (10,2).
37. Write program, which reads four points and finds the point of intersection of the line joining first two points and the last two points. Let input points be (4,6), (-2,10), (10,2) and (3,6) then the point of intersection is (10,2). [Hint: use above functions]
38. Define function float distance(point p, line l). Using it write program, which reads a point and a line. The program, finds the distance of the point 'p' from the line 'l'. e.g. input point (2,3) and line 8x+6y-12=0 distance = $(8*2+6*3-12)/(8^2+6^2)^{1/2}$ = 2.2.
39. Write program, which reads three points and finds the distance of first point from the line joining other two points. [Hint: use functions to find equation and distance]

Define class circle {point c;float r;}; c is the center and r is radius.

40. void print(circle x). It prints the equation of the circle. Let x be a circle whose centre is (2,3) and radius is 7. Call to a function circle(x) will output $x^2+y^2-4x-6y-36=0$. It is because $(x-2)^2+(y-3)^2=7^2$ is the equation of the circle.
41. circle form(float g,float h,float k). x=form(-4,-6,-36) will make a circle whose centre is (2,3) and radius is 7.
42. float chordlength(circle a,line b). It returns the length of the chord corresponding to circle 'a' and line 'b'. If a circle has centre (1,2) and radius 5 and the line is 5x+12y+10z=0 then chord length is 8. [Method: find distance (d) between centre and line by using appropriate function call now use $(r^2-d^2)^{1/2}$.] Assume that the line intersects with the circle.
43. float intersection(circle a, float b). It returns the y-coordinate of point of intersection of circle (s) and line (x=b). Let a circle (s) has centre (7,2) and radius 5. y=intersection(s,10) will make y as 6 (or -2). It is because circle $(x-7)^2+(y-2)^2=5^2$ intersects with the line x=10 at points (10,6) and (10,-2).

class complex {float r,i;}; r and i are real and imaginary parts respectively

44. Define Complex mul(complex a, complex b) for multiplication of two complex numbers. Similarly define function for addition of complex numbers.
45. Read complex number x and integer n. Find x^n. [Call multiplication function]

46. Read a number n and n complex numbers. Output that complex number, which has maximum magnitude. [Hint: Define function for magnitude of complex number]
47. Write program to find $1 + x + x^2 + x^3 + \ldots + x^n$. Here inputs are x(complex) and n(int). Use functions for multiplication and addition of complex number. [Input 0.75–0.433i output 1–1.732i, when large n is taken]
48. Define Complex mult(complex a, float b) for multiplication of a complex number with a float. Using it write program to find $1 + 2x + 3x^2 + 4x^3 + \ldots + nx^{n-1}$.

Complex numbers

```
#include<iostream>
#include<math.h>
using namespace std;
class comp
{ public: float re,im; };
comp operator+(comp a,comp b)
{ comp t;
  t.re=a.re+b.re;
  t.im=a.im+b.im;
  return(t);
}
comp operator-(comp a)
{  comp t;
   t.re=-a.re;t.im=-a.im;
   return t;
}
comp operator*(comp a,float b)
{ comp t;
  t.re=b*a.re;
  t.im=b*a.im;
  return(t);
}
comp operator*(comp a,comp b)
{ comp t;
 t.re=a.re*b.re-a.im*b.im;
 t.im=a.re*b.im+a.im*b.re;
 return(t);
}
comp readcomp()
{ comp t;
  cout<<"give number";
  cin>>t.re>>t.im;
  return(t);
}
void writecomp(comp a)
{ cout<<"the answer is";
  cout<<a.re<<"+"<<a.im<<"i";
}
main()
{ comp x,y,z;
  x=readcomp();
  y=readcomp();
  z=x*y;
  writecomp(z);
}
```

1. Define comp operator–(comp a, comp b)
2. Define comp operator/(comp a, float b)
   (2+3i)/4=0.5+0.75i.
3. Define comp operator*(float a, comp b)
4. Define comp conjugate(comp a)
   Conjugate(2+3i) is 2-3i
5. Define float magnitude(comp a)
   magnitude(3+4i) is 5
6. Define comp operator/(comp a, comp b)
   [Hint: a*conjugate(b)/magnitude^2(b)]
   (2+3i)/(7+9i)=0.315+0.023i
7. Define comp f(comp x) f(x)=1/(1-x)
   f(4+4i)= –0.12+0.16i
8. Define comp g(comp x) g(x)=x/(1-x)^2
   f(4+4i)= 0.1088–0.198i
9. Define comp expo(comp x) expo(x)=e^x
   expo(a+ib)=e^a(cos(b)+isin(b))
   expo(5+3i)=-146.92+20.94i

The following program finds summation of series:

$$1+x+\frac{x^2}{2!}+\frac{x^3}{3!}+\ \ldots..+\frac{x^n}{n!}$$

When n is large the above become e^x. input x=2+1.05i output e$^{2+\pi/3\,i}$=7.39(0.5+0.866i)
=3.67+6.40i

```
main()
{ int i,n;float fact,p;
  comp x,sum,term,t;
  cout<<"give n";cin>>n;
  x=readcomp();
  sum.re=1;sum.im=0;
  term=sum;fact=1;
  for(i=1;i<=n;i++)
  {  term=term*x;
     fact=fact*i;
     sum=sum+term*(1/fact);
  }
  writecomp(sum);
  cout<<"\nThe exact ";
  p=exp(x.re);
  t.re=cos(x.im);t.im=sin(x.im);
  writecomp(t*p);
}
```

Find summation of given series

Question	Input x	Output
1	0.15+0.35i	0.84+0.833i
1	0.2+0.5i	0.496+1.011i
2	0.2+0.5i	0.70-0.295i
3	0.15+0.4i	0.90-0.589i
4	0.3+0.6i	0.081+0.707i
5	0.3+0.5i	1.077-0.154i
6	0.15+0.5i	0.16+0.515i

27. ----------

Overloading

```
#include<iostream.h>
class comp
{ public:
    int re,im;
};
comp operator+(comp a,comp b)
{ comp t;
  t.re=a.re+b.re;
  t.im=a.im+b.im;
  return(t);
}
comp operator*(comp a,int b)
{ comp t;
  t.re=b*a.re;
  t.im=b*a.im;
  return(t);
}
comp operator*(comp a,comp b)
{ comp t;
 t.re=a.re*b.re-a.im*b.im;
 t.im=a.re*b.im+a.im*b.re;
 return(t);
}
comp pow(comp a,int n)
{ comp t;
  if(n==1) t=a;
  else t=a*pow(a,n-1);
  return(t);
}
comp readcomp()
{ comp t;
  cout<<"give complex number";
  cin>>t.re>>t.im;
  return(t);
}
void writecomp(comp a)
{ cout<<"the number is";
  cout<<a.re<<"+"<<a.im<<"i";
}
main()
{ int i,n;
  comp x,ans;
  cout<<"give n";cin>>n;
  x=readcomp();
  ans.re=1;ans.im=0;
  for(i=1;i<=n;i++)
    ans=ans+pow(x,i)*i;
  writecomp(ans);
}
```

Non Abstract Data Type

```
#include<iostream.h>
class stack
{ int sp;float x[200];
  public:
   stack(){sp=0;}
   void put(float e){ x[sp]=e;sp++;}
   float take(){ sp--; return(x[sp]);}
   int isempty(){ if (sp==0) return 0;else return 1;}
   void print()
   { int i;
     for(i=0;i<sp;i++)cout<<" "<<x[i]<<" ";
   }
};
main()
{ stack s;int choice;float ele;
  do{ cout<<"\n give choice\n 0 for put\n 1 for take";
      cout<<"\n 2 for print\n 3 for status\n 4 for exit";cin>>choice;
      switch(choice)
      { case 0: cout<<"give element";cin>>ele;s.put(ele);break;
        case 1: cout<<"\nElement take is "<<s.take();break;
        case 2: s.print();break;
        case 3: cout<<"The status is "<<s.isempty();break;
     }
   }while(choice!=4);
}
```

1. Add one more operation “size”. It returns the number of elements. put(11) put(34) size outputs 2.
2. Add one more operation “initialize”. After that the stack becomes empty. Put(11) put(14) initialize put(13) put(34) print will output 13 34 (not 11 14 13 34).
3. Define Queue. Put(11) put(13) put(10) Take Take will output 11 and 13.
4. Define priority system. The element taken is the biggest of existing elements. Output in above case 13,11.
5. Define queue using front and rear.
6. Define circular queue.
7. Define data type “sequence”. Its operations are put(e,k), take(k), empty and print. After operations put(12,1) put(37,2) put(41,1) put(67,2) put(95,3) the sequence will look as 41,67,95,12,37. Now if operation take(4) is done then element taken will be 12. Now print operation will output 41,67,95,37. [Hint: class sequence {int size;float x[200];…...]
8. Define data type “dynamic set”. Its operations are put(e), take(e), present(e) and print. After operations put(12) put(37) put(41) put(67) take (37) put(95) the set will look as {12,41,67,95}. In print operation the elements may be printed in any order [Hint: 12,95,41,67 will be easy and efficient to implement]. [Hint: class dy_set {int size;float x[200];…... int present(float e) it returns 1 if e is present in the set. 0 is returned otherwise.] present(41) outputs 1. present(49) outputs 0. present(37) outputs 0.
9. Define dynamic set using open address hashing. [Hash function is sum of digits, assume numbers are 2 digit positive integers] [class oa_hash{int x[25]; during present(e) the elements observed are also printed] Put(23) put(56) put (57) put(83) put (39) put(93) present(39) will output 57, 83 “1”. Now present(58) will output 83 39 93 “0”.

10. Define dynamic set using chain based hashing. [Hash function: (sum of digits) mod 5, assume numbers are 2 digit positive integers] [class ch_hash{int x[25], next[25];]
11. Define queue in such a manner so that its all operations (except put) are same as in stack.
12. Define queue in such a manner so that its all operations (except take) are same as in stack.
13. Define priority system such that its all operations (except put) are same as in stack.
14. Define priority system such that its all operations (except take) are same as in stack.
15. Define priority system such that its all operations (except put) are same as in queue. [Queue is defined using front and rear]
16. Define priority system such that its all operations (except take) are same as in queue.

Abstract Data Type

```
#include<iostream.h>
class stack
{ int sp;float x[200];
  public:
   stack(){sp=0;}
   void put(float e){ x[sp]=e;sp++;}
   float take(){ sp--; return(x[sp]);}
   int isempty(){ if (sp==0) return 0;else return 1;}
   void print()
   { int i;
     for(i=0;i<sp;i++)
       cout<<" "<<x[i]<<" ";
   }
};
class queue
{ stack s1,s2;
  public:
   void put(float e){ s1.put(e); }
   float take()
   { if (s2.isempty()==0) while (s1.isempty()!=0) s2.put(s1.take());
     return(s2.take());
   }
   int isempty()
   { if (s1.isempty()==0 && s2.isempty()==0)return 0; else return 1;}
   void print()
   {  cout<<"The first stack ";s1.print();cout<<"The second stack
";s2.print();}
};
main()
{ stack s;int choice;float ele;
  do{ cout<<"\n give choice\n 0 for put\n 1 for take";
      cout<<"\n 2 for print\n 3 for status\n 4 for exit";cin>>choice;
      switch(choice)
      { case 0: cout<<"give element";cin>>ele;s.put(ele);break;
        case 1: cout<<"\nElement take is "<<s.take();break;
        case 2: s.print();break;
        case 3: cout<<"The status is "<<s.isempty();break;
     }
   }while(choice!=4);
}
```

In above program a stack (non abstract) is defined. Using it abstract data type queue is defined. It is defined by using two stacks. An element is put in the queue by putting in first stack. After the operations put(12) put(19) put(16) put(11) then queue will look as [(12,19,16,11)()]. An element is taken from the queue by taking it from second stack. However if second stack is empty then all elements of the first stack are transferred into second stack. Now after take operation it will look as [()(11,16,19)]. (for some time the queue will be [()(11,16,19,12)]). The element taken will be 12. Now after put(17) put(28) put(22) the queue will look as [(17,28,22)(11,16,19)] Now after take operation it will look as [(17,28,22)(11,16)]. The element taken will be 19. After two more takes it will look as [(17,28,22)()]. Now if one more take is done it will look as [()(22,28)] (for some time it will be [()(22,28,17)]). The element take is 17.

1. Define stack using two queues. An element is put in stack by putting in first queue. An element is taken from stack by taking last element of first queue. Another queue acts as auxiliary queue.
2. Define stack using single queue. Assume all elements are positive.
3. Using stack define priority queue. In that element taken is the biggest of existing elements.
4. Define priority queue using queue.
5. Define stack using priority queue. Assume all elements are between 0 and 99.
6. Define non abstract data type “sequence”. Its operations are put(e,k), take(k), isempty and print. After operations put(12,1) put(37,2) put(41,1) put(67,2) put(95,3) take(4) then sequence will look as 41,67,95,37. The element taken will be 12.
7. Define abstract data type stack using sequence.
8. Define abstract data type sequence using stack.

```
class stack1
{ queue q1,q2;
  public:
   void put(float e)
   { q1.put(e); }

   float take()
   {   float k;
       do{
          k=q1.take();
          if (q1.isempty()!=0) q2.put(k);
       } while (q1.isempty()!=0);
       while (q2.isempty()!=0)
         q1.put(q2.take());
       return(k);
   }
   int isempty() { return q1.isempty(); }
   void print(){ q1.print();}
};
class stack2
{ queue q;
  public:
   void put(float e)
   { q.put(e); }

   float take()
   {   float k,l;
       q.put(-100);k=q.take();
       do{l=k;
          k=q.take();
          if (k!=-100) q.put(l);
       } while(k!=-100);
       return(l);
   }
   int isempty() { return q.isempty(); }
   void print(){ q.print();}
};
```

TCP (Java)

Type following program in a file named a.java. Compile the program by javac a.java. Now open two windows. In one window give command java server. In another window give command java client. The server has sent a string “ramprasad”.

```
import java.net.*;
import java.io.*;
class server
{  public static void main(String args[]) throws IOException
   {  ServerSocket a=new ServerSocket(9876);
      System.out.println("address binding over");
      Socket b=a.accept();
      System.out.println("got client");
      DataOutputStream d=new DataOutputStream(b.getOutputStream());
      d.writeUTF("ramprasad");
      a.close();
   }
}
class client
{  public static void main(String args[]) throws IOException
   {  Socket p=new Socket(InetAddress.getLocalHost(),9876);
      System.out.println(p+" connected");
      DataInputStream q=new DataInputStream(p.getInputStream());
      String r=new String(q.readUTF());
      System.out.println("received "+r);
      p.close();
   }
}
```

Matching Statements	
b=a.accept( );	p=new Socket(InetAddress.getLocalHost( ),9876);
writeUTF("ramprasad")	readUTF( )

1. Modify the above program so that the server sends two strings.
2. Modify the given program so that the client sends the string.
3. Modify the given program so that client sends a string “kapil” to the server. The server sends a string “gopal” to the client.
4. Write program so that client sends a string to the server. The server deletes the first letter and sends back the string to the server. [Notation: g.substring(2) will delete the first two letters of string ‘g’. Let g=”kapil” h=g.substring(2) will make h=”pil”]
5. The server sends two strings to the client. The client joins them and sends it to the server. [g+h joins strings g and h. Let g=”ram” h=”ok” p=g+h will make p=ramok]
6. Modify the server of the given example program (the client remains unchanged). The server gets connected to two clients. To first client the server sends “ram”. To second client the server sends “hari”. Open three windows: In one window execute the server. In another two windows execute the client. [Hint: use accept system call twice]
7. Make three classes: server, clienta and clientb. The clienta sends a string “ram” to the server. The clientb sends string “hari” to server.
8. In the context of above question: The server sends back the strings to opposite clients.
9. Make three classes: client, servera, serverb. Let ports of servers are 9876 and 9877. The client gets connected to both servers. To servera the client sends string “ram”. To serverb the client sends string “hari”.

10. In the context of above question: both servers send a string to the client. The Client joins them and prints. Let servers send “ram” and “ok” the client will output “ramok”.
11. In the context of above question: The client sends back the strings to opposite servers.

UDP (Java)

In following program Ram sends a string to Hari. Hari increments the first letter and sends back.

```
import java.net.*;
class ram
{  public static void main(String args[]) throws Exception
   {   DatagramSocket ds=new DatagramSocket(9846,InetAddress.getLocalHost());
       byte a[]=new byte[100];String b="gopal";a=b.getBytes();
       ds.send(new DatagramPacket(a,5,InetAddress.getLocalHost(),9847));
       DatagramPacket p=new DatagramPacket(a,5);ds.receive(p);
       System.out.println(new String(p.getData()));
   }
}
class hari
{  public static void main(String args[]) throws Exception
   {   DatagramSocket ds=new DatagramSocket(9847,InetAddress.getLocalHost());
       byte a[]=new byte[100];
       DatagramPacket p=new DatagramPacket(a,5);ds.receive(p);
       System.out.println(new String(p.getData())+" is sent by "+p.getPort());
       a=p.getData();a[1]++;
       ds.send(new DatagramPacket(a,5,p.getSocketAddress()));
    }
}
```

1. Make one more class Kapil. It is similar to Ram. Its port address is 9848 (in place of 9846). It sends string “mohan” (in place of “gopal”). No other modification in class Kapil should be made (The class Ram is identical to the example program). Modify class Hari. It receives strings from class Kapil and Ram. It prints both string. It sends those strings in opposite manner. [The string of Kapil is sent to Ram and vice versa]
2. Modify Ram. It reads a string and sends to Hari. The class Hari is same as in example. Assume string length is 5.
3. Modify Ram so that it reads 10 strings and sends to Hari. [Modify Hari to receive 10 strings]. Assume all strings have length 6.
4. DatagramSocket( ) is used for address binding. In Ram it was not needed. (because port number 9846 is not needed). Hence remove it from Ram. Ram reads a string and sends to Hari. Do not make any other modification in Ram. Execute Ram from two windows. Modify Hari so that it receives two strings. It sends them to opposite senders. [To remove address binding use `DatagramSocket ds=new DatagramSocket()`]
5. Modify the class Hari of the example. It reads a number (self port number). It does address binding to that (in place of 9847). Modify Ram so that it reads a number (the port number of Hari). Now Ram sends a string to Hari. [Caution: During execution give port numbers as big numbers (more than 5000)]
6. Execute the class Hari (without modification) of above program from two windows (with different self port numbers). Modify the class Ram. It reads two numbers (port numbers of class Hari). It reads two strings and sends one each to Hari.
7. Define class Ram. This class is executed from two ports (windows). Every port reads two numbers: self port number and the port number of another window. From both ports a string is read and is send to another port.
8. Modify class Ram so that it can be executed from three windows.

9. Ram reads a number and sends to Hari. Hari increments the number and sends back to Ram. Assume that the number has 5 digits. Let string received by Hari is ‘s’. Before converting to integer perform s=s.trim().
10. Define one more class Kapil. Ram sends a string to Hari. Hari sends the address of Ram (using getport(), not directly 9846) and the string to Kapil. The Kapil increments the 3rd letter and sends the string to Ram.
11. Define class Hari. It reads a number (self port number). Execute Hari from two windows. From both ports a string is read and sent to opposite port. It is performed by using class Ram. Ram reads two numbers (port numbers of Hari) and sends them to opposite port. (Let the numbers are X and Y. The number X is sent to port Y and the number Y is sent to X). Now both Hari(s) know the address of each other. Hence they can directly communicate. During communication between Hari(s), Ram does not play any role.

Applications (Java,udp: Lan Tree Ring)

1. Write 3 classes. Lan, sender and receiver. Execute LAN in one window and sender and receiver in few windows. [Do not perform address binding in sender and receiver]. Let the port number of Lan is 9846.
   (A) Whenever Lan receives a sting "XXX" it stores the address of the port (who has sent it) in an array.
   (B) Whenever Lan receives any other string it sends that (string) to all addresses stored in the array.
   (C) The sender reads string(s) and sends to the Lan. Assume all strings have length 3.
   (D) First time the receiver sends a string "XXX" to the LAN. After that it receives strings from LAN and displays them.
   [Hint: int p=`a.compareTo("XXX");` makes the value of 'p' as 0 if string 'a' is same as "XXX"]

2. Define Tree using two classes: node and sender
   (B) In the beginning the node reads two numbers. self port (X) and address (port) of some node (Y) already in the graph.
   (C) The address Y is stored in the list of adjacent nodes. [except when X=Y]
   (D) The node (X) sends a string "KKK" to node (Y).
   (E) When a node receives a string "KKK", it joins the sender's address in the list of adjacent nodes.
   (F) When a node receives a string other than "KKK" then it sends the same string to all nodes whose addresses are in the adjacency list.
   (G) To stop infinite loop: when a node receives a string, it does not sent it to the node, who has sent it.
   (H) In sender class a number (P) is read. It is the address of some node in the tree. After that strings are read (in the sender class) in an infinite loop. Every string is sent to P. [Because of interconnection it will be displayed in all nodes of the graph.]

3. Define ring using two classes: node and sender.
   (B) In the beginning a node reads two numbers: self port (X) and address (port) of the previous node (Y). It sends the string "KKK" to node Y.
   (C) When a node (Y) receives a string "KKK", it (node Y) sends to X the address (port number) of its next node (Z). The node Y also updates its next port as X.
   (D) The new node (X) records its next port as (Z).
   (E) When a node receives a string other than "KKK". It sends it to its next node. [To avoid infinite loop, when a node receives a string (which was previously received by it) it does not sends it to its next node]
   (F) In sender class a number (P) is read. It is the address of some node in the ring. After that strings are read (in the sender class) in an infinite loop. Every string is sent to P. [Because of interconnection it will be displayed to all nodes of the ring.
   [In step (A) if X and Y are same them (A)(B)(C) may by omitted. The next[X] may directly be recorded as X. But try to do this without using this special case.]

4. In above a node can be deleted as follows:

(B) The string “LLL” is given to sender, which sends it to some node (X) in the ring.
(C) When a node (X) gets a string “LLL” it sends another string “M” to its next node.
(D) When a node receives string “MMM” it sends to the sender (X) the port number of its next node (Y).
(E) The node (X) updates its next node as Y.

5. Modify above so that the node to whom L is sent is deleted.

JAVA (Overview)

Type following program in a file named a.java. Compile the program by javac a.java. Execute the programs using java ram. It input is a string. The first two letters of the string are deleted. [Input "prasad" Output "asad"]. In string "prasad" 0th letter is 'p'. 1st letter is 'r'. 5th letter is 'd'. substring(2) retains 2nd letters onward. [substring(3,6) retains 3rd, 4th, and 5th letters only ("omprakesh" $\rightarrow$"rak")].

```
import java.io.*;
class ram
{  public static void main(String args[]) throws Exception
   {   DataInputStream t=new DataInputStream(System.in);
       String k,p;k=t.readLine();
       p=k.substring(2);
       System.out.println(p);
   }
}
```

1. Write program, which reads two strings. Deletes 0th letters of both strings and joins them. [p=q+r will join strings q and r and store it in p]
2. Program that reads a string and deletes 2nd and 3rd letter. [Hint: substring(0,2)+(4)]
3. Program to exchange 2nd and 5th letters.

The following program reads a numbers and doubles it.

```
int m;m=Integer.parseInt(t.readLine());
m=m*2;System.out.println(m);
```

4. Read a number (x) and a string. Output x^{th} letter of the string.

The following program reads a number and doubles it. It considers the number of first and second digit only. It doubles that. A number can be converted (type conversion) to a string by appending null string. Input 474587 $\rightarrow$ 949174 $\rightarrow$ "949174" $\rightarrow$ "49" $\rightarrow$ 49 $\rightarrow$ 94.

```
int m;m=Integer.parseInt(t.readLine());m=m*2;
String p=m+"";              //Converts integer into String
p=p.substring(1,3);
m=Integer.parseInt(p);
m=m*2;System.out.println(m);
```

5. Read a number, double it and delete its second digit. Input 2745 output 540.
6. Read a number, find its first digit (x). Now find x^{th} digit. Input 478293 output 9.

The following program reads a string. Converts it into an array of letters (using getBytes). Now 0th letter is incremented. The array of letters is again converted into string.

```
class gopal
{  public static void main(String args[]) throws Exception
   {   DataInputStream t=new DataInputStream(System.in);
       String m;m=t.readLine();
       byte a[]=new byte[100];a=m.getBytes();
       a[0]++; m=new String(a);
       System.out.println(m);
   }
}
```

7. Read a string and exchange its 0th and 1st letters. Do not use substring.
8. Read a string and delete its 0th letter. Do not use substring.
9. Read a string and find how many letters are capital. [ASCII code between 65 and 90]

Read two strings. Output is 0 if both are same.

```
a=t.readLine();b=t.readLine();
int i=a.compareTo(b);System.out.println(i);
```

```
i=a.length();
```

It finds the length of the string.

TCP

Type following in a file name “server.c”. Compile it using [cc server.c –o server]

```
#include<stdio.h>
#include<sys/types.h>
#include<sys/socket.h>
#include<netinet/in.h>
#include<arpa/inet.h>
main()
{  int sockfd,fd,len,i,p;char b[200];
   struct sockaddr_in sa,ca;
   sockfd=socket(AF_INET,SOCK_STREAM,0);
   sa.sin_family=AF_INET;
   sa.sin_addr.s_addr=INADDR_ANY;
   sa.sin_port=6001;
   len=sizeof(sa);
   i=bind(sockfd,(struct sockaddr *) &sa,len);
   printf("(%d %d)\n",sockfd,i);
   listen(sockfd,5);
   fd=accept(sockfd, (struct sockaddr *) &ca,&len); printf("[%d]\n",fd);
   recv(fd,&p,4,0);printf("client sent %d\n",p);
   printf("\nGive string to send to client");scanf("%s",b);
   send(fd,b,50,0);
}
```

Type following in a file name “client.c”. Compile it using [cc client.c –o client]

```
main()
{  int sockfd,len,i,a;char q[200];
   struct sockaddr_in sa;
   sockfd=socket(AF_INET,SOCK_STREAM,0);
   sa.sin_family=AF_INET;
   sa.sin_addr.s_addr=inet_addr("127.0.0.1");
   sa.sin_port=6001;
   len=sizeof(sa);
   i=connect(sockfd,(struct sockaddr *) &sa,len);
   printf("(%d %d)\n",sockfd,i);
   printf("Give a number for server");scanf("%d",&a);
   send(sockfd,&a,4,0);
   recv(sockfd,&q,50,0);printf("server sent %s\n",q);
}
```

Run both these programs in two separate windows using [./client and ./server] respectively. First run server. If server creates problem in binding (address) then change the port number 6001 to some other number (to the same number in client also).

Matching Statements	
fd=accept(sockfd, ...	connect(sockfd, ...
recv(fd,&p,4,0);	send(sockfd,&a,4,0);
send(fd,b,50,0);	recv(sockfd,&q,50,0);

The accept system call accepts a client connection. It blocks a server until a client request for connection comes.

1. The client reads a number and sends to the server. The server doubles it sends back to the client.
2. The client reads a number(x) and sends to server. The server sends 2x and x^2 to the client. The Client adds them and sends the result to the server. The server displays it.
3. Server reads two numbers and sends them to client. The client displays those numbers and sends the sum to the server. The server displays it and sends its double (P) to the client. The client displays that. Now the client reads a number (Q). It sends the value of P+Q and P-Q to the server. The server displays them. Example:
   Server 17 23 (40) 93 67
   Client 17 23 (80) 13
4. The client reads 10 numbers and sends them to the server one by one. The server displays them one after another.
5. The client reads 10 numbers and sends them to server one by one. When server gets an even number it sends it double to the client. The client displays it (immediately before reading the next number). [Example: Input output behavior at client is as follows]
   13 17 12 (24) 32 (64) 11 19 17 90 (180) 19 23]
6. The client reads 10 numbers and sends them to server one by one. When server gets an even number it reads a number and sends it to the client. The client displays it (immediately before reading the next number). [Example:
   Client 13 17 12 (29) 32 (81) 11 19 17 90 (13) 19 23
   Server 29 81 13
   There shall be exactly three inputs at server. Unnecessary inputs should not be given.
7. The client and server both read a number (P) and send to each other. If both numbers are even or both are odd then both read one more number (Q). Both display counterpart(Q)+10. Otherwise both display the value of 2P+counterpart(P). [Example1: cl(P)=12 se(P)=18 cl(Q)=44 se(Q)=63. The output at client is 73 and at server is 54.][Example2: cl(P)=23 se(P)=18. The output at client is 64 and at server is 59. The value of Q should not be read.]
8. The client and server both read a number (X) and send to each other. If both numbers are even or both are odd then client reads one more number (Y) and sends to server. The server displays it. If one of the X's is even and another odd then the server reads two more numbers Z and W and sends them to client. The client displays them. [Example: cl(X)=12 se(X)=18 cl(Y)=44 The output at server is 44.]
9. The client and server both read numbers and send to each other. It is repeated 10 times.
10. There is a client and a server. Two players are sitting on them. They are playing a game. Initially a player writes a number. After that game starts. A player is supposed to write a number, which is 1 or 2 or 5 more than written by opponent. The player, who writes 50 or bigger is declared winner. Let game begins with 23. 23(28)30(31)36(41)43(45)50. The first player wins. Both client and server display the final result (who won)

TCP(multi client server)

```
#include<stdio.h>
#include<sys/types.h>
#include<sys/socket.h>
#include<netinet/in.h>
#include<arpa/inet.h>
#include<fcntl.h>
main()
{   int sockfd,fd1,fd2,length,i,a,b;
    struct sockaddr_in sa,ta1,ta2;
    sockfd=socket(AF_INET,SOCK_STREAM,0);
    sa.sin_family=AF_INET;
    sa.sin_addr.s_addr=INADDR_ANY;
    sa.sin_port=6001;
```

In place of `sa.sin_port=6001` following can also be used.

```
    printf("Give self port number:");
    scanf("%d",&sa.sin_port);
```

```
    i=bind(sockfd,(struct sockaddr *) &sa,sizeof(sa));
    printf("%d %d\n",sockfd,i);
    listen(sockfd,5);length=sizeof(sa);
    fd1=accept(sockfd, (struct sockaddr *) &ta1,&length);
    fd2=accept(sockfd, (struct sockaddr *) &ta2,&length);
    printf("%d %d\n",fd1,fd2);
    recv(fd1,&a,4,0);recv(fd2,&b,4,0);printf("%d %d\n",a,b);
    a=a+10;b=b*2;send(fd1,&b,4,0);send(fd2,&a,4,0);
}
```

The above is server. The following is client. Run the above program in a window and following program in two windows. There is one server and two clients. Both clients read a number and send to the server. The server adds 10 the number from the first client and doubles the number from the second client. These numbers are sent to the opposite clients.

```
main()
{   int sockfd,a;char s[50];
    struct sockaddr_in sa;
    sockfd=socket(AF_INET,SOCK_STREAM,0);
    sa.sin_family=AF_INET;
    sa.sin_addr.s_addr=inet_addr("127.0.0.1");
    sa.sin_port=6001;
```

In place of last two statements following can also be used.

```
    printf("Give server address and port no");
    scanf("%s %d",s,&a);
    sa.sin_addr.s_addr=inet_addr(s);
    sa.sin_port=a;
```

```
    i=connect(sockfd,(struct sockaddr *) &sa,sizeof(sa));
    printf("(%d %d)",sockfd,i);
    printf("Give a number to send");scanf("%d",&a);
    send(sockfd,&a,4,0);
    recv(sockfd,&a,4,0);
    printf("%d",a);
}
```

1. (A) The server reads a number and sends it to the first client. The first client displays it and sends the double of it to the server. The server displays it and sends it to the second client after adding 10. The second client displays it and sends the double of it to the server. The server displays it. If input to the server is 67 then output of clients are 67 and 144 respectively. The output of the server is 134 and 288 respectively.
   (B) Solve above problem when the second client sends the triple (in place of double) to the server. Now the second output of the server will be 432 in place of 288. [(B) is a hard problem. Solve it at the end.]
2. Both clients read 10 numbers. Both send them to the server. The server sends them to opposite clients. They display it.
3. Both clients read a number (P). The client, who has read bigger number, reads one more number (Q) and sends it to the server. The server sends it to the other client (the client with smaller number). It displays it. [Hint: Both clients send the number (P) to the server. It sends them to opposite clients.] Assume input (P) to both clients is different.
4. The client, who has read bigger numbers reads 10 more number and sends to the server. It displays all of them.
5. The server reads 10 pair of numbers [scanf("%d %d",&x,&y)] x is 1 or 2. If x is 1 then y is sent to the first client. If x is 2 then y is sent to the second client. Both clients display the numbers, which they have got.
   Let input to the server be 2 34 2 56 1 67 2 87 2 91 2 58 1 23 2 56 1 78 1 88
   The output produced by clients is 67 23 78 88 and 34 56 87 91 58 56 respectively.
6. In above question after producing output both clients output the sum of the numbers, which they have got. [Hint: After reading 10 numbers the server sends -1 to both clients. Hence they come to know that they have got all numbers. Assume numbers are all positive]
7. In above question after producing output both clients read one more number. They send it to the server. It displays both of them.
8. There are two servers and one client. Both servers read a number. They send to the client. The client displays both numbers.
9. There are two servers and one client. Intitially the client reads two numbers. These are port numbers of the servers. Then the client reads 3 more numbers P, Q, and R. P is the port number of one of the servers. The number P is sent to that server and the number R is sent to another server.
10. There is one server and two clients. The first client reads a number (x) and sends to server, who sends it another client. [How a client knows that it is first or second client: The server sends '1' to the first client and '2' to the second client]
11. In above question unnecessary communication takes place. It can be avoided by writing different programs for clients. [Caution: The first client is executed first]
12. The first client reads 10 numbers and sends to second client (via server).
13. There are 3 programs P, Q and R. P is server. Q is client. R is both. Q is connected to R. R is connected to P. Q reads a number and sends to R. R sends it to P.

TCP(no wait for input)

```
#include<stdio.h>
#include<fcntl.h>
main()
{  int i=1,x=-1;
   fcntl(0,F_SETFL,O_NDELAY);
   do{printf("%d\n",i++);
      scanf("%d",&x);
      if (x!=-1)
         printf("%d",2*x);
   }while(x==-1);
}
```

The following system call does not perform wait on input. Hence if data is not available during scanf(x) the next line is executed. The value of x remains unchanged.

```
fcntl(0,F_SETFL,O_NDELAY);
```

The given program prints 1,2,…. till an input is given. Finally double of the input is printed.

```
int i;
for(i=1;;i++)
{  printf("%d\n",i);
   sleep(5);
}
```

The given program prints 1,2,3... The next number is printed after the gap of 5 seconds. The sleep system call stops the execution of the program for specified time.

1. Write program, prints 1,2,3,…. As soon as it reads a number it stops printing for 2 seconds. After that it resumes printing.
2. Write program, which reads 10 numbers and outputs their sum. During the time of reading numbers it continues printing 1,2,3,…
3. Write program, which reads 10 numbers. After reading every number it outputs the time of input. (The time at the start of the program is taken as 0). [Hint: in loop use sleep(1)]
4. Write program, which reads 10 numbers. After reading every number (except for first number) it outputs the time gap between consecutive numbers.

Server	Client

`fd=accept(sockfd, (struct ...`
`printf("Give number to send to client");`
`scanf("%d",&b);`
`send(fd,&b,4,0);`
	`connect(sockfd,(struct ...`
	`fcntl(sockfd,F_SETFL,O_NDELAY);`
	`do{printf("%d\n",k++);`
	`  i=recv(sockfd,&q,4,0);`
	`} while(i==-1);`

In the above program, a client is printing 1,2,3,… As soon a server reads a number, it sends to the client. The client stops printing. Because of fcntl(sockfd,…) system call wait is not performed during recv(sockfd,…) system call. If nothing is sent to it then value of 'i' (the return value of recv(…)) is -1.

5. The server prints 1,2,3,… As soon as the client reads a number it sends to the server. The server prints the double of the number and stops execution.
6. Do above problem, when the server reassumes printing (1,2,3…) after outputting double of the number.
7. The client prints 1,2,3,…. for 1 second. [Hint: use sleep(1) in server]
8. The client is reading numbers for 20 seconds. As soon as it reads a number after time=20seconds it outputs the sum of the numbers, it has read.
9. The client is printing 1,2,3,…. After every 5 seconds it stops for 1 second.
10. The client reads numbers for 20 seconds. It outputs the sum of the numbers.
11. The server reads two numbers. As soon as it reads the first number the client starts printing 1,2,3,… As soon as server reads the second number the client stops printing.

12. The client is printing 1,2,3,….The server reads 10 numbers. After reading every number it sends to the client. The client stops for 2 seconds, then outputs double of the number, then again stops for 1 second. After that client resumes its printing.
13. The client is printing 1,2,3,…. The server reads 10 numbers. After reading every number it prints the number, which the client has printed at that time (approximately). [When server reads a number, it sends a message (some number). The client sends the value of 'k'.
14. The client and server are both reading numbers (in any order). As soon as they read a number they send to their counter part, who displays it. [Example:

    Client 12 71 (29) (34) 16 29 89

    Server (12) (71) 29 34 (16) (29) (89)
15. Implement chat. [In above program read strings in place of numbers]

No wait for input

```
#include <stdio.h>
#include <fcntl.h>
main()
{ int g,x,y,z,w;
  printf("Give a line of numbers");
  scanf("%d",&x);
  fcntl(0,F_SETFL,O_NDELAY);
  g=scanf("%d %d %d",&y,&z,&w);
  printf("%d\n",g+1);
}
```

```
main()
{ int g,x,y,z,w;
  printf("Give a line of numbers");
  fcntl(0,F_SETFL,O_NDELAY);
  do{ g=scanf("%d %d %d %d",&x,&y,&z,&w);
    }while(g==-1);
  printf("%d\n",g);
}
```

These programs read few (at most four) numbers in a line. Programs find the number of numbers. The fcntl( ) system call do not waits for input on scanf. The scanf returns the number of input numbers. If there is no input then -1 is returned.

1. The first program does not work correctly when only one number is given as input. Make suitable modifications.

```
int g,x,y,z,w;
printf("Give a line of numbers");
scanf("%d",&x);
fcntl(0,F_SETFL,O_NDELAY);
y=0;z=0;w=0;
g=scanf("%d %d %d",&y,&z,&w);
printf("%d\n",x+y+z+w);
```

```
int g,x,y;
printf("Give a line of numbers");
scanf("%d",&x);
fcntl(0,F_SETFL,O_NDELAY);
do{ g=scanf("%d",&y);
    if (g!=-1) x=x+y;
}while(g!=-1);
printf("%d\n",x);
```

These programs read few numbers in a line. The sum of numbers is printed. In the first program at most four numbers can be given as input. In the second program there is no limit.

2. Modify the first program so that only one scanf is used. (at most four numbers).

```
int g,s=0,x;
 fcntl(0,F_SETFL,O_NDELAY);
 sleep(10);
 do{ g=scanf("%d",&x);
     if (g!=-1)s=s+x;
 }while(g!=-1);
 printf("%d\n",s);
```

Program that reads as many numbers in 10 seconds as possible. It finds their sum. Give all numbers in separate line.

3. Program that reads a number. After reading that the program reads numbers for 10 seconds. The program outputs the product of the first number with remaining numbers.
4. Write program, which reads 10 lines of numbers. Every line has got at most five numbers. After reading every line the program outputs the number of numbers. Do this problem using only one scanf.
5. Do above problem using only one loop.
6. Do above problem, when there is no restriction on the number of numbers in a line.
7. Write program, that reads many lines of numbers. Every line has got at most five numbers. After reading every line the program outputs their sum. The program stops when a line with exactly 3 numbers is given.
8. Do above problem, when there is no restriction on number of numbers in a line.
9. Program that reads a line of numbers (at most three). If there is only one number then its square is printed. If there are two numbers then their product is printed. If there are three numbers then their sum is printed. Do this problem using one scanf only.
10. Do above problem without using any loop.

11. Do above problem when there is no restriction on the number of numbers in the line. If there are more than three numbers then their sum is printed.
12. Program reads two lines of numbers. Both lines have at most 5 numbers. The program finds the product of sum of numbers in both lines.
13. Client reads a 10 lines of numbers. After reading every number it sends to server. The server outputs the sum of numbers in the same line. [The server outputs10 numbers].
14. Write two programs. The second program is run in many windows. Each of these processes reads two numbers. The first number is the port number of the first program. The second number is sent to the first program. When first program receives a number (X) it stores the sender's address (port number) in an array. Then that number (X) is sent to all ports whose address is recorded.
15. Write two programs. The second program is run in many windows. Each of these processes read the port number of the first program. After that it reads numbers in an infinite loop. Each of these numbers is sent to the first program. When first program receives a number (X) it stores the sender's address (port number) in an array. Then that number (X) is sent to all ports whose address is recorded. [It is simulation of broadcast].

16.

(Communication using)Semaphore

```
#include<stdio.h>
#include<sys/sem.h>
main()
{  int p,x;
   p=semget(0,1,IPC_CREAT|0666);
   printf("semaphore no=%d\n",p);
   printf("give a number to send");
   scanf("%d",&x);
   semctl(p,0,SETVAL,x);
}
```

```
#include<stdio.h>
#include<sys/sem.h>
main()
{  int q,y;
   printf("Give semaphore no");
   scanf("%d",&q);
   y=semctl(q,0,GETVAL,0);
   printf("%d",y);
}
```

Run these program in different windows. The first program creates a semaphore (displays its number) and reads a number from the key board. The value the number is stored in the semaphore. The semaphore number is communicated to the second program (by typing). Now the second program fetches the value of the semaphore and displays it. The semaphore acts as a global variable. The second program is able to do so because it knows the number of the semaphore.

Suppose we want to first communicate the semaphore number to the second program and then input is given to the first program then `y=semctl(q,0,GETVAL,0);` is replaced by following

```
semctl(q,0,SETVAL,10);do{y=semctl(q,0,GETVAL,0);}while(y==10);
```

The second program waits (in a busy loop) till the value of the semaphore is modified. [We assume that the input number to the first process is not 10].

```
main()
{  int p,x,g;
   p=semget(0,1,IPC_CREAT|0666);
   printf("semaphore no=%d\n",p);
   semctl(p,0,SETVAL,0);
   printf("give a number to send");
   scanf("%d",&x);
   semctl(p,0,SETVAL,x);
   do{
     g=semctl(p,0,GETVAL,0);
   }while(g==x);
   printf("%d",g);
}
```

```
main()
{  int q,y,z;
   printf("Give semaphore no");
   scanf("%d",&q);
   do{
     y=semctl(q,0,GETVAL,0);
   }while(y==0);
   printf("%d",y);
   printf("Give a number to send");
   scanf("%d",&z);
   semctl(q,0,SETVAL,z);
}
```

The first program reads a number. It is displayed by the second program. Now the second program reads a number. It is displayed by the first program. [Do not give same number as input to both programs. It is because the second program is able to know that the another program has put the value in the semaphore only after observing the change]

Execute following programs using printf("%d\n",) Here \n is necessary.

1. One of the programs reads 10 numbers. The other program displays those numbers. Assume that consecutive numbers are not the same.
2. Do above problem assuming that consecutive numbers may be same but all numbers are positive. [Hint: After fetching the value of the semaphore it is made zero]
3. Let second program is not permitted to update the value of the semaphore. Do above problem. [Hint: the new number is added to the semaphore]
4. Both pr0grams read 10 numbers each. They are displayed by their counter parts. Assume that numbers are given to the programs alternatively. The numbers read by first program are displayed by second program. The numbers read by second program are displayed by first program.
5. Write three programs. The first program creates 2 semaphores. The number of first semaphore is communicated to the second program. The number of second semaphore is communicated to the third program. The second program reads a number the third program displays it.
6. Write two programs. The first program creates a semaphore. The second program is run in two different windows. The first program reads 10 numbers. As soon as it reads a number, it is displayed in both windows of second program. Here first program has one process and the second program has two processes.
7. Write two programs. The first program creates a semaphore. The second program is run in two different windows. The second program reads 5 numbers. As soon as a number is typed in any of the two windows, it is displayed by the first program. A total of 10 numbers are displayed by the first program.
8. Write two programs. The first program creates a semaphore. This program is run in two different windows. The second program reads two semaphore numbers. The second program reads 10 numbers. As soon as a number is given to the second program it is displayed in both windows of the first program.
9. In above problem each of the first programs reads 5 numbers. As soon as a number is typed in any of the two windows of first program, it is displayed by the second program. A total of 10 numbers are displayed by the second program.
10. Write two programs. Ten numbers are given as input to both programs. As soon as a number is given to the first program its double is displayed by the first program itself. As soon as a number is given to the second program it is displayed by the first program. [Hint: fcntl(0,F_SETFL,O_NDELAY); (with include<fcntl.h>)]
11. Ten numbers are given as input to both programs. As soon as a number is given to any program it is displayed by the other program.
12. We want to find the sum of numbers in a given range using parallel programming. A program reads two numbers 'a' and 'b'. It finds mid=(a+b)/2. It communicates the value of 'a' and 'mid' to the other program. These programs find (and display) the sum of numbers between 'a' and 'mid' and between 'mid+1' and 'b' respectively. Both communicate their results to each other. Both display the final sum. Input 6 and 14 mid=(6+14)/2=10. Here 6+7+8+9+10=40 and 11+12+13+14=50 are calculated by these programs. Final result is 90.

UDP

Type following in a file name “s.c”. Compile it using [cc s.c –o s]. Run this program in few windows by ./s. This program reads a destination (port number) and number. It sends the number at the destination.

```
#include<stdio.h>
#include<sys/types.h>
#include<sys/socket.h>
#include<netinet/in.h>
#include<arpa/inet.h>
#include<fcntl.h>
main()
{   int sockfd,a,b,i,len;
    struct sockaddr_in sa,da;
    sockfd=socket(AF_INET,SOCK_DGRAM,0);
    sa.sin_family=AF_INET;
    sa.sin_addr.s_addr=INADDR_ANY;
//    printf("Give self port number:");scanf("%d",&sa.sin_port);
//    i=bind(sockfd,(struct sockaddr *) &sa,sizeof(sa));
//    printf("%d %d\n",sockfd,i);
    len=sizeof(da);

    while(1)
    { da.sin_family=AF_INET;
      da.sin_addr.s_addr=inet_addr("127.0.0.1");
      printf("Give destination port number:");scanf("%d",&b);
      da.sin_port=b;
      printf("Give data to be send");scanf("%d",&a);
      sendto(sockfd,&a,4,0,(struct sockaddr *) &da, sizeof(da));
      recvfrom(sockfd,&a,4,0,(struct sockaddr *) &da,&len);
      printf("%d\n",a);
    }
}
```

Type following in a file name “r.c”. Compile it using [cc r.c –o r]. Run this program in few windows by ./r. When it receives a number then it sends back the double of the number.

```
main()
{   int sockfd,a,b,i,len;
    struct sockaddr_in sa,ta;
    sockfd=socket(AF_INET,SOCK_DGRAM,0);
    sa.sin_family=AF_INET;
    sa.sin_addr.s_addr=INADDR_ANY;
    printf("%d\n",sa.sin_port);
    printf("Give self port number:");scanf("%d",&sa.sin_port);
    i=bind(sockfd,(struct sockaddr *) &sa,sizeof(sa));
    printf("%d %d\n",sockfd,i);
    len=sizeof(ta);

    while(1)
    { ta.sin_family=AF_INET;
      recvfrom(sockfd,&a,4,0,(struct sockaddr *) &ta, &len);
      printf("received %d from port %d\n",a,ta.sin_port);
      a=a*2;
      sendto(sockfd,&a,4,0,(struct sockaddr *) &ta,sizeof(ta));
    }
}
```

1. Write two programs. The first program reads two numbers. The second program finds their sum. The sum is displayed by both the programs. Here port number of the second program is given as input to the first program [through scanf("%d",)]
2. Write two programs. The first program is run in two windows. Each of the instances (processes) of the first program reads a number. They also read the port number of the second program [through scanf("%d",)]. Both processes of the first program display each others input. [Method: Both processes communicate their numbers to the second program, which communicates them to opposite processes]

3. Write three programs. The first program reads 2 numbers 'p' and 'q'. The second program reads a number 't'. Both these programs also read the port number of the third program. Both these programs send their numbers to the third program. The third program computes the value of (q*t)+p. The numbers can be send in any order [pqt or ptq or tpq]. The structure of third program looks as

   recv(x,ta); recv(y,tb); recv(z,tc); a=ta.sin_port; b=tb.sin_port; c=tc.sin_port;
   if (a=b) ans=y*z+x; if (a=c)..; if (b=c) ..

   Structure of the first program scanf(a);scanf(p);send(p,a);scanf(q);send(q,a);
   [Notation: recv(p,q) means p=data from q send(p,q) means send p to destination q]
   . [This can be done in TCP without using "if" but here it is not possible]

4. Write two programs. The first program is run in two windows. Each of the instances (processes) of the first program reads two numbers. They also read the port number of the second program [through scanf("%d",)]. Both processes of the first program send their inputs to the second program. The second program adds numbers from each processes (of the first program) and multiplies it. Let input to these processes be 5,9 and 6,4 respectively. Then output is (5+9)*(6+4)=140. The structure of the first program appears as scanf(a);scanf(p);send(p,a);scanf(q);send(q,a);

   Do this problem under following assumption

   (A) Both numbers of same processes are communicated together. i.e. second program receives it in the order 5964 or 6459.
   (B) First number of both processes is communicated and then second number of both processes is communicated. [received order 5694 or 6549]
   (C) Do part (B) when received order is 5694 or 5649 or 6549 or 6594
   (D) Things can happen in any order.

5. Write two programs: sender and receiver. The sender program runs in two windows. Each of the sender processes reads 5 numbers and sends to the receiver. The receiver outputs the sum of the numbers send by each sender process separately.
6. Write two programs. The first program reads two numbers. The second program finds their sum. The sum is displayed by both the programs. Here port number of the first program is given as input to the second program [through scanf("%d",)] The second program sends his port number (or some other message) to the first program. Now the first program communicates its numbers to the second program.
7. Write a program. It is run in two windows. Each process reads three numbers: p,q,r. 'p' is own port number, 'q' is another process port number. 'r' is input. The value of 'r' is displayed by counter part processes. [Example: Let input to these processes be 12,34,56 and 34,12,45 then these processes will output 45 and 56 respectively].

8. Write two programs. The first program reads a string [through scanf(“%s”,)]. This string is sent to the second program. The second program opens the corresponding file and sends its contents to the first program. It is file transfer.

Message Queue

Sender Program

```
#include<sys/types.h>
#include<sys/ipc.h>
#include<sys/msg.h>
main()
{  int qid,j,i,qn,k;
   struct {long int type;char s[6];} message;
   printf("Give queue no");scanf("%d",&qn);
   qid=msgget((key_t)qn,IPC_CREAT|0666);
   printf("%d",qid);message.type=1;
   for (;;){
     scanf("%d",&i);strcpy(message.s,(char*)&i);message.s[4]='\0';
     j=msgsnd(qid,&message,5,0);printf("%d %d\n",i,j);
   }
}
```

Receiver Program

```
#include<sys/types.h>
#include<sys/ipc.h>
#include<sys/msg.h>
main()
{  int qid,j1,i,qn,k,j;
   struct {long int type;char s[6];} message;
   printf("Give queue no");scanf("%d",&qn);
   qid=msgget((key_t)qn,IPC_CREAT|0666);
   printf("%d\n",qid);
   for (;;){
     j=msgrcv(qid,&message,5,0,IPC_NOWAIT);
     if (j==-1) continue;
     i=0;strcpy((char*)&i,message.s);
     printf("%d %d\n",j,i); sleep(1);
   }
}
```

Open 4 windows. Execute sender and receiver program from 2 windows each.
Observe the output by replacing IPC_NOWAIT by 0. In this case sleep(1) can be removed.

Only mode of communication in following program is message queue.

1. Write a program(chat) and execute it from two different windows. Whenever a number a typed in a window, it is displayed in another window.
2. Write a program and run it in different windows. The program reads self identifier and identifier of other windows. Corresponding to every identifier a queue is open. A pair of numbers can be typed from any of the windows (type both number from same line). The first number is data and another number is window identifier. The corresponding number is displayed in the window, whose identifier is given. [Hint: Use fcntl(0,F_SETFL,O_NDELAY). You may have to include fcntl.h]
3. Write a program and run it in different windows. The program reads self identifier and identifier of few other windows. Corresponding to every identifier a queue is open. When a number is typed in any window, it is displayed in all other windows. [Hint: use concept of flooding] Assume that identifier of other windows are such that corresponding graph is connected.

Remote Login

```
#include<stdio.h>
main()
{  FILE *fp1,*fp2;int i;
   char s[100],t[10],u[10];
   printf("input file name");
   scanf("%s",t);
   printf("output file name");
   scanf("%s",u);
   fp1=fopen(t,"r");
   fp2=fopen(u,"w");
   while (1)
   {  fgets(s,40,fp1);
      if (feof(fp1)) break;
      printf("%s",s);
   }
}
```

This program reads two files names. The program displays the contents of the file. The program also copies the contents of the first file on the second file.

1. Write a program (client), which reads a file name. It sends that to server. The server opens the corresponding file and sends first 10 lines to the client. The client displays that. It if file transfer: get file.
2. Modify above program so that complete file is transferred. [Hint: The server goes through the file and counts the number of lines. Firstly the server sends the number of lines (n) to the client. The client executes the receive statement 'n' times. The server reopens the file and sends the file contents.
3. Define file transfer "put file".

```
#include<stdio.h>
main()
{  char t[20];
   gets(t);system(t);
}
```

This program reads a string. It is treated as a command. The contents of the corresponding commands are displayed. Observer the output of the commands like "ls", "cat a.c", "data", "ls –l".

4. Define client. It reads a string and sends to the server. The server executes the corresponding command and displays the contents.
5. When command "ls > tt" is given, the file names are put in file "tt". When command "cat a.c > tt" is given then contents of file a.c are put in file tt. Similar happens to other commands also. Define client. It reads a string (command of the type "…>tt"). The client sends the string to the server. The server executes it using system(). Now server opens file "tt" using (fp=fopen("tt","r");) The server sends the contents of the file to the client. The client displays it.

```
main()
{  char a[9],b[9],c[9],d[9];
   printf("Give 3 words");
   scanf("%s %s %s",a,b,c);
   if (strcmp(a,"ram")==0)
   {  strcat(b,c);
      printf("[%s]",b); }
   else
   {  strcpy(d,"hari");
      strcat(d,b);
      printf("[%s %s]",d,b);
   }
}
```

The given program reads a string of three words. If first word is ram then 2nd and 3rd words are joined. Otherwise "hari" and 2nd word are joined. The 3rd word is ignored.

6. Write program, which reads a string. The program joins string " > tt" to that and outputs that. Input "ram" output "ram > tt". Input "ram hari" output "ram hari > tt". [read string using gets()]
7. Define client. It reads a string (command). The client joins " > tt" to the string and sends to the server. The server executes it. After that server sends the contents of the file "tt" to the client.
8. Write program, which reads two words. If first word is "directory" then a string "ls > tt" is outputted. If first word is "type" then string "cat 2nd word" is outputted.
9. Define client, it reads two words. If first word is "directory" then files in the "directory" in which server is running is outputted. If first word is "type" then contents of corresponding file are displayed. If first word is "date" then time and date of server is displayed (on client port).

However command "cd pp" can not be executed using system(). When command "cd pp" is given then variable st (char st[40];initialized as ".") is modified as "./pp". Further execution of "cd tt" and "cd .." modified it as "./pp/tt/..". Now if command "ls" is given then it is executed as "ls ./pp/tt/..". Similarly "cat a.c" is executed as "cat ./pp/tt/../a.c".

10. Write program, which reads a sequence of pair of words in infinite loop. If “ram hari” is given then a string “./ram hari” is displayed. If “cd kapil” is given then a variable ‘st’ is changed to “./kapil”. Now if “ram hari” is given then “./kapil/ram hari” is displayed.
11. Write program, which reads a sequence of pair of words in infinite loop. If “cd kapil” is given than ‘st’ is updated as ./kapil. Now if “cd ravi” is given then it is updated as ./kapil/ravi. Now if “type a.c” is given then string “cat ./kapil/ravi/a.c” is displayed. Now if “directory hari” is given then “ls ./kapil/ravi/hari” is displayed. Now if “directory current” is given then “ls ./kapil/ravi” is displayed. [For commands of single word e.g. ls, data the second word is not part of string]

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <fcntl.h>
main()
{
        int sockfd,ii,tt,addr_len;char m[40];
      struct sockaddr_in self_addr,send_addr,recv_addr;
      sockfd = socket(AF_INET, SOCK_DGRAM, 0);
      self_addr.sin_family          = AF_INET;
      self_addr.sin_addr.s_addr     = INADDR_ANY;
      printf("Give self port no:");scanf("%d",&self_addr.sin_port);
      ii=bind(sockfd, (struct sockaddr *) &self_addr,sizeof(self_addr));
      printf("%d %d\n",sockfd,ii);
        send_addr.sin_family        = AF_INET;
      send_addr.sin_addr.s_addr     = inet_addr("127.0.0.1");
        recv_addr.sin_family        = AF_INET;
      recv_addr.sin_addr.s_addr     = inet_addr("127.0.0.1");
        fcntl(0,F_SETFL,O_NDELAY);addr_len=sizeof(recv_addr);
        fcntl(sockfd,F_SETFL,O_NDELAY);addr_len=sizeof(recv_addr);
        printf("Give Receiver's port no and a string to send:");
        for (;;){
          ii=-1;
          scanf("%d",&ii);send_addr.sin_port=ii;
          if (ii!=-1){gets(m);
            sendto(sockfd, &m, 10, 0,(struct sockaddr *)
&send_addr,sizeof(send_addr));}
        tt=recvfrom(sockfd, &m, 10, 0,(struct sockaddr *) &recv_addr,&addr_len);
          if (tt>0) {printf("Received %s from
%d\n",m,recv_addr.sin_port);puts(m);system(m);}
        }
}
```

Write two programs client and server. Run server program in few windows (ports). Make different directories
in these ports. Run client program in some window. It will ask a port number and will do (virtual) remote login
in that port. Any command typed in client will be executed in the context of that server.
When "ls" is typed then it is send to server. Server executes it as "ls>tt". Now contents of tt are
sent to client. Same hold for command like "cat a.c". Command "cd" will be handeled by above defined technique.

Broad Cast communication

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <fcntl.h>
main()
{
      int sockfd,ii,m,tt,addr_len;
      struct sockaddr_in self_addr,send_addr,recv_addr;
      sockfd = socket(AF_INET, SOCK_DGRAM, 0);
      self_addr.sin_family          = AF_INET;
      self_addr.sin_addr.s_addr     = INADDR_ANY;
      printf("Give self port no:");scanf("%d",&self_addr.sin_port);
      ii=bind(sockfd, (struct sockaddr *) &self_addr,sizeof(self_addr));
      printf("%d %d\n",sockfd,ii);
      send_addr.sin_family          = AF_INET;
      send_addr.sin_addr.s_addr     = inet_addr("127.0.0.1");
      recv_addr.sin_family          = AF_INET;
      recv_addr.sin_addr.s_addr     = inet_addr("127.0.0.1");
      fcntl(0,F_SETFL,O_NDELAY);addr_len=sizeof(recv_addr);
      fcntl(sockfd,F_SETFL,O_NDELAY);
      printf("Give Receiver's port no and message in same line:");
      for (;;){
          ii=-1;
          scanf("%d %d",&ii,&m);send_addr.sin_port=ii;
          if (ii!=-1)
            sendto(sockfd,&m,4,0,(struct sockaddr *) &send_addr,sizeof(send_addr));
          tt=recvfrom(sockfd, &m, 4, 0,(struct sockaddr *) &recv_addr,&addr_len);
          if (tt>0) printf("Received %d from %d\n",m,recv_addr.sin_port);
      }
}
```

Observe the behavior of above program by running in 4 different windows.

12. Write two programs lan.c and node.c.

(A) The first program (lan.c) asks for port number of lan (self port). Whenever any port sends any data to lan then the same data is sent to all nodes connected to lan. If a port, which is not connected to lan sends data to lan, it gets connected. [Hint: The lan.c program stores the address of all nodes connected to it in an array. It is maintained as follows: as soon as some data is received then the senders address is stored in the array (if not already there).]

(B) The second program asks for self port and lan port number. After that any input to this program is sent to lan. When this program receives any input from lan it displays. The program node.c is similar to what is given above. [The address of LAN (send_addr) is read outside the loop]

13. When a node wants to be disconnected it sends -1. [Hint: If -1 is received then senders address is removed from the array]

In following programs no modification in lan program should be made unless permitted.

14. Modify the program node.c. User can send sequence of numbers. The first number is receive's port number. After that sequence of data numbers is given. When a user wants to stop sending it sends -2. If it again wants to send then it will again give receiver's port number. [Hint: all numbers are sent to the lan. The lan sends all of them to all nodes. However only one of the node displays them (whose address is the first number)]

[Hint: Whatever is scanned is sent. Initially flag is 0. When something (X) is received

(A) If flag=0 and X=self port then flag=1

(B) If flag=1 then display X
(C) If flag=1 and received is -2 then flag=0]

15. CSMA: Modify the program of Node. When a user gives input from some node and some other node is already sending then the user is stopped from doing so. (i.e. tt and ii are bigger than 0).
    (A) If flag=0 and X $\neq$ self port then flag=2
    (B) If flag=2 and something is scanned then print stop
16. CSMA/CD: Use sleep(2) in lan.c. Here data to send to all nodes 2 seconds after it has received. Hence when a user gives input from some node and some other node is already sending then it can not be stopped. However as soon as it comes to know that some one is sending then it is stopped.

Ring

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <fcntl.h>
main()
{
      int sockfd,ii,m,tt,addr_len,past;
      struct sockaddr_in self_addr,send_addr,recv_addr;
      sockfd = socket(AF_INET, SOCK_DGRAM, 0);
      self_addr.sin_family          = AF_INET;
      self_addr.sin_addr.s_addr     = INADDR_ANY;
      printf("Give self port no and next port no:");
      scanf("%d %d",&self_addr.sin_port,&send_addr.sin_port);
      ii=bind(sockfd, (struct sockaddr *) &self_addr,sizeof(self_addr));
      printf("%d %d\n",sockfd,ii);
      send_addr.sin_family          = AF_INET;
      send_addr.sin_addr.s_addr     = inet_addr("127.0.0.1");
      recv_addr.sin_family          = AF_INET;
      recv_addr.sin_addr.s_addr     = inet_addr("127.0.0.1");
      fcntl(0,F_SETFL,O_NDELAY);addr_len=sizeof(recv_addr);
      fcntl(sockfd,F_SETFL,O_NDELAY);addr_len=sizeof(recv_addr);
      for (;;){
          m=-1;
          scanf("%d",&m);
          if (m!=-1){
              past=m;
              sendto(sockfd,&m,4,0,(struct sockaddr
*)&send_addr,sizeof(send_addr));
          }
          tt=recvfrom(sockfd, &m, 4, 0,(struct sockaddr *) &recv_addr,&addr_len);
          if (tt>0){
              printf("Received %d \n",m);sleep(1);
              if (past!=m)
               sendto(sockfd,&m,4,0,(struct
sockaddr*)&send_addr,sizeof(send_addr));
          }
        }
}
```

Open 4 windows. Give port numbers in them as 10 34, 12 10, 34 56, 56 12. Now a ring with port numbers 10 34 56 12 10 is ready. If we type any number in any of the windows, It will come in others in cyclic order.

1. When we type two numbers at a fast speed then infinite loop takes place. Make arrangement so that this does not happen.

2. Make arrangement so that any new node can be added in the ring. Let a node (say port number 91) wants to be added between 34 and 56. It will send to port number 34, its own port number. Port no. 34 will send it port no 56. Both ports will modify its send address appropriately. [Caution: Here port numbers should be sent in special manner so that they are not confused by ordinary number. e.g negative of port number might be sent.]

3. Make arrangement so that any node can be deleted from the ring. Let port number 12 wants to leave. It will put on ring its own port number and its next port number (send address). When node number 56 will receive it, it will modify its send address.

FIFO

Make a FIFO (named abc) by giving command mkfifo abc. Run the following programs in two different windows. Both Yes's are printed together. It shows that a FIFO can not be opened for reading till it is opened for writing (and vice versa).

```
#include<fcntl.h>
main()
{  int rd,i;char c;
   rd=open("abc",O_RDONLY,0);
   printf("yes\n");
   for (i=0;i<=100;i++)
   {  read(rd,&c,1);
      printf("%c\n",c);sleep(1);
   }
}
```

Observe the effect by putting following after open. fcntl(rd,F_SETFL,O_NDELAY);

```
#include<fcntl.h>
main()
{  int wr,i;char c[5];
   wr=open("abc",O_WRONLY,0);
   printf("yes\n");
   for (i=0;i<=10;i++)
   {  gets(c);
      write(wr,c,3);
   }
}
```

When the second program becomes over then first program does not wait.
If we reduce the size of loop in first program then second program terminates after some time (meaasge broken pipe). If we try to write into FIFO from where no one can read then this happens.

Make four windows. Run 2 instances of each of above programs.

A FIFO can also be made by executing following file. FIFO can be deleted by command rm abc.

```
#include<fcntl.h>
#include<sys/stat.h>
main()
{
   mkfifo("abc",(S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH));
}
```

1. Write a program, which reads n and creates ‘n’ FIFO’s. Their names are p1, p2, p3, p4 and p5.

2. Make three windows, whatever is typed in first window comes in other two windows. Only mode of communication is FIFO. [Hint: use following in place of scanf]

```
fcntl(0,F_SETFL,O_NDELAY);                no wait on input from keyboard
ii=read(0,s,100); if (ii>0) ........             equivalent to scanf.
  Or
t=-20;scanf("%d",&t);if (t!=-20) . . . . .
```

11.

TCP

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include<fcntl.h>
main()
{     int clilen,cfd,sockfd,ii;char s[100];
      struct sockaddr_in cli_addr, serv_addr;
      sockfd = socket(AF_INET, SOCK_STREAM, 0);
      serv_addr.sin_family          = AF_INET;
      serv_addr.sin_addr.s_addr     = INADDR_ANY;
      serv_addr.sin_port            = 6001;
      ii=bind(sockfd, (struct sockaddr *) &serv_addr,sizeof(serv_addr));
      printf("%d\n",ii);
      listen(sockfd, 5);
      clilen = sizeof(cli_addr);
      cfd = accept(sockfd, (struct sockaddr *) &cli_addr,&clilen) ;
      printf("Client connected\n");
        do{
          recv(cfd,s,100,0);
          printf("%s",s);
          gets(s);
          send(cfd,s,100,0);
        } while (1);
}
```

```
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include<fcntl.h>
main()
{     int ii,sockfd;char s[100];
      struct sockaddr_in serv_addr;
      sockfd=socket(AF_INET, SOCK_STREAM, 0);
      serv_addr.sin_family          = AF_INET;
      serv_addr.sin_addr.s_addr     = inet_addr("127.0.0.1");
      serv_addr.sin_port            = 6001;
      connect(sockfd, (struct sockaddr *) &serv_addr,sizeof(serv_addr));
      printf("%d\n",sockfd);
        do{
          gets(s);
          send(sockfd,s,100,0);
          ii=recv(sockfd,s,100,0);
          printf("%s",s);
        } while (1);
}
```

UDP Send Receive

Sender

```
#include<stdio.h>
#include<sys/types.h>
#include<sys/socket.h>
#include<netinet/in.h>
#include<arpa/inet.h>
#include<fcntl.h>
main()
{   int sockfd,a,b,i;
    struct sockaddr_in sa,da;
    sockfd=socket(AF_INET,SOCK_DGRAM,0);
    sa.sin_family=AF_INET;
    sa.sin_addr.s_addr=INADDR_ANY;
    printf("Give self port number:");scanf("%d",&sa.sin_port);
    i=bind(sockfd,(struct sockaddr *) &sa,sizeof(sa));
    printf("%d %d\n",sockfd,i);

    while(1)
    { da.sin_family=AF_INET;
      da.sin_addr.s_addr=inet_addr("127.0.0.1");
      printf("Give destination port number:");
      scanf("%d",&b); da.sin_port=b;
      printf("Give data to be send");scanf("%d",&a);
      sendto(sockfd,&a,4,0,(struct sockaddr *) &da, sizeof(da));
    }
}
```

Receiver

```
main()
{   int sockfd,a,b,i,len;
    struct sockaddr_in sa,ta;
    sockfd=socket(AF_INET,SOCK_DGRAM,0);
    sa.sin_family=AF_INET;
    sa.sin_addr.s_addr=INADDR_ANY;
    printf("Give self port number:");scanf("%d",&sa.sin_port);
    i=bind(sockfd,(struct sockaddr *) &sa,sizeof(sa));
    printf("%d %d\n",sockfd,i);
    len=sizeof(ta);

    while(1)
    { ta.sin_family=AF_INET;
      recvfrom(sockfd,&a,4,0,(struct sockaddr *) &ta, &len);
      printf("received %d from port %d\n",a,ta.sin_port);
    }
}
```

Modify programs of sender and receiver. When receiver gets a number, it multiplies it by 2 and send back to sender.

Implement File transfer from one system to another. Assistance of following program can be taken.

```
#include <stdio.h>
main()
{       FILE *fp1,*fp2;int i;char s[100],t[10];
        printf("Give  input file name");scanf("%s",&s);
        printf("Give output file name");scanf("%s",&t);
        fp1=fopen(s,"r");fp2=fopen(t,"w");
        for (i=1;i<=10;i++)
        {   fgets(s,40,fp1);
            fprintf(fp2,"%s\n",s);
            printf("%sttt %d %d\n",s,s[0],strlen(s));
        }
}
```

This program is for input-output from files.

UDP Chat

```
#include<stdio.h>
#include<sys/types.h>
#include<sys/socket.h>
#include<netinet/in.h>
#include<arpa/inet.h>
#include<fcntl.h>
main()
{   int sockfd,a,b,i,length;char id[50],m[50];
    struct sockaddr_in sa,da,ta;
    sockfd=socket(AF_INET,SOCK_DGRAM,0);
    sa.sin_family=AF_INET;
    sa.sin_addr.s_addr=INADDR_ANY;
    printf("Give self port number:");scanf("%d",&sa.sin_port);
    i=bind(sockfd,(struct sockaddr *) &sa,sizeof(sa));
    printf("%d %d\n",sockfd,i);
    fcntl(0,F_SETFL,O_NDELAY);fcntl(sockfd,F_SETFL,O_NDELAY);
    length=sizeof(sa);
    printf("Give destination address,port number
            and message in same line:");
    while(1)
    { da.sin_family=AF_INET;ta.sin_family=AF_INET;
      b=-1; scanf("%s %d %s",&id,&b,&m);
      if (b!=-1)
      {  da.sin_addr.s_addr=inet_addr(id);da.sin_port=b;
         sendto(sockfd,&m,20,0,(struct sockaddr *)&da,length);
      }
      i=recvfrom(sockfd,&m,20,0,(struct sockaddr *)&ta,&length);
      if (i!=-1) printf("Received %s from %d\n",m,ta.sin_port);
    }
}
```

Implement remote login. Assistant of following programs may be taken,

```
main()
{  char s[100],t[100],u[100];int i,j;
   printf("Give two strings");gets(s);gets(t);strcat(s,&t[3]);puts(s);
   s[3]='\0';puts(s);
   strcpy(u,"KL");strcat(u,s);puts(u);
   strcpy(t,&u[3]);puts(u);
   i=strcmp("ram","ram");j=strcmp("ram","hari");printf("%d %d\n",i,j);
}
main()
{   char t[20];
    system("ls");
    system("ls .. > tt"); Observe the contents of file tt
    system("cd ..");
    printf("Give a string");gets(t);system(t);
}
```

The above program reads a string and executes corresponding command. If input is "ls" the output is corresponding files. Same holds for command "cat a.c". However command "cd pp" can not be executed. When command "cd pp" is given then variable st (char st[40];initialized as ".") is modified as "./pp". Further execution of "cd tt" and "cd .." modifie it as "./pp/tt/..". Now if command "ls" is given then it is executed as "ls ./pp/tt/..". Similarly “cat a.c” is executed as "cat ./pp/tt/../a.c".

```
main()
{  FILE *fp1,*fp2;int i;char s[100],t[10];
   printf("Give input file name ");gets(s);
   fp1=fopen(s,"r");fp2=fopen("tk","w");
   while (1)
   {   fgets(s,40,fp1);
       if (feof(fp1)) break;
       fprintf(fp2,"%s",s);printf("%s",s);
   }
}
```

The above program displays contents of a file and puts them in a file "tk".

UDP

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
main()
{
      int sockfd,addr_len,ii;char s[100];
      struct sockaddr_in self_addr,fr_addr;
      sockfd = socket(AF_INET, SOCK_DGRAM, 0);
      self_addr.sin_family          = AF_INET;
      self_addr.sin_addr.s_addr     = INADDR_ANY;
      printf("Give self port no:");scanf("%d",&self_addr.sin_port);
      ii=bind(sockfd, (struct sockaddr *) &self_addr,sizeof(self_addr));
      printf("%d %d\n",sockfd,ii);
        addr_len=sizeof(fr_addr);
      recvfrom(sockfd, s, 100, 0,(struct sockaddr *) &fr_addr,&addr_len);
      printf("%s\n",s);s[3]='2';
      sendto(sockfd, s, 100, 0,(struct sockaddr *) &fr_addr,sizeof(fr_addr));
}
Run above and following program on two differnet windows.
main()
{     int sockfd,addr_len,ii;
      struct sockaddr_in self_addr,fr_addr;
      char s[100];
      sockfd = socket(AF_INET, SOCK_DGRAM, 0);
//    self_addr.sin_family          = AF_INET;
//    self_addr.sin_addr.s_addr     = INADDR_ANY;
//    printf("Give self port no:");scanf("%d",&self_addr.sin_port);
//    ii=bind(sockfd, (struct sockaddr *) &self_addr,sizeof(self_addr));
      printf("%d %d\n",sockfd,ii);
      fr_addr.sin_family            = AF_INET;
      fr_addr.sin_addr.s_addr       = inet_addr("127.0.0.1");
      printf("Give friend port no:");
        scanf("%d",&ii);fr_addr.sin_port=ii;
        printf("Give a string to send\n");scanf("%s",s);
        addr_len=sizeof(fr_addr);
      sendto(sockfd, s, 100, 0,(struct sockaddr *) &fr_addr,sizeof(fr_addr));
      recvfrom(sockfd, s, 100, 0,(struct sockaddr *) &fr_addr,&addr_len);
      printf("%s\n",s);
}
                        Program for input-output from files
#include <stdio.h>
main()
{       FILE *fp1,*fp2;int i;char s[100],t[10];
        printf("Give  input file name");scanf("%s",&s);
        printf("Give output file name");scanf("%s",&t);
        fp1=fopen(s,"r");fp2=fopen(t,"w");
        for (i=1;i<=10;i++)
        {   fgets(s,40,fp1);
            fprintf(fp2,"%s\n",s);
            printf("%sttt %d %d\n",s,s[0],strlen(s));
        }
}
```

Chat(TCP)

```
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <fcntl.h>
main()
{     int ii,sockfd;char s[100];
      struct sockaddr_in serv_addr;
      sockfd=socket(AF_INET, SOCK_STREAM, 0);
      serv_addr.sin_family          = AF_INET;
      serv_addr.sin_addr.s_addr     = inet_addr("127.0.0.1");
      serv_addr.sin_port            = 6001;
      connect(sockfd, (struct sockaddr *) &serv_addr,sizeof(serv_addr));
      printf("%d\n",sockfd);
      fcntl(sockfd,F_SETFL,O_NDELAY);   // no wait on receive
      fcntl(0,F_SETFL,O_NDELAY);        // no wait on input from keyboard
      do{
        ii=read(0,s,100);               // equivalent to scanf.
                                           Returns -1 if input not present.
        if (ii>0) send(sockfd,s,100,0);
        ii=recv(sockfd,s,100,0);        // ii is number of letters read
        if (ii>0) printf("%s %d\n",&s[1],(s[0]-48)*2);
      } while (1);
}


#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <fcntl.h>
main()
{     int clilen,cfd,sockfd,ii;char s[100];
      struct sockaddr_in cli_addr, serv_addr;
      sockfd = socket(AF_INET, SOCK_STREAM, 0);
      serv_addr.sin_family          = AF_INET;
      serv_addr.sin_addr.s_addr     = INADDR_ANY;
      serv_addr.sin_port            = 6001;
      ii=bind(sockfd, (struct sockaddr *) &serv_addr,sizeof(serv_addr));
      printf("%d\n",ii);
      listen(sockfd, 5);
      clilen = sizeof(cli_addr);
      cfd = accept(sockfd, (struct sockaddr *) &cli_addr,&clilen) ;
      printf("Client connected\n");
      fcntl(cfd,F_SETFL,O_NDELAY);
      fcntl(0,F_SETFL,O_NDELAY);
      do{
        ii=read(0,s,100);s[ii]='\0';
        if (ii>0) send(cfd,s,100,0);
        ii=recv(cfd, s, 100,0);
        if (ii>0) printf("%s\n",s);
      } while (1);
}
```

Remote Execution

Remote Server

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
main()
{
      int sockfd, clfd,clilen;int i,j,n;float t,x[100];
      struct sockaddr_in  cli_addr, serv_addr;

      sockfd = socket(AF_INET, SOCK_STREAM, 0);
      serv_addr.sin_family          = AF_INET;
      serv_addr.sin_addr.s_addr     = INADDR_ANY;
      serv_addr.sin_port            = 6000;
      bind(sockfd, (struct sockaddr *) &serv_addr,sizeof(serv_addr));
      listen(sockfd, 5);
      clilen = sizeof(cli_addr);
      clfd = accept(sockfd, (struct sockaddr *) &cli_addr,&clilen) ;

        printf("Howmany numbers");scanf("%d",&n);
        printf("Give those numbers");
        for (i=0;i<n;i++) scanf("%f",&x[i]);
        printf("\n Initial values are: ");
        for (i=0;i<n;i++) printf("(%d%5.1f) ",i,x[i]);

        printf("\nThe variables whose value is needed: ");
        do{
           recv(clfd,&i,4,0);
           if (i==-1) break;printf("%d ",i);
           send(clfd,&x[i],4,0);
        }while(2==2);

        printf("\nVariables whose value is modified: ");
        do{
           recv(clfd,&i,4,0);
           if (i==-1) break;printf("%d ",i);
           recv(clfd,&x[i],4,0);
        }while(2==2);
        printf("\nModified values are: ");
        for (i=0;i<n;i++) printf("(%d%5.1f) ",i,x[i]);
}
```

The above program gives following output

```
Howmany numbers  7
Give those numbers  4  12  1.5  3.9  4.6  22.7  4.7
 Initial values are: (0 4.0) (1 12.0) (2  1.5) (3 3.9) (4 4.6) (5 22.7) (6  4.7)
The variables whose value is needed: 2 5 6 4
Variables whose value is modified:   1 2 3 5
Modified values are: (0 4.0) (1 34.2) (2 25.9) (3 9.2) (4 4.6) (5 22.7) (6  4.7)
```

Remote Client

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
int sockfd,i,j;struct sockaddr_in serv_addr;
int valid[100],dirty[100];float y[100];
float g(int i)
{       if (valid[i]==0){
             send(sockfd,&i,4,0);
             recv(sockfd,&y[i],4,0);
             valid[i]=1;}
        return(y[i]);
}
void s(int i,float t)
{       y[i]=t;dirty[i]=1;valid[i]=1;}
void modify()
{       i=-1;send(sockfd,&i,4,0);
        for (i=0;i<100;i++)
           if (dirty[i]==1) {
                 send(sockfd,&i,4,0);
                 send(sockfd,&y[i],4,0);
           }
        i=-1;send(sockfd,&i,4,0);
}

main()
{     for (i=0;i<100;i++) { valid[i]=0;dirty[i]=0;}

      sockfd = socket(AF_INET, SOCK_STREAM, 0);
      serv_addr.sin_family          = AF_INET;
      serv_addr.sin_addr.s_addr     = inet_addr("10.xx.xx.xxx");
      serv_addr.sin_port            = 6000;
        connect(sockfd, (struct sockaddr *) &serv_addr,sizeof(serv_addr));

        s(3,g(2)*12);
        s(1,34.2);
        s(2,g(5)+g(6)-g(2));
        s(5,g(3)+g(6));
        s(3,2*g(4));

        modify();
}
The equivalent remote program is:
```

$V_3 = V_2 * 12$

$V_1 = 34.2$

$V_2 = V_5 + V_6 - V_2$

$V_5 = V_3 + V_6$

$V_3 = 2 * V_4$

Here V_0, V_1, V_2 ….V_6 are variables.

PROLOG

Boolean function
String
Arithmetic
List
Natural language processing
Multi valued function
Compound Object
Read/write
Simulation
File
Window (not typed)
Graphics (not typed)
Dynamic Database
Cut
Lisp

Fundamentals

In PROLOG

1. f(X)=Y is written as f(X,Y).
2. f(X,Y)=Z is written as f(X,Y,Z).
3. f(X)=true is written as f(X).
4. f(X)=false is written as not(f(X)).
5. X ≠ Y is written as not(X=Y).
6. ‘if’ is written as ‘:-‘. It means ‘ ← ’.
7. ‘and’ is written as ‘,’(comma).
8. ‘anything’ is written as ‘_’(underscore). (f(X,Y) can be written as f(_,Y) if in the clause variable X is not used anywhere else)
9. All variables are written in capital.
10. Range of integers is between –32768..32767.
11. ‘A div B’ gives quotient when A is divided by B. e.g. 74 div 7=10.
12. ‘A mod B’ gives remainder when A is divided by B. e.g. 4572 mod 10=2.
13. [2,7,9,3]=[A|B] will make A=2 and B=[7,9,3]. Similarily if P=4 and Q=[3,2,7,9] then R=[P|Q] will make R=[4,3,2,7,9].
14. A=[P|Q] returns false if A is null.

Boolean functions

```
predicates
  even(integer). abc(integer). pqr(integer). someeven(integer).
  allmore(integer,integer). xyz(integer,integer).first(integer,integer,integer).
  sum(integer,integer,integer).nf(integer,integer,integer).
clauses
  even(X):-X mod 2=0.
  abc(X):-X mod 2=0, X>50.
  pqr(X):-X mod 2=0.
  pqr(X):-X>50.
  someeven(X):-X mod 2=0.
  someeven(X):-X>10,X mod 2>0,T=X div 10,someeven(T).
  allmore(X,Y):-X<10,X>Y.
  allmore(X,Y):-X>=10,X mod 10>Y,S=X div 10,allmore(S,Y).
  xyz(X,Y):-someeven(X),allmore(X,Y).
  first(X,Y,Z):-X<10,X<Y.
  first(X,Y,Z):-X<10,X>Z.
  first(X,Y,Z):-X>=10,T=X div 10, first(T,Y,Z).
  nf(X,Y,Z):-Y>Z.
  nf(X,Y,Z):-Y<=Z,T=Y+1,nf(X,T,Z),X mod Y>0.
  sum(X,Y,Z):-Y=Z,Z=X.
  sum(X,Y,Z):-Y<Z,T=Y+1,G=X-Y,sum(G,T,Z).
```

1. even(x) returns true if x is even. All numbers are between -32768 and 32767. All variable names must be capital.
2. abc(x) returns true if x is an even number bigger than 50.
3. pqr(x) returns true if x is either an even number or is more than 50.
4. someeven(x) returns true if some digit in given number x is even. [x mod 2>0 can be removed]
5. allmore(x,y) returns true if all digits in x are more than y.
6. xyz(x,y) returns true if x has some even digit and all digits are more than y.
7. first(x,y,z) returns true if first digit of x is not between y and z. Assume $y \le z$. In the first clause first(X,Y,_) and in the second clause first(X,_,Y) can be used. '_' is underscore. It means that clause does not depend on corresponding variable.
8. nf(x,y,z) returns true if no factor of x is between y and z. nf(42,8,12) is true. Assume $y \le z$.
9. sum(x,y,z) returns true if x= $\sum_{i=y}^{i=z} i$. sum(18,3,6) is true. Assume $y \le z$. Here first clause can also be written as sum(X,X,X).

Define following functions [no other predicate should be used unless permitted(AP)**]**

1. Diff(x,y). It returns true if difference between x and y is less then 5. e.g. diff(7,9) and diff(9,6) are true but diff(5,12) and diff(12,5) are false.
2. del(X,Y). It returns true if Y can be obtained by deleting 2nd last digit from X. e.g. del(31563,3153) is true.
3. Present(x,y). It returns true if digit y is present in x. e.g. present(1343,3) is true. Present(1343,5) is false.
4. Absent(x,y). It returns true if digit y is absent in x.
5. Pr(x,y). It returns true if digit y is present in x exactly once.
6. Sm(x,y). It returns true if y is the smallest digit of x. Additional predicate can be used. [Hint: y is present in x and all digits of x are more than or equal to y]
7. Present2(x,y). It returns true if digit y is present in x at least two times. e.g. present2(1343,3) is true. Present2(1343,4) is false. Additional predicate can be used.
8. Present3(x,y). It returns true if digit y is present in x exactly two times. Additional predicate can be used.
9. Sod(x,y). It returns true if sum of all digits of x is y.

10. Fact(x,y). It returns true if factorial (x) is y. fact (5,120) is true. fact (5,125) and fact (6,96) are false.
11. Facti(x,y,z). It returns true if factorial(x) is between y and z. facti(5,90,124) is true. facti(5,125,190) and facti(6,96,200) are false. Assume $y \leq z$.
12. Alleven(x). It returns true if all digits in the given number are even.
13. Neven(x,y). It returns true if at least y digits of x are even.
14. Meven(x,y). It returns true if exactly y digits of x are even.
15. Somemore(x,y). It returns true if some digit in x is more than y.
16. Bd(x,y,z). It returns true if some digit of x is between y and z.
17. Nbd(x,y,z). It returns true if no digit of x is between y and z.
18. Deleteeven(x,y). It returns true if the number y is formed from x after deleting all even digits. e.g. deleteeven(5623,53), deleteeven(25437,537), deleteeven(58,8) and deleteeven(5337,0) are true. deleteeven(5297,573) is false.
19. Factor(x,y,z). It returns true if some factor of x is between y and z.
20. Prime(x). It returns true if x is a prime number. [Hint: use nf (not a factor)]
21. Gcd(x,y,z). It returns true if z is the greatest common divisor of x and y.
22. Sorted(x). It returns true if digits of x are in increasing order. sorted(357) and sorted(23567) are true. Sorted(5378) and sorted(2344) are false.
23. More(x,y). It returns true if every digit of x is bigger than corresponding digit of y. more(527,412) is true. more(5279,6322) is false. [Assume x and y have same number of digits]
24. Df(x,y). It returns true if at least one digit of x is different than corresponding digit of y. Df(2437,2737) and D(2439,2738) are true.
25. Dff(x,y). It returns true if exactly one digit of x is different than corresponding digit of y. Dff(2437,2737) is true but D(2439,2738) is false.
26. f(x,y,z). It is true if y is product of digits of x and z is special sum of product. Special sum of product of 5429 is 5*4*2*9+5*4*2+5*4+5=425. Hence f(5429,360,425) is true. Similarly f(235,30,38) is true. But f(235,28,38) and f(235,30,37) are false. Assume that the number x does not has any digit as 0.
27. g(x,y,z,w). It is true if z=x! and w= $\sum_{i=x}^{i=y} i!$. g(3,5,6,150) and g(2,6,2,872) are true. Assume $x \leq y$.
28. h(x,y,z,w). It is similar as above except z=y!.

String

frontchar(symbol,char,symbol) is an inbuilt predicates. frontchar(hari,X,Y) gives X=’h’ Y=ari. frontchar(X,’t’,yuio) gives X=tyuio. Symbol can have more than one word. In this case it is written in quotes(“) e.g. frontchar(“Ram is good”,X,Y) gives X=’R’ and Y=”am is good”.

fronttoken is another inbuilt predicate. fronttoken(“Ram is a good boy”,X,Y) gives X=Ram Y=”is a good boy”.

char_int(char,integer) is inbuilt function. char_int(‘A’,X) gives X=65. char_int(X,65) gives X=’A’. char_int(‘A’,65) gives true. Char_int(‘A’,66) gives false.

```
predicates
  second(symbol,char).more(char,char).last(symbol,symbol).
  count(symbol,integer).bigger(symbol,symbol).
clauses
  second(X,Y):-frontchar(X,_,P),frontchar(P,Y,_).
  more(X,Y):-char_int(X,A),char_int(Y,B),A>B.
  count(X,Y):-X="",Y=0.
  count(X,Y):-not(X=""),T=Y-1,frontchar(X,_,Z),count(Z,T).
  last(X,Y):-fronttoken(X,Y,"").
  last(X,Y):-fronttoken(X,_,Z),not(Z=""),last(Z,Y).
```

1. second(X,Y) returns true if Y is second letter of symbol X. second(ram,’m’) is true.
2. more(X,Y) return true if X is bigger letter than Y. more(‘k’,’g’) is true.
3. count(X,Y) is true if the number of letters in X are Y. The first clause can also be defined as count(“”,0).
4. last(X,Y) is true if Y is the last word of X. last(“ram is good”,good) is true.
5. bigger(X,Y) is true if X is lexicographically bigger than Y. bigger(rat,ramesh) is true.

Define following functions [First 5 questions should be done by using one clause only**]**

1. increment(X,Y). It is true if Ascii code of first letter of X is 1 more than that of the first letter of Y. increment(ram,qjkj) is true.
2. inc(X,Y). It is true if Ascii code of first letter of X is 1 more than that of the first letter of Y and remaining string is same. inc(ram,qjkj) is false. inc(ram,qam) is true.
3. del2(X,Y). It is true if Y can be obtained by deleting 2nd letter from X. del2(ram,rm) is true.
4. exchange(X,Y). It is true if Y can be obtained by exchanging first two letters of X. exchange(hari,ahri) is true. exchange(hari,ahry) is false.
5. join(X,Y,Z,W,T). It is true if X is a word of 4 letters. The first letters is Y. The remaing letters are Z, W and T respectively. Here X is a symbol and Y, Z, W and T are char. join(raja,’r’,’a’,’j’,’a’) is true. join(rajakl,’r’,’a’,’j’,’a’) is false.
6. more(X,Y). It is true if X and Y have same number of letters and Ascii code of every letter of X is one more than that of corresponding letter in Y. more(sbn,ram) is true.
7. place(X,Y,Z). It is true if Y^{th} letter of X is Z. place(“Rajeev”,3,’j’) is true.
8. Define ft(X,Y,Z). It works in the same way as fronttoken. Define it using frontchar only.
9. location(X,Y,Z,W). It is true if Y^{th} letter in Z^{th} word is W. location(“ram is the best student”,2,3,’h’) is true. [Hint: use place]
10. f(X,Y,Z). It is true if X can be obtained by joining reverse(Y) and Z. f(ramkumar,mar,kumar) is true.
11. Using f define rev(X,Y). It is true if X is reverse of Y. rev(hari,irah) is true.
12. sorted(X). It is true if all letters of X are in increasing order of Ascii code. sorted(ajkmst) is true. sorted(anil) is false.
13. unsorted(X). It is true if above is false. [Hint: not(sorted(X))]
14. Define unsorted, without using sorted.
15. consecutive(X). It is true if Ascii code of every letter in X is 1 more than that of preceding letter. consecutive(pqrstuvw) is true. consecutive(abcdfg) is false.
16. short(X,Y). It is true if Y is shortform of X. short(“Ram Prasad Kumar Dey”,RPKD) is true. Short(“Gopal Gupta”,GG) is true. Short(“ram Singh Yadav”,rSY) is true.

17. shrt(X,Y). It is true if short form is defined in the following manner. shrt("Ram Prasad Kumar Dey",RPKDey) is true. Shrt("Gopal Gupta",GGupta) is true.
18. delete(X,Y). It is true if Y can be obtained by deleting last letter from X. delete(ram,ra) is true.
19. reverse(X,Y). It is true if X is reverse of Y. [Hint: use delete and last]
20. bigger(X,Y). It is true if X is lexicographically bigger than Y. bigger(ram,kapil) and bigger (ramesh,ram) are true.
21. even(X) and odd(X). They return true if length of symbol is even and odd respectively. Do not use any other predicate.
22. sum(X,Y). It is true if sum of all digits of X is Y. sum(ra45w23,14) is true.
23. hexa(X,Y). It is true if hexadecimal representation of X is Y (reverse). Here X is an integer and Y is a symbol. hexa(631,"182") is true because hexadecimal representation of 631 is 281. hexa(641,"B82") is also true.
24. Do above problem when Y is hexadecimal representation (without reverse).
25. minmore(X,Y). It is true if ascii code of Y is more than or equal to that of minimum letter of X. Here minmore("tbhj",'d') is true minmore("tbhj",'a') is false. Here X is a symbol and Y is a letter.
26. Define minless(X,Y) and min(X,Y). To define min, minmore can be used.
27. maxminmore(X,Y). It is true if Y is more than or equal to maxmin(X). Here X is a string of words and Y is a letter. Maxmin is defined as follows: Find minimum letter of each word. Find maximum of those minimums. Maxmin("def rbi cgtck") is 'i'. It is because minimum letters of these words are 'd', 'i', and 'c' respectively.
28. maxminless(X,Y). maxminless("def rbi cgctk",'a') is true.
29. maxmin(X,Y). It is true if maxmin of X is Y. [Hint: use min(X,Y) and maxminless]
30. f(X,Y). It is true if minimum letter of every word of X is less than or equal to Y. [Hint: use minless]
31. g(X,Y). It is true if minimum letter of some of X is Y. [Hint: Use min(X,Y)]
32. Define maxmin(X,Y) using f and g.

Compound Object

```
domains     complex=pair(real,real).polar=cm(real,real).abc=x(symbol,integer).
predicates  add(complex,complex,complex).divide(complex,real,complex).
            convert(complex,polar).f(abc).
clauses
  add(X,Y,Z):-X=pair(A,B),Y=pair(C,D),P=A+C,Q=B+D,Z=pair(P,Q).
  divide(pair(A,B),C,pair(D,E)):-D=A/C,E=B/C.
  convert(X,Y):-X=pair(A,B),P=sqrt(A*A+B*B),Q=arctan(B/A)*57.27,Y=cm(P,Q).
  f(X):-X=x(A,B),frontchar(A,T,_),char_int(T,B).
```

1. add(pair(4,5),pair(7,2),T) outputs pair(11,7). It is addition of two complex numbers.
2. divide(pair(4,5),6,T) outputs pair(0.67,0.83). Here a complex number is divided by a real number.
3. convert(pair(1,1.732),T) outputs cm(2,60^0). It converts a complex number in Cartesian form to polar form. The function arctan is $\tan^{-1}$. The multiplication by 57.27 is done so that radian can be converted into degree.
4. abc is a compound object. It has two components: a symbol and a number. f(x(bimu,98)) returns true. It is because ascii code of first letter of symbol is 98.

Define following functions (no other predicate should be used unless permitted**)**

1. Define vector=v(real,real,real). Using it define dotpr(vector,vector,real). Given two vectors their dot product is computed. dotpr(v(2,3,1),v(5,2,3),K) outputs K=19.
2. Define magnitude(vector,real). magnitude(v(4,3,12),T) outputs 13. [Hint: use dotproduct]
3. Define unit(vector,vector). Here unit vector in the direction of a given vector is computed. Unit(v(4,3,12),T) outputs v(0.31,0.23,0.92). [Hint: use magnitude]
4. Define cross(vector,vector,vector). Here cross product of vectors is computed. cross(v(2,3,1),v(5,2,3),K) outputs K=v(7,-1,-11). (ai+bj+ck)x(pi+qj+rk)=(br-cq)i+(cp-ar)j+(aq-bp)k.
5. Define point=p(real,real,real). Using it define midpoint(point,point,point). midpoint(p(2,3,6),p(7,3,4),K) outputs p(4.5,3,5).
6. Define direction(point,point,vector). Given two points a vector from first point to second point is returned. direction(p(2,3,6),p(1,9,3),T) returns v(-1,6,-3).
7. Define area(A,B,C,D). Here A, B and C are points and D is the area of the triangle ABC. [Method: find directions corresponding to AB and AC. It can be done by using above function. Now call cross product and magnitude.] Area of triangle with end points (3,4,5), (7,3,6) and (6,2,3) is 6.36.
8. Define point=p(real,real) and line=l(real,real,real). l(a,b,c) stands for the line whose equation is ax+by+c=0. Define function equation(point,point,line). equation(p(2,3),p(4,7),T) outputs l(2,-1,-1). The equation be outputted in many ways. Alternate outputs are l(4,-2,-2), l(1,-0.5,-0.5). Given two points the equation of the line passing through them is outputted. equation(p(5,12),p(3,6),T) outputs l(3,-1,-3).
9. Define intersection(line,line,point). intersection(l(2,3,-26),l(4,7,-54),T) outputs p(10,2). It is because the point of intersection of lines 2x+3y-26=0 and 4x+7y-54=0 is (10,2).
10. Define intersect(A,B,C,D,P). Here P is the point of intersection of line joining A,B and C,D. intersect(p(5,12),p(3,6),p(6,11),p(5,9),T) outputs p(2,3). [Find the equation of lines. Then find their point of intersection].
11. Define distance(A,B,C). Distance of point A from line B is calculated. distance(p(2,3),l(8,6,-12),K) outputs $(8*2+6*3-12)/(8^2+6^2)^{1/2} = 2.2$.
12. Define dist(A,B,C,W). Distance of point A from line joining the points B and C is calculated. dist(p(5,2),p(2,1),p(6,-2),K) outputs 2.6.[Hint: use equation and distance].
13. Define pt(A,B,C,P,Q). Here A is equation of a line. B is some point on the line. We want to find two points on the line whose distance from B is C. pt(l(3,4,-7),p(1,1),5,P,Q) outputs P=p(5,-2) Q=p(-3,4). pt(l(15,17,-66),p(1,3),4,P,Q) outputs P=p(4,0.35) Q=p(-2,5.65).
14. Define circle=c(point,real). Centre and radius the circle are given. Define cv(A,B,C,D). A circle D is given. We want to find numbers A, B and C such that equation of circle is $x^2+y^2+Ax+By+C=0$. cv(A,B,C,c(p(2,3),7)) outputs A=-4 B=-6 C=–36.
15. Define cw. It is opposite of the above cw(-4,-6,-36,T) outputs c(p(2,3),7).

16. Define chord. A circle and a line is given, we want to find chord length. Chord(c(p(1,2),5),l(5,12,10),T) outputs 8. Assume that line intersects with circle.
17. A circle (A) and a line x=B is given. We want to find P and Q such that (B,P) and (B,Q) are points on the circle. yc(c(p(7,2),5),10,P,Q) outputs 6 and -2. It is because circle $(x-7)^2+(y-2)^2=5^2$ intersects with the line x=10 at points (10,6) and (10,-2).
18. Define f(A,B,C). It returns true if the point C is on the line AB and is in between points A and B. [Hint: find the distance of the point C from the line AB. If it is zero then check whether it is between A and B. It can be checked by comparing the x-coordinates of points A,B and C. If $A_x< C_x< B_x$ or $B_x< C_x< A_x$ then the point C is in between A and B. (assume that line AB is not vertical)]. f(p(2,5),p(7,15),p(4,9)) is true and f(p(2,5),p(7,15),p(1,3)) is false.
19. Define dst(A,B,C). It calculates the distance between points A and B.
20. Define projection(A,B,C,D). A is a given line. B is some point on the line. D is length of projection of BC on A. [Method: Find distance(k) of C from the line. $D=(BC^2-k^2)^{1/2}$]
21. Define line segment as ls(point,point). Now define distanc(A,B,C). Distance of point B from line segment A is calculated. Let A=ls(P,Q) if length of projects of both BP and BQ are less then the length of the line segment then distance between the line and the point B is taken. Otherwise it is minimum of distance between points P and B, distance between points Q and B.
22. Define intersectin(A,B,C). C is the point of intersection of line segments A and B. intersectin(ls(p(5,12),p(3,6)),ls(p(6,11),p(5,9)),T) outputs “no solution”. [Hint: find out the point of intersection of lines joining these points. Now check the condition whether the point is on both the line segments.]
23. Define mul(X,Y,Z). Here Z is multiplication of complex numbers X and Y.
24. Define convert(X,Y). It converts a complex number in Cartesian form to polar form. The definition given in example is not correct. convert(pair(1,-1.732),T) and convert(pair(-1,1.732),T) output cm(2,-60^0). In the second case the output should be cm(2,120^0). [In place of -60^0 you may write 300^0 also]
25. Define cvert(X,Y). It converts a complex number in polar form to Cartesian form. cvert(cm(2,60),T) outputs pair(1,1.732). [Hint: use functions sin and con and convert degree into radian by suitable division]
26. Define power(X,Y,Z). It calculates $Z=X^Y$. Here X is a complex number and Y is positive integer. power(pair(0.5,0.866),5,T) outputs pair(0.5,-.867). [Hint: convert X in polar form]
27. Define expo(X,N,Y,Z). It outputs Y=$\frac{x^n}{n!}$ and Z= $1+x+\frac{x^2}{2!}+\frac{x^3}{3!}+ \ldots\ldots +\frac{x^n}{n!}$ When n is large the above become e^x. expo(pair(0,1.05),10,A,B) outputs B=pair(0.50,0.87).

Arithmetic

```
predicates
  f(integer,integer). max(integer,integer,integer).fact(integer,integer).
  sod(integer,integer,integer).replace(integer,integer,integer,integer). md(integer,integer).
clauses
  f(X,Y):-Y=7*X*X-6*X.
  max(X,Y,Z):-X<Y,Z=Y.
  max(X,Y,Z):-X>=Y,Z=X.
  fact(X,Y):-X=1,Y=1.
  fact(X,Y):-X>1,T=X-1,fact(T,W),Y=X*W.
  sod(X,Y,Z):-X=0,Y=0,Z=1.                    It can also be written as sod(0,0,1).
  sod(X,Y,Z):-X>0,T=X div 10,sod(T,M,G),Y=M+X mod 10,Z=G*(X mod 10).
  replace(X,Y,Z,W):-X<10,X=Y,W=Z.
  replace(X,Y,Z,W):-X<10,not(X=Y),W=X.
  replace(X,Y,Z,W):-X>9,X mod 10=Y,T=X div 10,replace(T,Y,Z,M),W=M*10+Z.
  replace(X,Y,Z,W):-X>9,not(X mod 10=Y),T=X div 10,replace(T,Y,Z,M),W=M*10+X mod 10.
  md(X,Y):-X<10,Y=X.
  md(X,Y):-X>=10,S=X div 10,md(S,L),T=X mod 10,T>=L,Y=T.
  md(X,Y):-X>=10,S=X div 10,md(S,L),X mod 10<L,Y=L.
```

1. f(x,y) has input x and output y. f(7,T) outputs 301.
2. max(x,y,z) outputs z as maximum of x and y. max(5,7,Z) outputs Z=7. [Question: What difference will it make when X>=Y is replaced by X>Y? Similarly replace X<Y by X<=Y (without changing X>=Y]
3. fact(x,y) outputs y as factorial of x. fact(5,G) will output G=120.
4. sod(x,y,z) outputs y as sum of digits of x and z as product of digits of x. sod(32415,R,K) will output R=15 and K=120.
5. replace: It replaces all occurrences of digit Y by Z in number X. replace(16174,1,2,T) will output 26274.[Question: Write replace function in three clauses]
6. md finds maximum digit of number x. [It can be written in two clauses by using function max]

Define following functions (no other predicate should be used unless permitted**)**

1. First digit. fd(24135,T) outputs T=2.
2. Second digit.
3. Last even digit. lasteven(12363,T) outputs T=6.
4. Second last even digit. secondlasteven(12363,T) outputs T=2. [additional predicate]
5. First even digit. If the number does not have even digit then output is -1. firsteven(14258,Y) outputs Y=4. firsteven(1351,T) outputs T= –1.
6. Sum of even digits.
7. Greatest common divisor. Gcd(42,54,T) outputs 6.
8. Sum of product of consecutive digits. sum(3246,T) outputs 38 (since 3*2+2*4+4*6=38)
9. Power. Power(3,5,K) outputs K=243.
10. Maximum digit less than Y. mxm(7346,5,T) outputs T=4. mxm(7897,6,T) outputs -1.
11. Remove all even digits. rmv(25346,T) outputs 53. rmv(5237,T) outputs 537. Rmv(648,T) outputs 0.
12. Define cosinverse. Cosinverse(0.5,T) outputs 60^0. cosinverse(-0.5,T) outputs 120^0. cosinverse(0,T) outputs 90^0. [Hint: use arctan. P=arctan(1.732) gives p=1.05(radian)]. Here predicate is coninverse(real,real).
13. Sum(X,Y,Z,W,T). W is sum of those digits, which are between Y and Z. T is sum of those digits, which are not between Y and Z. sum(25497,3,6,P,Q) outputs P=5+4=6 and Q=2+9+7=18. Assume Y<Z.
14. Sum of factors between Y and Z. sf(24,3,9,T) outputs 21(Hint:3+4+6+8)
15. Smallest factor. sf(35,T) outputs 5. [Hint: Define a predicate (sfg) which finds smallest factor of X greater than Y. sfg(24,7,T) outputs T=8. sfg(55,7,K) outputs K=11]

16. Whether prime? prime(X,Y) outputs Y=0 if X is prime Y=1 otherwise. [Hint: Use smallest factor or sum of factors]
17. Whether prime? prime(X) outputs true if X is prime and false otherwise.[Additional Predicate]
18. Given X and Y find Z such that sum of numbers between Y and Z is X (assume $Y \leq Z$). sm(25,3,Z) outputs Z=7. sum(17,8,H) outputs H=9. [Observe sum(18,8,H)]
19. Lower bound of square root. Srt(441,T) outputs 21. Srt(440,T) outputs 20. [Hint: Sqaure root of a number(X) and (X-1) are differing atmost by 1.]
20. Square root. Sr(441,T) outputs 21. sr(442,T) outputs 'no solution'. [Additional predicate]
21. Greatest integer (less than or equal to). Gr(5.72,T) outputs T=5. Define predicate gr(real,integer). Assume that given number is more than 1.
22. First digit (of a number between 0 and 1). First(0.342,T) outputs 3. [Hint: use greatest integer).
23. facti(x,y,z). It outputs z= $\sum_{i=x}^{i=y} i!$. facti(3,5,T) outputs T=150. Assume $x \leq y$. [Additional Predicate]
24. g(x,y,z,w). It outputs z=x! and w= $\sum_{i=x}^{i=y} i!$. g(3,5,T,K) outputs T=6 and K=150.
25. h(x,n,y,z). It outputs $y=x^n$ and $z=1+x+x^2+x^3+\ldots+x^n$. h(2,4,A,B) outputs A=16, B=31.
26. expo(x,n,y,z). It outputs y= $\frac{x^n}{n!}$ and z= $1+x+\frac{x^2}{2!}+\frac{x^3}{3!}+\ \ldots\ldots+\frac{x^n}{n!}$ When n is large the above become e^x. It can be verified by giving command T=exp(x). expo(2,100,Y,Z) outputs $Z=e^2=7.39$

Write suitable predicates to find sum of following series

27. $x+\frac{x^2}{2}+\frac{x^3}{3}+\ \ldots\ldots+\frac{x^n}{n}$ It is –log(1– x). Input output
28. $1-\frac{x^2}{2!}+\frac{x^4}{4!}-\frac{x^6}{6!}+\ldots\ldots\ldots$ It is cos(x). Input 1.05 output 0.5
29. $x-\frac{x^3}{3!}+\frac{x^5}{5!}-\frac{x^7}{7!}+\ldots\ldots\ldots$ It is sin(x). Input 1.05 output 0.866
30. form(integer,integer,symbol). form(66,70,T) outputs "BCDEF". A word is formed, which has those letters whose ascii codes are between given numbers.
31. max(symbol,char). It finds maximum letter in a word. max(ramesh,T) outputs T='s'.

List

```
domains li=integer*.
predicates second(li,integer). exchange(li,li).join(li,integer).
           form(integer,integer,li).sp(li,integer,integer).
clauses
  second(X,Y):-X=[_|B],B=[Y|_].
  exchange(X,Y):-X=[A|B],B=[C|D],Z=[A|D],Y=[C|Z].
  join(X,Y):-X=[],Y=0.
  join(X,Y):-X=[A|B],join(B,T),Y=10*T+A.
  form(X,Y,Z):-X=Y,Z=[X].
  form(X,Y,Z):-Y>X,T=X+1,form(T,Y,P),Z=[X|P].
  sp(X,Y,Z):-X=[A|[]],Y=A,Z=A.
  sp(X,Y,Z):-X=[A|B],not(B=[]),sp(B,P,Q),Y=A+P,Z=A*Q.
```

These can be written as follows also

```
second([_|B],Y):-B=[Y|_].
exchange([A|B],[C|Z]):-B=[C|D],Z=[A|D].
join([],0).
join([A|B],Y):-join(B,T),Y=10*T+A.
form(X,X,[X]).
form(X,Y,Z):-Y>X,T=X+1,form(T,Y,P),Z=[X|P].
sp([A],A,A).
sp(X,Y,Z):-X=[A|B],sp(B,P,Q),Y=A+P,Z=A*Q.
```

It can also be written as sp([],0,1)

[2,7,9,3]=[A|B] will make A=2 and B=[7,9,3]. Similarly if P=4 and Q=[3,2,7,9] then R=[P|Q] will make R=[4,3,2,7,9]. A=[P|Q] returns false if A is null.

1. second([5,7,2,9],K) outputs K=7.
2. exchange([5,7,2,9],K) outputs K=[7,5,2,9]. It exchanges first two elements.
3. join([5,7,2,9],K) outputs K=9275.
4. form(5,9,T) outputs T=[5,6,7,8,9].
5. sp([5,2,6,3],P,Q) outputs P=16 and Q=180. Here sum and product of numbers is calculated. not(B=[]) is optional.

Define following functions (no other predicate should be used unless permitted**)**

1. Delete second element. del([5,4,2,3],T) outputs T=[5,2,3].
2. Replace first element. rep([5,4,2,3],11,T) outputs [11,4,2,3].
3. Increment first element. inc([5,4,2,3],T) outputs [6,4,2,3].
4. Insert second element. ins([5,4,2,3],12,T) outputs [5,12,4,2,3].
5. k^{th} element. kth([5,4,2,3,6,12,9],4,T) outputs T=3. kth([12,34,56],2,T) outputs T=34.
6. Last element. last([5,4,2,3,7,8,12,9],T) outputs T=9.
7. Increment all elements. incr([5,4,2,3],T) outputs T=[6,5,3,4].
8. Sum of even numbers. sm([12,5,7,18,5],T) outputs T=12+18=30.
9. Decompose. Decompose(5234,T) outputs T=[4,3,2,5].
10. Remove. All even numbers in the list are removed.
11. Member. Member(X,Y) outputs true if Y is present in list X at least once.
12. Mem. Mem(X,Y) outputs true if Y is present in list X exactly once. [Hint: Use not]
13. Union. Union([5,7,2],[6,7,2,1],T) outputs [5,7,2,6,7,2,1]. [2,7,5,6,7,2,1] is permitted.
14. Merge. Merge([5,7,12,14],[2,3,8,9,10],T) outputs [2,3,5,7,8,9,10,12,14]. Assume input lists are sorted. Output is sorted list of numbers, which has element of both list. All numbers are distinct.
15. In between every two elements insert sum of neighboring elements. insert([5,2,8,6],A) outputs A=[5,7,2,10,8,14,6].
16. Sum of product. F([[5,2],[3,9,2,4],[2]],A) will output A=5*2+3*9*2*4+2=228. [Hint: define a domain lii=li*]
17. form. form([1,2,14,768,2],T) outputs T=[[1],[2],[14],[768],[2]].
18. Partial decomposition. G([273,14,1311],A) outputs A=[[3,7,2],[4,1],[1,1,3,1]].

19. Break. Break([527,39,1523,4,17],A) outputs [5,2,7,3,9,1,5,2,3,4,1,7]. Following output will be easy to obtain. [7,2,5,9,3,3,2,5,1,4,7,1].
20. sum of complex numbers. sum([c(1,2),c(3,4),c(12,23)],T) outputs T=c(16,29). [Hint: define complex=c(integer,integer) and lc=complex*]
21. Define abc=p(li,integer). m(p([1,7,9,6],4),T) outputs T=(1+4)(7+4)(9+4)(6+4)=7150.
22. Last element and remaining list. Last([5,2,3,7],A,B) outputs A=7 and B=[5,2,3].
23. Last even number and total number of even elements. even([5,2,4,7,6,1,9],A,B) outputs A=6 and B=3. even([5,7,9],A,B) outputs A=-1 B=0.
24. First even number and total number of even elements.
25. Last and second last. Last([5,2,3,7,1],A,B) outputs A=7 and B=1.
26. n^{th} element and total number of elements. count([8,9,1,7,6,3],2,A,B) outputs A=9 and B=6. It is because second element is 9 and total number of elements are 6. count([8,9,1,7,6,3],9,A,B) outputs A=-1 B=6.
27. n^{th} element from last and total number of elements. cnt([8,9,6,3],2,A,B) outputs A=6 and B=4. It is because second element from last is 6 and there are 4 elements.
28. Maximum and second maximum element. mx([7,2,3,8,3,11,6],A,B) outputs 11 and 8.
29. Maximum element and its frequency. Max([2,5,8,3,2,8,6,8],A,B) outputs A=8 and B=3. It is because maximum element is 8 and it occurs 3 times.
30. f(X,Y). X is input list. Y is the list of 6 elements. The first element is sum of numbers, the second element is product, the third element is sum of squares, the fourth element is sum of product of consecutive numbers, the fifth element is the first number, the sixth element is the last number. f([3,4,2,6],A,B,C,D,E,F) outputs A=15 B=144 C=65 D=32 E=3 F=6.
31. Reverse. Reverse the elements of given list. Additional predicate may be used.
32. Put proper place. A given number is put in a sorted list of numbers at proper place. ppp([2,4,5,12,19],8,A) outputs [2,4,5,8,12,19]. ppp([2,4,8],9,A) outputs [2,4,8,9].
33. Sort. Additional predicate may be used. [Hint: use put proper place]
34. Uselessmore(X,Y,Z). We want to establish the truth of the sentence “Y is more than or equal to the smallest number in list X”. Z is the list of unobserved numbers. Uselessmore([8,9,3,7,2,4],5,T) outputs T=[7,2,4]. Uselessmore([8,9,6,12],5,T) outputs T=[]. It is because as soon as we observe 3, the sentence is certain to be false.
35. Define uselesslss and useless. In useless(X,Y,Z) Z is the list of unobserved words to establish the truth of the sentence “Y is smallest element of list X”. useless([8,9,3,7,2,4],5,T) outputs T=[7,2,4].
36. We want to find minimum element of a list if it is more than a given element. If minimum element is less than or equal to the given element then we are not interested in that. In that case we are interested in the list of unobserved elements. f([6,2,4,7,1],5,A,B) outputs A=some number B=[4,7,1]. f([6,4,7,8,],2,A,B) outputs A=4 and B=[].
37. We are given a list of lists. We want to find the maxmin (minimum element of every list and maximum of those) and the list of unobserved elements. maxmin([[7,2,5,6],[4,1,5],[7,4,6,3],[4,6,2,1]],A,B) outputs A=4 and B=[5,6,2,1]. It is because minimum elements of these lists are 2, 1, 4 and 1 respectively. Their maximum is 4. To find it in the second list [5] is unobserved. In the last list [6,2,1] is unobserved.
38. f(X,Y,Z). Z is the list of unobserved elements to establish truth of the sentence “maxmin of X is more than or equal to Y”. f([[7,2,5,6],[7,4,6],[4,2],[1,2],[5,7,9]],3,T) outputs A=[[5,6],[4,2],[1,2],[5,7,9]]. The minimum element of 3rd list is found to be 4. Hence it has become certain that maxmin is more than 3. Hence remaining portion is not observed. In the first list [5,6] is not observed because it has become certain that minimum element is less than 3.
39. g(X,Y,Z). Z is the list of unobserved elements to establish truth of the sentence “maxmin of X is less than or equal to Y”. g([[7,2,5],[2,4,1],[5,1,8,9]],3,A) outputs A=[[5],[4,1],[8,9]].
40. Define domain abc=pair(char,integer) and lt=abc*. Now define function f(lt). It is true if in every pair of input list the ascii code of letter is the corresponding number. f([p(‘a’,97),p(‘c’99),p(‘D’,68)]) is true. f([p(‘a’,97),p(‘c’,100)]) is false.

41. g(lt,lt). It outputs those pairs whose ascii code of letters is same as corresponding number. f([p(‘a’,97),p(‘c’,100),p(‘D’,68),p(‘+’,45),p(‘-‘,45),p(‘Z’,90)],A) outputs A= [p(‘a’,97),p(‘D’,68),p(‘+’,45),p(‘Z’,90)]
42. g(X,Y,Z). Given list of objects, it joins all symbols and finds the sum of numbers. g([x(ram,12),x(hari,23),x(kapil,56)],A,B) returns A=”ram hari kapil” and B=91.

Read/Write

```
predicates f(integer) .g( ). h(integer). k1( ). k2( ). ddbl( ). sum(integer,real). sy(symbol)
clauses
 f(P):-write("Give numbers"),readint(X),readint(Y),Z=X+Y,P=X*Y,write("Sum=",Z).
 g( ):-write("Give a number"),readint(X),X mod 2=0.
 h(X):-X mod 2=0,write("yes").
 h(X):-X mod 2=1,write("no").
 k1( ):-readint(X),X mod 2=0,write("yes").
 k1( ):-readint(X),X mod 2=1,write("no").
 k2( ):-readint(X),h(X).
 ddbl( ):-write("Give a number"),readreal(X),X>=0,Y=X*2,write(Y),ddbl().
 sum(N,P):-N>0,write("give a number"),readreal(X),T=N-1,sum(T,Q),P=Q+X.
 sum(N,P):-N=0,P=0.
 sy(R):-write("Give letter"),readchar(T),write("Givesymbol"),readln(G),frontchar(R,T,G).
```

1. f(P) reads two numbers and prints their sum and returns their product.
2. Predicate 'g' reads a number. It returns true if it is even. It returns false otherwise.
3. h(X) outputs "yes" if X is a even number. It X is odd then it outputs "no".
4. The predicate k1 is supposed to read a number and write "yes" if the number is even and "no" if it is odd. But in case of an odd number the second number is read. If it is odd then "no" is outputted. If second number is even then nothing is outputted.
5. The predicate k2 reads a number and writes "yes" if it is even and "no" otherwise.
6. The predicate ddbl reads numbers till a negative number is given. It outputs double of every number. [Question: What should be done so that true is returned in place of false]
7. sum(N,P) reads n numbers and returns their sum.
8. sy(R). It reads a symbol and a character and returns a symbol after joining. Readln is used to read a symbol.

Define following functions (Least number of additional predicates should be used**)**

1. Define sum(A). It reads two numbers and returns their sum.
2. Define maximum(A,B). It prints bigger of A and B. maximum(5,7) prints 7.
3. Define max( ). It reads two numbers and prints their maximum.
4. Define mx(A). It reads two numbers and returns their maximum. [Hint: Define a predicate mxm(X,Y,Z). mxm(5,7,Z) returns Z=7.]
5. Define between(X,Y). It prints all numbers between X and Y in separate lines.
6. Define predicate which reads a number and outputs it table. Input 7 output 7 14 ..70.
7. Define predicate which reads two numbers and outputs all numbers between them.
8. Define p. p(65,6) outputs ABCDEF. [Hint: char_int(T,65) gives T='A']
9. Define pic(int,int). pic(65,4) prints ABCDABCABA. pic(68,2) prints DED.
10. Define pc(symbol). pc(hari) prints hariaririi. pc(kapil) prints kapilapilpilill.
11. Define dtbl( ). It reads numbers till a negative number is given. It prints double of every even number and triple of every odd number.
12. Define dbl(N). It reads n numbers. It outputs double of every number after reading it. [Hint: It can be done in one clause, but it is advised not to do so. Why?]
13. Define dtl(N). It reads n numbers. It outputs double of every even number and triple of every odd number after reading it.
14. Define dts(P). It reads number till sum of all numbers exceeds P. It outputs double of all numbers.
15. Define dtt(P,Q). It reads numbers till sum of all numbers exceeds P. It returns their sum. e.g. if dtt(20,T) is executed and numbers are 6,3,8,7 then output is 24.
16. Define rd( ). It reads numbers till two consecutive even numbers are given. The program prints double of every number.
17. Define re( ). It reads numbers till a negative number or a even number is given. The program outputs double of every number.
18. Define smm(P). It reads numbers till a negative number is given. The program returns their sum. [Hint: The need of true return in example ddbl]

19. Define sp(N,P,Q). It reads n numbers and returns the first number and sum of product of consecutive numbers. If input numbers are 6,12,5,2 then sp(4,P,Q) will return P=6 and Q=6*12+12*5+5*2=142.
20. Define ddb( ). It reads numbers till a negative number is given. The predicate outputs double of every non-negative number. If the last number is -1 then “ram” is outputted otherwise “hari” is outputted.
21. Define f(X,Y). If X>45 then Y=X+5 otherwise if (X mod 3=1) then Y=X+2 otherwise Y=X+1.
22. Define gm( ). It reads a number till a number more than 50 is given. After reading every number (X) the program prints Y such that f(X,Y).
23. Define gam( ). It is similar to above. The numbers are read till X or Y exceeds 50. If X is more than 50 then program prints “you won”. If Y is more than 50 then the program prints “Computer won”.
24. Define g(X,Y). It reads a number (P). If P-X is 1 or 2 or 5 then Y=P otherwise program reads another number. It continues till user gives a number which is 1 or 2 or 5 more than X.
25. Define game(X). The computer prints a number (Y) such that f(X,Y). Now user is supposed to give a number, which is 1 or 2 or 5 more than Y. This process continues till the number given by user or printed by computer exceeds 50. It can be done by modifying gam( ), using g(X,Y).
26. Define pt to print all elements of a list. pt([2,3,8,5]) outputs 2 3 8 5.
27. Define ps to print all elements of a list separated by comma(,). In above case 2,3,8,5
28. Define pu to print a list. pu([2,3,8,5]) prints {2,3,8,5}. pu([8,7]) prints {8,7}.
29. Define pc to print a complex number. pc(c(2,3)) prints 2+3i.
30. Define sm(N,T). It reads n vectors (along the direction of principle axis). It returns their sum. sm(6,T) will return T=v(12,11,-4) if input is 8j 3j 7i -4k 8i -3i.
31. Define sl(n,X). It reads n numbers. After reading first number it outputs X. After reading any other number it outputs the second last number read. sl(5,11) will behave as follows: 19 **11** 17 **19** 21 **17** 8 **21** 18 **8**. The output is shown in bold letters.
32. Define tl(X). It reads numbers till a negative number is given. As soon as a negative number is given X is printed. This problem can be done without any other predicate.
33. Read a number till a negative number is given. Output the double of the negative number.
34. Read numbers till a even number is given. Output the sum of the input numbers.
35. Read numbers till a even number or a negative number is given. Output the sum of the read numbers.
36. Read numbers till two consecutive even number are given. Output the sum of all read numbers.
37. Read numbers till a negative number is given. After reading negative number one more number is read. The double of the last number is printed.

File

```
domains file=fp.
predicates f . g . h . l . t() . p(symbol) . aux .
clauses
f:-openwrite(fp,"abc"),writedevice(fp),write("Hari"),writedevice(screen),write("kapil").
g:-openread(fp,"pqr"),readdevice(fp),readchar(X),write(X),
   filepos(fp,3,0),readchar(Y),write(Y),filepos(fp,4,1),readchar(Z),write(Z),
   filepos(fp,-3,2),readchar(W),write(W).
h:-openmodify(fp,"pqr"),readdevice(fp),readchar(X),writedevice(fp),
    filepos(fp,4,0),write(X),writedevice(screen).
l:-openread(fp,"pqr"),readdevice(fp),filepos(fp,-1,2),filepos(fp,X,0),Y=X+1,write(Y).
t():-openread(fp,"abc"),readdevice(fp),readchar(P),closefile(fp),
        openmodify(fp,"pqr"),writedevice(fp),write(P),writedevice(screen).
p(X):-openread(fp,X),readdevice(fp),aux.
aux:-eof(fp).
aux:-not(eof(fp)),readchar(X),write(X),aux.
```

1. The predicate f writes "hari" in a file whose name is "abc". On the screen "kapil" is written. If writedevice(screen) is removed then "kapil" is also written in the file.
2. Let contents of the file "pqr" be "abcd…z". The predicate g( ) outputs adix. 'filepos' controls the file pointer. 3rd argument is mode. 0 means from beginning. 1 means from current position and 2 means from end. If contents of a file are "abcdef..z" and file pointer is at 'e' then filepos(fp,-2,1) will take the file pointer at 'c'. In this file 'a' is 0th letter, b is 1st letter and so on. After reading a letter file pointer is advanced automatically.
3. The predicate 'h' puts 0th letter of file "pqr" at 4th position. If initial contents are "abcdef.." then after the execution contents will be "abcdafgh..".
4. The predicate 'l' outputs the length of file.
5. t() reads a letter from file "abc" and puts it in file "pqr".
6. p(X) displays the contents of file X.

Define following functions (Least number of additional predicates should be used**)**

1. Output last 3 letters of a file. [Hint: use filepos(fp,-1,2) and filepos(fp,-2,1)]
2. Read a letter from a file and output its ascii code. [Hint: char_int('E',T) gives T=69]
3. Read first two letters from a file. Assume that they are digits. Output their sum.
4. Read a letter from a file. Output "yes" if its ascii code is even, "no" otherwise.
5. Increment 0st letter of a file. Let a file initially is "ram" then finally it is "sam".
6. Exchange first two letters of a file. Let initially a file is "ram" then finally is "arm".
7. Read 0st letter of a file. Assume that it is a digit (x). Output xth letter of the file. Let contents of a file are "5qwertyu" then output 't'.
8. Define between(X,Y,Z). All numbers between X and Y are written in a file whose name is Z.
9. Write program, which prints first letter from a file "pqr". Now a number is read from screen. The double of the number is printed. [Hint: use readdevice(keyboard)].
10. In a file named "hari" write "1 2 3 ….. 1000".
11. In a file named "kapil" write "1121231234…..123456789".
12. Define first(n). It outputs first n letters of a file.
13. Define last(n). It outputs last n letters of a file.
14. Write program to find the letter previous to first 'a' in a file.
15. Write program to find number of 'a''s in a file.
16. Write program to find the location of last 'a'.
17. Write program to find smallest letter of a file.
18. Write program to replace every 'a' by 'b' in a file.
19. Write program, which replaces every small letter by corresponding capital letter.
20. Write program to print the contents of a file in opposite order.

21. Write program to exchange consecutive letters of a file. Assume that size of the file is even. Let initial file contents be “qwrtsdfe” then after the execution of this the file contents will be “wqtrdsef”.
22. We are given two files, whose contents are in sorted order. Output all contents in sorted order. Let contents of file are “aghipq” and “bcdjxyz” then output is “abcdghijpqxyz”.
23. Write program to delete the first letter of a file.
24. Define sm(n). It reads n letters from a file. Assume they are digits output their sum.
25. Define predicate. It reads letters from a file till it gets a non digit. It outputs the sum of digits. Let contents of a file be “573k23” then output will be 15.

Simulation

```
predicates   f(integer).coin(integer).die(integer).sum(integer,real).d2(integer).
    opp(integer,integer,integer).dp(integer).avg(integer,real).dist(real).
    circle(real,real).opcr(real,real,real,real).sm(integer,integer).
clauses
  f(N):-N>0,random(Y),write(Y),nl,M=N-1,f(M).
  coin(T):-random(P),T=round(P).
  die(T):-random(P),T=round(P*6+0.5).
  dp(X):-die(P),die(Q),X=P*Q.
  sum(0,0).
  sum(N,S):-N>0,dp(Y),M=N-1,sum(M,T),S=T+Y.
  avg(N,A):-sum(N,K),A=K/N.
  d2(T):-die(P),die(Q),M=P+Q,opp(M,10,T).
  opp(A,B,C):-A<B,C=1.
  opp(A,B,C):-A>=B,C=0.
  dist(X):-random(A),random(B),X=sqrt(A*A+B*B).
  circle(X,Y):-random(P),random(Q),A=2*P-1,B=2*Q-1,opcr(A,B,X,Y).
  opcr(X,Y,X,Y):-sqrt(X*X+Y*Y)<1.
  opcr(X,Y,A,B):-sqrt(X*X+Y*Y)>=1,circle(A,B).
  sm(T,Y):-T<0,Y=0.
  sm(T,Y):-T>=0,die(P),R=T-P,sm(R,N),Y=N+1.
```

1. random(X) returns a (real) random number between 0 and 1. The call of f(100) will print 100 random numbers.
2. We want to define random outcome of a coin. It is 0 or 1. Let 1 be head and 0 be tail. When we call coin(X) then a random number 0 or 1 is generated. If in the function 'f' the random(Y) is replaced by coin(Y) then execution of f(100) will print 100 random numbers. Each random number is either 0 or 1.
3. The outcome of a die is a (integer) random number between 1 and 6.
4. sum(N,S) and avg(N,A) find the sum and average of N random numbers respectively.
5. Two dies are rolled. dp(X) returns the product of the outcomes. It is a random number. We want to know the expected value of the product. The method of simulation is to roll a pair of dies a large (say 100) number of times and find the average product. Hence execute avg(100,X). [ans:12.35]
6. Two dies are rolled. We want to know the probability that sum of the outcomes is less than 10. d2(X) returns a random number 0(failure) or 1 (success). A pair of dies is rolled. If sum is less than 10 then it is considered as success. In the function sum(N,S) replace dp(Y) by d2(Y). Now execute avg(100,X).[ans:0.84]
7. A point is generated inside a square of sides x=0, y=0, x=1, y=1. dist(X) returns its distance from (0,0). Using it we can solve the following problem: A point is randomly taken inside a square (side 1). What is the expected distance of the point from the centre? In the function sum(X,S) replace dp(Y) by dist(Y). [Answer: 0.76]
8. circle(X,Y) generates a random point inside the circle $x^2+y^2=1$. Following method is used. A point is generated inside a square of sides x=-1, y=-1, x=1, y=1. If it inside the circle then it is returned. Otherwise another point is generated. This is repeated till a point inside the circle is found.
9. A die is rolled till the sum of the outcomes exceeds T. sm(T,Y) returns the number of trials. The expected number of trials can be verified as 6.49 for T=20

Define following functions (Least number of additional predicates should be used)

1. Define square(X,Y). It returns two random number X and Y. Both are between 0 and 1. It is a random point inside a unit square. x=0, x=1, y=0, y=1.
2. Two points are randomly taken inside a square (side 1). What is expected distance between them [answer 0.53]? [Hint: Define dist(X). Put it in predicate sum.]

3. Two points are randomly taken inside a square. What is the probability that distance between then is more than half of the side of the square? [ans 0.51]
4. Two points are randomly taken inside a circle of radius 1. What is expected distance between them? [ans:0.9]
5. Two points are randomly generated inside the circle $x^2+y^2=5^2$ and $(x-10)^2+(y-8)^2=1$. What is expected distance between them? [ans: 13]
6. A coin and a die are rolled. What is the probability that coin shows 'H' and die shows 4. [ans: 0.83]
7. Define bc(X,Y). (X,Y) is a random point on the circumference of a circle $x^2+y^2=1$. [Hint: Generate a random number (A) between 0 and 6.28 (2π). X=cos(A) Y=sin(A)]
8. Two points are taken on the circumference of a circle $x^2+y^2=1$. What is expected chord length? [ans:1.3]
9. A point is generated on circumference of a circle. What is its expected distance from the line y=0.5?
10. A point is randomly generated inside the circle $x^2+y^2=1$. What is the probability that its distance from centre is less than 0.5? [ans: 0.25]
11. A point is randomly generated inside the circle $x^2+y^2=1$. What is the probability that the point is above the line Y=0.5? [ans:0.2]
12. Define bs(X,Y). (X,Y) is a random point on the boundary of the square (x=0 x=1 y=0 y=1). [Hint: generate a random number (P). If ($0 \le P < 0.25$) then X=0 Y=4P. If ($0.25 \le P < 0.50$) then X=4(0.5-P) Y=0. If ($0.50 \le P < 0.75$) then X=1 Y=4(P-0.5). If ($0.75 \le P < 1$) then X=4(1-P) Y=1].
13. Two points are randomly generated on the boundary of a square. What is expected distance between them? [ans: 0.73]
14. What is the probability that expected distance is more than 1? [ans:0.23]
15. Define bt(X,Y). It generates a random point on the boundary of the triangle, whose end points are (0,0), (3,0) and (0,4). [Hint: Generate a random number (P) between 0 and (3+4+5)=12. If ($0 \le P < 5$) then X=3-0.6P Y=0.8P. If ($5 \le P < 9$) then X=0 Y=(9-P). If ($9 \le P < 12$) then X=(P-9) Y=0.]
16. A point is randomly generated on the boundary of above triangle. Find its expected distance form origin.
17. Define expt(P,Q,T). A point is generated inside the square x=0 x=1 y=0 y=1. We want to find its expected distance from point (P,Q).
18. A coin is tosses N times. What are expected number of heads? [ans:]
    [Method: function cn(N,X). It returns a random number X between 0 and N]
19. A coin is tossed 10 times. What is the probability we get at least 4 heads? [ans 0.70]
20. A coin is tossed two times. What is the probability the both times it is head? [ans:.25]
    [Method: pt(Z):-coin(X),coin(Y),pm(X,Y,Z). pm(X,Y,Z) returns Z=1 iff X=Y=1.]
21. A biased coin (probability of head 0.4) is tossed two times. What is the probability that both times it is head? [ans: 0.16]
22. A die is rolled till the sum of outcomes exceeds 20. What is the probability that we shall get success in less than 8 trials? [ans: 0.804]
23. Define predicate tm. It prints random number (using random(X)) till a random number bigger than 0.6 is generated.
24. A random number (between 0 and 1) is generated till we get more than 0.6. What is expected number of generated numbers? [ans: 1/(1.-0.6)=2.5]
25. Define predicate tf(). It generates a random number (using random(X)) and outputs "yes" if it is more than 0.5 and "no" otherwise.
26. Define d3(X,Y). It returns X and Y, which are random outcomes of a die such that X+Y>8. Using it find probability for following problem: Two dies are rolled. Their sum is found to be more than 8. What is expected value of the product? [ans:24.5] [Method: define pd3(Z):-d3(X,Y),Z=X*Y In function sum(N,S) replace dp(Y) by pd3(Y)]
27. In above case what is the probability that product is more than 22? [ans:0.62]

28. A coin is tossed till we get 2 consecutive heads. What is expected number of tosses? [ans:6] In this case, what is the probability that we shall get success in less than 5 tosses? [ans: 19/32]

29. A point is randomly generated inside the circle $x^2+y^2=1$. What is the probability that its distance from centre is less than 0.5? [ans: 0.25]

30. Define crcle(X,Y). It generates two random number R and A. R is between 0 and 1 and A is between 0 and 6.28(2π). X=Rcos(A) Y=Rsin(A). Using it find probability for following: "A point is randomly generated inside the circle $x^2+y^2=1$. What is the probability that its distance from centre is less than 0.5?" We shall find answer as 0.5. Actual answer is 0.25. It shows that crcle(X,Y) is a wrong method to generate a random point inside the circle.

Multi valued functions

```
f(X,Y):-X<15,Y=X*X.
f(X,Y):-X>2,X mod 3=0,Y=X+10.
even(X,Y):-X mod 2=0,Y=X mod 10.
even(X,Y):-X>=10,T=X div 10,even(T,Y).
insert(X,Y,Z):-Z=[Y|X].
insert(X,Y,Z):-X=[A|B],insert(B,Y,T),Z=[A|T].
```

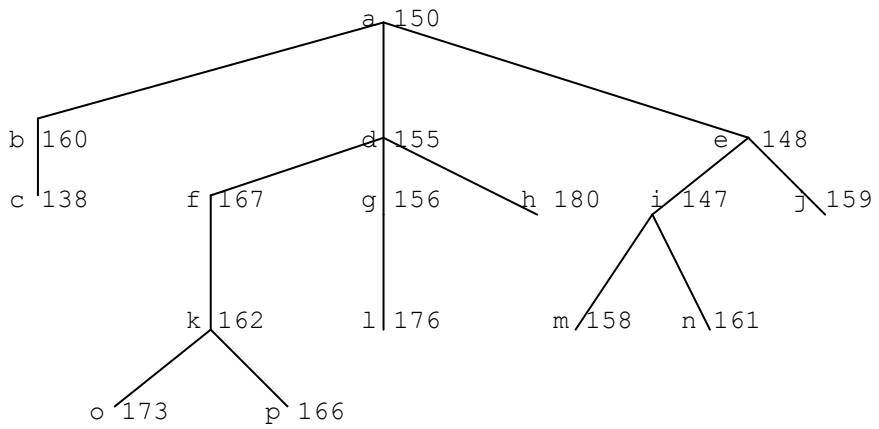
1. f(9,T) gives two solutions. f(20,T) gives no solution.
2. 'even' gives all even digits of a number,
3. In a list X the given element Y is inserted at some place. Insert([3,9,2],4,J) gives 4 solutions. These are [4,3,9,2], [3,4,9,2], [3,9,4,2], [3,9,2,4]. [Hint: X=[A|B] assumes that X is not null. A condition not(X=[]) is not needed.]

Define following functions (no other predicate should be used unless permitted**)**

1. Location (from last) of even digits. location(31476,T) outputs T=1 and T=3.
2. Multiply all numbers in a given range by some number. ml(3,7,10,Z) outputs 30,40,50,60,70.
3. Delete some even digit. delsome(32478,T) outputs 3247, 3278 and 3478.
4. Delete any even digit. delany(32478,T) outputs 8 solutions. Few of these are 37, 3247, 32478. Any even digit may or may not be deleted.
5. Insert a given digit after some even digit. ins(2478,1,T) outputs 21478, 24178 and 24781.
6. Divide some even digit by 2. divi(32478,T) outputs 32474, 32278, 31478.
7. All factors of X which are less than Y. factor(24,8,T) outputs 6, 4, 3, and 2.
8. All prime numbers in a given range. pm(6,18,X) outputs 7, 11, 13, 17. (additional predicate)
9. Even numbers and their location in a given list. f([4,3,7,6,12,19],X,Y) outputs X=4, Y=1 and X=6, Y=3 and X=12, Y=4.
10. Delete some element from the list. dellist([4,2,6],T) outputs [4,2], [4,6] and [2,6].
11. All subsets. subset([4,2,6,7],T) outputs 16 solutions. Few of them are [4,2], [2,6,7], [].
12. All proper subsets. psubset([4,2,6,7],T) outputs 15 solutions. [Hint: [4,2,6,7] is not outputted].
13. All non null subsets. subset([4,2,6,7],T) outputs 15 solutions. [Hint: [] is not outputted]
14. Given a list and a number find product of all elements of the list by the number. f([6,2,4,3],8,A) outputs 4 solutions. These are 48, 16, 32, 24.
15. Given two lists take an element from both lists and output their product. cp([5,2,7],[3,9,4,2],T) outputs 12 solutions. Few of them are 5*9=45, 7*3=21.
16. Given two sorted lists (one is ascending and another in descending order) take an element from both lists and output those pair of numbers, whose sum is 20.
    pair([2,5,9,11,13,14,19],[15,14,12,7,4,1],S,T) outputs S=5,T=15 and S=13,T=7 and S=19,T=1.
17. Do above problem, when lists may not be sorted.
18. Permutation. permute([5,2,7,9],X) outputs 24 solutions. Few of them are [7,9,5,2], [2,5,7,9].
19. Combination. combination([5,2,7,8,6,9],4,X) outputs ${}^{6}C_{4}$ solutions. Few are [5,2,6,9], [2,7,8,9].
20. Define suitable predicate to find all roots of a complex number. root(c(-8,13.84),4,T) will output 4 solutions: c(1.732,1), c(-1,1.732), c(-1.732,-1), c(1,-1.732).

Natural Language Processing

```
Predicates fa(symbol,symbol).h(symbol,integer).good(symbol).
           having_no_son(symbol).having_two_sons(symbol).gf(symbol,symbol).
Clauses  fa(a,b).fa(a,d).fa(d,f).fa(b,c).fa(a,e).fa(d,g).fa(d,h).fa(e,i).
         fa(e,j).fa(f,k).fa(g,l).fa(i,m).fa(i,n).fa(k,o).fa(k,p).
         good(d).good(e).good(f).good(g).good(n).
         h(a,150).h(b,160).h(d,155).h(e,148).h(c,138).h(f,167).h(g,156).h(h,180).
         h(i,147).h(j,159).h(k,162).h(l,176).h(m,158).h(n,161).h(o,173).h(p,166).
 gf(X,Y):-fa(X,Z),fa(Z,Y).
 having_no_son(X):-not(fa(X,_)).
 having_two_sons(X):-fa(X,Y),fa(X,Z),not(Y=Z).
```



fa(X,Y) states that X is father of Y. h(X,Y) states that height of person X is Y. good(X) states that X is a good person.

1. fa(d,Y) gives 3 solutions f, g and h. These are sons of ‘d’.
2. gf(T,k) gives grandfather of ‘k’, which is ‘d’. gf(X,Y) gives all pairs where first element is grandfather of second.
3. having_no_son(X) returns True if X is not father of any one.
4. having_two_sons(‘x’) returns True if X is father of at least two persons. having_two_sons(X) gives the names of those persons, who have at least two sons. [Caution: having_no_son(X) does not give the names of those persons who do not have any son]

Define following functions (no other predicate should be used unless essential**)**

1. Brother. Brother(g,T) gives T=f and h.
2. Uncle (father’s brother). Uncle(d,T) gives T=c,i and j.
3. Cosine (Common grandfather but different father). Cosine(f,T) gives T=c,i and j.
4. Define predicate to print all sons. allsons(a) prints b,d,e.
5. Define predicate to print all good sons. allgoodsons(a) prints d,e.
6. Define predicate to return all good sons. allgoodsons(a,T) gives T=d and T=e.
7. Define predicate to return all those persons, who are taller than their father. [taller(X) gives bdfghjlmnop]
8. Having brother. having_brother(T) gives T=b,d,e,f,g,h,i,j,m,n,o,p.
9. Having no brother. having_no_brother(c) is true. having_no_brother(d) is false.
10. Having no grandson (try to do it without any other predicate. Check for ‘d’).
11. Having (at least) two grandsons (use no other predicate).
12. Having no good son. e.g. ‘e’ has no good son. ‘o’ has no good son.
13. Having at least one good son.
14. non tallest son. non_tallest_son(a,X) gives two solutions d and e.
15. tallest son. tallest_son(a,X) gives X=b.
16. Having exactly one son. having_exactly_one_son(T) gives T=b,f and g.
17. Having exactly two sons. having_exactly_two_sons(T) gives T=k,e and i.

18. How many persons are good in a given list of persons. count([b,d,g,i],Y) gives Y=2.
19. Which persons in the given list are good? gd([b,d,g,i],T) gives T=[d,g].
20. Given list of persons find corresponding list of heights. hl([a,g,e,k],T) gives T=[150,156,148,162].
21. How many pairs in the given list are father son? cnt([[a,c],[d,g],[e,i],[i,e]],T) gives T=2 because [d,g] and [e,i] are father son.
22. Given list of persons. Form pair wise height. hl([a,g,e,k],T) gives T=[p(a,150), …]
23. Define f(X,Y). It returns true if X is father of every element of Y. f(d,[g,h]) is true. f(a,[d,g]) is false.
24. How many persons in a given list have height in a given range? rg([b,d,g,i],150,170,I) gives I=3.
25. sum of heights in given list. sum([b,z,g,y,x,h],T)=160+156+180=496. If height of an element is undefined then it is taken as 0.
26. Member. member(d,[c,k,e]) is false since the list does not has ‘d’.
27. All sons, which are not in the given list. g(a,[b],T) gives T=d and e. g(k,[o,p],T) gives no solution. [Hint: use member]
28. List of all sons, which are not in the given list. g(a,[b],T) gives T=[d,e]. g(k,[o,p],T) gives T=[].
29. List of all sons. son(d,T) gives T=[f,g,h]. [Hint: use above predicates]
30. Ancestor (either father or grandfather or great-grandfather and so on). Ancestor(T,‘k’) gives T=f, d and a.
31. Given fa(a,b).fa(a,e).fa(e,f).fa(b,c).fa(b,d).fa(e,g). Find solutions of following.

(A)fa(a,X),fa(X,Y),!. (B)fa(X,Y),fa(a,X),!. (C)fa(a,X),!,fa(X,Y) (D)fa(X,Y),!,fa(a,X).

Similarly define functions for second son, second grandson, first son of all sons.

Why (A) in above does not find the first son’s first son. Correct it to find first son.

Dynamic Database

```
domains li=integer*.
database   g(integer).f(integer).psh(integer).abc(integer,integer).
predicates   insert(li).sum(integer).last(integer).stack(li).first(integer).fib(integer,integer).
clauses
  insert(X):-X=[A|B],asserta(g(A)),insert(B).
  f(5).f(4).f(6).f(7).
  sum(X):-not(f(_)),X=0.  sum(X):-f(Y),retract(f(Y)),sum(T),X=T+Y.
  last(X):-f(G),f(I),G>I,retract(f(G)),last(X).
  last(X):-f(G),f(I),I>G,retract(f(G)),last(X).
  last(X):-f(X),retract(f(_)).
  fib(1,1). fib(2,1).
  fib(X,Y):-X>2,abc(X,Y),!.
  fib(X,Y):-X>2,T=X-1,fib(T,P),S=X-2,fib(S,Q),Y=P+Q,assertz(abc(X,Y)).
  stack(X):-X=[A|B],A>0,asserta(psh(A)),stack(B).
  stack(X):-X=[A|B],A<0,first(T),retract(psh(T)),stack(B).
  first(X):-psh(X),!.
```

1. insert ([5,12,3]) will insert 3 solutions of g(X). Now g(X) will give 3, 12, 5 as solutions. If assertz is used in place of asserta then solutions are 5, 12, 3. asserta inserts in the beginning. assertz inserts at end.
2. sum(X) returns 5+4+6+7. It is sum of all solutions of f(x). If sum(X) is executed again then answer is 0. It is because solutions are deleted. Hence it is advisable to recompile the program before reexecution. The retract(X) function removes the first solution. If X given then the first solution with that X is removed. e.g. if f(T) has solutions 3,4,5,7,5,1,2 then after the execution of retract(f(5)) solutions shall be 3,4,7,5,1,2.
3. last(X) finds the last solution of f(X).
4. fib(X,Y) computes Y=fib(X). If ordinary approach is used then Fibonacci for big numbers can not be computed because of recomputation involved.
5. stack([8,2,3,-1,7,6,4,-1,-1]) pushes 8,2,3 into stack and performs a take. Then 7,6,4 are pushed and two takes are performed. Now solutions of psh(X) are 7,2,8. Here any positive number is pushed and -1 acts for take. If asserta is used then it acts like queue.

Define following functions (Cut should not used unless permitted**)**

1. Define insrt. insrt([5,2,7,9,3,4,8) inserts h(5,2), h(2,7), .., h(4,8).
2. Define insrt. insqt([5,2,7,9,3,4,8]) inserts h(3,4), h(7,9), h(5,2), h(2,7), h(9,3), h(4,8).
3. Define remove. Remove([6,9,3]) will remove 6,9, and 3. Now solutions of f(X) will be 5,4,7.
4. Define Ackermann's function. Cut can be used. If logic is following.
   A(m,n)=n+1 if m=0 A(m,n) =A(m-1,1) if n=0 A(m,n) =A(m-1,A(m,n-1)) otherwise
5. Define function to find the number of solutions.
6. Define sum to find sum of solutions, without using not.
7. Define sum so that database is unchanged.
8. Define function for finding first solution, second solution, n^{th} solution.
9. Define function to find the biggest solution.
10. Define function to find the biggest solution less then Y.
11. Define listform. listform(X) returns [5,4,6,7]. All solutions of f are put into list.
12. Define retracta. The last solution is deleted.
13. Define retractb. The nth solution is deleted.
14. Define assertb. The solution is inserted at n^{th} place.
15. In the game of numbers every player is supposed to write 1, 2 or 5 bigger than what his opponent has written. A player who writes bigger than 50 wins. Write program to find good number. In place of 1, 2, and 5 variables x, y and z should be used. Cut can be used. Good(X) will give 51,45,42,39,.. as solutions. Bad(X) will give 50-46,44,43,… as solutions.
16. Find good numbers in the game with restriction. Cut can be used.

17. Define data base for father. Write program to find first son, the last son, the nth son, the first grandson.

18.

Cut

```
g(T,X):-T>5,X=T*T.  g(T,X):-T mod 2=0,!,X=T+100.   g(T,X):-T mod 3=0,X=T*10.
h(T,X):-g(T,X).   h(T,X):-P=T+T div 10,g(P,X).
f(4).f(7).f(3).f(9).f(2).
absent(X):-f(X),!,fail.          absent(_).
first(X):-f(X),!.
bigger(X,Y):-f(Y),Y>X,!.
biggest(X,Y):-bigger(X,Z),!,biggest(Z,Y).  biggest(X,X).
bgst(X,Y):-f(A),A>X,bgst(A,Y).    bgst(X,Y):-f(A),A>X,not(bgst(A,_)),A=Y.
max(X):-bgst(0,X).
```

34. The function g(x) return x^2 if x>5. If x is even then x+10 is returned. If x is not even, but is multiple of 3 then x*10 is returned. [Question: What difference will it make if cut is put after X=T+100?] When function h calls g then cuts of g are not applicable to h (find h(30)).
35. absent(x) returns true if x is not any solution of f. Similarly 'first' returns the first solution of f.
36. bigger(x) returns some solution produced by f which is bigger than x. biggest(x) returns the biggest number among solutions of f and x. It can be used to find the biggest solution. bgst is an alternative for biggest.

Define following functions

1. Define function g. If f has some even solution then square of that solution is returned. Otherwise if f has some solution more than 10 then its double is returned. Otherwise triple of some solution is returned.
2. Define function, which will produce the second biggest solution produced by f(X).
3. Define function which will produce biggest even solution.
4. Define function, which will produce the biggest solution less than Y.
5. Define function which will find how many solutions are produced by f.
6. Define function to find all solutions of f which are not present in the given list.
7. Define listform. listform(X) returns [4,7,3,9,2]. All solutions of f are put into list.

Answer following questions without using computer.

8. Given fa(a,b).fa(a,e).fa(e,f).fa(b,c).fa(b,d).fa(e,g). Find solutions of following.
   (A) fa(a,X),fa(X,Y),!. (B) fa(X,Y),fa(a,X),!. (C) fa(a,X),!,fa(X,Y) (D) fa(X,Y),!,fa(a,X).
   Similarly define functions for second son, second grandson, first son of all sons.
9. Let pf(X,Y) be a predicate. It produces prime factors of X in increasing order. e.g. pf(84,A) will produce A=2,3,7 as solutions. Find solutions of followings.
   (A) pf(30,A),pf(70,A). (B) pf(30,A),!,pf(70,A).
   (C) pf(84,A),A>4,pf(70,A) (D) pf(84,A),!,A>4,pf(70,A) (E) pf(84,A),A>4,!,pf(70,A).
10. Find solutions for fact(2,A) fact(3,A) fact(4,A) fact(5,A)
```
fact(1,1).
fact(X,Y):-X>1,T=X-1,fact(T,G),Y=G*X.
fact(X,Y):-X>1,T=X-1,fact(T,G),!,Y=G+X.
```
11. Find solutions for abc(3,A) abc(4,A) abc(5,A) abc(6,A)
```
abc(X,Y):-X>1,T=X-1,abc(T,P),Y=P+X.
abc(X,Y):-X<5,!,X<3,Y=10+X.
abc(X,Y):-T=X-2,abc(T,P),Y=P+1.
```
12. Find solutions for last(A,[4]) last(A,[2,4]) last(A,[3,2,4]) last(A,[5,3,2,4])
```
last(X,[X]).
last(X,[_|Z]):-last(X,Z).
last(X,[W|Z]):-last(T,Z),!,X=T+W.
```
13. Find solutions for ff(A,6,2) ff(A,10,2) ff(A,11,2) ff(A,35,32)
```
ff(X,Y,Z):-Y mod Z=0,X=Z.
ff(X,Y,Z):-Y>Z,T=Z+1,!,ff(X,Y,T).
ff(X,Y,Z):-Y>Z,T=Z+2,ff(L,Y,T),!,X=L+1.
```
14. Find solutions for
15. Find solutions for
16. Find solutions for

17. a
18.

Lisp

An S-expression is any one of the followings

1. Any number e.g. 12.5, 17.92, 34, -45
2. Any symbol e.g. ram, x, tt
3. List of S-expressions e.g. (ram 12.5 34.2), ( ), (12 (34 56)(2 3((3 4))))

Following rules are used to find value of an S-expression

1. Value of any number is the same number.
2. Value of any symbol is obtained from symbol table.
3. (A) Value of null list ( ) is same.
   (B) If a list contains more than one element then its value is guided by its first element. If first element of a list is an inbuilt function then its value calculated from the following rules.

S expression	Value of S-expression
(quote x)	x e.g. value of (quote (7 ram 12)) is (7 ram 12) (shortform ‘x)
(+ x y)	Sum of the values of x and y. Similar functions -,*,/,div,mod.
(> x y)	It is true if value of x is more than the value of y. Similar functions <,>=,=
(and x y)	It is true if values of both x and y is true. Similar function or, not.
(atom x)	It is true if value of x is a number, or symbol, or ( ).
(car x)	First element of the value of x.
(cdr x)	List obtained by deleting first element from the value of x.
(cons x y)	Value of x is inserted as the first element in the value of y.
(list x y z)	It forms the list of values of x, y and z.
(cond (x y)(z w)(s t))	If value of x is true then its value is equal to the value of y. otherwise (x is false) if value of z is true then it is equal to the value of w. Similarly for (s t). Normally the value of s must be true. It is made by writing T.
(eval x)	Value of value of x.

Observe value of followings.

42. (car (quote(5 1 7))) returns 5. It can also be written as (car ‘(5 1 7))
43. (cdr(car(quote((5 2 7)3 9)))) returns (2 7). Because (car ‘((5 2 7)3 9)) returns (5 2 7).
44. (and(> 15 7)(< 2 3)) returns true. It is because (> 15 7) and (< 2 3) are both true.
45. (cons ‘(2 3) ‘(7 9 6)) returns ((2 3) 7 9 6). (list ‘(2 3) ‘(7 9 6)) returns ((2 3) (7 9 6)).
46. (caddr(quote(1 + * -))) returns *. It is short form of (car(cdr(cdr(quote(1 + * -))))).
47. (cons(cadr (quote(5 + 7)))(quote (2 9))) returns (+ 2 9).
48. (eval (cdr ‘(4 * 6 7 5))) returns 210, because (cdr ‘(4 * 6 7 5)) returns (* 6 7 5)
49. (eval (cons (car (quote(+ * -))) (quote (3 9)))) returns 12.
50. (list ‘(3 4) ‘ram ‘hari) returns ((3 4) ram hari)

(C) If first element is user defined function then the value of the list is equal to the value of the function body. However before calculating value of the function body the value of every parameter is set (in the symbol table) as the value of the corresponding argument.

Let function f is defined as (Defun f (x y) (+ x y (* x (cond ((> x 5) y) (T 9)))))

Value of (f 6 7) will be calculated by following method:

(i) In symbol table make the values of x and y as 6 and 7 respectively.
(ii) Find the value of (+ x y (* x (cond ((> x 5) y) (T 9)))). It is 55.
(iii) Remove the values of x and y.

51. (defun f (x y) (+ (* x y) x)) (f 5 9) outputs 50.
52. (defun g (x) (cond ((> x 5)(+ x 7))(T(* x x)))) (g 3) outputs 9 (g 8) outputs 15.
53. (defun secd(x) (car (cdr x))) (secd ‘(3 6 5)) outputs 6.
54. (defun delsecond(x) (cons (car x)(cddr x))) (delsecond ‘(3 4 7 2)) outputs (3 7 2).
55. (defun putsec(x y) (cons (car x)(cons y (cdr x)))) (putsec ‘(3 2 4) 7) outputs (3 7 2 4).
56. (defun h (x) (list ‘ram ‘x x ‘mohan ‘hari)) (h 5) outputs (ram x 5 mohan rari).
57. (defun m1 (x y z) (list x y z)) (m1 ‘+ 7 9) outputs (+ 7 9).
58. (defun m (x y z) (eval (list x y z))) (m ‘+ 7 9) outputs 16. (m ‘f 5 9) outputs 50.
59. (defun kp (x) (list ‘defun ‘g ‘(p) (list ‘+ x ‘p))) (kp 9) outputs (defun g (p) (+ 9 p)).

60. (defun k (x) (eval (kp x))). After the execution of (k 9) if (g 7) is executed then output is 16. (g 18) outputs 27. After the execution of (k 25) they output 32 and 43 respectively.

Define following functions

1. Function to find sum of first two elements. (sm ‘(12 5 34)) returns 17.
2. Function to replace the first element by given element. (rep 8 ‘(4 5 7)) returns (8 5 7).
3. Function to increment first element. (inc ‘(5 2 6 7)) returns (6 2 6 7)
4. Function to find product of first element of given lists. (pr ‘(5 2) ‘(6 8 2)) returns 30.
5. Implement g(x)=x if x<10. g(x)=x^2 if x>20. g(x)=x+80 otherwise. (g 12) returns 92.
6. Implement f(x)=x^2 if x even. f(x)=g(x-1) if x odd. (f 31) returns 900.
7. Function to exchange first two elements. (exch ‘(4 6 2 5 9)) returns (6 4 2 5 9).
8. Define cf. It returns first element of given list if it is even. Otherwise if returns the second element. (cf ‘(12 4 6)) returns 12. (cf ‘(9 7 26 7)) returns 7.
9. Function to increment first element of a list if it is odd. (t ‘(9 11 2)) returns (10 11 2). (t ‘(4 3 2)) returns (4 3 2).
10. Define ab. (ab ‘ram ‘hari) returns (5 7 ram 9 hari 8). (ab ‘2 ‘9) returns (5 7 2 9 9 8).
11. Define gh. (gh 2 7) returns (5 (8 2) (4 7)). (gh ‘ram ‘hari) returns (5 (8 ram)(4 hari)).
12. Define st. (st ‘a ‘b ‘c) returns (5 (a (b c) c) ((8 hari))).
13. Define function sq, cube, inc, double. (sq 5) returns 25. (inc 6) returns 7.
14. Define function tt. (tt ‘a ‘b) returns (defun a (p) (b p)).
15. Define function tg. If (tg ‘mg ‘sq) is called then a function “mg” is defined. (mg 7) returns 49. (mg 5) returns 25. If (tg ‘hk ‘cube) is called then (hk 4) returns 64.
16. Define function mt. (mt 9) returns (defun gg(p) (cond ((> p 9)(* p p))(T(+ p 8)))).
17. Define function ms. If (ms 9) is called then (gg 5) and (gg 15) will return 13 and 225 respectively. If (ms 20) is called then (gg 5) and (gg 15) will return 13 and 23 respectively.
18. Define function fs. (fs ‘a ‘b) returns (a (+ b 1)).
19. Define function ft. (ft ‘sq 2) returns 9. (ft ‘cube 3) returns 64. (ft ‘double 3) returns 8.
20. Define function kt. (kt ‘sq ‘cube 5) returns $5^2 – 5^3$ =–100. (kt ‘double ‘inc 5) returns 2(5)-(5+1)=4.
21. Define function fm. (fm ‘a ‘b ‘(p q r s)) returns (- (a p q r s) (b p)).
22. Define function fl. (fl ‘+ ‘sq ‘(2 3 5 4)) returns 10. Because ((2+3+5+4)-2^2)=10.
23. Define function gen. Call to function (gen 5) will define a function adt. Now if (adt 12) is called then answer is 17. If (adt 3) is called then answer is 8. Similarly call to (gen 3) will define adt, which will add 3 to input

Recursion

```
(defun fact (x) (cond  [(= x 0) 1]   [T  (*  x  (fact (- x 1)))]))
(defun last  (x) (cond [(null (cdr x)) (car x)]  [T (last (cdr x))]))
(defun add (x) (cond  [(null x) 0]  [T   (+  (car x) (add (cdr x)))]))
(defun append(x y) (cond [(null x)y][T (cons (car x) (append (cdr x) y))]))
(defun flat(x)(cond [(null x)x] [(atom(car x))(cons(car x)(flat(cdr x)))]
                    [T(append(flat(car x))(flat(cdr x)))] ))
```

(fact 5) returns factorial 5. (last ‘(3 4 5)) returns 5. (add ‘(3 4 5 7)) returns 3+4+5+7=19.
(append ‘(2 3 4)) ‘(a b)) returns (2 3 4 a b). (cons ‘(2 3 4) ‘(a b)) returns ((2 3 4) a b).
(flat ‘(2 (3 4) ((a b) 4 5))) returns (2 3 4 a b 4 5).

1. Function to find the sum of numbers in given range. (sum 5 8) returns 5+6+7+8=26.
2. Function to find n^{th} element. (nth (quote (2 7 6 4)) 3) returns 6.
3. Function to find first even element. (first_even ‘(3 5 8 2 4 1)) returns 8.
4. Function to find the number of elements. (count ‘(2 3 (2 3) 6)) returns 4.
5. Function to find the number of even elements.
6. Product of consecutive numbers is taken and their sum is returned. (sop ‘(5 7 2 9)) returns 5.7+7.2+2.9=35+14+18=67.
7. Function to delete the last element. (delete_last (quote (2 7 6 4))) returns (2 7 6).
8. Function to put a given element at the end. (put_end ‘(2 4 1 5) 9) returns (2 4 1 5 9).
9. A list of complex numbers is given. Define function to find sum of all elements. Number 2+7i is represented as (2 7). (sum ‘(3 (4 5) 6 3 (8 2) (0 2))) returns (24 9).
10. Function to find smallest factor (greater than or equal to). (f 770 7)=7. (f 36 10)=12. It is because the smallest factor of 36, which is greater than (or equal to) 10 is 12.
11. Function to find smallest factor. (sf 385) returns 5. (additional function may be used)
12. Use above to check whether a given number is prime or not. (prime 17) returns T. (prime 10) returns NIL. [Hint: smallest factor of a prime number is equal to itself].
13. Function to find the sum of primes in a given list. (sum ‘(23 45 22 13 7)) returns 23+13+7=43. The function to check primacy can be used.
14. Function to find the maximum element (max (quote (2 7 6 4))) returns 7.
15. Function to check whether the list has any even element. (even (quote (2 7 6 4))) returns T. (even (quote (3 7 9))) returns False (or Nil).
16. Function to check whether elements are in increasing order. (odr ‘(1 4 6 8)) returns T.
17. Function to delete n^{th} element. (del_nth (quote (2 7 6 4)) 3) returns (2 7 4).
18. Function to increment every element. (increment (quote (2 7 6 4))) returns (3 8 7 5).
19. Function to increment every element. If some element is list then its elements are also incremented. (inc (quote (2 (7 (6 9)) 4))) returns (3 (8 (7 10)) 5).
20. All locations of a given element are returned in list form. (loc ‘(2 6 8 3 6 8 6 7) 6) returns (2 5 7). [Function to increment every element of list can be used.]
21. A list of binary functions and two numbers are given. Each of the given functions is applied on given numbers. All answers are put in a list. (abc (quote (+ * + –)) 5 7) returns (12 35 12 –2).
22. Define function gennth. Call to function (gennth 3) will define a function nth. Function nth will return 3rd element of a given list. (nth ‘(1 9 8 2 7 5)) will return 8. If we call (gennth 5) then call of (nth ‘(1 9 8 2 7 5)) will return 7.
23. Define function gen. If we call (gen ‘(ab pq rs) ‘(sq inc dbl) ‘(inc cube sq)) then functions ab, pq and rs are defined. Call of (ab 7) returns 7^2+1. Call of (pq 7) returns $(7+1)^3$=512. Call of (rs 7) returns $(2*7)^2$=196. Assume functions sq, inc and dbl are defined. If we call (gen ‘(t try wq yu pke) ‘(sq sq dbl cube sq) ‘(dbl cube inc cube inc)) and (wq 4) then (2*4)+1=9 is returned.
24. Function to place a given element at n^{th} place. (put ‘(2 7 6 4) 9 3) returns (2 7 9 6 4).
25. Put a given element at proper place in sorted list. (ppp ‘(2 7 6 8) 3) returns (2 3 7 6 8).
26. Use above to sort a given list by insertion sort technique.

27. Write program to sort a given list by selection sort technique. Define functions to find minimum element. Delete minimum element from the list and recursively sort it. Now put the minimum element at the beginning.
28. Write program to sort a given list by quick sort technique. Take the first element (say t). Decompose the given list in two parts. The first list contains all elements smaller than or equal to t. The second list contains all elements bigger than t. Sort both lists independently. Now append them.
29. Given two sorted lists, merge them into one sorted list.
30. Use above function to sort a list by merge sort technique.

31.
37.

Cut: Stop during backward motion

a(2,12) a(5,14) a(16,11) a(8,15) a(5,18) a(14,29) a(7,19) a(6,15) a(14,32) a(8,24) a(4,15) a(5,16) a(15,5)

1. a(5,?) is 14,18,16 because a(5,14) a(5,18) a(5,16) are present
2. a(?,15) is 8,6,4 a(?,21) is no solution
3. a(5,X),?=X+100 is 114,118,116
4. a(5,X),X>15,?=X is 18,16

5. a(5,X),a(X,Y),?=Y+100 is 129,132,111. Because a(5,X) gives 14,18,16. a(14,Y) gives 29,32. a(18,?) gives no solution. a(16,?) gives 11. Method: First solution of a(5,X) is found. It is X=14. Corresponding to that solution a(14,Y) is obtained. Its first solution is Y=29. Now ?=Y+100 is done. Now we go back and try to find another solution of a(14,Y). It is Y=32. Corresponding to it ?=Y+100 is done. Now we go back and try to find another solution for a(14,Y). It does not exist. Hence we try to find another solution for a(5,X). It gives X=18. Corresponding to it a(18,Y) is obtained. It has no solution. Now we try to find another solution for a(5,X) and proceed.

6. a(5,X),@,a(X,Y),?=Y+100 is 129,132. First solution of a(5,X) is X=14. Now @ is done. It does not do any thing. Now first solution for a(14,Y) is obtained. It is Y=29. Now ?=Y+100 is done. It gives 129. Now we go back and try to find another solution of a(14,Y). It is 32. Corresponding to it ?=Y+100 is 132. Now we go back and try to find another solution a(14,Y). It does not exist. Hence we go back and do @. It stops us. [@ stops only during backward motion].

7. a(5,X),a(X,Y),@,?=Y+100 is 129. First solution of a(5,X) is X=14. First solution a(14,Y) is 29. @ does not do any thing. Now ?=Y+100 is done. It gives 129. Now we go back and do @. It stops us.

8. a(5,X),X>15,@,?=X is 18. First solution for a(5,X) is X=14. Now 14>15 is done. It is false, hence we go back and try to find another solution for a(5,X). It is X=18. Since 18>15 is true, we proceed. @ does not do anything. ?=X gives solution as 18. Now we go back and do @. It stops.

9. a(5,X),@,X>15,?=X gives no solution. First solution for a(5,X) is X=14. @ does not do anything. Since 14>15 is false, hence we go back and do @. It stops.

10. a(5,X),a(8,Y),?=X+Y is 29,38,33,42,31,40
11. a(8,Y),a(5,X),?=X+Y is 29,33,31,38,42,40
12. a(5,X),a(8,Y),@,?=X+Y is 29.
13. a(5,X),@,a(8,Y),?=X+Y is 29,38.
14. a(8,?) is 15,24. a(8,X),?=2X is 30,48. a(8,X),a(8,Y),?=X+Y is 30,39,39,48
15. a(5,X),?=X is 14,18,16 a(5,X),a(8,Y),?=X is 14,14,18,18,16,16 a(8,Y),a(5,X),?=X is 14,18,16,14,18,16. a(8,Y),a(5,X),?=Y is 15,15,15,24,24,24.

F(x): $x<10$, x^2 F(x): $x\geq 10$, $x<20$, 5x+6 F(x): $x\geq 20$, y=x+5, 2y	F(5) is 25 F(12) is 66 F(15) is 81 F(25) is 60 F(40) is 90
G(x): (x even), y=x+7, F(y) G(x): (x odd), $x<5$, x^2+10 G(x): (x odd), $x\geq 5$, F(x)+100	G(2) is 81 G(4) is 61 G(3) is 19 G(7) is 149
H(x): $x<10$, x^2	H(5) is 25,31 H(7) is 49,41 H(8) is 64,46

H(x): $x \geq 4$, $x < 20$, 5x+6 H(x): $x \geq 30$, y=F(x), z=F(x-25), y+z	H(12) is 66 H(25) has no solution H(40) is 171
K_1(x): z=H(x), 2z	K_1(5) is 50,62
K_2(x): z=H(x), w=H(x), z+w	K_2(5) is 50,56,56,62
K_3(x): z=H(x), z+z	K_3(5) is 50,62
L(x,y): z=H(x), w=H(y), z+w	L(7,8) is 113,95,105,87 L(8,7) is 113,105,95,87
P(x): 2x+3 P(x): $x \geq 10$, $x < 20$, @, x+100 P(x): (x even) x^2	P(5) is 13 P(6) is 15,36 P(30) is 63,900 P(40) is 83,1600 P(16) is 35,116 @ means stop during backtrack
Q(x): $x \geq 15$, P(x) Q(x): $x < 30$, @, 2x, Q(x): 5x+2	Q(8) is 16 Q(40) is 83,1600,202 Q(16) is 35,116,32 (stop at P(16) has no effect in Q, It is done as independent problem)
R_1(x): $x \geq 4$, @,y=H(x),y+100 R_1(x): (x odd) x+200	R_1(5) is 125,131 R_1(3) is 203 R_1(2) has no solution R_1(25) is no solution (because H(25) has no solution)
R_2(x): $x \geq 4$, y=H(x),@,y+100 R_2(x): (x odd) x+200	R_2(5) is 125. We start with one solution of H(5). R_2(25) is 225. (Since H(25) has no solution, @ is not crossed)
L_1(x,y): z=H(x), w=H(y), @, z+w	L_1(7,8) is 113 L_1(8,7) is 113
L_2(x,y): z=H(x), @, w=H(y), z+w	L_2(7,8) is 113,95 L_1(8,7) is 113,105

NAME:	Roll No.

f(2,12) f(7,14) f(16,11) f(7,13) f(5,18) f(14,29) f(7,19) f(2,15) f(15,32)
f(15,24) f(4,15) f(5,16) f(2,5)

1. What is f(7,X),X>15,?=X+550 _____
2. What is f(2,X),f(7,Y),?=X+Y _____
3. What is f(2,X),f(X,?). _____
4. What is f(2,X),f(X,Y),@,?=Y+550 _____
5. What is f(2,X),@,f(X,Y),?=Y+550 _____

6. Let d(6,?) is 18,40,25,92 and d(5,?) is 12,30,40,51,52,58,66,72,73,78.
   (A) What is d(6,X),d(5,Y),X+Y>100,@,?=X+5002 _____
   (B) What is d(5,Y),d(6,X),X+Y>100,@,?=X+5002 _____

7. Let d(7,?) has 10 solutions and d(5,?) has 527 solutions.
   (A) How many solutions d(7,X),d(5,Y),?=X. will have _____
   (B) How many solutions d(7,X),@,d(5,Y),?=X will have _____
   (C) How many solutions d(5,Y),@,d(7,X),?=X will have _____

A(x): $x<10$, x^2 A(x): $x\geq 10$, $x<20$, 7x+6 A(x): $x\geq 20$, y=x+5, 524+y	What is A(40)?		
B(x): (x even), y=x+7, A(y) B(x): (x odd), $x<5$, x^2+10 B(x): (x odd), $x\geq 5$, A(x+1)+527	What is B(13)?		
C(x): (x even), y=x+7, A(y) C(x): (x odd), $x<5$, x^2+10 C(x): (x odd), $x\geq 5$, C(x+1)+527	What is C(13)?		
D(x): $x<10$, x+275 D(x): $x\geq 4$, $x<20$, x+513 D(x): $x\geq 30$, y=B(x), z=A(x−40), y+z	What is D(8)?	What is D(25)?	What is D(44)?
E(x): z=D(x), y=A(x), z+y+5000	What is E(5)?		
S(x,y): z=D(x), w=A(y), 2z+w	What is S(5,6)?	What is S(6,5)?	
T(x): 2x+520 T(x): $x\geq 10$, @, $x<20$, x+225 T(x): (x even) x^2	What is T(5)?	What is T(16)?	What is T(30)?
U(x): $x\geq 15$, T(x)+5000 U(x): $x<30$, @, 2x+54 U(x): x+509	What is U(10)?	What is U(40)	

Min-Max

1. Suppose we want to find minimum number in the set {6,3,4,2,5} we shall first observe 6, then 3, then 4, then 2 and then 5. Now we shall conclude that it is 2.

2. Suppose we want to know "whether minimum element of {6,7,2,4,1} is 3 or not", we shall observe 6, then 7, then 2. Now we shall conclude the minimum element can not be 3 (it should be 2 or less). Hence elements 4 and 1 are unobserved.

3. Suppose we want to know "whether minimum element of {6,2,5,7,1,8,9} is less than or equal to 4", we shall observe 6, then 2. Now we can conclude that minimum element is "less than or equal to 2" (hence it is certainly less than or equal to 4). Hence elements 5,7,1,8,9 are unobserved.

NOTATION: (a,b,c,d) means minimum of a,b,c,d. [a,b,c,d] means maximum of a,b,c,d. Hence (2,3,4,1,5,6) is 1 and [6,2,5,4,8,1] is 8. (4,[5,7,2,3]) is (4,7)=4.

4. (5,[3,6,x]) can be computed even when the value of x is not known. If x is a big number like 9 then [3,6,9]=9 hence (5,9)=5. If x is a small number like 2 then [3,6,2]=6 and (5,6)=5.

5. Suppose we want to know the value of (5,[1,4,3,6,2,8]) then elements 2 and 8 are unobserved. It is because after observing 6 it can be concluded $[1,4,3,6,\ldots]\geq 6$. Now $(5, \geq 6)=5$.
6. Similarly in [5,(8,9,5,7,11)] elements 7 and 11 are unobserved. During the processing of (8,9,5,7,11) after observing 5 it can be concluded that $(8,9,5,..)\leq 5$. Now $[5,\leq 5]=5$.

7. Suppose we want to know [(4,3,7,2),(3,7,1,5),(8,5,4,7),(3,7,1,5)], we shall first find (4,3,7,2)=2. Now $(3,7,1,..)\leq 2$ and [2,x,….] does not depend upon the value of 'x' if it is less than or equal to 2. Hence element 5 in unobserved. (8,5,4,7)=4. When we compute (3,7,1,5) then after observing (3,..) it can be concluded that it is (≤ 4). Now [2,x,4,y]=4 if x and y are less than or equal to 4. Hence unobserved elements are 5, 7, 1 and 5.

8. In [5,([3,2,4],[7,1,8],9,6)] unobserved elements are [7,1,8],9,6. It is because [3,2,4]=4 and [5,(4,x,y,z)] does not depend on the values of x, y and z.

9. In [5,([(8,3,7,6),18,2,19],91)] during processing of (8,3,7,6) unobserved elements are 7 and 6. It is because after observing 3 it can be concluded that $(8,3,\ldots)\leq 3$. [5,([x,y],z)] does not depend upon the value of 'x' if it is (≤ 5). It is because [x,y]=y if $x\leq y$. Otherwise $[x,y]=x\leq 5$. In second case $([x,y],z)\leq 5$ and [5,([x,y],z)]=5.

Notation

$(a,b,c,d)^p$=minimum element of a,b,c,d if it is more than p. Otherwise it is undefined.
$(a,b,c,d)_p$=minimum element of a,b,c,d if it is less than p. Otherwise it is undefined.
$[a,b,c,d]^p$=maximum element of a,b,c,d if it is more than p. Otherwise it is undefined.
$[a,b,c,d]_p$=maximum element of a,b,c,d if it is less than p. Otherwise it is undefined.

10. $(5,3,7,6)^4$=undefined. In its computation elements 7 and 6 are unobserved. It is because after observing 3 it has become certain that minimum element is less than or equal to 3 (hence less than or equal to 4).
11. Similarly $[2,4,1,7,9,3]_4$=undefined. In its computation elements 1,7,9 and 3 are unobserved. It is because after observing 4 it has become certain that maximum element is more than or equal to 4. It can not be less than 4.

12. $[3,2,5,1]^8$=undefined. However in its computation all elements are observed.
13. Similarly $(7,12,6)_4$=undefined. However in its computation all elements are observed.

14. To compute (a,b,c,d) we observe 'a' then try to find $(b,c,d)_a$. It is because minimum of (b,c,d) is not needed if it is more than or equal to 'a'. In the computation of (5,[2,6,4,8]) after observing 5, we need $([2,6,4,8])_5$ hence $[2,6,4,8]_5$. In its computation elements 4 and 8 are not observed. After observing 6, it has become certain that maximum element is more than (or equal to) 5.

15. To compute [a,b,c,d] we observe 'a' then try to find $[b,c,d]^a$. It is because maximum of [b,c,d] is not needed if it is less than or equal to 'a'. To compute [5,([3,2,4],[7,1,8],9,6)] after observing 5, we try to find $([3,2,4],[7,1,8],9,6)^5$. Now $[3,2,4]^5$ is computed. In its computation all elements are observed but it is undefined. Hence $([3,2,4],[7,1,8],9,6)^5$ is undefined and unobserved elements are [7,1,8],9,6. It is follows from following rule(1).

Rules

Rule 0: a^p=a if a>p it is undefined if $a \leq p$
Rule 1: During the processing of $(a,b,c,d)^p$ if a^p is undefined then $(a,b,c,d)^p$ is undefined and b,c,d are not observed.
Rule 2: During the processing of $[a,b,c,d]^p$
(A) if a^p is undefined then $[a,b,c,d]^p$ may not be undefined and b,c,d are observed.
(B) if all of a^p, b^p, c^p and d^p are undefined then $[a,b,c,d]^p$ is undefined.
(C) If a^p=q then $[b,c,d]^q$ is computed. If it is undefined then $[a,b,c,d]^p$=q.

16. In [5,([(8,3,7,6),18,2,19],91)] after observing 5, we try to find $(8,3,7,6)^5$. It is undefined. Here unobserved elements are 7 and 6. However $[(8,3,7,6),18,2,19]^5$ is defined. It follows from above rule 2(A).

17. In [5,([(8,3,7,6),(7,4,9)],91)], we try to find $(8,3,7,6)^5$ and $(7,4,9)^5$. Both are undefined. Hence (rule 2B) $[(8,3,7,6),(7,4,9)]^5$ is undefined. Hence $([(8,3,7,6),(7,4,9)],91)^5$ is undefined (rule 1). Here unobserved elements are 7,6,9,91.

NAME: Roll no.

Notation: ‘k’ is between 14 and 17 means k=14,15,16 or 17. End numbers are included.

All numbers involved in the answers of various questions are positive integers.

18. Possible values of [15,(12,x),(17,y)] are 15, 17 or y. [15,(12,x),(17,y)]=12 or [15,(12,x),(17,y)]=x is never possible. Similarly possible values of [21,([51,x],[47,y])] are _____.

19. The value [13,25,17,(87,31,71,x)] is between _____ and _____. Here x can be any integer.

20. Let x be between 15 and 78 and y is between 41 and 83. Hence [x,y] is between _____ and _____.

21. Let x be some number between 12 and 41 then value of [18,(52,x)] will be between _____ and _____.

22. The minimum possible value of [25,x,(32,100-x)] is _____. Here x can be any integer.

23. The maximum possible value of (75,2x+10,[42,100-x]) is _____. Here x can be any integer.

24. [77,([11,72,47],[42,135,115],[88,95,47,93],89,91,[112,99,89,94,98],79,96),39,81,47,59]= _____.

25. [77,([(11,72,47),(42,135,115)],[88,95,47,93],89,[112,99,89,94,98],79,96),47,91,49,59] = _____.

26. [712,(314,x)]=x. What is x? _____. [Caution: This question appears to be wrong but it is correct]

27. Which elements are unobserved when [5,(9,6,8,4,7,19),7,(9,6,8,4,7,19)] is computed? _____.

28. When [11,(52,x,62,101),(43,32,68,97,49,88,45)] is computed then all elements are observed. What is x? any number between _____ and _____.

29. During the processing of [15,19,(80,x,75),(63,84,45,37,89)] the unobserved elements are 37 and 89. What is x? any number between _____ and _____.

30. What are unobserved elements in [5,([(8,7,6,3),(7,4,9)],91)]? _____.

31. What are unobserved elements in [5,([8,3,7,6],18,2,19),91]? _____.

32. To compute $[(89,17,189),(61,52,89),(92,68,87)]^{31}$ we compute $(61,52,89)^{p}$ and $(92,68,87)^{q}$. What are p and q? p= _____ q= _____.

33. We want to compute [15,([(124,131,167),(81,31,71,61),18,2,19],91)]. Here $(81,31,71,61)^p$ is computed. What is the value of 'p'? [ ].

34. We want to compute [5,(51,62),(17,19),(13,45)]. Hence $(13,45)^p$ is computed. What is p? [ ].

35. $(112,164,p+23,115)^{2p}$ is undefined. What is p? [any number between and].

36. During the computation of $[5+(115,127,112),41+(189,141,179)]^{100}$ the processing of $(115,127,112)^{95}$ and $(189,141,179)^p$ is done. What is p? [ ].

37. During the processing of which of the followings all elements will be observed (for all values of p). Write all correct choices.(A)$(3,9,7)_p$ (B)$[3,9,7]_p$ (C)$(3,9,7)^p$ (D)$[3,9,7]^p$ [ ].

Recursive Programming

1. Suppose we want to know the factorial of a positive integer. factorial(5)=120. factorial(6)=720=6.factorial(5) In general factorial(x)=x.factorial(x-1). However this rule is not valid for x=1. factorial(1) $\neq$ 1.factorial(0). It is because factorial(0) is undefined. Hence rule is factorial(x)=x.factorial(x-1) (when x>1) and exception is factorial(1)=1. Alternatively factorial(0) can be defined as 1.

2. Suppose we want to find sum of numbers between 'p' and 'q'. sum(5,8)=5+6+7+8=26. Here it is assumed that $p \leq q$. It can be seen that sum(4,8)=4+5+6+7+8=30. Here rule is sum(p,q)=p+sum(p+1,q). However this rule is not valid when p=q. In sum(6,6)=6+sum(7,6) the value of sum(7,6) is undefined.
   Hence Rule is sum(p,q)=p+sum(p+1,q) when p<q exception is sum(p,p)=p.

3. Let k(x) gives smallest prime number, which is bigger than x. k(8)=11 k(5)=7 k(22)=23 k(17)=19. It can be seen that k(x) and k(x+1) are same if (x+1) is not prime. e.g. k(21)=k(22)=23. However when (x+1) is a prime number then they are different. e.g. k(6)=7 but k(7)=11.
   Rule k(x)=k(x+1) if (x+1) is not prime
   Exception k(x)=x+1 if (x+1) is prime

4. Let $g(x,n)=1 + x + x^2 + x^3 + \ldots + x^n$.
   Hence $g(x,3)=1+x+x^2+x^3$. $g(x,4)=1+x+x^2+x^3+x^4$. $g(x,4)-g(x,3)=x^4$.
   Hence g(x,5)=g(x,4)+x(g(x,4) –g(x,3)).
   Hence rule is g(x,n)=g(x,n-1)+x(g(x,n-1) –g(x,n-2)). However this rule is not valid for n=1 because in g(x,1)=g(x,0)+x(g(x,0)–g(x,-1)) the values of g(x,0) and g(x,-1) are undefined. Similarly in g(x,2)=g(x,1)+x(g(x,1) –g(x,0)) the value of g(x,0) is undefined.
   Rule: g(x,n)=g(x,n-1)+x(g(x,n-1)–g(x,n-2)) when n>2
   Exceptions: g(x,1)=1+x $g(x,2)=1+x+x^2$.

5. Let sr(x) be (integer) square root of a positive integer x. When x is not a perfect square then lower bound is taken. sr(16)=4 sr(27)=5 sr(63)=7. If sr(x)=y then sr(x+1)=y or y+1. However this rule is not valid for x=0 because sr(0)=sr(-1) or sr(-1)+1 is not possible. It is because sr(-1) is undefined.
   Rule1: sr(x)=1+sr(x-1) when x>0 and $(1+sr(x-1))^2=x$
   Rule2: sr(x)=sr(x-1) when x>0 and $(1+sr(x-1))^2>x$
   Exception: sr(0)=0.

Notation: A list is represented as [a,b,c...]. On a list following operations are defined. Head means the first element of the list. Tail means remove the first element.
Head([2,5,1,7])=2 Tail([2,5,1,7])=[5,1,7] Head([9])=9 Tail([9])=[] Tail([])=undefined
Join is opposite. Join(5,[7,4,8])=[5,7,4,8] Join(6,[])=[6] Join(5,[7])=[5,7]
Caution: join(5,7) is undefined. Tail([9,6])=[6]. It is different than 6.
[] is called null list.When a list x has only one element then tail(x)=[].

6. Head finds the first element of a list. second element of a list can be found as follows second(x)=head(tail(x)). e.g. second([4,3,8,9])=head(tail([4,3,8,9]))=head([3,8,9])=3
7. Last([4,5,2,3])=3 last([5,2,3])=3 last([17,12,34])=last([12,34])=34. Hence it can be concluded that last element of a list and the tail of the list are same. However it is not valid when the list has only one element. last([5])=5 but last(tail[5])=last([]) is undefined.
   Rule: last(x)=last(tail(x)) when tail(x) $\neq$ null Exception: last(x)=head(x) if tail(x)=null
   Here null means []. If a list has only one element then tail(x) is null. e.g. tail([12])=[]

8. sum([5,4,2,3])=5+4+2+3=14. sum([4,2,3])=4+2+3=9. They differ by 5. Which is first element of [5,4,2,3]. Hence rule is sum(x)=head(x)+sum(tail(x)). However this rule is not valid when x is null because in sum([])=head([])+sum(tail([])) head and tail of a null list are undefined.
   Rule: sum(x)=head(x)+sum(tail(x)) when x is not null exception: sum([])=0

9. Let f(x,k) be k^{th} element of a list x. f([5,2,7,8,4,9],4)=8. f([2,7,8,4,9],3) is also 8. Hence k^{th} of a list and $(k-1)^{th}$ element of tail of the same list are same. However this rule is not valid for k=1. Because f([5,2,7],1) is 5 but f([2,7],0) is undefined.
   Hence rule is f(x,k)=f(tail(x),k-1) if k>1 exception is f(x,1)=head(x)

10. Let g(x) be the first even element of a list x. g([9,5,2,7,8,4,9])=2 g([5,2,7,8,4,9])=2. Hence first even element of a list and its tail are same. However it is not valid if first element of the list is a even number. e.g. g([8,3,2,7])=8 and g([3,2,7])=2.
    Rule g(x)=g(tail(x)) if head(x) is odd number exception g(x)=head(x) if head(x) is even

11. Suppose rp(x,y) replaces first element of list x by number y. rp([12,1,7,4],8)=[8,1,7,4]. It can be defined as rp(x,y)=join(y,tail(x)).
12. Let rq(x,k,y) replaces k^{th} element of a list x by number y. rq([5,4,9,8],2,12)=[5,12,9,8]. rq([7,5,4,9,8],3,12)=[7,5,12,9,8]=join(7,[5,12,9,8])=join(7,rq([5,4,9,8],2,12))
    Rule rq(x,k,y)=join(head(x),rq(tail(x),k-1,y)). However this is not valid when k=1. rq([16,19,21],1,12)=[12,19,21] but rq([19,21],0,12) is undefined. Hence exception is when k=1 rq(x,1,y)=join(y,tail(x)).

13. Let sme(x) finds the sum of all even elements. sme([8,3,4,5,9,1,2,7,3])=8+4+2=14.
    Rule1: Sme(x)=head(x)+sme(tail(x)) when x is not null and head(x) is even
    Rule2: Sme(x)=sme(tail(x)) when x is not null and head(x) is odd
    Exception: Sme(x)=0 when x is null

14. Let le(x) finds the last even element of the list x. When x does not has any even element then le(x) gives 1. le([5,7,2,6,7,9,1])=6. le([3,5,7])=1
    Rule1: le(x)=le(tail(x)) when x is not null and head(x) is odd
    Rule2: le(x)=head(x) when x is not null and head(x) is even and le(tail(x)) is 1
    Rule3: le(x)=le(tail(x)) when x is not null and head(x) is even and le(tail(x)) is not 1

Exception: le(x)=1 when x is null

15. Let alleven(x) is true if all elements of the list x are even. alleven([6,8,12,4]) is true. alleven([12,3,8,4,2]) is false. alleven([12]) is true. alleven([17]) is false.

Rule1:	alleven(x)=alleven(tail(x))	when tail(x) is not null and head(x) is even
Exception1:	alleven(x)=false	when head(x) is odd
Exception2:	alleven(x)=true	when tail(x) is null and head(x) is even

NAME	Roll no

1. Let ct(x) is defined as
   ct(x)=join(head(tail(x)),join(head(x),tail(tail(x))))

What is ct([27,58,28,92])?

2. Let h(x) is defined as
   h(x)=400+h(x-1)–h(x-2) when x>2 h(1)=27 h(2)=49

What is h(5)?

3. Let gd(x,y) is defined as
   gd(x,y)=join(y+head(x),gd(tail(x),y+1)) when tail(x) is not null
   gd(x,y)=null when tail(x) is null

What is gd([27,94,57,92],4)?

4. Let pp(x,y) is defined as
   pp(x,y)=join(head(x),pp(tail(x),y)) when x is not null and head(x) $\leq$ y
   pp(x,y)=join(y,x) when x is not null and head(x) $>$ y
   pp(x,y)=join(y,[]) when x is null

What is pp([22,24,7,28,95,27],40)?

5. Let ad(x) is defined as
   ad(x)=join(head(ad(tail(x)))–head(tail(x))+head(x)+1,ad(tail(x)))when tail(x) is not null
   ad(x)=x when tail(x) is null

What is ad([22,24,89,56])?

6. Let df(x) is defined as:
   Rule df(x)=df(join(head(x) –head(tail(x)),tail(tail(x)))) when tail(x) in not null
   Exception df(x)=head(x) when tail(x) is null

What is df([97,22,24,4,25])?

7. Let tt(x) is defined as: tt(x)=join(tail(x),join(head(x),[]))

What is tt([5,4,7])?

8. Let kk(x) is defined as: kk(x)=tail(head(x))

What is kk([[22,54,56],[27,47,99],44,77])?

NAME	Roll no

9. Let sl(x) is second last element of a list. for example sl([5,2,3,7,6])=7.
   Rule: sl(x)=sl(tail(x)) when <missing> is not null
   Exception: sl(x)=head(x) when <missing> is null

What is missing?

10. Let ae(x) is the second number after first even number.
    ae([5,8,5,7,9,2,5])=7. It is because first even number is 8. Second number after that is 7.
    ae([7,3,6,4,2,8,9])=2. It is because first even number is 6. Second number after that is 2.
    Assume that x has at least one even number and there are at least two numbers after that.
    Rule: ae(x)=ae(tail(x)) when head(x) is an odd number
    Exception: ae(x)=missing when head(x) is an even number

What is missing?

11. Let del(x,k) deletes the kth element of the list x. del([22,13,36,14,19],2)=[22,36,14,19]
    Rule: del(x,k)=join(head(x),missing) when k>1
    Exception: del(x,1)=tail(x)

What is missing?

12. Let rp(x,y) replaces the last element of a list x by number y. rp([5,2,3],7)=[5,2,7]
    Rule: rp(x,y)=join(head(x),rp(tail(x),y)) when tail(x) is not null
    Exception: rp(x,y)=missing when tail(x) is null

What is missing?

13. Let loc(x) finds the location of last even element. loc([5,7,2,6,7,9,1])=4. loc([2,7,8])=3 loc([6,3])=1. When x does not has any even number then loc(x) gives 0. loc([3,1,5])=0.
    Rule1: loc(x)=1+loc(tail(x)) when x is not null and loc(tail(x)) is not 0
    Rule2: loc(x)=missing1 when x is not null and loc(tail(x)) is 0 and head(x) is odd
    Rule3: loc(x)=missing2 when x is not null and loc(tail(x)) is 0 and head(x) is even
    Exception: loc(x)=0 when x is null

What are missing1 and missing2?	Missing1	Missing2

14. Let sf(x)=1! + 2! + 3! + … +x!. sf(4)=1! + 2! + 3! + 4!=1+2+6+24=33. (! is factorial)
Rule: sf(x)=missing when x>2 Exceptions sf(1)=1 sf(2)=3

What is missing?

15. Let mg(23,[7,3])=[23,[7,3],7,3] mg(11,[43,78,26])=[11,[43,78,26],43,78,26]

How mg(x,y) is defined?

Boolean Function

Sum of product form Notation for this page

Inputs to the function are binary variables. Output is a non-negative number.
Variables x, y and z are Boolean variables (i.e. they take value 0 or 1).
abc is ordinary product. (5)(7)(3) is 105.
(a+b+c) is maximum of a, b and c. (12+34+15+19) is 34.
a’ is short form for (1−a)

The expression (1−y)z becomes 0 when y=1 or z=0. Hence only boxes 1 and 5 will have 1. All other boxes will be 0. Similarly expression xz will be 1 only in box 5 and 7. The expression (1−y)z+xz will become 2 in box 5, it will be 1 in boxes 1 and 7. In other boxes it will be zero. However, if '+' is treated as maximum then in box 5 it will be 1.

\ x yz	0	1
00	0	4
01	1	5
10	2	6
11	3	7

The function f (defined at right) can be written as 42y+39(1−x)(1−y). However if '+' means maximum then it can be also be written as 42y+39(1−x). It is because in boxes 2 and 3, 42 will suppress 39. Hence smallest size expression is 42y+39x'. Following method is used.

\ x yz	0	1
00	39	0
01	39	0
10	42	42
11	42	42

42y

\x yz	0	1
00	0	0
01	0	0
10	42	42
11	42	42

39(1−x)(1−y)

\x yz	0	1
00	39	0
01	39	0
10	0	0
11	0	0

39(1−x)=39x'

\x yz	0	1
00	39	0
01	39	0
10	39	0
11	39	0

max(42y,39x')

\x yz	0	1
00	39	0
01	39	0
10	42	42
11	42	42

The function f (defined below) can be written as 73xy'+42yz'+35x'.

The function f

\x yz	0	1
00	35	73
01	35	73
10	42	42
11	35	0

73x(1−y)=73xy'

\x yz	0	1
00	0	73
01	0	73
10	0	0
11	0	0

42y(1−z)=42yz'

\x yz	0	1
00	0	0
01	0	0
10	42	42
11	0	0

35(1−x)=35x'

\x yz	0	1
00	35	0
01	35	0
10	35	0
11	35	0

In the above function let us replace 42 by 22. Now above technique can not be applied. It is because 22 will not suppress 35 (rather 35 will suppress 22). Now the expression is 73xy'+35y'+35x'z +22z'.

The function g

\x yz	0	1
00	35	73
01	35	73
10	22	22
11	35	0

73x(1−y)=73xy'

\x yz	0	1
00	0	73
01	0	73
10	0	0
11	0	0

35y'+35x'z

\x yz	0	1
00	35	35
01	35	35
10	0	0
11	35	0

22(1−z)=22z'

\x yz	0	1
00	22	22
01	0	0
10	22	22
11	0	0

Product of sum form

Notation for this page

Here those functions are defined, whose input is binary and output is between 0 and 1.
(a+b+c) is ordinary sum. 5+7+12 is 24.
abc is minimum of a, b and c. (2)(3)(0.4) is 0.4.
a’ is short form for (1–a)

The function h (defined below) can be written as h(x,y,z)=(x+y)(y’+0.2)(x’+y+0.6)

The function h

\x yz	0	1
00	0	0.6
01	0	0.6
10	0.2	0.2
11	0.2	0.2

x+y

\x yz	0	1
00	0	1
01	0	1
10	1	2
11	1	2

y’+0.2

\x yz	0	1
00	1.2	1.2
01	1.2	1.2
10	0.2	0.2
11	0.2	0.2

x’+y+0.6

\x yz	0	1
00	1.6	0.6
01	1.6	0.6
10	2.6	1.6
11	2.6	1.6

The function (table at right) can be written as
(x’+z)(y+z’+0.2)(x+y’+0.4)(x+y+z+0.4)(x’+y’+z’+0.7)
However we can also write it in more compressed form as
(x’+z)(y+z’+0.2)(x+0.4)(0.7)
Following method is used to reach at above. Here we have used the fact the minimum is taken in place of product.

\x yz	0	1
00	0.4	0
01	0.2	0.2
10	0.4	0
11	0.4	0.7

x’+z

\x yz	0	1
00	1	0
01	2	1
10	1	0
11	2	1

y+z’+0.2

\x yz	0	1
00	1.2	1.2
01	0.2	0.2
10	2.2	2.2
11	1.2	1.2

x+0.4

\x yz	0	1
00	0.4	1.4
01	0.4	1.4
10	0.4	1.4
11	0.4	1.4

0.7

\x yz	0	1
00	0.7	0.7
01	0.7	0.7
10	0.7	0.7
11	0.7	0.7

The function (defined below) can be written as (y+z’+0.3)(x’+z’+0.3)(0.8). Those entries, which are bigger than 1 are not shown (because they will be suppressed)

The function

\x yz	0	1
00	0.8	0.8
01	0.3	0.3
10	0.8	0.8
11	0.8	0.3

y+z’+0.3

\x yz	0	1
00		
01	0.3	0.3
10		
11		

x’+z’+0.3

\x yz	0	1
00		
01		0.3
10		
11		0.3

0.8

\x yz	0	1
00	0.8	0.8
01	0.8	0.8
10	0.8	0.8
11	0.8	0.8

NAME:	Roll No:
For all questions (printed on either side)	a’ is used as a short form for (1−a)

1. Let f(x,y,z)=67xy’z+75yz+16
   Fill the table corresponding to this function.

   ab is product of a and b. (6)(1)(3)=18
   a+b is max(a,b). 12+17+13=17.

╲ x yz	0	1
00		
01		
10		
11		

2. Let f(x,y,z)=(x+y’)(x+y+z’+0.6)
   Fill the table corresponding to this function.

   ab is minimum of a and b. (2.7)(1.1)(1.6)=1.1
   a+b is max(a,b). 12+17+13=17.

╲ x yz	0	1
00		
01		
10		
11		

3. Let f(x,y,z)=(x’+0.7)(y+0.6)
   Fill the table corresponding to this function.

   ab is product of a and b. (2.7)(1.1)=2.97
   a+b is max(a,b). 12+17+13=17.

╲ x yz	0	1
00		
01		
10		
11		

4. Let f(x,y,z)=(x’+0.7)(y+0.6)
   Fill the table corresponding to this function.

   ab is product of a and b (2.7)(1.1)=2.97
   a+b is sum of a and b 2.2+1.3=3.5

╲ x yz	0	1
00		
01		
10		
11		

5. Let f(x,y,z)=0.66xy+8.3y’z
   Fill the table corresponding to this function.

   ab is minimum of a and b (2.7)(1.1)(1.8)=1.1
   a+b is maximum of a and b 1.8+2.2+2.3=2.3

╲ x yz	0	1
00		
01		
10		
11		

NAME:	Roll No:
For all questions (printed on either side)	a’ is used as a short form for (1−a)

6. Let f(x,y,z)=(0.96x+0.73y’z)(0.86)+0.51x’y
   Fill the table corresponding to this function.

   ab is minimum of a and b (2.7)(1.1)(1.8)=1.1
   a+b is maximum of a and b 1.8+2.2+2.3=2.3

x / yz	0	1
00		
01		
10		
11		

For following questions the answer of smallest size should be given. If there are more than one possible answers of same size then all of them should be given. Size of an expression is the number of multiplications (or min) and additions (or max) used.

7. Let the following function realizes the given table
   87x’+75z+missing

   What is missing (smallest size)?

   ab is product of a and b. (27)(11)=297
   a+b is maximum of a and b 22+13+19+11=22

x / yz	0	1
00	87	61
01	87	75
10	87	76
11	87	75

8. Let the following function realizes the given table
   (x’+y’+0.1)(w+0.11)(z+0.8)(0.9)(missing)

   What is missing (smallest size)?

   ab is minimum of a and b (7)(8)(5)=5
   a+b is normal sum 22+13=35

zw \ xy	00	01	10	11
00	0.11	0.11	0.11	0.1
01	0.8	0.8	0.6	0.1
10	0.11	0.11	0.11	0.1
11	0.9	0.9	0.9	0.1

9. Let the following function realizes the given table
   67x + 42y(1−w) + 37w + 19(1−x)(1−z) + missing

   What is missing (smallest size)?

   ab is normal product (7)(8)(5)=280
   a+b is normal sum 22+13=35

zw \ xy	00	01	10	11
00	19	61	67	109
01	56	56	104	104
10	38	80	88	130
11	58	58	125	125

10. In the function, which realizes the given table,

    What is coefficient of 55?(smallest size)

    ab is normal product (7)(8)(5)=280
    a+b is maximum of a and b 22+13+39+4=39
    [Hint: The coefficient of 85 is x’yz’]

zw \ xy	00	01	10	11
00	78	85	55	75
01	33	97	31	83
10	39	33	78	93
11	79	89	79	39

Conceptual Dependency (Representation of meaning)

Action	Name	Schema
Change of location	Loc-chg	agent, object, source,destination
Attribute change	Att-chg	agent, object, attribute, from, to
Telling	Speak	agent, object, contents
Movement of body part (s)	Move	agent, body parts, instruments used
Absorbed by the body	Digest	agent, object
Absorbed by Mind (Understand)	Grasp	agent, contents
Construction of objects	Make	agent, object, instruments used
Thinking (reasoning based)	Mbuild	agent, contents
Thinking (memory recall)	Mem-rec	agent, contents
Movement of object	Push	agent, Object
Causation (agent is action)	$\Rightarrow$	
When actual action is not known	DO	Agent
When some value (agent, object etc) is not known then it is written as Unknown (or U).		

Conceptual Dependency deals with representation of information about events. However all information may not be about events. Let us see representation of few sentences, which does not involve any action. They are propositions. They give attribute value.

1. Ram is Hari's father. Father(Hari,Ram). Here Hari's father attribute is Ram. It can also be represented as son(Ram,Hari). However son is not an attribute. It is because a person may not have any son or may have more than one son. Father is an attribute because a person has exactly one father.
2. The color of this ball is red. Color(ball,red) Here color is an attribute. Any object has exactly one color. [Here we are not considering those objects, which have more than one color or are color less]
3. Hari is sick. Health(Hari,sick). The health attribute of a person can be good, bad, sick, injured etc. [Here we are assuming that a person can not be sick and injured simultaneously]
4. Now temperature is 20^0C. temperature(environment,20^0C). Temperature is an attribute.
5. Hari's height is 160cms. height(Hari,160cms). Height is an attribute.
6. Ram is happy. Mood(Ram,happy). Mood of a person can be happy, angry, sad etc.
7. Ram is at Delhi. Position(Ram,Delhi).
8. Ram is poor in studies. Intelligence(Ram,low). Intelligence attribute of a person can be low, high, medium, fool, etc.
9. Ram is quarrelsome. Nature(Ram,quarrelsome). Nature of a person can be cool, quarrelsome, etc.

10. Cube of 6 is 216 (6^3=216). Cube(6,216). Similar attribute is square. Square(7,49).

16. Ram went from Kanpur to Delhi. Loc-chg(Ram,Ram,Kanpur,Delhi)
    Here agent and object of the action is Ram.
17. Gopal sent Kapil to Bombay. Loc-chg(Gopal,Kapil,Unknown,Bombay)
    The location of Kapil has been changed by Gopal. The initial location is not known.
18. Hari left IIT. Loc-chg(Hari,Hari,IIT,U) [Here U is used as short form of unknown]
19. Hari painted a green ball by red color. Att-chg(Hari,ball,color,green,red)
    Previously color of the ball was green. Now it has become red. Here color is an attribute of the ball. Its value has changed from green to red.
20. Bimu adopted Ravi. Att-chg(Bimu,Ravi,father,unknown,Bimu)
    Father of Ravi has changed. Here father is an attribute.
21. Gopal insisted Hari to adopt Ram's son Shaym. Att-chg(Gopal,Shyam,father,Ram,Hari)
    Previously the father of Shyam was Ram. Now Hari has become his father. Gopal is the agent of the adopt (father change) action.
22. AC (air-conditioner) has decreased the temperature of the room by 7^0C.
    Att-chg(AC,room,temperature,x,x–7)
23. Ram's weight has reduced to 40kg. Att-chg(Unkown,Ram,weight,x+40,40)
24. Mohan has fallen sick because of cold weather. Att-chg(cold,Mohan,health,good,sick)
25. Kapil has injured Ram. Att-chg(Kapil,Ram,health,good,injured)
26. Ram gave Hari's book to Mohan. Att-chg(Ram,book,owner,Hari,Mohan)
27. Ram told Mohan that Kapil is Hari's father. Speak(Ram,Mohan,father(Hari,Kapil))
28. Ram told Anil that Dipu is not Hari's father. Speak(Ram,Anil,NOT(father(Hari,Dipu)))
29. Anil told Bimu to go. Speak(Anil,Bimu,Loc-chg(Bimu,Bimu,U,U)) [Here U is unknown]
30. Ram told Om to paint the ball. Speak(Ram,Om,Att-chg(Om,ball,color,Unknown,Unown))
31. Ram asked Mohan the name of his father. Speak(Ram,Mohan,father(Mohan,?))
32. Ram asked Mohan "How much weight do you have lost?"
    Speak(Ram,Mohan,Att-chg(Unknown,Mohan,weight,x,x-?))
33. Ram is doing exercise using jogger and dumb-bell. Here Ram is moving his body parts. Jogger and dumb-bell are instruments used. Move(Ram,{legs,hands},{jogger,dumb-bell})
34. Ram is eating food. Digest(Ram,food)

Following sentences are syntactically same. However they carry different meaning. Hence they are represented differently. It is because basic action is different.

1. Ram is eating apple. Digest(Ram,apple)
2. Hari is eating chewing-gum. Move(Hari,{mouth},{chewing_gum})
3. Kapil has eaten glass balls. Loc-chg(Ram,glass-balls,ground,stomach)

In the first sentence the apple is digested by Ram's body.
Chewing-gum does not have any food value. Hari is only moving his mouth.

Glass balls are not digested. They are simply lying in Kapil's Stomoch.

35. Ram knows that Hari is Ravi's father. Grasp(Ram,father(Ravi,Hari))
36. Ram knows that Om has gone. Grasp(Ram,Loc-chg(Om,Om,U,U)) [Here U is unknown]
37. Kapil is reading book. Grasp(Kapil,Book)

38. Ram is making chair using hammer and screw driver. Make(Ram,chair,{hammer,sc-dvr})
39. Ram is solving problem by Laplace transformation. Make(Ram,solution to problem,LT)
40. Ram is measuring 1 liter milk using 3 and 5 lit pots. Make(Ram, 1 lit milk, {3,5 lit pots}).

There are two types of thinking. In one of them we recall information from memory. In another (reasoning based) we make new information out of old.

41. Ram is thinking, who is Mohan's father. Mem-rec(Ram,father(Mohan,?)).
42. Ram is thinking, where Kapil went. Mem-rec(Ram,Loc-chg(Kapil,Kapil,Unkwown,?))
43. Hari is thinking how to measure 1 liter milk using 3 and 5 lit pots.
    Mbuild(Hari,Make(Hari, 1 lit milk, {3,5 lit pots}))

When an $event_2$ is caused by another $event_1$ (i.e. the agent of $event_2$ is $event_1$), it is represented as ($event_1 \Rightarrow event_2$). In the representation of $event_2$, the agent is not mentioned.

44. Temperature has decreased because of snow fall.
    Loc-chg(Unknown,snow,sky,ground) $\Rightarrow$ Att-chg(,environment,temperature,x,x−y)
    It can also be represented as Att-chg(snow fall,environment,temperature,x,x−y)
    However it is not good. It is because the meaning of snow fall does not become clear.
45. Ram is feeding Kapil. Loc-chg(Ram,food,plate,Kapil's mouth) $\Rightarrow$ digest(Kapil,food)
46. Hari has eaten ice-cream. Hence he has fallen sick.
    Digest(Hari,ice-cream) $\Rightarrow$ Att-chg(,Hari,healh,good,sick)
47. Ravi is suffering from Malaria. Disease(Ravi,Malaria) $\Rightarrow$ Health(Ravi,bad)
48. Ram is watering plant. Loc-chg(Ram,water,ground,plant) $\Rightarrow$ Digest(plant,water)
49. Om injured Anil by throwing stone. Push(Om,stone) $\Rightarrow$ Att-chg(,Anil,health,good,injured)
50. Kapil gave Mohan an injection Push(Kapil,syringe) $\Rightarrow$ Digest(Mohan,medicine)
51. Ram called Mohan by moving hand. Move(Ram,{hand},{}) $\Rightarrow$ Loc-chg(,Mohan,U,U)
52. Mohan went from Kanpur to Delhi, because Ram told.
    Speak(Ram,Mohan,) $\Rightarrow$ Loc-chg(,Mohan,Kanpur,Delhi)
    It can also be represented as Speak(Ram,Mohan,Loc-chg(Mohan,Mohan,Kanpur,Delhi))
    But here it does not become clear whether Mohan actually went or not.
53. Hari prevented Ram from going. DO(hari) $\Rightarrow$ NOT(Loc-chg(,Ram,U,U))
    It should not be represented as Speak(Hari,Ram,NOT(Loc-chg(Ram,Ram,U,U)))
    It is because from the given sentence it is not clear, how Hari prevented Ram. Simply by saying or by another way. DO is used when the actual action is not known.
54. Anil prevented Ram from giving book to Hari.

DO(Anil) ⇒ NOT(Att-chg(,Book,owner,Ram,Hari))

55. Ram gave Mohan's letter to Hari.
Loc-chg(Ram,letter,Mohan's home, Hari's home) ⇒ Grasp(Hari,letter)

NAME: Roll No: Write representation for followings. They should be in the style described. Do not use any new attribute name, action name or convention.
Hari painted a green ball.
Someone went from the class room.
Mohan Moved table.
AC (air-conditioner) has decreased the temperature of the room to 20^0C. [The present temperature of the room is 20^0C]
Ram told Mohan to throw a stone on Ravi, so that he is injured.
Ram is thinking, who has eaten the apple.
Now Hari has lost hope that Mohan will go to Delhi.
Hari is not able to believe that Mohan will go to Delhi.
Kapil asked Mohan "what you told to Hari?"
. Since Ram reached Kanpur, Hari has to leave Kanpur.
. Initially X(M,P) Finally X(M,Q). Give representation.
. Hari went from Kanpur to Delhi. [Represent without using Loc-chg]

Kapil made Hari angry. [Represent without using Att-chg]
Ram told Mohan “you are a fool”. Hence Mohan concluded that Ram is quarrelsome.
. Ram is thinking, what is square root of 19.
. Hari is thinking, who measured 1 liter of water, using pots of 5 and 3 liters.
. Hari asked a person “Who is your father?” [Hint: Person to whom Hari asked is unknown]
. Hari wanted to paint a green ball by red color. Mohan stopped him from doing so.

Egg.c

```
#include<stdio.h>
main()
{  int w[100],i,j,k,n,p,q,r,min;
   scanf("%d %d",&n,&k);
   for (i=1;i<=k+1;i++) w[i]=i+k-2;
   for (i=k+2;i<=n;i++)
   {  min=10000;
      for(j=1;j<i;j++)
      {  p=w[j]+10;
         q=w[i-j]+1;
         if (p>q) r=p; else r=q;
         if (r<min) min=r;
      }
      w[i]=min;
   }
   for (i=1;i<=n;i++)
   {   printf("(%d %d) ",i,w[i]);
       if(i%10==0) printf("\n");
   }
}
```

postfix

```
#include<stdio.h>
float x[20];int sp=0;
float push(float e) { x[sp]=e; sp++; }
float pop(){ sp--; return(x[sp]); }
float operation(float p,float q,float r)
{
   if(r==-1000) return(p+q);
   if(r==-1001) return(q-p);
   if(r==-1002) return(q*p);
   if(r==-1003) return(q/p);
}
main()
{
   int i,n;float a[20],s,t;
   printf("give the size of the expression");
   scanf("%d",&n);
   printf("give the expression");
   for(i=0;i<n;i++) scanf("%f",&a[i]);
   for(i=0;i<n;i++)
   {
       if(a[i]>-1000) push(a[i]);
       else
       {  s=pop();t=pop();
          push(operation(s,t,a[i]));
       }
   }
   printf("the result is%f\n",pop());
}
```

prefix

```
#include<stdio.h>
float x[20];int sp=0;
float push(float e){ x[sp]=e; sp++;}
float pop(){ sp--; return(x[sp]);}
float operation(float p,float q,float r)
{
   if(r==-1000) return(p+q);
   if(r==-1001) return(q-p);
   if(r==-1002) return(q*p);
   if(r==-1003) return(q/p);
}
main()
{
   int i,n;float a[20],k,l,t;
   printf("give the size of the expression");scanf("%d",&n);
   printf("give the expression");
   for(i=0;i<n;i++) scanf("%f",&a[i]);
   for(i=0;i<n;i++)
   {
       if(a[i]<=-1000) push(a[i]);
       else
       {  k=pop();
          if(k<=-1000)
          {  push(k);
             push(a[i]);
          }
          else
```

```
          {  l=pop();
             t=(operation(a[i],k,l));
             a[i]=t;i--;
          }
       }
   }
   printf("the result is%f\n",pop());
}
```

pushlink

```
#include<stdio.h>
typedef struct node { float data; struct node *next;}node;
node *head;
void push(float e)
{
   node *t;
   t=(node *)malloc(sizeof(node));
   t->data=e;
   t->next=head;
   head=t;
}
float pop()
{
   float t;
   t=head->data;
   head=head->next;
   return(t);
}
void print()
{
   node *j;
   j=head;
   while(j!=NULL)
   {
       printf("%4.1f,",j->data);
       j=j->next;
   }
}
main()
{
   int i,choice;float y,t;head=NULL;
   do
   {
      printf("give the choice:0 for put 1 for take 2 for print 3 for exit");
scanf("%d",&choice);
      if(choice==0)
      {  printf("give the element to be pushed");scanf("%f",&y);
         push(y);
      }
      if(choice==1)
      {  t=pop();printf("The popped element is %f\n",t);}
      if(choice==2)print();
   }while(choice!=3);
}
```

Binary search

```
#include<stdio.h>
typedef struct node { float data; struct node *left,*right;}node;
node *root;
```

```
void put(float e)
{
   node *t,*i,*j;
   i=root;
   t=(node *)malloc(sizeof(node));
   t->data=e;t->left=NULL;t->right=NULL;
   if(i==NULL) root=t;
   else
   {
      while(i!=NULL)
      {
         j=i;
         if(i->data>e) i=i->left;
         else i=i->right;
      }
      if(j->data>e) j->left=t;
      else j->right=t;
   }
}
float search(float e)
{
   node *i;
   i=root;
   do
   {  printf("%f ",i->data);
      if(i->data>e) i=i->left;
      else i=i->right;
   }while(i!=NULL && i->data!=e);
   if(i==NULL) printf("the element does not exist");
   else printf("the element exists");
}
main()
{
   int i,choice;float y,m;
   do
   {  printf("give the choice: 0 for put 1 for take 2 for exit ");
      scanf("%d",&choice);
      if(choice==0)
      {  printf("give the element to be put");scanf("%f",&y);
         put(y);
      }
      if(choice==1)
      {
         printf("give the element to be searched");
         scanf("%f",&m);search(m);
      }
   }while(choice!=2);
}
```

Rearrange elements of two arrays alternatively

```
#include<stdio.h>
main()
{  int i,j,n,a[100],t,p;
   printf("Give number of elements in each array");scanf("%d",&n);
   printf("Give elements of first array");
   for (i=0;i<n;i++) scanf("%d",&a[i]);
   printf("Give elements of second array");
   for (i=n;i<2*n;i++) scanf("%d",&a[i]);
```

```
   for (j=1;j<2*n-1;j++)
   {  if (a[j]>0)
      {  i=j;p=a[i];
         do{ i=2*i % (2*n-1);
             t=a[i];
             a[i]=-p;
             p=t;
          } while(i!=j);
      }
   }
   for (i=0;i<2*n;i++) if (a[i]<0) a[i]=-a[i];
   for (i=0;i<2*n;i++) printf("%d ",a[i]);
}
```